A Property Kind Calculus for Metrology: proving quantity and unit laws in Lean, from ISO/IEC 80000 to measurement uncertainty

Compiled

2026-11-06T17:44:48Z

Project `0f1b7ffb`

Merge pull request #106 from nasa-jpl/devin/1793981098-physlib-solution-split

Lean

leanprover/lean4:v4.35.0

VersoBlueprint (https://github.com/leanprover/verso-blueprint)

`5a5bfc7e17ddba0cbce176d0fe3bfe6a257b16ec@5a5bfc7e17dd` (https://github.com/leanprover/verso-blueprint/commit/5a5bfc7e17ddba0cbce176d0fe3bfe6a257b16ec)

Mathlib (https://github.com/leanprover-community/mathlib4)

`v4.35.0@56a005855ea1` (https://github.com/leanprover-community/mathlib4/commit/56a005855ea17e0b0958f73ec801ae140b0a1679)

Nicolas F. Rouquette and Xiaolan Xu, *Jet Propulsion Laboratory, California Institute of Technology*

Measurement begins with a judgment that two properties are *mutually comparable* — that it is meaningful to ask which of them is the greater, or by how much. Dybkær's ontology on property (Dybkær, 2004)[1] takes that judgment as primitive and names what underwrites it: a *kind-of-property* is the "common defining aspect of mutually comparable properties" (§6.19). That is the first commitment, **comparability**: which properties are mutually comparable is something a model *states*, not a coincidence a reader may infer. Nearly everything else about a quantity hangs on it — the *scale*, which fixes whether `=,<,+,\times` are so much as defined between two values; the *unit*, which is one chosen value of one kind and of no other; and whether the values of the parts of a system bear on the value of the whole.

1. René Dybkær, 2004. *An Ontology on Property for Physical, Chemical, and Biological Systems* (https://ontology.iupac.org/). PhD Thesis, University of Copenhagen

What a *dimension* contributes is real, and bounded. It records the SI base-quantity exponents of a kind and nothing else, and it records them homomorphically: the exponents of a product add. That is enough for dimensional analysis to do its work — to reject a length added to a time, to catch the formula that returned a rate where a total was wanted, to constrain the form of a law before its coefficients are known

— by arithmetic alone, with no knowledge of what the quantities are quantities *of*. It is a genuine invariant, and worth keeping.

What a dimension cannot do is separate two kinds it sends to the same point, and the map that sends them is many-to-one by construction. Torque and energy both carry `\mathrm{M\,L^2\,T^{-2}}`, so no dimension check tells them apart. The limiting case is the quantities of *dimension one*

2. (Dybkær, 2004)[2] René Dybkær (2004). "Units for quantities of dimension one" (https://iopscience.iop.org/article/10.1088/0026-1394/41/1/010). *Metrologia.* **41**(1). , the ones usually called dimensionless: every ratio of like quantities lands on the single point `1`. Volumetric and gravimetric water content, relative permittivity, reflectivity, and all 115 characteristic numbers of ISO 80000-11 — Reynolds, Mach, and Prandtl among them — arrive there together. Inside that fiber dimensional analysis has no discriminating power left at all: it sees one type, *a real number*, for the whole family, and every substitution of one member for another is dimensionally correct. A language that checks dimensions therefore checks a necessary condition and reports it as a sufficient one — and what it admits, a gravimetric water content where a volumetric one was meant or a reflectivity where a permittivity was, does not halt the computation. It changes the number the computation returns.

A kind is a kind *of* something, and the second commitment, **the bearer**, concerns what that something is.

3. Lowe's four-category ontology (Lowe, 2005)[3] E. J. Lowe (December 2005). "The Four-Category Ontology: A Metaphysical Foundation for Natural Science" (https://doi.org/10.1093/0199254397.001.0001). *Oxford University Press.* . supplies the categorial frame Dybkær's vocabulary implies but never names. Two cuts make the four categories — universal against particular, and *substantial*, a thing, against *non-substantial*, a way a thing is — and every term of measurement lands in exactly one of the quadrants they produce. Here is Lowe's square (his Fig. 7.1), with the calculus's identifier for each corner set beneath his own term and its mechanism on each edge:

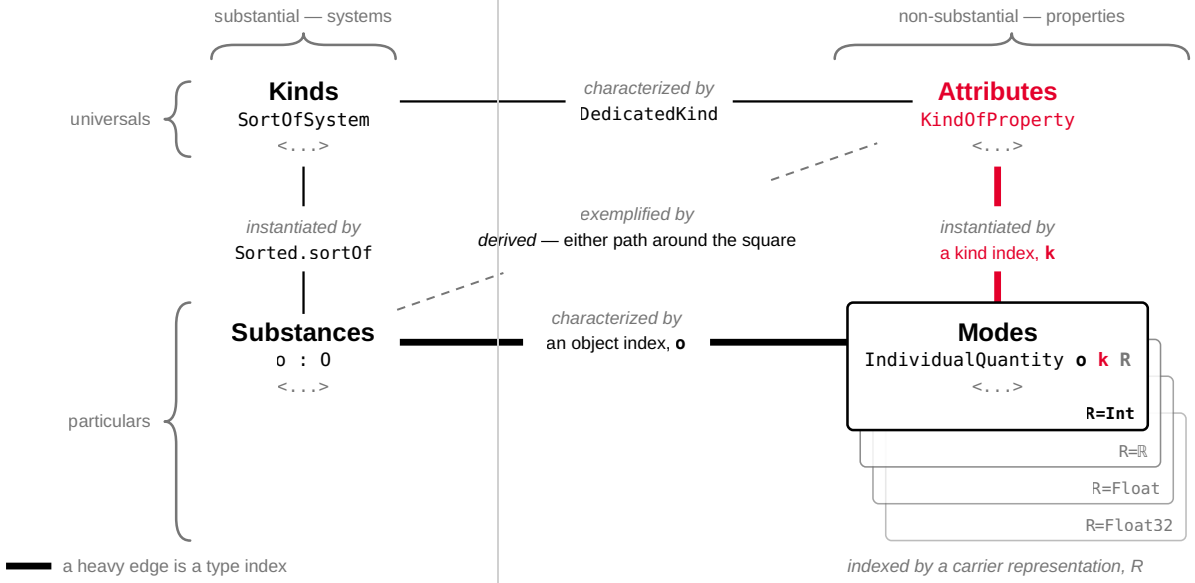

Figure 1. Lowe's ontological square with the calculus's identifiers at its corners and its mechanisms on its edges.

The two heavy edges are the two type indices of an individual quantity — the object index o along the bottom and the kind index k down the right — and the plates behind the *Modes* corner are the carrier R, which indexes that corner alone. Each corner's instance slot is left empty; a model fills it.

The four corners, taken in the order the square draws them, top row first (the numbers are Lowe's):

- **(3) Kinds — the substantial universal.** A *sort of system*: soil, a reference cell, an antenna. What a thing is, said once for all things of that sort.
- **(4) Attributes — the non-substantial universal.** A *kind-of-property*, the **comparability** commitment above: volumetric water content in general, torque in general.
- **(1) Substances — the substantial particular.** The *system* at hand: this soil column, under this footprint, at this hour. Particular systems are the *objects* of measurement — whatever a measured value is a value *of* — and object is the word the rest of this document uses for a substance.
- **(2) Modes — the non-substantial particular.** An *individual quantity*: the volumetric water content of *that* column. Neither the general kind nor a bare number.

And the three relations, which are why this is a square and not a list of four boxes:

- **Instantiation** runs down the sides, universal to particular: a sort of system is instantiated by the systems of that sort, a kind-of-property by the individual quantities of that kind. Both edges are declared by a model; neither is inferred.
- **Characterization** runs across each row, left to right, substantial to non-substantial: below, a system is characterized by the individual quantities it bears; above, a sort of system is characterized by the kinds-of-property that apply to it — the edge along which a kind may be *dedicated* to a sort, water content *of soil* rather than water content at large. A dimension has no counterpart for either.
- **Exemplification** is the diagonal, and it is derived rather than primitive: an object exemplifies an attribute by going either way around the square, so the calculus needs no construct joining an object directly to a kind.

Two consequences are load-bearing. First, an individual quantity is a *mode* — the length of *this* pencil, not length in general — so a magnitude detached from what it characterizes is a number rather than a quantity, and the general kind is never the same entity as the individual measurement. Second, characterization is not parthood: properties characterize objects, they do not compose them. Dybkær says as much in his own voice, refusing a partitive reading of the object–property relation on the ground that a duplicate object cannot be produced by putting an object's properties together (§2.23.3).

Keeping those relations apart raises the third commitment, **composition** — the one an unstated convention damages most: what a carving of a system into parts does to a value. Marmodoro's distinction is that "physical structure unites; while metaphysical structure unifies", and that "wholes are not always unities"; a physical structure brings no principle of counting with it: "Alternative carvings of the world deliver alternative numbers of entities" (Marmodoro, 2018)[4]

4. Anna Marmodoro, 2018. “Whole,

but not One”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).* . A description of a whole into parts is one carving among many, and no arithmetic follows from having made it. That the masses of the parts sum to the mass of the whole while the volumes of mixed liquids do not is a claim about the kind — refutable by a measurement, and therefore worth proving rather than assuming.

Each of the three is a claim about the world rather than a convention adopted for convenience: a model states it, and a measurement can refute it.

- **Comparability.** Which properties share a kind-of-property, and may therefore be compared, added, or given a common unit. Put torque and energy under one kind — their dimensions raise no objection — and the model licenses a sum that denotes nothing.
- **Bearer.** What each kind is a kind *of*: the sort of system it is dedicated to, and the object an individual quantity characterizes. Leave it unstated and one system's water content stands in for another's without complaint.
- **Composition.** What a carving of a system into parts does to a value — which kinds aggregate over that carving, and by what law. Assume additivity everywhere and the volumes of mixed liquids refute the model.

A metrology *taxonomy* answers a different question. It records which kinds exist, with their dimensions and defining relations — a list of entries, and no list forbids the sum, dedicates the kind, or fixes the aggregation law. What has to be added is the *calculus* that operates on those kinds: dimension arithmetic, the algebra of which kinds interact and to what, operations gated by scale, aggregation laws quantified over every carving; *parametricity* enough that one model definition survives the several numeric carriers a quantity must inhabit; and a logic in which each such law is *proved* rather than documented.

We present PropertyKindCalculus (PKC) 0.114.0, a machine-checked formalization, in the Lean 4 proof assistant, of Dybkær's seminal contribution to metrology — read through Lowe's square — and of several recent developments in this field (Flater

5. (Flater, 2018)[5] David Flater (2018). “Architecture for software-assisted quantity calculus” (https://
6. doi.org/10.6028/NIST.TN.1943). *Computer Standards & Interfaces.* **56**. , Willink (Willink, 2005)[6] Robin Willink (2005). “A procedure for the evaluation of measurement uncertainty based on moments” (https://iopscience.iop.org/article/10.1088/0026-1394/42/5/001). *Metrologia.* **42**(5). , and Degenhardt (Degenhardt *et*
7. *al.*, 2025)[7] Johannes Degenhardt, Rainer Tutsch, Xiukun Hu, and Gaoliang Dai (2025). “A practically oriented, efficient alternative to the Monte Carlo method for measurement uncertainty estimation” (https://iopscience.iop.org/article/10.1088/1681-7575/adb3ab). *Metrologia.* **62**(2).). PKC makes a kind a first-class type, kind-incompatible arithmetic a compile-time error, and classification a proof obligation. Of twenty-eight metrology requirements, twenty say something that is either true or false of the calculus — that two kinds of the same dimension stay distinct, that converting a value to another unit and back returns it unchanged — and every one of those is proved as a theorem the Lean kernel checks. The other eight ask a different question: not whether something is true, but whether the language can

express it at all — that a general kind and an individual quantity are different sorts of thing, that a unit is one chosen value of one kind. Each of those is met by a definition that compiles, and a type error is what its failure would look like. The standard is the test: all eleven quantity-and-unit parts of ISO/IEC 80000, item by item, 700+ kinds with checked dimensions and proved defining relations. Because a PKC model is carrier-polymorphic and its measurement uncertainty is an additive descriptor, one model definition serves proof, calculation, sensitivity analysis, and uncertainty quantification: what changes between the four is the carrier the model is instantiated at, never the model. That is the design principle we abbreviate *write once, correctly; go fast, automatically*. Measurement uncertainty, its propagation, and the coverage of the resulting interval are thereby machine-checked, while *accuracy* against nature — which needs a true value the kernel never has — remains an empirical matter; the boundary is exact, not rhetorical. The result is a modeling discipline whose specification documents — rendered from the checked sources — are verifiable rather than merely descriptive.

PKC is a library, and where it stands among the libraries it requires is worth drawing before anything is proved about it. A domain model written in the calculus traces its trust down through PhysLib, for dimensions and units, and cslib, for the labelled transition systems in which the capstone chapter states its bisimulation — the two peers on Mathlib — and through TorchLean, with FloatLib under it, for the tensor carriers and the executable float word, to Mathlib and then to the Lean kernel with its three classical axioms, the one band that is believed rather than derived, checked, or enumerated:

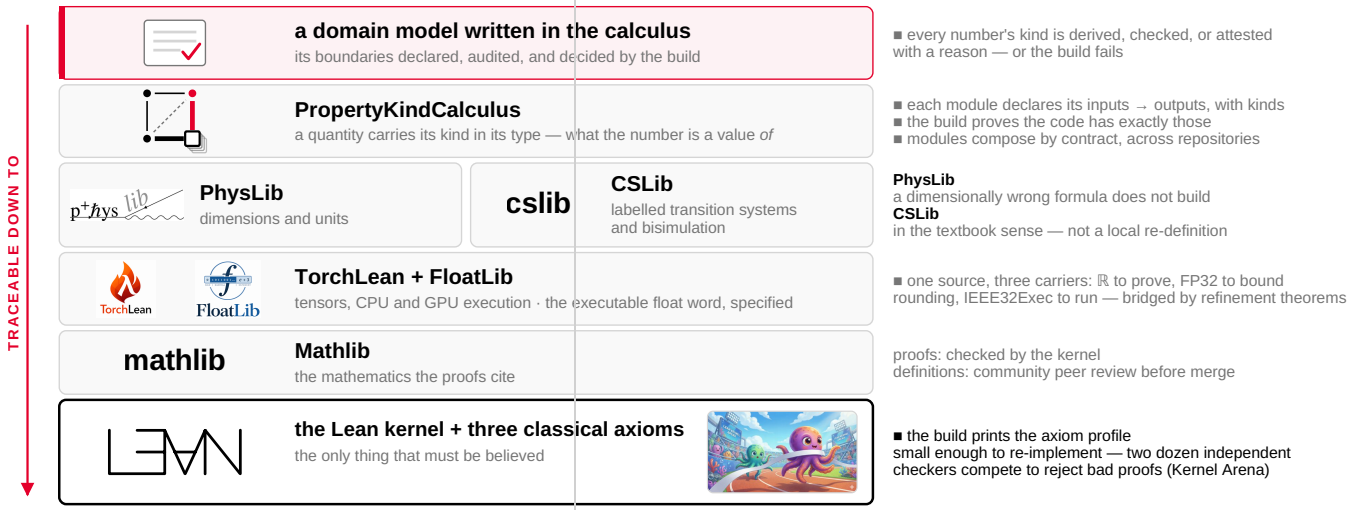

Figure 2. The stack a domain model's trust traces down through: PhysLib and cslib as peers on Mathlib, TorchLean with FloatLib beneath it, and the Lean kernel at the floor.

Each band says what it costs a reviewer in trust, and a filled square marks a property the build checks. The figure is a picture of a dependency graph — who requires whom, as the package manifests record it — and carries no measured claim of its own.

This blueprint is the design map for that formalization: every theorem node names the declaration that discharges it and reports that declaration's checked status, with the headline results tagged **capstone**. The dependency graph and a status summary come at the end.

What can the calculus *do* — what guarantee does a model earn by being written in it, stated as a theorem whose hypotheses the build discharges? Three capstone theorems answer that question, and they are the subject of the capstone chapter: the *seal of a module* — once its gates are declared, no other raw datum reaches its outputs; the *seal of the computation* — the same guarantee for the code at every carrier, with every kind change under an occurrence the source states; and *dimensional homogeneity* — dimension is a homomorphism along every path of a well-formed model. That chapter was written blueprint-first: the properties were architected and their proofs sketched there before the declarations were written, and every node now links the declaration that discharges it.

The whole blueprint is also available as a single paginated document: download the PDF.

The generated API reference for the checked sources is published alongside it: browse the API documentation. Declarations carrying `@[pkc_math]` render their defining equation as typeset mathematics there, next to the Lean source it was derived from.

Acknowledgment. The task "ROSE-L Soil Moisture Retrieval Using the SMAP and NISAR Algorithm", along with this research, was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration.

Table of Contents

List of figures

1. What PropertyKindCalculus provides

2. Requirements

3. Why a calculus, not a taxonomy

4. How to read the status

5. Onboarding: Human Context Protocols

6. The Proved Spine

7. Capstone theorems — what the calculus can do

8. Foundations: system, part, and the ontological square

9. Dedicated kinds-of-property

10. Units and the Dimension-1 Problem

11. Dimension as a Forgetful Functor

12. Scale-spanning units — a third unit category (R13)

13. The Interaction Algebra (Flater Appendix C)

14. The Function Calculus and Complex-Valued Carriers (R12)

15. Extensivity

16. The object type

17. Metrological modularity

18. Write Once, Correctly

19. Uncertainty quantification and numerical adequacy

20. ISO/IEC 80000 — The Standards Catalogue

21. ISO 80000-3 — Space and Time

22. ISO 80000-4 — Mechanics

23. ISO 80000-5 — Thermodynamics

24. IEC 80000-6 — Electromagnetism

25. ISO 80000-7 — Light and radiation

26. ISO 80000-8 — Acoustics

27. ISO 80000-9 — Physical chemistry and molecular physics

28. ISO 80000-10 — Atomic and nuclear physics

29. ISO 80000-11 — Characteristic numbers

30. ISO 80000-12 — Condensed matter physics

31. IEC 80000-13 — Information science and technology

32. External Cross-References

33. Using the library: annotations and generated indexes

34. Applying the calculus to a domain: the model template

35. Deploying a domain model: the deployment template

36. Terminology — the dictionary and the index

Dependency Graph

Blueprint Summary

37. License

List of figures

The document's figures, numbered in the order they appear; each entry links to its figure and names the section it stands in.

Figure 1. Lowe's ontological square with the calculus's identifiers at its corners and its mechanisms on its edges. — in the introduction

Figure 2. The stack a domain model's trust traces down through: PhysLib and cslib as peers on Mathlib, TorchLean with FloatLib beneath it, and the Lean kernel at the floor. — in the introduction

Figure 3. The three layers and their two ladders, drawn on the module `y = a · b + e` at three granularities. — in 7.1. The layers and their ladders

1. What PropertyKindCalculus provides

This project addresses **twenty-eight requirements** about formalizing *metrology* — the science of measurement — and discharges most of them as machine-checked theorems rather than prose. A substantial part of the library is grounded directly on the published **ISO and IEC 80000** metrology standards — *eleven* of the thirteen parts (every part but the two general ones, 1 *General* and 2 *Mathematics*), catalogued item by item. The table below is generated directly from the formalized catalogues, so its per-part counts and total stay in step with the Lean source:

Part	Title	Quantity kinds
ISO 80000-3	Space and time	42
ISO 80000-4	Mechanics	54
ISO 80000-5	Thermodynamics	54
IEC 80000-6	Electromagnetism	85
ISO 80000-7	Light and radiation	66
ISO 80000-8	Acoustics	18
ISO 80000-9	Physical chemistry and molecular physics	62
ISO 80000-10	Atomic and nuclear physics	125
ISO 80000-11	Characteristic numbers	115
ISO 80000-12	Condensed matter physics	60
IEC 80000-13	Information science and technology	42
Total		723

1.1. The paradigm: metrological provenance as taint tracking

The calculus is easiest to see through an analogy with *taint analysis* in software security. A taint analyzer marks what a program reads from outside as *tainted*, follows it through the code, and reports where tainted data reaches a sink that must only ever see trusted data. Read metrologically:

- A raw number is *tainted*. It could be a value of any kind — a velocity, an acceleration, a reflectivity — so nothing downstream may rely on what it means.
- A kinded quantity, `Quantity k R`, is *clean*. Its kind `k` is in its type, so everything downstream may rely on it, and the kind is what every operation on it is checked against.

Taint analysis tracks untrusted data through source code; the calculus tracks kinded quantities through the *kind algebra*. Here the analogy breaks in the calculus's favor. A taint analyzer has to approximate how data flows through a program, whereas here the kind rides in the type and the elaborator propagates it exactly, with nothing to approximate. What a taint analyzer must still be told — where data enters, where it leaves, and where trust is granted by hand — is exactly what an author still declares, and each declaration is an annotation the machine checks:

- **Sources.** Raw data enters through `@[kindIngest]`, a checked ingest — a mint admitted by a check; an adjudicated constant — a cited coefficient, a threshold — enters as a constant mint through `@[kindConst]`.
- **Sinks.** A kinded value leaves the calculus as a bare number at an emission, through `@[kindEmission]`: a deploy driver, a serializer, the parity apparatus the erasure theorems are stated over.
- **Trust granted by hand.** A value that already carries one kind and is re-typed as another, with no law of the algebra to derive it, is a crossing, tagged `@[kindCrossing]`; representation plumbing that drops to the carrier without changing the kind is a `@[carrierVocab]`.

Between the sources and the sinks nothing is declared, because nothing needs to be: a kind reached through a law of the algebra is *derived*, at a site the elaborator checked. That leaves three ways a value comes to carry a kind — its evidence tier — and the provenance layer names them (`Provenance.IntroTier`):

- `derived` — through a law, such as `ProductKind massK velocityK momentumK`. No judgment is made at the use site; it was made once, when the edge was registered, and the edge audits enumerate the edges.
- `gated` — through a check. A `KindAdmissible` instance gives a predicate on the raw value, `Quantity.certify` runs it, and the check is the declaration's evidence. It is necessary, not sufficient: no predicate on bits can establish that a number is a velocity rather than an acceleration.
- `attested` — through no check at all. `Quantity.attest "why" m` is definitionally the raw value `⟨m⟩`; the author states the classification claim, and the reason string, harvested into the audit, is the only content.

An attestation is a *hypothesis* of the model, not an axiom of the calculus. The distinction is the one Lean draws. An axiom is global — stated once, available to every proof, and a single false one makes everything provable — whereas a hypothesis is local to the statement that carries it, and a theorem with thirty of them is unremarkable. A model of any size carries dozens of attestations, because every place a number's kind rests on the author's knowledge rather than on a law or a check is one: an identification of a retrieved variable with the forward model's own, a convention such as NaN for no value, a re-entry from a table fetch the calculus cannot see, an operation the algebra has no edge for. Like a hypothesis, an attestation does no work in the computation — `Quantity.attest "why" m` is `⟨m⟩` — it licenses an interpretation, and the reason is its whole content.

The axiom's part is not played by the calculus. The core postulates nothing about any particular kind: its foundation is Dybkær's ontology of property (Foundations), and what it fixes — which operators a scale admits, that two kinds meet only under a witnessed law, that dimension is forgetful — is designed and proved. The system of quantities the calculus is instantiated with is ISO 80000's (the standards catalogue): a width is a kind of length, speed is path length per duration, each entry cited to its item. That is an axiomatization in the mathematician's sense — a system adopted by the consensus of the community whose subject it is, not a list of assumptions — and the calculus checks each transcribed relation: its scale preconditions close by `rfl`, and the catalogued dimensions are confirmed against it by computation. A domain model extends that system with kinds and relations of its own, and those are the postulates the trust model prices: authored, reviewed, and refutable wherever their kinds carry a dimension. Its attestations are its hypotheses.

For a theorem, the hypotheses stand gathered in its statement. A model scatters them over its source, and the boundary audit (the boundary family) gathers them back: `#kind_boundary_audit ns …` lists every boundary site in a namespace — each entry, exit and crossing, with its tier and, for an attestation, its harvested reason — so the pinned report is the model's *statement*, the list of what it assumes. Pinned under `#guard_msgs` (https://lean-lang.org/doc/reference/latest/Interacting-with-Lean/#hash-guard_msgs), an entry point nobody declared fails the build — not as a type error, since a raw `⟨x⟩` typechecks, but because the audit refuses it. The discipline is a referee's, not a no-axiom rule: derive what a law derives, check what a check can establish, and state everything else. Its end state is not zero attestations but zero *unstated* ones. And the honest limit is a hypothesis's: a false one leaves every downstream step valid and the conclusion worthless. It typechecks, appears as one line in the pinned report, and is exactly as good as the reading of that line — the reader is the caller who discharges it.

1.2. The calculus at a glance

The chapters that follow develop each piece in full, every node linked to the declaration that discharges it. This is the map, in the order the pieces rest on one another.

- **A kind is a type-level index.** `Quantity k R` is a magnitude in the carrier `R` at the kind `k`. Two quantities are comparable, addable, or given a common unit exactly when their kinds agree, and a kind of dimension one is as distinct from its neighbors as a length is from a mass. Which operators a kind admits at all — order, difference, ratio — is fixed by its *scale*; specialization (a radius *is a* length) is a lattice on kinds, and instantiation (this pencil's length) is term-of-type, never subtyping (The proved spine).
- **A kind is a kind of something.** The sort of system a kind is dedicated to and the object an individual quantity characterizes are the two sides of Lowe's square (Foundations, Dedicated kinds-of-property), and a value can carry the object it characterizes in its type, so one soil column's water content cannot stand in for another's (The object type).
- **Two kinds meet only under a law.** `ProductKind k₁ k₂ k` and `QuotientKind k₁ k₂ k` are the authored edges of the interaction algebra — the ISO 80000 defining relations are stated as them

— and the *operator table* registers `KindMul` and `KindDiv` notation over those laws, so `m * v` elaborates at the momentum kind and a pair with no entry has no product. Functions go through the function calculus, six families sorted by kind signature. What is checked is explicitness and propagation, not truth: an edge is an authored claim, enumerable like an axiom (the trust model).

- **Dimension certifies; it does not decide.** `\dim : \mathrm{Kind} \to \mathrm{Dimension}` is a forgetful functor: every edge the algebra admits is proved dimensionally coherent, and agreeing dimensions are necessary, never sufficient. A unit is a distinguished quantity of one kind, converted by proved round trips and never across kinds (Units and the dimension-1 problem); the units the base/derived split does not fit get a category of their own (Scale-spanning units).
- **Aggregation is a claim about the kind.** Which kinds sum over a carving of a system into parts, and under what law or tolerance, is declared and proved rather than assumed (Extensivity).
- **One model, several carriers.** A model is written once over a carrier typeclass and instantiated at `ℝ` to prove, at binary32 to bound rounding, at `Float` or a GPU tape to run; the kinded term erases to the bare-number kernel, so the checking is paid at compile time (Write once, correctly). The same parametricity carries measurement uncertainty through the model and flags where floating point loses information at the scale of the input uncertainties (Uncertainty quantification and numerical adequacy).
- **A module declares its boundary, what it computes, and what its quantities must satisfy.** A `Provenance.Contract` states a module's kind-typed ports and exits, a `Provenance.Relation` its measurement model as a theorem edge, and the aggregation classes on its outputs are what license sharding it (Metrological modularity). A `Provenance.Requirement` per produced port states what that port's quantity must satisfy and by what evidence — provable by a theorem, or empirical by a gate against a referent the model does not define — surveyed and censused, with the deferred ones counted (every claim a model makes, with its evidence).
- **The standard is the test.** Eleven parts of ISO/IEC 80000, item by item, with checked dimensions and proved defining relations (The standards catalogue); Dybkær's and the VIM's clauses are cross-referenced to the declarations that formalize them (External cross-references).
- **Annotations declare; commands ask.** The annotations an author writes and the commands that audit them are catalogued in Using the library, and the model and deployment templates fix what a domain document conforming to the calculus states.

1.3. In plain terms — the requirements

In plain terms, *rigorous metrology* here means:

- **Telling apart quantities that look identical.** Two measurements in the same units can still be different in nature — volumetric and gravimetric soil moisture are both "just a ratio", yet confusing them corrupts a result. The calculus keeps such quantities distinct *types*, so the mix-up is impossible rather than merely discouraged; agreeing units (or dimensions) are treated as necessary but never sufficient (R1, R7).

- **Rejecting nonsense arithmetic before it runs.** Adding a length to a mass — or even a torque to an energy, which share the same dimension — is a compile-time error, not something caught (or missed) at run time. Quantities that genuinely combine, such as a torque times an angle giving an energy, are sanctioned explicitly; everything else is refused (R4, R5).
- **Knowing which operations even make sense.** What you may do to a measurement depends on its scale: you can rank mineral hardness but not average it; you can subtract Celsius temperatures but only take ratios of thermodynamic ones. These rules are enforced automatically (R6).
- **A faithful hierarchy of kinds.** Width and height are special cases of length — usable where a length is wanted, never the reverse — while a general kind (length) is never confused with one individual measurement (the length of this pencil). A kind can even be tied to the system and component it is *dedicated* to, so "water content of soil" is named in its own right, and a value carries the object it characterizes in its own type, so one soil column's water content cannot stand in for another's (R2, R3, R19, R22).
- **Units that convert safely and never silently.** A unit is a chosen reference of one specific kind; a conversion round-trips exactly, and there is simply no conversion between units of different kinds (R8, R16, R17).
- **Summing only what may be summed.** The masses of the parts add up to the mass of the whole; the volumes of mixed liquids do not. The calculus tracks which quantities aggregate and refuses to assume it of the rest (R9).
- **Classifications you can trust because they are earned.** Labeling a value "an area" requires a proof that it really is a width times a height; an arbitrary number cannot wear the label (R12).
- **The same model from proof to running code.** One quantity can carry exact real numbers for proving properties and IEEE floating-point for execution — with a theorem guaranteeing the running code matches the proven specification, rounding and all — and a vector quantity is a list of numbers under a *single* shared unit, not a bag of separately-united components (R10, R11).
- **Components that remember the frame they were read in.** A vector quantity's numbers are coefficients in some chosen set of axes, and that choice is no part of the physics: turn the axes and every component changes, while the scalar product of two vectors does not. Readings taken in different frames do not combine, a change of frame acts on them through the law their kind of index obeys, and the combinations that are genuinely frame-independent are the ones proved to be (R20).
- **Honest unit classification.** The SI sorts units into *base* and *derived*, but not even the seven base units are alike: the mole is dimensionally a pure number and the candela a power, and each stays a base unit only because it carries a human-chosen coefficient sized to keep everyday values readable. Units like these get a category of their own — *scale-spanning* — instead of being forced into a base/derived split that does not fit them (R13).
- **Uncertainty propagated, with the cheap method licensed by the expensive one.** An input carries a spread as well as a magnitude, and the calculus carries it through the model by one of three methods of rising cost and fidelity: a linearization, a cumulant method that keeps the shape

of the tails, and a sampling method that keeps the full non-linearity. The three are proved to nest — each coarser one is a projection of the finer one — so reaching for the cheap method is an approximation with a stated relationship to the expensive one rather than an act of faith. The variance that comes out then certifies a coverage interval: a conservative one that assumes nothing about the distribution's shape, an exact one where the shape is known (R14, R18).

- **Knowing when single precision is enough.** Because one model runs at exact reals and at floating point alike, the two can be compared where it matters. An input whose contribution falls below half a unit in the last place of the running sum is *numerically invisible* — its uncertainty was silently dropped — and its dual is a subtraction that is exact yet magnifies the relative uncertainty of what remains. Both are flagged, so "enough precision" is a checkable property of a model at its inputs rather than a habit (R15).
- **Rigor that costs nothing at run time, and reads back as mathematics.** The kinded expression compiles to exactly the term a hand-written bare-number kernel would — same operations, same rounding, nothing added — so what the kinds buy is checking, paid at compile time and not at run time. In the other direction, a definition is rendered back out of its own elaborated form as a typeset equation with the kind bookkeeping suppressed, so what a reviewer reads is generated from what the machine checked, not written alongside it (R21, R25).
- **Where a number came from, and how much of a program is answered for.** A value can carry an auditable record of the graph that produced it — which inputs, through which steps, and where values left the calculus for bare numbers — against a boundary the author *declares* and the machine checks against the boundary the code actually has. Such an audit is always a claim about one named scope, never about the library, and a scope once blessed may only grow: a regression fails the check instead of quietly passing it (R23, R24).
- **Modules that state what they compute, and what may be split.** A module's declared boundary carries the relations its outputs stand in to its inputs — each backed by a theorem the kernel checks, each numerical bound naming a kinded tolerance. The same declaration says which outputs aggregate over a carving of the data, and that is what licenses running the computation in tiles or shards: a theorem says the total is preserved, rather than the schedule being trusted to preserve it (R26, R27).
- **What a module's quantities must satisfy, and by what evidence.** A requirement is stated beside the boundary it governs, in the author's words, with its evidence: a theorem, where the statement closes over the model, spot-checked where the model runs; or a gate against a named referent the model does not define — a datasheet value, a prior, a reference measurement — where it does not. Which of the two it is, the machine reads off the statement rather than taking the author's word, and every output either carries a requirement or says why it needs none (R28).

Between them these bullets name all twenty-eight requirements, each of which is stated precisely below; the status table at the end of that section maps every requirement to the checked declarations that address it.

1.4. What an author writes, and what the machine checks

The Using the library chapter explains each annotation in turn. This section is the summary to have before reading it: what a model written in the calculus asks of its author, and what the author gets back. Since the kind propagates in the type (the paradigm), an author states three things, and the machine checks all three.

First, the equations are written over kinded variables. Where a physics text writes `T = \tfrac{1}{2} m v^2` over bare reals, the model writes it over a `Quantity massK ℝ` and a `Quantity velocityK ℝ`, and every operation between them has to be licensed:

- Addition and comparison are gated by the kind index itself. Two quantities of the same kind add; a velocity added to an angular velocity is a type error (R4).
- Products and quotients go through the *operator table*. A `KindMul k₁ k₂ k` or `KindDiv k₁ k₂ k` entry is registered per sanctioned pair of kinds, each wrapping the `ProductKind` or `QuotientKind` law the ISO 80000 defining relations are stated as: the entry `KindMul massK velocityK momentumK` encodes ISO 80000-4 item 4-8, `p = m\,v`. A file opts in with `open scoped PropertyKindCalculus.OperatorTable`, after which `m * v` elaborates at the momentum kind. Where no entry exists, the product does not elaborate at all (R5, the interaction algebra).
- Functions go through the *function table* (the function calculus), which sorts them into six families by kind signature: kind-preserving (`abs`, `min`, `max`), powers and roots, the dimensionless transcendentals, the trigonometric functions, logarithmic levels, and re-expression against another reference. The trigonometric family is the showcase: a plane angle is dimension one but a distinct kind, so `sin` of an angle is fine while `sin` of a reflectivity is a category error a dimension-only system cannot forbid.

What the switch buys is the two things a bare equation cannot say. A kinded expression combines only where the algebra licenses it. And the kind separates what the dimension cannot: the ForPhysLib classical-mechanics benchmark alone has six distinct kinds at the joule (kinetic, potential, mechanical, the Lagrangian, and König's translational and rotational pair), two at `\mathrm{T^{-1}}` (angular frequency and angular velocity, the same letter `\omega` in two subtrees), and two at `\mathrm{T}` (duration and period), and the distinctions are decidable (`angularFrequency_ne_angularVelocity` is closed `by decide`). The kinds are paid for at compile time, not at run time (R21): the kinetic energy authored through the table, `\tfrac{1}{2}\langle p, v\rangle` built from the edges `m \cdot v \to p` and `p \cdot v \to 2T`, erases to PhysLib's `\tfrac{1}{2} m \langle \dot{x}, \dot{x}\rangle` by the theorem `kineticFromTableQ_erases`.

Second, the boundary is declared. Each source, sink and hand-granted crossing of the paradigm carries its annotation from the boundary family. Inside such a declaration, where no check applies, the mint itself is written as an *attestation*:

```
@[inline, expose] def attest (_why : String) (m : R) : Quantity k R := ⟨m⟩
```

It is definitionally the raw value: the physics context is in the type, `Quantity velocityK ℝ` say, not in the string. What the string does is put the judgment on the record. The boundary audit harvests it and prints it beside the site, so each authored mint is a one-line review artifact rather than an anonymous `⟨m⟩`. The discipline is that `attest` is the last resort, for a classification with no machine-checkable evidence, and the reason should say why no check applies; where a `KindAdmissible` check exists, `Quantity.certify` runs it and keeps the receipt in the type.

One command enumerates all of it: `#kind_boundary_audit ForPhysLib.ClassicalMechanics`, the boundary audit, prints one line per boundary site in that namespace — its tier and, for an attestation, its harvested reason — and ends `42 boundary site(s), all tagged — clean`. The report is pinned with `#guard_msgs` (https://lean-lang.org/doc/reference/latest/Interacting-with-Lean/#hash-guard_msgs), so an entry point nobody declared fails the build. That is not a type error, to be clear: a raw `⟨x⟩` typechecks. The audit is what refuses it (R23, R24).

Third, the requirements are stated. What a module's quantities must satisfy — a bound, a band, a tolerance — is a `Provenance.Requirement` beside the boundary that produces them, and it says by what evidence it is met. A provable requirement's statement closes over the model, so a theorem discharges it, with `Bool` spot checks the command evaluates where the model runs; an empirical requirement names a referent the model does not define — a datasheet value, a prior, a reference measurement, marked `@[kindReferent "where it comes from"]` — so a gate against that referent discharges it, or an attestation with its reason defers it. The category is read off the statement's constants, not chosen: a witness that names a referent is refused as provable, and a gate that names none is refused as empirical. `#kind_requirements` surveys a namespace's requirements as one pinned report, and `#kind_requirement_coverage` asks of every produced port whether a boundary/port-matching requirement governs it. For object scopes, the result member's type supplies the inferred object domain; a missing index or unresolved bearer is an unchecked violation unless the boundary is exempted. A `.sort` scope requires an indexed result and a `Sorted` instance; the census first reduces the actual result bearer's sort inside the member telescope. A definite match governs, a definite mismatch falls back to object scopes or is ungoverned, and a stuck sort follows the object-domain path and may be unchecked. `.decided` governs only when its `deciderProof` proves the decider holds for every result of the member; otherwise the census marks it unchecked. `.all` remains broad (every claim a model makes, with its evidence).

2. Requirements

What follows is the fixed set of requirements PropertyKindCalculus is **specified** to meet. The next section, **Why a calculus, not a taxonomy**, argues **why** a calculus is needed where a taxonomy stops, and the chapters after it develop and prove each requirement; this section states them up front, as the axes the design is judged against, with a status table at the end mapping each axis to the declaration that discharges it.

The twenty-eight requirements, R1–R28, fall into nine groups — the section headings of the generated traceability matrix at the end of this section:

- how kinds are **structured**, and how quantities characterize objects and named parts of named systems (R1–R3, R19, R22);
- how **operations** on kinds are gated (R4–R6);
- how the kind layer is kept **consistent** with the coarser dimension and unit layers (R7, R8, R16, R17);
- how values **aggregate** over parts (R9);
- how values are **represented** — the numeric carrier, vectors under one scalar unit, frames and variance (R10, R11, R20);
- how **classifications** are certified, of kinds and of units (R12, R13);
- how **uncertainty** is propagated, judged numerically adequate, and covered by intervals (R14, R15, R18);
- what the discipline **costs the author** –– erasure and rendering (R21, R25); and
- what **evidence** a value and a module carry — provenance, audit, the checked measurement model, licensed distribution, and the requirements stated on its quantities (R23–R28).

Below, a group whose axes are best read separately takes one heading per axis. R21–R25 entered the catalogue from the ForPhysLib benchmark's design-neutral MR requirements; the trace between the two numberings runs in both directions in the **Requirement validation** subsection at the end of this section.

2.1. Kind structure (R1, R2, R3, R19, R22)

R1 — Kinds are first-class and discriminate within a dimension. Two properties of the same physical dimension can still be different kinds, and the type system must keep them apart. Volumetric water content (`Quantity vwc`), gravimetric water content (`Quantity gwc`), relative permittivity, reflectivity, and emissivity are all dimension-one, yet `Quantity vwc` and `Quantity gwc` must be **different types** — not interchangeable values. (QUDT's and SysML v2's `QuantityKind` is subsumed here as one such kind — a scale-gated leaf — not as the root.) This reaches its limit on ISO 80000-11 *Characteristic numbers*, where **all 115** kinds — every one a dimensionless ratio — share dimension one, so the dimension

functor collapses the entire part to a single point and only the kind layer holds its members apart (see the ISO 80000-11 chapter). The **principled** form of this within-dimension discrimination — naming a kind by the system and component it is *dedicated to*, so volumetric and gravimetric water content differ by their kind-of-property rather than by an identity string — is Dybkær's dedicated kind-of-property (see the Dedicated kinds-of-property chapter).

R2 — Specialization is a lattice, with comparability but not identity. Width, Height, and Diameter each specialize Length, and a kind may specialize several parents at once — a lattice, not a tree. Specialization is a preorder (reflexive and transitive) and induces a **one-way** coercion: a `Quantity Width` may be used where a `Quantity Length` is wanted, never the reverse, and that up-cast is a visible, deliberate loss of information — realized as `Quantity.widen`, the lift along a proved `Specializes`, with the sum and the comparison of comparable kinds licensed **at the join** (`SpecializationLift`). Width and Height remain **mutually comparable** — they share the super-kind Length — while staying distinct kinds. This requirement is now realized on the real standard: the ISO 80000-3 length family (width, height, distance, radius, … of items 3-1.2 … 3-1.12, all of dimension `L`) is specified as a specialization lattice over the general length kind, each species individuated **not by fiat but by an explicit measurement principle** (see the ISO 80000-3 chapter), and the same pattern recurs on Part 4's force family, Part 5's thermodynamic potentials, and IEC 80000-6's AC power family — active, reactive, and apparent power as species of one power kind, carrying three different unit strings (`W`, `var`, `VA`) over one dimension (see the IEC 80000-6 chapter) — and on ISO 80000-7's radiation trios, where radiant, luminous, and photon flux are one measurand in three modes, the radiant and luminous members even sharing a dimension (see the ISO 80000-7 chapter). ISO 80000-11 pushes this furthest: the same name recurs as a *different kind* across its transport-phenomena clauses — the two Froude numbers, the five Stokes numbers, the four Bejan numbers — each sub-suffixed sibling sharing **both** name and dimension (one) with the others, so the measurement principle alone tells them apart (see the ISO 80000-11 chapter).

R3 — General versus individual is type versus term. A kind is the general notion (a **type**); a particular measured value is an individual (a **term** of that type). `Length` is a type; the length of this pencil is a term of type `Quantity Length`. The two are never conflated — the punning OWL permits between a class and its instances is structurally impossible here.

R19 — A quantity characterizes an object; object identity is carried in the type. R3 separates the general from the individual; R19 says **whose** individual. A measured value characterizes some object — this pencil, oscillator A — and that object rides in the type (`IndividualQuantity o k R`), so two same-kind quantities of different objects are different types and `ω_A · x_B` is a compile-time error, not a naming convention. Same kind, different object; the object axis is independent of the kind axis, which is exactly what dissolves the tagging dilemma (a dimension tag can make masses distinguishable **or** addable, never both — the object index makes them both). The object is drawn from an **arbitrary type**, not from the nominal system carrier alone, so a host library's own objects — the particles of a mechanical system, the cells of a mesh — are admissible indices without that library changing; see the object type chapter for what that costs and what it does not buy.

R22 — A quantity is dedicated to a named component of a named sort of system; dedications of distinct sorts are provably distinct, and the particular is the object index's. The sharpened successor of the part of R19 that "object identity" undersold, minted from the ForPhysLib benchmark's two-rover construction (see **Requirement validation** below): with one system there is no way to distinguish "the sum of the parts" from "the sum of some masses of the right dimension", because every mass in scope is a correct summand. `DedicatedKind` records the `System – Component ; kind` triple with the `System` slot naming the sort — one catalogue entry for the fleet (`distinct_of_sort`, `dedicatedFor_congr`) — and the object index is what makes cross-system substitution — rover 1's total mass drawing on rover 2's wheel — a type error rather than a modeling discipline.

2.2. Operation gating (R4, R5, R6)

R4 — Operations are gated by kind (the additive law). Arithmetic that keeps a quantity within its kind is well-typed; arithmetic across incompatible kinds is a **compile-time type error**, not a runtime check. `Width + Width = Width` type-checks; `Width + Height` does not — and, decisively, neither does `Torque + Energy`, even though torque and energy share a dimension. Matching dimension is not licence to add.

R5 — The interaction algebra is a partial, typed, ternary product. Some kinds combine **across** kinds to yield a third: torque times plane angle is energy, force times length is work. The combination is partial and curated — most pairs combine to **nothing** (fuel-consumption times rainfall is a category error, not a number) — and it carries a division dual, with multiplication and division proved inverse. This multiplicative combination is distinct from R4's additive, same-kind gate.

R6 — Operator availability is gated by the measurement scale, monotonically. Which operations even **exist** between two properties is fixed by their scale type, `\mathrm{nominal} \sqsubset \mathrm{ordinal} \sqsubset \mathrm{interval} \sqsubset \mathrm{ratio}`: a nominal property admits only `=`, a ratio property admits `\times` and `\div`, and a richer scale licenses every operation a poorer one does. This gate is **orthogonal** to kind — kind says **which** properties may combine, scale says **which operators** are defined at all. This requirement is now realized on the real standard: ISO 80000-5 lists thermodynamic temperature (ratio-scale) and Celsius temperature (interval-scale) at the **same** dimension `Θ`, so a ratio of Celsius temperatures is undefined where a ratio of thermodynamic temperatures is not — the scale type, not the dimension, separates them (see the ISO 80000-5 chapter). IEC 80000-6 repeats the pattern on electromagnetism: electric potential is gauge-dependent (fixed only up to an additive reference), hence interval-scale, while electric potential difference, of the same dimension `V`, is ratio-scale (see the IEC 80000-6 chapter).

2.3. Soundness bridges (R7, R8, R16, R17)

R7 — Dimension certifies coherence; it does not decide legality. Every kind has a dimension, but dimension is a **consistency check**, not the authorization: matching dimensions are necessary yet never sufficient — which is exactly what lets R4 reject `Torque + Energy`. Formally the dimension map `\dim :`

$\mathrm{Kind} \to \mathrm{Dimension}$ is a **forgetful functor** and a **homomorphism**: $\mathrm{KMul}\ k_1\ k_2\ k_3 \;\Longrightarrow\; \dim k_3 = \dim k_1 \cdot \dim k_2$.

In plain engineering terms: $\dim$ is a deliberately **lossy, one-way** translation from the rich kind layer down to the coarse SI-dimension layer. It keeps only the base-quantity exponents (length, mass, time, …) and **forgets** everything else, so many distinct kinds collapse onto the same dimension and there is no way back — that is what "forgetful" names. Calling it a **homomorphism** says the translation **respects multiplication**: the dimension of a product equals the product of the dimensions (the exponents add), so the kind layer and the dimension layer can never disagree about what multiplying means. It is the discipline of ordinary dimensional analysis, stated and proved as a law rather than performed by hand.

R8 — Units are chosen values of a kind; conversion is a faithful round-trip. A unit is a distinguished value of a kind (a metre is a chosen length), and a measured quantity is a number times a unit. Converting a quantity from one unit to another of the **same** kind and back is the identity — conversion is multiplication by a ratio, defined only within a kind. There is no conversion between units of **different** kinds; that too is a type error, not a runtime check.

R16 — Unit references are faithful. R8 states the capability; R16 and R17 are its proof obligations, split so each can be discharged exactly. R16 covers the **reference** side: commensurability of unit references is an equivalence relation, and the VIM's number-and-reference form of a quantity value round-trips — decompose a value into its number and its unit reference and recompose, and nothing is lost. Faithfulness here is what makes a recorded measurement citable: the reference is part of the value's identity, not a display convention.

R17 — Unit conversion between commensurable units is a faithful round-trip. The **conversion** side of R8's claim, made exact: converting between commensurable units is multiplication by an exact, reciprocal power-of-radix factor — SI decimal or IEC binary — so converting there and back is the identity **on the nose**, not up to floating-point error. The exactness is the point: a conversion factor that is a power of the radix commutes with the carrier's rounding, which is what lets the round-trip law hold at `Float` and not only at $\mathbb{R}$. R20's frame functoriality (`toFrameVector_id`, `toFrameVector_comp`) is this requirement's frame twin: what R17 proves for the unit axis, R20 proves for the frame axis.

2.4. Aggregation (R9)

R9 — Extensive quantities aggregate additively over parts; intensive ones do not. For an **extensive** kind the value over a whole is the sum of the values over its disjoint parts — the mass of an assembly is the sum of its parts' masses. Where this fails it must **not** be assumed: volume on mixing is sub-additive (ethanol and water), and that negation is stated to keep the extensive predicate honest. Tracking which kinds are extensive is the precondition for soundly summing measurements.

Beside the sum and the shared constant sits a third mode that is neither: the weighted mean. It is the first aggregation that **divides**, so it is the first that carries a side condition — the total weight is not zero

— and the first whose law does not survive the descent to an executable carrier. The mode is carried at any `Carrier`, its law is proved where cancellation is available, and its rung at genuine binary32 is a bound rather than an equation. Those three sentences are one structure read at three carriers, not three structures.

2.5. Representation parametricity (R10)

R10 — A quantity value is parametric in its numeric representation type. The magnitude of a scalar quantity is carried at a **representation type** `R`, a type parameter bounded by whatever algebraic structure a given task demands, so the **same** kind-indexed value can be instantiated at whichever `R` the task needs — without changing the kind, scale, dimension, interaction, or extensivity layers above it. This is the axis dependent types make not just possible but **necessary**: the carrier the kind sits over is exactly where proof and execution diverge, and a single quantity must serve both. Three **real** representations matter, and they are not interchangeable — and a fourth **complexifies** any of them:

- **Lean's `\mathbb{R}` — for proofs.** A complete ordered field: the continuous specification against which laws (monotonicity, conversion round-trips, extensivity) are stated and proved.
- **A finite floating-point rounding model — for numerical-accuracy reasoning.** TorchLean's `FP32` (`NeuralFloat` at binary32 precision: round-to-nearest, ties-to-even over `\mathbb{R}`, finite-only, with `toReal : FP32 → ℝ`). It is **non-computable** — it exists to bound rounding error against the `\mathbb{R}` spec, not to run.
- **An executable IEEE-754 type — for exact-execution reasoning and code generation.** TorchLean's `IEEE32Exec` (a bit-level `UInt32` binary32 kernel) that models NaN, infinities, signed zeros, and subnormals, with `toFloat`/`ofFloat` and `isNaN`. This is the representation that actually runs and the one in which exceptional values (NaN) can be reasoned about — what one might call the "exec" float, distinct from the `FP32` rounding **spec**.
- **The complexification `\mathrm{Complex}\ R` of any of the above — for complex-valued quantities.** Many physical quantities take a complex value (the relative permittivity of a lossy dielectric `\varepsilon = \varepsilon' + j\varepsilon''`, an impedance, an AC phasor); complex-ness is a property of the **carrier**, not the kind, so `\mathrm{Quantity}\ k\ (\mathrm{Complex}\ R)` is the **same** kind layer over a complexified carrier. As a functor `R \mapsto \mathrm{Complex}\ R` it lifts each of the three above — `\mathrm{Complex}\ \mathbb{R}` to prove, complex binary32 to certify rounding, `\mathrm{Complex}\ \mathrm{Float}` to run. It is a `Carrier` — and a `LawfulCarrier` whenever `R` is, since complex addition is componentwise so the additivity laws lift for free — carrying the full field `+\,-\,\times\,\div`; but it is **unordered** (there is no `\le` on `\mathbb{C}`), so unlike the real carriers it sits outside the ordinal/interval/ratio carrier tower: complex magnitudes are compared by modulus, not ranked.

Soundness **across** representations is itself a theorem, not an assumption: an exec/spec **refinement** (the executable float result is the rounding of the real result) lets a law proved over `\mathbb{R}` transfer to the executable run. In plain engineering terms: choose the number type to fit the job — reals to prove, the

rounding model to bound error, the IEEE kernel to run and to catch NaN — while the kind, dimension, and unit machinery above is written once and is identical for all three. This refines R1/R3/R4's kind-indexed value into a doubly-indexed `Quantity (k : KindOfProperty) (R : Type)` whose arithmetic is bounded by a `Carrier` typeclass on `R`. The value layer and its additivity laws are **realized** (sorry-free): `Quantity k R` with same-kind addition, the additivity laws proved once over any lawful carrier, and three carriers — `Int` and $\mathbb{R}$ (lawful, for proof) and `Float` (executable, deliberately not lawful). The exec/spec **refinement** across representations is now **also realized** (sorry-free): the abstract bridge `CarrierRefinement` and its kind-indexed capstone `Quantity.add_refines` — a law over the lawful spec carrier descends to the exec carrier as one rounding step — instantiated in the separately-built `Torch` library at TorchLean's binary32: `FP32` (the rounding **spec**, an **unconditional** refinement of `\mathbb{R}`) and `IEEE32Exec` (the **executable** kernel, refining `\mathbb{R}` on the finite/no-overflow path, with overflow surfaced as an explicit side condition rather than silently dropped).

The bridge spans the whole arithmetic surface, not the additive part alone: `MulRefinement` and `DivRefinement` carry the same forgetful map and rounding to `\times` and `\div`, and `Quantity.mul_refines` / `Quantity.div_refines` lift them across a **licensed kind change** — the exec product of a k_1 and a k_2 quantity, viewed in the spec carrier, is the rounding of the spec product at the product kind k. The rounding obligation and the kind license are independent and compose, so a rounding step cannot launder a product the kind calculus refuses. This is what lets a weighted mean, a ratio, or a sensitivity coefficient cross at all — none of them is an addition. The division law is unconditional at the **specification** rung only because `\mathbb{R}` totalizes `x/0`; the executable rung's `Quantity.div_refines_exec` asks that the executable quotient be finite, which is where a zero denominator is actually caught: a finite dividend over zero is an infinity or a NaN.

The real representations above and their complexifications, ordered from proof to execution, are the carrier ladder: each representation is a rung, and a definition written once over an abstract carrier is instantiated at every rung by one map. **Laws transfer across the ladder; side conditions do not.** That asymmetry is the counterpoint to everything above, and it is worth stating as a rule rather than as a property of one theorem. The bridge carries a **law** from the lawful carrier to the executable one at the cost of a single rounding step. It carries a **side condition** — a hypothesis a definition holds as a field, so that the degenerate case is not a term to be handled — not at all. The reason is visible in the statement: a condition phrased in the carrier's own arithmetic says a different thing at each carrier, and the different things are logically independent. A field that is one line of Lean, quantified over the carrier, is not thereby carrier-generic; it is carrier-relative.

The weighted mean is the worked instance, because it is the first aggregation mode with a side condition at all. A carving's license says its total weight is not the **carrier's** zero. At binary32 that is a claim about the **rounded** fold; the bound relating the computed mean to the exact one also needs a claim about the **exact** fold; and neither implies the other, in either direction:

- three weights `2^{24}`, `1`, `-2^{24}` sum exactly to `1` and total exactly `+0` on the grid — a mean that exists, refused;

- four weights `2^{24}`, `1`, `1`, `-(2^{24}+2)` sum exactly to `0` and total exactly `-2` on the grid — a mean that does not exist, computed, with a finite and plausible value and no exceptional flag anywhere to catch it.

So testing a computed denominator against zero is a guard on the number the machine formed, not on the quantity being defined. Two arrangements put the two conditions back together, and they rank. **Do not round the denominator**: where the refinement's rounding is the identity — a count folded in `\mathbb{N}`, converted once — forgetting the total **is** the total of the forgotten weights, so there is one condition rather than two. This is the strongest fix and it is available whenever the weights are counts or indicators, which is the ordinary case for a validity-masked average. **Or keep the weights nonnegative**: both failures above are built from cancellation, and ruling it out restores the equivalence for weights that are genuinely real-valued — masses, areas, durations, coverage fractions.

The reading generalizes to every `\mathrm{Prop}` field phrased in carrier operations. The other instance already in the library is a real-valued unit's nonzero-magnitude license.

2.6. Value representation: vectors and scalar units (R11)

R11 — Units are scalar; a vector quantity is a numerical array times one scalar unit. Following ISO 80000-2 §18 (scalars, vectors and tensors), a vector (or tensor) quantity is written as a **numerical** vector multiplied by a single unit, and "all units are scalars" — **not** as a collection of per-coordinate quantity values each carrying its own number × unit; the quantity itself is independent of the coordinate system while its numerical components are not. This is exactly the R10 carrier taken at a numerical-array type: a vector quantity is `Quantity (k : KindOfProperty) (Fin n → R)` — one kind `k`, the numbers an `n`-vector in the carrier `R`, and one **scalar** `MetrologicalUnit` of the kind for the whole vector — and the additivity laws transfer to the vector carrier by the **same** parametric proof used for scalars (the pointwise `Carrier (Fin n → R)`). It is the ISO 80000-2 §18 **numerical-array** reading of value representation, and it aligns with the VIM's "a quantity is scalar; a vector/tensor is a composite of scalar quantities" — the priority SysML v2 inverts (see **Why a calculus, not a taxonomy**). The **structural** apparatus is a separate axis, specified as R20 below.

One consequence of the numerical-array reading needs stating here because it is a gate rather than a capability. A carrier supplying `Mul` is not thereby supplying **the** multiplication of magnitudes: Lean's `Mul (Fin n → R)` is pointwise, so the componentwise product of two position vectors would elaborate through `Quantity.mul` as a correctly kinded, correctly dimensioned area that is no physical quantity at all. `ScalarCarrier` is the carrier-side gate that refuses it — the missing half of the doctrine `QuantityClassification` already stated, that a kind's scale says an operation is **meaningful** while a carrier's tier says the representation can **compute** it. It is a marker class, and deliberately so: the same `\iota \to R` is a scalar carrier when its indices are independent **samples** (the batch carriers) and not when they are **components**, and no property of the type decides which. The operations §18 does license on a vector quantity — the scalar and vector products — are not pointwise and belong to R20.

2.7. Value representation in the structural sense: frames and variance (R20)

R20 — Components carry the frame they were read in and the variance they transform with; a scalar product is invariant under an orthonormal change of frame and a component is not. R11 above specifies the numerical half of ISO 80000-2 §18 and is silent on the sentence the standard puts next to it: the quantity is independent of the choice of coordinate system while its numerical components are not. Independence of a choice is not a property an array has — it is a property of how the array behaves when the choice changes — so `Quantity k (Fin n → R)` does not record that a choice was made at all.

`InFrame f var k R` adds the two indices that do: the `Frame` the components were read in, and the `Variance` (scalar, vector, rank-2) whose transformation law they obey. Three points make this an axis of its own rather than a second kind of unit:

- **A frame change is dimension-blind.** A **unit** change acts through the dimension — the factor `UnitScale.dimScale` computes is a function of it — so the dimension layer supplies unit covariance unaided (R7, R17). A **frame** change acts through the carrier: position, velocity, force and field all turn by the same matrix, and their dimensions have no bearing on it. Nothing multiplicative reaches variance, so no refinement of the dimension basis reaches it either.
- **The frame is an index, where the unit is not.** A kind determines a canonical unit and a conversion is recoverable from the dimensions; neither holds of a frame, and two readings of one quantity in two frames agree in kind, dimension, unit, object and carrier. There is nothing else left to catch the mixture.
- **What survives is a theorem.** `dot_toFrameVector` proves the scalar product invariant under an orthonormal change, and `component_not_invariant` exhibits a component that differs — stated with a witness so the first result cannot be over-read. Together with the functoriality of the action (`toFrameVector_id`, `toFrameVector_comp`) these are the frame twin of R17's unit round-trip.

The definitions are carrier-parametric and Mathlib-free, so a change of frame runs at `Float` and specifies at `\mathbb{R}`; the laws live in `FrameReal`, the same split `Quantity`/`QuantityReal` already uses.

Scope, stated as scope. The transformations are linear changes of **Cartesian** frame, and under an orthonormal change the covariant and contravariant laws coincide (`C^{-1} = C^{\top}`), so one `vector` variance suffices. `IsOrthonormal` is a hypothesis on the theorems that need it and never a field of `FrameChange`, so a general linear change remains expressible while the results state their price. Still owed, and named here rather than assumed away: the covariant/contravariant split for non-orthogonal changes, curvilinear coordinates, bound-versus-free vectors, and affine frames — an origin as well as a basis, which positions need and velocities do not.

2.8. Verified classification: kind-laws that instantiate at the quantity level (R12)

R12 — A classification is a certificate, not an assertion; kind-laws instantiate at the quantity level. By default a kind is a **tag**: `Quantity k R` records the kind but the magnitude is arbitrary, so a value can be labelled with a kind it has not earned. R12 upgrades a classification to a **certificate** — a proof that a quantity satisfies its kind's **defining relation** (the formalized ISO 80000 **Remark**) to quantities of the kinds it is built from. A length built as speed × time is certified by construction; a bare `⟨999⟩` cannot be certified against those factors. The kind becomes a **refinement** (a tag plus its earned proof).

The reason this strengthens the calculus is **instantiation**. Every kind-law is stated as a carrier-parametric universal over quantities with the kind-relation as a premise, so it instantiates to a quantity-level fact by ordinary application — the inter-kind generalization of R10's carrier-parametric laws — and any property of a kind's defining relation transports to **every** quantity certified under it (every certified surface area is `≥ 0`, because the area element is). The pattern comes in three algebraic shapes — **product** (area = length · length, energy = force · length), **quotient** (speed = length / duration, plane angle = arc / radius), and **reciprocal** (frequency = 1 / period, curvature = 1 / radius) — each a kind-law family in the core; analysis-shaped relations such as area as a surface integral `∬ √g du dv` follow the same discipline in the layer where their mathematics lives. These are exercised directly on ISO 80000-3's own **Remarks** (see the ISO 80000-3 chapter), where, for instance, the plane angle is **computed** to be dimension one because it is a ratio of two lengths. R12 unifies the two halves of the standards work: the formalized remark **is** the defining relation, and classification is the witnessed instantiation of it.

2.9. Unit classification (R13)

R13 — Unit classification needs a third category beyond base and derived: scale-spanning units. The SI sorts units into **base** (kilogram, metre, second, ampere, kelvin, candela, mole) and **derived** (everything built from them). Finkelstein and Whitehead (*Eur. J. Phys.* 46 (2025) 035701) argue this dichotomy is insufficient: of the seven base units only four — the kilogram, metre, second, and ampere — are genuinely **dimensionally independent**, while the kelvin (the dimension of energy, via the Boltzmann constant), the candela (the dimension of power, via the luminous efficacy), and the mole (a dimensionless number, via the Avogadro constant) are not — yet each is kept a distinct coherent unit because it carries a **human-selected coefficient** sized to span vast scales. They are **scale-spanning units**, a third category. R13 specifies that category, and its point is the unit-layer twin of R1: **whether a unit is scale-spanning is not a function of its dimension alone**. The dimension layer can detect that the candela and the mole are **mechanically reducible** (to power and to one), but it cannot detect the kelvin — this work keeps thermodynamic temperature an independent dimension (ISO 80000-5 needs it), so the kelvin's scale-spanning character is visible only through its coefficient, never through dimensional analysis. The ampere, by contrast, **is** dimensionally independent (it carries the electromagnetic generator), a genuine base unit — so the third category is non-empty and distinct from

the first. In plain engineering terms: just as a **kind** carries more than its dimension, a **unit's category** carries more than its dimension; the candela and mole (ISO 80000-7) and the kelvin (ISO 80000-5) are where this lands on the standard, the ampere (IEC 80000-6) the base-unit foil. The category's **nonlinear** extreme is the level — the decibel of ISO 80000-8 and of EM practice (dBm, field levels) — and it is now construction rather than aspiration: `LevelKind` presents a ratio root kind through a logarithmic chart against a **named reference**, with the reference and the ISO 80000-1 power/root-power role carried as kind identity (dBm and dBW are distinct kinds), the level kind ordinal-as-a-kind so the certified addition is unprovable at it ($L_1 + L_2$, the classic domain error, is a type error), differences landing in a provably reference-free **gain** kind that composes by ordinary certified addition, and the role-independent energetic combination as the licensed way two sources meet. The kelvin is the linear member of R13's category; the decibel is its chart-bearing one, and it lives beside the kind — deliberately not in a unit system of pure rescalings, which cannot express even the affine °C, let alone a logarithm with a reference.

2.10. Uncertainty and numerical adequacy (R14, R15, R18)

R14 — Output uncertainty is computed by a provably nested ladder of methods. A measured input quantity carries not just a magnitude but a **dispersion**; given the uncertainty of the inputs of a model $Y = f(X_1,\dots,X_k)$, the calculus computes the uncertainty of Y. The dispersion is an **additive** descriptor placed alongside a quantity — moments and cumulants, an inverse-CDF sampler, and a support range — never a change to the carrier tower, the same discipline R11 used for vectors. Three methods estimate the output uncertainty at increasing fidelity and cost: GUM linearization (combine variances), Willink's cumulants method (combine the second **and** fourth cumulants, recovering the tail shape), and the derivative-free SSPRC sampling method (propagate each input separately and combine by convolution, capturing full non-linearity). The organizing claim is that these form a **provably nested ladder** $\mathrm{GUM} \subset \mathrm{Willink} \subset \mathrm{SSPRC}$: each coarser method is a projection of the finer one, all resting on the additivity of cumulants under independent summation. The linearized methods' sensitivity coefficients $c_i = \partial f/\partial X_i$ are **not** hand-supplied — they come from instantiating the write-once model at an autograd carrier, R10's representation parametricity put to a new use. This requirement is realized through Stages 1–2: the reference Monte Carlo propagator, the GUM and Willink combines (reproducing both source papers' headline numbers as checked facts), the autograd coefficients, the derivative-free SSPRC pipeline (recovering the fictive model's full non-linear output uncertainty at ~67× fewer model evaluations than Monte Carlo), and all five ladder theorems — cumulant additivity (T1), $\mathrm{gum} = \mathrm{willink}|_{\kappa_4 = 0}$ (T2), convolution adds cumulants (T5), Willink as the (κ_2,κ_4)-projection of the linearized SSPRC (T3), and the affine reference equalling the mean (T4) — are proved over $\mathbb{R}$. The development is the Uncertainty quantification and numerical adequacy chapter, and the full design is recorded in the project's `UNCERTAINTY.md`.

R15 — A floating-point representation is numerically adequate iff it loses no information at the scale of the input uncertainties. R10 gives a model exact reals to prove with and IEEE floats to run with; R15 asks the sharper question the pairing makes possible — does the floating-point representation silently drop an input uncertainty that **matters**? "Matters" is made precise by R14's descriptor: the scale

is the input dispersion itself, which is exactly why the two areas **share** one descriptor rather than being two features. The textbook failure becomes checkable — an input whose contribution `c_i\,u_i` falls below half a unit in the last place of the accumulated sum is **numerically invisible**, a silent corruption of the uncertainty result; its dual is exact-by-Sterbenz cancellation that nonetheless amplifies **relative** uncertainty. Because the model is written once over `[NumCarrier α]`, the check is obtained **for free** by instantiating it at an analysis carrier that tracks each value's magnitude and carried uncertainty and flags swamping and harmful cancellation — sound because the swamping threshold is exactly half a ulp, proved on the rounding grid over `ℝ` and grounded in TorchLean's binary32 `FP32` unit-in-the-last-place lemmas and its proven interval arithmetic. This requirement is realized through Stage 3: the executable `Adequacy` carrier (a `NumCarrier` running the swamping and cancellation checks over `Float`, with a worked flagged/clean example pair), and the three theorems over `ℝ` — A1 absorption (a sub-½-ulp perturbation is invisible, its converse fixing the threshold), A2 Sterbenz (near-equal subtraction is exact, its dual amplifying relative uncertainty), and A3 verdict soundness (the flag holds **iff** the contribution is lost — sound and complete). Stage 3.1 then lifts A3 to the whole evaluation: A3′ (`DagBound`) abstracts a write-once model as a binary32 +/− DAG and proves that the `FP32` measurand's variation over an input box reproduces the exact `ℝ` variation up to a DAG-additive rounding budget (composed from the per-operation `FP32` half-ulp bounds), collapsing to **equality** when no site rounds anywhere — so rounding is the sole source of the gap. Stage 3.2 then discharges the FLT-level Sterbenz: a TorchLean PR (`neural_generic_format_FLT_sterbenz`) lifts A2 from the self-contained `FLX` model to the gradual-underflow format binary32 actually uses (`fexp32`), so `\mathrm{round}_{32}(u - v) = u - v` for near-equal representable operands is a theorem about the real rounding operator (`round32_sterbenz_exact`), and a near-equal binary32 subtraction is lossless. Stage 3.3 then closes the executable ↔ spec gap: Lean's host `Float` is binary64 (model-backed on the pinned toolchain, but not the float32 this verdict is about, and its `log2`/`exp2` are `opaque` externs), so a second TorchLean PR gives TorchLean's **computable** `IEEE32Exec` model an executable ULP query (`ulpExp?`, its answers proved equal to ulp_{32}, answering on exactly the finite fragment) and an absorption test (`absorbs`, the float32 sum unchanged) that is **sound** against the specification — when it fires, the exact real sum rounds back under round_{32} (`exec_verdict_sound`). So the computed adequacy verdict is provably the specified one, the residual `Float32 ↔ IEEE32Exec` step being an upstream assumption typeclass rather than an axiom. Stage 3.4 then wires the Axis-U significance yardstick: `analyze` drives **both** areas from one `InputDist` descriptor — instantiating a single write-once kernel at the autograd carrier for the sensitivities c_i and at the adequacy carrier (seeded from the same descriptor) for the verdict — so a model is adequacy-checked at the scale $c_i \cdot u_i$ its own uncertainty descriptor defines (the soundness inherited from the A3 verdict). What remains is extending the DAG to ×/÷, scoped in the project's `UNCERTAINTY.md`.

R18 — The recorded variance certifies a coverage interval. Once an output's uncertainty is a distribution, VIM 2.36–2.38 ask for a **coverage interval** — a probability attached to an interval about the mean. The `variance` the R14 descriptor already carries must certify one, in two tiers graded by what is assumed about the distribution's shape: distribution-free, the interval of `k` standard uncertainties about the mean has coverage at least `1 - 1/k^2` — the complement of Chebyshev's inequality, honest but conservative (`k = 2` gives `\ge 75\%`, not the Gaussian `95\%`, which is a shape assumption); and, for a bounded family, **exact** — the centered interval of half-width `h` of a uniform on `[m - \delta, m + \delta]`

has coverage exactly `h/\delta`. Neither tier needs a **true value** — the sense in which coverage is provable while **accuracy**, closeness to nature, is not. The development is the Coverage intervals section of the uncertainty chapter; what a coverage factor is then **for** at a limit — the guard band of ISO/IEC Guide 98-4, derived from a stated risk rather than chosen, and a deployed band read as coverage or as a systematic — is the chapter's Conformity assessment section, which no catalogue requirement claims.

2.11. Ergonomics and erasure (R21, R25)

R21 — The kind layer erases: the ceremony is pure. The standing objection to any kind layer is authoring cost, and the honest answer is not that the cost is zero — the ForPhysLib benchmark scores PKC **worst** of four designs at longhand authoring — but that the cost buys checking and nothing else: the kinded authoring computes **exactly** the bare-carrier value, and the curated-operator form (`KindMul`/`KindDiv`, the `OperatorTable`) elaborates to the **same term** as the longhand form (`hmul_eq_mul`, one `rfl` that transports every certificate). So `m * (ω * ω) * (x * x)` is character-identical to bare reals, and what remains of the cost is the per-model operator-table setup — a price the requirement makes measurable rather than deniable. This is the **write once correctly, go fast automatically** slogan stated as an obligation: correctness lives in the types and is paid at compile time, and the number that runs is the one a hand-written kernel would compute.

R25 — A kinded definition renders back as typeset mathematics. The reading-side twin of R21's writing-side claim. A definition annotated `@[pkc_math]` is lifted from its own elaborated term into a presentation language (`MathTerm`), normalized faithfully, and pretty-printed to a LaTeX equation in the declaration's docstring — with the kind bookkeeping suppressed and the notation (an object index as a subscript, a conventional symbol like `\sigma^0`) recovered rather than lost. The rendering is pinned with `#guard_msgs` (https://lean-lang.org/doc/reference/latest/Interacting-with-Lean/#hash-guard_msgs), so it is a regression artifact, not a comment.

2.12. Provenance and audit (R23, R24)

R23 — A value's provenance is recordable and auditable. Beyond **what kind** a quantity is (R1) and **whose** it is (R19, R22): **where it came from**. `Provenance ν κ` records a kind-typed value-flow graph — ports, node introductions each carrying an evidence tier, hyperedge occurrences, and the exits where values leave the calculus for the bare carrier — and `Contract` is the boundary an author **declares** for a scope, checkable against the boundary the scope actually computes (`Contract.discharges`). A membership choice can be wrong, and this is the structure it can be wrong **about**: a changed part list changes a difference list, never passes silently.

R24 — An audit is a claim about its scope, never about the library. The discipline that keeps every other requirement's evidence honest at adoption time. The unkinded surface of a scope is **enumerable** (`boundarySites`), reported against a tier vocabulary that includes the interface tier — the boundary where erasing to a bare carrier is the sanctioned design, so an audit does not score a Mathlib `fderiv` call as a defect — and a blessed scope only grows: the ratchet (`#kind_mint_ratchet`) compares the current

ledger against the pinned one and **throws** on regression, so a clean audit cannot silently shrink to stay clean.

2.13. The measurement model, licensed distribution, and stated requirements (R26–R28)

R26 — A module's behavior is a checked measurement model. The metrological-modularity chapter's behavior clause, stated as an expressiveness requirement: a declared boundary carries relation edges (`Provenance.Relation`) whose witnesses are kernel-checked theorems; every relation kind has a conclusion-shape check; a `boundedBy` edge names a kinded tolerance; named side conditions are checked against the witness statement; and every carrier rung beyond the witness's own is answered for by a named repair or restatement (`RelationLicense`) — a rung claimed with neither is refused. The checks are the elaborator's (`#kind_relation`, surveyed by `#kind_relations`), pinned by acceptance and refusal probes for all four relation kinds.

R27 — A module's distribution is licensed by declared extensivity. The mereology clause as a verifiable requirement: produced ports carry aggregation classes (`Provenance.AggregationClass` — the §13.5 vocabulary of the extensivity chapter, extended by the count-keyed, parameter-conditioned, and interface-licensed classes, each naming the tolerance, sortal, transport, or law that is the content of its claim), and recarving or sharding a module's batch axis is licensed by theorem from those declarations: `Recarving.distribution_license` — one map, every declared-extensive output's total preserved — with the quasi-extensive price at most both carvings' joins times the per-join tolerance, tied to the rounding budget where the carrier demands it. The clause's hygiene is the elaborator's (`checkAggregations`, rendered in every `#kind_contract` report), pinned by acceptance and refusal probes.

R28 — A module's requirements are stated with their evidence. The requirement clause of the metrological-modularity chapter, stated as an expressiveness requirement: every produced port of a model boundary is governed by a `Provenance.Requirement` or exempted with a reason (`@[kindRequirementFree]`); object scopes are compared with the result domain inferred from the member's return type, while missing indices or unresolved object domains are unchecked violations unless exempted; `.sort` requires an indexed result and a `Sorted` instance; the census first reduces the actual result bearer's sort inside the member telescope, then falls back to object scopes or is ungoverned on a definite mismatch, while a stuck sort follows the object-domain path and may be unchecked. A `.decided` scope requires a `deciderProof` theorem proving that its decider holds for every result of the governed member; otherwise the port is unchecked. A provable requirement's witness is a sorry-free theorem, or a theorem edge to a specification boundary, whose statement names no `@[kindReferent]`-marked declaration, with `Bool` spot checks the command evaluates where the model runs; an empirical requirement names marked referents and a gate mentioning one, and no witness; a claimed absence is a license the environment holds no instance of; and an attestation never stands beside a discharge. The checks are the elaborator's (`#kind_requirement`, surveyed by `#kind_requirements`, censused by `#kind_requirement_coverage` and gated by

`#kind_requirement_clean`), pinned by acceptance and refusal probes; what the clause buys is argued in every claim a model makes, with its evidence.

2.14. Requirement validation — the ForPhysLib benchmark

R21–R25 did not come from introspection; they came from **validation**, in the systems engineering sense: not "did we build the calculus right" (verification — the traceability matrix below) but "did we specify the right calculus". The instrument is the ForPhysLib benchmark (`ForPhysLib/REQUIREMENTS.md`, MR1–MR32): thirty-two requirements stated design-neutrally against concrete physics — the same harmonic oscillator typed four honest ways, and a survey of PhysLib's twelve Markdown `API-map.md` files, whose prose cites numbered Lean References — where each MR is scored not only for PKC but for three rival designs and for a shipping library none of them authored. Where an MR restates an R, that is the R **validated adversarially**: R1 asserts PKC discriminates within a dimension; MR4 hands the same claim a physics problem that plain reals, `WithDim`, and Buckingham-π tagging attempt first and fail structurally. The benchmark can also **fail** PKC — it scores MR11 against it — which is what makes its confirmations evidence rather than advertising.

The trace, in both directions:

- **Adversarial restatements** — the MR is the R given a physics test case: MR4 ↔ R1; MR5 ↔ R6; MR7 ↔ R19; MR8 ↔ R9; MR13 and MR27 ↔ R10; MR17 ↔ R11; MR18 and MR26 ↔ R20; MR28 ↔ R12; MR20 ↔ R22; MR24 ↔ R23; MR30 and MR31 ↔ R24; MR11 ↔ R21; MR12 ↔ R25.
- **Delegated** — the MR is discharged by the layer **beneath** the kind layer, which the calculus consumes rather than reproves: MR1 → R7 + R4, MR2 → R5 + R7, MR3 → R17 + R16. This is why the benchmark's Tier 1 is the tier PhysLib's own `WithDim` wins outright, and it is a feature of the layering, not a gap in it.
- **A numbering trap, recorded so it stays harmless:** MR15 (exec and spec agree) traces to **R10** — the `CarrierRefinement` bridge and `Quantity.add_refines` — and **not** to R15, despite the number. R15 is the strictly sharper claim (adequacy relative to the **input uncertainties**, not mere rounding-step refinement), and no MR states it yet; it appears in the unvalidated list below.
- **Minted** — MRs whose obligations no R stated, now R21–R25: the erasure and operator-table purity of MR11 (R21), the rendering of MR12 (R25), the system-scale dedication of MR20 (R22), the provenance of MR24 (R23), and the audit-scope discipline of MR30/MR31 (R24).
- **Deliberately not minted** — MR23 (interface quantities naming both systems they join) and MR25 (genericity over the geometric dimension) have no PKC machinery to annotate yet, and a catalogue row with no possible annotation is exactly the drift the matrix exists to prevent. They are staged in `ForPhysLib/PLAN.md` and enter the catalogue when their first declaration does.
- **Not yet validated** — R13 (scale-spanning units), R14 (the uncertainty ladder), R15 (numerical adequacy), and R18 (coverage intervals) have no MR counterpart. For R13, R14 and R18 the

benchmark's physics lacks the phenomena; R15 falls with R14, being defined relative to input uncertainties. That bounds what the benchmark's confirmation is evidence **for**, and `ForPhysLib/PLAN.md` stages the case studies that would reach them.

- **Validated late, in both directions** — R2 (the specialization lattice) was the near miss: the oscillator's `H = T + V` is a specialization family the first scoring pass never interrogated. MR32, appended, now does — probes in all four attempts and the scorecard — and validates R2's lattice while exposing what nothing above it consumed: `Quantity`'s addition was same-kind and no quantity-level operation took a `Specializes` proof, so the licensed sum at the join (`T + V` landing at the kind both terms provably specialize) was machinery owed — the specialization twin of R9's licensed aggregation. That debt is now paid: `SpecializationLift` supplies `Quantity.widen` (the one-way lift along a proved specialization), the sum and the comparison at the join, and the curated `KindJoin` table through which `T + V` elaborates — and the MR32 probes flipped from to , leaving MR11 (whose residue `kind_algebra` has since pushed to the kind equations themselves) as the one requirement the calculus's own benchmark does not let it sweep.

2.15. Out of scope (for now)

User-specified base units — making the dimension system parametric in the chosen base set (so a quantity's dimensional decomposition is relative to a choice of generators, expressing e.g. Gaussian-CGS electromagnetism, natural units, or the four-base Finkelstein system) — was out of scope in an earlier draft and is now **delivered**. We made PhysLib's `Dimension` itself parametric in its basis and contributed the change upstream (`github.com/leanprover-community/physlib/issues/1441`, pull request 1447), where it has since been **merged**; PKC consumes it from `leanprover-community/physlib` master. The former fixed five-generator type is recovered as the default instance, so no existing result changes. The contribution did not stop at the dimension side: the **unit** twin is parametrized in the same basis (`UnitScale B` with its conversion-factor homomorphism `dimScale`, which PKC's VIM4 §1.22 prefix conversion instantiates at a single generator), and the seven-generator ISQ basis together with the embedding/projection bridge to the five-generator one are upstream as well. The division of labour that leaves is the one this development argues for throughout: the dimensional algebra — bases, homomorphisms between them, unit scaling — belongs in the physics library, while the kind layer and the catalogue's **stances** belong here. PKC then **realizes** the angle-augmented basis — PhysLib's five generators together with **angle** — as one instantiation among several (Gaussian-CGS, natural units, the four-base Finkelstein system, the seven-generator current-based ISQ of ISO/IEC 80000-1): in it plane angle, solid angle, and a pure number are dimensionally distinct (solid angle the square of plane angle, `sr = rad²`) and torque is energy **per angle** (so `torque ≠ energy`), the very separations the five-generator group cannot draw. The ISO 80000 catalogue stays faithful to the standard (SI convention, plane angle dimension one) in the five-generator basis and lifts into any of these along the injective embedding of those generators, with every kind invariant under the lift — a proved change-of-basis theorem. Not every such lift is an embedding of generators: passing to the current-based ISQ sends the charge generator to the **product** `I·T` (the coulomb as ampere-second), so it is realized not by reindexing generators but as a genuine group homomorphism, proved injective and hence lossless, and paired upstream with a

projection that retracts it (`Dimension.toISQHom`, `toISQHom_injective`, `fromISQHom_comp_toISQHom`; consumed in `IsqBase`). The catalogue turns this to account for **citation**: its kinds are dimensioned internally over PhysLib's charge generator, yet the item-index dimensional formulae it prints are rendered in the ISQ base quantity current, so a capacitance reads `M⁻¹·L⁻²·T⁴·I²` exactly as IEC 80000-6 tabulates it rather than in the internal charge encoding. The same instantiation shows the mole/candela reduction to be a stance rather than a lack: the ISQ **offers** independent generators for amount of substance and luminous intensity, and the catalogue declines them (the mole stays dimension one, the candela stays power under the lift), so R13 is a modeling choice the change of basis preserves, not an artifact of the five-generator type. Committing the catalogue to one reformed basis would merely privilege a single contested proposal, which is exactly what the parametricity avoids. So the controversy over which units are base is answered on both layers: a kind is invariant under the choice of base, and only its dimensional shadow changes. R13 captures the specific base/scale-spanning/derived controversy over the five-generator basis.

Value representation in the structural sense — coordinate frames, tensor variance, bound-versus-free vectors, and frame transformations — is a **separate, orthogonal** axis, distinct from R10's **numeric carrier** `R` and R11's **numerical array**. Its principled home is an index **over** the kind, never a layer the kind hangs beneath (the inversion the **Why a calculus, not a taxonomy** section charges against SysML v2), and R20 above now occupies exactly that position: `InFrame f var k R` indexes a reading by its frame and its variance, and the invariance of a scalar product under an orthonormal change of frame is a theorem rather than a convention.

The axis is **specified for Cartesian frames**, which is less than the full requirement. What R20 states as owed, and this section keeps owed with it: the covariant/contravariant split for non-orthogonal changes (under an orthonormal change the two laws coincide, which is why one `vector` variance suffices there), curvilinear coordinates, bound-versus-free vectors, and affine frames. Naming the remainder is what keeps the boundary honest now that the first part is discharged.

2.16. The requirements at a glance — the traceability matrix

The requirements fall into two kinds of obligation. A **verifiable** requirement is a truth-apt claim about the calculus — one that is either true or false of it — discharged by a **checked theorem**: PropertyKindCalculus **proves** it. An **expressiveness** requirement is a capability the type system must afford — general-versus-individual (R3), units as chosen values of a kind (R8) — discharged not by a theorem but by a **construction that typechecks**: that the witnessing declaration elaborates under CI **is** the demonstration. Conflating the two would make a capability look like an unfinished proof; separating them lets each be reported in its own honest vocabulary.

PropertyKindCalculus *proves all 20 verifiable requirements* and *demonstrates all 8 expressiveness requirements* — the capabilities a type system affords by construction rather than by theorem.

The matrix below is **generated** from the typed `@[requirement …]` annotations carried by the Lean declarations (harvested by `PropertyKindCalculus.Requirements`), not maintained by hand. It is organized

by the requirement groups introduced above; within each group, every requirement lists the declarations that *specify*, *prove*, *implement*, or *exemplify* it, linked to that declaration's blueprint node. The **Status** column is likewise derived, not asserted, in the vocabulary of each requirement's kind: a **verifiable** requirement reads *proved* and shows links to the very theorems that discharge it; an **expressiveness** requirement reads *demonstrated* and links to the constructions that exhibit it; an undischarged one reads *specified* (addressed) or *unaddressed*.

Because every annotation attaches to a **real** declaration and every status is a function of those annotations and the requirement's kind, the matrix cannot drift from the source — a renamed declaration or a missing proof is a build-time fact, not a stale cell. (This replaces the hand-maintained requirements-at-a-glance table.)

Requirement	Status	Role	This work
Kind structure			
R1 — Kinds are first-class and discriminate within a dimension	dim_not_injective	specifies	Quantity
		specifies	KindOfProperty
		proves	dim_not_injective
		exemplifies	`vwc`
R2 — Specialization is a lattice, with comparability but not identity	`widen_widen`, distinct_of_examPrinciple, trans, `comparable`	specifies	`widen`
		specifies	Specializes
		specifies	MutuallyComparable
		specifies	ExaminationPrinciple
		specifies	`leAt`

Requirement	Status		Role	This work
			specifies	`addAt`
			specifies	`KindJoin`
			proves	`widen_widen`
			proves	distinct_of_examPrinciple
			proves	trans
			proves	`comparable`
			exemplifies	`mr32_attempt4_swept`
			exemplifies	`width`
R3 — General versus individual	`pencilWidth`		specifies	Quantity

Requirement	Status		Role	This work
is type versus term				
			specifies	KindOfProperty
			exemplifies	`pencilWidth`
R19 — A quantity characterizes an object; object identity is carried in the type	`wc1`		specifies	IndividualQuantity
			proves	`distinct_of_sort`
			exemplifies	`wc1`
R22 — A quantity is dedicated to a named component of a named system; dedications of distinct systems are provably distinct, so a part-quantity of	`distinct_of_sort`		specifies	DedicatedKind

Requirement	Status		Role	This work
one system cannot stand in for a part-quantity of another				
			proves	`distinct_of_sort`
			exemplifies	`no_dilemma`
Operation gating				
R4 — Operations are gated by kind (the additive law)	laws_parametric		proves	laws_parametric
			implements	`add`
			exemplifies	`lenA`
R5 — The interaction algebra is a partial, typed, ternary product	kMul_iff_kDiv		specifies	KDiv
			specifies	`InteractionAlgebra`

Requirement	Status		Role	This work
			proves	kMul_iff_kDiv
			exemplifies	torque_angle_work
R6 — Operator availability is gated by scale, monotonically	allows_mono		specifies	ScaleType
			proves	allows_mono
			exemplifies	`bloodGroup`
Soundness bridges				
R7 — Dimension certifies coherence; it does not decide legality	dim_homomorphism		specifies	toDimension
			proves	dim_homomorphism
			exemplifies	`torque_eq_energy`

Requirement	Status	Role	This work
R8 — Units are chosen values of a kind	`centimetre`	specifies	MetrologicalUnit
		exemplifies	`centimetre`
R16 — Unit references are faithful: commensurability is an equivalence and the number-and-reference form round-trips	trans, `symm`, `measure_numeral`, `refl`, measure_eq_of_measures	specifies	`measure`
		specifies	`Commensurable`
		proves	trans
		proves	`symm`
		proves	`measure_numeral`

Requirement	Status	Role	This work
		proves	`refl`
		proves	measure_eq_of_measures
		exemplifies	`fiveCm`
R17 — Unit conversion between commensurable units is a faithful round-trip (an exact, reciprocal power-of-radix factor — SI decimal or IEC binary)	`convertReal_roundtrip`, `shift_add_symm`, convertExp_roundtrip, `ratio_mul_ratio_symm`, `ofNumber_measure`	specifies	`conversionExponent`
		specifies	SIPrefix
		specifies	BinaryPrefix

Requirement	Status	Role	This work
		specifies	PrefixedUnit
		specifies	`realFactor`
		specifies	RealUnit
		proves	`convertReal_roundtrip`
		proves	`shift_add_symm`
		proves	convertExp_roundtrip
		proves	`ratio_mul_ratio_symm`
		proves	`ofNumber_measure`

Requirement	Status	Role	This work
		exemplifies	`kib_mib_real_roundtrip`
		exemplifies	`cm_km_real_roundtrip`
		exemplifies	`kib_mib_roundtrip`
		exemplifies	`cm_km_roundtrip`
Aggregation			
R9 — Extensive quantities aggregate additively over parts; intensive ones do not	extensive_additive, mixing_subadditive, leafSum_invariant, `extensiveAbout_mixed`, coalesce_count_ne, assemble_ne_measured, intensive_uniform, mean_const, `mk?_eq_none_iff`, not_extensive_of_intensive, `extensiveAbout_mixed_ne`, mean_magnitude, quasiExtensive_leafSum, parallelAxis, `mixing_not_quasiExtensive`, extensive_iff_quasiExtensive_zero	specifies	`countMeasurement`

Requirement	Status		Role	This work
			specifies	Recarving
			specifies	Extensive
			specifies	Intensive
			specifies	WholeProper
			specifies	ExtensiveAbout
			specifies	`Transports`
			specifies	WeightedCarvingQ
			specifies	WeightedCarving
			specifies	`mean`

Requirement	Status		Role	This work
			specifies	`mean`
			specifies	QuasiExtensive
			proves	extensive_additive
			proves	mixing_subadditive
			proves	leafSum_invariant
			proves	`extensiveAbout_mixed`
			proves	coalesce_count_ne

Requirement	Status		Role	This work
			proves	assemble_ne_measured
			proves	intensive_uniform
			proves	mean_const
			proves	`mk?_eq_none_iff`
			proves	not_extensive_of_intensive
			proves	`extensiveAbout_mixed_ne`
			proves	mean_magnitude

Requirement	Status		Role	This work
			proves	quasiExtensive_leafSum
			proves	parallelAxis
			proves	`mixing_not_quasiExtensive`
			proves	extensive_iff_quasiExtensive_
			implements	mk?
			exemplifies	`mass_extensive`
Representation parametricity				
R10 — A quantity value is			specifies	DivRefinement

Requirement	Status	Role	This work
parametric in its numeric representation type	add_refines, mul_refines, div_refines, mean_div_refines, refinementFixes_iff_exactRepresentable		
		specifies	Carrier
		specifies	`LawfulCarrier`
		specifies	MulRefinement
		specifies	CarrierRefinement
		specifies	`RefinementFixes`
		proves	add_refines
		proves	mul_refines
		proves	div_refines

Requirement	Status		Role	This work
			proves	mean_div_refines
			proves	refinementFixes_iff_exactRepr
			exemplifies	`lenF`
R11 — Units are scalar; a vector quantity is a numerical array times one scalar unit	instLawfulCarrierPi		proves	instLawfulCarrierPi
			implements	`instCarrierPi`
			exemplifies	`doubled_east`

Requirement	Status	Role	This work
		exemplifies	`vector_add_comm`
R20 — A vector quantity's numbers are coefficients in a chosen basis, so a reading is indexed by that basis and by the law its coefficients obey when the basis changes; readings in different bases do not combine, the change acts functorially, and a contraction of two readings is basis-independent while each coefficient separately is not	`toFrameVector_comp`, `toFrameVector_id`, `dot_toFrameVector`, `component_not_invariant`	specifies	`InFrame`
		specifies	`Frame`
		specifies	`Variance`

Requirement	Status		Role	This work
			specifies	`IsOrthonormal`
			specifies	`ScalarCarrier`
			proves	`toFrameVector_comp`
			proves	`toFrameVector_id`
			proves	`dot_toFrameVector`
			proves	`component_not_invariant`
			exemplifies	`component_moves`

Requirement	Status		Role	This work
			exemplifies	`contraction_invariant`
Classification				
R12 — A classification is a certificate, not an assertion	`mul_isProduct`		specifies	IsProduct
			specifies	ProductKind
			specifies	QuotientKind
			specifies	ReciprocalKind
			proves	`mul_isProduct`
			exemplifies	`d`
			exemplifies	`length_is_speed_times_time`
R13 — Unit classification			specifies	`combineEnergetic`

Requirement	Status	Role	This work
needs a third category: scale-spanning units	`toKind_not_allowsDifference`, `gainKind_ref_irrelevant`, scaleSpanning_not_determined_by_dimension		
		specifies	`LevelKind`
		specifies	`PowerRole`
		specifies	`toKind`
		specifies	UnitCategory
		specifies	ScaleSpanningUnit
		specifies	`sub`
		specifies	MechanicallyReducible

Requirement	Status		Role	This work
			proves	`toKind_not_allowsDifference`
			proves	`gainKind_ref_irrelevant`
			proves	scaleSpanning_not_determine
			exemplifies	ampere_is_physicalBase
			exemplifies	kelvin_reduction_invisible_to_c
Uncertainty and numerical adequacy				
R14 — Output uncertainty is computed by a provably nested ladder of methods	willinkCumulants_cons, gum_eq_willink_of_normal, `willinkCumulants_kappa2`		specifies	`Cumulants`
			proves	willinkCumulants_cons

Requirement	Status	Role	This work
		proves	gum_eq_willink_of_normal
		proves	`willinkCumulants_kappa2`
		implements	gumStdUnc
		implements	willinkCombine
		exemplifies	`normalTerms`
R15 — A floating-point representation is numerically adequate iff it loses no information at the scale of the input uncertainties	mean_fp32_within_errBound, `regular_meanExpr`, verdict_sound, `toSpec_box_exact`, `relUnc_amplifies`, `round32_eq_self_iff`, flagFree_iff_exactRepresentable	specifies	`meanExpr`
		specifies	`specCarving`

Requirement	Status	Role	This work
		specifies	`AdequacyReport`
		specifies	`ExactRepresentable`
		proves	mean_fp32_within_errBound
		proves	`regular_meanExpr`
		proves	verdict_sound
		proves	`toSpec_box_exact`
		proves	`relUnc_amplifies`
		proves	`round32_eq_self_iff`

Requirement	Status	Role	This work
		proves	flagFree_iff_exactRepresentab
		implements	Adequacy
		exemplifies	`a1_absorb`
		exemplifies	`a3_verdict_sound`
		exemplifies	`a2_relunc`
		exemplifies	`doubling_box_exact`
R18 — The recorded variance certifies a coverage interval: distribution-free (Chebyshev ≥ 1	`join_within_tolerance`, `coverageBound`, coverageBound_stdUnc, uniform_coverage_exact	specifies	stdUnc

Requirement	Status	Role	This work
− 1/k²), exact for bounded families			
		proves	`join_within_tolerance`
		proves	`coverageBound`
		proves	coverageBound_stdUnc
		proves	uniform_coverage_exact
		exemplifies	`central_half_coverage`
Ergonomics and erasure			
R21 — The kind layer erases: the kinded authoring computes exactly the bare-carrier value, and the operator-table	`hmul_eq_mul`, `hdiv_eq_div`	specifies	`KindDiv`

Requirement	Status		Role	This work
form elaborates to the same term as the longhand form — the ceremony is pure				
			specifies	KindMul
			proves	`hmul_eq_mul`
			proves	`hdiv_eq_div`
			exemplifies	`mr11_operators_same_term`
			exemplifies	`mr11_pkc_same_value`
R25 — A kinded definition renders back as typeset mathematics from its own elaborated	`avsForward`		specifies	`MathTerm`

Requirement	Status		Role	This work
source, with the kind bookkeeping suppressed				
			implements	`pkcMathUses`
			implements	`prettyEquation`
			exemplifies	`avsForward`
Provenance and audit				
R23 — A value's provenance is recordable and auditable: a kind-typed value-flow graph with a declared boundary contract, checked against what the scope actually computes	`wcmBoundary`		specifies	Contract
			specifies	`Provenance`

Requirement	Status	Role	This work
		implements	wellFormed
		implements	`discharges`
		exemplifies	`wcmBoundary`
R24 — An audit is a claim about its scope, never about the library: the unkinded surface of a scope is enumerable, and a blessed scope only grows	`ingestBoundary`, `interiorScope`	specifies	`BoundaryTier`
		implements	`ledgerOf`
		implements	`boundarySites`
		exemplifies	`ingestBoundary`

Requirement	Status	Role	This work
		exemplifies	`interiorScope`
R26 — A module's behavior is a checked measurement model: a declared boundary carries relation edges whose witnesses are kernel-checked theorems, every relation kind has a conclusion-shape check, a bound names a kinded tolerance, and every rung beyond the witness's own is answered for by a named repair or restatement	`retrievalInvertsForward`	specifies	RelationLicense

Requirement	Status		Role	This work
			specifies	Relation
			implements	`checkRelation`
			implements	`checkDeciders`
			exemplifies	`retrievalInvertsForward`
R27 — A module's distribution is	distribution_license		specifies	AggregationClass

Requirement	Status	Role	This work
licensed by declared extensivity: output ports carry aggregation classes, and recarving or sharding a batch axis is licensed by theorem from those declarations, with the quasi-extensive tolerance tied to the rounding budget where the carrier demands it			
		proves	distribution_license

Requirement	Status		Role	This work
			implements	`checkAggregations`
R28 — A module's requirements are stated with their evidence: every produced port of a model boundary is governed by a requirement or exempted with a reason; a provable requirement's witness is a sorry-free theorem or theorem edge naming no referent, with spot checks evaluated where the model runs; an empirical requirement names marked referents and a gate mentioning one; and an	`he5`, `hp1`, `hp6`		specifies	RequirementScope

Requirement	Status		Role	This work
attestation never stands beside a discharge				
			specifies	Requirement
			specifies	RequirementKind
			specifies	BoundaryRole
			implements	`checkRequirement`

Requirement	Status		Role	This work
			implements	`coverageRows`
			exemplifies	`he5`
			exemplifies	`hp1`
			exemplifies	`hp6`

Requirement	Status	Role	This work

2.17. What *proved* is checked to mean

A *proved* status is stronger than "a theorem of that name compiles", and two guarantees that `lake build` does not enforce on its own are carried by a dedicated validation suite (`PropertyKindCalculus.Tests`, one probe per requirement group), built by CI alongside the dimension and uncertainty layers so that *all twenty* verifiable requirements — not only those whose theorems live in the Mathlib-free core — are under regression on every change.

First, each verifiable requirement's theorem has its *axiom profile* pinned with `#guard_msgs` (https://lean-lang.org/doc/reference/latest/Interacting-with-Lean/#hash-guard_msgs) over `#print axioms` (https://lean-lang.org/doc/reference/latest/Interacting-with-Lean/#hash-print), so *proved* means **sorry-free as certified by the axiom set**, not merely that no `sorry` keyword appears: a proof relocated behind a `sorry` raises no warning at its call sites, and only the axiom profile exposes it. (The exportable core is largely axiom-free — most of its capstones depend on nothing at all, the rest on `propext` and `Quot.sound` from `omega` and structure equality, with `Classical.choice` reached by a single declaration; the Mathlib-backed layers use the three standard classical axioms. What the suite pins is each theorem's profile exactly, whatever it is, so a change to any of them is a build failure rather than a footnote.) Second, each theorem is applied to a *concrete witness* whose premises are discharged — and, at the degenerate boundary where a premise *fails*, shown genuinely excluded — so that no requirement is satisfied **vacuously**: an empty quantifier or an unsatisfiable hypothesis would leave a theorem true but empty, and the witness is what rules that out.

What a *proved* status rests on is the stack Figure 2 draws: a domain model's trust traces down through PhysLib and cslib, the two peers on Mathlib, and through TorchLean with FloatLib under it, to Mathlib and then to the Lean kernel with its three classical axioms — the one band that is believed rather than derived, checked, or enumerated. The axiom profile pinned above is that band's cost, stated per theorem.

3. Why a calculus, not a taxonomy

PropertyKindCalculus continues two lines of machine-checkable metrology modeling — the descriptive one, which runs from QUDV through an OML vocabulary to SysML v2, and the mechanized one, which indexes quantities by dimension inside a proof assistant — and inherits its problem statement from exactly where each of them stops.

3.1. From QUDV to an OML metrology vocabulary

OMG SysML 1.2 introduced QUDV (https://www.omgwiki.org/OMGSysML/doku.php?id=sysml-qudv:quantities_units_dimensions_values_qudv) (Quantities, Units, Dimensions, Values) as a library for declaring quantity kinds, units, and their dimensional factoring; QUDV's foundations were then revised substantially across SysML 1.3 and 1.4.

In parallel, a formalization in OML — leveraging the semantics of OWL2-DL for classification-based reasoning — reworked those foundations, the quantity-kind and unit *stereotypes* carried by SysML, to align them with Dybkær's *An Ontology on Property*. That effort became the opencaesar metrology vocabulary (https://github.com/opencaesar/metrology-vocabularies), whose current version combines the updates and motivations of the VIM4 2nd Committee Draft (2CD, 2023-07-31) with Dybkær's ontological grounding. OWL2 (SROIQ) carries the metrology *taxonomy* well: specialization, the general-vs-individual quantity distinction (encoded as concept vs. instance), dimensional factoring, and OWL2-DL/SPARQL consistency checks over units.

Where a description logic stops is visible in that vocabulary's own *dimensional analysis*: computing a derived quantity's dimension from its factors is an arithmetic induction that has to be pushed out of OWL2 into SPARQL 1.1 Update, and even there it is brittle — updates report success while inserting no triples, because "update succeeded" is not a statement about *what was established*. Dimension arithmetic sits right at that boundary; the kind interaction algebra and the algebraic *laws* are past it.

3.2. What a description logic cannot express

A description logic captures the *taxonomy* but is structurally unable to capture the *calculus*. The recurring gap is arithmetic, operations, definedness-conditions, laws, and proofs:

- **Dimension arithmetic** (`\dim(a\cdot b) = \dim a + \dim b`): OWL2 has no datatype arithmetic; in Lean it is an ordinary function with a homomorphism theorem.
- **The interaction algebra** (torque `\times` angle = energy): role chains are binary, regular, and carry no arithmetic side-conditions; in Lean it is a ternary relation with a dimensional-coherence proof obligation.

- **Scale-type operator gating** (which of `=,<,+,\times` are **defined**): DL has no operations; in Lean the scale indexes which typeclasses apply, and monotonicity of availability is a proved meta-theorem.
- **Extensivity** (`\mathrm{value}(\text{whole}) = \sum \mathrm{value}(\text{parts})`): no quantified arithmetic in DL; in Lean it is a `\forall`-theorem.
- **Units** (a unit is a chosen value of a kind; conversion is a ratio): no arithmetic in DL; in Lean it is a definition with a round-trip theorem.
- **Meta-theorems** (specialization is a preorder; `\dim` is a homomorphism): a DL reasoner checks **consistency**, not **laws** — proving the laws is the point.

3.3. SysML v2: a quantity model organized by value representation

OMG's current generation ships a standard-library treatment of quantities and units, examined here at tag SysML v2, 2026-04 (https://github.com/Systems-Modeling/SysML-v2-Release/tree/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units) (its `Quantities`, `MeasurementReferences`, and ISQ packages). Read against Dybkær, it is built around a different organizing axis than the one taken here as primary, and the contrast is the cleanest way to say what a **kind-primary** model buys.

3.3.1. The evidence: representation is the root, kind is a leaf

The quantity-value hierarchy is rooted in **value representation**, not in quantity kind. The most general value is `TensorQuantityValue`, a tensor of any order, and the specialization chain runs **toward** the scalar, each more specific type being a lower-order tensor — `VectorQuantityValue :> TensorQuantityValue`, then `ScalarQuantityValue :> VectorQuantityValue` (SysML v2, 2026-04 — Quantities.sysml, lines 17–46 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L17-L46)): `\text{Scalar} \sqsubseteq \text{Vector} \sqsubseteq \text{Tensor}`. So a scalar **is a** tensor of order 0 and a vector **is a** tensor of order 1. What a value's type primarily records is the apparatus of that representation:

- its tensor order,
- the split of that order into **contravariant** and **covariant** indices — the attributes `contravariantOrder` and `covariantOrder` on `TensorQuantityValue`, constrained to sum to the order (SysML v2, 2026-04 — Quantities.sysml, lines 33–36 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L33-L36))
- whether a vector is bound or free, and
- a matching measurement reference (tensor / vector / scalar) carrying the coordinate frame and its transformations.

Each of these is a statement about **how a quantity is measured, coordinatized, and valued**, not about **what kind of property** it is. The contravariant/covariant split is the sharpest case of this representation-centric view — the one record-field that is **unambiguously** representational, on three escalating counts.

1. **Pure coordinate bookkeeping.** It records only how a value's components transform under a change of frame — nothing about what the quantity **is**. The other three fields each record a facet of the quantity itself:
   - tensor order tracks its mathematical character,
   - bound-versus-free is a genuine physical distinction (and one the library actually uses), and
   - the measurement reference names a dimension and a frame.

   The variance split alone carries no information about the quantity kind.
2. **Not quantity-invariant.** Using the metric to raise a covariant (lower) index or lower a contravariant (upper) one moves a unit between the two counts — say from `contravariantOrder` 2, `covariantOrder` 0 to a mixed 1 and 1 — while the order, and the physical quantity, stay fixed. That the quantity is unchanged is a fact of **tensor algebra** — the metric's raising/lowering isomorphism — not a SysML v2 statement: the library defines no raise- or lower-index operation, so it would record the two splits as distinct `TensorQuantityValue`s with nothing to mark them as one quantity. A feature that a pure change of representation can move while the quantity is held fixed is thus a property of the representation, not of the quantity's kind. Hold the quantity fixed and the other three fields are pinned with it — order, bound-status, and reference do not change — so the variance split is the one field free to vary while the modeled quantity stays put (worked through just below).
3. **Empirically inert.** The split is declared on the abstract root and named by its constraint, yet assigned by **no** quantity anywhere in the library: even the genuine order-2 kinds — `Cartesian3dStressTensor`, with strain and moment of inertia alongside it — inherit the two slots and leave them unset, fixing only `isBound`, `num`, and `mRef` (SysML v2, 2026-04 — ISQMechanics.sysml, lines 730–746 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQMechanics.sysml#L730-L746)). The apparatus is carried at the root whether or not any kind exercises it. By contrast `isBound` **is** used (88 free against 7 bound), so the point is precise rather than blanket: bound-versus-free is live machinery, while the variance split is declared-but-unexercised overhead inherited from the tensor root.

To see why the second count bites — and what the model **loses** by it — follow the library's own machinery. `Cartesian3dStressTensor` stores its value as `num: Real[9]`, nine bare reals, with the variance split left blank (same listing). In an orthonormal Cartesian frame that omission is harmless: under the Euclidean metric the contravariant components `\sigma^{ij}`, the mixed `\sigma^i{}_j`, and the covariant `\sigma_{ij}` are numerically equal, so it does not matter which the nine numbers are. But `TensorCalculations::transform` is defined to carry a tensor value from a source frame to a target frame through a `CoordinateTransformation` (SysML v2, 2026-04 — TensorCalculations.sysml, lines 45–49 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/

TensorCalculations.sysml#L45-L49), MeasurementReferences.sysml, lines 126–135 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L126-L135)), and the transformation rule is keyed on the split: a contravariant index transforms by the Jacobian of the coordinate change, a covariant index by its inverse — mutually inverse rules, so the two generally yield different numbers. Move to a frame where they differ — any non-orthonormal frame, which is to say the everyday cylindrical, spherical, or material coordinates of continuum mechanics — and the nine numbers transform by **different** rules depending on a datum the model declared but never filled in.

What is missing, then, is the ability to transform an order-2 (or higher-order) quantity between frames **at all**: the `transform` contract cannot be met without the split, so for exactly the quantities where it would matter the operation is underdetermined. And the gap is **silent**. The concrete types are pinned to a Cartesian frame, where the omission never shows; the model type-checks and looks complete, and fails only where one would actually rely on it — in the curvilinear frames it was never exercised in. That is precisely a representational hole that validation cannot see: correct on every case anyone tested, wrong on the first case anyone needed.

The lone constraint the library attaches to the split sharpens the reading. `orderSum` requires `contravariantOrder + covariantOrder == order` (SysML v2, 2026-04 — Quantities.sysml, line 36 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L36)). Metrologically it is a well-formedness invariant, not a statement about the quantity: it says only that the chosen split must be a valid partition of the quantity's actual order. Its content is that **order is the frame-invariant datum and the split is a convention that must respect it** — raising or lowering an index slides one unit between `covariantOrder` and `contravariantOrder` but holds their sum, the order, fixed. So `orderSum` is exactly what makes **the same quantity, a different split** formally coherent: the order — which carries the quantity's mathematical character — is conserved while the representation changes underneath it. It constrains nothing metrological: not dimension, unit, scale, or kind, only that the coordinate bookkeeping adds up. The one rule SysML v2 writes about variance is thus itself a pure-representation consistency check — the section's thesis in miniature.

The kind-of-quantity sits one level **down**, as a leaf, and the source is decisive on this point: there is no first-class "kind" entity. Length, mass, temperature, and the other base quantities are each declared as a **subtype of the scalar value type** — `attribute def LengthValue :> ScalarQuantityValue` — with the dimension carried only by the corresponding unit's `quantityDimension`, e.g. `LengthUnit` (SysML v2, 2026-04 — ISQBase.sysml, lines 16–38 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQBase.sysml#L16-L38)), and that `quantityDimension` attribute itself lives on `ScalarMeasurementReference` (SysML v2, 2026-04 — MeasurementReferences.sysml, lines 82–99 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L82-L99)). In other words, "length-the-kind" exists only as (i) a nominal subtype hanging beneath `ScalarQuantityValue` and (ii) a power-product of base quantities on its unit — never as a property whose nature is named in its own right. To reach a kind one must already have committed to a representation branch: the kind is **nested under** the representation. The proportions confirm the weight — across the ISQ library 319 kinds are declared on the scalar branch, against 2 vector, 4 tensor, and 51 fixed-three-vector kinds.

That places the concern precisely. A valued quantity has two independent classification axes:

- **(A) kind-of-property** — its nature, fixed by Dybkær's examination principle, common to all mutually comparable quantities;
- **(B) value representation** — scalar / vector / tensor order, coordinate frame, bound-versus-free — what SysML v2 packages as the measurement reference.

Dybkær's ontology lives entirely on axis (A) and is deliberately **agnostic** to (B): whether a position is recorded as one number, three Cartesian components, or a tensor in a chosen frame does not change **what kind of property** it is. SysML v2 makes axis (B) the root and nests (A) beneath it. The model even flags the departure from its own source: it quotes the VIM3 "quantity" Note 5 — "A quantity as defined here is a scalar. However, a vector or a tensor, the components of which are quantities, is also considered to be a quantity" — then notes that "the rest of [VIM] does not explicitly define how tensor and vector quantities can be or should be supported" (SysML v2, 2026-04 — MeasurementReferences.sysml, lines 29–31 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L29-L31)). The tensor-rooted hierarchy is thus SysML's own interpretation, and it **inverts** the VIM's priority — both the quoted VIM3 and the VIM4 2CD (quantity, 1.1 Note 4: "As defined here, quantities are scalar … vectors, and tensors are considered quantities in a broader sense if their components are (scalar) quantities") make the quantity fundamentally scalar, with vectors and tensors as composites **of** scalar quantities; SysML v2 makes the tensor primary and the scalar the degenerate order-0 case.

3.3.2. How PropertyKindCalculus takes the kind as root instead

PropertyKindCalculus inverts the nesting: axis (A) is the root. A kind-of-property is a first-class value (`KindOfProperty`), and a quantity is indexed **by its kind** — `Quantity (k : KindOfProperty)` — so the kind is fixed before, and independently of, any choice of how the value is recorded. Dimension is not the organizing principle but a forgetful functor `\dim : \mathrm{Kind} \to \mathrm{Dimension}` that **certifies** coherence without **deciding** what is allowed (developed in the next section). Two consequences fall out directly: two kinds that share a dimension remain distinct types — so the dimension-1 disambiguation is expressible — and an operation across kinds is a type error rather than a silent success. Value representation — the axis SysML v2 puts at the root — sits here as an **orthogonal** index over the kind, never a layer the kind hangs beneath: R20's `InFrame f var k R` adds the frame and the variance above a `Quantity k R` whose kind was already fixed without reference to either.

3.3.3. The two designs side by side

Aspect	SysML v2 (2026-04)	PropertyKindCalculus
Root organizing axis	value representation (tensor order, frame)	kind-of-property (Dybkær)
A *kind of quantity* is…	a leaf subtype of `ScalarQuantityValue`	a first-class type index, `Quantity (k : KindOfProperty)`
Role of dimension	the discriminator for commensurability, on the reference	a forgetful functor `dim`; certifies, never decides
Two kinds, one dimension (vwc vs gwc)	indistinguishable	distinct by construction
Adding a length to a mass	well-typed (`Scalar × Scalar → Scalar`)	a type error
Scalar / vector / tensor, frames, transforms	first-class and rich	an orthogonal index (R20): Cartesian frames, three variances, invariance proved; curvilinear and the covariant/contravariant split still owed
Algebraic laws	a modelling library; none machine-checked	proved theorems (preorder, homomorphism, monotonicity)

The table is honest in both directions: SysML v2's representation-first design buys real, first-class machinery for coordinate frames, transformations, and stress and strain tensors that systems engineering genuinely needs. R20 supplies the Cartesian part of it — frames, the three variances, the scalar product and its invariance — while curvilinear coordinates and the covariant/contravariant split remain machinery PropertyKindCalculus does not yet have. The claim is not that one model is right; it is that the axes are **orthogonal and should not be nested**, and R20 is that claim carried out rather than asserted: the frame index sits **over** a kind that was fixed without it.

3.3.4. Limitations of the SysML v2 approach

Rooting the hierarchy in representation has three consequences that bear directly on PropertyKindCalculus's problem statement:

1. **A kind cannot be named without first choosing a representation, so one kind fragments.** Position is a single kind-of-property, yet appears as `CartesianPosition3dVector`, `CylindricalPosition3dVector`, `SphericalPosition3dVector`, and `PlanetaryPosition3dVector` — individuated by **coordinate frame**, not by the nature of the property. The cylindrical and spherical forms even carry mixed-dimension components (a length and two angles), so the single documented "quantity dimension `\mathrm{L^1}`" does not survive its own representations (SysML

v2, 2026-04 — ISQSpaceTime.sysml, lines 304–353 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQSpaceTime.sysml#L304-L353)).

2. **The within-dimension conflation is left untouched.** On the scalar branch the dimension is the only discriminator, so volumetric and gravimetric water content, relative permittivity, reflectivity, and emissivity remain indistinguishable — exactly the confusion that motivates PropertyKindCalculus. Putting representation at the root does nothing about it; the two are orthogonal.
3. **The calculus is kind- and dimension-blind.** Scalar addition is declared `ScalarQuantityValue × ScalarQuantityValue → ScalarQuantityValue`, so adding a length to a mass is well-typed; nothing gates an operation on agreeing kind or dimension (SysML v2, 2026-04 — QuantityCalculations.sysml, line 29 (https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/QuantityCalculations.sysml#L29)).

3.4. Mechanization: what a dimension index still cannot separate

The other line is mechanization, and its strongest prior result is a formalization of the ISQ and the SI in a proof assistant (Foster and Wolff, 2023)[8]. Simon Foster and Burkhart Wolff (2023). "Automated Reasoning for Physical Quantities, Units, and Measurements in Isabelle/HOL" (https://isa-afp.org/browser_info/current/AFP/Physical_Quantities/). *27th International Conference on Engineering of Complex Computer Systems (ICECCS)*. . : a quantity carries its dimension as a type index, so dimensionally incoherent arithmetic is rejected by the type checker rather than deferred to a run-time check.

That is a real calculus, and it is one this work carries too — as `\dim`, the forgetful functor of point 5 of the next subsection, whose job is to certify. What it is not is a kind layer. Indexing on the dimension collapses the fibers of `\dim`: volumetric and gravimetric water content, relative permittivity and reflectivity, torque and energy each share an index and are therefore one type, so the distinction a kind-of-property exists to draw cannot be stated there, let alone enforced. Neither do the axes that hang off a kind rather than off a dimension follow from it — the scale that decides which operators are defined, the interaction algebra, the dedicated kind, the aggregation laws. Dimensional coherence remains necessary; it was never sufficient.

3.5. How the calculus maps into Lean

Two things in particular resist a DL encoding: the *interaction algebra* (which kind `\times` which kind `=` which kind, a *partial* relation with arithmetic side-conditions, not a role chain), and the difference between *instantiation* and *subtyping* (that a radius *is a* length, versus that this pencil's length *is an instance of* the length kind — conflated by punning in OWL). In Lean both map cleanly, and the mapping *is* the design:

1. **Kinds are type-level parameters.** `Quantity (k : KindOfProperty) (R : Type)` — a kind indexes a *type*, not a value of one universal `Quantity` class, and a unit `RealUnit k` is a distinguished

8.

quantity of that same kind (the Units chapter), so "a metre" and "a unit of gravimetric water content" inhabit different types. This is the move OWL cannot make.

2. **Specialization (⊑) induces coercions between types.** It lives on that parameter: `Quantity Radius R → Quantity Length R` is sound — a radius *is a* length — and the reverse is not. Up-casting to a parent kind is an explicit, visible loss of information.
3. **Instantiation is term-of-type, not subtyping.** The length of a pencil is a *term* `pencil : Quantity Length ℝ := .attest "read off the ruler" 5.2` — a value of that type, related to its kind by typing/instantiation, never by subtyping (and, being a mint, one of the boundary sites the audit enumerates). The general/individual distinction *is* the type/term distinction.
4. **The kind algebra is a partial typed algebra, not a group.** PhysLib's `Dimension` is a *total* `CommGroup` — multiply, divide, invert anything. Kinds are not: `Torque × Angle = Work` is meaningful; `Torque + Energy` must be *blocked* even though the dimensions match; `FuelConsumption × Rainfall` is an error (Flater's own example). The meaningful products are encoded as a relation surfaced through typeclass resolution — Flater's "map kinds onto classes + operator overloading," lifted into Lean's elaborator.
5. **Dimension is a soundness functor, not the primary type.** In QUDV and most quantity libraries dimension is the *primary* classifier: two quantities interoperate exactly when their dimensions agree — which is precisely the move that collapses every dimension-one quantity into one indistinguishable type. PropertyKindCalculus inverts the dependency: the *kind* is the primary type-level index, and dimension is recovered as a forgetful functor `\dim : \mathrm{Kind} \to \mathrm{Dimension}`. Its job is *certification*, not decision — the coherence theorem `KMul k₁ k₂ k₃ → dim k₃ = dim k₁ · dim k₂` proves every product the kind algebra admits is dimensionally consistent — but `dim` never decides whether an operation is *allowed*; the kind relation does. Agreeing dimensions are necessary, not sufficient.
6. **Specialization (⊑) is a preorder — a lattice, not a tree.** Flater stresses this (citing Formal Concept Analysis): a kind can specialize several parents. Unit-1 subtyping (Flater §6) is then just the *dimension-1 slice* of that kind lattice — and nothing forces it to be uniform: units of plane angle, for one, could carry their own subtyping reflecting the different measurement principles by which they are realized.
7. **The numeric carrier is a type parameter, bounded by a typeclass (R10).** A scalar quantity is `Quantity (k : KindOfProperty) (R : Type)` with the arithmetic on `R` supplied by a `Carrier` typeclass — the kind fixes **what is measured**, `R` fixes **in what numbers**. Everything above `R` (kind discrimination, scale gating, the dimension functor, the interaction algebra, extensivity) is written once, against the typeclass, and is reused verbatim at every `R` (the additivity laws, for one, are proved a single time over any lawful carrier and hold at `ℝ` and `Int` with no per-carrier proof). This is the architecture TorchLean already runs: models are written against a `Context α` typeclass (arithmetic + transcendentals
    - comparison) and instantiated at `α := ℝ` for the **spec**, `α := FP32` for the finite **rounding** model, and `α := IEEE32Exec` for the **executable** IEEE-754 kernel — the **same** source

elaborated at three carriers. The kind layer here is the missing index **above** that carrier: TorchLean makes a tensor polymorphic in its scalar type; PropertyKindCalculus makes the scalar polymorphic in its **kind**, and the two indices compose. Crucially the carriers are bridged by **refinement theorems** (`toReal (op_exec x) = round (op_real (toReal x))`), so a law proved at `\mathbb{R}` is not stranded in the proof world — it descends to the float that runs. A description logic has no type parameters and no typeclasses, so this whole axis is unavailable to it; here it is the ordinary discipline of parametric polymorphism.

The trigger was concrete. PhysLib's `Dimension` makes all dimension-one quantities equal, so it cannot distinguish volumetric (`\mathrm{L^3/L^3}`) from gravimetric (`\mathrm{M/M}`) soil moisture, nor either from permittivity, reflectivity, or emissivity. Every dangerous soil-moisture confusion is dimension one. Indexing quantities and units by **kind** is what restores the distinction — this is the dimension-1 disambiguation capstone in the Units chapter.

4. How to read the status

Nodes that link a real declaration with `(lean := "PropertyKindCalculus.…")` report their **proved** status straight from the checked, sorry-free core. The nodes that link none are the authoring steps of the *write once, correctly* recipe: they say how a model is composed, and carry nothing to prove. The headline deliverables are tagged `capstone`, and the status summary at the end reports the current counts. The blueprint dependency graph drawn beside that summary is the graph of these nodes and their `uses` edges — a picture of this document, and not one of the graphs the calculus itself defines; those are named and kept apart in the capstone chapter, and every term of the calculus is fixed by the dictionary.

5. Onboarding: Human Context Protocols

This part is for a *PhysLib contributor*: a physicist who writes Lean in PhysLib and already knows Lean, PhysLib, `Dimension`, and `WithDim`. The method is top-down refinement: begin with the broad system context, then state the attributes of interest, interleaving context and attribute refinement as new subsystems or phenomena enter the model.

Lowe's four-category square separates substantial things from ways things are, and universals from particulars:

Axis	Substantial things	Non-substantial ways things are
Universal	Sort of system — `SortOfSystem`	Kind of property — `KindOfProperty`
Particular	System or object — index `o`	Individual quantity — `IndividualQuantity o k R`

The left column is system context; dimension-only metrology attends to the right.

The ladder starts with one black-box oscillator and adds a second oscillator as another particular of the same sort. L2 opens each box into body and spring(s), then treats the pair as a whole. Later steps open the body to its mass distribution, move out of the line, optionally add damping, and finally state what must hold:

Ladder step	Refinement	Context change	New attributes	What goes wrong	PhysLib stage/source
L0 — The black box	One homogeneous, rigid, point-like body on one ideal spring to a wall, along a line	One system; body and spring inside, wall outside; the wall is fixed and infinitely massive	Mass, stiffness, displacement, ω vs f, period	One oscillator. Bare reals accept the inverted period formula `√(m/k)`; `WithDim` refuses that but accepts angular frequency ω plus frequency f, which share dimension T^{-1}. Only distinct kinds refuse both.	Stages 0–1; upstream oscillator Bas WithDim; ForPhysLib Kinds and Me Exhibit C
L1 — Two bodies	One body inside each black-box oscillator	Two particulars of sort harmonic oscillator; each black box contains one body and its wall spring; the oscillators remain uncoupled	Same kinds, now indexed by the oscillator whole; its mass and stiffness share an index	Two separate oscillators, A and B. Cross-system contamination: with kinds but no bearer, B's mass can be used in A's frequency, because the kind is right. False identity: giving both oscillators one name makes them one system. Indexing each quantity by its oscillator refuses the	Stage 2; CaseStudies/HarmonicOsc Attempt 4; HCP L1

Ladder step	Refinement	Context change	New attributes	What goes wrong	PhysLib stage/source
				first; an injective designation the second.	
L2 — The pair is a system too	The pair becomes a system in its own right	Opens each oscillator into body and spring(s); pair's own sort, whole and parts; coupling, parallel and series springs	Normal-mode frequency ω_+, total mass, effective stiffness	One coupled pair, opened into bodies A and B and springs kA, kC, kB. Aggregating what does not aggregate: the pair's mass is m*A* + *m*B, but its normal-mode frequency ω_+ is not ωA + ωB. Wrong composition law: the licence to add masses belongs to the sort 'coupled pair', not to the kind 'mass'; stiffnesses add in parallel but not in series.	Stage 3; HCP `pairMass`/`parallelSt` Composition finding 4
L3 — The body opens up	Body A's centre of mass is off-axis, producing torque and wobble	Body A of pair P becomes a nominally indexed rigid composite	Per-half mass and position, centre of mass, inertia tensor, frame-specific	Body A opened into two halves. Wrong carving: counting a	Stages 2–3 on RigidBody; Exhibits RigidBody and Composition; upstre `RB-sphereI` informal lemma proved `SolidSphereInertia.lean`

Ladder step	Refinement	Context change	New attributes	What goes wrong	PhysLib stage/source
		with left/right halves; a body-fixed frame enters beside the lab frame	angular velocity	half twice or leaving one out gives another mass. Weighted mean vs sum: the centre of mass is the mass-weighted mean of the halves' positions, and ω is the same for every half. Degenerate whole: a massless body's centre of mass evaluates to the origin, because Lean defines 1/0 = 0. Frames: the lab-frame ω put into the body-frame energy type-checks in PhysLib.	
L4 — Out of the line	Motion in three dimensions, seen by a moving observer	Motion in 3-D, seen from the lab and a moving cart	Position vector, displacement, distance, velocity per frame	Pair P in 3-D, seen from the lab and a moving cart. Bearer erased: `distanceQ` cannot tell P's	Stages 0–4 on Space; Exhibit ReferenceFrame

Ladder step	Refinement	Context change	New attributes	What goes wrong	PhysLib stage/source
				separation from Q's. Observer left implicit: one bare vector type accepts all three frame-conversion laws (displacement, velocity, relative position), though only one is right for each kind of vector.	
L5 — The oscillation dies out	Add a damper (optional)	Add a damper to the nominally named oscillator	Damper γ, whole-system decay rate δ, and damping regime	Oscillator A with a damper. PhysLib's γ (kg/s), ISO's damping coefficient $\delta = \gamma/(2m)$ (s^{-1}), and ω_0 (rad/s) share symbols or dimensions; bare reals accept $\gamma < \omega_0$ and dimensions accept $\delta + \omega_0$.	Stage 1; upstream `DampedHarmonicOscillator`; ISO 80 items 3-18 and 3-24
L6 — What must hold	What must hold	Requirement scope over refined context	Requirement, referent, gate	Requirements over pair P and the sort 'coupled pair'. Unstatable scope: without	Stage 4; `Tutorial.HarmonicOscillatorRequi` Audits, `checked_by`

Ladder step	Refinement	Context change	New attributes	What goes wrong	PhysLib stage/source
				named sorts and particulars, 'every pair' and 'pair P' have nothing to quantify over; a scope naming a bare number is rejected.	

Every lesson line is quoted live from the `HCP` library through SubVerso anchors; the blueprint build refuses drift. L1–L6 are included here, with L5 optional; the `Tutorial` library remains the model-writer's separate track.

5.1. L0 — The black box

5.1.1. Synopsis

5.1.1.1. Purpose

Show a PhysLib contributor, on one oscillator (a rigid point body of mass m on an ideal spring of stiffness k fixed to a wall, moving in one dimension), what each level of typing checks. With bare `ℝ`, Lean accepts every expression built from m and k, so `√(m/k)` type-checks where `√(k/m)` was meant. With PhysLib's `WithDim`, each expression has a dimension in its type, so an expression of the wrong dimension fails to type-check, but two quantities of equal dimension are interchangeable. With PKC kinds, each quantity also has a kind in its type, so quantities of equal dimension but different kind, such as angular frequency (rad/s) and frequency (Hz), cannot be added. The rest of the ladder builds on this distinction.

5.1.1.2. PKC vocabulary in scope

New at this step:

- `Quantity` — a magnitude of one fixed kind of property, with no bearer yet.
- `angularFrequencyDK` — a dimensioned angular-frequency kind (`.kind`, `.dim`).
- `angularFrequency`, `frequency` — two ISO 80000-3 kinds that share dimension T^{-1}.
- `massQ`, `springQ` — read PhysLib's oscillator mass and spring constant as kinded quantities.

5.1.1.3. Verifiable properties

- The potential energy ½ k x² of PhysLib's `WithDim` oscillator has dimension M L² T⁻² (type-checked) `HCP.L0_BlackBox#potential_energy_dimension`.
- Under `WithDim`, `m/k` has dimension T² and `k/m` has dimension T⁻² (type-checked) `HCP.L0_BlackBox#quotient_dimensions`.
- The kinds angular frequency and frequency are unequal, and both have dimension T⁻¹ (theorem) `HCP.L0_BlackBox#distinct_kinds_same_dimension`.
- `ω + f`, for ω a `Quantity` of kind angular frequency and f a `Quantity` of kind frequency, fails to type-check: there is no `HAdd` instance for the two kinds (refused) `HCP.L0_BlackBox#refused_kind_addition`.
- `massQ S / springQ S`, the radicand of the inverted formula `√(m/k)`, fails to type-check: there is no `HDiv` instance dividing a mass by a spring constant (refused) `HCP.L0_BlackBox#refused_reciprocal_radicand`.

5.1.1.4. Errors prevented

- Inverted formula: `√(m/k)` written where `√(k/m)` was meant. Bare reals accept it; with kinds, the quotient m/k fails to type-check (`HCP.L0_BlackBox#bare_reals`, `HCP.L0_BlackBox#refused_reciprocal_radicand`).
- Same dimension, different kind: an angular frequency ω (rad/s) added to a frequency f (Hz). `WithDim` accepts the sum because both have dimension T⁻¹; with kinds, it fails to type-check (`HCP.L0_BlackBox#frequency_alias`, `HCP.L0_BlackBox#distinct_kinds_same_dimension`, `HCP.L0_BlackBox#refused_kind_addition`).

5.1.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean and PhysLib and trust `Dimension` and `WithDim`. This lesson begins in that vocabulary.

5.1.3. Pull

The oscillator is one body on an ideal, massless spring fixed to an infinitely massive wall, moving along a line. The body is homogeneous, rigid, and point-like; those assumptions are revisited at L3. Its bare-real model has no type-level record of what mass and stiffness mean and accepts the reciprocal-frequency expression `√(m/k)` where `√(k/m)` was intended. A dimension tag can catch dimensional mistakes but cannot distinguish kinds that share a dimension. The system context comes before the attributes it needs.

5.1.4. Risks at this step

- **Bare-real and dimension-only mistakes.** Bare reals accept `√(m/k)` where `√(k/m)` is the angular-frequency formula, and dimension-only types accept angular frequency plus cyclic frequency because both have dimension T^{-1}. The kinded checks pin these risks at `HCP.L0_BlackBox#bare_reals`, `HCP.L0_BlackBox#frequency_alias`, `HCP.L0_BlackBox#refused_kind_addition`, and `HCP.L0_BlackBox#refused_reciprocal_radicand`.

5.1.5. Small rigorous improvement

At this level, treat the oscillator as one black box with one body, one ideal linear spring, a fixed wall, and a line of motion. The body is assumed homogeneous, rigid, and point-like; those assumptions are revisited at L3.

Question	Context change	Attribute change	Risk shown	PhysLib stage
Which quotient is the oscillator's angular frequency?	One oscillator, one body and spring.	Mass, stiffness, angular frequency.	Bare reals accept both `√(k/m)` and the mistaken `√(m/k)`.	Stages 0–1; HarmonicOscillator Basic and WithDim.
What does a dimension tag check?	Keep the same black box.	Potential energy carries the product dimension.	`WithDim` rejects assigning `m/k` to `k/m`'s dimension, but does not identify kinds.	PhysLib `HarmonicOscillatorWithDim`.
What does it not see?	Same one system.	Frequency and angular frequency share `T⁻¹`.	PhysLib `Frequency` has no kind distinction.	PhysLib `Optics.Frequency`; ISO 80000-3.
Which properties are these?	Record the measurement convention.	Frequency and angular frequency become separate kinds.	Same dimension does not imply same kind.	ForPhysLib Kinds and Metrology.

At L0 the bearer stays informal: the System-context card section names one body, while the code uses values such as `Quantity k ℝ`. With only one of each part, no two bearers can yet be confused. L1 is where the bearer enters the type.

5.1.6. Learning path

#	Do	Checked by `HCP.L0_BlackBox` anchor	Concept disclosed
1	In the bare-real oscillator, write the reciprocal quotient under the square root.	`HCP.L0_BlackBox#bare_reals`	Bare reals do not distinguish the two formulas.
2	Return the tagged potential energy at its declared dimension; inspect both quotient types and try the cross-assignment.	`HCP.L0_BlackBox#potential_energy_dimension`; `HCP.L0_BlackBox#quotient_dimensions`	`WithDim` accepts each quotient at its own dimension and rejects assigning `m/k` to `k/m`'s type.
3	Pass an angular-frequency value typed as `WithDim T`$\boldsymbol{d}^{-1}$ `ℝ≥0` to a parameter typed as `Frequency`.	`HCP.L0_BlackBox#frequency_alias`	The alias accepts it because the dimensions and carrier types coincide; there is no kind distinction.
4	Decide kind inequality and compare the paired dimensions.	`HCP.L0_BlackBox#distinct_kinds_same_dimension`	One dimension, two kinds.
5	Try to add kinded angular frequency and frequency.	`HCP.L0_BlackBox#refused_kind_addition`	The kinded layer refuses cross-kind addition.
6	Put the reciprocal mass/stiffness quotient under a square root in the kinded model.	`HCP.L0_BlackBox#refused_reciprocal_radicand`	The quotient fails before the square root: no registered edge.
7	Write the L0 system context in prose.	This chapter's System-context card section	State the system boundary and assumptions before adding attributes.

Worksheet: `HCP/Worksheets/L0_BlackBox.lean` · Solution: `HCP/L0_BlackBox.lean`

5.1.7. Lesson

1. Bare reals. Write the reciprocal quotient in the upstream oscillator. Both `√(k/m)` and `√(m/k)` elaborate as reals; the latter is accepted despite being the wrong formula for angular frequency. A dimension library would reject the reciprocal quotient because its dimension is not time^{-1}.

```
-- Bare reals: √(S.m / S.k) is accepted as a real although oscillator frequency uses
example (S : ClassicalMechanics.HarmonicOscillator) : ℝ :=
  √(S.m / S.k)
```

2. Dimensions. Return the potential energy with the dimension declared by PhysLib. Inspect both quotient result types, then try to assign `m/k` to the result type of `k/m`. The build accepts each quotient at its own dimension and rejects that cross-assignment. `WithDim` distinguishes the quotients by dimension, but it does not say which expresses the intended physics.

```
-- Dimensions: `S.potentialEnergy x` is a `WithDim` value of dimension M·L²·T⁻²; it
example (S : ClassicalMechanics.HarmonicOscillatorWithDim)
    (x : WithDim L𝒹 (EuclideanSpace ℝ (Fin 1))) :
    WithDim (M𝒹 * L𝒹 * L𝒹 * T𝒹⁻¹ * T𝒹⁻¹) ℝ :=
  S.potentialEnergy x

-- Dimensions: The dimension-only, bearer-free `WithDim` quotient m/k has dimension
example (m : WithDim M𝒹 ℝ)
    (k : WithDim (M𝒹 * T𝒹⁻¹ * T𝒹⁻¹) ℝ) :
    WithDim (M𝒹 * (M𝒹 * T𝒹⁻¹ * T𝒹⁻¹)⁻¹) ℝ := m / k

-- Dimensions: The dimension-only, bearer-free `WithDim` quotient k/m has dimension
example (m : WithDim M𝒹 ℝ)
    (k : WithDim (M𝒹 * T𝒹⁻¹ * T𝒹⁻¹) ℝ) :
    WithDim ((M𝒹 * T𝒹⁻¹ * T𝒹⁻¹) * M𝒹⁻¹) ℝ := k / m

-- Dimensions: Refused: type mismatch between m/k's T² dimension and the required k/
/--
error: Type mismatch
  m / k
has type
  WithDim (M𝒹 * (M𝒹 * T𝒹⁻¹ * T𝒹⁻¹)⁻¹) ℝ
but is expected to have type
  WithDim (M𝒹 * T𝒹⁻¹ * T𝒹⁻¹ * M𝒹⁻¹) ℝ
-/
```

```
#guard_msgs (whitespace := lax) in
example (m : WithDim M𝒹 ℝ)
    (k : WithDim (M𝒹 * T𝒹⁻¹ * T𝒹⁻¹) ℝ) :
    WithDim ((M𝒹 * T𝒹⁻¹ * T𝒹⁻¹) * M𝒹⁻¹) ℝ := m / k
```

3. A dimension cannot distinguish kinds. Pass the angular frequency to the PhysLib frequency type: it is the same `WithDim T𝒹⁻¹ ℝ≥0` type. A dimension library accepts interchange here because the types coincide.

```
-- Kind vs dimension: A WithDim T⁻¹ value is accepted as Frequency; equal dimensions
example (ω : WithDim T𝒹⁻¹ ℝ≥0) : Frequency :=
  (fun f : Frequency => f) ω
```

4. Name the kinds. The angular-frequency kind and ISO 80000-3 frequency kind differ by `decide`; the Metrology pairing gives them the same dimension by `rfl`. A dimension library sees one dimension, while the kind layer retains the distinction.

```
-- Kind vs dimension: Proves that angular-frequency and frequency are distinct kinds
example :
    ForPhysLib.ClassicalMechanics.Kinds.angularFrequency ≠
      PropertyKindCalculus.Iso80000.Part3.frequency.kind := by decide

-- Kind vs dimension: Proves that angular frequency and frequency have equal dimensi
example :
    ForPhysLib.ClassicalMechanics.Metrology.angularFrequencyDK.dim =
      PropertyKindCalculus.Iso80000.Part3.frequency.dim := rfl
```

5. Refuse unlike kinds. Write the sum of kinded angular frequency and frequency. The build pins the elaborator's refusal; a dimension-only library would permit the sum because both have dimension `T⁻¹`.

```
-- Kind vs dimension: Refused: no HAdd instance combines angular-frequency and frequ
/--
error: failed to synthesize instance of type class
  HAdd (Quantity ForPhysLib.ClassicalMechanics.Kinds.angularFrequency ℝ)
    (Quantity (Iso80000.Part3.frequency.kind LTMCTDimensionBase) ℝ) ?m.6

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs (whitespace := lax) in
example := fun (ω : Quantity ForPhysLib.ClassicalMechanics.Kinds.angularFrequency ℝ)
    (f : Quantity PropertyKindCalculus.Iso80000.Part3.frequency.kind ℝ) => ω + f
```

6. Refuse the reciprocal radicand. Put `m/k` beneath the square root using the kinded oscillator attributes. Elaboration fails at the quotient because `mass / spring constant` is not a registered edge; the square root is never formed. A dimension-only library would accept the quotient as a value with some dimension.

```
-- Kind vs dimension: Refused: no HDiv instance licenses mass divided by spring-cons
/--
error: failed to synthesize instance of type class
  HDiv (Quantity ForPhysLib.ClassicalMechanics.massK ℝ) (Quantity ForPhysLib.Classic

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs (whitespace := lax) in
example := fun (S : ClassicalMechanics.HarmonicOscillator) =>
  √((ForPhysLib.ClassicalMechanics.massQ S /
    ForPhysLib.ClassicalMechanics.springQ S).magnitude)
```

7. Record the context. The System-context card section of this chapter records the boundary and assumptions; no Lean data is needed for the prose context. At L0 the bearer remains informal and the code has quantities such as `Quantity k ℝ`. There is one of each part, so no bearers can yet be confused; L1 introduces the bearer as a type index.

5.1.8. AI assistance contract

- Keep the contributor's terms: body, spring, wall, frequency, dimension.
- Quote each Lean check from its anchored lesson region; do not invent replacement declarations.
- State only what the build demonstrates. A guarded cross-assignment shows that `WithDim` distinguishes the quotient dimensions.
- Treat errors as evidence only when the pinned message is the elaborator's actual output.

5.1.9. Questions to ask the model so far

Question	Check	What the verdict means
Does the bare-real model catch the reciprocal quotient?	`bare_reals`	No: both expressions have type `ℝ`.
Does `WithDim` distinguish `m/k` and `k/m`?	`quotient_dimensions`	Yes: the build accepts each at its own dimension and rejects the cross-assignment.
Can a dimension-only type tell frequency from angular frequency?	`frequency_alias`	No: both have type `Frequency`.
Do the kinds differ despite the shared dimension?	`distinct_kinds_same_dimension`	Yes: the kinds differ while their paired dimensions are equal.
Why does the kinded reciprocal quotient fail?	`refused_reciprocal_radicand`	`mass / springConstant` has no registered `KindDiv`, so no square-root argument is formed.

5.1.10. System-context card

Scope	System or component	Sort / status	L0 assumptions and attributes
System	The oscillator	Spring–mass oscillator	One body, one linear spring, a wall, and a line of motion.
Inside	Body	Homogeneous, rigid, point-like body	One mass; assumptions to be revisited at L3.
Inside	Spring	Linear spring	One spring; massless and ideal.
Outside	Wall	Fixed, infinitely massive wall	Boundary condition for the spring.
Outside	Line	One-dimensional line of motion	The motion is constrained to this line.

This section records the bearer informally. The Lean expressions are dimension- or kind-indexed quantities without an object index; L1 introduces the bearer into the type.

5.2. L1 — Two bodies

5.2.1. Synopsis

5.2.1.1. Purpose

Introduce the bearer: the particular system a quantity is about. The context has two black-box oscillators, A and B, each a body on a wall spring. An `IndividualQuantity o k ℝ` is a real magnitude of kind k about the object o, so the oscillator a quantity is about is part of its type. Dividing oscillator A's spring constant by oscillator A's mass type-checks and gives an `IndividualQuantity` of kind `omegaSquaredK` about A. Supplying oscillator B's mass where a mass about A is expected fails to type-check with a type mismatch. A `Designated` instance gives A and B distinct nominal names, so they remain two systems even when their parameter values are equal.

5.2.1.2. PKC vocabulary in scope

New at this step:

- `SortOfSystem` — the universal category of a system, here "harmonic oscillator".
- `Sorted` (`sortOf`) — an object type whose objects know their sort.
- `Designated` (`designation`, `designation_inj`) — an object type whose objects have distinct nominal names.
- `IndividualQuantity` — a magnitude of kind `k` about one object `o`.
- `IndividualQuantity.add`, `IndividualQuantity.div` — same-object, kind-licensed arithmetic.
- `DifferenceKind.ofScale` — the difference law that licenses comparing two records of one quantity.
- `massK`, `springConstantK`, `omegaSquaredK` — the oscillator's kinds; k/m has kind `omegaSquaredK`.

Carried from earlier steps: `Quantity`, `massQ`, `springQ`.

5.2.1.3. Verifiable properties

- Oscillators A and B have the same sort, harmonic oscillator (theorem) `HCP.L1_TwoBodies#oscillator_context_and_sort`.
- The designations of A and B are different, and a designation that gives every oscillator the same name is not injective, so it cannot be a `Designated` instance (theorem) `HCP.L1_TwoBodies#designation`.
- For each oscillator o, `omegaSqOf` divides o's spring constant by o's mass with `IndividualQuantity.div` and type-checks as an `IndividualQuantity` of kind `omegaSquaredK` about o, whose magnitude is o's k/m (type-checked) `HCP.L1_TwoBodies#oscillator_indexed_quantities`.

- Oscillator B's mass, supplied where a mass about A is expected, fails with a type mismatch: as a value, as an argument of `IndividualQuantity.add` or `IndividualQuantity.div`, and when oscillator A of rig R_1 is mixed with oscillator A of rig R_2 (refused) `HCP.L1_TwoBodies#refused_cross_oscillator`.
- Subtracting an earlier record of A's mass from a later one, both on rig R_1, type-checks as an `IndividualQuantity` of kind `massK` about A, using `DifferenceKind.ofScale`; its magnitude is the later mass minus the earlier mass (type-checked) `HCP.L1_TwoBodies#same_rig_estimate_difference`.
- Dedicating a kind to an object whose type has no `Sorted` instance fails to type-check (refused) `HCP.L1_TwoBodies#refused_unsorted_object`.

5.2.1.4. Errors prevented

- Cross-system contamination: oscillator B's mass used in oscillator A's k/m. Bare reals and `Quantity` values without a bearer accept it; `IndividualQuantity` rejects it with a type mismatch (`HCP.L1_TwoBodies#bare_real_cross_oscillators`, `HCP.L1_TwoBodies#kinded_bearer_free_cross_oscillators`, `HCP.L1_TwoBodies#refused_cross_oscillator`).
- False identity: one name used for both oscillators, so A and B are reported as one system. The `designation_inj` proof that every `Designated` instance needs rules this out (`HCP.L1_TwoBodies#designation`).
- Unsorted bearer: a quantity about an object of no stated sort. It fails to type-check because no `Sorted` instance exists for the object's type (`HCP.L1_TwoBodies#refused_unsorted_object`).

5.2.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean and PhysLib and trust `Dimension` and `WithDim`; L0 has already shown what those dimensions can and cannot distinguish.

5.2.3. Pull

Two bodies are now in view, one inside each black-box oscillator: whose mass is this? Bare reals and bearer-free kinded values let one oscillator's mass enter another's formula. L1 indexes mass and stiffness by an oscillator bearer scoped to a nominal `Rig`, so its own `k/m` is accepted and cross-oscillator operations are refused. Each oscillator is the whole characterized by ω_0, which is why `k` and `m` share its index. One name is one system: records under that name are its states at different times, configurations, or rival estimates of it; comparing them is the point. Different names, R_1 and R_2, are distinct systems even when their parameters match. L2 opens each box into body and spring(s), revisits which component bears mass and stiffness, and considers the pair as a whole.

5.2.4. Risks at this step

- **Cross-system contamination.** Oscillators A and B have the same kinds, so B's mass can enter A's frequency if the bearer is omitted; the cross-oscillator refusal is pinned at `HCP.L1_TwoBodies#refused_cross_oscillator`.
- **False identity.** Giving A and B one name makes two oscillators look like one system; the injective designation and its constant-name counterexample are pinned at `HCP.L1_TwoBodies#designation`.

Further reading

- `Exhibits/TwoRovers` (rovers A and B).
- `Exhibits/PointParticle`, finding 2.

5.2.5. Small rigorous improvement

Question	Context change	Attribute change	Risk shown	PhysLib stage
There are two bodies now — whose mass is this?	Two particulars, Oscillator A and B, of one sort (harmonic oscillator); each black box contains one point-like body and its own wall spring. The oscillators do not interact. One name is one system; its records are states, configurations, or rival estimates to compare. R_1 and R_2 are distinct names even when parameters match. L2 opens each box into body and spring(s), and introduces their whole/part relations.	No new kinds: mass and stiffness share a rig-indexed oscillator bearer, allowing its `k/m` quotient.	B's mass can enter A's formula when only the kind is checked; one constant designation would collapse A and B into one system.	Stage 2: the kinded form carries its bearer.

Under L0's assumptions, each oscillator is a homogeneous, rigid, point-like body on one ideal, massless spring attached to a fixed, infinitely massive wall, along a line. The oscillator's mass is its body's mass, and its stiffness is its spring's. The oscillator is the whole characterized by ω_0, so `k` and `m` must carry the same oscillator index. The two oscillators do not interact in L1. L2 revisits that assumption as it opens each box into body and spring(s) and considers the pair as a whole.

5.2.6. Learning path

#	Do	Checked by `HCP.L1_TwoBodies` anchor	Concept disclosed
1	Put oscillator B's mass in oscillator A's bare-real frequency radicand.	`HCP.L1_TwoBodies#bare_real_cross_oscillators`	The cross-oscillator expression elaborates as `ℝ`.
2	Form the same spring-constant/mass quotient from kinded, bearer-free values.	`HCP.L1_TwoBodies#kinded_bearer_free_cross_oscillators`	The registered `k/m` edge accepts values from different oscillators.
3	Declare oscillators A and B, give both the harmonic-oscillator sort, and compare their sorts.	`HCP.L1_TwoBodies#oscillator_context_and_sort`	Two particulars instantiate one sort.
4	Give the oscillators distinct designations and prove a constant designation is not injective.	`HCP.L1_TwoBodies#designation`	A valid `Designated` instance cannot collapse distinct oscillators.
5	Index mass and stiffness by one rig-scoped oscillator, form its `k/m`, and compare the mass	`HCP.L1_TwoBodies#oscillator_indexed_quantities`	The accepted quotient reads the oscillator's fields; the dedicated mass kind is shared.

#	Do	Checked by `HCP.L1_TwoBodies` anchor	Concept disclosed
	catalogue entries.		
6	Try cross-oscillator substitutions and addition; compare two weighings of oscillator A at different times on R₁.	`HCP.L1_TwoBodies#refused_cross_oscillator`; `HCP.L1_TwoBodies#same_rig_estimate_difference`	Cross-oscillator operations and cross-rig addition are refused; the same-rig time states admit a typed mass difference whose magnitude reads the real fields. Total mass is meaningful, but whose total requires a whole; that is L2.
7	Ask for the mass kind dedicated to a nominal object with no sort.	`HCP.L1_TwoBodies#refused_unsorted_object`	A nominal `Object` cannot supply the `Sorted` instance required by `dedicatedFor`.

Worksheet: `HCP/Worksheets/L1_TwoBodies.lean` · Solution: `HCP/L1_TwoBodies.lean`

5.2.7. Lesson

1. Bare reals. Put oscillator B's mass in oscillator A's frequency radicand. The upstream oscillator accepts the cross-oscillator expression as a real:

```
-- Bare reals: √(S₁.k / S₂.m) accepts oscillator A's stiffness and oscillator B's ma
example (S₁ S₂ : ClassicalMechanics.HarmonicOscillator) : ℝ :=
  √(S₁.k / S₂.m)
```

2. Kinded, but bearer-free. The spring-constant/mass quotient also accepts values from different oscillators. It is still a `Quantity`, with no oscillator index:

```
-- Bearer safety: A bearer-free Quantity omegaSquaredK accepts oscillator A's stiffn
example (S₁ S₂ : ClassicalMechanics.HarmonicOscillator) :
```

```
    Quantity ForPhysLib.ClassicalMechanics.omegaSquaredK ℝ :=
  ForPhysLib.ClassicalMechanics.springQ S₁ /
    ForPhysLib.ClassicalMechanics.massQ S₂
```

3. Declare the context. Oscillators A and B are distinct particulars of one sort. Each contains one body and one wall spring; neither component is separately indexed yet:

```
inductive Oscillator | A | B
deriving DecidableEq, Repr

def oscillatorS : SortOfSystem := ⟨"harmonic oscillator"⟩

instance : Sorted Oscillator := ⟨fun _ => oscillatorS⟩

-- Same sort, distinct systems: Proves that sortOf assigns oscillator A and oscillat
example : Sorted.sortOf Oscillator.A = Sorted.sortOf Oscillator.B := rfl
```

4. Designate without collapsing. A designation gives each oscillator a nominal name. `Designated` requires a proof, `designation_inj`, that the designation is injective: two different oscillators never get the same name. Without that proof, anything keyed by name, such as a report or a SysML export, could merge A and B into one system. This is the false-identity risk. The instance below names A "oscillator A" and B "oscillator B" and proves injectivity by cases. The theorem `constant_designation_not_injective` shows why a lazy designation fails: the function that names every oscillator "oscillator" sends A and B to the same name, so it is not injective, and no `Designated` instance can use it:

```
instance : Designated Oscillator where
  designation
    | .A => ⟨"oscillator A"⟩
    | .B => ⟨"oscillator B"⟩
  designation_inj {x y} h := by
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

/-- Nominal identity: Proves that the constant designation gives oscillator A and os
theorem constant_designation_not_injective :
    ¬ Function.Injective (fun _ : Oscillator => (⟨"oscillator"⟩ : Object)) := by
  intro h
  have hAB : Oscillator.A = Oscillator.B := h rfl
  cases hAB
```

5. Carry the bearer. The oscillator is the whole characterized by ω_0; mass and stiffness therefore share its index. One `Rig` name is one system: two `TwoOscillators r` records are its states at different times, configurations, or rival estimates, and comparing their quantities is the point. `OscillatorOf r` keeps

those records' oscillator bearers shared; different rig names are distinct systems regardless of matching parameters. Under L0's massless-spring, fixed-wall assumptions, these magnitudes read the body's mass and spring's stiffness. The existing kind table supplies the quotient law; the magnitude theorem reads the real fields. The catalogue entry remains shared because both oscillators have the same sort:

```
/--
One rig name is one system; its records are states, configurations, or rival estimat
compare. Different names denote distinct systems.
-/
inductive Rig | R₁ | R₂
deriving DecidableEq, Repr

structure TwoOscillators (r : Rig) where
  osc : Oscillator → ClassicalMechanics.HarmonicOscillator

structure OscillatorOf (r : Rig) where
  osc : Oscillator

def massOf {r : Rig} (sys : TwoOscillators r) (o : OscillatorOf r) :
    IndividualQuantity o ForPhysLib.ClassicalMechanics.massK ℝ :=
  ⟨(sys.osc o.osc).m⟩

def stiffnessOf {r : Rig} (sys : TwoOscillators r) (o : OscillatorOf r) :
    IndividualQuantity o ForPhysLib.ClassicalMechanics.springConstantK ℝ :=
  ⟨(sys.osc o.osc).k⟩

def omegaSqOf {r : Rig} (sys : TwoOscillators r) (o : OscillatorOf r) :
    IndividualQuantity o ForPhysLib.ClassicalMechanics.omegaSquaredK ℝ :=
  IndividualQuantity.div
    (KindDiv.law
      (k₁ := ForPhysLib.ClassicalMechanics.springConstantK)
      (k₂ := ForPhysLib.ClassicalMechanics.massK)
      (k := ForPhysLib.ClassicalMechanics.omegaSquaredK))
    (stiffnessOf sys o)
    (massOf sys o)

/-- Bearer safety: Proves that oscillator O's indexed omegaSquaredK magnitude equals
theorem omegaSqOf_magnitude {r : Rig} (sys : TwoOscillators r) (o : OscillatorOf r)
    (omegaSqOf sys o).magnitude = (sys.osc o.osc).k / (sys.osc o.osc).m := by
  rw [omegaSqOf, IndividualQuantity.div_magnitude, stiffnessOf, massOf]
```

```
def whole : Component := ⟨"whole"⟩

/-- Kind vs bearer: Proves that the mass kind dedicated to oscillator A's whole equa
theorem mass_kind_shared_by_oscillators :
    ForPhysLib.ClassicalMechanics.massK.dedicatedFor Oscillator.A whole =
      ForPhysLib.ClassicalMechanics.massK.dedicatedFor Oscillator.B whole :=
  KindOfProperty.dedicatedFor_congr rfl whole
```

The sort-level catalogue comparison uses `Sorted Oscillator`; the rig-scoped wrapper needs no additional `Sorted` or `Designated` instance.

6. Compare states; refuse cross-system substitution. Two weighings of oscillator A at different times on rig R_1 have a typed mass difference whose magnitude reads the later and earlier real fields. B's mass cannot fill A's slot, A's stiffness cannot be divided by B's mass, and the two different oscillators' quantities cannot be added at this layer. Nor can same-named oscillator A masses be added across R_1 and R_2: those names identify distinct systems. Total mass is physically meaningful, but whose total requires a whole; that is L2's question:

```
-- Bearer safety: Refused: type mismatch between oscillator B's mass bearer and the
/--
error: Type mismatch
  massOf sys { osc := Oscillator.B }
has type
  IndividualQuantity { osc := Oscillator.B } ForPhysLib.ClassicalMechanics.massK ℝ
but is expected to have type
  IndividualQuantity { osc := Oscillator.A } ForPhysLib.ClassicalMechanics.massK ℝ
-/
#guard_msgs in
example {r : Rig} (sys : TwoOscillators r) :
    IndividualQuantity (⟨.A⟩ : OscillatorOf r)
      ForPhysLib.ClassicalMechanics.massK ℝ :=
  massOf sys ⟨.B⟩

-- Bearer safety: Refused: type mismatch between oscillator A's and oscillator B's m
/--
error: Application type mismatch: The argument
  massOf sys { osc := Oscillator.B }
has type
  IndividualQuantity { osc := Oscillator.B } ForPhysLib.ClassicalMechanics.massK ℝ
but is expected to have type
  IndividualQuantity { osc := Oscillator.A } ForPhysLib.ClassicalMechanics.massK ℝ
in the application
```

```
  IndividualQuantity.add … (massOf sys { osc := Oscillator.A }) (massOf sys { osc :=
-/
#guard_msgs in
example {r : Rig} (sys : TwoOscillators r) :=
  IndividualQuantity.add DifferenceKind.ofScale
    (massOf sys ⟨.A⟩) (massOf sys ⟨.B⟩)

-- Bearer safety: Refused: type mismatch between oscillator A's stiffness bearer and
/--
error: Application type mismatch: The argument
  massOf sys { osc := Oscillator.B }
has type
  IndividualQuantity { osc := Oscillator.B } ForPhysLib.ClassicalMechanics.massK ℝ
but is expected to have type
  IndividualQuantity { osc := Oscillator.A } ForPhysLib.ClassicalMechanics.massK ℝ
in the application
  IndividualQuantity.div h (stiffnessOf sys { osc := Oscillator.A }) (massOf sys { o
-/
#guard_msgs in
example {r : Rig} (sys : TwoOscillators r) :=
  let h :=
    KindDiv.law
      (k₁ := ForPhysLib.ClassicalMechanics.springConstantK)
      (k₂ := ForPhysLib.ClassicalMechanics.massK)
      (k := ForPhysLib.ClassicalMechanics.omegaSquaredK)
  IndividualQuantity.div h (stiffnessOf sys ⟨.A⟩) (massOf sys ⟨.B⟩)

-- Bearer safety: Refused: type mismatch between oscillator A on R₁ and oscillator A
/--
error: Application type mismatch: The argument
  massOf { osc := o } { osc := Oscillator.A }
has type
  IndividualQuantity (@OscillatorOf.mk Rig.R₂ Oscillator.A) ForPhysLib.ClassicalMech
but is expected to have type
  IndividualQuantity (@OscillatorOf.mk Rig.R₁ Oscillator.A) ForPhysLib.ClassicalMech
in the application
  IndividualQuantity.add … (massOf { osc := o } { osc := Oscillator.A }) (massOf { o
-/
#guard_msgs in
example (o : Oscillator → ClassicalMechanics.HarmonicOscillator) :=
  IndividualQuantity.add DifferenceKind.ofScale
```

```
    (massOf (⟨o⟩ : TwoOscillators .R₁) ⟨.A⟩)
    (massOf (⟨o⟩ : TwoOscillators .R₂) ⟨.A⟩)

def oscillatorAMassDifference (earlier later : TwoOscillators .R₁) :
    IndividualQuantity (⟨.A⟩ : OscillatorOf .R₁)
      ForPhysLib.ClassicalMechanics.massK ℝ :=
  IndividualQuantity.add DifferenceKind.ofScale
    (massOf later ⟨.A⟩)
    (-(massOf earlier ⟨.A⟩))

-- Same-name records: Constructs a massK IndividualQuantity about oscillator A on R₁
#guard_msgs in
example (earlier later : TwoOscillators .R₁) :
    IndividualQuantity (⟨.A⟩ : OscillatorOf .R₁)
      ForPhysLib.ClassicalMechanics.massK ℝ :=
  oscillatorAMassDifference earlier later

/-- Same-name records: Proves that oscillator A's R₁ mass-difference magnitude is la
theorem oscillatorAMassDifference_magnitude (earlier later : TwoOscillators .R₁) :
    (oscillatorAMassDifference earlier later).magnitude =
      (later.osc .A).m - (earlier.osc .A).m := by
  simp [oscillatorAMassDifference, IndividualQuantity.add_magnitude,
    IndividualQuantity.neg_magnitude, massOf]
  change (later.osc .A).m + (-(earlier.osc .A).m) =
    (later.osc .A).m - (earlier.osc .A).m
  ring
```

7. A name is not a sort. A nominal `Object` does not supply `Sorted System`, so the catalogue cannot be dedicated through that name:

```
-- Bearer safety: Refused: no Sorted System instance lets Lean dedicate a mass kind
/--
error: failed to synthesize instance of type class
  Sorted System

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs in
example :=
  ForPhysLib.ClassicalMechanics.massK.dedicatedFor
    (⟨"oscillator A"⟩ : Object) whole
```

5.2.8. AI assistance contract

- Keep the contributor's vocabulary: oscillator, body, mass, spring, and bearer.
- Quote each Lean check from its anchored L1 region; do not redefine PhysLib or PKC declarations.
- Distinguish a bearer-free `Quantity` from an `IndividualQuantity` indexed by an oscillator.
- Distinguish the oscillator label from its `OscillatorOf r` bearer, which is scoped to a nominal rig identity.
- Treat one `Rig` name as one system: its records are states at different times, configurations, or rival estimates to compare; distinct rig names denote distinct systems.
- Treat each oscillator as the whole at L1; body and spring become separate bearers when L2 opens the box.
- Treat the guarded elaboration messages as evidence only for the refusals they actually show.

5.2.9. Questions to ask the model so far

Question	Check	What the verdict means
Can oscillator B's mass enter A's bare-real frequency?	`bare_real_cross_oscillators`	Yes; the expression has type `ℝ`.
Does kinding alone prevent the cross-oscillator quotient?	`kinded_bearer_free_cross_oscillators`	No; the registered `springConstantK / massK` edge accepts the two `Quantity` values.
Are oscillator A and B different sorts?	`oscillator_context_and_sort`	No; both have sort `harmonic oscillator`.
Can both oscillators be given one constant designation?	`designation`	The function is not injective, so it cannot witness `Designated Oscillator`.
Does the shared oscillator index admit its own `k/m` and refuse cross-oscillator `k/m`?	`oscillator_indexed_quantities`, `refused_cross_oscillator`	Yes; same-oscillator division is accepted, while A's stiffness over B's mass is refused.
Does the object index keep their masses apart?	`refused_cross_oscillator`	Yes; assignment and addition across A and B are refused.
Can two weighings of oscillator A at different times be compared on R_1?	`same_rig_estimate_difference`	Yes; the typed difference has magnitude `(later.osc .A).m - (earlier.osc .A).m`.
Can same-named oscillator A masses from R_1 and R_2 be added when their parameters are identical?	`refused_cross_oscillator`	No; `OscillatorOf .R₁` and `OscillatorOf .R₂` are distinct bearers by nominal identity.
Can a nominal name reach a dedicated kind?	`refused_unsorted_object`	Not without a sort; the nominal `Object` has no `Sorted` instance.

5.2.10. System-context card

Scope	System or component	Sort / status	L1 assumptions and attributes
System	Oscillator A	One particular of sort `harmonic oscillator`	Black box: one homogeneous, rigid, point-like body on one ideal, massless wall spring; mass mA and stiffness kA.
System	Oscillator B	One particular of sort `harmonic oscillator`	Same L0 assumptions; mass mB and stiffness kB.
Relation	A and B	Distinct, designated oscillators	The oscillators do not interact; L2 revisits this assumption.
Inside each box	Body and spring	Not separately indexed in L1	The oscillator's mass is the body's mass; its stiffness is the spring's. L2 opens these components.
Outside	Walls	Fixed, infinitely massive walls	Each ideal spring has its own fixed-wall boundary.
Outside	Lines	One-dimensional lines of motion	Each oscillator's body remains constrained to its line.

The Lean model takes each complete oscillator as its bearer, scoped to a nominal `Rig`: `massOf`, `stiffnessOf`, and `omegaSqOf` share an `OscillatorOf r` index. One name is one system; records under it are states at different times, configurations, or rival estimates whose quantities are meant to be compared. R_1 and R_2 are distinct systems even when their oscillator parameters match. The box opens into body and spring(s) at L2, where the question of which component bears mass or stiffness becomes explicit.

5.3. L2 — The pair is a system too

5.3.1. Synopsis

5.3.1.1. Purpose

Open each oscillator into its body and wall spring, add a coupling spring between bodies A and B, and treat the coupled pair as a system of its own sort, `pairS`. `assemble` sums a quantity of kind k over the parts of a whole of sort σ only if an `Assembles σ k` instance exists; the kind alone does not license the sum. The lesson provides `Assembles` instances for mass over a pair's bodies and for stiffness over springs in parallel. It provides none for angular frequency over a pair or for stiffness over springs in series, so those sums fail to type-check.

5.3.1.2. PKC vocabulary in scope

New at this step:

- `Composite` (`whole`, `part`) — the object type of a whole and its parts.
- `Decomposition` (`atom`, `union`) — how a whole is carved into parts.
- `Assembles` — the aggregation licence, keyed by the sort of the whole and the kind.
- `assemble` — fold a licensed quantity over a decomposition.
- `Extensive` — a kind whose value over a union is the sum over its parts.
- `DedicatedKind`, `KindOfProperty.dedicatedTo` — a kind dedicated to one sort, such as the pair's normal-mode frequency.
- `Component` — a named component of a system's characterization.
- `displacementK` — spring extension; negative compressed, positive stretched.

Carried from earlier steps: `IndividualQuantity`, `IndividualQuantity.add`, `DifferenceKind.ofScale`, `SortOfSystem`, `Sorted`, `Designated`, `massK`, `springConstantK`.

5.3.1.3. Verifiable properties

- `assemble` of the two bodies' masses gives the pair's mass as an `IndividualQuantity` about the pair as a whole (type-checked) `HCP.L2_ThePair#pair_mass`.
- Adding the pair's mass to body A's mass fails with a type mismatch, because one is about the whole and the other about a part; adding pair P's mass to pair Q's mass fails the same way (refused) `HCP.L2_ThePair#refused_whole_part`.
- `assemble` of angular frequency over the pair's bodies fails to type-check: there is no `Assembles pairS` instance for angular frequency (refused) `HCP.L2_ThePair#refused_frequency_assembly`.
- The normal-mode frequency ω_+ is an angular-frequency kind dedicated to the sort `pairS`, and it is provably different from any angular-frequency kind dedicated to a body or a spring (type-checked) `HCP.L2_ThePair#normal_mode`.
- `assemble` of stiffness over springs in parallel type-checks; over springs in series it fails: there is no `Assembles seriesS springConstantK` instance (refused) `HCP.L2_ThePair#parallel_stiffness`, `HCP.L2_ThePair#refused_series_stiffness`.
- The parallel assembly equals the measured stiffness of the parallel combination, and the series law $1/k = 1/k_1 + 1/k_2$ gives $k = 2$ for springs of stiffness 6 and 3 (theorem) `HCP.L2_ThePair#parallelLaw`, `HCP.L2_ThePair#licenseCashed`, `HCP.L2_ThePair#seriesLaw`.

5.3.1.4. Errors prevented

- Summing what does not sum: the normal-mode frequency taken as the sum of the bodies' angular frequencies. `assemble` fails because no `Assembles` instance exists (`HCP.L2_ThePair#refused_frequency_assembly`).
- Wrong composition law: the stiffnesses of springs in series added as if the springs were in parallel (`HCP.L2_ThePair#refused_series_stiffness`).
- Whole/part confusion: pair P's mass added to body A's mass, a quantity about the whole combined with a quantity about one of its parts (`HCP.L2_ThePair#refused_whole_part`).

5.3.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean and PhysLib and trust `Dimension` and `WithDim`; L0 and L1 have established the oscillator context and the bearer-indexed quantities.

5.3.3. Pull

Once A and B are coupled, what bears the total mass and the oscillation frequency? Open the L1 black boxes into bodies A and B and springs kA, kC, and kB; kC is the new coupling spring. Each wall-spring oscillator supplies its body's mass and wall-spring stiffness, and the coupling spring makes the oscillators interact, so L1's no-interaction assumption falls. Mass is borne by the bodies and stiffness by the springs; the L1 oscillator-indexed `ω² = k/m` does not survive coupling. The normal-mode angular frequency belongs to the pair whole, which has its own sort. Following L1, one `PairId` name is one system: records under it are its states at different times, configurations, or rival estimates; comparing their shared-bearer quantities is the point. Different names denote distinct systems, so P and Q remain distinct even when their parameters match. Each spring has a signed extension, negative when compressed and positive when stretched; two configurations of pair P can compare their coupling-spring extensions with a typed displacement difference.

5.3.4. Risks at this step

- **Aggregation.** In coupled pair P, the pair mass assembles from bodies A and B, but their normal-mode frequencies do not add; the mass assembly and frequency refusal are pinned at `HCP.L2_ThePair#pair_mass` and `HCP.L2_ThePair#refused_frequency_assembly`.
- **Sort versus particular.** The license to add body masses belongs to the coupled-pair sort, and parallel stiffness adds where series stiffness does not; the accepted parallel case and refused series case are pinned at `HCP.L2_ThePair#parallel_stiffness` and `HCP.L2_ThePair#refused_series_stiffness`.

Further reading

- `Exhibits/PointParticle`.
- `CaseStudies/PointParticle`.
- `Composite.Assembles` with `Exhibits/TwoRovers`.

5.3.5. Small rigorous improvement

Question	Context change	Attribute change	Risk shown	PhysLib stage
Once A and B are coupled, what bears the total mass and the oscillation frequency?	Open the L1 black boxes into bodies A and B and springs kA, kC, kB; kC joins A and B, so they interact. The coupled pair has its own sort and a nominal PairId independent of its parameters; spring configurations also carry signed extensions.	Per-part mass, stiffness, and displacement; total mass of the pair; normal-mode angular frequency of the pair; effective stiffness.	The pair's mass adds from its bodies, but normal-mode frequency and series stiffness do not; parallel stiffness adds under its own sort-specific license.	Stage 3: sort-specific assembly licenses.

With B clamped, A is held by its wall spring kA and the coupling spring kC in parallel. The total mass belongs to the pair whole; angular frequency does not add across the bodies, and a normal mode is a property of the coupled whole. The parallel stiffness is extensive for the parallel sort, while series stiffness is whole-proper rather than additive. At L3, each body is still assumed rigid, homogeneous, and point-like, with its mass on the spring axis; an imbalance would make it wobble.

5.3.6. Learning path

#	Do	Checked by `HCP.L2_ThePair` anchor	Concept disclosed
1	Open the boxes into bodies A/B and springs kA/kC/kB, give each part a nominal pair-indexed bearer, and compare compressed/stretched kC configurations of pair P.	`HCP.L2_ThePair#opened_pair`	Mass belongs to a body, stiffness and signed extension belong to a spring; `extensionOf` has a typed same-pair difference.
2	Assemble both body masses into the pair whole.	`HCP.L2_ThePair#pair_mass`	The pair-specific whole mass is the sum of its parts' masses, read from the real fields.
3	Try to add the pair mass to body A's mass and to a pair with identical parameters but a different PairId.	`HCP.L2_ThePair#refused_whole_part`	Whole and part are distinct bearers; P and Q are different systems even when all parameter values and part names match.
4	Try to assemble the bodies' angular frequencies into the pair.	`HCP.L2_ThePair#refused_frequency_assembly`	Angular frequency has no `Assembles` license for the pair sort.
5	Give the normal mode its dedicated kind at the pair sort.	`HCP.L2_ThePair#normal_mode`	The normal mode is neither a body nor a spring property.
6	With B clamped, assemble kA and kC as springs in parallel.	`HCP.L2_ThePair#parallel_stiffness`	The parallel whole has the sum of the two stiffness magnitudes.
7	Try to assemble kA and kB as two springs chained end to end.	`HCP.L2_ThePair#refused_series_stiffness`	No series-sort assembly license is registered.
8	Check the aggregation licenses on the		Parallel stiffness adds; series kA–kB gives 6

#	Do	Checked by `HCP.L2_ThePair` anchor	Concept disclosed
	exhibited pairs: parallel kA–kC and series kA–kB.	`HCP.L2_ThePair#parallelLaw`; `HCP.L2_ThePair#seriesLaw`; `HCP.L2_ThePair#licenseCashed`	‖ 3 = 2. Integer measurements are exact only for the exhibited pairs and do not model longer chains.

Worksheet: `HCP/Worksheets/L2_ThePair.lean` · Solution: `HCP/L2_ThePair.lean`

5.3.7. Lesson

1. Open the box and represent configurations. Each body's upstream harmonic oscillator supplies its wall-spring parameters; the coupling spring supplies stiffness, and each spring has a signed extension. Two `CoupledPair .P` configurations admit a typed difference of their kC extensions:

```
inductive Body | A | B
deriving DecidableEq, Repr

inductive Spring | kA | kC | kB
deriving DecidableEq, Repr

/--
One pair name is one system; its records are states, configurations, or rival estima
compare. Different names denote distinct systems.
-/
inductive PairId | P | Q
deriving DecidableEq, Repr

def bodyS : SortOfSystem := ⟨"point mass"⟩
def springS : SortOfSystem := ⟨"linear spring"⟩
def pairS : SortOfSystem := ⟨"coupled oscillator pair"⟩
def parallelS : SortOfSystem := ⟨"springs in parallel"⟩
def seriesS : SortOfSystem := ⟨"springs in series"⟩

instance : Sorted Body := ⟨fun _ => bodyS⟩
instance : Sorted Spring := ⟨fun _ => springS⟩

instance : Designated Body where
  designation
    | .A => ⟨"body A"⟩
```

```
    | .B => ⟨"body B"⟩
  designation_inj {x y} h := by
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

instance : Designated Spring where
  designation
    | .kA => ⟨"spring A"⟩
    | .kC => ⟨"coupling spring"⟩
    | .kB => ⟨"spring B"⟩
  designation_inj {x y} h := by
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

structure CoupledPair (π : PairId) where
  osc : Body → ClassicalMechanics.HarmonicOscillator
  kC : ℝ
  extension : Spring → ℝ

structure PartOf (π : PairId) where
  part : Body ⊕ Spring
deriving DecidableEq, Repr

def bodyPart (π : PairId) (b : Body) : PartOf π := ⟨Sum.inl b⟩

def springPart (π : PairId) (s : Spring) : PartOf π := ⟨Sum.inr s⟩

def springMagnitude {π : PairId} (p : CoupledPair π) : Spring → ℝ
  | .kA => (p.osc .A).k
  | .kC => p.kC
  | .kB => (p.osc .B).k

def massOf {π : PairId} (p : CoupledPair π) (b : Body) :
    IndividualQuantity
      (Composite.part (σ := pairS) (bodyPart π b))
      ForPhysLib.ClassicalMechanics.massK ℝ :=
  ⟨(p.osc b).m⟩

def stiffnessOf {π : PairId} (p : CoupledPair π) (s : Spring) :
    IndividualQuantity
      (Composite.part (σ := pairS) (springPart π s))
      ForPhysLib.ClassicalMechanics.springConstantK ℝ :=
  ⟨springMagnitude p s⟩
```

```
def extensionOf {π : PairId} (p : CoupledPair π) (s : Spring) :
    IndividualQuantity
      (Composite.part (σ := pairS) (springPart π s))
      ForPhysLib.ClassicalMechanics.displacementK ℝ :=
  ⟨p.extension s⟩

def couplingSpringExtensionDifference
    (compressed stretched : CoupledPair .P) :
    IndividualQuantity
      (Composite.part (σ := pairS) (springPart .P .kC))
      ForPhysLib.ClassicalMechanics.displacementK ℝ :=
  IndividualQuantity.add DifferenceKind.ofScale
    (extensionOf stretched .kC)
    (-(extensionOf compressed .kC))

-- Same-name records: Constructs a displacementK IndividualQuantity about pair P's c
#guard_msgs in
example (compressed stretched : CoupledPair .P) :
    IndividualQuantity
      (Composite.part (σ := pairS) (springPart .P .kC))
      ForPhysLib.ClassicalMechanics.displacementK ℝ :=
  couplingSpringExtensionDifference compressed stretched

/-- Same-name records: Proves that pair P's coupling-spring displacement difference
theorem couplingSpringExtensionDifference_magnitude
    (compressed stretched : CoupledPair .P) :
    (couplingSpringExtensionDifference compressed stretched).magnitude =
      stretched.extension .kC - compressed.extension .kC := by
  simp [couplingSpringExtensionDifference, IndividualQuantity.add_magnitude,
    IndividualQuantity.neg_magnitude, extensionOf]
  change stretched.extension .kC + (-(compressed.extension .kC)) =
    stretched.extension .kC - compressed.extension .kC
  ring
```

Mass is now indexed by a body part and stiffness by a spring part. The oscillator-indexed L1 `ω²` was the frequency of an isolated oscillator; it is not the normal-mode frequency of this coupled pair.

2. Assemble the pair's mass. The two body masses are parts of `Composite.whole`; `rfl` checks that the total reads the upstream oscillator fields:

```
instance : Assembles pairS ForPhysLib.ClassicalMechanics.massK :=
  ⟨DifferenceKind.ofScale⟩

def bodies : Decomposition Body := .union (.atom .A) (.atom .B)

def pairMass {π : PairId} (p : CoupledPair π) :
    IndividualQuantity
      (Composite.whole : Composite pairS (PartOf π))
      ForPhysLib.ClassicalMechanics.massK ℝ :=
  assemble (k := ForPhysLib.ClassicalMechanics.massK) pairS
    (Composite.whole : Composite pairS (PartOf π))
    (fun b : Body => Composite.part (σ := pairS) (bodyPart π b))
    bodies
    (fun b => massOf p b)

/-- Licensed composition: Proves that for pair π, the assembled whole-mass magnitude
theorem pairMass_magnitude {π : PairId} (p : CoupledPair π) :
    (pairMass p).magnitude = (p.osc .A).m + (p.osc .B).m := by
  rfl
```

3. Refuse whole plus part, and one pair plus another. Total mass is meaningful, but its whole is not body A. A second pair is another system even when all parameters and part names match, because PairId is nominal:

```
-- Bearer safety: Refused: type mismatch between pair P's whole-mass bearer and body
/--
error: Application type mismatch: The argument
  massOf p Body.A
has type
  IndividualQuantity (Composite.part (bodyPart π Body.A)) ForPhysLib.ClassicalMechan
but is expected to have type
  IndividualQuantity Composite.whole ForPhysLib.ClassicalMechanics.massK ℝ
in the application
  IndividualQuantity.add … (pairMass p) (massOf p Body.A)
-/
#guard_msgs in
example {π : PairId} (p : CoupledPair π) :=
  IndividualQuantity.add (k := ForPhysLib.ClassicalMechanics.massK)
    DifferenceKind.ofScale (pairMass p) (massOf p .A)

-- Bearer safety: Refused: type mismatch between pair P's and pair Q's whole-mass be
```

```
/--
error: Application type mismatch: The argument
  pairMass q
has type
  IndividualQuantity (@Composite.whole pairS (PartOf PairId.Q)) ForPhysLib.Classical
but is expected to have type
  IndividualQuantity (@Composite.whole pairS (PartOf PairId.P)) ForPhysLib.Classical
in the application
  IndividualQuantity.add … (pairMass p) (pairMass q)
-/
#guard_msgs in
example (o : Body → ClassicalMechanics.HarmonicOscillator) :=
  let p : CoupledPair .P := ⟨o, 2, fun _ => 0⟩
  let q : CoupledPair .Q := ⟨o, 2, fun _ => 0⟩
  IndividualQuantity.add (k := ForPhysLib.ClassicalMechanics.massK)
    DifferenceKind.ofScale (pairMass p) (pairMass (π := .Q) q)
```

4. Refuse to assemble angular frequency. The pair's license table has no entry for it:

```
-- Licensed composition: Refused: no Assembles pairS angularFrequency instance licen
/--
error: failed to synthesize instance of type class
  Assembles pairS ForPhysLib.ClassicalMechanics.Kinds.angularFrequency

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs in
example :=
  fun {π : PairId} (p : CoupledPair π) =>
    fun (f : (b : Body) → 
      IndividualQuantity
        (Composite.part (σ := pairS) (bodyPart π b))
        ForPhysLib.ClassicalMechanics.Kinds.angularFrequency ℝ) =>
      assemble (k := ForPhysLib.ClassicalMechanics.Kinds.angularFrequency) pairS
        (Composite.whole : Composite pairS (PartOf π))
        (fun b : Body => Composite.part (σ := pairS) (bodyPart π b))
        bodies f
```

5. Give the normal mode its whole. A dedicated angular-frequency kind at `pairS` is distinct from one dedicated to a body or a spring:

```
def whole : Component := ⟨"whole"⟩

def dkOmegaPlus : DedicatedKind :=
  ForPhysLib.ClassicalMechanics.Kinds.angularFrequency.dedicatedTo
    pairS ⟨"normal mode +"⟩

/-- Kind vs dimension: Proves that pair P's normal-mode kind differs from angular fr
theorem normal_mode_not_of_a_body :
    dkOmegaPlus ≠
      ForPhysLib.ClassicalMechanics.Kinds.angularFrequency.dedicatedFor Body.A whole
  DedicatedKind.distinct_of_sort (by decide)

/-- Kind vs dimension: Proves that pair P's normal-mode kind differs from angular fr
theorem normal_mode_not_of_a_spring :
    dkOmegaPlus ≠
      ForPhysLib.ClassicalMechanics.Kinds.angularFrequency.dedicatedFor Spring.kC wh
  DedicatedKind.distinct_of_sort (by decide)
```

6. Assemble parallel stiffness. With B clamped, A is held by kA and kC in parallel:

```
instance : Assembles parallelS ForPhysLib.ClassicalMechanics.springConstantK :=
  ⟨DifferenceKind.ofScale⟩

def parallelSprings : Decomposition Spring := .union (.atom .kA) (.atom .kC)

def stiffnessInSort {π : PairId} (p : CoupledPair π) (σ : SortOfSystem) (s : Spring)
    IndividualQuantity (Composite.part (σ := σ) (springPart π s))
      ForPhysLib.ClassicalMechanics.springConstantK ℝ :=
  ⟨springMagnitude p s⟩

def parallelStiffness {π : PairId} (p : CoupledPair π) :
    IndividualQuantity
      (Composite.whole : Composite parallelS (PartOf π))
      ForPhysLib.ClassicalMechanics.springConstantK ℝ :=
  assemble (k := ForPhysLib.ClassicalMechanics.springConstantK) parallelS
    (Composite.whole : Composite parallelS (PartOf π))
    (fun s : Spring => Composite.part (σ := parallelS) (springPart π s))
    parallelSprings
    (fun s => stiffnessInSort p parallelS s)

/-- Licensed composition: Proves that pair π's parallelS whole has magnitude A.k + k
```

```
theorem parallelStiffness_magnitude {π : PairId} (p : CoupledPair π) :
    (parallelStiffness p).magnitude = (p.osc .A).k + p.kC := by
  rfl
```

7. Refuse series stiffness. For kA and kB chained end to end, no additive series license is registered:

```
def seriesSprings : Decomposition Spring := .union (.atom .kA) (.atom .kB)

-- Licensed composition: Refused: no Assembles seriesS springConstantK instance; an
/--
error: failed to synthesize instance of type class
  Assembles seriesS ForPhysLib.ClassicalMechanics.springConstantK

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs in
example {π : PairId} (p : CoupledPair π) :=
  assemble (k := ForPhysLib.ClassicalMechanics.springConstantK) seriesS
    (Composite.whole : Composite seriesS (PartOf π))
    (fun s : Spring => Composite.part (σ := seriesS) (springPart π s))
    seriesSprings
    (fun s => stiffnessInSort p seriesS s)
```

8. Check the licenses against measurements. The measurements use HCP's own `Spring`. `PropertyValue.numeral` is fixed at `Int`, so the exhibited kA–kB series chain is exact (6 ‖ 3 = 2); these integer numerals do not model longer chains:

```
def stiffnessNumeral : Spring → Int
  | .kA => 6
  | .kC => 2
  | .kB => 3

def parallelNumeral : Decomposition Spring → Int
  | .atom s => stiffnessNumeral s
  | .union a b => parallelNumeral a + parallelNumeral b

def parallelMeasurement : Measurement Spring := fun d =>
  { kind := ForPhysLib.ClassicalMechanics.springConstantK
    numeral := parallelNumeral d
    reference := "N/m" }

/-- Licensed composition: Proves that parallel springConstantK measurements add over
```

```
theorem parallel_extensive :
    Extensive ForPhysLib.ClassicalMechanics.springConstantK parallelMeasurement :=
  ⟨fun _ => rfl, fun _ _ => rfl⟩

def seriesNumeral : Decomposition Spring → Int
  | .atom s => stiffnessNumeral s
  | .union a b =>
      (seriesNumeral a * seriesNumeral b) / (seriesNumeral a + seriesNumeral b)

def seriesMeasurement : Measurement Spring := fun d =>
  { kind := ForPhysLib.ClassicalMechanics.springConstantK
    numeral := seriesNumeral d
    reference := "N/m" }

/-- Composition law: Proves that series springs kA = 6 and kB = 3 measure 2, not the
theorem series_kA_kB_exact : (seriesMeasurement seriesSprings).numeral = 2 := by d

/-- Composition law: Proves that series measurement is WholeProper—neither additive
theorem series_whole_proper :
    WholeProper ForPhysLib.ClassicalMechanics.springConstantK seriesMeasurement wher
  ofKind := fun _ => rfl
  notAdditive := ⟨.kA, .kB, by decide⟩
  notUniform := ⟨.kA, .kA, ⟨rfl, by decide⟩⟩

/-- Composition law: Proves that series springConstantK measurement is not extensive
theorem series_not_extensive :
    ¬ Extensive ForPhysLib.ClassicalMechanics.springConstantK seriesMeasurement :=
  series_whole_proper.not_extensive

/-- Licensed composition: Proves that parallel assembly of kA = 6 and kC = 2 for pai
theorem parallel_assembled_is_measured {π : PairId} (_p : CoupledPair π) :
    (assemble (k := ForPhysLib.ClassicalMechanics.springConstantK) parallelS
        (Composite.whole : Composite parallelS (PartOf π))
        (fun s => Composite.part (σ := parallelS) (springPart π s)) parallelSprings
        (fun s => ⟨(parallelMeasurement (.atom s)).numeral⟩)).magnitude
      = (parallelMeasurement parallelSprings).numeral :=
  assemble_eq_measured parallelS Composite.whole
    (fun s => Composite.part (σ := parallelS) (springPart π s))
    parallel_extensive parallelSprings

/-- Licensed composition: Proves that if seriesS springConstantK assembly is license
```

```
theorem series_registered_would_lie
    {π : PairId} (_p : CoupledPair π)
    [Assembles seriesS ForPhysLib.ClassicalMechanics.springConstantK] :
    ∃ d : Decomposition Spring,
      (assemble (k := ForPhysLib.ClassicalMechanics.springConstantK) (R := Int) seri
          (Composite.whole : Composite seriesS (PartOf π))
          (fun s => Composite.part (σ := seriesS) (springPart π s)) d
          (fun s => ⟨(seriesMeasurement (.atom s)).numeral⟩)).magnitude
        ≠ (seriesMeasurement d).numeral :=
  assemble_ne_measured seriesS Composite.whole
    (fun s => Composite.part (σ := seriesS) (springPart π s)) series_whole_proper
```

The measurements verify the curation: parallel stiffness is extensive, while series stiffness is whole-proper. `Measurement` carries an `Int` numeral, so the exhibited kA–kB series value is exact (6 ∥ 3 = 2); these integer numerals do not model longer chains. Registering the series sum would produce a magnitude that disagrees with its measurement.

The System-context card section of this chapter lists the bodies, all three springs, the fixed walls, and the line of motion. The Lean model's nominal `PartOf π` index keeps parts of differently identified pairs distinct even when their parameters match, while body mass, spring stiffness, and the pair's normal-mode frequency stay on their respective bearers. The on-axis, rigid, homogeneous, point-like body assumption returns at L3.

5.3.8. AI assistance contract

- Use the HCP-owned `Body`, `Spring`, `PartOf`, and `CoupledPair` types; do not import or reuse the Tutorial model.
- Keep body mass, spring stiffness, and pair normal-mode frequency on their respective bearers.
- Give each spring a signed displacement-kind extension (negative when compressed, positive when stretched), and compare configurations under the same `PairId`.
- Treat the pair as a whole distinct from every body and spring part.
- State that the L1 no-interaction assumption falls here and that L3 revisits the on-axis, rigid-body assumption.
- Use sort-specific `Assembles` licenses: mass for the pair and stiffness for parallel springs, but neither angular frequency for the pair nor stiffness for springs in series.
- Treat guarded elaboration errors as evidence only for the refusals they show.

5.3.9. Questions to ask the model so far

Question	Check	What the verdict means
Where does total mass live?	`pair_mass`	On `Composite.whole`; the two body masses assemble there.
Can compressed and stretched configurations of pair P be compared by kC extension?	`opened_pair`	Yes; the typed displacement difference has magnitude `stretched.extension .kC - compressed.extension .kC`.
Can a whole mass be added to one of its parts?	`refused_whole_part`	No; their composite object indices differ.
Can two pairs' whole masses be added when their parameters are identical?	`refused_whole_part`	No; nominal `PartOf .P` and `PartOf .Q` have distinct bearer types independent of parameter values.
Does angular frequency assemble across the pair's bodies?	`refused_frequency_assembly`	No; the pair sort has no angular-frequency license.
What sort bears the normal mode?	`normal_mode`	The dedicated kind is attached to `pairS`, not `bodyS` or `springS`.
What happens to stiffness when kA and kC act in parallel?	`parallel_stiffness`, `parallelLaw`	The effective magnitude is their sum, and the measurement is extensive.
May the same summation rule be used for springs in series?	`refused_series_stiffness`, `seriesLaw`, `licenseCashed`	No; kA–kB reads 6 ∥ 3 = 2, and a hypothetical sum license reports 9. The `Int` examples are exact only for the exhibited pairs, not longer chains.

5.3.10. System-context card

Scope	System or component	Sort / status	L2 assumptions and attributes
System	Coupled pair	`pairS`; composite whole	Bodies A and B interact through kC. The normal-mode angular frequency belongs to the pair.
Part	Body A	`Body.A`, sort `bodyS`	Homogeneous, rigid, point-like; mass mA lies on the spring axis.
Part	Body B	`Body.B`, sort `bodyS`	Homogeneous, rigid, point-like; mass mB lies on the spring axis.
Part	Wall spring kA	`Spring.kA`, sort `springS`	Linear spring from body A to the fixed wall; stiffness kA and signed extension (negative compressed, positive stretched).
Part	Coupling spring kC	`Spring.kC`, sort `springS`	Linear spring joining bodies A and B; stiffness kC and signed extension (negative compressed, positive stretched).
Part	Wall spring kB	`Spring.kB`, sort `springS`	Linear spring from body B to the fixed wall; stiffness kB and signed extension (negative compressed, positive stretched).
Outside	Walls and line	Fixed, infinitely massive walls; one-dimensional motion	The wall springs remain attached to fixed boundaries and both bodies move along the line.

The Lean model keeps each body's oscillator as the source of its mass and wall-spring stiffness; `kC` is the pair's new coupling field, and `extensionOf` indexes each signed displacement by its spring part. One `PairId` name is one system: records under it are states at different times, configurations, or rival estimates to compare; distinct names are distinct systems. `PartOf π` uses nominal identity, so identical parameter records under P and Q still yield different pair part types. The `Measurement` numeral is an `Int`: the series example is exact for kA–kB (6 ∥ 3 = 2), but these integer numerals do not model longer chains. L3 revisits the assumption that each body is rigid, homogeneous, and point-like with its mass on the spring axis: if the mass is off-axis, the body wobbles.

5.4. L3 — The body opens up

5.4.1. Synopsis

5.4.1.1. Purpose

Open body A of pair P into two rigid halves, with a body-fixed frame beside the lab frame. Mass assembles over the halves as a sum. The centre of mass is the mass-weighted mean of the halves' positions, which is meaningful only when the total mass is nonzero. Every half of a rigid body has the same angular velocity, so angular velocity is intensive and has no `Assembles` instance. Each frame-dependent quantity has its frame in its type (`InFrame`), so a body-frame rotational energy computed from a lab-frame angular velocity fails to type-check.

5.4.1.2. PKC vocabulary in scope

New at this step:

- `WeightedCarving` (`parts`, `weight`, `mean`, `total_ne_zero`) — a carving with per-part weights whose mean requires nonzero total weight.
- `totalWeight`, `weightedSum` — the pieces of a weighted mean.
- `Decomposition.fold` — a value per part, combined over a decomposition.
- `InFrame` — components read in a stated frame.
- `union_mass`, `union_centerOfMass`, `union_inertiaTensor` — how mass, centre of mass and inertia combine over a union.
- `labOmega`, `bodyOmega`, `bodyInertia`, `rotationalContraction` — rigid-body quantities indexed by frame.

Carried from earlier steps: `Composite`, `Decomposition`, `Assembles`, `assemble`, `IndividualQuantity`, `DifferenceKind.ofScale`, `SortOfSystem`, `Sorted`, `Designated`, `displacementK`.

5.4.1.3. Verifiable properties

- Body A's centre of mass is the mass-weighted mean of its halves' positions: halves of mass 3 and 5 at positions 0 and 10 give 25/4, not the midpoint 5 (theorem) `HCP.L3_TheBodyOpens#center_of_mass_mean`.
- PhysLib's `centerOfMass` of a body with zero mass is the origin, because 1/0 = 0 in Lean; body A's carving comes with a proof that its total mass is nonzero (theorem) `HCP.L3_TheBodyOpens#massless_center_of_mass`.

- Angular velocity is equal on both halves and is not a sum over them, and `assemble` of angular velocity over the halves fails: there is no `Assembles rigidBodyS angularVelocityK` instance (refused) `HCP.L3_TheBodyOpens#angular_velocity_intensive`.
- `rotationalContraction` of the body-frame inertia with the lab-frame ω fails with a type mismatch between `InFrame lab` and `InFrame body` (refused) `HCP.L3_TheBodyOpens#framed_rotational_energy`.
- With the body-frame ω, the kinded rotational energy equals PhysLib's rotational kinetic energy (theorem) `HCP.L3_TheBodyOpens#framed_rotational_energy`.
- A homogeneous solid sphere of mass m and radius R has moment of inertia (2/5) m R² (theorem) `HCP.L3_TheBodyOpens#solid_sphere`.

5.4.1.4. Errors prevented

- Sum or midpoint instead of weighted mean: body A's centre of mass computed as the sum or the midpoint of its halves' positions (`HCP.L3_TheBodyOpens#center_of_mass_mean`).
- Degenerate whole: a centre of mass reported for a body with zero total mass, which PhysLib returns as the origin (`HCP.L3_TheBodyOpens#massless_center_of_mass`).
- Wrong frame: PhysLib accepts the lab-frame ω in the body-frame energy; with `InFrame`, it fails to type-check (`HCP.L3_TheBodyOpens#bare_frame_mismatch`, `HCP.L3_TheBodyOpens#framed_rotational_energy`).

5.4.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean and PhysLib and trust `Dimension` and `WithDim`; L0–L2 have established kinds, bearers, and sort-specific assembly licenses.

5.4.3. Pull

Body A of pair P wobbles instead of sliding along the line. Its centre of mass is off the springs' line of action, so the spring force misses the centre of mass, produces a torque, and rotates the body. Body A therefore stops being a point mass: it opens into left and right halves, acquires a mass distribution and inertia tensor, and needs a body-fixed frame beside the lab frame. One name remains one system: records under that name are its states at different times, configurations, or rival estimates; comparing them is the point. Distinct names are distinct systems. `HalfOf .P .A` keeps body A's halves shared across descriptions of pair P, while a half of pair Q has a different bearer type.

5.4.4. Risks at this step

- **Wrong carving.** For body A of pair P, counting a half twice or leaving one out changes the whole mass; the pinned decomposition gives one slot to each half and proves the total at `HCP.L3_TheBodyOpens#opened_body`.
- **Weighted mean, not sum.** The centre of mass uses each half's mass and position, while both halves share the same angular velocity; the pinned checks are `HCP.L3_TheBodyOpens#center_of_mass_mean` and `HCP.L3_TheBodyOpens#angular_velocity_intensive`.
- **Degenerate whole and frame.** A massless body's `centerOfMass` evaluates to the origin because Lean defines 1/0 = 0, which is not a physical centre of mass; and lab-frame angular velocity is not the body-frame input to rotational energy; the checks are `HCP.L3_TheBodyOpens#massless_center_of_mass`, `HCP.L3_TheBodyOpens#bare_frame_mismatch`, and `HCP.L3_TheBodyOpens#framed_rotational_energy`.

Further reading

- `Exhibits/Composition`, findings 2 and 5.

5.4.5. Small rigorous improvement

Question	Context change	Attribute change	Risk shown	PhysLib stage
Body A wobbles instead of sliding along the line. Why, and what must the model now say about the body?	Body A of pair P becomes a rigid composite carved into left and right halves; a body-fixed frame appears beside the lab frame.	Per-half mass and position, centre of mass, inertia tensor, and body- versus lab-frame angular velocity.	Counting the wrong halves changes total mass; angular velocity is intensive; a massless body's centre of mass evaluates to the origin, which is not physical; lab-frame angular velocity does not match the body-frame energy input.	Stages 2–3 on rigid bodies; proved solid-sphere inertia theorem.

The L2 on-axis point-mass assumption has fallen. L4 revisits the remaining assumption of motion along one line seen by one observer.

5.4.6. Learning path

#	Do	Checked by `HCP.L3_TheBodyOpens` anchor	Concept disclosed
1	Open body A into left and right halves, index them by pair and body identity, and assemble their masses.	`opened_body`	`HalfOf .P .A` names the bearers; mass assembles to the rigid-body whole, while P and Q remain distinct systems.
2	Weight the half positions by this state's half masses, then try to assemble positions by addition.	`center_of_mass_mean`	`centerOfMass_eq` uses the `OpenBody.mass` and `.position` fields; masses 3 and 5 at positions 0 and 10 give 25/4, not the unit-weight midpoint 5. `rigidBodyS` has no displacement assembly license.
3	Inspect the massless whole and the nonzero-total condition on a weighted carving.	`massless_center_of_mass`	PhysLib reports the origin for zero mass; the license is hypothesis `h`, which a massless state cannot satisfy to build the carving.
4	Check angular velocity over halves and try to assemble it.	`angular_velocity_intensive`	The halves share ω; adding their angular velocities would double it, so no license exists.
5	Pass lab-frame angular velocity to upstream rotational kinetic energy.	`bare_frame_mismatch`	The bare expression type-checks even though the inertia tensor is body-fixed, so the accepted term is physically wrong.
6	Apply PKC's frame-indexed contraction to lab and body angular velocities.	`framed_rotational_energy`	The lab-frame term is refused, the body-frame term is accepted, and the accepted contraction agrees with PhysLib energy.
7	Specialize the old homogeneity assumption to a solid sphere.	`solid_sphere`	The proved inertia tensor is `(2/5) m R² · 1`; upstream records the target as `informal_lemma CM-RB-sphereI`.

Worksheet: `HCP/Worksheets/L3_TheBodyOpens.lean` · Solution: `HCP/L3_TheBodyOpens.lean`

5.4.7. Lesson

1. Open body A. `HalfOf π b` adds nominal pair-and-body identity to the upstream left/right `Half`. `OpenBody π b` records per-half mass and one position coordinate. The existing `Assembles rigidBodyS massK` license produces the whole-body total and its magnitude reads the two real fields. Upstream's union theorems state the corresponding mass and inertia-tensor laws. Same-name records share bearers, while P and Q do not:

```
/--
One pair name is one system. Records under that name describe states, configurations
rival estimates of it; distinct names denote distinct systems.
-/
inductive PairId | P | Q
deriving DecidableEq, Repr

inductive Body | A | B
deriving DecidableEq, Repr

def halfS : SortOfSystem := ⟨"half of a rigid body"⟩

structure HalfOf (π : PairId) (b : Body) where
  half : Half
deriving DecidableEq, Repr

instance {π : PairId} {b : Body} : Sorted (HalfOf π b) :=
  ⟨fun _ => halfS⟩

instance {π : PairId} {b : Body} : Designated (HalfOf π b) where
  designation
    | ⟨.left⟩ => ⟨"left half"⟩
    | ⟨.right⟩ => ⟨"right half"⟩
  designation_inj {x y} h := by
    rcases x with ⟨x⟩
    rcases y with ⟨y⟩
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

structure OpenBody (π : PairId) (b : Body) where
  mass : Half → ℝ
  position : Half → ℝ

def halfPart (π : PairId) (b : Body) (h : Half) : HalfOf π b := ⟨h⟩
```

```
def bodyHalves : Decomposition Half := .union (.atom .left) (.atom .right)

def massOf {π : PairId} {b : Body} (state : OpenBody π b) (h : HalfOf π b) :
    IndividualQuantity
      (Composite.part (σ := rigidBodyS) h)
      massK ℝ :=
  ⟨state.mass h.half⟩

def bodyMass {π : PairId} {b : Body} (state : OpenBody π b) :
    IndividualQuantity
      (Composite.whole : Composite rigidBodyS (HalfOf π b))
      massK ℝ :=
  assemble (k := massK) rigidBodyS
    (Composite.whole : Composite rigidBodyS (HalfOf π b))
    (fun h : Half => Composite.part (σ := rigidBodyS) (halfPart π b h))
    bodyHalves
    (fun h => massOf state (halfPart π b h))

/-- Composition law: Proves that the assembled whole-mass magnitude of a body equals
theorem bodyMass_magnitude {π : PairId} {b : Body} (state : OpenBody π b) :
    (bodyMass state).magnitude = state.mass .left + state.mass .right := by
  rfl

-- Composition law: Proves that PhysLib's union-mass law is mass(A ∪ B) = mass(A) +
example := ForPhysLib.Exhibits.Composition.union_mass

-- Composition law: Proves that PhysLib's union-inertia law is I(A ∪ B) = I(A) + I(B
example := ForPhysLib.Exhibits.Composition.union_inertiaTensor

-- Same-name records: Constructs a massK IndividualQuantity about body A's left half
#guard_msgs in
example (earlier later : OpenBody .P .A) :
    IndividualQuantity
      (Composite.part (σ := rigidBodyS) (halfPart .P .A .left))
      massK ℝ :=
  IndividualQuantity.add DifferenceKind.ofScale
    (massOf later (halfPart .P .A .left))
    (-(massOf earlier (halfPart .P .A .left)))

-- Bearer safety: Refused: type mismatch between body A's left-half mass bearers in
```

```
/--
error: Application type mismatch: The argument
  massOf q (halfPart PairId.Q Body.A Half.left)
has type
  IndividualQuantity (Composite.part (halfPart PairId.Q Body.A Half.left)) massK ℝ
but is expected to have type
  IndividualQuantity (Composite.part (halfPart PairId.P Body.A Half.left)) massK ℝ
in the application
  IndividualQuantity.add … (massOf p (halfPart PairId.P Body.A Half.left))
    (massOf q (halfPart PairId.Q Body.A Half.left))
-/
#guard_msgs in
example (p : OpenBody .P .A) (q : OpenBody .Q .A) :=
  IndividualQuantity.add DifferenceKind.ofScale
    (massOf p (halfPart .P .A .left))
    (massOf q (halfPart .Q .A .left))
```

The HCP total is the narrow restatement: upstream `union_mass` and `union_inertiaTensor` are about PhysLib's `RigidBody`, not the nominal `HalfOf .P .A` bearer.

2. Centre of mass is a weighted mean. Upstream supplies both the rigid-body union law and the general weighted-mean machinery. HCP's `centerOfMassCarving` uses this state's `OpenBody.mass` for its weights, and `centerOfMass_eq` gives the resulting formula. For example, masses 3 and 5 at positions 0 and 10 give 25/4 (6.25), not the unit-weight midpoint 5. HCP's position quantities use the same nominal half bearers as mass, but an attempted sum has no `Assembles` license:

```
-- Centre of mass: Proves that with nonzero masses, (mA + mB)·COM(A ∪ B) = mA·COM(A)
example := ForPhysLib.Exhibits.Composition.union_centerOfMass

noncomputable def centerOfMassCarving {π : PairId} {b : Body}
    (state : OpenBody π b)
    (h : state.mass .left + state.mass .right ≠ 0) :
    WeightedCarving ℝ Half where
  parts := bodyHalves
  weight := state.mass
  total_ne_zero := by
    change state.mass .left + state.mass .right ≠ 0
    exact h

/-- Centre of mass: Proves that a body's two-half carving mean is (mL·xL + mR·xR)/(m
theorem centerOfMass_eq {π : PairId} {b : Body} (state : OpenBody π b)
    (h : state.mass .left + state.mass .right ≠ 0) :
```

```
    (centerOfMassCarving state h).mean state.position =
      (state.mass .left * state.position .left + state.mass .right * state.position
        (state.mass .left + state.mass .right) := by
  rfl

def unequalMassExample : OpenBody .P .A where
  mass
    | .left => 3
    | .right => 5
  position
    | .left => 0
    | .right => 10

/-- Centre of mass: Proves that body A in pair P, with half masses 3 and 5 at positi
theorem unequalMassExample_centerOfMass :
    (centerOfMassCarving unequalMassExample (by norm_num [unequalMassExample])).r
      unequalMassExample.position = (25 : ℝ) / 4 := by
  norm_num [centerOfMassCarving, unequalMassExample, WeightedCarving.mean, weighted
    totalWeight, bodyHalves, Decomposition.fold, Carrier.add]

def positionOf {π : PairId} {b : Body} (state : OpenBody π b) (h : HalfOf π b) :
    IndividualQuantity
      (Composite.part (σ := rigidBodyS) h)
      ForPhysLib.ClassicalMechanics.displacementK ℝ :=
  ⟨state.position h.half⟩

-- Centre of mass: Refused: no Assembles rigidBodyS displacementK instance; the halv
/--
error: failed to synthesize instance of type class
  Assembles rigidBodyS ForPhysLib.ClassicalMechanics.displacementK

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs in
example {π : PairId} {b : Body} (state : OpenBody π b) :=
  assemble (k := ForPhysLib.ClassicalMechanics.displacementK) rigidBodyS
    (Composite.whole : Composite rigidBodyS (HalfOf π b))
    (fun h : Half => Composite.part (σ := rigidBodyS) (halfPart π b h))
    bodyHalves
    (fun h => positionOf state (halfPart π b h))
```

The physical content is `(mA + mB) com = mA comA + mB comB`, not `com = comA + comB`.

3. The degenerate whole. PhysLib defines the massless body's centre of mass to be the origin by convention. The license to build `centerOfMassCarving` is hypothesis `h`, that the state's total mass is nonzero; a massless state cannot satisfy it, so it cannot build the carving:

```
-- Centre of mass: proves that a massless body's `centerOfMass` is the origin, becau
example := ForPhysLib.Exhibits.Composition.massless_centerOfMass

-- Centre of mass: Proves that the two mass-weighted halves have total weight 2 ≠ 0,
example := ForPhysLib.Exhibits.Composition.halvesByMass_total_ne_zero
```

4. Angular velocity is intensive. The two rigid halves share one angular velocity; adding their measurements would double it. The upstream finding proves the intensive law and refutes extensivity, while the HCP attempted assembly is rejected:

```
-- Composition law: Proves that angular velocity is intensive: each rigid-body half
example := ForPhysLib.Exhibits.Composition.omegaOfHalf_intensive

-- Composition law: Proves that angular velocity is not extensive across the rigid b
example := ForPhysLib.Exhibits.Composition.omegaOfHalf_not_extensive

-- Licensed composition: Refused: no Assembles rigidBodyS angularVelocityK instance;
/--
error: failed to synthesize instance of type class
  Assembles rigidBodyS angularVelocityK

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs in
example :=
  fun {π : PairId} {b : Body}
      (ω : (h : Half) →
        IndividualQuantity
          (Composite.part (σ := rigidBodyS) (halfPart π b h))
          angularVelocityK ℝ) =>
    assemble (k := angularVelocityK) rigidBodyS
      (Composite.whole : Composite rigidBodyS (HalfOf π b))
      (fun h : Half => Composite.part (σ := rigidBodyS) (halfPart π b h))
      bodyHalves ω
```

5. Upstream frames. PhysLib's bare rotational-energy API accepts the lab-frame `M.angularVelocity t` against the body-fixed inertia tensor. The term below type-checks, but it is physically wrong:

```
-- Frame safety: The bare ℝ example accepts rotational energy from M's body inertia
noncomputable example (M : RigidBodyMotion 3) (t : Time) : ℝ :=
  M.toRigidBody.rotationalKineticEnergy (M.angularVelocity t)
```

6. PKC frames. `rotationalContraction` retains the frame in the type. Lab-frame ω is therefore refused, body-frame ω is accepted, and the bridge theorem equates the accepted components with twice PhysLib's own rotational kinetic energy:

```
-- Frame safety: Refused: type mismatch between lab-frame ω and the body-frame ω rec
/--
error: Application type mismatch: The argument
  ForPhysLib.Exhibits.RigidBody.labOmega M t
has type
  InFrame ForPhysLib.Exhibits.RigidBody.lab Variance.vector ForPhysLib.Exhibits.Rigi
but is expected to have type
  InFrame ForPhysLib.Exhibits.RigidBody.body Variance.vector ForPhysLib.Exhibits.Rig
in the application
  ForPhysLib.Exhibits.RigidBody.rotationalContraction ForPhysLib.Exhibits.RigidBody.
    ForPhysLib.Exhibits.RigidBody.angularVelocity_mul_angularMomentum (ForPhysLib.Ex
    (ForPhysLib.Exhibits.RigidBody.labOmega M t)
-/
#guard_msgs in
noncomputable example (M : RigidBodyMotion 3) (t : Time) :=
  ForPhysLib.Exhibits.RigidBody.rotationalContraction
    ForPhysLib.Exhibits.RigidBody.inertia_mul_angularVelocity
    ForPhysLib.Exhibits.RigidBody.angularVelocity_mul_angularMomentum
    (ForPhysLib.Exhibits.RigidBody.bodyInertia M)
    (ForPhysLib.Exhibits.RigidBody.labOmega M t)

-- Frame safety: Constructs a bearer-free body-frame kineticEnergyK scalar from M's
noncomputable example (M : RigidBodyMotion 3) (t : Time) :
    InFrame ForPhysLib.Exhibits.RigidBody.body .scalar
      ForPhysLib.Exhibits.RigidBody.kineticEnergyK ℝ :=
  ForPhysLib.Exhibits.RigidBody.rotationalContraction
    ForPhysLib.Exhibits.RigidBody.inertia_mul_angularVelocity
    ForPhysLib.Exhibits.RigidBody.angularVelocity_mul_angularMomentum
    (ForPhysLib.Exhibits.RigidBody.bodyInertia M)
    (ForPhysLib.Exhibits.RigidBody.bodyOmega M t)
```

```
-- Frame safety: Proves that body-frame rotational contraction equals PhysLib's rota
example := ForPhysLib.Exhibits.RigidBody.rotationalContraction_eq_physlib
```

7. Homogeneity returns as a special case. For a nonzero-radius solid sphere, the proved inertia tensor is `(2/5) m R²` times the identity matrix:

```
-- Inertia tensor: Proves that the 3D solid sphere's inertia tensor is (2/5) m R² ti
example := ForPhysLib.ClassicalMechanics.SolidSphereInertia.solidSphere_inertiaTenso
```

PhysLib records this target as `informal_lemma CM-RB-sphereI`. The proof in `ForPhysLib.ClassicalMechanics.SolidSphereInertia` is therefore a ready first upstream contribution for the PhysLib contributor. L4 next revisits motion along one line seen by one observer.

5.4.8. AI assistance contract

- Define L3-local `PairId` and `Body`: the preceding lesson modules are independent rather than importing one another.
- Reuse the ForPhysLib findings directly when their upstream bearers already express the claim; restate only the HCP-specific half bearers and the refused assembly probes.
- Preserve nominal identity: same-name records share bearers, while P and Q do not.
- Treat `Assembles` as an explicit physical license, not a generic arithmetic operation.
- Make the weighted-mean license the state's nonzero total mass; do not substitute unit weights.
- Keep the body- and lab-frame indices visible in rotational expressions.
- Label the accepted upstream lab-ω expression physically wrong even though Lean accepts it.
- Leave the one-dimensional, one-observer assumption for L4.

5.4.9. Questions to ask the model so far

Question	Check	What the verdict means
What bears the mass of body A's left half?	`opened_body`	`Composite.part (HalfOf .P .A)` bears it; the body whole bears the assembled total.
Can another record of body A in pair P use the same bearer?	`opened_body`	Yes; it is another state, configuration, or estimate of the named system.
Can pair Q's left-half mass be mixed with pair P's?	`opened_body`	No; nominal pair identities produce distinct bearer types.
How does this state's centre of mass depend on its halves?	`center_of_mass_mean`	`centerOfMassCarving` uses the actual half masses; 3 at 0 and 5 at 10 yield 25/4 rather than 5. Position addition is unlicensed.
What happens when total mass is zero?	`massless_center_of_mass`	PhysLib returns the origin by convention, but hypothesis `h` fails, so the state cannot construct a weighted carving.
Does angular velocity add over a carving?	`angular_velocity_intensive`	No; it is intensive, and the attempted assembly is refused.
Can the body inertia tensor contract with lab-frame ω?	`bare_frame_mismatch`, `framed_rotational_energy`	The bare PhysLib API accepts it, but the PKC frame index refuses it.
Where does the homogeneous-body assumption survive?	`solid_sphere`	As the solid-sphere theorem `(2/5) m R² · 1`, ready to contribute upstream.

5.4.10. System-context card

Scope	System or component	Sort / status	L3 assumptions and attributes
System	Coupled pair P	Nominal `PairId.P`	The L2 coupled pair remains the named system; records under P may describe different states, configurations, or estimates.
Part	Body A	Rigid body in pair P	No longer point-like or homogeneous; carved into left and right halves with per-half mass and position.
Part	Left half of A	`HalfOf .P .A`	Mass-bearing rigid-body part, designated “left half.”
Part	Right half of A	`HalfOf .P .A`	Mass-bearing rigid-body part, designated “right half.”
Attribute	Centre of mass	Weighted mean	Uses each state's half masses and positions; defined only under nonzero-total hypothesis `h`. Masses 3 and 5 at positions 0 and 10 give 25/4. An off-axis value makes the spring force produce torque.
Attribute	Inertia tensor	Body-fixed tensor	The union theorem adds part tensors about a common origin; the solid sphere is the homogeneous special case.
Attribute	Angular velocity	Lab or body frame	Intensive over rigid halves; rotational contraction requires the body-frame value.
Outside / deferred	Observer and full 3-D motion	L4	L3 introduces two frames but still defers the one-observer, one-line assumption to L4.

The HCP bearer is narrower than the upstream `RigidBody` theorems: `HalfOf π b` adds nominal pair-and-body identity. The mass total, state-dependent weighted mean, and refused assembly probes are therefore stated over HCP bearers. The physical union, massless, intensive-ω, frame-bridge, and solid-sphere results are reused directly from ForPhysLib.

5.5. L4 — Out of the line

5.5.1. Synopsis

5.5.1.1. Purpose

Move pair P into three-dimensional space, seen from two reference frames: the lab, and a cart moving at constant velocity relative to it. Positions, displacements and distances get distinct ISO 80000-3 kinds, so adding a position to a displacement fails to type-check. The A–B separation is an `IndividualQuantity` about the pair it belongs to, so pair P's separation cannot be used where pair Q's is expected. Each

frame conversion accepts one kind of vector: `kGeometricTransport` converts displacements and `kBoostTransport` converts velocities, so passing a displacement to `kBoostTransport` fails to type-check.

5.5.1.2. PKC vocabulary in scope

New at this step:

- `positionVector`, `distance`, `area` — ISO 80000-3 space kinds (with `displacement`, carried from L2: the spring extension is the same kind).
- `positionQ`, `displacementQ`, `distanceQ`, `lengthOf` — read PhysLib's `Space` values at their kinds.
- `KVector` — a frame vector with its physical kind stated.
- `kGeometricTransport`, `kBoostTransport` — frame conversions, each typed for one kind of vector.
- `velocityK`, `forceK`, `durationK` — kinds that share PhysLib's one vector type.
- `DifferenceKind` — a kind-level difference law.

Carried from earlier steps: `Quantity`, `IndividualQuantity`, `DifferenceKind.ofScale`, `SortOfSystem`, `Sorted`, `Designated`.

5.5.1.3. Verifiable properties

- Adding a position to a displacement fails to type-check, and so does adding a distance to the length of a displacement: there is no `HAdd` instance for either pair of kinds (refused) `HCP.L4_OutOfTheLine#points_vectors_and_distances`.
- Pair P's A–B separation is an `IndividualQuantity` of kind distance about P, and pair Q's separation supplied where P's is expected fails with a type mismatch (refused) `HCP.L4_OutOfTheLine#whose_distance`.
- The distance between two points equals the length of the displacement between them (theorem) `HCP.L4_OutOfTheLine#distance_is_length`.
- `kBoostTransport` rejects a displacement with a type mismatch, and adding a velocity to a force fails to type-check (refused) `HCP.L4_OutOfTheLine#kinded_frames`.
- Converting P's displacement from frame F to frame G with `kGeometricTransport` gives the displacement that G computes from its own coordinates (theorem) `HCP.L4_OutOfTheLine#kinded_frames`.
- The displacement of the cart's origin between times t_1 and t_2 equals the elapsed time times the cart's velocity (theorem) `HCP.L4_OutOfTheLine#observer_motion`.

5.5.1.4. Errors prevented

- Bearer erased: pair P's A–B separation used where pair Q's is meant (`HCP.L4_OutOfTheLine#whose_distance`).
- Observer left implicit: with PhysLib's single vector type, the geometric, boost and origin-shift conversions all accept the same vector; with kinded vectors, each conversion accepts only the kind it is correct for (`HCP.L4_OutOfTheLine#bare_frames`, `HCP.L4_OutOfTheLine#kinded_frames`).
- Point/vector confusion: a position added to a displacement, or a distance added to the length of a displacement (`HCP.L4_OutOfTheLine#points_vectors_and_distances`).

5.5.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean and PhysLib and trust `Dimension` and `WithDim`; L0–L3 have established kinds, bearers, composition, and body- and lab-frame angular velocity.

5.5.3. Pull

The wobble throws body A's centre of mass off the line, and a second observer on a moving cart sees the pair differently. What must the model now say about where things are, and for whom? L4 moves the two centres of mass into `Space 3`, distinguishes points from vectors and distances, and states the observer whose frame supplies a vector's context. One name is one system: records under it are states at different times, configurations, or rival estimates; comparing them is the point. Distinct names are distinct systems. Pair P's separation has one bearer across its states, while pair Q has a different bearer. The lab frame F and cart frame G are explicit observers.

5.5.4. Risks at this step

- **Bearer erased.** `distanceQ` cannot distinguish the A–B separation of pair P from that of pair Q; the typed cross-pair refusal is pinned at `HCP.L4_OutOfTheLine#whose_distance`.
- **Observer left implicit.** Bare vectors accept geometric, boost, and origin-shift transport even though only one law is right for each vector kind; `HCP.L4_OutOfTheLine#bare_frames` shows the wrong accepted forms and `HCP.L4_OutOfTheLine#kinded_frames` pins the role-specific refusals.

Further reading

- `Exhibits/ReferenceFrame`, MOTIVATION M7.

5.5.5. Small rigorous improvement

Question	Context change	Attribute change	Risk shown	PhysLib stage
The wobble throws body A's centre of mass off the line, and a second observer on a moving cart sees the pair differently. What must the model now say about where things are, and for whom?	The pair's centres become points in `Space 3`; lab F and cart G are named frames.	Position vector, displacement, pair distance, and velocity per frame.	Pair P's distance can be confused with pair Q's, and bare frame vectors accept transport laws that are wrong for the vector's role.	Space Stages 0–4.

The remaining one-dimensional, one-observer assumption is gone. The no-dissipation assumption remains for L5.

5.5.6. Learning path

#	Do	Checked by `HCP.L4_OutOfTheLine` anchor	Concept disclosed
1	Put the two centres in a `PairState3`; read position, displacement, and distance, then try the two kind collisions.	`points_vectors_and_distances`	Position and displacement are distinct vector kinds; distance and the length of displacement are distinct scalar kinds, even where magnitudes agree. HCP's body and separation bearers retain nominal identity.
2	Attach the distance to the named pair, compare two states, and mix P with Q.	`whose_distance`	`distance` admits `DifferenceKind.ofScale`; same-name states have a typed change whose magnitude reads their own centres, while Q's separation cannot be added to P's.
3	Relate distance to displacement length and apply the triangle inequality to state points.	`distance_is_length`	`distanceQ_eq_lengthOf_displacementQ` and `distanceQ_triangle` operate directly on the pair state's centres.
4	Feed one state-derived bare frame vector to all three upstream transports and add the cart's origin velocity.	`bare_frames`	Both terms compile, but are physically wrong when the vector's role is displacement: bare `F.Vector` cannot select a transport law or distinguish displacement from velocity.
5	Use `KVector` for displacement and velocity; try the wrong boost and velocity-plus-force.	`kinded_frames`	Frame components enter through each frame's own `dispEquiv`; the observer-independence theorem verifies geometric transport. Boost-on-displacement and velocity-plus-force are pinned refusals. Average velocity is computed from two same-pair states and an elapsed duration.
6	State the cart-origin	`observer_motion`	`origin_displacement_eq` makes observer motion an attribute of the cart frame; the

#	Do	Checked by `HCP.L4_OutOfTheLine` anchor	Concept disclosed
	displacement law and try adding its frame velocity to the pair displacement.		kinded displacement-plus-velocity sum is refused.
7	Map Space Stages 0–4 and exercise Stage 3's operator table.	`adoption_ladder`	`spanArea` and its magnitude law are accepted on a displacement from the state and a probe point; area × area is refused. Stage 4's boundary and API-map audits are the final rung.

Worksheet: `HCP/Worksheets/L4_OutOfTheLine.lean` · Solution: `HCP/L4_OutOfTheLine.lean`

5.5.7. Lesson

1. Points, vectors, and distances. `PairState3 π` stores this state's centre-of-mass points. `BodyOf π` keeps a body's nominal pair identity, while `SeparationOf π` names the pair's A–B separation. The upstream `positionQ`, `displacementQ`, and `distanceQ` are applied to the state fields. Position plus displacement and distance plus the length of that displacement are both pinned refusals:

```
/--
One pair name is one system. Records under that name are its states, configurations,
rival estimates; comparing them is the point. Distinct names are distinct systems.
-/
inductive PairId | P | Q
deriving DecidableEq, Repr

inductive Body | A | B
deriving DecidableEq, Repr

structure PairState3 (π : PairId) where
  com : Body → Space 3

structure BodyOf (π : PairId) where
  body : Body
deriving DecidableEq, Repr

structure SeparationOf (π : PairId) where
```

```
deriving DecidableEq, Repr

def bodyS : SortOfSystem := ⟨"body centre"⟩
def separationS : SortOfSystem := ⟨"pair separation"⟩

instance {π : PairId} : Sorted (BodyOf π) := ⟨fun _ => bodyS⟩

instance {π : PairId} : Designated (BodyOf π) where
  designation
    | ⟨.A⟩ => ⟨"body A"⟩
    | ⟨.B⟩ => ⟨"body B"⟩
  designation_inj {x y} h := by
    rcases x with ⟨x⟩
    rcases y with ⟨y⟩
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

instance {π : PairId} : Sorted (SeparationOf π) := ⟨fun _ => separationS⟩

instance {π : PairId} : Designated (SeparationOf π) where
  designation _ := ⟨"A–B separation"⟩
  designation_inj {x y} _ := by cases x; cases y; rfl

def bodyPositionOf {π : PairId} (s : PairState3 π) (b : BodyOf π) :
    IndividualQuantity b ForPhysLib.Kinds.Space.positionVector (EuclideanSpace ℝ (Fi
  ⟨(ForPhysLib.Kinded.Space.positionQ (s.com b.body)).magnitude⟩

def pairDisplacementOf {π : PairId} (s : PairState3 π) :
    IndividualQuantity (⟨⟩ : SeparationOf π) ForPhysLib.Kinds.Space.displacement
      (EuclideanSpace ℝ (Fin 3)) :=
  ⟨(ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B)).magnitude⟩

-- Point vs vector: Constructs body A's location as a bearer-free `Quantity position
example (s : PairState3 .P) :
    Quantity ForPhysLib.Kinds.Space.positionVector (EuclideanSpace ℝ (Fin 3)) :=
  ForPhysLib.Kinded.Space.positionQ (s.com .A)

-- Point vs vector: Constructs pair P's A-to-B difference as a bearer-free `Quantity
example (s : PairState3 .P) :
    Quantity ForPhysLib.Kinds.Space.displacement (EuclideanSpace ℝ (Fin 3)) :=
  ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B)
```

```
-- Point vs vector: Constructs pair P's scalar separation as a bearer-free `Quantity
example (s : PairState3 .P) : Quantity ForPhysLib.Kinds.Space.distance ℝ :=
  ForPhysLib.Kinded.Space.distanceQ (s.com .A) (s.com .B)

-- Point vs vector: Refused: no HAdd instance combines positionVector and displaceme
/--
error: failed to synthesize instance of type class
  HAdd (Quantity positionVector (EuclideanSpace ℝ (Fin 3))) (Quantity displacement (

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs (whitespace := lax) in
example (s : PairState3 .P) :=
  ForPhysLib.Kinded.Space.positionQ (s.com .A) +
    ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B)

-- Kind vs dimension: Refused: no HAdd instance combines distance and length, prever
/--
error: failed to synthesize instance of type class
  HAdd (Quantity distance ℝ) (Quantity length ℝ) ?m.12

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs (whitespace := lax) in
example (s : PairState3 .P) :=
  ForPhysLib.Kinded.Space.distanceQ (s.com .A) (s.com .B) +
    ForPhysLib.Kinded.Space.lengthOf
      (ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B))
```

2. Whose distance. `distanceQ` returns a `Quantity` and so erases the HCP pair bearer. `separationOf` puts that same distance on `SeparationOf π`. Distance is a ratio-scale length species, so `DifferenceKind.ofScale` applies: two records under P can express a change in separation, while Q's pair bearer does not unify with P's.

```
def separationOf {π : PairId} (s : PairState3 π) :
    IndividualQuantity (⟨⟩ : SeparationOf π) ForPhysLib.Kinds.Space.distance ℝ :=
  ⟨(ForPhysLib.Kinded.Space.distanceQ (s.com .A) (s.com .B)).magnitude⟩

/-- Bearer safety: Proves that pair P's indexed separation magnitude is the distance
lemma separationOf_magnitude {π : PairId} (s : PairState3 π) :
    (separationOf s).magnitude = dist (s.com .A) (s.com .B) := rfl
```

```
-- Distance is ratio-scale in ISO 80000-3, so its difference is licensed.
-- Composition law: Constructs a DifferenceKind for distance from its ratio scale, l
example : DifferenceKind ForPhysLib.Kinds.Space.distance := DifferenceKind.ofScale

def separationChange (earlier later : PairState3 .P) :
    IndividualQuantity (⟨⟩ : SeparationOf .P) ForPhysLib.Kinds.Space.distance ℝ :=
  IndividualQuantity.add DifferenceKind.ofScale
    (separationOf later) (-(separationOf earlier))

/-- Same-name records: Proves that pair P's separation-change magnitude is later A–B
lemma separationChange_magnitude (earlier later : PairState3 .P) :
    (separationChange earlier later).magnitude =
      dist (later.com .A) (later.com .B) - dist (earlier.com .A) (earlier.com .B) :=
  simp [separationChange, IndividualQuantity.add_magnitude,
    IndividualQuantity.neg_magnitude, separationOf]
  change dist (later.com .A) (later.com .B) +
      (-(dist (earlier.com .A) (earlier.com .B))) =
    dist (later.com .A) (later.com .B) - dist (earlier.com .A) (earlier.com .B)
  ring

-- Bearer safety: Refused: type mismatch between pair P's and pair Q's SeparationOf
/--
error: Application type mismatch: The argument
  qDistance
has type
  IndividualQuantity (@SeparationOf.mk PairId.Q) distance ℝ
but is expected to have type
  IndividualQuantity (@SeparationOf.mk PairId.P) distance ℝ
in the application
  IndividualQuantity.add … (separationOf p) qDistance
-/
#guard_msgs (whitespace := lax) in
example (p : PairState3 .P) (q : PairState3 .Q) :=
  let qDistance : IndividualQuantity (⟨⟩ : SeparationOf .Q)
      ForPhysLib.Kinds.Space.distance ℝ := separationOf q
  IndividualQuantity.add DifferenceKind.ofScale (separationOf p) qDistance
```

3. Distance is displacement length. These laws are applied to the state's own A and B centres. The magnitudes agree, but the kinds remain distinct; the triangle law is another statement over the points carried by one `PairState3`.

```
-- Point vs vector: Proves that pair P's A–B distance equals the length of its displ
example (s : PairState3 .P) :
    (ForPhysLib.Kinded.Space.distanceQ (s.com .A) (s.com .B)).magnitude =
      (ForPhysLib.Kinded.Space.lengthOf
        (ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B))).magnitude :=
  ForPhysLib.Kinded.Space.distanceQ_eq_lengthOf_displacementQ (s.com .A) (s.com .B)

-- Point vs vector: Proves that pair P's triangle inequality is distance(A,A) ≤ dist
example (s : PairState3 .P) :
    (ForPhysLib.Kinded.Space.distanceQ (s.com .A) (s.com .A)).magnitude ≤
      (ForPhysLib.Kinded.Space.distanceQ (s.com .A) (s.com .B)).magnitude +
        (ForPhysLib.Kinded.Space.distanceQ (s.com .B) (s.com .A)).magnitude :=
  ForPhysLib.Kinded.Space.distanceQ_triangle (s.com .A) (s.com .B) (s.com .A)
```

4. Bare frames. The vector fed to the three upstream laws is derived from this state's two centres. All three transports accept it because `F.Vector` records a frame but not a physical role. The displacement-plus-origin-velocity sum also elaborates. Both are accepted but physically wrong when that vector is meant to be a displacement.

```
def barePairDisplacement {π : PairId} (s : PairState3 π)
    (F : ReferenceFrame 3) (t : Time) : F.Vector :=
  (ReferenceFrame.Vector.dispEquiv (frame := F) (F.timeEquiv.symm t)).symm
    (s.com .A -ᵥ s.com .B)

-- Frame safety: Bare vectors erase physical frame and kind, so all three transports
example (s : PairState3 .P) (F G : ReferenceFrame 3) (t : Time)
    (u : EuclideanSpace ℝ (Fin 3)) :
    G.Vector × G.Vector × G.Vector :=
  let v := barePairDisplacement s F t
  (geometricTransport F G t v, boostTransport F G t u v, originShiftTransport F G t

-- Frame safety: Bare EuclideanSpace vectors accept adding pair P's displacement to
example (s : PairState3 .P) (cart : ReferenceFrame 3) (hCart : cart.IsInertial) :
    EuclideanSpace ℝ (Fin 3) :=
  (s.com .A -ᵥ s.com .B) + hCart.velocity
```

5. Kinded frames. Frame components must enter through the frame's own `dispEquiv`. With `KVector`, the displacement goes through geometric transport, and an average velocity computed from two same-pair centre states goes through the boost law. The observer-independence theorem shows that geometric transport preserves the displacement. The wrong boost on a displacement and a velocity-plus-force sum are pinned refusals.

```
/- Frame components must enter through the frame's own `dispEquiv`. -/
def pairDisplacementInFrame {π : PairId} (F : ReferenceFrame 3) (t : Time)
    (s : PairState3 π) :
    KVector F ForPhysLib.Kinds.Space.displacement :=
  ⟨⟨(barePairDisplacement s F t).components⟩⟩

def averageVelocityOfA {π : PairId} (earlier later : PairState3 π)
    (elapsed : Quantity durationK ℝ) (h : elapsed.magnitude ≠ 0) :
    Quantity velocityK (EuclideanSpace ℝ (Fin 3)) :=
  ⟨(if hzero : elapsed.magnitude = 0 then False.elim (h hzero)
    else elapsed.magnitude⁻¹) • (later.com .A -ᵥ earlier.com .A)⟩

def averageVelocityInFrame {π : PairId} (F : ReferenceFrame 3) (t : Time)
    (earlier later : PairState3 π) (elapsed : Quantity durationK ℝ)
    (h : elapsed.magnitude ≠ 0) : KVector F velocityK :=
  ⟨⟨((ReferenceFrame.Vector.dispEquiv (frame := F) (F.timeEquiv.symm t)).symm
    (averageVelocityOfA earlier later elapsed h).magnitude).components⟩⟩

-- Frame safety: Constructs a bearer-free `KVector G displacement` for pair P via ge
example (s : PairState3 .P) (F G : ReferenceFrame 3) (t : Time) :
    KVector G ForPhysLib.Kinds.Space.displacement :=
  kGeometricTransport F G t (pairDisplacementInFrame F t s)

/-- Frame safety: Proves that geometric transport of pair P's displacement from F to
theorem pairDisplacement_observer_independent {π : PairId}
    (F G : ReferenceFrame 3) (t : Time) (s : PairState3 π) :
    kGeometricTransport F G t (pairDisplacementInFrame F t s) =
      pairDisplacementInFrame G t s := by
  have hη : (⟨(barePairDisplacement s F t).components⟩ : F.Vector) =
      barePairDisplacement s F t := rfl
  simp only [kGeometricTransport, pairDisplacementInFrame]
  rw [hη]
  simp [barePairDisplacement]

-- Frame safety: Constructs a bearer-free `KVector G velocityK` for body A's average
example (earlier later : PairState3 .P) (elapsed : Quantity durationK ℝ)
    (h : elapsed.magnitude ≠ 0) (F G : ReferenceFrame 3) (t : Time)
    (u : Quantity velocityK (EuclideanSpace ℝ (Fin 3))) :
    KVector G velocityK :=
  kBoostTransport F G t u (averageVelocityInFrame F t earlier later elapsed h)
```

```
-- Frame safety: Refused: type mismatch between pair P's KVector displacement and th
/--
error: Application type mismatch: The argument
  pairDisplacementInFrame F t s
has type
  KVector F displacement
but is expected to have type
  KVector F velocityK
in the application
  kBoostTransport F G t u (pairDisplacementInFrame F t s)
-/
#guard_msgs (whitespace := lax) in
example (F G : ReferenceFrame 3) (t : Time)
    (u : Quantity velocityK (EuclideanSpace ℝ (Fin 3))) (s : PairState3 .P) :=
  kBoostTransport F G t u (pairDisplacementInFrame F t s)

-- Kind vs dimension: Refused: no HAdd instance combines frame-F velocity and force
/--
error: failed to synthesize instance of type class
  HAdd (KVector F velocityK) (KVector F forceK) ?m.9

Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs (whitespace := lax) in
example (F : ReferenceFrame 3) (v : KVector F velocityK) (force : KVector F forceK)
  v + force
```

6. The observer moves. `origin_displacement_eq` states the cart frame's own origin motion over elapsed time. The pair's state-derived displacement still cannot be added to that frame's kinded velocity.

```
-- Frame safety: Proves that the inertial cart's origin-displacement magnitude equal
example (cart : ReferenceFrame 3) (hCart : cart.IsInertial) (t₁ t₂ : Time) :
    (ForPhysLib.Kinded.Space.displacementQ (cart.origin t₂) (cart.origin t₁)).magnit
      (elapsedQ t₁ t₂).magnitude • (frameVelocityQ hCart).magnitude :=
  origin_displacement_eq cart hCart t₁ t₂

-- Kind vs dimension: Refused: no HAdd instance combines displacement and velocity c
/--
error: failed to synthesize instance of type class
  HAdd (Quantity displacement (EuclideanSpace ℝ (Fin 3))) (Quantity velocityK (Eucli
```

```
Hint: Type class instance resolution failures can be inspected with the `set_option
-/
#guard_msgs (whitespace := lax) in
example (s : PairState3 .P) (cart : ReferenceFrame 3) (hCart : cart.IsInertial) :=
  ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B) + frameVelocityQ hCart
```

7. Adoption ladder. The five Space stages go from Kinds, through Metrology, Kinded and Operators, to Audits. Stage 3's operator-built area is computed from this state's A–B displacement and a probe-point displacement; its magnitude law is accepted, while area × area is refused. Stage 4 audits the boundary and anchors the API map.

```
def pairSpanArea {π : PairId} (s : PairState3 π) (p : Space 3) :
    Quantity ForPhysLib.Kinds.Space.area ℝ :=
  ForPhysLib.Operators.Space.spanArea
    (ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B))
    (ForPhysLib.Kinded.Space.displacementQ (s.com .A) p)

/-- Point vs vector: Proves that pair P's span-area magnitude from A–B and A–p displ
lemma pairSpanArea_magnitude {π : PairId} (s : PairState3 π) (p : Space 3) :
    (pairSpanArea s p).magnitude =
      ‖(ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B)).magnitude‖ *
        ‖(ForPhysLib.Kinded.Space.displacementQ (s.com .A) p).magnitude‖ :=
  ForPhysLib.Operators.Space.spanArea_magnitude _ _

-- Point vs vector: Constructs a bearer-free `Quantity area` from pair P's A–B and A
example (s : PairState3 .P) (p : Space 3) : Quantity ForPhysLib.Kinds.Space.area ℝ :
  pairSpanArea s p

-- Point vs vector: Proves that pair P's span-area magnitude equals the product of i
example (s : PairState3 .P) (p : Space 3) :
    (pairSpanArea s p).magnitude =
      ‖(ForPhysLib.Kinded.Space.displacementQ (s.com .A) (s.com .B)).magnitude‖ *
        ‖(ForPhysLib.Kinded.Space.displacementQ (s.com .A) p).magnitude‖ :=
  pairSpanArea_magnitude s p

-- Kind vs dimension: Refused: no HMul instance licenses area times area, preventing
/--
error: failed to synthesize instance of type class
  HMul (Quantity area ℝ) (Quantity area ℝ) ?m.9

Hint: Type class instance resolution failures can be inspected with the `set_option
```

```
-/
#guard_msgs (whitespace := lax) in
example (s : PairState3 .P) (p : Space 3) := pairSpanArea s p * pairSpanArea s p
```

The Space adoption files, in order, are `ForPhysLib/Kinds/Space.lean`, `ForPhysLib/Metrology/Space.lean`, `ForPhysLib/Kinded/Space.lean`, `ForPhysLib/Operators/Space.lean`, and `ForPhysLib/Audits/Space.lean`. The final audit stage is wired to `Space/API-map.md` by `Space.checked_by.yaml`.

5.5.7.1. Space adoption ladder

Stage	File	What it contributes
0	`ForPhysLib/Kinds/Space.lean`	Kind vocabulary for length, distance, position vector, displacement, and area.
1	`ForPhysLib/Metrology/Space.lean`	Dimension pairings and the kind algebra.
2	`ForPhysLib/Kinded/Space.lean`	`positionQ`, `displacementQ`, `distanceQ`, and `lengthOf`.
3	`ForPhysLib/Operators/Space.lean`	The curated operator table: `spanArea` is accepted, area × area is not.
4	`ForPhysLib/Audits/Space.lean` and `Space.checked_by.yaml`	Boundary and API-map audits under build checks.

The Stage-3 accepted form is `spanArea v w : Quantity area ℝ`, with magnitude ‖`v.magnitude`‖ * ‖`w.magnitude`‖; the product of two areas has no registration. Stage 4 audits the authored boundary and ties the API map to its Lean declarations.

5.5.8. AI assistance contract

- Keep L4 independent: define local `PairId | P | Q` and `Body | A | B`.
- Preserve nominal identity: one name is one system; records under it are states, configurations, or rival estimates; distinct names are distinct systems.
- Use the exact `PairState3.com` fields for all accepted position, displacement, distance, velocity, and area forms.
- Keep the `distance` bearer pair-indexed in HCP; do not alter ForPhysLib's `distanceQ`.
- Reuse the Space and ReferenceFrame APIs, and pin each requested kind refusal.
- Label the accepted unkinded frame examples wrong; they do not encode physical role.
- Recap, rather than rewrite, Space Stages 0–4 and point to Stage 4 audits.
- Leave the no-dissipation assumption for L5.

5.5.9. Questions to ask the model so far

Question	Check	What the verdict means
Where is body A in this state?	`points_vectors_and_distances`	`positionQ` reads the state's `com .A` point from the conventional origin, and `BodyOf π` retains the body's nominal pair identity.
Can position be added to displacement?	`points_vectors_and_distances`	No; the Space kind layer refuses the distinct vector roles.
Is distance interchangeable with displacement length?	`points_vectors_and_distances`, `distance_is_length`	No; their scalar magnitudes are related by a theorem, not kind identity.
Whose separation is this?	`whose_distance`	`SeparationOf π` retains the pair name erased by the upstream `distanceQ`.
Can the separation change between P states?	`whose_distance`	Yes; distance is ratio-scale and the difference magnitude is the later distance minus the earlier one, computed from each state's centres.
Can Q's distance be mixed with P's?	`whose_distance`	No; distinct pair identities yield distinct separation bearers.
Which frame law applies to a displacement?	`bare_frames`, `kinded_frames`	Bare `F.Vector` accepts all three; `KVector` allows geometric transport, proves observer independence, and refuses the boost law.
Where does the observer's velocity live?	`observer_motion`	On the cart frame context, and not as a summand of the pair displacement.
What does the adoption ladder contribute?	`adoption_ladder`	Stage 3 curates operators; Stage 4 audits the Space boundary and API map.

5.5.10. System-context card

Scope	System or component	Sort / status	L4 assumptions and attributes
System	Coupled pair P or Q	Nominal `PairId`	One name denotes one system; records under it can be different states, configurations, or rival estimates. P and Q are distinct systems.
State	Pair state	`PairState3 π`	Stores the centre-of-mass point of each body in `Space 3`.
Part	Body A or B	`BodyOf π`	Its position vector is read from the point in this state.
Attribute	A–B separation	`SeparationOf π`	`distance` in $\mathbb{R}$; its change is comparable between states of one pair.
Attribute	Pair displacement	`SeparationOf π` bearer	A vector from body B's centre to body A's centre, kind `displacement`.
Observer	Lab F or cart G	`ReferenceFrame 3`	The frame is part of the context; the cart's origin velocity and motion law are explicit.
Deferred assumption	No dissipation	L5	Not changed by this spatial and observer refinement.

5.6. L5 — The oscillation dies out

5.6.1. Synopsis

5.6.1.1. Purpose

Optional. Add a viscous damper to oscillator A, so that its energy is no longer conserved. Three quantities are easy to confuse here: PhysLib's damping constant γ (kg/s); ISO 80000-3's damping coefficient δ = γ/(2m) (s^{-1}), which PhysLib calls the decay rate; and the angular frequency ω_0 (rad/s), which has the same dimension as δ. The lesson gives each its own kind, so adding δ to ω_0 fails to type-check. `decayRateOf` computes δ from one oscillator's γ and mass with `IndividualQuantity.div`, so oscillator B's mass cannot be used for oscillator A's δ.

5.6.1.2. PKC vocabulary in scope

New at this step:

- `KindOfProperty` (`id`, `scale`) — define a kind of property; the lesson defines its own kind for γ.

- `ScaleType.ratio` — the scale type that licenses × and ÷.
- `dampingCoefficient` — ISO 80000-3 damping coefficient δ, dimension T^{-1}.
- `KindDiv`, `QuotientKind.ofRatio` — a curated quotient law, here δ = γ/(2m).
- `Quantity.add` — kind- and scale-gated addition.
- `IndividualQuantity.toQuantity` — forget the bearer.
- `angularFrequencyK` — the oscillator's ω.

Carried from earlier steps: DimensionedKind, `IndividualQuantity` and its `add`/`div`, `DifferenceKind.ofScale`, `SortOfSystem`, `Sorted`, `Designated`, `massK`, `springConstantK`.

5.6.1.3. Verifiable properties

- The damped oscillator's energy decreases at the rate stated by PhysLib's energy-dissipation theorem (theorem) `HCP.L5_Damping#energy_dissipation`.
- `decayRateOf` divides A's γ by A's 2m, and its magnitude equals PhysLib's `decayRate`; using B's mass instead fails with a type mismatch (refused) `HCP.L5_Damping#kinds_and_decay_rate`.
- δ and ω_0 have the same dimension, T^{-1}, and adding them fails to type-check (refused) `HCP.L5_Damping#same_dimension_different_kinds`.
- PhysLib's `IsUnderdamped` is equivalent to $\delta < \omega_0$. Comparing the two as kinded quantities fails with a type mismatch, because their kinds differ; `IsUnderdamped` implies that the magnitude of δ is less than the magnitude of ω_0 (theorem) `HCP.L5_Damping#underdamped_regime`.
- Subtracting an earlier record of A's δ from a later one type-checks as an `IndividualQuantity` about A of the damping-coefficient kind, whose magnitude is the difference of the two decay rates; subtracting B's δ from A's fails with a type mismatch (type-checked) `HCP.L5_Damping#same_oscillator_states`.

5.6.1.4. Errors prevented

- Similar names, different kinds: PhysLib's damping constant γ (kg/s) used as ISO's damping coefficient δ (s^{-1}) (`HCP.L5_Damping#kinds_and_decay_rate`).
- Same dimension, different kind: δ + ω_0, which dimensions alone accept (`HCP.L5_Damping#same_dimension_different_kinds`).
- Cross-system contamination: oscillator B's mass used in oscillator A's δ (`HCP.L5_Damping#kinds_and_decay_rate`).

5.6.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean, PhysLib, `Dimension`, and `WithDim`. L0–L4 modeled an ideal, conservative oscillator; this optional rung adds the damping that makes its energy decay.

5.6.3. Pull

The oscillator loses energy as its motion fades. This optional rung adds a damper to the nominally named whole-system model. Do not confuse the damper's viscous constant γ with the whole oscillator's decay rate δ. One name is one system: records under that name are its states at different times, configurations, or rival estimates; comparing them is the point. Distinct names are distinct systems.

5.6.4. Risks at this step

- **Similar symbols and dimensions.** Bare reals accept $\gamma < \omega_0$ although the damper's viscous constant γ and angular frequency ω_0 are different kinds; dimension-only types accept $\delta + \omega_0$ because both have dimension T^{-1}. The kinded checks pin the distinction at `HCP.L5_Damping#kinds_and_decay_rate` and `HCP.L5_Damping#same_dimension_different_kinds`.
- **Cross-system contamination.** Oscillator B's mass cannot be used to compute oscillator A's decay rate, and states of A and B cannot be mixed; see `HCP.L5_Damping#kinds_and_decay_rate` and `HCP.L5_Damping#same_oscillator_states`.

5.6.5. Small rigorous improvement

Question	Context change	Attribute change	Risk shown	PhysLib source
The oscillator loses energy. What must the model add, and how should it distinguish the damper's measurement from the whole oscillator's decay?	Add a damper to the oscillator and represent the whole as a nominally named `DampedRig`.	γ is measured on the damper and `m` on the body; both retain the whole oscillator's bearer. ISO damping coefficient δ and the damping regime belong to the whole.	Bare $\gamma < \omega$ and $\delta + \omega_0$ compile despite wrong or distinct kinds; cross-system division and mixing A's and B's states are refused.	`DampedHarmonicOscillator.Basic`; ISO 80000-3 items 3-18, 3-24, and 3-25.

L0–L4's conservation assumption ends here. The PhysLib dissipation law says that the energy derivative is negative when the damper is active and the velocity is nonzero. This rung is optional; the requirements lesson remains L6.

5.6.6. Learning path

#	Do	Checked by `HCP.L5_Damping` anchor	Concept disclosed
1	Apply the upstream energy laws to the state stored in `DampedRig`.	`energy_dissipation`	The HCP wrapper does not restate the physics: it supplies its own `r.osc` to PhysLib's `energy_dissipation_rate` and `energy_not_conserved`.
2	Read `m` on the body and `γ` on the damper; form their typed decay-rate quotient on the whole oscillator.	`kinds_and_decay_rate`	Both readings retain bearer `o`; `m + m` expresses `2m` without dropping it. The local `KindDiv` licenses `γ/(m + m)`, and A's `γ` cannot be divided by B's mass. Bare `γ < ω` is accepted but wrong.
3	Compare `δ` with `ω₀` first as reals, then as kinds.	`same_dimension_different_kinds`	Both are `T⁻¹`, but damping coefficient and angular frequency are distinct kinds. The bare sum is accepted; kinded addition is refused.
4	State the underdamped criterion and make its erasure to real magnitudes visible.	`underdamped_regime`	PhysLib proves the decay-rate inequality; the kinded comparison is refused. The accepted magnitude comparison exposes the rad = 1 convention.
5	Compare earlier and later states of oscillator A; try to mix oscillator B.	`same_oscillator_states`	A typed difference measures A's change in decay rate; distinct nominal names cannot be combined.

Worksheet: `HCP/Worksheets/L5_Damping.lean` · Solution: `HCP/L5_Damping.lean`

5.6.7. Lesson

```
/--
One name is one system; records under that name are its states at different times,
configurations, or rival estimates of it—comparing them is the point. Distinct names
are distinct systems.
```

```
-/
inductive Osc | A | B
deriving DecidableEq, Repr

structure DampedRig (o : Osc) where
  osc : _root_.ClassicalMechanics.DampedHarmonicOscillator

inductive DampedPart | Body | Spring | Damper
deriving DecidableEq, Repr

structure PartOf (o : Osc) where
  part : DampedPart
deriving DecidableEq, Repr

def dampedPartS : SortOfSystem := ⟨"damped oscillator component"⟩

instance {o : Osc} : Sorted (PartOf o) := ⟨fun _ => dampedPartS⟩

instance {o : Osc} : Designated (PartOf o) where
  designation
    | ⟨.Body⟩ => ⟨"body"⟩
    | ⟨.Spring⟩ => ⟨"spring"⟩
    | ⟨.Damper⟩ => ⟨"damper"⟩
  designation_inj {x y} h := by
    rcases x with ⟨x⟩
    rcases y with ⟨y⟩
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

def bodyPart (o : Osc) : PartOf o := ⟨.Body⟩
def springPart (o : Osc) : PartOf o := ⟨.Spring⟩
def damperPart (o : Osc) : PartOf o := ⟨.Damper⟩

def massOf {o : Osc} (r : DampedRig o) :
    IndividualQuantity o massK ℝ :=
  ⟨r.osc.m⟩

def springConstantOf {o : Osc} (r : DampedRig o) :
    IndividualQuantity (springPart o) springConstantK ℝ :=
  ⟨r.osc.k⟩
```

1. Energy is no longer conserved. The HCP state wraps PhysLib's damped oscillator, so the upstream dissipation results apply to `r.osc`. L0–L4's conservation assumption ends here:

```
/-- Dissipation: Proves that under oscillator O's damped equation, the energy deriva
lemma energy_dissipation_on_state {o : Osc} (r : DampedRig o)
    (xₜ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time)
    (h1 : r.osc.EquationOfMotion xₜ) (hx : ContDiff ℝ ∞ xₜ) :
    ∂ₜ (r.osc.energy xₜ) t =
      - r.osc.γ * inner ℝ (∂ₜ xₜ t) (∂ₜ xₜ t) :=
  r.osc.energy_dissipation_rate xₜ t h1 hx

/-- Dissipation: Proves that positive γ and nonzero velocity make oscillator O's ene
lemma energy_not_conserved_on_state {o : Osc} (r : DampedRig o)
    (xₜ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time)
    (h1 : r.osc.EquationOfMotion xₜ) (hx : ContDiff ℝ ∞ xₜ)
    (hdx : ∂ₜ xₜ t ≠ 0) (hγ : 0 < r.osc.γ) :
    ∂ₜ (r.osc.energy xₜ) t < 0 :=
  r.osc.energy_not_conserved xₜ t h1 hx hdx hγ
```

2. One name, two kinds. `γ` is the viscous damping constant read on the damper (kg/s), and `m` is measured on the body. Both quantities retain the whole oscillator's bearer `o`. ISO 3-24's damping coefficient `δ` (s^{-1}) is instead PhysLib's `decayRate`, attached to the whole. The local quotient law makes the typed `γ/(m + m)` formula possible:

```
def viscousDampingConstantK : KindOfProperty :=
  { id := "viscous damping constant", scale := .ratio }

def dampingCoefficientK : KindOfProperty :=
  Iso80000.Part3.dampingCoefficient.kind

def dampingConstantOf {o : Osc} (r : DampedRig o) :
    IndividualQuantity o viscousDampingConstantK ℝ :=
  ⟨r.osc.γ⟩

instance : KindDiv viscousDampingConstantK massK dampingCoefficientK :=
  ⟨QuotientKind.ofRatio _ _ _⟩

def decayRateOf {o : Osc} (r : DampedRig o) :
    IndividualQuantity o dampingCoefficientK ℝ :=
  -- Write 2m as m + m to keep the mass on the oscillator bearer.
  IndividualQuantity.div
    (KindDiv.law (k₁ := viscousDampingConstantK) (k₂ := massK)
```

```
      (k := dampingCoefficientK))
    (dampingConstantOf r)
    (IndividualQuantity.add DifferenceKind.ofScale (massOf r) (massOf r))

/-- Kind vs dimension: Proves that oscillator O's indexed damping-coefficient magnit
lemma decayRateOf_magnitude {o : Osc} (r : DampedRig o) :
    (decayRateOf r).magnitude = r.osc.decayRate := by
  simp only [decayRateOf, IndividualQuantity.div_magnitude,
    IndividualQuantity.add_magnitude, dampingConstantOf, massOf]
  change r.osc.γ / (r.osc.m + r.osc.m) = r.osc.decayRate
  rw [← two_mul, r.osc.gamma_eq_two_mul_m_mul_decayRate]
  have hm : (2 * r.osc.m) ≠ 0 :=
    mul_ne_zero (by norm_num) r.osc.m_ne_zero
  field_simp [hm]

def angularFrequencyOf {o : Osc} (r : DampedRig o) :
    IndividualQuantity o angularFrequencyK ℝ :=
  ⟨r.osc.ω⟩

/-- Kind vs dimension: Proves that oscillator O's indexed angular-frequency magnitud
lemma angularFrequencyOf_magnitude {o : Osc} (r : DampedRig o) :
    (angularFrequencyOf r).magnitude = r.osc.ω := rfl

-- Kind vs dimension: Bare reals let Lean compare oscillator A's γ (kg/s) with ω (s
example (r : DampedRig .A) : Prop := r.osc.γ < r.osc.ω

-- Kind vs dimension: Proves that oscillator A's indexed decay-rate magnitude equals
example (r : DampedRig .A) :
    (decayRateOf r).magnitude = r.osc.decayRate :=
  decayRateOf_magnitude r

-- Dissipation: Proves that PhysLib's law for oscillator A is γ = 2m·decayRate, rela
example (r : DampedRig .A) :
    r.osc.γ = 2 * r.osc.m * r.osc.decayRate :=
  r.osc.gamma_eq_two_mul_m_mul_decayRate

-- Bearer safety: Refused: type mismatch between oscillator B's mass bearer and osci
/--
error: Application type mismatch: The argument
  b
has type
```

```
  DampedRig Osc.B
but is expected to have type
  DampedRig Osc.A
in the application
  massOf b
-/
#guard_msgs in
example (a : DampedRig .A) (b : DampedRig .B) :=
  IndividualQuantity.div
    (KindDiv.law (k₁ := viscousDampingConstantK) (k₂ := massK)
      (k := dampingCoefficientK))
    (dampingConstantOf a) (massOf b)
```

The real-valued `γ < ω` above is accepted, but wrong: `γ` is kg/s, not the whole system's decay rate. `massOf` and `dampingConstantOf` both retain the oscillator's nominal bearer: mass is measured on the designated body and `γ` on the designated damper. Writing `2m` as `m + m` keeps the quotient bearer-safe; the pinned refusal below prevents dividing A's damper reading by B's mass.

3. Same dimension, different kind. Both `δ` and natural angular frequency `ω₀` have dimension `T⁻¹`; the bare-real sum is accepted, while the kinded sum is refused:

```
-- Kind vs dimension: Proves that damping coefficient δ and angular frequency ω both
example :
    Iso80000.Part3.dampingCoefficient.dim =
      Iso80000.Part3.angularFrequency.dim :=
  rfl

-- Kind vs dimension: Proves that dampingCoefficientK and angularFrequencyK are dist
example : dampingCoefficientK ≠ angularFrequencyK := by
  decide

-- Kind vs dimension: Bare reals accept oscillator A's δ + ω as a real although the
example (r : DampedRig .A) : ℝ :=
  r.osc.decayRate + r.osc.ω

-- Kind vs dimension: Refused: type mismatch between dampingCoefficientK and angular
/--
error: Application type mismatch: The argument
  (angularFrequencyOf r).toQuantity
has type
  Quantity angularFrequencyK ℝ
but is expected to have type
```

```
  Quantity dampingCoefficientK ℝ
in the application
  Quantity.add … (decayRateOf r).toQuantity (angularFrequencyOf r).toQuantity
-/
#guard_msgs in
example (r : DampedRig .A) :=
  Quantity.add DifferenceKind.ofScale
    (decayRateOf r).toQuantity (angularFrequencyOf r).toQuantity
```

4. Regime. PhysLib's `isUnderdamped_iff_decayRate_lt` reduces the regime to the numerical condition. Its kinded `<` is refused; the accepted HCP comparison states explicitly that magnitudes are being compared:

```
-- Regime: Proves that oscillator A is underdamped exactly when its numerical decay
example (r : DampedRig .A) :
    r.osc.IsUnderdamped ↔ r.osc.decayRate < r.osc.ω :=
  r.osc.isUnderdamped_iff_decayRate_lt

-- Regime: Refused: type mismatch between angularFrequencyK and dampingCoefficientK;
/--
error: Type mismatch
  (angularFrequencyOf r).toQuantity
has type
  Quantity angularFrequencyK ℝ
but is expected to have type
  Quantity dampingCoefficientK ℝ
-/
#guard_msgs in
example (r : DampedRig .A) :=
  (decayRateOf r).toQuantity < (angularFrequencyOf r).toQuantity

/-- Regime: Proves that an underdamped oscillator O has indexed decay-rate magnitude
lemma underdamped_magnitude_test {o : Osc} (r : DampedRig o)
    (h : r.osc.IsUnderdamped) :
    (decayRateOf r).magnitude < (angularFrequencyOf r).magnitude := by
  rw [decayRateOf_magnitude, angularFrequencyOf_magnitude]
  exact r.osc.isUnderdamped_decayRate h
```

The honest criterion is the dimensionless ratio $\delta/\omega_0 < 1$. Directly ordering the kinded quantities erases that conversion and silently assumes `rad = 1`.

5. States of one oscillator. Two `DampedRig .A` records can describe the same oscillator before and after its damper ages. Their typed difference computes the change from those states; B remains a different system:

```
def decayRateChange (earlier later : DampedRig .A) :
    IndividualQuantity Osc.A dampingCoefficientK ℝ :=
  IndividualQuantity.add DifferenceKind.ofScale
    (decayRateOf later) (-(decayRateOf earlier))

/-- Same-name records: Proves that oscillator A's indexed decay-rate difference has
lemma decayRateChange_magnitude (earlier later : DampedRig .A) :
    (decayRateChange earlier later).magnitude =
      later.osc.decayRate - earlier.osc.decayRate := by
  simp [decayRateChange, IndividualQuantity.add_magnitude,
    IndividualQuantity.neg_magnitude, decayRateOf_magnitude]
  change later.osc.decayRate + -(earlier.osc.decayRate) =
    later.osc.decayRate - earlier.osc.decayRate
  ring

-- Bearer safety: Refused: type mismatch between oscillator A's and oscillator B's d
/--
error: Type mismatch
  -decayRateOf b
has type
  IndividualQuantity Osc.B dampingCoefficientK ℝ
but is expected to have type
  IndividualQuantity Osc.A dampingCoefficientK ℝ
-/
#guard_msgs in
example (a : DampedRig .A) (b : DampedRig .B) :=
  IndividualQuantity.add DifferenceKind.ofScale
    (decayRateOf a) (-(decayRateOf b))
```

`Body`, `Spring`, and `Damper` are designated particulars of the damped oscillator. `γ` is measured on the damper; `δ` and `IsUnderdamped` describe the whole. Records under one name may be states, configurations, or rival estimates; distinct names are distinct systems. L5 is optional and revisits the no-dissipation assumption left by L0–L4.

5.6.8. AI assistance contract

- Keep the lesson independent and use nominal `Osc | A | B` identity.

- Treat records under one name as states, configurations, or rival estimates of one system; compare them, and refuse different names.
- Keep `Body`, `Spring`, and `Damper` as designated particulars. Measure `γ` on the damper and `m` on the body, while retaining the whole oscillator as the bearer of both; place `δ` and the regime on the whole.
- Apply PhysLib laws to `r.osc`; every accepted HCP quantity must compute from the lesson's own state fields.
- Keep accepted formulas at the `IndividualQuantity` level; do not drop the oscillator bearer to build the decay-rate quotient.
- Keep the `γ` kind distinct from ISO 3-24 damping coefficient, even though the upstream formula relates them.
- Compare `δ` and `ω₀` by explicit magnitudes when showing the underdamped criterion; do not present their kinded comparison as licensed.
- Do not change ForPhysLib, PKC core, PhysLib, or Tutorial.

5.6.9. Questions to ask the model so far

Question	Check	What the verdict means
Does the damper conserve mechanical energy?	`energy_dissipation`	No; with positive γ and nonzero velocity, the energy derivative is negative.
What does γ measure?	`kinds_and_decay_rate`	The viscous damping constant measured on the damper, not the ISO whole-system damping coefficient.
Can A's γ be divided by B's mass?	`kinds_and_decay_rate`	No; the quotient requires both quantities to share the same oscillator bearer.
What is δ?	`kinds_and_decay_rate`	PhysLib's `decayRate`, carried on the named whole and kinded as ISO 3-24 damping coefficient.
Can δ be added to ω_0?	`same_dimension_different_kinds`	Not as kinded quantities: both have dimension T^{-1}, but they are different kinds.
How is underdamping tested?	`underdamped_regime`	Compare magnitudes explicitly; the physical criterion is the dimensionless ratio $\delta/\omega_0 < 1$.
Can the damper-aging state be compared with another state of A?	`same_oscillator_states`	Yes; the typed change reads the later and earlier state fields.
Can A's δ be mixed with B's?	`same_oscillator_states`	No; distinct nominal names are distinct systems.

5.6.10. System-context card

Scope	System or component	Sort / status	L5 assumptions and attributes
System	Oscillator A or B	Nominal `Osc`	One name is one system; records under it are states, configurations, or rival estimates. Distinct names are distinct systems.
State	Damped oscillator	`DampedRig o`	Wraps the upstream damped oscillator; same-name records may describe different states.
Part	Body, spring, damper	Designated `PartOf o`	Mass `m` is measured on the body and viscous constant γ on the damper; both quantities retain the whole oscillator's bearer. The spring bears stiffness.
Attribute	Viscous damping constant	Custom ratio-scale kind	γ has units kg/s and is measured on the damper.
Attribute	Damping coefficient δ	ISO 80000-3, 3-24	`decayRate`, in s^{-1}, belongs to the whole damped oscillator.
Attribute	Natural angular frequency ω_0	ISO 80000-3, 3-18	Also T^{-1}, but a different kind from δ.
Regime	Underdamped	Whole-system predicate	`IsUnderdamped` is characterized upstream by `decayRate < ω`; the lesson compares real magnitudes explicitly.
Optional rung	L5	Damping	L0–L4's no-dissipation assumption ends here.

5.7. L6 — What must hold

5.7.1. Synopsis

5.7.1.1. Purpose

State what must hold of the refined context as requirements that the build checks. Each `Provenance.Requirement` names the boundary it governs (a `Provenance.Contract`), the output port it is about, and its scope, the objects it quantifies over. R1 and R2 are scoped to the sort `pairS`: R1 is proved by a theorem, and R2 holds because no `Assembles` instance exists. R3 is scoped to pair P and is decided by a gate function over every state of P. The coverage census lists every output port of every boundary with the requirements that govern it, and `#kind_requirement_clean` fails if any port is ungoverned or unchecked.

5.7.1.2. PKC vocabulary in scope

New at this step:

- `Provenance.Contract` — a declared model boundary with its members and ports.
- `Provenance.Requirement` (`statement`, `governs`, `port`, `scope`, `witness`, `absent`, `gate`, `referents`, `spotChecks`) — a requirement on a boundary.
- `RequirementKind.provable`, `RequirementKind.empirical` — discharged by a proof, or decided by a gate.
- `RequirementScope.sort`, `RequirementScope.objects` — what a requirement quantifies over.
- `NodeId.result`, `PortDir.output` — which port a requirement governs.
- `#kind_requirement_clean` — the gate that fails on any ungoverned or unchecked output.
- `@[kindReferent]`, `@[kindCounterexample]` — mark an empirical referent and a refuting witness.

Carried from earlier steps: `Composite`, `Decomposition`, `Assembles`, `assemble`, `IndividualQuantity`, `Quantity`, `KindOfProperty`, `SortOfSystem`, `Sorted` (`sortOf`), `Designated`, `massK`, `displacementK`, `angularFrequencyK`.

5.7.1.3. Verifiable properties

- R1: for every pair, the pair's mass equals the assembly of its two bodies' masses, proved by a theorem (theorem) `HCP.L6_Requirements#r1_pair_mass`.
- R2: there is no `Assembles pairS normalModeK` instance, so a pair's normal-mode frequency cannot be assembled from its bodies (theorem) `HCP.L6_Requirements#r2_no_aggregation`.
- R3: in every state of pair P, the coupling spring's extension lies within its rated range, decided by the gate `withinRatedRange` (gate) `HCP.L6_Requirements#r3_empirical`.
- `#kind_requirement` rejects with an error a requirement scoped to a bare real number, or to a declaration that is not a `SortOfSystem` (refused) `HCP.L6_Requirements#invalid_scopes`.
- The pinned census lists three governed output ports and none ungoverned or unchecked, and `#kind_requirement_clean` passes (census) `HCP.L6_Requirements#requirement_census`.
- A requirement scoped to pair P, on an accessor that returns a value for every pair, leaves the row for pair Q marked ⚠ UNGOVERNED (census) `HCP.L6_Requirements#scope_gap`.

5.7.1.4. Errors prevented

- Unstatable scope: "every pair" or "pair P" written without a declared sort or a nominal object to quantify over (`HCP.L6_Requirements#invalid_scopes`).
- Over-claimed coverage: a requirement about pair P counted as governing pair Q's output (`HCP.L6_Requirements#scope_gap`).

- Ungoverned output: a model output that no requirement governs goes unreported (`HCP.L6_Requirements#requirement_census`).

5.7.2. Persona

You are a PhysLib contributor: a physicist who writes Lean in PhysLib. You know Lean and PhysLib and trust `Dimension` and `WithDim`; the earlier HCP levels have named the pair sort, its particulars, and its parts.

5.7.3. Pull

The pair now has two bodies, a coupling spring, and multiple records. What must hold, for which systems or objects? A requirement states its quantification scope, writable because the earlier levels named the pair sort and its nominal particulars. One name is one system: records named P are states, configurations, or rival estimates of that same pair and can be compared; Q is a distinct pair. An object-scoped requirement on P governs every state or record of P, not only records used for a spot check. The full worked requirements example is `examples/Tutorial/HarmonicOscillatorRequirements.lean`; this lesson mirrors its machinery without importing that file.

5.7.4. Risks at this step

- **Unstatable scope.** A requirement about every coupled pair needs the pair sort, and one about every state of P needs the nominal pair P; without those named systems, it has nowhere to range. The pinned `invalid_scopes` exercise (`HCP.L6_Requirements#invalid_scopes`) rejects a bare real and a name that is not a sort.

5.7.5. Small rigorous improvement

Question	Context change	Attribute change	Risk shown	PhysLib stage
The pair now has bodies, a coupling spring, and multiple records. What must hold, for which systems or objects?	Give the nominal pair IDs P and Q a `Sorted` instance into `pairS`, and designate bodies and the coupling spring.	Pair mass, normal-mode frequency, coupling-spring extension, requirement scope, referent, and gate.	An unscoped claim cannot say whether it applies to every pair or to every state of P; a bare real and a non-sort are rejected as scopes.	Stage 4 requirements and audits.

5.7.6. Learning path

#	Do	Checked by `HCP.L6_Requirements` anchor	Concept disclosed
1	Assemble the two body masses into a pair mass and state R1 over `pairS`.	`r1_pair_mass`	The theorem reads the pair's own body quantities; the executable check evaluates 3 + 5 = 8.
2	State R2 as the absence of the pair-sort assembly license for normal-mode frequency.	`r2_no_aggregation`	A whole-only frequency does not inherit the body-mass aggregation license.
3	Scope the coupling-spring rating to P and compare two P records against a marked datasheet limit.	`r3_empirical`	The empirical requirement is gated by a Bool that names the referent and is spot-checked on two states of P.
4	Replace P with a bare real, then replace `pairS` with a declaration that is not a sort.	`invalid_scopes`	Object scope requires a `Sorted` instance; sort scope requires a `SortOfSystem`.
5	Survey requirements, census every produced model port, and run the clean gate.	`requirement_census`	Every produced port in these boundaries is governed; the invalid-scope twins are pinned counterexamples.

Worksheet: `HCP/Worksheets/L6_Requirements.lean` · Solution: `HCP/L6_Requirements.lean`

5.7.7. Lesson

1. Pair mass. R1 quantifies over every object of `pairS`. Its pair mass is assembled from the body masses indexed to the named pair, and a witness theorem plus an executable check shows 3 + 5 = 8:

```
inductive PairId | P | Q
deriving DecidableEq, Repr

inductive Body | A | B
deriving DecidableEq, Repr

inductive Spring | kC
deriving DecidableEq, Repr

def pairS : SortOfSystem := ⟨"coupled oscillator pair"⟩
```

```
def bodyS : SortOfSystem := ⟨"body"⟩
def springS : SortOfSystem := ⟨"coupling spring"⟩

instance : Sorted PairId := ⟨fun _ => pairS⟩
instance : Sorted Body := ⟨fun _ => bodyS⟩
instance : Sorted Spring := ⟨fun _ => springS⟩

instance : Designated Body where
  designation
    | .A => ⟨"body A"⟩
    | .B => ⟨"body B"⟩
  designation_inj {x y} h := by
    cases x <;> cases y <;> first | rfl | exact absurd (System.mk.inj h) (by

instance : Designated Spring where
  designation
    | .kC => ⟨"coupling spring"⟩
  designation_inj {x y} _ := by cases x; cases y; rfl
```

The body masses remain individual quantities on their body-in-pair bearers. Only pair mass has the `Assembles pairS massK` license:

```
instance : Assembles pairS massK := ⟨DifferenceKind.ofScale⟩

def bodyParts : Decomposition Body := .union (.atom .A) (.atom .B)

/-- A pair record keeps its measured quantities on their own body or spring. -/
structure PairState (π : PairId) where
  massA : IndividualQuantity (Composite.part (σ := pairS) (bodyPart π .A)) massK Int
  massB : IndividualQuantity (Composite.part (σ := pairS) (bodyPart π .B)) massK Int
  couplingExtension :
    IndividualQuantity (Composite.part (σ := pairS) (springPart π .kC)) displacement

/-- Body A's mass in a particular pair record. -/
def bodyMassA {π : PairId} (state : PairState π) :
    IndividualQuantity (Composite.part (σ := pairS) (bodyPart π .A)) massK Int :=
  state.massA

/-- Body B's mass in a particular pair record. -/
def bodyMassB {π : PairId} (state : PairState π) :
    IndividualQuantity (Composite.part (σ := pairS) (bodyPart π .B)) massK Int :=
```

```
  state.massB

/-- The pair mass, assembled from the two nominal body bearers. -/
def pairMassOf {π : PairId}
    (mA : IndividualQuantity
      (Composite.part (σ := pairS) (bodyPart π .A)) massK Int)
    (mB : IndividualQuantity
      (Composite.part (σ := pairS) (bodyPart π .B)) massK Int) :
    IndividualQuantity (Composite.whole : Composite pairS (PartOf π)) massK Int :=
  assemble (k := massK) pairS
    (Composite.whole : Composite pairS (PartOf π))
    (fun b : Body => Composite.part (σ := pairS) (bodyPart π b))
    bodyParts
    (fun b => match b with
      | .A => mA
      | .B => mB)

/-- The pair mass read from one record of the named pair. -/
def pairMassInState {π : PairId} (state : PairState π) :
    IndividualQuantity (Composite.whole : Composite pairS (PartOf π)) massK Int :=
  pairMassOf (bodyMassA state) (bodyMassB state)

/-- The pair mass equals the sum of the masses recorded for its two bodies. -/
theorem pairMass_assembled {π : PairId} (state : PairState π) :
    (pairMassInState state).magnitude = state.massA.magnitude + state.massB.magnitud
  rfl

/-- The pair-mass boundary: two body masses in, one pair mass out. -/
def pairMassBoundary : Provenance.Contract NodeId KindRef where
  name := "pair mass from body masses"
  members := [``pairMassOf]
  ports := [
    ⟨(NodeId.binder "mA").within ``pairMassOf, .decl ``massK, .input⟩,
    ⟨(NodeId.binder "mB").within ``pairMassOf, .decl ``massK, .input⟩,
    ⟨NodeId.result.within ``pairMassOf, .decl ``massK, .output⟩
  ]
  exits := []

#kind_contract_decide pairMassBoundary
```

```
/-- A spot check with body A's mass 3 and body B's mass 5. -/
def pairP_earlier : PairState PairId.P where
  massA := ⟨3⟩
  massB := ⟨5⟩
  couplingExtension := ⟨-4⟩

/-- A second record of the same nominal pair P. -/
def pairP_later : PairState PairId.P where
  massA := ⟨3⟩
  massB := ⟨5⟩
  couplingExtension := ⟨6⟩

/-- R1's executable check: the assembled mass is 3 + 5 = 8. -/
def r1Spot : Bool := decide ((pairMassInState pairP_earlier).magnitude = 8)

/-- R1: pair mass is the sum of the masses of its two bodies. -/
def r1PairMass : Provenance.Requirement where
  name := "R1"
  statement := "the pair's mass is assembled from the masses of its two bodies"
  kind := .provable
  governs := ``pairMassBoundary
  port := some (NodeId.result.within ``pairMassOf)
  scope := .sort ``pairS
  witness := ``pairMass_assembled
  spotChecks := [``r1Spot]

/--
info: kind requirement 'R1' (provable) on 'pair mass from body masses'
statement: the pair's mass is assembled from the masses of its two bodies
governs: 'pair mass from body masses' at pairMassOf/result
scope: sort HCP.L6_Requirements.pairS
witness: HCP.L6_Requirements.pairMass_assembled
axioms: propext, Classical.choice, Quot.sound
spot check: HCP.L6_Requirements.r1Spot = true
-/
#guard_msgs (whitespace := lax) in #kind_requirement r1PairMass
```

2. Whole-only frequency. R2 is provable by absence: the pair sort has no assembly license for the normal-mode kind. The input and output of the boundary remain attached to the pair whole; the absence rules out summing body frequencies:

```
/-- A pair's normal-mode frequency does not have a body-assembly license. -/
def pairNormalModeOf {π : PairId}
    (ω : IndividualQuantity
      (Composite.whole : Composite pairS (PartOf π)) normalModeK Int) :
    IndividualQuantity (Composite.whole : Composite pairS (PartOf π)) normalModeK In
  ω

/-- The normal-mode boundary has no operation over the bodies. -/
def pairNormalModeBoundary : Provenance.Contract NodeId KindRef where
  name := "pair normal-mode frequency"
  members := [``pairNormalModeOf]
  ports := [
    ⟨(NodeId.binder "ω").within ``pairNormalModeOf, .decl ``normalModeK, .input⟩,
    ⟨NodeId.result.within ``pairNormalModeOf, .decl ``normalModeK, .output⟩
  ]
  exits := []

#kind_contract_decide pairNormalModeBoundary

/-- The aggregation license whose absence is R2's evidence. -/
def pairNormalModeLicense : Prop := Assembles pairS normalModeK

/-- R2: the normal-mode frequency is a whole property, not a sum of body frequencies
def r2NormalMode : Provenance.Requirement where
  name := "R2"
  statement := "the pair's normal-mode frequency is not assembled by summing body fr
  kind := .provable
  governs := ``pairNormalModeBoundary
  port := some (NodeId.result.within ``pairNormalModeOf)
  scope := .sort ``pairS
  absent := ``pairNormalModeLicense

/--
info: kind requirement 'R2' (provable) on 'pair normal-mode frequency'
statement: the pair's normal-mode frequency is not assembled by summing body frequen
governs: 'pair normal-mode frequency' at pairNormalModeOf/result
scope: sort HCP.L6_Requirements.pairS
absent: no instance of HCP.L6_Requirements.pairNormalModeLicense
-/
#guard_msgs (whitespace := lax) in #kind_requirement r2NormalMode
```

3. Empirical range. R3 is scoped to the nominal object P. Under the one-name-one-system rule, it governs every state or record of P; the predicate is defined for every P record, and two Bool spot checks run it against the spring's datasheet limit. The gate mentions the referent, so the requirement is discharged rather than attested:

```
/-- P's state accessor keeps coupling extension on that pair's spring bearer. -/
def couplingExtensionOfP (state : PairState .P) :
    IndividualQuantity (Composite.part (σ := pairS) (springPart .P .kC))
      displacementK Int :=
  state.couplingExtension

/-- The coupling-extension boundary is limited to the particular pair P. -/
def couplingExtensionBoundary : Provenance.Contract NodeId KindRef where
  name := "coupling-spring extension"
  members := [``couplingExtensionOfP]
  ports := [
    ⟨((NodeId.binder "state").field "massA").within ``couplingExtensionOfP,
      .decl ``massK, .input⟩,
    ⟨((NodeId.binder "state").field "massB").within ``couplingExtensionOfP,
      .decl ``massK, .input⟩,
    ⟨((NodeId.binder "state").field "couplingExtension").within ``couplingExtensionO
      .decl ``displacementK, .input⟩,
    ⟨NodeId.result.within ``couplingExtensionOfP, .decl ``displacementK, .output⟩
  ]
  exits := []

#kind_contract_decide couplingExtensionBoundary

/-- The spring's rated extension, from its datasheet. -/
@[kindReferent "coupling-spring manufacturer datasheet"]
def couplingSpringRatedExtension : Quantity displacementK Int := ⟨6⟩

/-- The rated-range predicate applies to every state or record of P. -/
def withinRatedRange (s : PairState .P) : Bool :=
  decide (Int.natAbs (couplingExtensionOfP s).magnitude ≤
    Int.natAbs couplingSpringRatedExtension.magnitude)

def pairP_earlierWithinRatedRange : Bool := withinRatedRange pairP_earlier
def pairP_laterWithinRatedRange : Bool := withinRatedRange pairP_later

/-- R3: every state or record of P must keep the spring within its rated range. -/
```

```
def r3CouplingSpringRange : Provenance.Requirement where
  name := "R3"
  statement := "P's coupling-spring extension stays within its rated range in every
  kind := .empirical
  governs := ``couplingExtensionBoundary
  port := some (NodeId.result.within ``couplingExtensionOfP)
  scope := .objects [``PairId.P]
  referents := [``couplingSpringRatedExtension]
  gate := ``withinRatedRange
  spotChecks := [``pairP_earlierWithinRatedRange, ``pairP_laterWithinRatedRange]

/--
info: kind requirement 'R3' (empirical) on 'coupling-spring extension'
statement: P's coupling-spring extension stays within its rated range in every state
governs: 'coupling-spring extension' at couplingExtensionOfP/result
scope: objects HCP.L6_Requirements.PairId.P
referent: HCP.L6_Requirements.couplingSpringRatedExtension — coupling-spring manufac
gate: HCP.L6_Requirements.withinRatedRange
spot check: HCP.L6_Requirements.pairP_earlierWithinRatedRange = true
spot check: HCP.L6_Requirements.pairP_laterWithinRatedRange = true
-/
#guard_msgs (whitespace := lax) in #kind_requirement r3CouplingSpringRange
```

The two failed scopes are counterexamples, so the survey can pin their diagnostics without counting them as valid requirements. A bare value is not a nominal object, and a name is not a sort merely because it is a declaration:

```
/-- A bare real is not a nominal object with a sort. -/
def someBareReal : ℝ := 7

/-- A name whose declaration is not a system sort. -/
def notASort : Nat := 1

@[kindCounterexample]
def r3BareRealScope : Provenance.Requirement :=
  { r3CouplingSpringRange with scope := .objects [``someBareReal] }

/--
error: the scope object 'HCP.L6_Requirements.someBareReal' is of type 'ℝ', which has
  'Sorted' instance — an object of no sort is a bare value, not a particular a requi
-/
```

```
#guard_msgs (whitespace := lax) in #kind_requirement r3BareRealScope

@[kindCounterexample]
def r3NotASortScope : Provenance.Requirement :=
  { r3CouplingSpringRange with scope := .sort ``notASort }

/--
error: the scope 'HCP.L6_Requirements.notASort' is not a 'SortOfSystem' — a scope ov
-/
#guard_msgs (whitespace := lax) in #kind_requirement r3NotASortScope
```

4. Census. The valid requirements are surveyed, every produced model port has a governing requirement, and the clean command enforces that result. The census infers a result port's object domain from its member's type: R3 covers a P-only result, but leaves Q ungoverned on a generic pair boundary. An unresolved bearer is an unchecked violation unless exempted. A `.sort` scope needs an indexed result whose object type has a `Sorted` instance: R1 and R2's results are borne by the pair whole, and the instance above maps the whole to `pairS` and each part to its own sort, so the census checks their scope. An object of another sort is reported ungoverned. A `.decided` scope is checked against a `deciderProof` theorem proving its decider returns `true` for every result of the governed member; without that proof the census reports the port unchecked.

```
/--
info: kind requirements — 3 requirement(s), 2 exempted: 2 provable, 1 empirical, 0 a
  HCP.L6_Requirements.r1PairMass: 'R1' (provable) on 'pair mass from body masses' —
  HCP.L6_Requirements.r2NormalMode: 'R2' (provable) on 'pair normal-mode frequency'
  HCP.L6_Requirements.r3CouplingSpringRange: 'R3' (empirical) on 'coupling-spring ex
  ⊘ HCP.L6_Requirements.r3BareRealScope: counterexample, exempted
  ⊘ HCP.L6_Requirements.r3NotASortScope: counterexample, exempted
-/
#guard_msgs (whitespace := lax) in #kind_requirements HCP.L6_Requirements

/--
info: requirement coverage:
[governed] HCP.L6_Requirements.couplingExtensionBoundary ('coupling-spring extension
[governed] HCP.L6_Requirements.pairMassBoundary ('pair mass from body masses') pairM
[governed] HCP.L6_Requirements.pairNormalModeBoundary ('pair normal-mode frequency')
3 row(s) over 3 model boundary(ies): 3 governed — clean
requirements: 2 provable, 1 empirical, 0 attested
-/
#guard_msgs (whitespace := lax) in #kind_requirement_coverage HCP.L6_Requirements
#kind_requirement_clean HCP.L6_Requirements
```

All three requirements are discharged: R1 has a theorem and a spot check, R2 has an absent aggregation license, and R3 has a referent-backed gate. The invalid scopes remain visible as counterexamples rather than silently entering the requirement census.

The sibling-namespace probe shows the scope gap directly: a P-scoped requirement governs P's result while the generic pair boundary leaves Q ungoverned.

```
/-- A generic pair-state result retains its pair index. -/
def couplingExtensionOfAny {π : PairId} (state : PairState π) :
    IndividualQuantity (Composite.part (σ := pairS) (springPart π .kC)) displacement
  state.couplingExtension

def couplingExtensionAnyBoundary : Provenance.Contract NodeId KindRef where
  name := "coupling-spring extension, generic pair"
  members := [``couplingExtensionOfAny]
  ports := [
    ⟨((NodeId.binder "state").field "massA").within ``couplingExtensionOfAny,
      .decl ``massK, .input⟩,
    ⟨((NodeId.binder "state").field "massB").within ``couplingExtensionOfAny,
      .decl ``massK, .input⟩,
    ⟨((NodeId.binder "state").field "couplingExtension").within ``couplingExtensionO
      .decl ``displacementK, .input⟩,
    ⟨NodeId.result.within ``couplingExtensionOfAny, .decl ``displacementK, .output⟩
  ]
  exits := []

#kind_contract_decide couplingExtensionAnyBoundary

def r3POnlyOnAny : Provenance.Requirement :=
  { HCP.L6_Requirements.r3CouplingSpringRange with
    governs := ``couplingExtensionAnyBoundary
    port := some (NodeId.result.within ``couplingExtensionOfAny) }

/--
info: kind requirement 'R3' (empirical) on 'coupling-spring extension, generic pair'
statement: P's coupling-spring extension stays within its rated range in every state
governs: 'coupling-spring extension, generic pair' at couplingExtensionOfAny/result
scope: objects HCP.L6_Requirements.PairId.P
referent: HCP.L6_Requirements.couplingSpringRatedExtension — coupling-spring manufac
gate: HCP.L6_Requirements.withinRatedRange
spot check: HCP.L6_Requirements.pairP_earlierWithinRatedRange = true
```

```
spot check: HCP.L6_Requirements.pairP_laterWithinRatedRange = true
-/
#guard_msgs (whitespace := lax) in #kind_requirement r3POnlyOnAny

/--
info: requirement coverage:
[governed] HCP.L6_ScopeGap.couplingExtensionAnyBoundary ('coupling-spring extension,
⚠ UNGOVERNED HCP.L6_ScopeGap.couplingExtensionAnyBoundary ('coupling-spring extensio
2 row(s) over 1 model boundary(ies): 1 governed, 1 UNGOVERNED — requirement-coverage
requirements: 0 provable, 1 empirical, 0 attested
-/
#guard_msgs (whitespace := lax) in #kind_requirement_coverage HCP.L6_ScopeGap
```

5.7.7.1. Requirement scope

The full worked requirements example is `examples/Tutorial/HarmonicOscillatorRequirements.lean`; L6 mirrors its Provenance boundaries, witnesses, referents, gates, surveys, and coverage commands without importing the Tutorial file. The census infers a result port's object domain from its member's type: R3 covers a P-only result, but leaves Q ungoverned on a generic pair boundary. An unresolved bearer is an unchecked violation unless exempted. A `.sort` scope needs an indexed result whose object type has a `Sorted` instance: R1 and R2's results are borne by the pair whole, and the instance above maps the whole to `pairS` and each part to its own sort, so the census checks their scope. An object of another sort is reported ungoverned. A `.decided` scope is checked against a `deciderProof` theorem proving its decider returns `true` for every result of the governed member; without that proof the census reports the port unchecked.

5.7.8. AI assistance contract

- Keep L6 independent, with local nominal `PairId | P | Q`, `Body | A | B`, and the coupling spring.
- Preserve nominal identity: one pair name is one system; same-name records are comparable states or configurations; P and Q are distinct.
- Keep pair mass on the pair whole and body masses on their body-in-pair bearers.
- Use the `pairS` sort scope for R1 and the singleton object scope containing P for empirical R3.
- Mark the rated limit as an external referent; make the gate mention it and evaluate it on two states of P.
- Pin both invalid scopes and run the requirement survey, coverage census, and clean gate.
- Do not import the Tutorial worked example. The full worked example is `examples/Tutorial/HarmonicOscillatorRequirements.lean`.

- Leave optional `.decided` scope out of this lesson. If used, provide a `deciderProof` theorem proving that its decider returns `true` for every result of the governed member; without that proof the census reports the port unchecked.

5.7.9. Questions to ask the model so far

Question	Check	What the verdict means
What does R1 quantify over?	`r1_pair_mass`	Every object of sort `pairS`; its theorem relates the assembled pair mass to the masses of the two bodies.
Does normal-mode frequency sum over the bodies?	`r2_no_aggregation`	No `Assembles pairS normalModeK` instance exists.
What does the spring rating govern?	`r3_empirical`	Every state or record of P; the gate checks two such records against the same datasheet limit.
Can a bare real or a non-sort name state the scope?	`invalid_scopes`	No; the object must have a `Sorted` instance and a sort must be a `SortOfSystem`.
Are all produced ports governed?	`requirement_census`	Yes; the census is clean and the command enforces that invariant.

5.7.10. System-context card

Scope	System or component	Sort / status	L6 requirements
System	Coupled pair P or Q	Nominal `PairId`, sorted into `pairS`	One name denotes one pair; same-name records are states or configurations, and Q differs from P.
Part	Body A or B	Designated body	Each body has its own mass bearer within the named pair.
Part	Coupling spring kC	Designated spring	Its extension is state-indexed and checked against an external rated limit.
Requirement	Pair mass R1	`pairS` sort	Provable from the pair's two body masses; spot check gives 8 from 3 and 5.
Requirement	Normal-mode frequency R2	`pairS` sort	Provable negative: no aggregation license for `normalModeK`.
Requirement	Spring range R3	Singleton object scope containing P	Empirical, gated by the datasheet referent; applies to every P state or record.
Audit	Requirement census	Stage 4	The valid requirements govern every produced model port; invalid-scope twins are counterexamples.

6. The Proved Spine

This chapter records what PropertyKindCalculus already establishes, sorry-free, in its Mathlib-free core. Every node here links to a real declaration with `(lean := "PropertyKindCalculus.…")`, so its *proved* status in the dependency graph is read directly from the checked library rather than asserted in prose.

The point to keep in view: each of these is a law or an operation, not a class membership. A description-logic reasoner (OWL2/SROIQ) can record that `width` specializes `length`; it cannot state, let alone prove, that specialization is transitive or that operator availability is monotone in scale. Those are the entries below. The systems those properties characterize — and the part–whole and four-category structure they carry — are the foundations chapter's subject.

6.1. Scale — the operator-based division of ⟨property⟩ (Dybkær Ch. 12)

Definition 6.1.1. group uses 0 used by 7 ✓

The four *scale types* of Dybkær Fig. 12.21, ordered by operator richness: `\mathrm{nominal} \sqsubset \mathrm{ordinal} \sqsubset \mathrm{interval} \sqsubset \mathrm{ratio}`, admitting respectively `\{=,\neq\}`, then `\{<,>\}`, then `\{+,-\}`, then `\{\times,\div\}`.

Lean code for Definition 6.1.1. 1 definition

inductive (4 constructors) · defined in `PropertyKindCalculus/Scale.lean` complete

```
inductive PropertyKindCalculus.ScaleType : Type
```

Proof for Definition 6.1.1 uses 0

A four-constructor `inductive` with a `rank : ScaleType → Nat` and the induced ≤. The lemmas `le_def`, `le_refl`, `le_trans`, and `le_antisymm` establish that ≤ is the expected linear order.

Theorem 6.1.2. group statement">used 1 used by 0 ✓

Owner	Tags
Nicolas Rouquette	proved

> *The scale order is a linear order: reflexive (`le_refl`), transitive (`le_trans`), and antisymmetric (`le_antisymm`) on [the scale types](#).*

Lean code for Theorem 6.1.2.

1 theorem

theorem · defined in PropertyKindCalculus/Scale.lean

complete

```
theorem PropertyKindCalculus.ScaleType.le_antisymm
  {a b : PropertyKindCalculus.ScaleType} (h₁ : a ≤ b) (h₂ : b ≤ a) :
  a = b
```

Proof for Theorem 6.1.2

uses 0

Reflexivity and transitivity are inherited from `Nat.le` on `rank`. Antisymmetry reduces to `Nat.le_antisymm` on the ranks followed by a four-by-four case split (`cases a <;> cases b <;> simp_all`).

Theorem 6.1.3.

group statement">used by 2 ✓

Owner	Priority	Tags
Nicolas Rouquette	high	capstone, proved

> ***Operator-availability monotonicity.*** *A richer scale licenses every operation a poorer one does: if `a \le b` then `AllowsOrder`, `AllowsDifference`, and `AllowsRatio` each transport from `a` to `b`. This is a meta-theorem OWL2's reasoner cannot even phrase, since it quantifies over which operations are defined for a kind. Builds on [the scale types].*

Lean code for Theorem 6.1.3.

1 theorem

theorem · defined in PropertyKindCalculus/Scale.lean

complete

```
theorem PropertyKindCalculus.ScaleType.allows_mono
  {a b : PropertyKindCalculus.ScaleType} (h : a ≤ b) :
  (a.AllowsOrder → b.AllowsOrder) ∧
    (a.AllowsDifference → b.AllowsDifference) ∧
      (a.AllowsRatio → b.AllowsRatio)
```

Proof for Theorem 6.1.3

uses 0

Decide all sixteen ordered pairs: `cases a <;> cases b <;> simp_all [AllowsOrder, AllowsDifference, AllowsRatio, le_def, rank]`.

6.2. Kind-of-property and kind-of-quantity (Dybkær Ch. 6, 13)

Definition 6.2.1. group statement">used by 17 ✓

A kind-of-property (§6.19) bundles a terminological `id`, a `scale` (from the scale types), and an optional examination principle. A kind *is a kind-of-quantity* (§13.3.1) iff its scale has magnitude (is at least ordinal).

Lean code for Definition 6.2.1. 1 definition

`structure` (3 fields) · defined in `PropertyKindCalculus/Kind.lean` — complete

```
structure PropertyKindCalculus.KindOfProperty : Type
```

Proof for Definition 6.2.1 uses 0

A `structure` with `DecidableEq`. `IsQuantity`, and the scale divisions `IsNominal`/`IsOrdinal`/`IsDifferential`/`IsRational` (§13.2), are definitional predicates over the `scale` field.

Theorem 6.2.2. group statement">us

Owner: Nicolas Rouquette — Tags: proved

A nominal kind is, by construction, not a kind-of-quantity: it has no magnitude. Uses kind-of-property.

Lean code for Theorem 6.2.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Kind.lean` — complete

```
theorem PropertyKindCalculus.KindOfProperty.nominal_not_quantity
  {k : PropertyKindCalculus.KindOfProperty} (h : k.IsNominal) :
  ¬k.IsQuantity
```

Proof for Theorem 6.2.2 uses 0

Unfold `IsQuantity`, `IsNominal`, and `HasMagnitude`; the scale is `nominal`, so `scale ≠ nominal` is false.

Theorem 6.2.3. group statement">us

Owner	Tags
Nicolas Rouquette	proved

A rational kind is a kind-of-quantity. Uses kind-of-property.

Lean code for Theorem 6.2.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Kind.lean` — complete

```
theorem PropertyKindCalculus.KindOfProperty.rational_isQuantity
  {k : PropertyKindCalculus.KindOfProperty} (h : k.IsRational) :
  k.IsQuantity
```

Proof for Theorem 6.2.3

uses 0

The scale is `ratio`, so `scale ≠ nominal` holds; unfold and simplify.

Definition 6.2.4.

group — uses 0

A kind is a *ratio quantity* (VIM4 2CD 1.3) when its scale is `ratio`: the operators `\times, \div` are defined, so its quantities have an absolute zero and form ratios.

Lean code for Definition 6.2.4.

1 definition

`def` · defined in `PropertyKindCalculus/Kind.lean` — complete

```
def PropertyKindCalculus.KindOfProperty.IsRational
  (k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 6.2.4

uses 0

`IsRational k := k.scale = .ratio`.

Definition 6.2.5.

group — uses 0

A kind is an *interval quantity* (VIM4 2CD 1.4) when its scale is `interval`: the operators `+, -` are defined (differences are meaningful), but ratios are not.

Lean code for Definition 6.2.5. 1 definition

`def` · defined in `PropertyKindCalculus/Kind.lean` — complete

```
def PropertyKindCalculus.KindOfProperty.IsDifferential
  (k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 6.2.5 uses 0

```
IsDifferential k := k.scale = .interval.
```

Definition 6.2.6. group uses 0

A kind is an *ordinal quantity* (VIM4 2CD 1.33) when its scale is `ordinal`: only the order `<, >` is defined, with neither differences nor ratios.

Lean code for Definition 6.2.6. 1 definition

`def` · defined in `PropertyKindCalculus/Kind.lean` — complete

```
def PropertyKindCalculus.KindOfProperty.IsOrdinal
  (k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 6.2.6 uses 0

```
IsOrdinal k := k.scale = .ordinal.
```

6.3. Specialization and mutual comparability (OML / Dybkær §6)

Definition 6.3.1. group statement">used by 8 ✓

Specialization `Specializes E` is the reflexive–transitive closure of a direct-parent edge relation `E` over kind-of-property. Different `E` give different per-application hierarchies — the extensibility a fixed `inductive` would deny.

Lean code for Definition 6.3.1. 1 definition

`inductive` (2 constructors, Prop, 3 parameters) · defined in `PropertyKindCalculus/Specialization.lean` — complete

```
inductive PropertyKindCalculus.Specializes
  (E :
    PropertyKindCalculus.KindOfProperty →
      PropertyKindCalculus.KindOfProperty → Prop) :
  PropertyKindCalculus.KindOfProperty →
    PropertyKindCalculus.KindOfProperty → Prop
```

Proof for Definition 6.3.1

uses 0

An `inductive` with `refl` and `step` constructors; `of_edge` lifts a single edge.

Theorem 6.3.2.

group statement">uses 1 by 0 ✓

Owner	Priority	Tags
Nicolas Rouquette	high	capstone, proved

Specialization is a preorder. *It is reflexive (the `refl` constructor) and transitive (`Specializes.trans`). Stating — let alone proving — an algebraic law of the taxonomy is exactly what an OWL2 reasoner cannot do; it checks consistency, not laws. Uses specialization.*

Lean code for Theorem 6.3.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Specialization.lean` — complete

```
theorem PropertyKindCalculus.Specializes.trans
  {E :
    PropertyKindCalculus.KindOfProperty →
      PropertyKindCalculus.KindOfProperty → Prop}
  {a b c : PropertyKindCalculus.KindOfProperty}
  (h₁ : PropertyKindCalculus.Specializes E a b)
  (h₂ : PropertyKindCalculus.Specializes E b c) :
  PropertyKindCalculus.Specializes E a c
```

Proof for Theorem 6.3.2

uses 0

Transitivity is by induction on the first derivation: `refl` returns the second derivation, `step` re-applies the constructor under the inductive hypothesis.

Definition 6.3.3.

group statement">uses 1 by 2 ✓

Two kinds are *mutually comparable* iff they share a common super-kind (`\exists p,\ a \sqsubseteq p \wedge b \sqsubseteq p`). This is how `width` and `height` stay comparable as lengths while remaining distinct sub-kinds. Uses specialization.

Lean code for Definition 6.3.3.

1 definition

`def` · defined in `PropertyKindCalculus/Specialization.lean` — complete

```
def PropertyKindCalculus.MutuallyComparable
  (E :
    PropertyKindCalculus.KindOfProperty →
      PropertyKindCalculus.KindOfProperty → Prop)
  (a b : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 6.3.3

uses 0

An existential over a shared upper bound.

Theorem 6.3.4.

group statement"> used by 0 ✓

Owner	Tags
Nicolas Rouquette	proved

Mutual comparability is reflexive (`refl`) and symmetric (`symm`), and any specialization induces it (`of_specializes`). Uses mutual comparability.

Lean code for Theorem 6.3.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Specialization.lean` — complete

```
theorem PropertyKindCalculus.MutuallyComparable.symm
  {E :
    PropertyKindCalculus.KindOfProperty →
      PropertyKindCalculus.KindOfProperty → Prop}
  {a b : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.MutuallyComparable E a b) :
  PropertyKindCalculus.MutuallyComparable E b a
```

Proof for Theorem 6.3.4

uses 0

> Reflexivity takes the witness `a` itself; symmetry swaps the two halves of the conjunction.

6.4. Examination — principle, method, procedure (Dybkær Ch. 7, 14)

Definition 6.4.1. group uses 0 used by 17 ✓

The examination chain has three layers — an *examination principle* (§7.5 / VIM4 2CD 2.4), a *method* based on a principle (VIM4 2CD 2.5), and a *procedure* based on a method (VIM4 2CD 2.7) — gathered into one carrier `ExaminationItem` so that refinement is a homogeneous relation. The forgetful projection `basePrinciple` sends any item down to the principle it rests on.

Lean code for Definition 6.4.1. 1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/Examination.lean` complete

```
structure PropertyKindCalculus.ExaminationPrinciple : Type
```

Proof for Definition 6.4.1 uses 0

> Three small `structure`s with `DecidableEq`, united by an `ExaminationItem` inductive; `basePrinciple` is the projection to the principle layer.

Definition 6.4.2. group statement">statement">used by ✓

An *examination method* (Dybkær Ch. 7 / VIM4 2CD 2.5) is a generic description of an examination, *based on* an examination principle.

Lean code for Definition 6.4.2. 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Examination.lean` complete

```
structure PropertyKindCalculus.ExaminationMethod : Type
```

Proof for Definition 6.4.2 uses 0

> A `structure` carrying an identifier and the `ExaminationPrinciple` it is based on.

Definition 6.4.3. group statement">used by 0 ✓

An *examination procedure* (Dybkær Ch. 7 / VIM4 2CD 2.7) is a detailed description, *based on* an examination method; its principle is the method's principle.

Lean code for Definition **6.4.3.** 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Examination.lean` — complete

```
structure PropertyKindCalculus.ExaminationProcedure : Type
```

Proof for Definition 6.4.3 uses 0

A `structure` carrying an identifier and the `ExaminationMethod` it is based on.

Definition **6.4.4.** group statement">used by 2 ✓

Refinement `Refines` is the reflexive–transitive closure of the "based-on" edges (a procedure is based on its method, a method on its principle): procedure ⊑ method ⊑ principle. It mirrors specialization over kinds — the same closure construction, one level down at the defining aspect.

Lean code for Definition **6.4.4.** 1 definition

`inductive` (2 constructors, Prop, 2 parameters) · defined in `PropertyKindCalculus/Examination.lean` — complete

```
inductive PropertyKindCalculus.Refines :
  PropertyKindCalculus.ExaminationItem →
    PropertyKindCalculus.ExaminationItem → Prop
```

Proof for Definition 6.4.4 uses 0

An `inductive` with `refl` and `step` constructors over a `BasedOn` edge relation; `of_basedOn` lifts a single edge.

Theorem **6.4.5.** group statement">used by 0 ✓

Owner	Tags
Nicolas Rouquette	proved

> ***Refinement is a preorder.*** *It is reflexive (the `refl` constructor) and transitive (`Refines.trans`) on the examination chain — the same algebraic law proved for specialization, which an OWL2 reasoner can neither state nor prove.*

Lean code for Theorem 6.4.5.

1 theorem

theorem · defined in PropertyKindCalculus/Examination.lean

complete

```
theorem PropertyKindCalculus.Refines.trans
  {a b c : PropertyKindCalculus.ExaminationItem}
  (h₁ : PropertyKindCalculus.Refines a b)
  (h₂ : PropertyKindCalculus.Refines b c) :
  PropertyKindCalculus.Refines a c
```

Proof for Theorem 6.4.5

uses 0

Transitivity is by induction on the first derivation, exactly as for `Specializes`: `refl` returns the second derivation, `step` re-applies the constructor under the inductive hypothesis.

Theorem 6.4.6.

group statement"> uses 1 used by 0 ✓

Owner	Priority	Tags
Nicolas Rouquette	high	capstone, proved

Refinement preserves the principle. *Every examination in one refinement chain rests on the same base principle: $\mathrm{Refines}\ a\ b \;\Longrightarrow\; \mathrm{basePrinciple}\ a = \mathrm{basePrinciple}\ b$. The forgetful projection to the principle layer is invariant along refinement — the examination-layer analogue of $\dim$ being a homomorphism, and one provable here in the Mathlib-free core without PhysLib. Builds on refinement.*

Lean code for Theorem 6.4.6.

1 theorem

theorem · defined in PropertyKindCalculus/Examination.lean

complete

```
theorem PropertyKindCalculus.Refines.basePrinciple_eq
  {a b : PropertyKindCalculus.ExaminationItem}
  (h : PropertyKindCalculus.Refines a b) :
  a.basePrinciple = b.basePrinciple
```

Proof for Theorem 6.4.6

uses 0

By induction on the refinement derivation: each `BasedOn` edge preserves the base principle (`cases … <;> rfl`), and the inductive step chains the resulting equalities.

Theorem 6.4.7. group uses 2 statement">used by

Owner	Tags
Nicolas Rouquette	proved

The examination principle is a defining aspect. *Two kinds examined by different principles are distinct kinds — independently of scale and dimension. This is what keeps width and height distinct while both remain rational lengths, mutually comparable under their shared super-kind Length.*

Lean code for Theorem 6.4.7. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Examination.lean` — complete

```
theorem PropertyKindCalculus.KindOfProperty.distinct_of_examPrinciple
  {k₁ k₂ : PropertyKindCalculus.KindOfProperty}
  (h : k₁.examPrinciple ≠ k₂.examPrinciple) : k₁ ≠ k₂
```

Proof for Theorem 6.4.7 uses 0

The kinds differ in their `examPrinciple` field, so they are unequal by congruence: substitute the assumed kind-equality and derive a contradiction from the field inequality.

6.5. Property value and value scale (Dybkær Ch. 9, 16, 10, 17)

Definition 6.5.1. group statement">used by 4

A *property value* (§9.15) bundles its kind-of-property, a numeral (the numerical value, §16.10), and the reference the numeral is taken against — a metrological unit for a quantity value, a designation for a nominal one. A value *is a quantity value* (§16.10) iff its kind has magnitude, and two values are *comparable* iff they are of the same kind.

Lean code for Definition 6.5.1. 1 definition

`structure (3 fields)` · defined in `PropertyKindCalculus/PropertyValue.lean` — complete

```
structure PropertyKindCalculus.PropertyValue : Type
```

Proof for Definition 6.5.1

uses 0

> A `structure` with `DecidableEq`. `scale`, `IsQuantityValue`, and `Comparable` are definitional projections over the `kind` field.

Theorem 6.5.2.

group statement">us

Owner	Tags
Nicolas Rouquette	proved

> ***Comparability of values is an equivalence.*** *Two property values are comparable iff they are of the same kind (§9.15); the relation is reflexive, symmetric, and transitive. Comparable values are moreover governed by the same scale type, so the same operators are defined on both (`scale_eq_of_comparable`). Transitivity is exactly the algebraic law a description logic cannot state.*

Lean code for Theorem 6.5.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/PropertyValue.lean`

complete

```
theorem PropertyKindCalculus.PropertyValue.Comparable.trans
  {u v w : PropertyKindCalculus.PropertyValue} (h₁ : u.Comparable v)
  (h₂ : v.Comparable w) : u.Comparable w
```

Proof for Theorem 6.5.2

uses 0

> Comparability unfolds to equality of the `kind` field, so reflexivity, symmetry, and transitivity are those of `Eq`; `scale_eq_of_comparable` is `congrArg` on the `scale` projection.

Definition 6.5.3.

group statement">state

A *property value scale* (§10.14) is the ordered set of possible, mutually comparable values of one kind. It carries that kind and a provenance — *true* (consistent with the property's definition, §10.16.1) or *examined* (obtained by an examination procedure, §10.16.2). Its `scaleType` is the kind's scale, the datum Table 17.4 uses to gate which manipulations a scale admits, and `KindOfProperty.valueScale` gives the canonical true scale of a kind (§9.15 ↔ §10.14).

Lean code for Definition 6.5.3.

1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/ValueScale.lean` — complete

```
structure PropertyKindCalculus.ValueScale : Type
```

Proof for Definition 6.5.3

uses 0

A `structure` carrying `kind` and a `ScaleProvenance`; `Admits v` is `v.kind = s.kind`, and `KindOfProperty.valueScale` builds the canonical true scale.

Theorem 6.5.4.

group · uses 3 · used by 0 ✓

Owner	Priority	Tags
Nicolas Rouquette	high	capstone, proved

A value scale holds only mutually comparable values. *Any two values a value scale admits are of the scale's kind, hence comparable (§10.14) — the defining property of a value scale, quantified over its members. The scale type further licenses operations monotonically (`allows_mono_of_le`): a richer value scale supports every manipulation a poorer one does (Table 17.4), inherited from operator monotonicity.*

Lean code for Theorem 6.5.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/ValueScale.lean` — complete

```
theorem PropertyKindCalculus.ValueScale.comparable_of_mem
  {s : PropertyKindCalculus.ValueScale}
  {v w : PropertyKindCalculus.PropertyValue} (hv : s.Admits v)
  (hw : s.Admits w) : v.Comparable w
```

Proof for Theorem 6.5.4

uses 0

Membership unfolds to equality with the scale's kind, so two members' kinds are equal by `trans`/`symm`; operator monotonicity is the scale-layer `allows_mono` re-exported through `scaleType`.

6.6. Metrological units (Dybkær Ch. 18, §13.3.3)

Definition 6.6.1. group uses 2 used by 22 ✓

A *metrological unit* (§18.12) carries the kind it references and a terminological `symbol` (the unit id, e.g. `"m"`, `"kg"`). A kind *bears a unit* (`BearsUnit`, §13.3.3) iff its scale is at least differential, and a unit is *well-formed* iff its kind bears one; `measure u n` builds the property value "`n` × `symbol`".

Lean code for Definition 6.6.1.

1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Unit.lean` — complete

```
structure PropertyKindCalculus.MetrologicalUnit : Type
```

Proof for Definition 6.6.1 uses 0

A `structure` with `DecidableEq`. `BearsUnit` is `scale.AllowsDifference`, `WellFormed` is `kind.BearsUnit`, and `measure` fills the value's `numeral` and `reference` fields from `n` and the unit's `symbol`.

Theorem 6.6.2. group statement">state

Owner: Nicolas Rouquette

Tags: proved

> ***Commensurability is an equivalence.*** *Two units are commensurable iff they reference the same kind (§9.13.4); the relation is reflexive, symmetric, and transitive. A metre and a centimetre are commensurable; a metre and a kilogram are not — incommensurability across kinds is a fact of the types, not a runtime check. Transitivity is exactly the law a description logic cannot state.*

Lean code for Theorem 6.6.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Unit.lean` — complete

```
theorem PropertyKindCalculus.MetrologicalUnit.Commensurable.trans
  {u₁ u₂ u₃ : PropertyKindCalculus.MetrologicalUnit}
  (h₁ : u₁.Commensurable u₂) (h₂ : u₂.Commensurable u₃) :
  u₁.Commensurable u₃
```

Proof for Theorem 6.6.2 uses 0

Commensurability unfolds to equality of the `kind` field, so reflexivity, symmetry, and transitivity are those of `Eq`.

Theorem 6.6.3. group statement">us

Owner	Tags
Nicolas Rouquette	proved

Only a unitary kind bears a unit. *A nominal kind has no magnitude, and — the distinctive §9.13.4 fact — an ordinal kind, though rankable, has no reference-times-number magnitude either, so neither bears a metrological unit (`not_wellFormed_of_nominal`, `not_wellFormed_of_ordinal`). Conversely a rational kind does (`rational_bears_unit`), and bearing a unit implies being a kind-of-quantity (`bearsUnit_isQuantity`). This gate is invisible to a dimension-and-scale-only view that treats every magnitude alike. Uses the metrological unit.*

Lean code for Theorem 6.6.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Unit.lean` — complete

```
theorem PropertyKindCalculus.MetrologicalUnit.not_wellFormed_of_ordinal
  {u : PropertyKindCalculus.MetrologicalUnit} (h : u.kind.IsOrdinal) :
  ¬u.WellFormed
```

Proof for Theorem 6.6.3 uses 0

`BearsUnit` is `scale.AllowsDifference`, which is `False` at nominal and ordinal; substitute the scale from `IsNominal`/`IsOrdinal` and the goal is `¬ False`. The rational case and `bearsUnit_isQuantity` are the same four-way case split on the scale.

Theorem 6.6.4. group uses 3 used by 0 ✓

Owner	Priority	Tags
Nicolas Rouquette	high	capstone, proved

The number-and-reference form is a faithful round-trip (§13.3.3). *Measuring a quantity in a unit yields a numeral, and re-applying the unit to that numeral recovers the value: `\mathrm{numeral}\, (\mathrm{measure}\ u\ n) = n \qquad\text{and}\qquad \mathrm{measure}\ u\ (\mathrm{numeral}\ v) = v \;\text{ when } v \text{ is measured in } u`. So `quantity / unit = number` and `number × unit = quantity` invert one another — the §13.3.3 "reference quantity multiplied by a number", checked as a law. Measuring in a well-formed unit moreover yields a quantity value*

> (`measure_isQuantityValue`), and values measured in *commensurable* units are comparable. Builds on *the metrological unit*.

Lean code for Theorem 6.6.4.

1 theorem

theorem · defined in `PropertyKindCalculus/Unit.lean`

complete

```
theorem PropertyKindCalculus.MetrologicalUnit.measure_eq_of_measures
  {u : PropertyKindCalculus.MetrologicalUnit}
  {v : PropertyKindCalculus.PropertyValue} (h : u.Measures v) :
  u.measure v.numeral = v
```

Proof for Theorem 6.6.4

uses 0

> `measure_numeral` is `rfl`. For the converse, destructure "measured in `u`" into the kind- and reference-equalities, unfold `measure`, and rewrite the unit's `kind` and `symbol` back to the value's fields; structure eta closes the goal.

7. Capstone theorems — what the calculus can do

A reader who has followed the chapters so far knows what the calculus *is*: a kind is a type index, two kinds meet only under a witnessed law, a boundary is declared and then decided, and a model carries its attestations as hypotheses. The question a reviewer asks first is a different one: *what can the calculus do?* What guarantee does a model earn by being written in it — stated as a theorem whose hypotheses the build discharges and whose conclusion a reviewer can use without reading the model? Three theorems answer it, one per layer of the calculus — the provenance layer, the computation layer and the dimension layer, laid out below — read through the taint-tracking paradigm:

1. **The seal of a module.** Once a module's inputs, constants and attestations are declared, no other raw datum reaches its outputs: every leaf of an output's pedigree is a declared source. Stated over the metrological provenance hypergraph the build harvests from the module's members; its hypotheses are the two kernel-decided judgments the build already pins per boundary.
2. **The seal of the computation.** The same guarantee for the computation itself, at every carrier the module is instantiated at: an output depends only on the sources the hypergraph names for it, every constant it uses is one the audit lists, and every kind change in the computation sits at a declared crossing. Stated over the computational tape graph of the tape a carrier-generic module records, related to the provenance hypergraph by a kind-transporting weak bisimulation.
3. **Dimensional homogeneity of the kind algebra.** Every derived kind in a well-formed provenance hypergraph carries the dimension the family rules compute from its operands, so the dimension functor is a homomorphism along every occurrence — not only along the edges an interaction algebra curates.

Four named objects appear in these statements, and this chapter keeps them apart. The metrological provenance hypergraph is the kinded object the first capstone is stated over, harvested from the source, and it is where every kind originates; the value-flow digraph is its binary shadow, the graph reachability is proved on. The tape is not a graph but a record — the sequence of carrier operations a module writes when evaluated at the recording carrier — and the computational tape graph is the directed acyclic graph that record determines, the object the second capstone relates to the provenance hypergraph; it carries no kinds until the kind-transporting weak bisimulation transports them. None of the four is the blueprint dependency graph drawn at the end of this document, which is a picture of the document itself.

This chapter was written blueprint-first, in the discipline the proved spine records after the fact: each theorem was stated informally with its hypotheses and its proof sketched against the declarations that existed, and the declarations were then written to the sketches. Every node now links the declaration that discharges it, and the ledger at the end records what the sketches priced and what remains an instrument rather than a theorem. Two rules govern every statement below. **Non-vacuity is a deliverable, not a hope**: each capstone owes three witnesses — an instance whose hypotheses the kernel decides, a mutant that fails the hypothesis *and* the conclusion, so that the hypothesis is shown to carry the weight, and a pinned axiom profile. **The seal is no undeclared entry, not no entry**: raw data

enters a sealed module through its declared gates — a checked ingest, a constant mint, an attestation with its reason — and the theorem's content is that the build's list of gates is complete.

7.1. The layers and their ladders

Each capstone is stated over one object, and those objects are the calculus's three layers.

- **The provenance layer** — its object the metrological provenance hypergraph: what the source declares. Every node has a kind, every source has a tier, every hyperedge is a use of a law. Its value is that a module's assumptions become one object, and the seal of a module says that object's list of gates is complete for every output.
- **The computation layer** — its object the computational tape graph: what the code computes, at every carrier. Its value is that the seal becomes a claim about the executable rather than about a reading of the source.
- **The dimension layer** — its object the dimension functor over the kinds the hypergraph carries: what the laws in use must satisfy. Its value is that it is the only layer able to refute an authored law. Witnesses and attestations are claims; dimension can say no wherever the kinds carry one.

There is no single ladder from the first layer through the second to the third. There are two ladders, and both stand on the provenance layer, because the computation layer checks that the computation realizes the hypergraph while the dimension layer checks that the laws inside the hypergraph cohere. A rung below both carries the source up to the provenance layer. Each rung's status is the status of the nodes it names, which the dependency graph and the ledger at the end of this chapter carry.

Rung	What carries it	Lost without it
Source to the provenance layer	the boundary audit; the harvest; well-formedness and agreement of the declared boundary, decided per boundary; the reachability reading; the seal of a module	Per-site verdicts never compose to outputs: an anonymous mint two hops upstream of an output is visible only to a reader of the report.
Provenance layer to computation layer	the cone lemma; the bisimulation with its soundness and its strong form after contraction; the denotation bridge and the closure tactic `tape_hom`; the seal of the computation; the denotation of the rendering — one program, printed by every backend	The seal is about the harvest's transcript. A constant the recorder bakes, a path compaction introduces, an opaque callee the harvest saw as one node: all go unseen, and the harvest's attributions stay trusted instead of cross-checked.
Provenance layer to dimension layer	the curated homomorphism; the coverage command's verdict per authored edge; dimensional homogeneity	Outputs rest on declared sources through possibly wrong laws: a wrong edge is a kind-level fact nobody checks.

The kind-transporting weak bisimulation is therefore the relation between the provenance layer and the computation layer. It never touches the dimension layer directly, but that layer's verdicts ride along it: every tape sub-graph that realizes an occurrence realizes a law the dimension layer has judged coherent.

The three layers and the two ladders, drawn on one module — `y = a · b + e`, the module the bisimulation witnesses below decide — seen at three granularities:

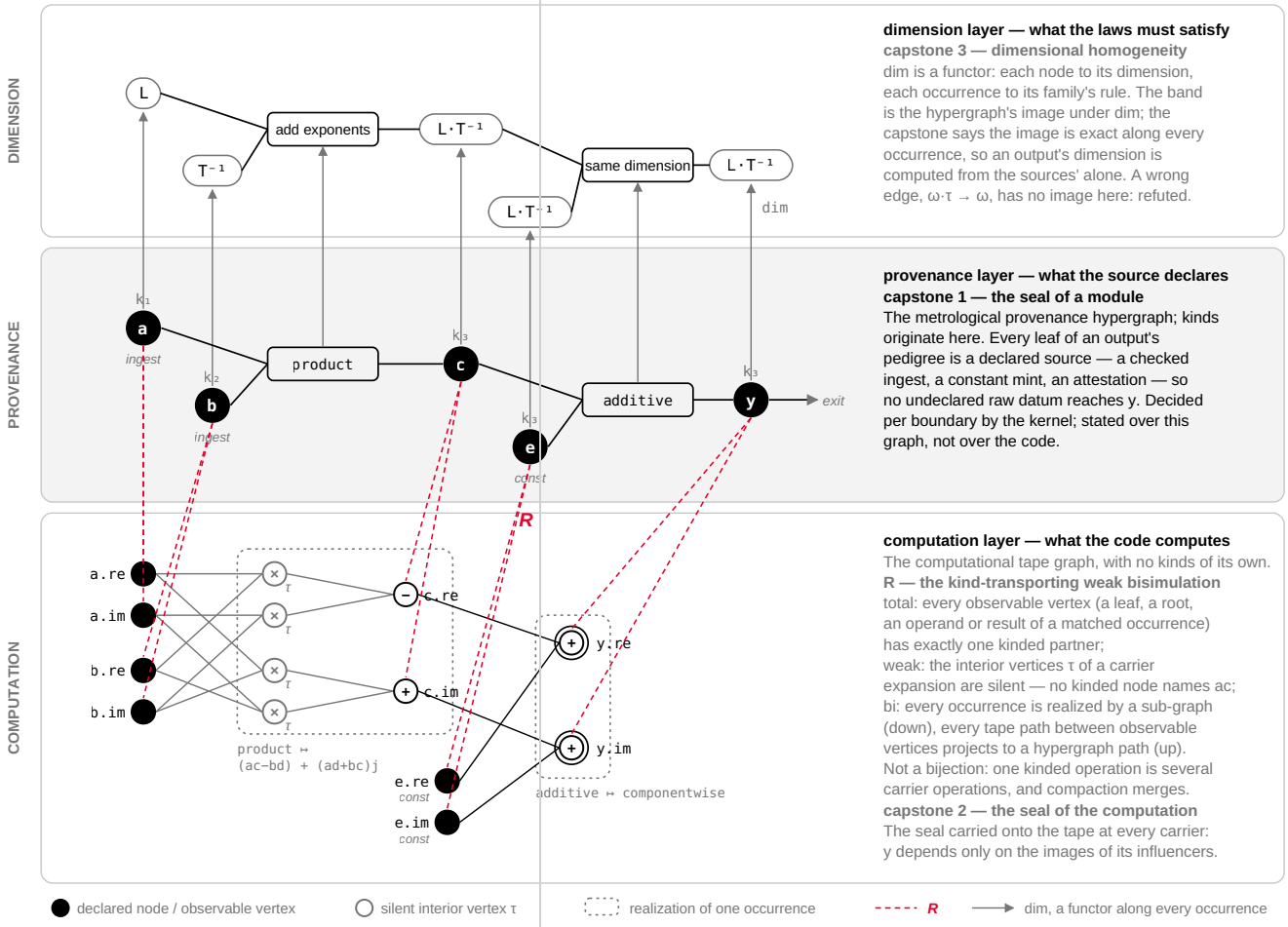

Figure 3. The three layers and their two ladders, drawn on the module `y = a · b + e` at three granularities.

The middle band is the metrological provenance hypergraph, the kinded one. The top band is its image under the dimension functor, node by node and occurrence by occurrence, with the family rule each occurrence must satisfy. The bottom band is the computational tape graph the same module records at a complex carrier, where the one `product` occurrence expands to four multiplications, a subtraction and an addition: the four products are the interior vertices no node of the hypergraph names, the silent steps that make the bisimulation weak, and the `additive` occurrence expands componentwise, with no interior at all. `R` runs from each node to each of its observable components; the dotted boxes are the realizations of the two occurrences.

7.2. The seal of a module — the provenance hypergraph

The first capstone is stated over the metrological provenance hypergraph — the object every audit report presents one relation of. Its nodes are kinded values; its hyperedges are occurrences of the witness families, each relating its operands to its result *in order*, since a quotient's numerator and denominator are different ports of one edge; its sources carry the evidence tier the boundary audit assigns them; and its exits mark where a value leaves the calculus for the bare carrier. It is data — prelude-only, parametric in the node and kind types — so it can be authored by hand in a probe or produced by the harvest: the elaboration-time walk that reads a declared boundary's member definitions and builds their hypergraph from them, ports from the members' signatures, occurrences from the kinded operations in their bodies, sources from the tiers the boundary audit assigns and from the reason each attestation states at its call site. The harvest's output for one boundary is its assembly: the hypergraph together with the signature positions found to carry no kind and the level each node belongs to. The capstone's hypotheses are the two judgments already decided per boundary: structural well-formedness of the hypergraph, and agreement between the declared boundary and the one its members compute. The order matters for what the seal rests on: the audit's tiers and the attestations' reasons are inputs to the harvest, the harvest builds the hypergraph, and the theorem is stated over that hypergraph with the two judgments as its hypotheses — the audits establish the hypotheses, the harvest builds the object, and the theorem gives the seal.

Definition 7.2.1. group uses 0 used by 2 ✓

Structural well-formedness of the provenance hypergraph: every node declared exactly once; every occurrence typed at its family's operand count, with operands and result carrying their nodes' declared kinds; every occurrence result a derivation target — a `derived` introduction or a produced port, never a source; and every derived node, produced port and exit reached from the sources through the conjunctive closure of the occurrences. The last clause is the boundary audit's "raw mints = 0" made compositional: a derived node no occurrence chain produces is an anonymous mint, and an occurrence cycle feeding itself licenses nothing, because the closure starts from the sources. `#kind_assembly_decide` reflects the judgment into a kernel theorem per boundary.

Lean code for Definition 7.2.1. 1 definition

`def` · defined in `PropertyKindCalculus/Provenance.lean` complete

```
def PropertyKindCalculus.Provenance.wellFormed {ν κ : Type} [BEq ν] [BEq κ]
  (g : PropertyKindCalculus.Provenance ν κ) : Bool
```

Proof for Definition 7.2.1 uses 0

Four structural recursions over lists, conjoined; decided by evaluation in a probe and by kernel reduction in a theorem. The sources are the non-produced ports together with the gated and attested introductions (`sources`), and the closure is a fuel-bounded fixpoint that saturates in one sweep per occurrence (`reachableFrom`).

Definition 7.2.2. group uses 0 statement">used by ✓

The pedigree of a node set: everything it is derived from, the set itself included — the backward closure through the occurrences, computed as the forward influence closure of the reversed incidence, one reversed unit occurrence per operand position. The sources among an output's pedigree are its influencers: the term list of an uncertainty budget over the hypergraph's edges, and the source set whose attached conditions a downstream flag may claim for that output. Split by what each source *is*, the same set is the output's assumption ledger: the ports left open at the boundary, the gated ingests, the attested mints with their harvested reasons.

Lean code for Definition 7.2.2. 1 definition

`def` · defined in `PropertyKindCalculus/Influence.lean` — complete

```
def PropertyKindCalculus.Provenance.ancestorsOf {ν κ : Type} [BEq ν]
  (g : PropertyKindCalculus.Provenance ν κ) (of : List ν) : List ν
```

Proof for Definition 7.2.2 uses 0

One engine for both directions: the disjunctive sweep that adds a result when *some* operand is known, run over the occurrence list reversed edge by edge. Fuel is one sweep per reversed occurrence plus one, the sweep that observes saturation.

Theorem 7.2.3. group uses 0 used by 2 ✓

Effort	Tags
small	proved

Membership in the executable pedigree is reachability in the value-flow digraph — the binary shadow of the provenance hypergraph, one vertex per node and one edge per operand-to-result pair of an occurrence: `a` is among the ancestors of `b` exactly when a path of such edges leads from `a` to `b`. The same saturation argument yields the pedigree hop by hop — a member is a root or an operand of some occurrence, and the pedigree is closed under one backward hop — which is how the capstone below is argued while its hypotheses are decided over lists.

Lean code for Theorem 7.2.3.

1 theorem

theorem · defined in PropertyKindCalculus/Graph/Flow.lean

complete

```
theorem PropertyKindCalculus.Provenance.mem_ancestorsOf_iff {ν κ : Type} [BEq
ν]
  [LawfulBEq ν] {g : PropertyKindCalculus.Provenance ν κ} {of : List ν}
  {a : ν} : a ∈ g.ancestorsOf of ↔ ∃ b ∈ of, g.flowDigraph.Reachable a b
```

Proof for Theorem 7.2.3 uses 0

The saturation argument for the forward closure — a sweep only appends, appended nodes are pairwise-distinct occurrence results, so with one sweep per occurrence plus one the closure has either observed a fixpoint or would have outgrown its own bound — applied to the reversed incidence, whose step relation is the converse of the flow digraph's adjacency.

Theorem 7.2.4.

group uses 3 used by 2 ✓

Effort	Priority	Tags
medium	high	capstone, proved

The seal of a module. *Let `g` be a provenance hypergraph and `c` a declared boundary, with `g` well-formed and `c` agreeing with `g` — the two judgments `#kind_assembly_decide` and `#kind_contract_decide` pin (the declared boundary). Then for every produced port or exit `y` of `c`, every node `a` in the pedigree of `y` is either a source of `g` — a non-produced port of `c`, a gated ingest, or an attested mint carrying its reason — or the result of some occurrence all of whose operands are themselves in the pedigree of `y`. Equivalently: the pedigree of an output has no leaf outside the sources, and the assumption ledger the build prints for `y` is the complete list of what `y` rests on. In the paradigm's words, once the module's gates are declared, no other raw datum reaches its outputs.*

Lean code for Theorem 7.2.4.

1 theorem

theorem · defined in PropertyKindCalculus/Graph/Seal.lean

complete

```
theorem PropertyKindCalculus.Provenance.pedigree_seal {ν κ : Type} [BEq ν]
  [LawfulBEq ν] [BEq κ] {g : PropertyKindCalculus.Provenance ν κ}
  (hwf : g.WellFormed) {y : ν} (hy : y ∈ g.outputs) {a : ν}
  (ha : a ∈ g.ancestorsOf [y]) :
```

```
  a ∈ g.sources ∨
    ∃ o ∈ g.occurrences,
      o.result = a ∧ ∀ oc ∈ o.operands, oc.1 ∈ g.ancestorsOf [y]
```

Proof for Theorem 7.2.4 proof">uses 1

Read hop by hop (the two closure facts beside the reachability reading), a node `a` in the pedigree of `y` is `y` itself or an operand of some occurrence, and `occurrencesTyped` makes it a declared node — a port or an introduction. If `a` is a non-produced port or a gated or attested introduction, it is a source. Otherwise `a` is a `derived` introduction or a produced port, and `sourcesReach` places it in the conjunctive closure `known`, so some occurrence produces `a` with every operand in `known`; each such operand is in the pedigree of `y`, which is closed under a backward hop. The hypothesis on `c` enters only to identify the non-produced ports of `g` with the boundary's declared inputs, configuration and parameters, so that the conclusion speaks of the declared interface rather than of whatever the harvest happened to port. The case `a = y` and the step from a known node to a source or an occurrence result are the induction principle of the conjunctive closure (`reachableFrom_sound`, in the core), which the dimension capstone reuses. The executable form — a decidable `undeclaredLeaves g y`, empty exactly under the conclusion — is `sealed_of_wellFormed`, what an instance pins with `decide`; the boundary reading, `seal_of_agrees`, adds the hypothesis on `c`. All three stand beside the reachability reading, in the graph library.

Theorem 7.2.5. group statement">uses 1 by 0 ✓

Effort	Tags
small	proved

Non-vacuity of the seal. *Three witnesses, each a pinned probe. (i) Two instances: an authored hypergraph — the influence probe, labelled by numbers so the kernel decides it — whose well-formedness and seal `decide` closes, with the theorem applied to it; and a harvested one, the interval composition's declared boundary, whose well-formedness and agreement the kernel already decides, for which `#kind_seal_decide` adds the seal theorem by kernel reduction of the executable conclusion at every output. (ii) A mutant: the same hypergraph with one anonymous mint added — a `derived` introduction no occurrence produces, wired into an output — fails `wellFormed` and has a non-empty `undeclaredLeaves` at that output, exactly the mint, so the hypothesis is the discriminating one and the conclusion is not closed by the shape of the statement. (iii) The axiom profile of the seal, pinned to the three classical axioms.*

Lean code for Theorem 7.2.5. 1 theorem

theorem · defined in `PropertyKindCalculus/Tests/Graph/Seal.lean` complete

```
theorem PropertyKindCalculus.Tests.GraphSeal.seal_witnesses :
  PropertyKindCalculus.Tests.GraphSeal.GN.WellFormed ∧
    PropertyKindCalculus.Tests.GraphSeal.GN.Sealed ∧
      ¬PropertyKindCalculus.Tests.GraphSeal.GNmut.WellFormed ∧
        ¬PropertyKindCalculus.Tests.GraphSeal.GNmut.Sealed
```

Proof for Theorem 7.2.5 uses 0

> (i) and (ii) are `decide` theorems over the authored hypergraph and its mutant, conjoined as the linked declaration, and a kernel-decided seal over the harvested one; (iii) is `#print axioms` under `#guard_msgs`. The mutant is the check the methodology asks of every gate — a tactic that closes a goal is not evidence that the hypothesis fired — carried into the capstone itself.

What the first capstone does not claim: it is a theorem about the provenance hypergraph the harvest produced. That the hypergraph is the code — that the harvest attributed every mint, erasure and occurrence to the right node and missed none — is what the boundary audit, the mint ratchet and the unkinded sweep assert about the source, and each of those is an instrument, not a hypothesis of the theorem. The seal of the computation is what turns that residual trust into a cross-check.

7.3. The seal of the computation — the tape

A carrier-generic module has a second record of its computation, one that no harvest produces. The tape is automatic differentiation's word for the record of the operations a program performs, in the order it performs them. TorchLean's autograd tape is that record as data: a grow-only array of nodes, each one operation with the ids of its parents in operand order, its forward value and its local backward rule, and every operation appends exactly one node. The name comes from reverse-mode differentiation, which replays the record backward to accumulate gradients; the calculus never runs that pass. It reads the tape forward — the code generator lowers it, and the evaluator the cone lemma below is about re-computes every node from an environment of leaf values. A tape is therefore a straight-line program, not a graph. A module writes one when instantiated at the recording carrier — the carrier whose values are not numbers but thunks that append their sub-expression to the tape and return the result node. Because the branchless carrier class carries no ordering to `Bool`, a module written against it cannot branch on its data, so one recording covers every input.

The computational tape graph is the directed acyclic graph the record determines: one vertex per tape node, an edge from each parent to its node, the in-edges of a vertex ordered as its parents are — acyclic because every parent precedes its node in the array. Its leaves are of exactly two sorts, a named input and a baked constant; its roots are the recorded outputs; and it carries no kinds, since a value of any kind records the same node. The name pairs it with the metrological provenance hypergraph: one is what the source declares about kinds, the other is what the code computes, and the second capstone is the relation between them. Graph or hypergraph? A tape node *is* its operation, so a vertex's ordered in-edges are the one hyperedge into it, and reading the tape as an ordered hypergraph adds nothing to the

graph. The provenance hypergraph is a hypergraph in a stronger sense: its occurrences are objects apart from its nodes, and several occurrences may derive one node. The compacted tape — the tape after its identical sub-expressions are merged, so that its graph is the graph of the *distinct* sub-expressions rather than of their unfolding — is the very object the code generator lowers to a kernel.

Definition 7.3.1. group uses 0 used by 0 ✓

The recording carrier, as a structure: one deferred build action that, run inside the tape monad, appends this sub-expression's nodes and yields its result node. Every branchless-carrier operation is realized as one appended node, so instantiating a module at this carrier *builds* its tape, with one leaf per named input and one per host constant.

Lean code for Definition 7.3.1. 1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/Torch/Paradigm/TapeCarrier.lean` complete

```
structure PropertyKindCalculus.Paradigm.TapeBuilder (s : Spec.Shape) : Type
```

Proof for Definition 7.3.1 uses 0

Each arithmetic operation runs its operands and emits one node. Sharing is not recorded — a sub-expression named twice emits twice — and the compaction that merges identical sub-expressions is a separate pass over the recorded tape.

Definition 7.3.2. group uses 0 statement">used by ✓

The denotation bridge: a recording-carrier value *evaluates* to a tensor when, on any tape, running it succeeds, its result node carries that tensor, and the run only extends the tape. Each carrier operation preserves the bridge with its matching elementwise operation, proved once per operation, so a module's parity with its own eager value at the same source is the mechanical chaining of those lemmas — and needs no numerical side condition, both carriers using the same clamped operations.

Lean code for Definition 7.3.2. 1 definition

`def` · defined in `PropertyKindCalculus/Torch/Paradigm/TapeParity.lean` complete

```
def PropertyKindCalculus.Paradigm.TapeParity.Evaluates {s : Spec.Shape}
  (b : PropertyKindCalculus.Paradigm.TapeBuilder s)
  (v : TorchLean.Tensor Float s) : Prop
```

Proof for Definition 7.3.2

uses 0

> A four-conjunct predicate; the per-operation preservation lemmas are proved from the tape's forward-value faithfulness lemmas through a run bridge, and a composite kernel's parity is a chain of them.

Theorem 7.3.3.

group statement">proof">used by 1 ✓

Effort	Tags
small	proved

> ***Evaluation reads only the cone.*** *Re-evaluating a tape from an environment of leaf values gives, at every node, a value that depends only on the leaves in that node's backward cone in the tape graph: two environments agreeing on the cone's named leaves give the same value at the node. The tape-side counterpart of [the pedigree reading](#).*

Lean code for Theorem 7.3.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Torch/Paradigm/TapeSeal.lean` — complete

```
theorem PropertyKindCalculus.Paradigm.TapeSeal.evalTape_cone
  (t : Runtime.Autograd.Tape Float)
  (hwf : PropertyKindCalculus.Paradigm.TapeCodegen.WF t)
  {env₁ env₂ : String → Float} {vals₁ vals₂ : Array Float}
  (h₁ :
    PropertyKindCalculus.Paradigm.TapeCodegen.evalTape env₁ t =
      Except.ok vals₁)
  (h₂ :
    PropertyKindCalculus.Paradigm.TapeCodegen.evalTape env₂ t =
      Except.ok vals₂)
  (v : ℕ)
  (hagree :
    ∀ (u : ℕ),
      PropertyKindCalculus.Paradigm.TapeSeal.Reaches
          (PropertyKindCalculus.Paradigm.TapeCodegen.ofTape t) u v →
        ∀ (n : Runtime.Autograd.Node Float),
          t.getNode? u = some n →
            n.parents.isEmpty = true →
              ∀ (nm : String), n.name = some nm → env₁ nm = env₂ nm) :
  vals₁.getD v 0.0 = vals₂.getD v 0.0
```

Proof for Theorem 7.3.3

uses 0

> Strong induction over the vertex id, through the node-value characterisation of the evaluator's fold (`evalTape_node_value`): a leaf reads its own name or constant, and an operation applies its scalar to values already established for its parents, each earlier in the tape and each in the cone.

Definition 7.3.4. group uses 0 used by 2 ✓

The kind-transporting weak bisimulation. The relation that ties the two graphs of one module together: the provenance hypergraph `g`, harvested from the source, where every kind originates, and the computational tape graph of the tape `T` the same module records, which carries none. `R` relates nodes of `g` to vertices of the tape graph. It is total on the *observable* vertices — leaves, roots, and the operands and results of every matched occurrence — and silent on the interior vertices a single kinded operation expands to at the carrier: the real projections of a complex product, the sub-graph of a principal square root, the plumbing an operation registered as carrier vocabulary performs. That silence is what makes it *weak*: an interior vertex is a step neither side observes. It simulates in both directions, which is what makes it a *bisimulation*. Hypergraph to tape: every occurrence of `g` is matched to a tape sub-graph whose frontier is the images of the occurrence's operands and whose operations realize the occurrence's family, per a realization table with one entry per family and carrier. Tape to hypergraph: every path of the tape graph between observable vertices projects along `R` to a path of `g` — the direction the seal consumes. `R` matches leaves to sources, ports by name and constants by value. A `step` edge to a member whose audit tier licenses a re-typing is realized by a wire, and it is the only place the kinds transported along `R` may change across a tape edge. The coarsest reading `R` preserves is the boundary's propagation relation — which source ports reach which produced ports through the occurrences — and comparing that relation, rather than occurrence lists, is what makes the match robust to sharing. `R` is not a bijection, for four reasons present in the code: compaction merges tape nodes many-to-one; one kinded operation is several carrier operations; two uses of one witness are two occurrences by design; and a nominal selection is resolved at recording time into one tape per branch.

The relation is a weak bisimulation in the textbook sense — Sangiorgi's, through cslib's labelled transition systems — between two systems over one label alphabet: a visible step is an edge family with the operand position it enters at, and the silent step is the wiring the harvest records as `copy`, a procedure edge, and every carrier-level interior step. The provenance layer's transitions are the hops of its occurrences; the tape graph's are its edges, labelled through the match. As data the relation is a **match**: a component table — each node of `g` with the tape vertices realizing it, one for a real carrier, two for a complex one — and one realization per occurrence, the interior vertices of its sub-graph. The observable relation is node to component; the interior of a realization is related to the occurrence's result, the result under construction. Acceptance is a decidable judgment on the match, each clause of which some proof below consumes.

Lean code for Definition 7.3.4.

1 definition

```
def · defined in PropertyKindCalculus/Graph/Bisimulation.lean
```

complete

```
def PropertyKindCalculus.Paradigm.Match.weak {ν κ : Type} [BEq ν]
  (g : PropertyKindCalculus.Provenance ν κ)
  (m : PropertyKindCalculus.Paradigm.Match ν) : ν → ℕ → Prop
```

Proof for Definition 7.3.4

uses 0

Decided, never assumed: acceptance is a Boolean over the match, decided by evaluation in a probe and by kernel reduction in a theorem. A matcher that *finds* the match from the leaves of a recorded tape — ports by name, constants by recording site, each occurrence by a realization table per family and carrier — is the instrument the ledger prices, and the refactoring it needs stands: the recorder must name its constant leaves, and either mark the scope of each member call on the tape or the matcher must widen the member set to the closure the tape sees through.

Theorem 7.3.5.

group statement">proof">used by 1 ✓

Effort	Tags
medium	proved

The accepted match is a weak bisimulation. *When acceptance holds for `R` between a well-formed `g` and `T`, [the bisimulation] is a weak bisimulation, in both directions: every hop of `g` from a node is matched, from each of the node's components, by a tape step with the same label up to silent steps, and every tape step from a vertex is matched from every node the vertex is related to. The direction the seal consumes follows as a corollary: every path of the tape graph from a component of a node `s` to a component of a node `b` projects along `R` to a path of the value-flow digraph from `s` to `b` (`reachable_of_tape_path_comps`); and every tape edge between two observable vertices realizes one hop of one occurrence of `g`, with the kinds that occurrence names for its operand and its result (`occurrence_of_edge`) — so the kinds transported along `R` change across a tape edge only where an occurrence of `g` states the change. The tape-side analogue of the lemma that makes the influence probe's `false` a theorem — so an accepted match is evidence rather than a report.*

Lean code for Theorem 7.3.5.

1 theorem

theorem · defined in `PropertyKindCalculus/Graph/Bisimulation.lean`

complete

```
theorem PropertyKindCalculus.Paradigm.Match.isWeakBisimulation_weak {ν κ :
Type}
  [BEq ν] [LawfulBEq ν] [BEq κ]
  {g : PropertyKindCalculus.Provenance ν κ}
  {T : PropertyKindCalculus.Paradigm.TapeGraph}
  {m : PropertyKindCalculus.Paradigm.Match ν} (hwf : g.WellFormed)
  (hA : PropertyKindCalculus.Paradigm.Match.Acc g T m) :
```

```
  g.lts.IsWeakBisimulation
    (PropertyKindCalculus.Paradigm.Match.tapeLts g T m)
    (PropertyKindCalculus.Paradigm.Match.weak g m)
```

Proof for Theorem 7.3.5 proof">uses 1

By factoring rather than by induction over an acceptance derivation: the strong bisimulation with the contracted tape graph composed with the contraction (the contraction lemma), cslib's composition of weak bisimulations. The projection is the tape-to-hypergraph half of the saturated bisimulation, iterated along the path, with the observable vertices at both ends read back to their nodes through injectivity of the component table; the kind clause is covering and closedness of the sub-graphs read at one edge, with `occurrencesTyped` for the kinds.

Definition 7.3.6. group uses 0 statement">used by

The contracted tape graph. Let acceptance hold for `R` between `g` and `T`, so that each occurrence `o` of `g` has a realizing sub-graph `S_o` of the tape graph: its interior, its roots — the components of the result — and its frontier, the components of the operands in operand order. The contraction collapses each `S_o` to one ordered hyperedge from the frontier to the roots, labelled by the occurrence's family and the operand position; a wire is a silent step. It is defined — an ordered hypergraph of the same signature as `g`, with no interior — under the clauses acceptance decides of the sub-graphs: *closed*, every parent of a sub-graph vertex is an interior or a frontier vertex, so no value enters except through the operands; *interior-private*, every edge leaving an interior vertex stays inside `S_o`, so no interior value is read from outside; *progressing*, every interior vertex feeds the sub-graph, so it lies on a path to a root; *used*, every frontier vertex feeds the sub-graph; *disjoint*, the interiors of distinct sub-graphs do not overlap and no interior vertex is a component; and *covering*, every operation vertex of the tape graph lies in some `S_o`. Leaves are matched by recording site, not by value, so that two constants of one value stay two leaves.

Lean code for Definition 7.3.6. 1 definition

def · defined in `PropertyKindCalculus/Graph/Bisimulation.lean` complete

```
def PropertyKindCalculus.Paradigm.Match.contracted {ν κ : Type} [BEq ν]
  (g : PropertyKindCalculus.Provenance ν κ)
  (m : PropertyKindCalculus.Paradigm.Match ν) :
  Cslib.LTS ℕ PropertyKindCalculus.Paradigm.KindLabel
```

Proof for Definition 7.3.6 uses 0

One hyperedge per accepted occurrence, between components; the clauses are Boolean conjuncts of acceptance, and a sub-graph violating one is a match acceptance refuses. The recorder's obligation is the named constant leaf, the same one the bisimulation's definition already prices.

Theorem 7.3.7. group uses 2 proof">used by 1 ✓

Effort	Tags
medium	proved

Contraction makes the bisimulation strong. *Under acceptance and well-formedness, `R` restricted to the observable vertices — node to component — is a strong bisimulation between `g` and the contracted tape graph: every hop of `g` is matched by one hyperedge of the contraction and conversely, with no silent step. The relation is a bisimulation and not a bijection: a node carries as many components as the match gives it, for two reasons — compaction, which matches against a hypergraph quotiented by derivation equality, and nominal selection, which matches per branch. The contraction itself — a component to every component of its node, a root to every interior vertex of its realization — is a weak bisimulation between the contracted and the actual tape graph (`isSWBisimulation_contraction`), and the weak bisimulation between `g` and the tape graph is their composite: the weak bisimulation factors as this strong one composed with the contraction, which is the standard relation between weak and strong bisimulation up to silent-step abstraction. What the strong form deliberately loses is the interior, and the interior is where the numerical side conditions live: the license clause and the adequacy layer keep working on the uncontracted tape graph.*

Lean code for Theorem 7.3.7.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Graph/Bisimulation.lean` — complete

```
theorem PropertyKindCalculus.Paradigm.Match.isBisimulation_contracted
  {ν κ : Type} [BEq ν] [LawfulBEq ν] [BEq κ]
  {g : PropertyKindCalculus.Provenance ν κ}
  {T : PropertyKindCalculus.Paradigm.TapeGraph}
  {m : PropertyKindCalculus.Paradigm.Match ν} (hwf : g.WellFormed)
  (hA : PropertyKindCalculus.Paradigm.Match.Acc g T m) :
  g.lts.IsBisimulation
    (PropertyKindCalculus.Paradigm.Match.contracted g m) m.obs
```

Proof for Theorem 7.3.7

uses 0

The strong half is a corollary of acceptance alone — totality and injectivity of the component table, one realization per occurrence, and well-formedness for the result of every occurrence to be a declared node. The contraction's half is where the sub-graph clauses work: an interior vertex reaches a root

through silent steps (progress, with the tape's order for termination), a frontier vertex enters the sub-graph (used), and a step from an interior vertex stays inside it (interior-private), while an interior vertex is related to nothing observable (disjointness). No new instrument: acceptance already decides every clause.

Theorem 7.3.8. group uses 2 used by 2 ✓

Effort	Priority	Tags
large	high	capstone, proved

The seal of the computation. *Let `f` be a module written once against the branchless carrier class with fixed-extent iteration, `g` its provenance hypergraph with `c` its declared boundary, and `T` the well-formed tape `f` records. Suppose (H1) `g` is well-formed, so that the seal of the module holds of it and, under agreement, of `c`; (H2) acceptance holds for a match `R` between `g` and the computational tape graph of `T`; and (H3) at a carrier, the value of `f` at an output `o` evaluates to the re-evaluation of `T` at the vertex realizing `o`. Then for every node `o` — in particular every produced port — and every vertex `r` realizing it: two environments that agree on the leaves realizing the influencers of `o` — the sources `g` names for it — give the same value at `r`, hence the same value of `f` at that carrier (`semantic_seal_of_denotes`); every leaf in the cone of `r`, a named input or a baked constant, realizes an influencer of `o` — every constant the computation uses is one the assumption ledger lists; and every tape edge between observable vertices realizes one hop of one occurrence of `g`, with the kinds that occurrence names — every kind change in the computation sits under an occurrence the source states. The hypergraph's absence claims become semantic: an input the harvest says cannot reach `o` provably does not, in the computation, at every carrier the bridge reaches.*

Lean code for Theorem 7.3.8.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Torch/Paradigm/TapeSeal.lean` — complete

```
theorem PropertyKindCalculus.Paradigm.TapeSeal.semantic_seal {ν κ : Type}
  [BEq ν] [LawfulBEq ν] [BEq κ]
  {g : PropertyKindCalculus.Provenance ν κ} (hwf : g.WellFormed)
  (t : Runtime.Autograd.Tape Float)
  (hwft : PropertyKindCalculus.Paradigm.TapeCodegen.WF t)
  {m : PropertyKindCalculus.Paradigm.Match ν}
  (hacc :
    PropertyKindCalculus.Paradigm.Match.accepts g
        (PropertyKindCalculus.Paradigm.TapeCodegen.ofTape t) m =
      true)
  {o : ν} {r : ℕ} (hr : r ∈ m.comps o) :
```

```
(∀ (env₁ env₂ : String → Float) (vals₁ vals₂ : Array Float),
    (∀ s ∈ g.influencers o,
        ∀ u ∈ m.comps s,
          ∀ (nm : String),
            (PropertyKindCalculus.Paradigm.TapeCodegen.ofTape t).name?
                  u =
                some nm →
              env₁ nm = env₂ nm) →
      PropertyKindCalculus.Paradigm.TapeCodegen.evalTape env₁ t =
          Except.ok vals₁ →
        PropertyKindCalculus.Paradigm.TapeCodegen.evalTape env₂ t =
            Except.ok vals₂ →
          vals₁.getD r 0.0 = vals₂.getD r 0.0) ∧
  (∀ (u : ℕ),
      PropertyKindCalculus.Paradigm.TapeSeal.Reaches
          (PropertyKindCalculus.Paradigm.TapeCodegen.ofTape t) u r →
        (PropertyKindCalculus.Paradigm.TapeCodegen.ofTape t).isLeaf
              u =
            true →
          ∃ s ∈ g.influencers o, u ∈ m.comps s) ∧
    ∀ (u w : ℕ),
      (PropertyKindCalculus.Paradigm.TapeCodegen.ofTape t).edge u w =
          true →
        ∀ (a b : v),
          u ∈ m.comps a →
            w ∈ m.comps b →
              ∃ oc ∈ g.occurrences,
                oc.result = b ∧
                  ∃ i k,
                    oc.operands[i]? = some (a, k) ∧
                      (g.kindOf? a == some k) = true ∧
                        (g.kindOf? b == some oc.resultKind) = true
```

Proof for Theorem 7.3.8 uses 2

By (H3) the value of `o` at the carrier is the tape's value at `r`; by the cone lemma that value depends only on the named leaves in the cone of `r`; by the accepted match each such leaf is a component of a source whose tape path to `r` projects to a path of the value-flow digraph to `o`, so the source is an influencer of `o` — the assumption ledger (H1) prints. The kind clause is the accepted match's edge reading.

Theorem 7.3.9. group statement"> uses 1 used by 0 ✓

Effort	Tags
medium	proved

Non-vacuity of the seal of the computation. *(i) An instance in which acceptance is decided by the kernel and the recorded tape is the accepted graph: the module* `y = a · b + e` *at the complex carrier over the recording carrier, whose hypergraph has a product and a same-kind sum and whose tape expands the product to* `(ac − bd) + (ad + bc)j` *— four interior vertices the bisimulation is silent on. Acceptance is closed by* `decide`*; the strong bisimulation, the contraction, the weak bisimulation, the cone lemma and the seal of the computation are instantiated on it; and the tape the recorder writes, compacted, projects vertex for vertex onto that graph — a runtime check, since the tape's stored tensors are opaque to the kernel — with the recorded values checked against the seal's value clause. (ii) A mutant: a tape that bakes one constant the hypergraph does not list — acceptance refuses it, at the leaf clause the constant clause rests on. (iii) The axiom profiles of the seal of the computation, the cone lemma and the bisimulation theorems, pinned. The worked model's branchless dielectric chain — whose hypergraph is pinned, whose tape is recorded and whose (H3) is a chain of* `Evaluates` *lemmas — is the instrument the ledger still prices: the match between its two objects is what the recorder decision fixes.*

Lean code for Theorem 7.3.9.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Tests/Graph/Bisimulation.lean`

complete

```
theorem PropertyKindCalculus.Tests.Bisimulation.bisimulation_witnesses :
  PropertyKindCalculus.Paradigm.Match.Accepts
      PropertyKindCalculus.Tests.Bisimulation.g
      PropertyKindCalculus.Tests.Bisimulation.T
      PropertyKindCalculus.Tests.Bisimulation.m ∧
    ¬PropertyKindCalculus.Paradigm.Match.Accepts
        PropertyKindCalculus.Tests.Bisimulation.g
        PropertyKindCalculus.Tests.Bisimulation.Tmut
        PropertyKindCalculus.Tests.Bisimulation.m
```

Proof for Theorem 7.3.9

uses 0

(i) and (ii) are kernel-decided theorems over an authored pair, conjoined as the linked declaration, and a runtime check over the recorded tape; (iii) is `#print axioms`. Where (H3) is only a bit-exact runtime gate — the iterated retrieval, the table lookups — an instance is gated, not proved, and the ledger says so.

Theorem 7.3.10.

group

statement">used 1

used by 0

✓

Effort	Tags
medium	proved

The emitted program denotes as the tape — for every backend at once. *Every emitter — the CUDA kernel, the C stub, the Julia function — is factored as a* ***printer*** *of one backend-neutral straight-line program:* `SL.genSL` *is the only function that reads the tape, and produces one line per node (a read of a named input, an exact literal, an alphabet operation on earlier slots, a table fetch);* `SL.run` *gives the program its meaning in Lean's* `Float`*, with* `cOp`*'s own clauses (*`BinOp.cOp_eval`*,* `UnOp.cOp_eval`*). The theorem: if* `genSL` *produces a program for a tape, the table-extended interpreter* `evalTapeT` *succeeds on that tape and returns exactly* `SL.run` *of the program — slot* `i` *of the emitted kernel denotes node* `i` *of the tape, for every node; and conversely (*`SL.genSL_isOk_iff_evalTapeT`*)* `genSL` *produces a program exactly when the interpreter runs the tape, so no tape the seal is stated over is left un-emittable and no program is emitted for a tape the interpreter rejects. With it the two clauses the previous reading of this node could only describe are theorems about the program: a literal occurs in the emitted body exactly at an unnamed leaf and is that leaf's stored scalar (*`SL.genSL_lit_iff`*), an input read exactly at a named leaf with that leaf's name (*`SL.genSL_input_iff`*) — equivalences, both directions — and the program's own graph is the tape's:* `SL.ofProg` *projects a program onto a computational tape graph as* `ofTape` *projects a tape, and* `SL.ofProg_eq_ofTape` *says the two are equal when the program is* `genSL` *of the tape, so [the seal of the computation] holds of the emitted program alone (*`TapeSeal.semantic_seal_program`*: the match accepted against the program's graph, the values read off* `SL.run`*, the tape only in the hypothesis that the program came from it) — no number enters the emitted code that is not a leaf of the tape, and no argument that is not a named leaf. The per-backend rendering tables (*`rendering_table_binary/_unary`*, for C and for Julia) and the agreement of the two emitters on where they are defined (*`jlExpr_isOk_eq_cExpr`*, over the whole alphabet, and* `genBodyJulia_isOk_eq_genBody`*, over every tape) are corollaries of the factoring.*

The technique, since it is the point. Before the factoring, each backend walked the tape itself and wrote text, so "the emitted kernel computes what the tape computes" was a claim per backend about strings. Now a backend is a total function `SLExpr → String` *(*`printC`*,* `printJulia`*) that cannot see the tape — it is handed the program and nothing else — so it cannot introduce an input, a literal or an operation the program, hence the tape, hence the sealed hypergraph does not have. The denotation theorem is proved once, about the one tape reader, and is inherited by every printer with no proof per backend; what a backend must still be trusted for shrinks to "each of the printer's dozen fixed forms means, in the host language, what* `SLExpr.denote` *means" — a statement about* `v3 * v4`*,* `ifelse(a <= b, a, b)` *and* `expf(v9)`*, never about a kernel. A new target is a printer: a few lines and the same theorem. And "what* `SLExpr.denote` *means in Lean's* `Float`*" is itself a specification, not an oracle: on the pinned toolchain* `Float` *is a structure over* `Float.Model`*, core's bit-level IEEE 754 binary64 model, and every op of the alphabet but* `exp`*/*`log` *unfolds by* `rfl` *to model arithmetic (*`examples.tape_host_float_model`*), so the obligation reads "the host's binary64 + rounds to nearest even" — a conformance claim about the host. The host compiler, its* `exp`*/*`log` *(the two* `opaque` *externs, the whole of "libm"), the absence of FMA contraction, and (CUDA) the* `float` *narrowing of the double program remain the trusted base*

> *each printer names and the parity executables measure; the methodology chapter, Julia, emitted, says so for the Julia host.*

Lean code for Theorem **7.3.10.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Torch/Paradigm/TapeStraightLine.lean` complete

```
theorem PropertyKindCalculus.Paradigm.TapeCodegen.SL.genSL_denotes
  {tbls : Array PropertyKindCalculus.Paradigm.LutTable}
  {tables : String → Option PropertyKindCalculus.Paradigm.LutTable}
  (hT :
    PropertyKindCalculus.Paradigm.TapeCodegen.SL.TablesAgree tbls
      tables)
  (env : String → Float) {t : Runtime.Autograd.Tape Float}
  {prog : PropertyKindCalculus.Paradigm.TapeCodegen.SL.SLProg}
  (h :
    PropertyKindCalculus.Paradigm.TapeCodegen.SL.genSL t tbls =
      Except.ok prog) :
  PropertyKindCalculus.Paradigm.TapeCodegen.evalTapeT tables env t =
    Except.ok
      (PropertyKindCalculus.Paradigm.TapeCodegen.SL.run env prog)
```

Proof for Theorem 7.3.10 uses 0

`evalTapeT` is reformulated as a left fold of its step function over the nodes (`evalTapeT_eq_foldlM`). The node law `nodeSL_denote`: for a node the generator accepts, the interpreter's step equals the denotation of the generated line against the same slots — by cases on the node (leaf or not, named or not, a fetch or an alphabet op), the fetch case using the agreement of the resolved tables with the interpreter's table function and the op case the `cOp_eval` clauses through `mkOp_denote`. The fold law `foldlM_run_aux` lifts the node law along the list of nodes `mapM` produced lines for, by induction with the slot array generalised; the theorem is its instance at the empty array, read back through `Array.foldlM_toList` and `Array.toList_mapM`. Completeness is the same induction on the failing side: `mkOp` fails only where `cOp` fails (`cOp_error_of_mkOp_error`, by splitting the interpreter's string match), a fetch fails only where `cOpT` does, and a failing `mapM` makes the fold fail (`foldlM_error_of_mapM_error`). The literal and input clauses read slot `i` of the program back to node `i` (`genSL_getElem?`, a `mapM`-indexing lemma) and case on the generator; their converses read node `i` forward to slot `i` (`genSL_getElem?_of_node`, the same lemma the other way) and unfold the generator on a leaf.

What the second capstone changes, and what it leaves. It covers bits and, through `R`, kinds: the tape graph's kinds are the hypergraph's, transported, and the matcher checks the hypergraph's occurrence structure — each family against the operation that realizes it — against an instrument the harvest does not share. Read as a guarantee about the code: every occurrence maps to a tape sub-graph realizing its

family, every tape path between observable vertices projects to a hypergraph path, and the kinds transported along `R` change only at the image of a declared crossing — so the computation is kind-preserving in exactly that sense, on a graph that carries no kinds of its own; and speaking of values at every carrier needs (H3), the denotation bridge, beside it. What remains trusted is smaller: that the harvest read each node's kind off the kernel-checked term at the right node, since `R` transports kinds and does not re-derive them; and the code generator and toolchain below the tape, which the deployment template lists as such.

7.4. Dimensional homogeneity along the hypergraph

Three results grow in scope here, and the third capstone is the jump from the first two to the third. The curated homomorphism is proved: for a curated interaction algebra, whenever the algebra sanctions a product, the result's dimension is the product of the operands' dimensions. It holds because the algebra carries that side condition in its definition, so it is per algebra, and only for the product and quotient edges an author chose to mirror into one. The coverage command is mechanical: it walks every authored edge under a namespace across all nine witness families and evaluates each family's dimensional rule. It is total, but per edge, and its output is a report rather than a theorem. The capstone below composes both: given a well-formed hypergraph whose occurrences all carry coherent families, dimension is a homomorphism along every occurrence and hence along every path, so the dimension of every derived node and every output is the one computed from the sources' dimensions by composing the rules. A per-edge verdict says each step balances; the capstone says the whole model balances end to end — dimensional analysis of the entire algorithm as a theorem — and it covers the families the model actually uses, transcendental, power, copy and difference among them, which no interaction algebra curates. In the paradigm's words: the seal of a module says nothing undeclared enters, and homogeneity says every law traversed coheres, so an output's kind rests on declared sources through coherent laws.

Theorem 7.4.1. uses 2 used by 0 ✓ L∃∀N

Effort	Priority	Tags
medium	high	capstone, proved

> ***Dimensional homogeneity along the hypergraph.*** *Let `g` be a well-formed provenance hypergraph and `dimOf` the declared dimension of each kind, such that the declared assignment of dimensions to nodes is homomorphic along every occurrence — the family's rule, applied to the operands' dimensions, gives the result's: a product adds the operand exponents, a quotient subtracts them, a reciprocal negates them, a power scales them by its exponent, a transcendental demands dimension one, a copy and a reference preserve, a same-kind sum demands equal operands — the verdict the coverage command reports as `[coherent]`, stated per occurrence and decided by the executable `coherent`. Then every assignment that follows the rules along every occurrence from the same source*

> *dimensions agrees with the declared one on every known node — every derived node, produced port and exit: the dimension of every output is the one the rules compute from the sources' alone, and the dimension functor is a homomorphism along every path of `g`. Stated over any rule algebra in the core (`Provenance.homogeneity`) and instantiated on PhysLib's dimensions with the coverage command's rules (`dimRule`); the hypergraph-level, total form of [the curated homomorphism], which is per edge and per algebra.*

Lean code for Theorem 7.4.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/DimensionalHomogeneity.lean` — complete

```
theorem PropertyKindCalculus.DimensionalHomogeneity.dimensional_homogeneity
  {B : Type} [DimensionBasis B] [Fintype B] {ν κ : Type} [BEq ν]
  [LawfulBEq ν] [BEq κ] {g : PropertyKindCalculus.Provenance ν κ}
  (hwf : g.WellFormed) (dimOf : κ → Option (Dimension B))
  (hcov :
    ∀ o ∈ g.occurrences,
      PropertyKindCalculus.Provenance.Homomorphic
        PropertyKindCalculus.DimensionalHomogeneity.dimRule
        (PropertyKindCalculus.Provenance.declaredDim dimOf g) o)
  {δ : ν → Option (Dimension B)}
  (hδ :
    ∀ o ∈ g.occurrences,
      PropertyKindCalculus.Provenance.Homomorphic
        PropertyKindCalculus.DimensionalHomogeneity.dimRule δ o)
  (hsrc :
    ∀ s ∈ g.sources,
      δ s = PropertyKindCalculus.Provenance.declaredDim dimOf g s) :
  (∀ x ∈ g.known,
      δ x = PropertyKindCalculus.Provenance.declaredDim dimOf g x) ∧
    (∀ y ∈ g.outputs,
        δ y = PropertyKindCalculus.Provenance.declaredDim dimOf g y) ∧
      ∀ i ∈ g.intros,
        i.tier = PropertyKindCalculus.Provenance.IntroTier.derived →
          δ i.node =
            PropertyKindCalculus.Provenance.declaredDim dimOf g i.node
```

Proof for Theorem 7.4.1

uses 0

> Induction over the conjunctive closure from the sources (`reachableFrom_sound`, the same principle the seal of a module rests on), one case per occurrence, closed by the rule: two homomorphic assignments agreeing on the operands agree on the result. Well-formedness supplies that every

> derived node and every output is in the closure. The coverage rows as hypergraph data are the declared assignment `declaredDim dimOf g` and the per-occurrence verdict `coherent`, which `coherent_iff` identifies with the hypothesis. Non-vacuity: the refuted `L · L → L` edge as a mutant occurrence that fails the verdict and, together with it, the conclusion — the rule-following assignment gives the output L^2, not the declared `L` (`homogeneity_witnesses`). What remains an instrument is reading `dimOf` off the environment's `DimensionedKind` registry for a harvested hypergraph, which the coverage command does by elaboration and the theorem takes as a hypothesis.

7.5. Status ledger

Done — what each capstone rests on, and the capstones themselves.

- The provenance hypergraph with its well-formedness judgment, the pedigree closure and its reachability reading, the boundary judgments `agrees` and `discharges`, and the per-boundary kernel theorems the worked model pins for each of its declared boundaries.
- The recording carrier, the compaction pass, the code generator, and the denotation bridge with whole-kernel parity theorems for the branchless dielectric chain, its reflectivity, and the fit's forward, residual and Jacobian kernels.
- The Julia emitter beside the CUDA/C pair, with its rendering table over the whole alphabet and the `min`/`max` reading as Lean's `ite`; the coverage and ABI `#guard`s on the recorded AVS tape; the `julia_parity` executable that runs the emitted file against the tape evaluator.
- The denotation of the rendering, for every backend at once: the one straight-line program `genSL` reads off the tape, the three emitters as printers of it, `genSL_denotes` (the program runs as `evalTapeT`, node for node), its completeness (`genSL_isOk_iff_evalTapeT`: a program exists exactly when the interpreter runs the tape) and the literal and input clauses `genSL_lit_iff`/ `genSL_input_iff` as equivalences about the emitted program.
- The seal of the computation over the emitted program: `ofProg`, the program's graph, equal to the tape's under `genSL` (`ofProg_eq_ofTape`), and `semantic_seal_program`, the second capstone restated with the program's graph and `SL.run` in place of the tape and `evalTape`.
- Slot discipline on the program: every operand slot of line `i` precedes it on a well-formed tape (`genSL_slots_lt`, `ofProg_ordered`); the generators decide that invariant (`checkWF`, `gen_wf`, `genJulia_wf`) and reject an output id outside the tape (`checkOutputs`, `gen_outIds_lt`, `genJulia_outIds_lt`) — the printed kernels are well-scoped, for any tape handed to them.
- The scalar-leaf obligation: the generators decide that every constant leaf stores exactly one scalar (`checkScalarLeaves`, `gen_scalar_leaves`, `genJulia_scalar_leaves`), so the literal baked at a const leaf (`genSL_lit_iff`) is the leaf's whole stored value, never the head of a buffer read as `0.0`.
- The chain composed: `pipeline_denotes` (`examples.tape_end_to_end_chain`) — for every well-formed tape, const leaves included, tape, compacted tape and emitted program hold the same value at every original node; the program's existence supplies the compacted tape's evaluation,

and Batteries' `Float.toBits_inj` (`cseCompact_denotation_unconditional`) the merged-const-leaf case.

- The host `Float` as a specification: `Float` is a structure over core's `Float.Model` (IEEE 754 binary64, round-to-nearest-even), and `BinOp.eval`/`UnOp.eval` unfold by `rfl` to model arithmetic for every op of the alphabet but the two `opaque` externs `exp`/`log` (`examples.tape_host_float_model`), so the per-printer obligation is the host's IEEE conformance, not a claim about Lean.
- One `min`/`max` reading on NaN across the interpreter and the three renderings: Lean's `ite`, written as the ≤-ternary in C and CUDA and as `ifelse` in Julia; the table fetch's clamp likewise follows `refFetch` in every rendering.
- The dimension functor, the curated homomorphism, and the coverage command with its pinned refutation of an incoherent edge.
- The three capstones and their supporting lemmas, each linked above: the seal of a module in the graph library, with its executable form, its boundary reading and the induction principle of the conjunctive closure it and the third capstone share; the kind-transporting weak bisimulation in cslib's labelled transition systems, factored as the strong bisimulation with the contracted tape graph composed with the contraction; the cone lemma and the seal of the computation on the tape evaluator; dimensional homogeneity over any rule algebra, instantiated on PhysLib's dimensions. Each with its instance, its mutant and its pinned axiom profile.

Open — each item with whose move it is.

- Capstone 2, the matcher: acceptance is decided on a match a probe authors. A matcher that finds the match from the leaves of a recorded tape — ports by name, constants by recording site, each occurrence by a realization table per family and carrier — is the instrument to run on the dielectric pair, where the hypergraph, the tape and the parity theorem all exist, reading what disagrees before the seal of the computation is claimed of the worked model; the expected disagreement classes are the four named in the bisimulation's definition (`cap_def_bisimulation`). **Library and worked model.**
- Capstone 2, the recorder decision: scope markers per member call, or membership widened to the closure the tape sees through — the choice that fixes the matcher's cost, and what makes an assembly's procedure edges matchable beside the occurrences they summarize. **Owner.**
- Capstone 2, the iterated retrieval's parity theorem, today a bit-exact runtime gate. **Worked model.**
- Capstone 3, the declared dimensions from the environment: the coverage command resolves each kind to its `DimensionedKind` by elaboration; the assignment the capstone consumes is authored per probe today, and the walk that reads it off the registry for a harvested hypergraph is the bridge from the command's report to the theorem's hypothesis. **Library.**

7.6. What the capstones do not claim

- The host that receives an emitted kernel. An emitter hands a host the program and nothing else — the denotation of the rendering says the program runs as the tape and has the tape's graph, so the seal is a theorem about the program (`semantic_seal_program`); that the printed text means in the host language what the program means in Lean's `Float` — which is core's IEEE binary64 model for every op but `exp`/`log` (`examples.tape_host_float_model`), so a conformance claim about the host — the host's compiler, its `exp`/`log`, and every line of the host's own model are its trusted base, named in the methodology chapter and measured — not proved — by `julia_parity` and the landing stubs.
- The truth of an attestation. It is a hypothesis of the model; the seal says only that the build's list of them is complete. Where the attestation defers a requirement, the requirement clause counts it, with its reason, as one the apparatus still owes.
- The harvest's kind readings. The bisimulation transports them and does not re-derive them.
- Anything below the tape — the code generator, the toolchain, the foreign interface — which the deployment template lists as what remains trusted.
- Accuracy against nature, which needs a true value the kernel never has; that boundary is the uncertainty chapter's, and the requirement clause is where a demand of that kind is stated as an empirical requirement against a named referent, gated or attested, rather than left unsaid.

8. Foundations: system, part, and the ontological square

PKC's foundation is a combination of two sources. Dybkær's ontology on property
9. (Dybkær, 2004)[9] René Dybkær, 2004. *An Ontology on Property for Physical, Chemical, and Biological Systems* (https://ontology.iupac.org/). PhD Thesis, University of Copenhagen supplies the metrological content — system, component, property, kind-of-property, scale, examination — and the discipline that every construct be a terminological commitment a model states rather than a convention a
10. reader infers. Lowe's four-category ontology (Lowe, 2005)[10] E. J. Lowe (December 2005). "The Four-Category Ontology: A Metaphysical Foundation for Natural Science" (https://doi.org/10.1093/0199254397.001.0001). *Oxford University Press.* . supplies the categorial frame: which of those constructs is a universal and which a particular, and along which relations they meet. The combination is load-bearing, and this chapter says where: Dybkær's text runs part–whole structure at two levels, in two vocabularies, without ever relating them, and the square is what relates them. A kind is ultimately a kind **of** some system — in the universal or the particular sense — and which sense a construct consumes decides what may be summed, what may be counted, and what a whole even is. Those decisions are made here, before the first aggregate, rather than discovered mid-proof.

8.1. The mereology Dybkær implies

The word *mereology* does not occur in Dybkær's text. The structure does, at two levels, and the levels never meet.

At the level of **concepts**, the vehicle is the *partitive relation* that ISO 704 and ISO 1087-1 provide and Fig. 2.1 (item 2) sets in notation: a *comprehensive concept* over its *partitive concepts*, beside the generic and associative relations. Dybkær puts it to work as a relation among universals. The quantity taxonomy itself is drawn with partitive rather than generic edges (Fig. 6.9); the representation of a unitary quantity value has the numerical value and the metrological unit as coordinate concepts in partitive relation to it (Fig. 9.25); a property-value scale stands in partitive relation to ⟨property value⟩ (§11.8). And he **rules** with it: ⟨property⟩ over ⟨property value⟩ cannot be comprehensive-over-partitive, because a property value is not a detachable piece of a property (§9.13.2).

At the level of **systems**, the vocabulary is plain part-talk, and it is load-bearing from the opening of Chapter 3. ⟨system⟩ was adopted "to accommodate any partition into components" (§3.2, where Bunge is quoted: "there are no simple structureless entities"). The definition itself is mereological — a system is a "demarcated arrangement of a set of elements and a set of relationships or processes between these elements" (§3.3), an element being "any definable part except a relationship or a process" (§3.3 Note 3) — and its worked examples are a six-deep chain of parts: a human being, the blood of that human being, the leukocytes of that blood, a single leukocyte among them, the DNA of that leukocyte, a given gene in that DNA. ⟨component⟩ is then defined as "part of a system" outright (§3.4), with the

complementary part — the chemist's *matrix* — named beside it (§3.4 Note 2), and the improper case, system and component "designated identically", admitted (§3.4 Note 3). Fig. 3.5 splits ⟨object⟩ into composite and non-composite. The relation even carries definitional weight one level up: the minimal definition of ⟨property⟩ — "inherent feature of a system" — is available exactly because "a relevant component is a part of a system and therefore might not need special mention" (§5.5.1).

Two further facts fix how these two levels must be read, and both are Dybkær's own. First, they are genuinely two: the partitive relation holds between concepts — universals — while a component is part of a particular system, and nothing in the text identifies the two relations or maps one onto the other. Second, at the one point where the levels could have been collapsed, he refuses. Asked whether ⟨object⟩ stands in a partitive relation to its properties — whether properties are the parts objects are made of — §2.23.3 answers that "the description of a biological object by a set of all its properties hardly means that a duplicate instance can be created by putting the properties together", and the object–property relation is fixed as associative (§3.1, Fig. 2.25). That is a refusal of the bundle theory in one sentence: properties **characterize** objects, they do not **compose** them. What is needed, then, is a frame that keeps properties and their bearers in distinct categories related by characterization rather than parthood, distinguishes universals from particulars on both sides, and leaves room for a part–whole relation among the particulars and another among the universals without confusing either with instantiation. That frame exists, and it is Lowe's.

8.2. The ontological square

Definition 8.2.1. group uses 0 used by 5 ✓

A *system* (Dybkær §3.3) is a demarcated arrangement of elements and their relationships; *object* (§3.3 Note 6) is given as a synonym. Specified abstractly by identity: what parts a system has is deliberately not a field, because Dybkær's own Note 5 makes the parts observer-relative — "the extent and structure of a system is essentially defined by the observer for some purpose" — so parthood arrives as a *carving*, below, rather than as stored structure.

Lean code for Definition 8.2.1. 1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/Foundations.lean` complete

```
structure PropertyKindCalculus.System : Type
```

Proof for Definition 8.2.1 uses 0

A one-field `structure` with `DecidableEq`; nothing to prove. The synonym `Object := System` keeps the instance layer reading as "characterizes an object".

Definition 8.2.2. group uses 0 used by 5 ✓

The *sort of system* — the substantial **universal** as against the substantial **particular**: *plasma* as against this sample, *rover* as against rover 1. Dybkær's dedicated kind-of-property is defined over exactly this — "kind-of-property with given **sort of** system and any pertinent component" (Ch. 20, quoted from the source's own definition) — and the mereological registry is keyed by it, because how parts unify into a whole is dictated by the sort of the whole.

Lean code for Definition 8.2.2. 1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/Foundations.lean` complete

```
structure PropertyKindCalculus.SortOfSystem : Type
```

Proof for Definition 8.2.2 proof">uses 1

A one-field structure with `DecidableEq`, mirroring the system carrier one ontological level up. `Sorted O` is the instantiation arrow from an object type to its sorts (`sortOf : O → SortOfSystem`), declared per model and never derived. With these, the calculus carries all four corners of Lowe's ontological square (the correspondence is spelled out edge by edge below) — of which three were already present and the fourth was being played, by convention, by the particular. `DedicatedKind.sort` stores exactly what the quoted definition asks for, and `dedicatedFor` closes the square: dedicating a kind through an object contributes the object's sort, so two objects of one sort instantiate one catalogue entry while their quantities stay apart by type.

Definition 8.2.3. group uses 2 statement">used by ✓

An *individual quantity* is a magnitude of a kind-of-property that *characterizes an object* (Ch. 3), with the object carried in the type alongside the kind. Its object is drawn from an arbitrary type, so a nominal system and a host library's own structural object are both admissible indices — the census of admissible shapes is the object-type chapter's subject.

Lean code for Definition 8.2.3. 1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/IndividualQuantity.lean` complete

```
structure PropertyKindCalculus.IndividualQuantity.{u} {O : Type u} (o : O)
  (k : PropertyKindCalculus.KindOfProperty) (R : Type) : Type
```

Proof for Definition 8.2.3 uses 0

A one-field structure `IndividualQuantity {O : Type u} (o : O) (k : KindOfProperty) (R : Type)`. `add` is gated on the object, the kind, and a `DifferenceKind` scale witness; `mul`, `div` and `recip` on the object and a kind law. None of these reads `o`, which is why the generalization from `Object` to an arbitrary `O` changed no proof.

Definition 8.2.4. group statement"><statement">used by

A *designated* object type carries a map into the nominal system type, injectively, so an individual quantity of such an object can name it. The nominal type designates itself; a type whose objects are positions in a structure has no instance.

Lean code for Definition 8.2.4. 1 definition

class (2 methods) · defined in `PropertyKindCalculus/Foundations.lean` complete

```
class PropertyKindCalculus.Designated.{u} (O : Type u) : Type u
```

Proof for Definition 8.2.4 uses 0

A two-field class: `designation : O → Object` and `designation_inj`. The injectivity field is the whole content. Without it, any object type is satisfiable by the constant map, and the resulting designation reports two distinct objects as one system — silently, wherever it is rendered. With it, the absence of an instance is a true report about the type.

11. The correspondence with Lowe's four-category ontology ((Lowe, 2005)[11] E. J. Lowe (December 2005). "The Four-Category Ontology: A Metaphysical Foundation for Natural Science" (https://doi.org/10.1093/0199254397.001.0001). *Oxford University Press.* . , Fig. 7.1 — the ontological square) is worth stating edge by edge, because the calculus carries the **relations** of the square and not only its corners. The corners: `SortOfSystem` is his *Kinds* (the substantial universal), the object index his *Substances* (the substantial particular), `KindOfProperty` his *Attributes* (the non-substantial universal), `IndividualQuantity` his *Modes*. The edges: `Sorted.sortOf` is the left *instantiated by* edge — substances instantiate kinds. The right *instantiated by* edge — modes instantiate attributes — is the kind index of an individual quantity, and the bottom *characterized by* edge — substances characterized by modes — is its object index: both edges into the mode are carried in the **type** of an individual quantity, which is what puts the arithmetic under the type checker — two quantities disagreeing on either edge are of two different types, so nothing combines them, and the refusal is the compiler's rather than a convention's. The top *characterized by* edge — kinds characterized by attributes — is the dedicated kind (the dedicated-kinds chapter), refined by Dybkær's pertinent component, a refinement the square itself does not carry.

Figure 1, up front, draws the square with the calculus's identifiers at its corners and these mechanisms on its edges, and two things it shows are worth reading against the correspondence just stated. The two heavy edges are the two type indices of an individual quantity — the object index `o` along the bottom and

the kind index `k` down the right — and the plates behind the *Modes* corner are the carrier `R`, which indexes that corner alone: `R` is no ontological category and has neither a corner nor an edge, so a change of carrier stacks copies of the mode and moves nothing else. The columns and rows are braced because the square's whole content is a two-by-two — universals over particulars, substantial beside non-substantial — and naming the columns keeps the two senses of *kind* apart: Lowe's *Kinds* corner is a kind of **system**, while the `k` of `Quantity k R` is a kind-of-property, the *Attributes* corner.

A word on *index*, which this chapter uses throughout. In Lean's own vocabulary the arguments of `IndividualQuantity` are structure *parameters*: a structure has no indices, those being the arguments an inductive declares to the right of the colon and may vary from one constructor to the next. The prose says *index* deliberately, because what the correspondence needs to name is the role rather than the declaration form — each argument is fixed in the type of any given quantity, so that two quantities differing in one of them are of two different types, and that is the property the square's edges are identified with here. A reader who wants the declaration's own term should read *parameter* wherever this chapter writes *index*.

The diagonal is a structural agreement rather than a construct. Lowe's *exemplified by* — a substance exemplifying an attribute — is derivative for him, factoring through either path around the square, and the calculus likewise has no primitive object-to-kind-of-property construct: the dispositional route (up the left edge, then across the top) is `dedicatedFor`, and the occurrent route (across the bottom, then up the right) is an inhabitant of `IndividualQuantity o k R`. The reading is not idiosyncratic: surveying the square's grounding relations, Simons states the occurrent half as a definition — an object exemplifies an attribute "when a mode which instantiates an attribute characterizes an object" — and counts exemplification "definable in terms of instantiation, characterization, and some logic" (Simons, 2018)[12]. Where the calculus narrows Lowe: the attributes here are kinds-of-property specifically — quantitative attributes carrying a scale and an examination — not attributes at large.

One stance the implementation takes silently is worth making explicit. The calculus's universals are catalogue **entries**: a `SortOfSystem` and a `KindOfProperty` are data, declared by a model, and a sort no object instantiates is idle data rather than a Platonic surplus. That is the immanentist reading Lowe himself settled on — universals as "abstractions from, or invariants across, particulars", incapable of existing uninstantiated (Lowe 2012, quoted in Heil (Heil, 2018)[13]) — and it is the only reading a terminological ontology in Dybkær's style needs.

12. Peter Simons, 2018. “Lowe, the Primacy of Metaphysics, and the Basis of Categorial Distinctions”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).* .

13. John Heil, 2018. “Existents and Universals”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).* .

8.3. The carving — mereology as data, wholes by sortal

With the square in place, the two levels of Dybkær's implied mereology get one carrier each, and the flexibility both sources demand is a theorem-shaped commitment rather than a mood. Marmodoro's distinction says why flexibility is forced: "physical structure unites; while metaphysical structure unifies. The former brings about wholes, the latter unities. But wholes are not always unities" — and a physical structure is "numberless", bringing no count principle with it: "Alternative carvings of the world deliver alternative numbers of entities" (Marmodoro, 2018)[14] . The particular-side mereology is therefore **data**, quantified over: a carving of a system, never the carving. Unification — being one whole of a sort, with a license to aggregate — is a separate act, supplied when a model declares a sort and instantiates a composite (the object-type chapter), "under the individuation principle of the sortal"; Dybkær's *sort of system* is that sortal under another name. The host library this work targets confirms the design from the other side: its rigid body is a single mass functional whose parts occur only as bound points inside integrands, so a foundation that made the parts of a system a stored fact would misdescribe the very objects it aims at.

14. Anna Marmodoro, 2018. “Whole, but not One”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).*

Definition 8.3.1. group statement">used by 4 ✓

A *carving* of a whole into parts drawn from an arbitrary type — a system's subsystems, a host library's own particles — as a binary tree of joins. Deliberately not a census: nothing in a carving claims the parts are disjoint, exhaustive, or all there are, and nothing fixes how many parts a system has. It is Dybkær's §3.2 "partition into components" with §3.3 Note 5's observer-relativity taken at face value: the carving is the observer's act, so every law that consumes one is quantified over all of them.

Lean code for Definition 8.3.1. 1 definition

inductive (2 constructors, 1 parameter) · defined in PropertyKindCalculus/Mereology.lean — complete

```
inductive PropertyKindCalculus.Decomposition.{u} (O : Type u) : Type u
```

Proof for Definition 8.3.1 uses 0

A two-constructor inductive `Decomposition O` — `atom` and `union` — with `fold` as its one eliminator, realized in the Mathlib-free core as the `Mereology` module, which imports nothing at all: the carving is pure structure, and the laws that consume it (the `Extensivity` module) import it rather than housing it. The abstinence is load-bearing: `coalesce_count_ne` and `count_sortal_ne` (the extensivity chapter) prove that a part count is not a function of the whole, which is exactly what a carving-as-census would wrongly supply.

Definition 8.3.2. group statement">us

Part and whole as roles. A portion of a thing is a quantity of the **same** kind as the thing — which is what leaves an interface taking a portion and its total open to a silent argument swap. `Part k R` and `Whole k R` put the two mereological roles into the type; `Part.WithinWhole` — the portion does not exceed its total — is the only relation between them, stated with the part on the left, where the mereology puts it, so the reversed reading is unwritable rather than merely wrong. Dybkær's improper case (§3.4 Note 3, system and component "designated identically") is the pair wrapping one quantity.

Lean code for Definition 8.3.2.

1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/PartWhole.lean` — complete

```
structure PropertyKindCalculus.Part (k : PropertyKindCalculus.KindOfProperty)
  (R : Type) : Type
```

Proof for Definition 8.3.2

uses 0

One-field role wrappers over `Quantity k R`, with `castCarrier` on each so a pair born as counts crosses to a float carrier in its roles. The eliminator `Part.fractionOf` writes the portion-over-total division once, in the only order the roles admit, and its result is deliberately a **different** kind, licensed by a quotient kind-law — dividing a portion by its total is what mints the "fraction of total" role. The roles claim no partition: exhaustiveness and disjointness are further commitments, and the kernels this serves do not make them.

Where the foundation does work downstream: the extensivity chapter states Dybkær's §13.5 aggregation modes as laws quantified over all carvings; the object-type chapter adds the whole a carving cannot supply (`Composite`, one sort at a time) and keys the license to aggregate by that sort (`Assembles`); the dedicated-kinds chapter consumes the sort as Dybkær's Ch. 20 asks; and the count — the SI's "count of a specified elementary entity" — is keyed by a sortal, because the refutations above leave it nowhere else to live.

9. Dedicated kinds-of-property

A bare kind-of-property — *length*, *volume fraction*, *substance concentration* — is generic: it names *what is measured* without saying *of what*. Dybkær's Chapter 20 closes that gap with the *dedicated kind-of-property*, defined there as "a kind-of-property with given sort of system and any pertinent component". It is the syntax laboratory medicine already uses for a fully-specified measurand — `System – Component ; kind-of-property`, as in *"Plasma — Glucose ; substance concentration"*, or, in this project's home domain, *"Soil — Water ; volume fraction"* (volumetric water content). Note what the definition asks for: a **sort** of system. *Plasma* and *soil* name sorts, and the catalogue entry is one entry for every plasma sample and every soil sample there will ever be. The *particular* system a measured value characterizes enters one layer down, as the object index of an individual quantity — dedication to the sort, individuation by the object.

This is the construct the QUDV → OML/OWL2 lineage cannot express. A description logic can record that a quantity *has* a sort of system and *has* a component, but it cannot make *"Soil — Water ; volume fraction"* and *"Soil — Water ; mass fraction"* provably distinct kinds *while keeping both dedicated to the same sort and the same component*. Here that distinctness is a theorem, and it is the *principled* form of the dimension-1 disambiguation: volumetric and gravimetric water content are the **same measurement target** — soil, water — examined as two different kinds-of-property, so they differ **because the kinds-of-property differ**, not because someone chose two different identity strings.

9.1. The dedicated kind (Dybkær Ch. 20)

Definition 9.1.1. group statement">used by 3 ✓

A *component* (§20) is the pertinent component of the sort of system a dedicated kind is about — *water* in a *soil* sample, *glucose* in *plasma*. Specified abstractly by identity, exactly mirroring the sort carrier.

Lean code for Definition 9.1.1. 1 definition

`structure` (1 field) · defined in `PropertyKindCalculus/DedicatedKind.lean` complete

```
structure PropertyKindCalculus.Component : Type
```

Proof for Definition 9.1.1 uses 0

A one-field `structure` with `DecidableEq`; nothing to prove.

Definition 9.1.2. group uses 3 used by 3 ✓

A *dedicated kind-of-property* (§20) bundles a sort of system, a component, and a kind-of-property. Its `systematicTerm` renders the three as `System – Component ; kind`, and it *is a dedicated kind-of-quantity* iff its underlying kind has magnitude (§13.3.1). `dedicatedTo` builds one from a bare kind.

Lean code for Definition 9.1.2.

1 definition

`structure` (3 fields) · defined in `PropertyKindCalculus/DedicatedKind.lean` — complete

```
structure PropertyKindCalculus.DedicatedKind : Type
```

Proof for Definition 9.1.2

uses 0

A three-field `structure` with `DecidableEq`. `systematicTerm` is string interpolation of the three ids; `IsQuantity` is the underlying kind's `IsQuantity`.

Definition 9.1.3.

group uses 1 used by 0 ✓

Dedicating a kind *through an object*: the object contributes exactly its sort, via the model's `Sorted` claim. This is the arrow from the individual layer to the catalogue — an individual quantity carries the particular in its type, and `dedicatedFor` says which catalogue entry that particular's readings instantiate. In the terms of the foundations chapter's ontological square, it is Lowe's dispositional route to exemplification: up the *instantiated by* edge, then across the top.

Lean code for Definition 9.1.3.

1 definition

`def` · defined in `PropertyKindCalculus/DedicatedKind.lean` — complete

```
def PropertyKindCalculus.KindOfProperty.dedicatedFor.{u} {O : Type u}
  [PropertyKindCalculus.Sorted O]
  (k : PropertyKindCalculus.KindOfProperty) (o : O)
  (component : PropertyKindCalculus.Component) :
  PropertyKindCalculus.DedicatedKind
```

Proof for Definition 9.1.3

uses 0

`dedicatedTo` applied to `Sorted.sortOf o`.

Theorem 9.1.4.

group uses 0 used by 0 ✓

Tags

proved

> ***Two objects of one sort dedicate to one kind.*** *The catalogue entry cannot tell two rovers apart, and is not supposed to: individuation is the object index's job, where combining quantities across the two is a compile-time type error. This is the load-bearing half of the corrected §20 reading — dedication to the sort, individuation by the object.*

Lean code for Theorem 9.1.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/DedicatedKind.lean`

complete

```
theorem PropertyKindCalculus.KindOfProperty.dedicatedFor_congr.{u} {O : Type
u}
  [PropertyKindCalculus.Sorted O]
  {k : PropertyKindCalculus.KindOfProperty} {o₁ o₂ : O}
  (h :
    PropertyKindCalculus.Sorted.sortOf o₁ =
      PropertyKindCalculus.Sorted.sortOf o₂)
  (component : PropertyKindCalculus.Component) :
  k.dedicatedFor o₁ component = k.dedicatedFor o₂ component
```

Proof for Theorem 9.1.4

uses 0

Congruence: the two dedications differ only in the sort argument, and the hypothesis equates the sorts.

Theorem 9.1.5.

group uses 2 used by 0 ✓

Priority

high

Tags

capstone

proved

> ***Same sort and component, a different kind-of-property: a distinct dedicated kind.*** *This is the principled volumetric-versus-gravimetric distinctness. Both water contents are "Soil — Water", but volume fraction and mass fraction are different kinds-of-property, so the dedicated kinds differ — with the sort and component held identical, and without appeal to any hand-chosen identity string. This is exactly the discrimination an OWL2 reasoner cannot phrase.*

Lean code for Theorem 9.1.5.

1 theorem

`theorem` · defined in `PropertyKindCalculus/DedicatedKind.lean`

complete

```
theorem PropertyKindCalculus.DedicatedKind.distinct_of_kind
  {d₁ d₂ : PropertyKindCalculus.DedicatedKind} (h : d₁.kind ≠ d₂.kind) :
  d₁ ≠ d₂
```

Proof for Theorem 9.1.5

uses 0

Assume the two dedicated kinds are equal; then their `kind` projections are equal by congruence, contradicting the hypothesis (`intro he; apply h; rw [he]`). The companion laws `distinct_of_sort` and `distinct_of_component` are the same one-line argument on the other two projections.

Theorem 9.1.6.

group uses 4 statement">used by

Tags

proved

Same sort and kind, a different component: a distinct dedicated kind. *The water content and the air content of soil share their sort of system and their kind-of-property, differing only in the component — and that alone makes them distinct dedicated kinds.*

Lean code for Theorem 9.1.6.

1 theorem

`theorem` · defined in `PropertyKindCalculus/DedicatedKind.lean`

complete

```
theorem PropertyKindCalculus.DedicatedKind.distinct_of_component
  {d₁ d₂ : PropertyKindCalculus.DedicatedKind}
  (h : d₁.component ≠ d₂.component) : d₁ ≠ d₂
```

Proof for Theorem 9.1.6

uses 0

Congruence on the `component` projection, exactly as for `distinct_of_kind`.

10. Units and the Dimension-1 Problem

This chapter states the layer that originally motivated PropertyKindCalculus. Every node in it is *proved* and carries a `(lean := …)` declaration: the motivating capstone — the dimension-1 disambiguation, whose forgetful map the *Dimension as a Forgetful Functor* chapter supplies — and the unit-arithmetic nodes around it alike.

PhysLib's `Dimension` (a $\mathbb{Q}$-exponent free commutative group over the SI base quantities) makes every dimension-one quantity equal. Volumetric water content (`\mathrm{L^3/L^3}`), gravimetric water content (`\mathrm{M/M}`), relative permittivity, reflectivity, and emissivity are all dimension one — so a dimension-only type system cannot tell them apart. In soil-moisture retrieval that is precisely the class of confusion that silently corrupts a result. The kind layer is what restores the distinction.

10.1. Quantities and units (VIM4 2CD, Dybkær §13.3.3)

Definition 10.1.1. group uses 0 used by 4 ✓

A *numeric carrier* `Carrier R` is the minimal arithmetic a kind-indexed magnitude needs, supplied once per representation type `R` (a zero and an addition; a `LawfulCarrier` adds the additive-monoid laws). In plain engineering terms it is the *number type plug*: everything above it — kind discrimination, scale gating, the dimension functor, the interaction algebra, extensivity — is written against this class and so reused verbatim at every `R`. This is the role TorchLean's `Context α` typeclass plays for tensor element types; the kind index is the layer *above* the carrier that TorchLean does not have.

Lean code for Definition 10.1.1. 1 definition

`class` (2 methods) · defined in `PropertyKindCalculus/Quantity.lean` — complete

```
class PropertyKindCalculus.Carrier (R : Type) : Type
```

Proof for Definition 10.1.1 uses 0

Realized as the `Carrier` / `LawfulCarrier` classes. `Carrier R` bundles `zero` and `add`; `LawfulCarrier R` extends it with associativity, commutativity, and the unit laws. `Int` and (in the `Dimension` library) $\mathbb{R}$ are lawful carriers; `Float` is a `Carrier` but deliberately *not* lawful (floating-point addition is not associative).

Definition 10.1.2. group uses 2 used by 21 ✓

A *quantity* `Quantity k R` is a magnitude of a fixed kind `k`, carried at a representation type `R` (R10). It is indexed by `k`, a kind-of-property, so the type system forbids forming or comparing a `Quantity k₁ R` with a `Quantity k₂ R` when `k_1 \neq k_2` — even if both kinds are dimension one. This indexing *is* the fix for the dimension-1 conflation. The second index `R` is the numeric carrier: the same value type is

instantiated at `\mathbb{R}` and `Int` to prove, at `Float` to run today, and (the planned float carriers) at `FP32` to bound rounding and at `IEEE32Exec` to run with NaN — the kind machinery above it written once for all of them.

Lean code for Definition 10.1.2.

1 definition

structure (1 field) · defined in `PropertyKindCalculus/Quantity.lean` — complete

```
structure PropertyKindCalculus.Quantity (k :
PropertyKindCalculus.KindOfProperty)
  (R : Type) : Type
```

Proof for Definition 10.1.2

uses 2

Realized as `structure Quantity (k : KindOfProperty) (R : Type)` carrying a magnitude in `R`, with same-kind addition `Quantity.add : Quantity k R → Quantity k R → Quantity k R` (R4: the type forces both summands to share the kind). The algebraic operations come from the carrier typeclass on `R`; which operators are even *admissible* is the orthogonal scale gate (the proved operator stratification, operator monotonicity).

Theorem 10.1.3.

group · uses 2 · used by 2 ✓

Effort
medium

Tags
capstone · proved

Representation-parametric additivity (R10). *The additivity laws on quantities — commutativity, associativity, and the zero unit — `q_1 + q_2 = q_2 + q_1, \quad (q_1+q_2)+q_3 = q_1+(q_2+q_3), \quad 0 + q = q` hold over every lawful carrier, established by a single proof and so available at each `R` at once: at `\mathbb{R}` (the proof carrier) and at `Int` with no `R`-specific argument. The very carrier where these laws are absent — an executable `Float`, a carrier but not a lawful one — is what the exec/spec refinement bridge below exists to reconcile. Builds on the quantity layer.*

Lean code for Theorem 10.1.3.

1 theorem

theorem · defined in `PropertyKindCalculus/Quantity.lean` — complete

```
theorem PropertyKindCalculus.Quantity.laws_parametric
  {k : PropertyKindCalculus.KindOfProperty} {R : Type}
```

```
  [PropertyKindCalculus.LawfulCarrier R]
  (h : PropertyKindCalculus.DifferenceKind k)
  (x y z : PropertyKindCalculus.Quantity k R) :
  PropertyKindCalculus.Quantity.add h x y =
      PropertyKindCalculus.Quantity.add h y x ∧
    PropertyKindCalculus.Quantity.add h
          (PropertyKindCalculus.Quantity.add h x y) z =
        PropertyKindCalculus.Quantity.add h x
          (PropertyKindCalculus.Quantity.add h y z) ∧
      PropertyKindCalculus.Quantity.add h
            PropertyKindCalculus.Quantity.zero x =
          x ∧
        PropertyKindCalculus.Quantity.add h x
            PropertyKindCalculus.Quantity.zero =
          x
```

Proof for Theorem 10.1.3

uses 0

Proved in `PropertyKindCalculus.Quantity` (axiom-free). Each law unfolds `Quantity.add` to its carrier operation and rewrites by the corresponding `LawfulCarrier` field; `laws_parametric` bundles the four. Specializing to `Int` (core) and `ℝ` (in the `Dimension` library) reuses the same proof verbatim, which *is* the R10 payoff.

Definition 10.1.4.

group statement">used by 3 ✓

A *carrier refinement* `CarrierRefinement E S` specifies how an *exec* carrier `E` (fast, lossy) stands in for a *spec* carrier `S` (exact): a forgetful map `toSpec : E → S` (the analogue of TorchLean's `toReal`) and a spec-side `round : S → S`, with the single law that forgetting an exec addition equals rounding the spec addition, `\mathrm{toSpec}(x +_E y) = \mathrm{round}\,(\mathrm{toSpec}\,x +_S \mathrm{toSpec}\,y)`. In plain engineering terms it is the *rounding contract* between a slow exact number type and the fast machine number that stands in for it. Builds on the carrier.

Lean code for Definition 10.1.4.

1 definition

class (4 methods) · defined in `PropertyKindCalculus/QuantityRefinement.lean`

complete

```
class PropertyKindCalculus.CarrierRefinement (E S : Type)
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S] :
  Type
```

Proof for Definition 10.1.4

uses 0

Realized as the `CarrierRefinement` class (core, axiom-free). TorchLean's `FP32` gives an *unconditional* instance over `\mathbb{R}`, and `IEEE32Exec` refines `\mathbb{R}` on the finite path — both in the separately built `Torch` library.

Theorem 10.1.5.

group uses 2 statement">used by

Effort	Priority	Tags
large	high	capstone, proved

Exec refines spec across representations (R10). *For a kind-`k` quantity, the exec sum viewed in the spec carrier is the rounding of the spec sum, `\mathrm{toSpec}(x +_E y) = \mathrm{round}\,(\mathrm{toSpec}\,x +_S \mathrm{toSpec}\,y)`, established over any carrier refinement and so holding at each at once. A law proved over the lawful spec carrier therefore transfers to the executable run with one rounding step — the bridge that carries the representation-parametric laws from the lawful `ℝ` onto a float that is not lawful, so one kind-indexed value serves both proof and execution.*

Lean code for Theorem 10.1.5.

1 theorem

`theorem` · defined in `PropertyKindCalculus/QuantityRefinement.lean` — complete

```
theorem PropertyKindCalculus.Quantity.add_refines
  {k : PropertyKindCalculus.KindOfProperty} {E S : Type}
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S]
  [PropertyKindCalculus.CarrierRefinement E S]
  (h : PropertyKindCalculus.DifferenceKind k)
  (x y : PropertyKindCalculus.Quantity k E) :
  (PropertyKindCalculus.Quantity.add h x y).toSpec =
    PropertyKindCalculus.Quantity.roundBy
      (PropertyKindCalculus.CarrierRefinement.round E)
      (PropertyKindCalculus.Quantity.add h x.toSpec y.toSpec)
```

Proof for Theorem 10.1.5

uses 0

Proved as `Quantity.add_refines` (core, axiom-free): the `CarrierRefinement` bridge law lifted along the kind index. It is instantiated concretely in the separately built `Torch` library at TorchLean's binary32 — an *unconditional* `CarrierRefinement FP32 ℝ` (the rounding spec has no overflow, so the law holds by computation) and a *conditional* `IEEE32Exec` refinement on the finite/no-overflow path (`Quantity.add_refines_exec`), where overflow is an explicit hypothesis rather than a silent failure. This is TorchLean's `Bridge/FP32` pattern ("compute in `\mathbb{R}`, then round").

Definition 10.1.6. group statement">stat

The bridge's multiplicative law. `MulRefinement E S` says that the *same* forgetful map and rounding a carrier refinement supplies for `+` also relate the exec product to the spec product, `\mathrm{toSpec}(x \cdot_E y) = \mathrm{round}\,(\mathrm{toSpec}\,x \cdot_S \mathrm{toSpec}\,y)`. Held apart from the refinement rather than bundled into it, for two reasons. A carrier can refine additively without having a `\times` at all — the carrier class supplies only zero and addition — and an expression that uses `+`, `\times` and `\div` at once would, under bundling, put two independent paths to `toSpec` and `round` in scope simultaneously.

The line the split follows is data against proof, and it is what keeps the two extra classes from re-creating the problem they avoid. A carrier refinement carries **functions**, and two of them are two different functions, so it must reach a use site by exactly one path. These two carry only equations, and a class in `\mathrm{Prop}` is a subsingleton — any two of its instances are the same instance — so they may be inherited, re-derived and diamonded freely without a term ever depending on which one arrived.

Lean code for Definition 10.1.6.

1 definition

class (1 method) · defined in `PropertyKindCalculus/QuantityRefinement.lean` — complete

```
class PropertyKindCalculus.MulRefinement (E S : Type)
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S]
  [Mul E] [Mul S] [PropertyKindCalculus.CarrierRefinement E S] : Prop
```

Proof for Definition 10.1.6

uses 0

Realized as the `MulRefinement` class (core, axiom-free), a `Prop` class parametrized by a `CarrierRefinement` rather than extending it, so one carrier has one forgetful map and one rounding. Instantiated at TorchLean's `FP32` in the separately built `Torch` library, where `NF` multiplication is `ofReal (a·b)` and the law is therefore `rfl`.

Definition 10.1.7. group statement">us

The bridge's division law, in the same shape as the multiplicative one — and unconditional, which is a statement about specification carriers and not about machines. Both sides read the spec carrier's total convention for `x / 0`, so at a zero divisor there is nothing to exclude. A real executable format does not totalize division; it produces an infinity or a NaN, and no rounding of a real quotient is either. So the hazard sits one rung down, as an explicit hypothesis on the executable carrier's own theorem.

Lean code for Definition 10.1.7.

1 definition

class (1 method) · defined in PropertyKindCalculus/QuantityRefinement.lean — complete

```
class PropertyKindCalculus.DivRefinement (E S : Type)
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S]
  [Div E] [Div S] [PropertyKindCalculus.CarrierRefinement E S] : Prop
```

Proof for Definition 10.1.7

uses 0

Realized as the `DivRefinement` class (core, axiom-free), with the unconditional `DivRefinement FP32 ℝ` instance in the `Torch` library. The executable rung's `Quantity.div_refines_exec` asks that the executable quotient be finite, and the Torch-tier probes discharge that on concrete binary32 values and then refute it at a zero denominator, where the divisor's decoded mantissa is shown to be zero.

Theorem 10.1.8.

group · uses 2 · used by 0 ✓

Effort: medium

Tags: proved

Exec refines spec across a licensed kind product (R10). *For a product law `k_1 \times k_2 = k`, the exec product viewed in the spec carrier is the rounding of the spec product. The additive bridge preserves one kind throughout; this one does not, and that is its content. The kinds move, but they move under the same product law on both sides, so the rounding obligation and the kind license are independent and compose: a rounding step cannot launder a product the kind calculus refuses, and a licensed product does not lose its license by being computed in floats.*

Lean code for Theorem 10.1.8.

1 theorem

theorem · defined in PropertyKindCalculus/QuantityRefinement.lean — complete

```
theorem PropertyKindCalculus.Quantity.mul_refines {E S : Type}
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S]
  [Mul E] [Mul S] [PropertyKindCalculus.ScalarCarrier E]
  [PropertyKindCalculus.ScalarCarrier S]
  [PropertyKindCalculus.CarrierRefinement E S]
  [PropertyKindCalculus.MulRefinement E S]
  {k₁ k₂ k : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.ProductKind k₁ k₂ k)
```

```
  (x : PropertyKindCalculus.Quantity k₁ E)
  (y : PropertyKindCalculus.Quantity k₂ E) :
  (PropertyKindCalculus.Quantity.mul h x y).toSpec =
    PropertyKindCalculus.Quantity.roundBy
      (PropertyKindCalculus.CarrierRefinement.round E)
      (PropertyKindCalculus.Quantity.mul h x.toSpec y.toSpec)
```

Proof for Theorem 10.1.8

uses 0

Proved as `Quantity.mul_refines` (core, depending on no axioms at all): the `MulRefinement` law lifted along the kind product. Specialized at binary32 as `Quantity.mul_refines_fp32`, and conditionally at the executable carrier as `Quantity.mul_refines_exec` under decoding and finiteness hypotheses.

Theorem 10.1.9.

group uses 2 used by 0 ✓

Effort	Tags
medium	proved

And across a licensed kind quotient (R10) *— the rung a ratio, a conversion factor, or a [weighted carving](#)'s denominator rides. An additive bridge carries the [extensive mode](#) and nothing else; the mean of a constant, the parallel-axis correction, and every sensitivity coefficient are* `+/\ times/\ div` *expressions, and none of them can cross on an additive contract alone.*

Lean code for Theorem 10.1.9.

1 theorem

`theorem` · defined in `PropertyKindCalculus/QuantityRefinement.lean` — complete

```
theorem PropertyKindCalculus.Quantity.div_refines {E S : Type}
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S]
  [Div E] [Div S] [PropertyKindCalculus.ScalarCarrier E]
  [PropertyKindCalculus.ScalarCarrier S]
  [PropertyKindCalculus.CarrierRefinement E S]
  [PropertyKindCalculus.DivRefinement E S]
  {k₁ k₂ k : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.QuotientKind k₁ k₂ k)
  (x : PropertyKindCalculus.Quantity k₁ E)
  (y : PropertyKindCalculus.Quantity k₂ E) :
  (PropertyKindCalculus.Quantity.div h x y).toSpec =
    PropertyKindCalculus.Quantity.roundBy
      (PropertyKindCalculus.CarrierRefinement.round E)
      (PropertyKindCalculus.Quantity.div h x.toSpec y.toSpec)
```

Proof for Theorem 10.1.9

uses 0

> Proved as `Quantity.div_refines` (core, depending on no axioms at all), with `Quantity.div_refines_fp32` at the binary32 rounding spec and `Quantity.div_refines_exec` at the executable carrier. Both are exercised over a genuinely lossy toy carrier in the core probes — where `3 \times 3` reads 9 exactly but forgets to 8 — so neither capstone is satisfied vacuously by an identity rounding.

Definition 10.1.10.

group uses 3 used by 0 ✓

Vector quantities: numerical array × one scalar unit (R11, ISO 80000-2 §18). A vector (or tensor) quantity is a *numerical* array carried at one kind with one *scalar* unit — not a per-coordinate collection of `number × unit` values; "all units are scalars". This is the carrier taken at a function-space type: `Fin n → R` is a (lawful) carrier whenever `R` is, pointwise, so a vector quantity is `Quantity k (Fin n → R)` and the additivity laws transfer to it by the same parametric proof. The quantity is coordinate-independent; only its numerical components depend on the frame.

Lean code for Definition 10.1.10.

1 definition

def · defined in `PropertyKindCalculus/QuantityVector.lean` — complete

```
def PropertyKindCalculus.instLawfulCarrierPi {ι R : Type}
  [PropertyKindCalculus.LawfulCarrier R] :
  PropertyKindCalculus.LawfulCarrier (ι → R)
```

Proof for Definition 10.1.10

uses 0

> Realized as the pointwise `Carrier (ι → R)` / `LawfulCarrier (ι → R)` instances (core). The fuller structural apparatus — coordinate frames, tensor variance, transforms — remains a separate, owed axis.

Definition 10.1.11.

group uses 2 **statement** proof ✓

A *unit* `RealUnit k` is a distinguished quantity of kind `k` chosen as the reference: measuring expresses any quantity of kind `k` as `\text{number} \times \text{unit}`. Because it is indexed by the same `k`, "a metre" and "a unit of gravimetric water content" inhabit different types. This is the real-valued refinement of the proved metrological unit: it replaces the symbolic numeral-and-reference form with a magnitude in `\mathbb{R}`, so that conversion *ratios* become arithmetic.

Lean code for Definition 10.1.11.

1 definition

structure (2 fields) · defined in `PropertyKindCalculus/UnitReal.lean` — complete

```
structure PropertyKindCalculus.RealUnit
  (k : PropertyKindCalculus.KindOfProperty) : Type
```

Proof for Definition 10.1.11 proof">uses 1

Realized as `RealUnit k`, a `Quantity k ℝ` chosen as the reference together with the one condition the choice must satisfy: `ref_ne_zero`. That field is what the phrase "express the ratio of the two quantities as a number" presupposes and the symbolic layer never has to state — a reference of magnitude zero expresses no ratio — and carrying it makes a degenerate unit not a term of the type. Measuring is `measure : Quantity k ℝ → RealUnit k → ℝ`, division by the reference; `ofNumber` is the multiplication back; and the §13.3.3 number-and-reference round-trip is then a bijection in both directions (`measure_ofNumber`, `ofNumber_measure`) rather than a definitional unfolding. `ratio` is the §1.22 conversion factor for an *arbitrary* chosen reference, nonzero from the two licenses and reciprocal in the two directions, of which the power-of-radix factor is the special case. The refinement inherits the §9.13.4 exclusions rather than widening them: `WellFormed` is the symbolic layer's `BearsUnit`, so a nominal or ordinal kind bears no real-valued unit either.

Theorem 10.1.12. group uses 3 used by 2 ✓

Effort	Priority	Tags
medium	high	capstone, proved

Unit conversion is a faithful round-trip. *For two commensurable units `u_1, u_2` of the same kind `k`, converting a magnitude from `u_1` to `u_2` and back is the identity: `\mathrm{convert}_{u_2 \to u_1}\bigl(\mathrm{convert}_{u_1 \to u_2}(x)\bigr) = x`. This is proved for the realized case: the SI-prefixed (decimal) and binary-prefixed (IEC 80000-13) units, where the §1.22 conversion factor is a **power of the radix** — ten for SI, two for binary. That factor is kept as its integer **exponent**, so the two directions are reciprocal on the nose (`+5` then `-5` for `\mathrm{km}\leftrightarrow\mathrm{cm}`; `+10` then `-10` for `\mathrm{MiB}\leftrightarrow\mathrm{KiB}`) — exact over `\mathbb{Z}` (`convertExp_roundtrip`) and, numerically, over `\mathbb{R}` via a power-of-radix factor that is **structurally** nonzero (`convertReal_roundtrip`). The exponent bookkeeping is blind to the radix, so **one** proof serves both prefix families, gated to a common radix; conversion across radices (a power of ten is never a power of two) and across kinds is simply undefined — a type-level fact, not a runtime check. The fully general conversion by an arbitrary chosen-reference ratio, over the real-valued unit, remains the deeper refinement below.*

Lean code for Theorem 10.1.12. 1 theorem

`theorem` · defined in `PropertyKindCalculus/UnitPrefix.lean` complete

```
theorem PropertyKindCalculus.PrefixedUnit.convertExp_roundtrip
  (p q : PropertyKindCalculus.PrefixedUnit) (decExp : ℤ) :
  q.convertExp p (p.convertExp q decExp) = decExp
```

Proof for Theorem 10.1.12

proof">uses 1

Because the conversion factor is a power of ten, its exponent is an `Int` — an element of an additive group — so the shift `u_1\to u_2` (`shift`, the difference of the prefix exponents) and the shift back sum to zero (`shift_add_symm`). The round-trip on the decimal exponent is then `x + (e_1 - e_2) + (e_2 - e_1) = x`, closed by `omega` with no rounding. The numeric `\mathbb{R}` companion multiplies by `10^{e_1-e_2}` then `10^{e_2-e_1}`; `zpow_add₀` on `10\neq 0` collapses the exponents to `10^0 = 1`. Neither needs a chosen-reference nonzeroness hypothesis; the general real-valued case is where one is needed, and it carries `ref_ne_zero` as a field so that `ratio_ne_zero` follows from the two licenses and from nothing else.

Theorem 10.1.13.

group uses 2 statement">used by

Effort	Priority	Tags
small	high	capstone, proved

The dimension-1 disambiguation (the motivating capstone). *There exist distinct kinds that map to the same dimension. Concretely, for volumetric and gravimetric water content, `\mathrm{vwc} \neq \mathrm{gwc} \qquad\text{yet}\qquad \dim(\mathrm{vwc}) = \dim(\mathrm{gwc}) = \mathbf{1}`, and likewise for permittivity and reflectivity. So the kind layer is a strict refinement of the dimension layer: `\dim` is not injective. This is the statement PhysLib — and any dimension-only model, whether OWL2 or a representation-rooted hierarchy like SysML v2's — structurally cannot make. Uses kind-of-property and the dimension map.*

Lean code for Theorem 10.1.13.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Dimension.lean` — complete

```
theorem PropertyKindCalculus.dim_not_injective :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.toDimension = b.toDimension ∧ a.toDimension = 1
```

Proof for Theorem 10.1.13

uses 0

Proved in the `Dimension` library. `vwc` and `gwc` are declared as dimensioned kinds with distinct kind `ids`, so `vwc.kind ≠ gwc.kind` is `by decide`; both carry `dim = 1`, so their dimensions agree by `rfl`. Together they witness `\exists a\ b,\ a.\mathrm{kind} \neq b.\mathrm{kind} \wedge \dim a = \dim b \wedge \dim a = 1`.

Definition 10.1.14. group uses 0 **state**

A *decimal SI prefix* `SIPrefix` is a name and symbol denoting a power-of-ten factor (e.g. *centi* = `10^{-2}`). Applied to a unit it produces a multiple (VIM4 2CD 1.20) or submultiple (1.21) of that unit.

Lean code for Definition 10.1.14. 1 definition

structure (3 fields) · defined in `PropertyKindCalculus/UnitPrefix.lean` complete

```
structure PropertyKindCalculus.SIPrefix : Type
```

Proof for Definition 10.1.14 uses 0

A `structure SIPrefix` carrying `name`, `symbol`, and an integer `exponent` (the factor is `10^{\text{exponent}}`); the full SI set (quetta … quecto) is enumerated as `defs`.

Definition 10.1.15. group **statement">state**

A *binary prefix* `BinaryPrefix` (IEC 80000-13) is the information-technology counterpart of an SI prefix, denoting a power-of-*two* factor: *kibi* = `2^{10}`, *mebi* = `2^{20}`, …. IEC 80000-13 introduced these to disambiguate the "kilobyte" between `10^3` and `2^{10}` bytes — *kibi* names the binary one. They are **not** SI prefixes (the SI is decimal-only), so they are a distinct type; but they form the same multiples/submultiples and bear the same §1.22 conversion factor, and so reuse one prefixed-unit and one conversion round-trip.

Lean code for Definition 10.1.15. 1 definition

structure (3 fields) · defined in `PropertyKindCalculus/UnitPrefix.lean` complete

```
structure PropertyKindCalculus.BinaryPrefix : Type
```

Proof for Definition 10.1.15 uses 0

A `structure BinaryPrefix` carrying `name`, `symbol`, and an integer `exponent` (the factor is `2^{\text{exponent}}`); the eight prefixes kibi … yobi are enumerated as `defs`. A `PrefixedUnit` records

the resolved `radix` (10 or 2) so that a single `convertExp_roundtrip` serves both families, gated to a common radix.

10.2. Verified classification and instantiable kind-laws (R12)

A quantity `Quantity k R` records its kind `k` as a *tag*: the magnitude is an arbitrary `R`, so a value can be labelled with a kind it has not earned (`⟨999⟩` type-checks as an area). R12 upgrades a classification from an *assertion* to a *certificate*: a quantity is classified under a kind by a proof that it satisfies the kind's *defining relation* — the formalized ISO 80000 *Remark* — to quantities of the kinds it is built from. The kind becomes a *refinement* (a tag plus its earned proof), not a label.

The payoff is *instantiation*. Every kind-law is stated as a carrier-parametric universal over quantities with the kind-relation as a premise; "instantiating it at the quantity level" is then ordinary application, and any property of a kind's defining relation transports to *every* quantity certified under it. This is the inter-kind generalization of the carrier-parametric laws of R10.

Definition 10.2.1. group uses 0 used by 2 ✓

A *product kind-law* `ProductKind k₁ k₂ k` records that `k` is the product kind of `k₁` and `k₂`. In the core it carries the scale precondition — all three are ratio-scale, since only ratio quantities multiply (Dybkær §13.3.5); the dimension layer refines it with the dimensional equation `k.\dim = k_1.\dim \cdot k_2.\dim`.

Lean code for Definition 10.2.1. 1 definition

structure (3 fields) · defined in `PropertyKindCalculus/QuantityClassification.lean` complete

```
structure PropertyKindCalculus.ProductKind
  (k₁ k₂ k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 10.2.1 uses 0

Realized as `structure ProductKind` with three `IsRational` fields. A concrete law such as area = length · length is the value `⟨rfl, rfl, rfl⟩`.

Definition 10.2.2. group statement uses 1 used by 0 ✓

The *certificate* `q.IsProduct h a b` states that `q` (of kind `k`) is the product of `a` (of `k₁`) and `b` (of `k₂`): its magnitude is the product of theirs. A proof of this certifies `q`'s classification, rather than asserting it. It is a separate proposition, so `Quantity` stays a clean tag and certificates are carried only when needed. The smart constructor `Quantity.mul` yields a quantity whose certificate holds by construction.

Lean code for Definition 10.2.2.

1 definition

`def` · defined in `PropertyKindCalculus/QuantityClassification.lean` — complete

```
def PropertyKindCalculus.Quantity.IsProduct {R : Type} [Mul R]
  [PropertyKindCalculus.ScalarCarrier R]
  {k₁ k₂ k : PropertyKindCalculus.KindOfProperty}
  (_h : PropertyKindCalculus.ProductKind k₁ k₂ k)
  (q : PropertyKindCalculus.Quantity k R)
  (a : PropertyKindCalculus.Quantity k₁ R)
  (b : PropertyKindCalculus.Quantity k₂ R) : Prop
```

Proof for Definition 10.2.2

uses 0

`Quantity.IsProduct h q a b := q.magnitude = a.magnitude * b.magnitude` over a carrier with multiplication; `Quantity.mul h a b := ⟨a.magnitude * b.magnitude⟩`, and `mul_isProduct` is `rfl`.

Theorem 10.2.3.

group · uses 0 · used by 0 ✓

Effort	Tags
small	proved

A kind-law that instantiates at the quantity level. *A quantity certified as the product of `a` and `b` is unique — and any certified product equals the smart-constructed one (`eq_mul_of_isProduct`, canonicity). Supplying concrete quantities gives the quantity-level fact by application. Axiom-free.*

Lean code for Theorem 10.2.3.

1 theorem

theorem · defined in PropertyKindCalculus/QuantityClassification.lean — complete

```
theorem PropertyKindCalculus.Quantity.isProduct_unique {R : Type} [Mul R]
  [PropertyKindCalculus.ScalarCarrier R]
  {k₁ k₂ k : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.ProductKind k₁ k₂ k)
  {q q' : PropertyKindCalculus.Quantity k R}
  {a : PropertyKindCalculus.Quantity k₁ R}
  {b : PropertyKindCalculus.Quantity k₂ R}
  (hq : PropertyKindCalculus.Quantity.IsProduct h q a b)
  (hq' : PropertyKindCalculus.Quantity.IsProduct h q' a b) : q = q'
```

Proof for Theorem 10.2.3

uses 0

`isProduct_unique` rewrites both magnitudes to `a.magnitude * b.magnitude` and closes by structure η; `eq_mul_of_isProduct` is `isProduct_unique` against `mul_isProduct`. Both depend on no axioms.

Definition 10.2.4.

group · uses 0 · used by 9 ✓

A *quotient kind-law* `QuotientKind k₁ k₂ k` records that `k` is the quotient kind `k = k₁ / k₂` (speed = length / duration; plane angle = arc / radius, where the lengths cancel). Like the product, it carries the ratio-scale precondition in the core and is refined by `k.dim = k₁.dim / k₂.dim` in the dimension layer. Its certificate `Quantity.IsQuotient` and smart constructor `Quantity.div` mirror the product family exactly, over the core `Div` class.

Lean code for Definition 10.2.4.

1 definition

structure (3 fields) · defined in PropertyKindCalculus/QuantityClassification.lean — complete

```
structure PropertyKindCalculus.QuotientKind
  (k₁ k₂ k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 10.2.4

uses 0

`structure QuotientKind` with three `IsRational` fields; `Quantity.div h a b := ⟨a.magnitude / b.magnitude⟩`, `div_isQuotient` is `rfl`, and `isQuotient_unique` / `eq_div_of_isQuotient` are the product family's proofs transcribed. Axiom-free.

Definition 10.2.5.

group · uses 0 · statement">used by ✓

A *reciprocal kind-law* `ReciprocalKind k₁ k` records that `k = 1 / k₁` (frequency = 1 / period; curvature = 1 / radius; repetency = 1 / wavelength) — the unary special case, over the core `Inv` class, refined by `k.dim = (k₁.dim)⁻¹`. All three families (product, quotient, reciprocal) stay Mathlib-free, using only toolchain-level arithmetic classes.

Lean code for Definition 10.2.5.

1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/QuantityClassification.lean` — complete

```
structure PropertyKindCalculus.ReciprocalKind
  (k₁ k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 10.2.5

uses 0

`structure ReciprocalKind` with two `IsRational` fields; `Quantity.recip h a := ⟨a.magnitude⁻¹⟩`, `recip_isReciprocal` is `rfl`, with `isReciprocal_unique` / `eq_recip_of_isReciprocal` as before. Axiom-free.

11. Dimension as a Forgetful Functor

PropertyKindCalculus does not discard PhysLib's `Dimension`; it keeps it as a forgetful functor. Dimension answers the coarse question — "are these even commensurable in the SI base quantities?" — and the kind layer answers the fine one. The forgetful map, its multiplicativity, and the dimension-1 disambiguation it makes possible are now *realized*, sorry-free, in the `Dimension` library; the full coherence law — that `\dim` is a homomorphism over the interaction algebra — is realized in the `Interaction` module and stated as the homomorphism capstone below.

Two terms recur below; in plain engineering terms they mean this. A *forgetful functor* is a deliberately lossy, one-way map: `\dim` keeps only a kind's SI base-quantity exponents (length, mass, time, …) and discards everything else that distinguishes it, so many kinds share one dimension and there is no inverse. A *homomorphism* is a map that respects an operation: here `\dim` respects multiplication — the dimension of a product is the product of the dimensions, the exponents adding — which is exactly the bookkeeping of everyday dimensional analysis, stated and proved here as a law rather than performed by hand.

11.1. The forgetful map (Dybkær Ch. 19)

Definition 11.1.1. group uses 2 used by 43 ✓

The *dimension map* `\dim` sends a *dimensioned kind* — a kind-of-property paired with a PhysLib `Dimension` (ℚ-exponents over length/time/mass/charge/temperature) — to that dimension. It is deliberately many-to-one: that is what makes the dimension-1 disambiguation true.

Lean code for Definition 11.1.1. 1 definition

`def` · defined in `PropertyKindCalculus/Dimension.lean` — complete

```
def PropertyKindCalculus.DimensionedKind.toDimension {B : Type}
  [DimensionBasis B] (dk : PropertyKindCalculus.DimensionedKind B) :
  Dimension B
```

Proof for Definition 11.1.1 uses 0

Realized in the `Dimension` library (`dimension/`). `DimensionedKind` is the structure pairing a kind with its `Dimension`; `toDimension` is the forgetful projection. PhysLib + Mathlib enter only here, as a separate library; the spine stays Mathlib-free.

Theorem 11.1.2. group uses 2 statement">used by ✓

Tags

proved

The forgetful functor preserves products. *The dimension of a product of dimensioned kinds is the product of their dimensions, `\dim (a \cdot b) = \dim a \cdot \dim b`, the SI exponents adding — ordinary dimensional bookkeeping, proved as a law. This is the dimension-component of functoriality, stated for the provisional product; the full interaction-algebra version is the homomorphism capstone below. Builds on the dimension map.*

Lean code for Theorem 11.1.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Dimension.lean`

complete

```
theorem PropertyKindCalculus.DimensionedKind.toDimension_times {B : Type}
  [DimensionBasis B] (a b : PropertyKindCalculus.DimensionedKind B) :
  (a.times b).toDimension = a.toDimension * b.toDimension
```

Proof for Theorem 11.1.2

uses 0

The provisional product multiplies the dimension components (PhysLib's `Dimension.mul`, i.e. exponent addition), so the equation holds by `rfl`. The unit law `\dim 1 = 1` holds likewise.

Theorem 11.1.3.

group · uses 2 · used by 5 ✓

Effort	Priority	Tags
medium	high	capstone, proved

Dimensional coherence — `\dim` is a homomorphism over the full interaction algebra. *Whenever the interaction algebra says `k_1` and `k_2` combine to `k_3`, their dimensions combine the same way: `\mathrm{KMul}\ k_1\ k_2\ k_3 \;\Longrightarrow\; \dim k_3 = \dim k_1 \cdot \dim k_2`. This is the theorem that licenses forgetting to the dimension layer without losing soundness: a kind-level product is always dimensionally consistent. It upgrades the proved multiplicativity from the provisional product to Flater's gated kind product.*

Lean code for Theorem 11.1.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Interaction.lean`

complete

```
theorem PropertyKindCalculus.InteractionAlgebra.dim_homomorphism
  (A : PropertyKindCalculus.InteractionAlgebra)
  {a b c : PropertyKindCalculus.DimensionedKind LTMCTDimensionBase}
  (h : A.KMul a b c) : c.toDimension = a.toDimension * b.toDimension
```

Proof for Theorem 11.1.3

uses 0

Realized in the `Interaction` module as `InteractionAlgebra.dim_homomorphism`. Coherence is a well-formedness field of every `InteractionAlgebra`, so the homomorphism is its projection: any algebra that can be formed at all already guarantees `\dim k_3 = \dim k_1 \cdot \dim k_2` on every sanctioned edge. The content sits in the worked algebra, where the obligation is discharged by the `Dimension`-group computation, case by case.

12. Scale-spanning units — a third unit category (R13)

The SI sorts its units into two categories — *base* (the kilogram, metre, second, ampere, kelvin, candela, mole) and *derived* (everything built from them). Finkelstein and Whitehead (*Eur. J. Phys.* **46** (2025) 035701) argue this two-way split is *insufficient*. Of the seven base units, only four — the kilogram, metre, second, and ampere — are genuinely *dimensionally independent*, each fixed by a natural constant (`h`, `c`, `Δν_Cs`, `e`). The other three are *not* dimensionally independent of those four: the kelvin has the dimension of *energy* (`M·L²·T⁻²`, via the Boltzmann constant `k_B`); the candela has the dimension of *power* (`M·L²·T⁻³`, via the luminous efficacy `K_cd`); and the mole is a dimensionless *number* (via the Avogadro constant `N_A`). Yet each is kept as a distinct, coherent unit because it carries a *human-selected coefficient* — chosen for historical significance and a size that spans vast scales without scientific notation (a temperature difference of 100 K, not `1.38 × 10⁻²¹ J` per degree of freedom). The authors propose a third category for them: *scale-spanning units*.

R13 specifies that third category. Its point is the unit-layer twin of *R1* (a kind is more than its dimension): *whether a unit is scale-spanning is not a function of its dimension alone*. ISO 80000-7 (the *Light and radiation* chapter) is where this lands on the standard — the candela and the mole appear there, and are handled by reduction, not by a new base generator.

12.1. A criterion the dimension layer can apply — and one place it cannot

Definition 12.1.1. group uses 0 used by 3 ✓

A dimension is *mechanically reducible* when it is expressible in mass, length, and time alone — it carries no charge and no temperature exponent. This is the Finkelstein–Whitehead test for "not dimensionally independent of the four physical base units", specialized to the mechanical generators: such a unit needs no electromagnetic, thermal, luminous, or chemical base of its own.

Lean code for Definition 12.1.1. 1 definition

`def` · defined in `PropertyKindCalculus/ScaleSpanning.lean` — complete

```
def PropertyKindCalculus.Dimension.MechanicallyReducible
  (d : Dimension LTMCTDimensionBase) : Prop
```

Proof for Definition 12.1.1 uses 0

`Dimension.MechanicallyReducible d := d.charge = 0 ∧ d.temperature = 0`. Power and energy are reducible (`power_mechanicallyReducible`, `energy_mechanicallyReducible`), as is dimension one; electric current is *not* (`current_not_mechanicallyReducible` — it carries the electromagnetic generator), and neither is thermodynamic temperature in this work's algebra (`temperature_not_mechanicallyReducible`).

Definition 12.1.2. group statement">statement">used by

The proposed three-way classification of SI units: *physical base* (the four dimensionally independent units `kg, m, s, A`), *scale-spanning* (the kelvin, candela, and mole), and *derived* (every plain product of powers, with no special coefficient). The scale-spanning units are the new middle category the two-way base/derived split omits.

Lean code for Definition 12.1.2. 1 definition

inductive (3 constructors) · defined in PropertyKindCalculus/ScaleSpanning.lean — complete

```
inductive PropertyKindCalculus.UnitCategory : Type
```

Proof for Definition 12.1.2 uses 0

`inductive UnitCategory | physicalBase | scaleSpanning | derived`. Realized as a plain enumeration; a scale-spanning unit's `category` is `scaleSpanning` by construction.

Definition 12.1.3. group statement">used by 4

A *scale-spanning unit* pairs a metrological unit with its mechanical-dimension reduction and the human-selected coefficient that defines it. The worked set is the candela (reducing to power, via `K_cd = 683 lm/W`), the mole (to dimension one, via `N_A`), and the kelvin (to energy, via `k_B`). The coefficient and its value are *citation locators* — the published constants — not reproduced normative text.

Lean code for Definition 12.1.3. 1 definition

structure (4 fields) · defined in PropertyKindCalculus/ScaleSpanning.lean — complete

```
structure PropertyKindCalculus.ScaleSpanningUnit : Type
```

Proof for Definition 12.1.3 uses 0

> `structure ScaleSpanningUnit` carries the `unit`, the `reducesTo` dimension, and the `coefficient` symbol and value. The three canonical instances `candela`, `mole`, `kelvin` are declared in the `ScaleSpanning` namespace (so their names do not collide with the standards parts that also declare them).

12.2. The requirement, proved

Theorem 12.2.1. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***The candela reduces to power — the dimension layer can see it.*** *Luminous intensity carries the dimension of power, which is mechanically reducible; so the dimension layer correctly detects that the candela is not dimensionally independent. The mole is likewise reducible (to dimension one). Uses mechanical reducibility.*

Lean code for Theorem 12.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/ScaleSpanning.lean` — complete

```
theorem PropertyKindCalculus.ScaleSpanning.candela_reduction_reducible :
  PropertyKindCalculus.Dimension.MechanicallyReducible
    PropertyKindCalculus.ScaleSpanning.candela.reducesTo
```

Proof for Theorem 12.2.1 uses 0

> `candela_reduction_reducible : Dimension.MechanicallyReducible candela.reducesTo`, from `power_mechanicallyReducible` (`power.charge = 0`, `power.temperature = 0`, checked in the PhysLib dimension group). The companion `mole_reduction_reducible` covers dimension one. Axiom-free.

Theorem 12.2.2. group uses 2 used by 0 ✓

Effort	Tags
small	proved

> ***The kelvin's scale-spanning character is invisible to the dimension layer.*** *Its proposed reduction (energy, via k_B) is mechanically reducible, yet the dimension this work assigns thermodynamic temperature (θ) is not — so no dimensional analysis reveals the kelvin to be scale-spanning. The fact is carried at the unit-classification layer, through the coefficient k_B, exactly as a kind carries more than its dimension. Uses mechanical reducibility.*

Lean code for Theorem 12.2.2.

1 theorem

theorem · defined in PropertyKindCalculus/ScaleSpanning.lean

complete

```
theorem PropertyKindCalculus.ScaleSpanning.kelvin_reduction_invisible_to_dimens
  PropertyKindCalculus.Dimension.MechanicallyReducible
      PropertyKindCalculus.ScaleSpanning.kelvin.reducesTo ∧
    ¬PropertyKindCalculus.Dimension.MechanicallyReducible
        PropertyKindCalculus.Dim.temperature
```

Proof for Theorem 12.2.2

uses 0

`kelvin_reduction_invisible_to_dimension : Dimension.MechanicallyReducible kelvin.reducesTo ∧ ¬ Dimension.MechanicallyReducible Dim.temperature`, the first conjunct from `energy_mechanicallyReducible`, the second from `temperature_not_mechanicallyReducible` (`Dim.temperature.temperature = 1 ≠ 0`). Axiom-free.

Theorem 12.2.3.

group

statement">used by 0

✓

Effort: small

Tags: proved

> ***The ampere is a genuine physical base — not scale-spanning.*** *Electric current is not mechanically reducible: it carries the electromagnetic generator, so it is dimensionally independent. This is the contrast that makes the third category non-empty and distinct from the first — the ampere (IEC 80000-6) belongs to the base set, the candela (ISO 80000-7) does not. Uses mechanical reducibility.*

Lean code for Theorem 12.2.3.

1 theorem

theorem · defined in PropertyKindCalculus/ScaleSpanning.lean

complete

```
theorem PropertyKindCalculus.ScaleSpanning.ampere_is_physicalBase :
  ¬PropertyKindCalculus.Dimension.MechanicallyReducible
      PropertyKindCalculus.Dim.current
```

Proof for Theorem 12.2.3

uses 0

`ampere_is_physicalBase : ¬ Dimension.MechanicallyReducible Dim.current`, from `Dim.current_charge = 1 ≠ 0`. Axiom-free.

Theorem 12.2.4. group uses 2 used by 0 ✓

Effort	Priority	Tags
small	high	capstone, proved

> ***R13 — the base/derived dichotomy is insufficient; scale-spanning is a third category, not a function of dimension.*** *Two scale-spanning units disagree on mechanical reducibility — the candela's dimension (reducible) and the kelvin's actual dimension `θ` (not reducible) — so membership in the third category cannot be decided from the dimension. The classification is a genuine extra datum, the unit-layer analogue of R1. Uses the unit categories and the scale-spanning units.*

Lean code for Theorem 12.2.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/ScaleSpanning.lean` — complete

```
theorem PropertyKindCalculus.ScaleSpanning.scaleSpanning_not_determined_by_dimer
  ∃ a b,
    PropertyKindCalculus.Dimension.MechanicallyReducible a.reducesTo ∧
      ¬PropertyKindCalculus.Dimension.MechanicallyReducible
            PropertyKindCalculus.Dim.temperature ∧
        b.unit = PropertyKindCalculus.ScaleSpanning.kelvin.unit
```

Proof for Theorem 12.2.4

uses 0

`scaleSpanning_not_determined_by_dimension : ∃ a b : ScaleSpanningUnit, Dimension.MechanicallyReducible a.reducesTo ∧ ¬ Dimension.MechanicallyReducible Dim.temperature ∧ b.unit = kelvin.unit` witnessed by `⟨candela, kelvin, …⟩`. Axiom-free. This is the requirement ISO 80000-7's candela and mole, and ISO 80000-5's kelvin, exercise on the standard.

13. The Interaction Algebra (Flater Appendix C)

This is the contribution beyond the VIM4 2CD and OML, and the reason Flater's note is titled *full tracking of kinds of quantities*. Dimensional analysis says torque and energy share a dimension (`\mathrm{M\,L^2\,T^{-2}}`), yet they are not the same kind and not interchangeable. The interaction algebra records which kinds legitimately combine, with the dimensional product as a coherence side-condition rather than the definition. It is now *realized*, sorry-free, in the `Interaction` module: the curated product, its division dual, the multiplication–division round-trip, and the worked torque-versus-energy instance.

13.1. The ternary product of kinds (Flater App. C)

Definition 13.1.1. group uses 2 used by 7 ✓

`\mathrm{KMul}\ k_1\ k_2\ k_3` holds when kind `k_1` times kind `k_2` yields kind `k_3`. It is partial and curated: an application asserts the legitimate products of its kinds, each carrying the dimensional coherence obligation discharged by the homomorphism theorem.

Lean code for Definition 13.1.1. 1 definition

`abbrev` · defined in `PropertyKindCalculus/Interaction.lean` — complete

```
abbrev PropertyKindCalculus.InteractionAlgebra.KMul
  (self : PropertyKindCalculus.InteractionAlgebra) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase →
    PropertyKindCalculus.DimensionedKind LTMCTDimensionBase →
      PropertyKindCalculus.DimensionedKind LTMCTDimensionBase → Prop
```

Proof for Definition 13.1.1 proof">uses 1

Realized as the `KMul` field of `InteractionAlgebra`: a ternary relation over dimensioned kinds, paired in the same structure with the well-formedness field `kMul_coherent` that requires every edge to satisfy `\dim k_3 = \dim k_1 \cdot \dim k_2`. The structure cannot be formed for an algebra that violates coherence.

Definition 13.1.2. group statement">used by 2 ✓

`\mathrm{KDiv}\ k_3\ k_2\ k_1` is the division dual of the kind product: `k_3` divided by `k_2` yields `k_1`. Fuel-consumption `\times` rainfall is the classic Flater example whose careless handling yields a category error rather than a number.

Lean code for Definition 13.1.2.

1 definition

`def` · defined in `PropertyKindCalculus/Interaction.lean` — complete

```
def PropertyKindCalculus.InteractionAlgebra.KDiv
  (A : PropertyKindCalculus.InteractionAlgebra)
  (c b a : PropertyKindCalculus.DimensionedKind LTMCTDimensionBase) :
  Prop
```

Proof for Definition 13.1.2

uses 0

Realized as `InteractionAlgebra.KDiv`, defined as the product read backwards (`\mathrm{KDiv}\ c\ b\ a := \mathrm{KMul}\ a\ b\ c`), so it inherits the same partiality and coherence side-condition.

Theorem 13.1.3.

group · uses 3 · used by 0 ✓

Effort	Priority	Tags
medium	high	capstone, proved

> ***Multiplication and division are inverse on kinds.*** *A product and its quotient hold together — the round-trip is an equivalence:* `\mathrm{KMul}\ k_1\ k_2\ k_3 \;\Longleftrightarrow\; \mathrm{KDiv}\ k_3\ k_2\ k_1`*. Together with dimensional coherence this is the calculus that OWL2 cannot host: an algebraic law with arithmetic side-conditions, proved rather than consistency-checked. Builds on the kind product and the kind quotient.*

Lean code for Theorem 13.1.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Interaction.lean` — complete

```
theorem PropertyKindCalculus.InteractionAlgebra.kMul_iff_kDiv
  (A : PropertyKindCalculus.InteractionAlgebra)
  (a b c : PropertyKindCalculus.DimensionedKind LTMCTDimensionBase) :
  A.KMul a b c ↔ A.KDiv c b a
```

Proof for Theorem 13.1.3

uses 0

Immediate from defining `KDiv` as the converse of `KMul`: the biconditional is `Iff.rfl`. The round-trip is the symmetry of that definition.

Theorem 13.1.4. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***Worked instance.*** *Torque times plane angle is energy, and energy is not torque, even though* `\dim(\mathrm{torque}) = \dim(\mathrm{energy})`*. Formally* `\mathrm{KMul}\ \mathrm{torque}\ \mathrm{angle}\ \mathrm{energy}` *holds while* `\mathrm{energy} \neq \mathrm{torque}` *— the small example that shows the algebra carrying information dimension alone discards. Uses the kind product.*

Lean code for Theorem 13.1.4. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Interaction.lean` — complete

```
theorem PropertyKindCalculus.torque_angle_work :
  PropertyKindCalculus.siMech.KMul PropertyKindCalculus.torqueKind
      PropertyKindCalculus.angleKind PropertyKindCalculus.energyKind ∧
    PropertyKindCalculus.energyKind.kind LTMCTDimensionBase ≠
        PropertyKindCalculus.torqueKind.kind LTMCTDimensionBase ∧
      PropertyKindCalculus.energyKind.toDimension =
        PropertyKindCalculus.torqueKind.toDimension
```

Proof for Theorem 13.1.4 uses 0

> Proved in the `Interaction` module. `torque`, `angle` (dimension one), and `energy` are declared as dimensioned kinds, with `torque` and `energy` both given dimension `force · length`; the product edge `torque_angle_energy` is asserted in the worked `siMech` algebra and its coherence checked by the `Dimension`-group computation, while `energy.kind ≠ torque.kind` is `by decide`.

13.2. The trust model: what is checked, what is authored

The negative pole — kinds in the name only. In an unkinded model every quantity is a bare numeric: a vegetation-attenuation kernel reads `attenuation (b ndvi : Float) : Float`, and the kind of each argument lives only in its *name*. The type system then cannot see any of the kind algebra: the swapped call `attenuation ndvi b` elaborates and silently computes the role-swapped exponent; a damping `lam : Float` adds to a Gram-diagonal entry `jtj : Float` whatever the entry's kind; a fitted gain from one forward model substitutes for the differently-kinded gain of another. Dimension checking does not recover the distinctions: attenuations, damping factors, and vegetation indices are all dimension one, so a dimensional type system waves every one of these confusions through (the torque-versus-energy

instance above is the same failure at dimension `\mathrm{M\,L^2\,T^{-2}}`). Name-only kinds are documentation; nothing enforces them.

The positive pole — the kind algebra is explicit. The same kernel written over `Quantity` makes the kind a type index and gates every kind-changing operation on a witness: `attenuationQ (b : Quantity paramB α) (ndvi : Quantity vegetationIndex α)` rejects the swapped call at compile time, the cross-kind sum `jtj + lam` fails to elaborate because addition is defined only at a *shared* kind (there is deliberately no opt-in for heterogeneous addition), and each product or exponential names its edge in full — `ProductKind.ofRatio paramB vegetationIndex attenuationExponent` — where it is used. Downstream models certify the rejections as build artifacts: a `#check_failure` probe succeeds only if the forbidden term fails to elaborate, so the build proves the negative examples *stay* negative.

Definition 13.2.1. group uses 0

The liberal certificate authority. `ProductKind.ofRatio k₁ k₂ k` signs *any* three ratio-scale kinds: the scale gate is checked, the kind equation is not. This is deliberate — the same dimension-one ratio can legitimately land in different kinds by role (a quotient of two backscatters is an attenuation in one column of a Jacobian and a pure number in another), so the landing kind is curation the calculus must let the author state. The cost is the trust model above: an edge's truth is a reviewed claim. The guarantee that remains is that the claim must be written, in full, where it is used — a wrong edge is not impossible, it is impossible to write *silently*.

Lean code for Definition 13.2.1.

1 theorem

theorem · defined in `PropertyKindCalculus/QuantityClassification.lean` — complete

```
theorem PropertyKindCalculus.ProductKind.ofRatio
  (k₁ k₂ k : PropertyKindCalculus.KindOfProperty)
  (h₁ : k₁.IsRational := by rfl) (h₂ : k₂.IsRational := by rfl)
  (h : k.IsRational := by rfl) :
  PropertyKindCalculus.ProductKind k₁ k₂ k
```

Proof for Definition 13.2.1 uses 0

Realized in `QuantityClassification` ("The trust model" in that module's header): the witness Props (`ProductKind`, `QuotientKind`, `TranscendentalKind`) carry only the ratio-scale gates, `ofRatio` discharges them by `rfl` for any concrete kinds, and `Quantity.mul`/`div`/`exp` refuse to operate without the witness.

Definition 13.2.2. group uses 0

Narrowing by instance table. `KindMul k₁ k₂ k` (with `k` an `outParam`) and its division dual `KindDiv` re-house authored edges as typeclass instances: a model registers at most one signed edge per operand pair next to its kind declarations, and adopting call sites use scoped `*`/`/` whose result kind is resolved

— not chosen — at each use. Where the table is adopted, an unlisted edge is no longer merely unwritten; it is unresolvable.

Lean code for Definition **13.2.2.** 1 definition

class (1 method) · defined in `PropertyKindCalculus/OperatorTable.lean` complete

```
class PropertyKindCalculus.KindMul
  (k₁ k₂ : PropertyKindCalculus.KindOfProperty)
  (k : outParam PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 13.2.2 proof">uses 1

> Realized in `OperatorTable`: the instance classes wrap the curated ternary product as resolution-driven registrations, with `hmul_eq_mul` bridging the scoped operator back to the witness-passing `Quantity.mul` by `rfl`.

The audit is mechanical. A witness is to the kind algebra what an axiom is to a proof, and the analogy extends to the tooling: soundness is judged by *enumerating* the trusted base, never by the absence of errors. Three queries cover it. Instance-table registrations are listed by the stock `#instances KindMul` command. Named witness theorems are found by scanning the environment for declarations whose type mentions the witness Props. And every *call-site* witness is lifted by Lean into an auxiliary theorem whose type *is* the edge (`attenuationQ._proof_2 : ProductKind paramB vegetationIndex attenuationExponent`), so one environment scan over constant types — internal names attributed to their parents — enumerates the complete authored edge set of a kind, named and inline alike: the kind-algebra analog of `#print axioms`.

14. The Function Calculus and Complex-Valued Carriers (R12)

The product / quotient / reciprocal calculus covers the *algebraic* defining relations of the ISQ (area is length times length, speed is length over duration). Scientific models do two further things the algebra alone does not reach: they apply *functions* to quantities (roots, exponentials, logarithms, the trigonometric functions), and they take *complex values* (the relative permittivity of a lossy dielectric, an impedance, a phasor). This chapter adds the two carrier-level extensions that cover both — the **function calculus** (a carrier *capability*, plus kind-relations for functions) and the **complex carrier** (a carrier *construction*) — and the point of each is that the kind layer is left untouched: only what a magnitude can *do*, or what numbers it lives *in*, changes.

14.1. Functions on quantities

Definition 14.1.1. group uses 0 statement">used by

`\mathrm{MathCarrier}\ R` is the **function operation surface** a kind-indexed magnitude needs — `\exp`, `\log`, `\sin`, `\cos`, the hyperbolic functions, `\sqrt{\,}`, `|\cdot|`, and the constant `\pi` — supplied once per representation type `R`. It is the function-layer analogue of the numeric carrier `Carrier`, and is by construction the *same* surface as the underlying tensor toolchain's elementary functions, so every carrier that supports them — `\mathbb{R}`, `Float`, the binary32 carriers — is a clean `\mathrm{MathCarrier}`.

Lean code for Definition 14.1.1. 1 definition

class (10 methods) · defined in `PropertyKindCalculus/QuantityFunction.lean` complete

```
class PropertyKindCalculus.MathCarrier (R : Type) : Type
```

Proof for Definition 14.1.1 uses 0

Realized as the `MathCarrier` class. The richer surface (`\tan`, the inverse trig, `\operatorname{atan2}`, cube root, rounding, rational power) is split off into `MathCarrierExt`, so the executable binary32 carrier — whose function vocabulary is deliberately smaller — is a `MathCarrier` without being a `MathCarrierExt`. The algebraic *identities* (`\sin^2+\cos^2=1`, `\exp(a+b)=\exp a\cdot\exp b`) live in `LawfulMathCarrier`, which holds at `\mathbb{R}` and *fails* in floating point — the same exec/spec gap the lawful numeric carrier draws for `+`. The `Float` instance is computable (the run carrier); the `\mathbb{R}` instance is the noncomputable proof carrier in the PhysLib-backed `Dimension` layer.

Definition 14.1.2. group statement">used by 3

`\mathrm{PowerKind}\ p\ k_1\ k` records that raising a `k_1`-quantity to the rational power `p` yields a `k`-quantity (`k = k_1^{\,p}`). In the core it carries only the ratio-scale gate (Dybkær §13.3.5 — only ratio quantities have powers); the dimension layer strengthens it to the exponent-scaling certificate `\dim k = (\dim k_1)^{p}`. This is the one function family that is *not* dimension-one-in-out — the dimension-scaling extension of multiplication and division.

Lean code for Definition 14.1.2.

1 definition

structure (2 fields) · defined in `PropertyKindCalculus/QuantityFunction.lean` — complete

```
structure PropertyKindCalculus.PowerKind (p : ℚ)
  (k₁ k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 14.1.2

proof">uses 1

Realized as the `PowerKind` structure (two ratio-scale fields), with the smart constructors `Quantity.sqrt` (the half-power), `Quantity.cbrt`, and `Quantity.rpow`. The exponent-scaling refinement is `DimPowerKind`, well-defined because PhysLib's `Dimension` carries `\mathbb{Q}` exponents (`Pow Dimension ℚ`), making a half-power a genuine group element.

Definition 14.1.3.

group uses 2 used by 5 ✓

Tags: proved

`\mathrm{Quantity.sqrt}` is the **verified half-power**, the canonical `PowerKind (1/2)` smart constructor: `\sqrt{\,}` of a `k_1` yields a `k`, its magnitude `\sqrt{a}` taken in the carrier. It rides on the base `MathCarrier` (not the extended surface), so it runs even on the binary32 executable carrier.

Lean code for Definition 14.1.3.

1 definition

`def` · defined in `PropertyKindCalculus/QuantityFunction.lean`

complete

```
def PropertyKindCalculus.Quantity.sqrt
  {k₁ k : PropertyKindCalculus.KindOfProperty} {R : Type}
  [PropertyKindCalculus.MathCarrier R]
  (_h : PropertyKindCalculus.PowerKind (1 / 2) k₁ k)
  (a : PropertyKindCalculus.Quantity k₁ R) :
  PropertyKindCalculus.Quantity k R
```

Proof for Definition 14.1.3

uses 0

Realized as `Quantity.sqrt`, taking a `PowerKind (1/2) k₁ k` and returning `(\mathrm{MathCarrier.sqrt} \ a.\mathrm{magnitude})`; the projection `sqrt_magnitude` is `rfl`, so an erased computation is exactly the carrier's `\sqrt{\,}`.

Theorem 14.1.4.

group uses 2 used by 0 ✓

Effort	Tags
small	proved

> ***The square root of an area is a length — checked, not annotated.*** *The half-power of the area dimension `\mathrm{L}^2` is the length dimension `\mathrm{L}`: `\dim(\sqrt{\mathrm{area}}) = (\dim \mathrm{area})^{1/2} = \mathrm{L}`. The certificate `Quantity.sqrt` carries, here discharged in the PhysLib `Dimension` group. Builds on the power law.*

Lean code for Theorem 14.1.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Function.lean`

complete

```
theorem PropertyKindCalculus.sqrt_area_coherent :
  PropertyKindCalculus.DimPowerKind (1 / 2) PropertyKindCalculus.areaK
    PropertyKindCalculus.lengthKind
```

Proof for Theorem 14.1.4

uses 0

Proved in the `Function` module: `areaK` is declared with dimension `\mathrm{L}^2` and the goal `lengthKind.dim = areaK.dim ^ (1/2 : ℚ)` is closed component-wise by `norm_num` over the rational exponents of the `Dimension` group.

Definition 14.1.5. group uses 3 used by 0 ✓

`\mathrm{TranscendentalKind}\ k_1\ k` licenses the smart constructors for `\exp`, `\log`, the hyperbolic and trigonometric functions. The core records the ratio-scale gate; the dimension layer strengthens it to "argument and result are dimension one" (Families C), and — for the trigonometric functions — to a *curated* algebra (the unary analogue of the interaction algebra) that keeps `\sin` on plane angles only. That a plane angle is dimension one yet a distinct *kind* is exactly why `\sin` of an angle is fine while `\sin` of a reflectivity is the category error a dimension-only system cannot forbid.

Lean code for Definition 14.1.5. 1 definition

structure (2 fields) · defined in `PropertyKindCalculus/QuantityFunction.lean` — complete

```
structure PropertyKindCalculus.TranscendentalKind
  (k₁ k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 14.1.5 uses 0

Realized as `TranscendentalKind` with the constructors `Quantity.exp` / `log` / `sin` / `cos` / `tanh` (base surface) and `tan` / `asin` / `acos` / `atan` / `atan2` (extended surface). The dimensional strengthening is `DimTranscendental` (`tanh_lossTangent_coherent` is the worked edge: the conductive-loss `\tanh` argument is dimensionless and so is its value), and the trigonometric curation is the `Dimension`-layer trig algebra.

14.2. Complex-valued carriers

Definition 14.2.1. group uses 0 used by 0 ✓

Tags: proved

`\mathrm{Complex}\ R` is the **complexification of a carrier** — the functor `R \mapsto \mathrm{Complex}\ R` that, given the real arithmetic of `R`, equips `(\mathrm{re}, \mathrm{im})` with the complex arithmetic. A complex-valued quantity is `\mathrm{Quantity}\ k\ (\mathrm{Complex}\ R)`; the carrier varies (`\mathbb{R}` to prove, `Float` to run) while the kind `k` does not.

This is a **fourth** point on the representation axis (R10): the three scalar carriers (`\mathbb{R}` to prove, a binary32 rounding spec to certify, `Float` to run) are all real, and `\mathrm{Complex}\ R` lifts each of them

— `\mathrm{Complex}\ \mathbb{R}` to prove, `\mathrm{Complex}\ (\mathrm{binary32})` to certify rounding, `\mathrm{Complex}\ \mathrm{Float}` to run. It is a `Carrier`, and a `LawfulCarrier` whenever `R` is (complex addition is componentwise, so the additivity laws lift for free); it carries a full field `+\,-\,\times\,\div` but is *unordered*, so — unlike the real carriers — it sits outside the ordinal/interval/ratio carrier tower: complex magnitudes are compared by modulus, not ranked.

Lean code for Definition 14.2.1.

1 definition

structure (2 fields) · defined in `PropertyKindCalculus/Complex.lean` — complete

```
structure PropertyKindCalculus.Complex (R : Type) : Type
```

Proof for Definition 14.2.1

uses 0

Realized as `Complex R` with the toolchain instances (`+`, `-`, `\times`, `\div`, negation, zero, one), a `Carrier` instance, and `|z|^2` / `|z|`. Why parametric rather than Mathlib's `\mathbb{C}`? For the same reason the carrier is a parameter at all: `\mathbb{C}` is *noncomputable* (its parts are `\mathbb{R}`, which does not reduce — no `\mathbb{C}`-valued program runs or compiles, yet the executable carrier must be `Float`-backed), and it is *hardwired to* `\mathbb{R}` (there is no "`\mathbb{C}` over `Float`"), whereas the representation axis needs complex over `Float`, over a binary32 rounding spec, and over `\mathbb{R}` from one definition. As with the reals — where `\mathbb{R}` is *one instance* of the carrier, not the universal one — Mathlib's `\mathbb{C}` is the natural *proof-carrier target*: a ring isomorphism `\mathrm{Complex}\ \mathbb{R} \simeq \mathbb{C}` (its home the `Dimension` layer) transports any property of complex quantities at `\mathbb{R}` to `\mathbb{C}` and borrows its analysis library, while the run side stays on `\mathrm{Complex}\ \mathtt{Float}`.

Definition 14.2.2.

group — uses 0 — proof">used by 1 ✓

Tags: proved

The **principal complex square root**. The algebraic operations are total and single-valued, so they are plain instances; the square root is the exception — it has a branch cut along the negative real axis. Selecting the principal branch needs exactly one sign decision (the sign of the imaginary part, with `\operatorname{sign} 0 = +`), isolated as the one-field capability `NegTest` so the branch-*free* complex algebra needs only the toolchain.

Lean code for Definition 14.2.2.

1 definition

`def` · defined in `PropertyKindCalculus/Complex.lean`

complete

```
def PropertyKindCalculus.Complex.sqrt {R : Type} [One R] [Add R] [Sub R]
  [Mul R] [Div R] [Neg R] [PropertyKindCalculus.MathCarrier R]
  [PropertyKindCalculus.Complex.NegTest R]
  (z : PropertyKindCalculus.Complex R) : PropertyKindCalculus.Complex R
```

Proof for Definition 14.2.2

proof">uses 1

Realized as `Complex.sqrt`: `\sqrt{z} = \sqrt{\tfrac{|z|+\mathrm{re}}{2}} \;+\; \operatorname{sign}(\mathrm{im})\,\sqrt{\tfrac{|z|-\mathrm{re}}{2}}\,j`, the real and imaginary magnitudes two real square roots and the imaginary *sign* the one `NegTest` decision (`Float` deciding it computably, `\mathbb{R}` classically). `Complex` is deliberately *not* made a full `MathCarrier`: a complex `\exp`/`\log`/`\sin` suite would drag in a quadrant-aware angle and an unprovable lawfulness story the metrology does not need — the one transcendental the electromagnetic forward model uses is given directly and honestly as its own branch-tracked constructor.

Theorem 14.2.3.

group uses 2 used by 0 ✓

Effort	Tags
small	proved

The kind-tracked complex square root. *The complex analogue of `Quantity.sqrt`, licensed by the same `PowerKind (1/2)`: the complex square root of a (dimension-one) relative permittivity is a (dimension-one) refractive index — a checked half-power crossing of kinds, now over a complex carrier. The complexification leaves the kind layer untouched, so this is the only new kind-level constructor the complex numbers require.*

Lean code for Theorem 14.2.3.

1 definition

`def` · defined in `PropertyKindCalculus/Complex.lean`

complete

```
def PropertyKindCalculus.Quantity.csqrt
  {k₁ k : PropertyKindCalculus.KindOfProperty} {R : Type} [One R]
  [Add R] [Sub R] [Mul R] [Div R] [Neg R]
  [PropertyKindCalculus.MathCarrier R]
  [PropertyKindCalculus.Complex.NegTest R]
```

```
(_h : PropertyKindCalculus.PowerKind (1 / 2) k₁ k)
(a :
  PropertyKindCalculus.Quantity k₁ (PropertyKindCalculus.Complex R)) :
PropertyKindCalculus.Quantity k (PropertyKindCalculus.Complex R)
```

Proof for Theorem 14.2.3

proof">uses 1

Realized as `Quantity.csqrt`, the smart constructor wrapping the principal complex square root under the same `PowerKind (1/2)` certificate as the real half-power; named `csqrt` because the complex root is a genuinely distinct operation (it has a branch cut) even though its kind discipline is identical.

15. Extensivity

Dybkær §13.5 divides kinds-of-quantity "according to physical (and arithmetic) additivity", presenting Bunge's four types: *unconditionally extensive* (§13.5.1, the value for the total equals the arithmetic sum over the parts), *quasiextensive* (§13.5.2, approximately equal — an uncertainty question, not a mereological one), *conditionally extensive* (§13.5.3, the addition exists but the total may differ "due to the respective internal and environmental conditions" — volume on mixing), and *intensive* (§13.5.4, no physical addition, the value "invariant with the extent of a system of constant composition"). These are quantified arithmetic laws over the mereology of systems — again something a description logic cannot state — and all four are *realized*, sorry-free: three of them in the Mathlib-free `Extensivity` module — the predicates over the `Mereology` module's carving, the two inductions that lift a single split to the whole tree, and a checked witness for each — and §13.5.2 in `Uncertainty.QuasiExtensive`, because "approximately" is a claim about measurement and not about the mereology.

Four cases Bunge's four do not name are added. A value the parts do not determine at all (*whole-proper*); a value that adds only about a *shared parameter* — an axis, a frame, a reference point — where the parallel-axis theorem lives and the transport tower with it; a *weighted mean*, which is neither a sum nor a shared constant and whose license is that the total weight is not zero; and a value that reads no part at all but an *interface* between two, whose additivity is purchased by a cancellation law. The chapter closes on what a carving cannot say at all: how many parts there are.

15.1. Extensive and conditionally-extensive kinds (Dybkær §13.5)

Definition 15.1.1. group uses 2 used by 10 ✓

A kind is *extensive* when, for any carving of a system into parts, the quantity of the whole equals the sum over the parts — the mereological structure the foundations chapter supplies.

Lean code for Definition 15.1.1. 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
structure PropertyKindCalculus.Extensive.{u} {O : Type u}
  (k : PropertyKindCalculus.KindOfProperty)
  (m : PropertyKindCalculus.Measurement O) : Prop
```

Proof for Definition 15.1.1 uses 0

Realized as `Extensive k m`, a predicate over a kind `k` and a measurement `m` (the property value observed on each node of a `Decomposition`): it asserts that every part is measured as a value of kind `k`, and that the numeral on a disjoint union is the sum of the numerals on its two parts.

Theorem 15.1.2. group statement">statement proof ✓

Effort: medium

Tags: capstone, proved

> ***Extensive aggregation.*** *For an extensive kind `k` and a system decomposed into parts `s_1, \dots, s_n`, `\mathrm{value}_k\Bigl(\bigsqcup_i s_i\Bigr) = \sum_i \mathrm{value}_k(s_i)`. The `\forall`-quantified additivity law itself — not a single instance — is the deliverable, and it is exactly what OWL2 has no vocabulary for. Builds on extensive kinds.*

Lean code for Theorem 15.1.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
theorem PropertyKindCalculus.extensive_additive.{u} {O : Type u}
  {k : PropertyKindCalculus.KindOfProperty}
  {m : PropertyKindCalculus.Measurement O}
  (h : PropertyKindCalculus.Extensive k m)
  (d : PropertyKindCalculus.Decomposition O) :
  (m d).numeral = PropertyKindCalculus.leafSum m d
```

Proof for Theorem 15.1.2

uses 0

Proved in the `Extensivity` module by induction on the `Decomposition`: at a leaf the law is reflexivity; at a union the single-split `additive` field plus the two induction hypotheses give the numeral of the whole as the sum of the leaf sums.

Theorem 15.1.3. group statement">uses 1 used by 0 ✓

Effort: small

Tags: proved

> ***Counterexample — volume on mixing is not extensive.*** *Volume fails the additivity law for a water/ethanol mixture: the volume of the mixture is strictly less than the sum of the component volumes. Stating the negation keeps the extensive predicate honest. Uses extensive kinds.*

Lean code for Theorem 15.1.3. 1 theorem

theorem · defined in PropertyKindCalculus/Extensivity.lean — complete

```
theorem PropertyKindCalculus.mixing_subadditive :
  (PropertyKindCalculus.volMix PropertyKindCalculus.mixture).numeral <
      (PropertyKindCalculus.volMix
            PropertyKindCalculus.waterPart).numeral +
        (PropertyKindCalculus.volMix
            PropertyKindCalculus.ethanolPart).numeral ∧
    ¬PropertyKindCalculus.Extensive PropertyKindCalculus.volume
        PropertyKindCalculus.volMix
```

Proof for Theorem 15.1.3 uses 0

Proved in the `Extensivity` module. 50 mL of water and 50 mL of ethanol measure 96 mL when mixed; the strict inequality `96 < 50 + 50` is `by decide`, and the same `volMix` measurement witnesses `\neg\, \mathrm{Extensive}\ \mathrm{volume}` by refuting its `additive` field on that two-part decomposition.

15.2. Intensity, whole-properness, and what a carving cannot say

A `Decomposition` is a *carving* and not a census. Nothing in it fixes how many parts a system has, and every law of this chapter is quantified over all carvings rather than stated for one — which is why the `\forall`-quantified form is the deliverable: a total that depends on how the whole was cut is not a total. That is Marmodoro's point about physical structure, which "unites" without bringing a count principle with it, so that "It is an open question how many entities a physical structure is" (Marmodoro, 2018)[15] Anna Marmodoro, 2018. “Whole, but not One”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).* . What a carving therefore cannot supply is the *whole*; that arrives with a sort of system (the foundations chapter) and its aggregation license (the object-type chapter).

Additivity is one answer to what composition does to a value, and the alternatives are not the absence of an answer. Both below are stated against the same measurement, so a model that claims one is claiming something refutable.

Definition 15.2.1. group uses 0 used by 2 ✓

A kind is *intensive* (§13.5.4) when composing two parts **that already agree** leaves the value where it was: temperature, density, the angular velocity of a rigid body. The hypothesis is Dybkær's own, read literally — his definition asks that the value be "invariant with the extent of a system of constant

composition", and *of constant composition* is the clause a shorter phrasing drops. Without it the law would assert that mixing water at 20 °C with water at 80 °C leaves either reading intact.

Lean code for Definition 15.2.1.

1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
structure PropertyKindCalculus.Intensive.{u} {O : Type u}
  (k : PropertyKindCalculus.KindOfProperty)
  (m : PropertyKindCalculus.Measurement O) : Prop
```

Proof for Definition 15.2.1

proof">uses 1

Realized as `Intensive k m`, the same shape as extensivity with the additivity field replaced by a conditional constancy field. What the structure says about parts that *disagree* is nothing at all: an intensive kind licenses no reading of the whole from unequal parts, which is a narrower claim than a total law and an honest one.

Theorem 15.2.2.

group statement">used by 0 ✓

Effort
medium

Tags
capstone
proved

Intensive aggregation. *For an intensive kind, a whole all of whose atomic parts read v reads v itself, over any carving and to any depth. Builds on intensive kinds.*

Lean code for Theorem 15.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
theorem PropertyKindCalculus.intensive_uniform.{u} {O : Type u}
  {k : PropertyKindCalculus.KindOfProperty}
  {m : PropertyKindCalculus.Measurement O} {v : ℤ}
  (h : PropertyKindCalculus.Intensive k m)
  (d : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.Decomposition.Forall
      (fun s =>
        (m (PropertyKindCalculus.Decomposition.atom s)).numeral = v)
```

```
    d →
  (m d).numeral = v
```

Proof for Theorem 15.2.2

uses 0

By induction on the decomposition, exactly as the extensive capstone: at a leaf the hypothesis is the conclusion; at a union the two induction hypotheses make the parts agree, which is what the conditional constancy field consumes. The leaf-wise quantifier is `Decomposition.Forall` — extensivity reaches its leaves through arithmetic (`leafSum`) and intensivity has no arithmetic to reach them with.

Theorem 15.2.3.

group uses 2 used by 0 ✓

Effort	Tags
small	proved

The branches exclude each other. *An intensive kind is not extensive as soon as two parts agree on a* ***nonzero*** *value: additivity doubles it, constancy keeps it, and only `0 = 0 + 0` satisfies both. Uses intensive kinds and extensive kinds.*

Lean code for Theorem 15.2.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
theorem PropertyKindCalculus.not_extensive_of_intensive.{u} {O : Type u}
  {k : PropertyKindCalculus.KindOfProperty}
  {m : PropertyKindCalculus.Measurement O}
  (hi : PropertyKindCalculus.Intensive k m)
  {a b : PropertyKindCalculus.Decomposition O}
  (hab : (m a).numeral = (m b).numeral) (hne : (m a).numeral ≠ 0) :
  ¬PropertyKindCalculus.Extensive k m
```

Proof for Theorem 15.2.3

uses 0

Cancellation on `v + v = v + 0`. The nonzero hypothesis is stated rather than assumed away, because a kind whose every reading is zero really does satisfy both laws, vacuously. The library's witness is density: two parcels of one homogeneous fluid read 1000 kg/m³ and so does the parcel they compose, which refutes additivity outright.

Definition 15.2.4.

group uses 0 statement">used by ✓

A kind is *whole-proper* when the value of the whole is produced by **no** aggregation over the parts: neither by summing them nor by sharing a common reading with them. The normal-mode frequency of a coupled pair is the standard case — a lone oscillator has no normal mode, so there is nothing to sum

and nothing to inherit. Bunge's four types do not name this case; under the formalizations here it satisfies none of them, and in particular it is not intensive, the value not being invariant with the extent of the system.

Lean code for Definition 15.2.4.

1 definition

`structure` (3 fields) · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
structure PropertyKindCalculus.WholeProper.{u} {O : Type u}
  (k : PropertyKindCalculus.KindOfProperty)
  (m : PropertyKindCalculus.Measurement O) : Prop
```

Proof for Definition 15.2.4

uses 0

A `Prop` structure whose two refutation fields are **exhibited**, as pairs of atomic parts, rather than stated as negated universals: a negated universal is satisfied by a measurement that is merely undefined somewhere, while a witness pins the failure to two parts a reader can point at. What names the case is Marmodoro's distinction rather than Dybkær's vocabulary: her structural powers constitute a whole out of parts, where the substantial power "**is** the electron" (Marmodoro, 2018)[16] Anna Marmodoro, 2018. “Whole, but not One”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).* — borne by the whole, and by nothing the whole is made of.

16.

Theorem 15.2.5.

group statement"> uses 1 by 0 ✓

Effort	Tags
small	proved

The whole-proper case is occupied. *Two oscillators at `\omega_0 = 10` rad/s, coupled so the upper normal mode sits at `\omega_+ = 14` rad/s: the pair's reading is neither the parts' sum (20) nor their common value (10). Uses whole-proper kinds.*

Lean code for Theorem 15.2.5.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Extensivity.lean` — complete

```
theorem PropertyKindCalculus.normalMode_wholeProper :
  PropertyKindCalculus.WholeProper
```

```
PropertyKindCalculus.oscillatorFrequency
PropertyKindCalculus.normalModeFreq
```

Proof for Theorem 15.2.5 proof">uses 1

Both refutations are `decide` on the witness measurement, and the two derived theorems — not extensive, not intensive — follow from the exhibited pair alone. The quantity-level consequence is assembling it anyway, in the object-type chapter: with a license registered the sum still elaborates, and reports 20 rad/s for a mode at 14.

15.3. Additivity to within a tolerance (§13.5.2)

The remaining branch of Bunge's four says the value for the total is "approximately equal" to the sum over the parts. *Approximately* is a claim about measurement, so the predicate is stated where the uncertainty is, over `\mathbb{R}` rather than over the numeral, and its tolerance is a coverage factor rather than a number chosen to make the claim come out true.

Definition 15.3.1. group statement">used by 2 ✓

A kind is *quasiextensive to within* `t` (§13.5.2) when composing two parts is additive up to `t` — at *each join*, not once for the whole. Naming the tolerance is what turns "approximately" into a claim: two quasiextensive claims about one measurement are then comparable, and the sharper is the one with the smaller `t`. Refines extensivity.

Lean code for Definition 15.3.1. 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Uncertainty/QuasiExtensive.lean` complete

```
structure PropertyKindCalculus.Uncertainty.QuasiExtensive.{u'} {O : Type u'}
  (k : PropertyKindCalculus.KindOfProperty)
  (m : PropertyKindCalculus.Measurement O) (t : ℝ) : Prop
```

Proof for Definition 15.3.1 uses 2

Realized as `Uncertainty.QuasiExtensive k m t`, the same two-field shape as `Extensive` with the additivity equation replaced by `\lvert v(a \sqcup b) - (v(a) + v(b)) \rvert \le t`. Where the tolerance comes from is R18's business: `join_within_tolerance` specializes the Chebyshev coverage bound to a mean-zero join discrepancy, so a per-join tolerance of `k` standard uncertainties carries probability at least `1 - 1/k^2`.

Theorem 15.3.2. group statement">used by 0 ✓

Effort
medium

Tags
capstone
proved

> ***Quasiextensive aggregation.*** *For a kind quasiextensive to within `t`, the value measured on the whole differs from the sum over all atomic parts by at most the carving's join count times the tolerance, `\Bigl\lvert v\Bigl(\bigsqcup_i s_i\Bigr) - \sum_i v(s_i) \Bigr\rvert \le \mathrm{joins} \cdot t`. That the bound grows with the carving is the content, not a weakness of the proof: a library reporting one tolerance for a total however the total was assembled is reporting the wrong number, and this says by how much. Builds on quasiextensive kinds.*

Lean code for Theorem 15.3.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/QuasiExtensive.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.quasiExtensive_leafSum.{u'}
  {O : Type u'} {k : PropertyKindCalculus.KindOfProperty}
  {m : PropertyKindCalculus.Measurement O} {t : ℝ}
  (h : PropertyKindCalculus.Uncertainty.QuasiExtensive k m t)
  (d : PropertyKindCalculus.Decomposition O) :
  |↑(m d).numeral - ↑(PropertyKindCalculus.leafSum m d)| ≤ ↑d.joins * t
```

Proof for Theorem 15.3.2

uses 0

> By induction on the decomposition, exactly as the §13.5.1 capstone: at a leaf the discrepancy is zero over zero joins, and at a union the triangle inequality splits it into this join's tolerance plus the two sub-carvings' accumulated bounds. The witness is a balance whose composite readings carry one digit of rounding — quasiextensive to within 1, and not extensive, so the branch is occupied by something §13.5.1 excludes.

Theorem 15.3.3.

group · uses 2 · used by 0 ✓

Effort
small

Tags
proved

> ***§13.5.1 is §13.5.2 at zero tolerance.*** *A kind is extensive exactly when it is quasiextensive to within nothing at all — so the branches are nested rather than parallel, and a tolerance is not hiding a weaker law. Uses quasiextensive kinds and extensive kinds.*

Lean code for Theorem 15.3.3. 1 theorem

theorem · defined in PropertyKindCalculus/Uncertainty/QuasiExtensive.lean — complete

```
theorem PropertyKindCalculus.Uncertainty.extensive_iff_quasiExtensive_zero.
{u'}
  {O : Type u'} {k : PropertyKindCalculus.KindOfProperty}
  {m : PropertyKindCalculus.Measurement O} :
  PropertyKindCalculus.Extensive k m ↔
    PropertyKindCalculus.Uncertainty.QuasiExtensive k m 0
```

Proof for Theorem 15.3.3 uses 0

Forward, the additivity equation makes the discrepancy zero; backward, `\lvert x \rvert \le 0` forces `x = 0`. The tolerance is shown to be doing work by the other direction of the same witness family: volume on mixing refutes *every* tolerance below the 4 mL water and ethanol actually contract by (`mixing_not_quasiExtensive`), so §13.5.3 reads as §13.5.2 with a tolerance the conditions set.

15.4. Aggregating about a shared parameter

Some quantities add only relative to something the parts must share — a moment of inertia about a common axis, an energy about a common datum. A `Measurement` reads a carving and nothing else, so the parameter is made an explicit argument; what that buys is not a weaker law but a statable one, about the case a library that forgets the axis actually hits.

The parameter is kind-specific — an axis for a moment of inertia, a **frame** for a momentum, a **reference point** for an angular momentum — and the transports relating two parameter values form a tower: each correction is priced in a summary that is itself extensive one rung down, grounding out in mass. The worked witness is a rover (`RoverExtensivity`, examples): whether its four wheels' spins, each read about its own axle, may be summed against the chassis is decided by the priced correction — each assembly's momentum — which vanishes parked and does not while driving.

Definition 15.4.1. group statement">used by 2 ✓

A kind is *extensive about* a parameter when fixing the parameter — an axis, an origin, a datum — makes the ordinary additivity law hold. This is extensivity at every parameter at once, not a weakening of it, which is what makes the mixed-parameter statement a claim about the predicate rather than about one lucky choice.

Lean code for Definition 15.4.1.

1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Extensivity.lean`

complete

```
structure PropertyKindCalculus.ExtensiveAbout.{u} {A : Type} {O : Type u}
  (k : PropertyKindCalculus.KindOfProperty)
  (m : PropertyKindCalculus.ParamMeasurement A O) : Prop
```

Proof for Definition 15.4.1

uses 0

Realized as `ExtensiveAbout k m` over a `ParamMeasurement`, with `ExtensiveAbout.at` recovering `Extensive k (m a)` at each parameter so both §13.5 capstones apply unchanged. Its companion `Transports` carries the correction as an argument rather than defining it as the difference: a transport law is useful only when the correction is computable from data the part already carries.

Theorem 15.4.2.

group statement">statement">used by

Effort
medium

Tags
capstone
proved

> ***The parallel-axis theorem, over any carving.*** *The moment of inertia about `a` is the moment about `b` plus a correction built from the carving's own first moment and total mass, `I_a = I_b + 2(b-a)\sum_i w_i x_i + (a^2 - b^2)\sum_i w_i`. Both summaries are themselves extensive over the same carving, which is why this is a metrological statement and not only an algebraic identity: transporting an aggregate costs only quantities the parts already carry. Taking `b` at the centre of mass kills the first moment and leaves the textbook `M d^2`. Builds on extensivity about a parameter.*

Lean code for Theorem 15.4.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/AggregationLaws.lean`

complete

```
theorem PropertyKindCalculus.parallelAxis.{u} {O : Type u} (w x : O → ℤ)
  (a b : ℤ) (d : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.inertiaAbout w x a d =
    PropertyKindCalculus.inertiaAbout w x b d +
      (2 * (b - a) * PropertyKindCalculus.firstMoment w x d +
        (a * a - b * b) * PropertyKindCalculus.partTotal w d)
```

Proof for Theorem 15.4.2

uses 0

By induction on the carving: at a leaf it is ring normalization of `w(x-a)^2 = w(x-b)^2 + 2(b-a)wx + (a^2-b^2)w`, and at a join the three folds distribute. The `Int` ring arithmetic is why it lives in the Mathlib-backed `AggregationLaws` rather than the core. What the core exhibits is the failure it corrects: two point masses read about axes through themselves contribute nothing each, where the rod about its centre reads 2.

Theorem 15.4.3.

group statement">statement">used by

Effort	Tags
small	proved

A momentum transports between frames, priced in mass. *The momentum read in the frame at `u_1` is the momentum read at `u_2` plus the carving's mass times the frame difference, `p_{u_1} = p_{u_2} + \Bigl(\sum_i w_i\Bigr)(u_2 - u_1)`. The parameter of extensivity about a parameter is a* ***frame*** *here rather than an axis, and the failure it prices is reading each part in its own rest frame: every reading is zero, so the untransported sum reports a system with no momentum at all.*

Lean code for Theorem 15.4.3.

1 theorem

theorem · defined in `PropertyKindCalculus/AggregationLaws.lean`

complete

```
theorem PropertyKindCalculus.momentumBoost.{u} {O : Type u} (w v : O → ℤ)
  (u₁ u₂ : ℤ) (d : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.momentumIn w v u₁ d =
    PropertyKindCalculus.momentumIn w v u₂ d +
      PropertyKindCalculus.partTotal w d * (u₂ - u₁)
```

Proof for Theorem 15.4.3

uses 0

By induction on the carving: `ring` at a leaf on `w(v-u_1) = w(v-u_2) + w(u_2-u_1)`, folds distributing at a join. The core exhibits the failure (`drift_rest_frames_wrong`: two drifting masses read 0 in their own rest frames where the laboratory reads the pair at 3) and `drift_rest_frames_corrected` closes the gap from the general law, with the two corrections being the parts' own laboratory momenta.

Theorem 15.4.4.

group uses 2 used by 0

Effort	Tags
small	proved

> ***An angular momentum transports between reference points, priced in momentum.*** *Componentwise over any carving, `L_a = L_b + (b - a) \times p`, with the momentum components entering as the first moments of the velocities. This is the middle rung of the transport tower: inertia's correction is priced in mass and first moment, this one in momentum, and momentum's in mass — each price itself extensive one rung down, so mixed-parameter aggregation is checkable all the way to the ground.*

Lean code for Theorem **15.4.4.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/AggregationLaws.lean` — complete

```
theorem PropertyKindCalculus.angularMomentumTransport.{u} {O : Type u}
  (w x y vx vy : O → ℤ) (a b : ℤ × ℤ)
  (d : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.angularMomentumAbout w x y vx vy a d =
    PropertyKindCalculus.angularMomentumAbout w x y vx vy b d +
      ((b.1 - a.1) * PropertyKindCalculus.firstMoment w vy d -
        (b.2 - a.2) * PropertyKindCalculus.firstMoment w vx d)
```

Proof for Theorem 15.4.4 uses 0

> By induction on the carving, `ring` at a leaf. Taking `b` at a part's own mass centre and `a` at the system's shows the spin/orbital split of an angular momentum is this law and not a new principle, and a part whose momentum vanishes reads the same about every point. The rover instantiates both readings: each wheel's spin reads 2 about its own axle, parked or driving; parked, every assembly's momentum vanishes and the four spins sum against the chassis unchanged, while driving each transport costs −15 and the untransported sum misses the whole by exactly the four prices.

15.5. The interface ledger –– what parts exert across a cut

Every measurement above reads a **node** of a carving. A drive torque on a hub, a contact force at a joint, the heat crossing a wall read no node at all: each is indexed by an ordered **pair** of parts — an interface — and a `Measurement` has no slot for a pair. The ledger is that slot, and its one law is the reason such quantities nonetheless aggregate.

Definition **15.5.1.** group statement">statement">used by

> An *interface ledger* over the parts of a carving is a pairwise action carrying its cancellation law as a field: the two readings of one interface sum to zero — for mechanical actions, Newton's third law. A pair of readings that does not cancel is not two readings of one interface, so the obligation is a field for the reason a re-carving's whole-preservation is.

Lean code for Definition 15.5.1.

1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/InterfaceLedger.lean` — complete

```
structure PropertyKindCalculus.InterfaceLedger.{u} (O : Type u) : Type u
```

Proof for Definition 15.5.1

uses 0

Realized as `InterfaceLedger` — `act` and `antisymm` — with `netOn` and `netTotal` reading per-part nets against a carving and `mutualTotal` the flow across a cut. A ledger is entered one direction at a time: `fun p q => raw p q - raw q p` antisymmetrizes any raw table, and its law is `omega`. `act_self` is the law's own erasure clause: a carving that merges the two ends of an interface hands both ends one name and the entry reads zero — which is why the rover example's two drive trains (2/2 and 3/1 hub-to-rim splits) coarsen to the **same** ledger, so internal-versus-external is the carving's fact rather than the machine's — Dybkær's observer clause (§3.3 Note 5) with a proof attached.

Theorem 15.5.2.

group statement">used by 2 ✓

Effort	Tags
small	proved

The exact price of additivity. *For* ***any*** *pairwise action over a carving, lawful or not, rolling up per-part nets over a union overshoots the two halves' own rollups by precisely the two cross-flows over the cut. A ledger that drops a reaction does not fail additivity vaguely — it fails it by this number. Stated with no law in scope, so it is the identity a violated law is measured against.*

Lean code for Theorem 15.5.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/InterfaceLedger.lean` — complete

```
theorem PropertyKindCalculus.netTotal_union.{u} {O : Type u} (act : O → O → ℤ)
  (ext : O → ℤ) (a b : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.netTotal act ext (a.union b) =
    PropertyKindCalculus.netTotal act ext a +
        PropertyKindCalculus.netTotal act ext b +
      (PropertyKindCalculus.mutualTotal act a b +
        PropertyKindCalculus.mutualTotal act b a)
```

Proof for Theorem 15.5.2

uses 0

Six instances of the fold's linearity, assembled by `omega`. The rover example presents the invoice: log the motor torques and forget the reactions, and the whole's rollup exceeds its halves' by 16 — four motors' worth of uncancelled flow across the chassis–arms cut.

Theorem 15.5.3.

group uses 2 proof">used by 1 ✓

Effort	Tags
small	proved

The rollup. *Under the cancellation law the price at every cut is zero, so the sum over the parts of the per-part nets is the exogenous total alone. The reaction a motor induces on the body is real in the per-part net — it loads a bearing — and absent from the whole; both facts are this theorem, read at a part and at the carving. Builds on the ledger and the price identity.*

Lean code for Theorem 15.5.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/InterfaceLedger.lean`

complete

```
theorem PropertyKindCalculus.InterfaceLedger.netTotal_eq_extTotal.{u}
  {O : Type u} (L : PropertyKindCalculus.InterfaceLedger O)
  (ext : O → ℤ) (d : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.netTotal L.act ext d =
    PropertyKindCalculus.partTotal ext d
```

Proof for Theorem 15.5.3

uses 0

The flows across any cut cancel (`mutualTotal_cancel`, by induction from the field), so the interior of one carving cancels outright and only the exogenous column survives. In the rover: the chassis's net reads −16 (four motors' reactions), each hub's balances to zero, and the rover's total is the ground torque alone.

Theorem 15.5.4.

group uses 2 used by 0 ✓

Effort	Tags
medium	capstone, proved

Additivity, purchased. *The net over a ledger satisfies the extensive law — but where mass's `additive` field is the model's free choice, this one is a theorem with a hypothesis: delete the cancellation field and the price identity prices exactly what remains. One predicate, two different*

> *licenses, which is the sense in which the **scope** of extensivity is kind-specific: what varies between kinds is not whether a sum appears but which law licenses it.*

Lean code for Theorem **15.5.4.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/InterfaceLedger.lean` — complete

```
theorem PropertyKindCalculus.InterfaceLedger.netMeasurement_extensive.{u}
  {O : Type u} (L : PropertyKindCalculus.InterfaceLedger O)
  (k : PropertyKindCalculus.KindOfProperty) (ref : String)
  (ext : O → ℤ) :
  PropertyKindCalculus.Extensive k (L.netMeasurement k ref ext)
```

Proof for Theorem 15.5.4 proof">uses 1

> Both fields follow from [the rollup]: the kind is fixed by construction, and additivity collapses to the fold's union case once each rollup is its exogenous total. The rover example instantiates it at the torque kind, with the lawless counter-witness beside it.

15.6. The weighted mean, and its license

A third aggregation mode, neither additive nor constant. Its side condition is the one a formula written without it silently violates — where a centre of mass defined as a ratio of integrals with no mass hypothesis returns the origin for a massless body.

Definition **15.6.1.** group uses 0 used by 7 ✓

> A *weighted carving* is a carving, a weight per part, and the license the mean needs: the total weight is not zero. Carrying the condition as a field is the point — a whole with no weight is not a term of the type, so the degenerate case is unreachable rather than handled.

Lean code for Definition **15.6.1.** 1 definition

`structure (3 fields)` · defined in `PropertyKindCalculus/Aggregation.lean` — complete

```
structure PropertyKindCalculus.WeightedCarving.{u} (R : Type) (P : Type u)
  [PropertyKindCalculus.Carrier R] : Type u
```

Proof for Definition 15.6.1 uses 0

Realized as `WeightedCarving R P` in the Mathlib-free core (`Aggregation`), carried at whatever numbers the task uses: `WeightedCarving.mean` is the ratio of the weighted fold to the weight fold, and needs the carrier's `*` and `/` beside the `+` every mode needs. What cannot live there is the mode's law — see below.

Definition 15.6.2. group uses 0 used by 0 ✓

The license *decided* rather than assumed. A program does not hold a proof that its weights sum to something nonzero; it holds the weights. The run-time constructor tests the total and either produces the carving — license included — or refuses. The refusal is the value a formula written without the hypothesis would have had to invent.

Lean code for Definition 15.6.2. 1 definition

`def` · defined in `PropertyKindCalculus/Aggregation.lean` complete

```
def PropertyKindCalculus.WeightedCarving.mk?.{u} {R : Type} {P : Type u}
  [PropertyKindCalculus.Carrier R] [DecidableEq R]
  (parts : PropertyKindCalculus.Decomposition P) (weight : P → R) :
  Option (PropertyKindCalculus.WeightedCarving R P)
```

Proof for Definition 15.6.2 proof">uses 1

`WeightedCarving.mk?` is a `dif` on the carrier's own equality test, and `mk?_eq_none_iff` says it refuses exactly the carvings whose total is the carrier's zero. Uses weighted carvings.

What the test **rejects** is what the carrier calls zero, and that is where the ladder starts mattering. At `Float` the license is satisfied by `NaN`: an all-`NaN` carving is accepted and its mean of a constant is `NaN`, refuting at that carrier the law proved below. Nothing is wrong with the field; what is absent is a carrier law to spend it on, which is why the executable rungs state their own gate rather than inherit this one.

Theorem 15.6.3. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

The mean of a constant is that constant. *Put every part at the same place and the whole is at that place, whatever the weights — the law that distinguishes this mode from a sum and from a shared reading at once. Uses weighted carvings.*

Lean code for Theorem 15.6.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/AggregationLaws.lean`

complete

```
theorem PropertyKindCalculus.WeightedCarving.mean_const.{u} {P : Type u}
  (c : PropertyKindCalculus.WeightedCarving ℝ P) (v : ℝ) :
  (c.mean fun x => v) = v
```

Proof for Theorem 15.6.3

uses 0

A constant factors out of the fold (`fold_mul_const`, by induction), and the license discharges the cancellation. Cancellation is a field law, which is why this statement is in the PhysLib-backed `AggregationLaws` over `\mathbb{R}` while the mode it is about is in the core: the structure travels down to every carrier and the law stops at the lawful ones.

That the weights are load-bearing is checked separately: a carving weighted 1 and 3 over values 3 and 1 has mean `3/2` where the unweighted average of the same values is `2`, so a `mean` ignoring its weights would fail the probe.

Theorem 15.6.4.

group statement">statement">used by

Effort	Tags
medium	proved

The binary32 weighted mean is within the evaluation DAG's rounding budget of the exact one.

The rung between the mode and its law: at genuine binary32 the mean is computed on a rounded grid, so the constant law holds only up to an accumulated budget, and the theorem says what that budget is. Uses weighted carvings.

Lean code for Theorem 15.6.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Adequacy/MeanBound.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.mean_fp32_within_errBound.
{u}
  {P : Type u}
  (c : PropertyKindCalculus.WeightedCarving TorchLean.Floats.FP32 P)
  (v : P → TorchLean.Floats.FP32) (ρ : ℕ → TorchLean.Floats.FP32)
  (hS :
    PropertyKindCalculus.totalWeight (fun p => (c.weight p).val)
```

```
      c.parts ≠
    PropertyKindCalculus.Carrier.zero) :
|(c.mean v).val -
      (PropertyKindCalculus.Uncertainty.Adequacy.specCarving c
           hS).mean
        fun p => (v p).val| ≤
  PropertyKindCalculus.Uncertainty.Adequacy.errBound
    (PropertyKindCalculus.Uncertainty.Adequacy.meanExpr c.weight v
      c.parts)
    ρ
```

Proof for Theorem 15.6.4

uses 0

A mean is the first aggregation mode that is not a fold — it multiplies, sums, and finally divides — so it is exactly the shape the adequacy layer's forward-error theorem (`dag_fp32_error_bound`) was built for. `MeanBound` compiles a carving into that layer's `Expr` (weights and values as exact binary32 constants, the folds as `add` nodes, the ratio as the one `div` node) and reads the bound off it. Every rounding the mean incurs is a node of the budget, and the quotient rule's magnitude factors are what a small total weight costs.

The load-bearing detail is that the **license does not cross the bridge**. `WeightedCarving`'s field says the total is not the **carrier's** zero, which at binary32 is a statement about the **rounded** fold; the theorem also needs one about the **exact** fold, and neither implies the other. Both directions are ordinary floating-point behavior and both are exhibited: weights `2^{24}`, `1`, `-2^{24}` sum exactly to `1` while their binary32 fold is exactly `+0`, because the first addition absorbs the `1`; and weights `2^{24}`, `1`, `1`, `-(2^{24}+2)` sum exactly to `0` while their binary32 fold is exactly `-2`, because two absorbed `1`s leave the running total short. So forgetting a binary32 carving to the real carving it specifies takes the specification's license as an **argument**, and the signature is where that is recorded.

Theorem 15.6.5.

group statement">used by 2 ✓

Effort	Tags
small	proved

A denominator that is never rounded has one license, not two. *Where the refinement's rounding is the identity, forgetting a carving's total gives the total of the forgotten weights, so the executable condition and the specification's are the same statement read through the forgetful map — and a run-time test of the computed total is a test of the quantity being defined. Uses weighted carvings.*

Lean code for Theorem 15.6.5.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Aggregation.lean` — complete

```
theorem PropertyKindCalculus.licenses_agree_of_exact.{u} {E S : Type}
  {P : Type u} [PropertyKindCalculus.Carrier E]
  [PropertyKindCalculus.Carrier S]
  [PropertyKindCalculus.CarrierRefinement E S]
  (hround :
    ∀ (s : S), PropertyKindCalculus.CarrierRefinement.round E s = s)
  (hzero :
    ∀ (x : E),
      PropertyKindCalculus.CarrierRefinement.toSpec x =
          PropertyKindCalculus.Carrier.zero →
        x = PropertyKindCalculus.Carrier.zero)
  (w : P → E) (d : PropertyKindCalculus.Decomposition P) :
  PropertyKindCalculus.totalWeight w d ≠
      PropertyKindCalculus.Carrier.zero ↔
    PropertyKindCalculus.totalWeight
        (fun p => PropertyKindCalculus.CarrierRefinement.toSpec (w p))
        d ≠
      PropertyKindCalculus.Carrier.zero
```

Proof for Theorem 15.6.5

uses 0

By induction on the carving: identity rounding makes the bridge law an additive homomorphism, so the fold commutes with the forgetting. Stated in the core, over an arbitrary refinement, since nothing about binary32 enters.

This is the strongest of the two remedies and the one to reach for first, because it removes the question rather than answering it. It is available exactly when the weights are counts or indicators — a validity mask, a sample tally, a pixel count — which is the ordinary shape of a masked average: fold the denominator where addition is exact and convert to the numerator's carrier once, rather than accumulating it in the same floating-point type as the numerator.

Theorem 15.6.6.

group statement">used by 2

✓

Effort	Tags
medium	proved

Nonnegative weights make the two licenses equivalent. *Where the denominator genuinely is a binary32 fold of real-valued weights, requiring them to be nonnegative and on the grid restores the equivalence: the rounded total is zero exactly when the exact total is. Uses weighted carvings.*

Lean code for Theorem 15.6.6.

1 theorem

theorem · defined in PropertyKindCalculus/Uncertainty/Adequacy/MeanBound.lean

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.licenses_agree_of_nonneg.{u}
  {P : Type u} (w : P → TorchLean.Floats.FP32)
  (hnn : ∀ (p : P), 0 ≤ (w p).val)
  (hg :
    ∀ (p : P), FloatLib.Floats.Formats.Flocq.NF.IsRepresentable (w p))
  (d : PropertyKindCalculus.Decomposition P) :
  (PropertyKindCalculus.totalWeight w d).val ≠ 0 ↔
    PropertyKindCalculus.totalWeight (fun p => (w p).val) d ≠ 0
```

Proof for Theorem 15.6.6

uses 0

Both counterexamples are built from cancellation, so nonnegativity is exactly what excludes them. The floating-point content is one lemma — a grid point does not shrink when a nonnegative is added to it and the sum is rounded, which is monotonicity of rounding together with rounding fixing the grid. Absorption may return the grid point unchanged; it cannot return anything smaller. Induction then gives both halves: a positive exact total forces a positive rounded one, and a zero exact total forces a zero rounded one.

Grid membership is a real hypothesis, not bookkeeping: the rounding-spec format is a bare record over `\mathbb{R}` with representability a separate predicate, so "this is a binary32 number" has to be said, and without it a weight below half the smallest subnormal would round away and the argument would fail at the leaf it starts from.

This covers the weights metrology actually uses — masses, areas, durations, coverage fractions, validity indicators — but it is the second-best fix. Where the weights are counts, the theorem above applies and no floating-point reasoning is needed at all.

Definition 15.6.7.

group statement">uses 1 by 0 ✓

The mode with its kinds on. A weighted carving over a bare carrier is silent about three things a kind layer can say. The weight kind has to be one that sums, so `WeightedCarvingQ` carries its carving's difference-kind proof as a field and a carving of a non-summable kind cannot be built. The weight, value and product kinds have to be related, so the mean asks for a product license and a quotient license that say so. And the quotient the mean performs is a licensed kind change, not a bare division.

Lean code for Definition 15.6.7.

1 definition

`structure` (4 fields) · defined in `PropertyKindCalculus/Aggregation.lean` — complete

```
structure PropertyKindCalculus.WeightedCarvingQ.{u} (R : Type)
  [PropertyKindCalculus.Carrier R]
  (kw : PropertyKindCalculus.KindOfProperty) (P : Type u) : Type u
```

Theorem 15.6.8.

group statement"> uses 1 by 0 ✓

Effort: small

Tags: proved

The kinded mean computes what the carrier mean computes. *Its magnitude is the carrier-level mean of the same data. So the kind index gates which folds may be written and contributes nothing to the value — which is what lets the binary32 bound, stated about magnitudes, apply unchanged to a mean a user wrote with kinds on.*

Lean code for Theorem 15.6.8.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Aggregation.lean` — complete

```
theorem PropertyKindCalculus.WeightedCarvingQ.mean_magnitude.{u} {R : Type}
  {P : Type u} [PropertyKindCalculus.Carrier R] [Mul R] [Div R]
  [PropertyKindCalculus.ScalarCarrier R]
  {kw kv kwv : PropertyKindCalculus.KindOfProperty}
  (c : PropertyKindCalculus.WeightedCarvingQ R kw P)
  (hp : PropertyKindCalculus.ProductKind kw kv kwv)
  (hq : PropertyKindCalculus.QuotientKind kwv kw kv)
  (hd : PropertyKindCalculus.DifferenceKind kwv)
  (v : P → PropertyKindCalculus.Quantity kv R) :
  (c.mean hp hq hd v).magnitude =
    c.toCarving.mean fun p => (v p).magnitude
```

Proof for Theorem 15.6.8

uses 0

Two inductions over the carving, one per fold, each with the same shape: a leaf is the erasure by definition, and a join is `Carrier.add` on both sides once the operands have been rewritten. Realized in the Mathlib-free core (`Aggregation.lean`), depending on `propext` alone.

The theorem is what makes the kinded layer honest about its cost. Nothing is recomputed and no numerical claim is restated: the three obligations above are discharged once at construction, and everything proved downstream about magnitudes continues to hold.

Theorem 15.6.9. group uses 0 used by 0 ✓

Effort	Tags
small	proved

The mean's division is an R10 site. *Forgetting the executable kinded mean to the specification carrier is the rounding of the specification quotient of the forgotten numerator and denominator — the quotient bridge read at the mean's own division node.*

Lean code for Theorem 15.6.9.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Aggregation.lean` — complete

```
theorem PropertyKindCalculus.WeightedCarvingQ.mean_div_refines.{u} {E S :
Type}
  {P : Type u} [PropertyKindCalculus.Carrier E]
  [PropertyKindCalculus.Carrier S] [Mul E] [Mul S] [Div E] [Div S]
  [PropertyKindCalculus.ScalarCarrier E]
  [PropertyKindCalculus.ScalarCarrier S]
  [PropertyKindCalculus.CarrierRefinement E S]
  [PropertyKindCalculus.DivRefinement E S]
  {kw kv kwv : PropertyKindCalculus.KindOfProperty}
  (c : PropertyKindCalculus.WeightedCarvingQ E kw P)
  (hp : PropertyKindCalculus.ProductKind kw kv kwv)
  (hq : PropertyKindCalculus.QuotientKind kwv kw kv)
  (hd : PropertyKindCalculus.DifferenceKind kwv)
  (v : P → PropertyKindCalculus.Quantity kv E) :
  (c.mean hp hq hd v).toSpec =
    PropertyKindCalculus.Quantity.roundBy
      (PropertyKindCalculus.CarrierRefinement.round E)
      (PropertyKindCalculus.Quantity.div hq
        (PropertyKindCalculus.weightedSumQ hp hd c.weight v
```

```
        c.parts).toSpec
    c.total.toSpec)
```

Proof for Theorem 15.6.9 proof">uses 1

Immediate from the quotient bridge, and that is the point: until the mean acquired kinds, the bridge had no consumer that was not a test of itself. A weighted mean is the first model-level construct in the library that divides across a licensed kind change, so it is the first place the law does work.

What the theorem does **not** say is that the whole mean commutes with the forgetful map. The folds above the division round at every join, and pushing the forgetful map through them is exactly what the exact-carrier hypothesis supplies. The division is one step of that argument, stated where it holds with no side condition. Instantiated at the binary32 rung on an indicator-weighted carving whose license is discharged rather than assumed.

15.7. Re-carving, and what a count is keyed to

A carving is not a census, and the two halves of that claim are provable. What survives a re-carving is every licensed aggregate; what does not is how many parts there are.

Definition 15.7.1. group uses 0 used by 2 ✓

A *re-carving* is a map from one carving of a whole to another, carrying the obligation that the measured whole is the same whole. It is the generalization of a pattern physics libraries prove one functional at a time — merging indiscernible parts and checking that each additive functional is unchanged.

Lean code for Definition 15.7.1. 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Recarving.lean` complete

```
structure PropertyKindCalculus.Recarving.{u} {O : Type u}
  (m : PropertyKindCalculus.Measurement O) : Type u
```

Proof for Definition 15.7.1 uses 0

Realized as `Recarving m`, a map and its `preserves` field. The witness `coalesce` replaces any carving by the single part whose mass is the total: fewer entities, same whole, with `preserves` holding by `rfl`.

Theorem 15.7.2. group uses 2 statement proof ✓

Effort	Tags
medium	capstone, proved

> ***Every licensed aggregate is re-carving invariant.*** *For an extensive kind the total over the parts is the value of the whole, so a map that preserves the whole preserves the total — whatever it does to the parts. Builds on re-carvings and extensive aggregation.*

Lean code for Theorem 15.7.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Recarving.lean` — complete

```
theorem PropertyKindCalculus.Recarving.leafSum_invariant.{u} {O : Type u}
  {k : PropertyKindCalculus.KindOfProperty}
  {m : PropertyKindCalculus.Measurement O}
  (h : PropertyKindCalculus.Extensive k m)
  (r : PropertyKindCalculus.Recarving m)
  (d : PropertyKindCalculus.Decomposition O) :
  PropertyKindCalculus.leafSum m (r.map d) =
    PropertyKindCalculus.leafSum m d
```

Proof for Theorem 15.7.2

proof">uses 1

Two rewrites by the extensive capstone reduce the claim to the `preserves` field. This is what makes the `\forall`-quantified form the deliverable rather than a convenience: a total that depended on how the whole was cut would fail exactly here.

Theorem 15.7.3.

group statement">used 1 ✓

Effort	Tags
small	proved

> ***The count does not survive.*** *The same re-carving that preserves the mass takes a two-part carving to a one-part one. So there is no function from the whole to how many parts it has, and a count is licensed by a sortal rather than by the whole — the SI's own reading of amount of substance as a count of a specified elementary entity. Uses re-carvings.*

Lean code for Theorem 15.7.3.

1 theorem

theorem · defined in `PropertyKindCalculus/Recarving.lean`

complete

```
theorem PropertyKindCalculus.coalesce_count_ne :
  PropertyKindCalculus.Decomposition.count (fun x => true)
      (PropertyKindCalculus.coalesce.map
        PropertyKindCalculus.twoParts) ≠
    PropertyKindCalculus.Decomposition.count (fun x => true)
      PropertyKindCalculus.twoParts
```

Proof for Theorem 15.7.3

uses 0

`decide` on the exhibited pair, against `thm_recarving_invariant` on the same map: 2 kg before and after, two parts before and one after. `countMeasurement` takes the sortal as an argument and `count_sortal_ne` checks that the argument matters — one carving, two predicates, counts 2 and 0 — so a count is extensive over a fixed carving and yet not a property of the whole. This is Marmodoro's "It is an open question how many entities a physical structure is" (Marmodoro, 2018)[17] Anna Marmodoro, 2018. “Whole, but not One”. In *Ontology, Modality, and Mind: Themes from the Metaphysics of E. J. Lowe (Oxford University Press).* as a refutation rather than a remark.

17.

16. The object type

A measured quantity is always the property of some object — the length of *this* pencil, the glucose concentration of *this* plasma sample — and R19 puts that object in the type. What kind of thing an object may *be* is a separate question, and this chapter is its answer: the object is drawn from an arbitrary type, and the consequences of that decision are not symmetric.

Two things are wanted of an object, and they turn out to have different prices.

Discrimination — telling two objects apart — is what the gate needs, and it is free. It is definitional equality of a type index, a mechanism every type in Lean already has. The evidence is a measurement of the instance layer itself: of the declarations in `IndividualQuantity`, **exactly one reads its object**. `add`, `mul`, `div`, `recip` and all three classification certificates carry `o` and never inspect it. What stops a length of `R1` multiplying a width of `R2` is that the two types differ, and nothing else.

Designation — naming an object in the terminological vocabulary — is what Dybkær's instance layer consumes, and it is not free. It is supplied by `Designated` (the foundations chapter), and an object type that has no identity of its own has no instance. (The Chapter 20 dedicated kind consumes the object's *sort*, not its name — that half is `Sorted`'s, in the same chapter.)

The asymmetry is the point. A host library's own objects — the particles of a mechanical system, the cells of a mesh, the pixels of a scene — can be *gated* with nothing supplied, on their own types, without that library changing. They cannot be *named*, and the layer says so rather than inventing a name that is not there.

16.1. What an object type supplies

16.2. The census of object types

The useful distinction is not between objects this calculus defines and objects a host library defines. It is how an object carries its identity, crossed with whether its type is primitive or derived from another. Six shapes arise, and each is exhibited — acceptances as `example`, refusals as `#check_failure` — in the `MiniObjectTypes` module of the `Examples` library.

Nominal. Identity **is** the name: `System`, a terminological id and nothing else. Naming and telling apart are one act, so both questions are answered at once. This is the reading the layer shipped with, and the right one for a sample, an instrument, a rover.

Structural. Identity is a **position**: a particle of a mechanical system, a cell of a mesh. Gated for free — the exhibit states this at an arbitrary type variable, about which nothing whatever is assumed — and not designatable, because there is no name anywhere to designate with.

Indexed. The position **determines** a name: a pixel, a spectral band, a layer. Between the two — gated like a structural object, designatable like a nominal one, with the injectivity discharged rather than assumed.

Composite. Derived: the parts of an assembly **and the whole they compose**, in one type. The construct aggregation needs, below.

Joint. Derived: an ordered pair, for a quantity that characterizes a **coupling** rather than a system — a flow, a transfer, a torque across an interface. The gate falls out with nothing added, so an interface quantity is an instance of the scheme rather than a construct of its own. Designation is where this shape instructs: the obvious nominal encoding, concatenating the two names with a separator, is *not* injective, because the separator can occur inside a name. The exhibit gives the two couplings that collide, and that counterexample is the argument for the injectivity field.

Singleton. One object, `Unit`. Worth naming because it bounds the cost: a model with nothing to disambiguate pays nothing for the object layer, since there is exactly one term to write.

A seventh shape suggests itself and is not one. An object *at a time* — the same apparatus examined on Tuesday — is not a new object type: the time dependence belongs to the quantity, which is a function of time, and the difference between two occasions is an *examination*. Whether a sample drawn at one moment is a different system from the sample drawn at the next is a modeling decision about the system, not something the parameterization settles.

16.3. The generality of the paradigm

It is tempting to read the census as a taxonomy the calculus provides — six kinds of object, pick one. It is the opposite. **PKC contributes no taxonomy of objects; it contributes the fact that it needs none.**

What a metrology layer must have of an object is discrimination, and discrimination is what a type **is**. So every entry in the census is either a type the host library already has (nominal, structural, indexed, singleton) or an ordinary type former applied to such types (composite, joint) — and the **gate** on all of them is definitional equality, which comes with the type and costs nothing to carry.

That is why the shapes compose without any provision being made for it. A quantity of **this instrument**, at **this pixel**, of **that particle** is a quantity of the triple `Object × Cell × Particle`, and its gate is the triple's own equality: differ in any one factor and the sum is refused. Nothing in the library knows that this combination exists.

16.3.1. What the library ships, and what it refuses to ship

Exactly two things do not come from the type former:

The one object type a host cannot have. A whole is not one of its parts, so a part type does not contain the assembly, so the total has nowhere to live. The *composite* is that type, and it is the only object **construction** this library ships.

Closure of designation. Naming a derived object is a terminological commitment, not a derivation, and the library states the obligation instead of guessing the convention. The composition principle is that a designation travels along an injection; the injectivity is the author's claim, `decide` for a finite object type and a lemma otherwise.

The refusal is the substantive half. An automatic designation for pairs would have to join two names into one, and the encoding an author reaches for first — concatenate with a separator — is not injective, because the separator can occur inside a name. Two different couplings, one name, silently. The `Examples` library carries that counterexample as a compiled fact; it is the reason `Designated` carries an injectivity field rather than a docstring.

Definition 16.3.1.1. group statement">uses 1 used by 0 ✓

Designation travels along an injection. An object type that injects into a designated one is designated: name each object by the name of its image. This is the composition principle for the whole census — an index family, a combination of factors, a sub-selection of a named set — and the only one most models need.

Lean code for Definition 16.3.1.1. 1 definition

`def` · defined in `PropertyKindCalculus/Foundations.lean` complete

```
def PropertyKindCalculus.Designated.ofInjective.{u, v} {O : Type u}
  {P : Type v} [PropertyKindCalculus.Designated P] (f : O → P)
  (hf : ∀ {x y : O}, f x = f y → x = y) :
  PropertyKindCalculus.Designated O
```

Proof for Definition 16.3.1.1 uses 0

Compose the two maps and the two injectivity proofs. A **definition**, not an instance: which naming a model uses is the model's decision, and instance search finding one by accident is the failure `designation_inj` exists to prevent. `Designated.prod` is its specialization to a pair, taking the join of two names and the proof that the join is injective.

16.4. Operations the object index changes

Two operations look ordinary until the object is in the type, and then say something they could not say before.

Definition 16.4.1. group uses 0 used by 0 ✓

The kind-licensed scalar action. A scalar quantity of an object scaling a vector quantity **of the same object**, licensed by the same product law the homogeneous product takes. `p = m·v` and `F = m·a` have this shape, and the homogeneous product cannot express them: it wants one carrier for both operands, and here one side is a number and the other a numerical vector.

Lean code for Definition 16.4.1. 1 definition

`def` · defined in `PropertyKindCalculus/IndividualQuantity.lean` — complete

```
def PropertyKindCalculus.IndividualQuantity.smulK.{u} {O : Type u} {o : O}
  {R V : Type} [SMul R V] [PropertyKindCalculus.ScalarCarrier R]
  {k₁ k₂ k : PropertyKindCalculus.KindOfProperty}
  (_h : PropertyKindCalculus.ProductKind k₁ k₂ k)
  (a : PropertyKindCalculus.IndividualQuantity o k₁ R)
  (b : PropertyKindCalculus.IndividualQuantity o k₂ V) :
  PropertyKindCalculus.IndividualQuantity o k V
```

Proof for Definition 16.4.1 uses 0

The carrier's own `SMul` on the magnitudes, with `ScalarCarrier` demanded of the **acting** side — a vector carrier is deliberately not a `ScalarCarrier`, so this cannot be misread as a componentwise product of two vectors. The object gate is as much the point as the kind law: one particle's mass scaling another's acceleration is exactly the substitution an object-blind model has no way to refuse, and it is the shape of an ordinary copy-paste error. Without this operation an application has to mint an unlicensed action and attest it at the interface tier, which is where the kind stops being checked.

Definition 16.4.2. group uses 0 used by 0 ✓

The transpose of a joint quantity. The same magnitude, read as a quantity of the reversed pair — the equal-and-opposite reading, and the **only** re-indexing of an individual quantity the library offers. Its value is that a law becomes a type: a "reverse" that negates a magnitude and forgets to exchange the endpoints has the untransposed type and does not elaborate where a reversed quantity is expected.

Lean code for Definition 16.4.2. 1 definition

`def` · defined in `PropertyKindCalculus/Composite.lean` — complete

```
def PropertyKindCalculus.IndividualQuantity.transpose.{u, v} {O₁ : Type u}
  {O₂ : Type v} {a : O₁} {b : O₂}
```

```
  {k : PropertyKindCalculus.KindOfProperty} {R : Type}
  (q : PropertyKindCalculus.IndividualQuantity (a, b) k R) :
  PropertyKindCalculus.IndividualQuantity (b, a) k R
```

Proof for Definition 16.4.2 uses 0

The magnitude, at the swapped index; the involution is `rfl`. There is deliberately no general re-indexing: moving a magnitude from one object to another is what the object index exists to prevent, and an operation that did it on request would return the layer to a naming convention. The transpose is safe because it is not a move — `(a, b)` and `(b, a)` are one coupling read from its two ends, and which end is the source is the content of a law like Newton's third.

16.5. Assembly over a composite

Gating on the object refuses a sum across two objects, which is what it is for — and it refuses, with equal correctness, the *total* mass of an assembly, because the total characterizes an object the part type does not contain. Aggregation therefore needs an eliminator, and the eliminator is where the §13.5 license enters.

Three decisions shape the eliminator, each an application of the unite/unify distinction the foundations chapter states: the whole is **added** rather than derived, because no carving yields it; the same parts under two sorts (the foundations chapter) are two object types with two sets of licenses, because the statue and the lump are two wholes; and the license is indexed by the sort rather than by the kind alone, because how parts make a whole is settled by what the whole is — the one claim the parts cannot make on their own behalf.

Definition 16.5.1. group statement">statement">used by

The *composite* object type over a part type, **as a sort of whole**: the whole, or one of its parts. The whole and the parts must inhabit one type or their quantities cannot be related at all; the whole is a distinct term from every part, so a whole-assembly quantity and a part quantity do not combine. The sort index is what lets two wholes stand over one part type — the statue and the lump over the same clay — and their quantities do not combine either, because the types differ. Extensionality about objects is not assumed.

Lean code for Definition 16.5.1.

1 definition

`inductive` (2 constructors, 2 parameters) · defined in `PropertyKindCalculus/Composite.lean` — complete

```
inductive PropertyKindCalculus.Composite.{u}
  (σ : PropertyKindCalculus.SortOfSystem) (P : Type u) : Type u
```

Proof for Definition 16.5.1

uses 0

A two-constructor inductive with `DecidableEq` and `Repr`, indexed by the sort and the part type. Naming a composite is a definition rather than an instance, taking the whole's name and the proof that no part already answers to it — a terminological commitment instance search has no business guessing.

Definition 16.5.2.

group uses 1 used by 0 ✓

The *assembly license*, keyed by the sort of the whole and the kind: which kinds' part quantities may be summed into a quantity of **that sort of** whole. Registration is opt-in, and the entry carries the same scale gate addition demands, so a nominal or ordinal kind cannot be licensed at all.

Lean code for Definition 16.5.2.

1 definition

`class` (1 method) · defined in `PropertyKindCalculus/Composite.lean` — complete

```
class PropertyKindCalculus.Assembles (σ : PropertyKindCalculus.SortOfSystem)
  (k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 16.5.2

uses 0

A `Prop`-valued class with one field, a `DifferenceKind` witness. The sort index is forced by the library's own counterexample: volume aggregates over the parts of a rigid assembly and contracts over the parts of a mixture (`mixing_subadditive`), and both facts are facts about volume — a registry keyed by the kind alone must either license the mixture or refuse the assembly, and either answer is wrong. The validation suite carries both verdicts side by side, each backed by its witness: an `Extensive` measurement for the rigid sort, the ethanol/water contraction for the mixture.

Definition 16.5.3.

group uses 2 used by 0 ✓

Assembly: the quantity of the whole, folded from the quantities of the parts over a carving. The whole and the part injection are arguments rather than fixed, so the eliminator serves a composite and equally a host library's own object type with its own naming of parts. Its part function is dependent — the quantity supplied for a part must characterize *that* part — which is the gate.

Lean code for Definition 16.5.3.

1 definition

`def` · defined in `PropertyKindCalculus/Composite.lean`

complete

```
def PropertyKindCalculus.assemble.{u, v} {O : Type u} {P : Type v}
  {k : PropertyKindCalculus.KindOfProperty} {R : Type}
  [PropertyKindCalculus.Carrier R]
  (σ : PropertyKindCalculus.SortOfSystem)
  [PropertyKindCalculus.Assembles σ k] (whole : O) (part : P → O)
  (d : PropertyKindCalculus.Decomposition P)
  (f : (p : P) → PropertyKindCalculus.IndividualQuantity (part p) k R) :
  PropertyKindCalculus.IndividualQuantity whole k R
```

Proof for Definition 16.5.3

uses 0

`Decomposition.fold` of the parts' magnitudes under `Carrier.add`: the same traversal `leafSum` uses, so the quantity-level arithmetic and the §13.5 law are one recursion read at two carriers. The counterpart over a finite index (`assembleOver`, `assembleAll`) sums a `Finset` instead, which is the shape a host library's own aggregate is written in — a `Finset.sum`, recovered by `rfl` rather than by a rewriting lemma.

Theorem 16.5.4.

group statement">proof">used by 1 ✓

Effort	Tags
small	proved

The license refused. For a whole-proper kind — one whose value the parts do not determine, in either of the two ways they could — there is a carving on which the assembled magnitude is not the value the whole has. Not merely unjustified: wrong, on an exhibited pair of parts. Uses assembly.

Lean code for Theorem 16.5.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Composite.lean`

complete

```
theorem PropertyKindCalculus.assemble_ne_measured.{u, v} {O : Type u}
  {P : Type v} {k : PropertyKindCalculus.KindOfProperty}
  (σ : PropertyKindCalculus.SortOfSystem)
  [PropertyKindCalculus.Assembles σ k] (whole : O) (part : P → O)
  {m : PropertyKindCalculus.Measurement P}
```

```
  (h : PropertyKindCalculus.WholeProper k m) :
∃ d,
  (PropertyKindCalculus.assemble σ whole part d fun p =>
        {
          magnitude :=
            (m
                (PropertyKindCalculus.Decomposition.atom
                  p)).numeral }).magnitude ≠
    (m d).numeral
```

Proof for Theorem 16.5.4 uses 0

> The `Assembles` instance is still required, and that is the finding: the class licenses the sum to be *written*, and nothing about writing it makes it true, so an author who registers a whole-proper kind gets a term that elaborates and a number that is not the physics. The witness's own non-additive pair supplies the carving; the negative twin of `assemble_eq_measured`, same eliminator and same measurement, with the extensivity hypothesis replaced by its refutation. This is why the registry is opt-in and curated per sort rather than derived.

16.5.1. Two sums, and only the object tells them apart

A sum of quantities appears in applied models in two shapes that are identical on the page.

A *resultant* combines several quantities **of one object and one kind** into one of the same object and kind: the forces acting on a particle, the currents into a node. Nothing about the object changes, so nothing beyond the scale gate addition already demands is needed.

An *assembly* combines the quantities of the **parts** into a quantity of the **whole**. The object changes, and whether that is meaningful at all is a fact about the kind — which is what the *license* records.

Summing the particles' velocities is refused because it is an assembly of an unlicensed kind. Summing the forces on one particle must not be refused, because it is not an assembly. A layer that indexes quantities only by kind writes both with the same ∑ at the same type and can refuse neither; this is the clearest single thing the object index buys, and the `PointParticle` case study scores four designs on exactly it.

Theorem 16.5.1.1. group statement"> uses 1 used by 0 ✓

Effort
medium

Tags
capstone
proved

> ***The license, cashed.*** *For a kind that is genuinely extensive under a measurement, the assembled magnitude over any decomposition* ***is*** *the value that measurement reports for the whole. This is what makes the license a claim about the world rather than a permission slip: the class licenses the sum to be written, and this theorem is what shows the sum is right. Volume on mixing has no* `Extensive` *witness, so it has no instance of this.*

Lean code for Theorem 16.5.1.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Composite.lean`

complete

```
theorem PropertyKindCalculus.assemble_eq_measured.{u, v} {O : Type u}
  {P : Type v} {k : PropertyKindCalculus.KindOfProperty}
  (σ : PropertyKindCalculus.SortOfSystem)
  [PropertyKindCalculus.Assembles σ k] (whole : O) (part : P → O)
  {m : PropertyKindCalculus.Measurement P}
  (h : PropertyKindCalculus.Extensive k m)
  (d : PropertyKindCalculus.Decomposition P) :
  (PropertyKindCalculus.assemble σ whole part d fun p =>
        {
          magnitude :=
            (m
                (PropertyKindCalculus.Decomposition.atom
                  p)).numeral }).magnitude =
    (m d).numeral
```

Proof for Theorem 16.5.1.1

uses 0

`extensive_additive` applied to the decomposition, symmetrized: the assembled magnitude is `leafSum` by definitional unfolding of the shared fold, and `leafSum` is the whole's measured numeral by the aggregation capstone.

17. Metrological modularity

The chapters so far give the calculus's vocabulary (kinds, quantities, units, dimensions, objects and their parts) and its evidence discipline (the boundary audit and the metrological provenance hypergraph); the authoring recipe and the uncertainty calculus follow. This chapter states the organizing principle they add up to: software organized as **metrology modules**. Nothing in the principle is specific to science software — it applies wherever a computed value is presented as a measurement result in the VIM sense, whatever the domain; science software is simply where this development exercises it. A metrology module is a software component — the declarations one boundary names as its members, which need not be a file or a namespace — whose

1. **interface** is a declared boundary of kind-typed ports — input quantities, output quantities, parameters, configuration, and conditional outputs with their deciders;
2. **behavior** is a measurement model in the VIM sense — VIM 4 2CD §2.12 [VIM3: 2.48], the mathematical relation among the quantities involved, in the general implicit form `h(Y, X_1, \ldots, X_n) = 0` — attached to the boundary as a checked theorem edge, not as prose;
3. **implementation** is a **measurement function** (VIM 4 2CD §2.13 [VIM3: 2.49], whose Note allows that "f" may symbolize an algorithm): a carrier-parametric kinded definition whose relation to the measurement model is `equals`, `inverts`, `refines`, or `boundedBy` a declared tolerance;
4. **licenses** are stated per rung of the carrier ladder, as the relation's license clause: the measurement-model claim is proved at one carrier, and each further carrier the module runs at — binary32, `Float`, the tape — gets its own entry saying how the claim holds there. A law proved over the reals survives the move up to one rounding per operation; a side condition phrased in the carrier's own arithmetic, such as "the total is not zero", constrains the exact total at one rung and the rounded total at the next, and neither implies the other;
5. **mereology** is declared — each output port carries its aggregation class, and the license to distribute the module's computation over a carving is **derived** from that declaration, not assumed;
6. **requirements** are stated — what the module's quantities must satisfy, each a requirement beside the boundary it governs with the evidence by which it is met: a provable requirement by a theorem, an empirical requirement by a gate against a referent the model does not define.

The name pairs the discipline (metrology — the interface vocabulary is Dybkær/VIM kinds and quantities, the spec vocabulary is the VIM measurement model) with the software principle (modularity — boundaries, contracts, composition). Why the **relational** form matters: VIM's general form is implicit, and its §2.12 Note 1 has the measurand's value **inferred** from the relation. A retrieval module's honest specification is therefore "the output satisfies the forward model's equation" — exactly, or within a declared tolerance — not "the output is what this loop computes". The paradigm demotes the loop to evidence **for** the relation, with the relation as the published behavior. `MODULARITY.md` in the repository root is the working plan for carrying the paradigm into the downstream model and deployment tiers.

17.1. The interface — the declared boundary

Definition 17.1.1. group uses 0 used by 2 ✓

The **declared boundary**: a name, the member declarations the boundary is claimed for, the kind-typed ports with the role each plays — `input`, `config`, `param`, `output`, `conditional` — the exits where values leave the calculus, and, per conditional port, the **decider**: the name of the declaration that decides its cases, so a validity domain is read off the boundary rather than excavated from a constructor choice. Roles carry **binding time**: a `param` is a source this tier leaves for the tier below to bind, and `Contract.discharges` is the decidable tier relation — every inherited parameter bound or restated, never forgotten, and no exit lost on the way up. `Contract.agrees` is the decidable scope judgment: the declared boundary is the one the members actually compute, in both directions. `#kind_contract_decide` and `#kind_discharges_decide` reflect both into kernel-checked theorems. At scale the obligation moves off the author's memory: `#kind_contracts` sweeps every contract declared under a namespace — membership is by type, so declaring a boundary enrolls it — and `#kind_contracts_decide` is the same sweep as a hard gate, adding each passing boundary's kernel receipt. A deliberate falsification probe opts out explicitly with `@[kindCounterexample]`, and the sweeps count the exemptions in their pinned headers.

Lean code for Definition 17.1.1. 1 definition

structure (8 fields) · defined in `PropertyKindCalculus/Provenance.lean` complete

```
structure PropertyKindCalculus.Provenance.Contract (ν κ : Type) : Type
```

Proof for Definition 17.1.1 uses 0

A four-list structure over node and kind types, plus the decider association list; the judgments are structural recursions decided by evaluation in a probe and by kernel reduction (`decide`) in a theorem. Nothing about it is trusted from construction — the doctrine is to decide the object.

17.2. The behavior — the measurement model as a theorem edge

Definition 17.2.1. group uses 2 statement">used by ✓

The **theorem edge** between two declared boundaries — the formal seat of the VIM 4 2CD §2.12 measurement model. It names the two contracts, what is claimed (the relation kind), the witness theorem, the claim in the author's own words, and three optional clauses: a **tolerance** (a declaration whose type is a `Quantity` at the kind of an output port of the governed boundary — a bound is a kinded quantity, not a float in prose), the **named side conditions** the claim holds under (each checked to be a

declaration the witness statement mentions), and the **license clause** (one entry per carrier rung). The edge is load-bearing because the semantic claim is often unstatable structurally: a closed-form retrieval contains none of its forward's members, a Newton retrieval contains all of them, a lookup-table retrieval contains the forward only while building its table — yet the scientific claim has one shape in all three cases, and only the theorem edge states it uniformly. `#kind_relation` checks everything around the proof — the witness a sorry-free theorem, the conclusion in the claimed shape, members of both boundaries mentioned, every optional clause answered for by name — and `#kind_relations` surveys a namespace's edges as one pinnable report — membership by type, violations rendered in the report itself, `@[kindCounterexample]`-marked probes as counted exemptions.

Lean code for Definition **17.2.1.** 1 definition

`structure` (11 fields) · defined in `PropertyKindCalculus/Provenance.lean` complete

```
structure PropertyKindCalculus.Provenance.Relation : Type
```

Proof for Definition 17.2.1 uses 0

> A record of names; nothing about it is decidable, because the witness already **is** a kernel-checked proposition. What the elaborator adds is the checking of everything around the proof, throwing on every violation, so the command is the report and the gate at once.

Definition **17.2.2.** group uses 0 statement">used by

The four ways an implementing **measurement function** (VIM 4 2CD §2.13) may relate to its measurement model, each with a checked conclusion shape: `equals` (an `Eq`), `inverts` (an `Eq` one of whose sides composes members of both boundaries — the round trip must be in the statement), `refines` (an `Eq` under at least one bound hypothesis — the smaller domain, stated), and `boundedBy` (an order relation, its tolerance nameable as a kinded quantity). The exact-versus-approximate axis of a module's honesty is carried here: a fixed-iteration solver's true relation to its model is `boundedBy`, and the vocabulary makes that statable rather than silently rounding it to `equals`.

Lean code for Definition **17.2.2.** 1 definition

`inductive` (4 constructors) · defined in `PropertyKindCalculus/Provenance.lean` complete

```
inductive PropertyKindCalculus.Provenance.RelationKind : Type
```

Proof for Definition 17.2.2 uses 0

A four-constructor enumeration; the shape checks live in the elaborator, pinned by acceptance and refusal probes for all four kinds.

17.3. The license clause — side conditions do not transfer

A relation's license clause is the part of its declaration that extends its witness beyond the carrier rung the witness is proved at: one entry per further rung of the carrier ladder, each naming how the claim holds there — a repair theorem, or an independent witness restated at that rung.

Definition 17.3.1. group uses 2 statement">used by

One rung of a relation's license clause: the carrier rung the claim is extended to, and how it holds there. Laws transfer across the carrier ladder at the cost of one rounding step per join; side conditions do not — a nonzero-total hypothesis checked over the reals constrains the exact total, over binary32 the rounded fold, and neither implies the other. So a relation claimed beyond the rung its witness is stated at owes, per additional rung, either a named repair — exact, where rounding is the identity on the values in play, or nonneg, where nonnegative on-grid weights rule the cancellation out — or `restated`: an independent witness proved at that rung. A rung claimed with no repair and no restatement is refused, because a side condition does not transfer by being assumed to.

Lean code for Definition 17.3.1. 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Provenance.lean` complete

```
structure PropertyKindCalculus.Provenance.RelationLicense : Type
```

Proof for Definition 17.3.1 uses 0

A rung name and a three-constructor transfer status, each constructor carrying the name of its evidence; the elaborator checks the named evidence is a sorry-free theorem, and the refusal of an unanswered rung is a pinned probe.

17.4. The mereology clause, and the footprint audit

Definition 17.4.1. group uses 0

The **aggregation class** of a produced port — the declared mereology of a boundary. Dybkær's §13.5 vocabulary, extended by the classes the mereology layer proves laws over: `extensive`, `quasiExtensive` (naming its per-join tolerance, a declaration whose type is a `Quantity` at the governed port's kind — without the name, "approximately" is not a claim), `conditionallyExtensive` (naming its condition), `intensive`, `wholeProper`, `countKeyed` (naming the sortal the count is **of** — a count is extensive over a fixed carving yet not a property of the whole, so it travels with its carving rather than surviving a re-

carving), `extensiveAbout` (naming the transport law that prices a parameter change), and `interfaceLicensed` (naming the cancellation law that purchases additivity over interior interfaces). A contract carries one entry per classed port; the checks are hygiene — a class governs a **produced** port, and each name the class carries exists, a tolerance as a kinded quantity. The truth of a class is the author's curated claim, exactly as an `Assembles` entry is.

Lean code for Definition 17.4.1.

1 definition

inductive (8 constructors) · defined in `PropertyKindCalculus/Provenance.lean` — complete

```
inductive PropertyKindCalculus.Provenance.AggregationClass : Type
```

Proof for Definition 17.4.1

uses 0

An eight-constructor enumeration whose payloads are the names the classes must answer for, plus the `aggregations` association list on the contract; the elaborator checks are pinned by acceptance and refusal probes, and the class renders in every `#kind_contract` report.

Theorem 17.4.2.

group statement">state

Effort: small

Tags: capstone, proved

The distribution license. *A module whose batched outputs are all extensive commutes with a re-carving of its batch axis: one map, every output's total preserved, however the axis is cut into tiles, blocks, or shards. The quantified boundary-level form of [re-carving invariance], stated over the list of output measurements because a boundary is a list. What it deliberately does not cover: a count-keyed output — the re-carving that merges parts preserves every extensive total while changing the count, which is why a count travels per pixel or per shard; and the quasi-extensive case, where each total stands within its carving's `joins · t` of the preserved whole, so a re-carving costs at most both carvings' joins times the per-join tolerance.*

Lean code for Theorem 17.4.2.

1 theorem

theorem · defined in `PropertyKindCalculus/Recarving.lean` — complete

```
theorem PropertyKindCalculus.Recarving.distribution_license.{u} {O : Type u}
  (outs :
```

```
  List
    (PropertyKindCalculus.KindOfProperty ×
      PropertyKindCalculus.Measurement 0))
(map :
  PropertyKindCalculus.Decomposition 0 →
    PropertyKindCalculus.Decomposition 0)
(hext : ∀ o ∈ outs, PropertyKindCalculus.Extensive o.1 o.2)
(hpres :
  ∀ o ∈ outs,
    ∀ (d : PropertyKindCalculus.Decomposition 0),
      (o.2 (map d)).numeral = (o.2 d).numeral)
(d : PropertyKindCalculus.Decomposition 0)
(o :
  PropertyKindCalculus.KindOfProperty ×
    PropertyKindCalculus.Measurement 0) :
o ∈ outs →
  PropertyKindCalculus.leafSum o.2 (map d) =
    PropertyKindCalculus.leafSum o.2 d
```

Proof for Theorem 17.4.2 **proof">uses 1**

> Each output alone is [the re-carving invariance capstone]; the license applies it under the quantifier. The quasi-extensive price is proved in the uncertainty layer beside the `joins · t` accumulation law.

What the vocabulary **does** already support module-by-module is the audit of how much checking the kind algebra gives a boundary for free. `#kind_scc` reads the kind vocabulary's inter-derivability clusters; `#kind_footprint` reads one declared boundary against them and reports the component partition of its port and interior kinds (a single-cluster collapse is flagged), the same-kind port groups (swappable however separated the components), and the review-strength sites (crossings, attests, exits) — closing with a pinnable verdict line answering one question per module: where does this module's guarantee actually come from?

17.5. The requirement clause — what the model must satisfy

The five clauses above say what a module computes, how that relates to its measurement model, at which carriers, and how it distributes. None says what the module's quantities **must** satisfy — the input/output ratio that may not exceed a bound, the output that must lie in a band, the idealization whose error must stay below a tolerance — nor by what evidence each demand is met. A requirement is that statement, as data beside the declared boundary it governs, in the shape the theorem edge set: declaration names the environment resolves, the author's words where the calculus cannot read them, and nothing the commands do not check.

Two categories, told apart by the statement and not by the effort. A provable requirement closes over the model's own declarations and kinded quantities — every constant its statement mentions is the model's or the library's — so a theorem about the model discharges it: a theorem edge from the governed boundary to a specification boundary, a declared boundary whose one member computes the bound and whose role says so, or a bare theorem; a negative one — "this is never summed over the pair" — by the absence of a license the elaborator asks the environment for and must not find. Where the model also runs, the requirement names its spot checks: `Bool` declarations the requirement command evaluates and requires `true`, the way an inhabitation check spot-checks a proof — evidence that the proof reaches the carrier the model runs on and the operational inputs, not a discharge. An empirical requirement names a referent the model does not define — a datasheet value, a prior, a reference measurement, a population — so no theorem about the model alone can discharge it, however much proof is spent: the referent is a free variable only observation closes, which makes this the falsifiable category and testing its correct instrument rather than a fallback. It is discharged by a **gate**, a decider whose threshold is the referent, or deferred. The referent is a kinded quantity carrying `@[kindReferent "where it comes from"]`, and the mark is what makes the demarcation mechanical: a provable requirement's statement may name no referent, and an empirical requirement's gate must name one. Either category may be attested, in the sense of attestation: deferred with a reason in the author's words. That is not a third category but a ratchet, counted by the census and meant to fall.

A requirement has a requirement scope: the objects it quantifies over — every object the boundary is evaluated for, every object of a named sort, reached through the `Sorted` edge of the ontological square, named objects, or the objects a named decider selects. "Every spring" is a sort, "the coupling spring" is an object, and neither is statable over a boundary whose objects are nominal.

Definition 17.5.1. group uses 0 used by 0 ✓

A requirement on a declared boundary: its name as a document cites it, the statement in the author's words, its category, the model boundary it governs and the governed port (or the boundary as a whole), its scope, and the evidence — for a provable requirement the witness (a theorem edge whose left side is the governed boundary and whose right side is a specification boundary, or a sorry-free theorem), the spot checks, and the absence claimed; for an empirical requirement the referents and the gate; for either, the attestation that defers it. `#kind_requirement` checks everything around the statement and throws on every failure — the governed boundary is a model's, the port is one it declares, the scope names a sort, sorted objects, or a decider, a provable statement names no referent, an empirical gate names one, every spot check evaluates to `true`, a claimed absence is absent, and an attestation never stands beside a discharge. `#kind_requirements` surveys a namespace's requirements as one pinnable report; `#kind_requirement_coverage` is the census — every produced port of every model boundary in scope governed by a boundary/port-matching requirement whose object scope covers the inferred result domain, or marked `@[kindRequirementFree "reason"]`, with category tallies and the attested count — and `#kind_requirement_clean` its gate. The result member's type infers that domain: an index binder means all objects, constants of that type mean only those objects; `.noIndex` means the result carries no object index, while `.unresolved` means its object cannot be determined statically. Both produce unchecked violations unless the boundary is exempted. A `.sort` scope requires an object

index and a `Sorted` instance; the census first reduces the actual result bearer's sort inside the member telescope. A definite match governs, a definite mismatch falls back to object scopes or is ungoverned, and a stuck sort follows the object-domain path and may be unchecked. A `.decided` scope is governed only when its `deciderProof` theorem proves the decider holds for every result of the governed member; otherwise the census reports it unchecked.

Lean code for Definition **17.5.1.** 1 definition

`structure` (13 fields) · defined in `PropertyKindCalculus/Requirement.lean` complete

```
structure PropertyKindCalculus.Provenance.Requirement : Type
```

Proof for Definition 17.5.1 uses 0

A record of names and strings, nothing about it decidable; what the elaborator adds is the checking of everything around the statement, the evaluation of the spot checks, and the instance search behind a claimed absence, each refusal pinned by a probe.

Definition 17.5.2. group uses 0 used by 0 ✓

The two categories, `provable` and `empirical`, as the demarcation reads them: the constants a provable requirement's witness statement mentions may include no `@[kindReferent]`-marked declaration, and an empirical requirement's gate must mention one. A requirement declared in the wrong category is refused on its constants.

Lean code for Definition **17.5.2.** 1 definition

`inductive` (2 constructors) · defined in `PropertyKindCalculus/Requirement.lean` complete

```
inductive PropertyKindCalculus.Provenance.RequirementKind : Type
```

Proof for Definition 17.5.2 uses 0

A two-constructor enumeration; the demarcation is the elaborator's constant walk over the witness statement and the gate's body, pinned by acceptance and refusal probes.

Definition 17.5.3. group uses 0 used by 0 ✓

The four scope forms: `all`, `sort` (a `SortOfSystem` declaration, which the objects reach through their `Sorted` instance), `objects` (declarations whose type carries a `Sorted` instance, so each is an object of

some sort and not a bare value), and `decided` (a named decider selecting a batch's subset). The square's left edge is what makes the first three statable.

Lean code for Definition **17.5.3.** 1 definition

`inductive` (4 constructors) · defined in `PropertyKindCalculus/Requirement.lean` complete

```
inductive PropertyKindCalculus.Provenance.RequirementScope : Type
```

Proof for Definition 17.5.3 uses 0

A four-constructor enumeration; the elaborator checks a named sort is one, synthesizes the `Sorted` instance for each named object's type, and refuses an empty object list as a vacuous scope.

Definition 17.5.4. group uses 0 statement">used by

Which side of a requirement a declared boundary stands on: `model` (the default — what a requirement governs and what the coverage census asks for) or `specification` (what a requirement is checked against, and itself the subject of no requirement). The same structure under the same checks, so a specification is kernel-accepted exactly as a model is; only the role tells them apart, and `#kind_requirement` refuses a requirement that governs a specification or is checked against a model.

Lean code for Definition **17.5.4.** 1 definition

`inductive` (2 constructors) · defined in `PropertyKindCalculus/Provenance.lean` complete

```
inductive PropertyKindCalculus.Provenance.BoundaryRole : Type
```

Proof for Definition 17.5.4 uses 0

A two-constructor enumeration carried as a defaulted field of the contract, rendered in the contract's own report when it is not the default.

17.6. What the requirement clause adds — every claim a model makes, with its evidence

The five clauses before it answer the reviewer's question about a module's **computation**: the declared boundary says what it reads and produces, the theorem edge says how that relates to its measurement model and the license clause at which carriers, and the mereology clause says how it distributes.

Together they are evidence that the module computes what it is specified to compute. They leave open the question a model is asked when it is put to use — do the quantities it produces satisfy what that use demands of them? — and that question has a different shape: its answer names a bound, a band, a tolerance, and sometimes a number the model does not define. The sixth clause answers it the way the other five answer theirs, as data beside the boundary, checked by the elaborator, surveyed and censused by commands whose reports are pinned, and gated. What follows is what that buys, point by point, against the mechanisms the definitions above name.

The demarcation is read off the statement, not granted to the author. A requirement is provable or empirical according to the constants its evidence mentions, and the `@[kindReferent]` mark is what the check reads: a provable requirement's witness statement may name no referent, and an empirical requirement's gate must name one. The consequence is a symmetry no prose ledger enforces. A claim about the world cannot be presented as a theorem, since a witness whose statement names a datasheet value is refused as provable; and a claim about the model cannot shelter behind a test, since an empirical requirement that names a witness is refused as empirical. The two categories are kinds of evidence, not degrees of confidence: a reader who meets `provable` in the survey knows the discharge is a theorem the command checked for `sorry`, and `empirical` names the referent only observation closes. Metrology draws a neighboring distinction, between verification against a specification and validation for an intended use; the clause cuts along a different line — how a claim is discharged, not what it is about — because that is the line the elaborator can decide. That an output never exceeds a physical bound is an intended-use demand and provable; a rounding budget over the operational input distribution is a design demand and empirical, its referent the distribution.

Non-vacuity, carried from the capstones down to each requirement. The capstone chapter owes every theorem three witnesses — an instance the kernel decides, a mutant that fails, a pinned axiom profile — so that a proved statement is shown to carry weight. A provable requirement owes the same, and the command collects it. Its witness is a theorem edge, checked as the license clause checks one, or a bare theorem whose axioms the report prints; on either route a witness that depends on `sorryAx` is refused. Its spot checks are the instance: `Bool` declarations evaluated where the model runs, on the operational inputs. And the construct's own test pins a refusal for each clause of the check beside the acceptances, which is the mutant discipline applied to the checker itself. The spot check is where the carrier ladder bites. The theorem is proved at $\mathbb{R}$; the license clause above records that a side condition does not transfer across carriers on its own; so the `Bool` the command evaluates at `Float` is the evidence that the proof reaches the carrier the model runs on and the inputs it meets there — evidence of reach, not a second discharge. The asymmetry with the empirical category is exactly that there the test **is** the discharge, and its oracle is the referent rather than the requirement's own statement.

A negative claim becomes a discharge. What a module must never do — sum an angular frequency over the pair, distribute a whole-proper quantity over its parts — is as much a requirement as what it must do. The kind algebra can refute such a sum as a theorem, and the oscillator case study does; but the fact a reviewer needs is about the registry: that the environment the model is built in holds no license for the sum. A provable requirement may name that license as the one that must be absent, and the command discharges it by asking the environment for an instance and finding none. The refusals the

calculus is built around are thereby citable as the evidence for a stated claim, rather than visible only as a probe that fails to elaborate.

The referent is the constant mint of the validation side. An empirical requirement's gate decides against a threshold, and a threshold that is a bare carrier value with a docstring is the docstring problem in a new place: invisible to every census, its provenance a comment. The referent is a kinded quantity carrying `@[kindReferent "where it comes from"]`, so a datasheet band, a prior, a reference measurement enters the model through a declared tier with its provenance on the record, as a constant mint enters the computation; the mark is what the demarcation reads and what the survey prints beside the requirement. A gate whose threshold is the model's own is refused: a decision against nothing the world supplied is not an observation.

Scope, through the ontological square. "Every spring", "body A", "the objects a decider selects" are quantifiers over particulars, and a requirement scope states them through the `Sorted` edge of the object type: a sort, named objects, or a decider. Over a boundary whose objects are nominal none of the three is statable. This is where the object layer pays for itself a second time: besides keeping one object's quantity from standing in for another's, it makes a demand about particular objects statable at all — a bound that holds of the coupling spring and not of the wall springs, a band every spring of one sort must lie in — and it is why a requirement is declared beside a boundary whose objects have sorts, not beside a bare function.

What the capstones do not claim becomes what the model states and counts. The capstone chapter closes with what no theorem there asserts — the truth of an attestation, accuracy against nature — and leaves them to the uncertainty chapter and the reader. The requirement clause is where those become data: each an empirical requirement with its referent and gate, or deferred by an attestation whose reason the survey prints. The coverage census then asks of every produced port of every model boundary whether a boundary/port-matching requirement governs it. For object scopes it compares the requested objects with the domain inferred from the result member's type; a missing index or unresolved bearer yields an unchecked violation unless the boundary is exempted. A `.sort` scope requires an object index and a `Sorted` instance; the census first reduces the actual result bearer's sort inside the member telescope. A definite match governs, a definite mismatch falls back to object scopes or is ungoverned, and a stuck sort follows the object-domain path and may be unchecked. A `.decided` scope needs a `deciderProof` theorem proving the decider holds for every result of the governed member; without it, the census reports the port unchecked. A `@[kindRequirementFree "reason"]` mark says why no requirement is demanded, and the census counts the attested ones. That count is a ratchet: it moves only by a conscious edit, and it is meant to fall as the apparatus supplies its numbers. A model's remaining dependence on the world is then a number in a pinned report, with a reason per entry, rather than a sentence in a limitations section.

What a reader gets. For each output of each module, the survey and the census answer in one pinned report: what must hold of it, in the author's words; over which objects; whether the evidence is a theorem or an observation; where a theorem's proof reaches, by its spot checks; which referent an observation is decided against and where that referent comes from; and what is still attested, with the reason. That is

the per-output form of the question the mereology clause closes with — where does this module's guarantee actually come from? — answered from data the elaborator checked rather than from a reading of the source.

The tutorial's harmonic oscillator carries the clause in both categories over the ForPhysLib case study, and its reports are pinned in `Tutorial.HarmonicOscillatorRequirements`:

Requirement	Category	Governs, over	Evidence
HP1 — the static displacement of body A per unit force never exceeds its wall spring's compliance	provable	the static-compliance boundary, over body A	a `boundedBy` edge to the specification boundary computing `1/k_A`, proved at `ℝ` on a named domain; a spot check at `Float`
HP2 — the velocity response per unit driving force never exceeds `1/c`	provable	the squared-impedance boundary, over every driven oscillator	a `boundedBy` edge to the specification boundary computing `c²`, unconditional; a spot check at the complex carrier
HP5 — the pair's mass is the sum of its bodies' masses	provable	the pair's mass boundary, over every coupled pair	a theorem, the licensed assembly; a spot check
HP6 — angular frequency is never summed over the pair	provable, negative	the pair's mass boundary, over every coupled pair	the absence of the license that would assemble it
HE1 — the linear model is used only within the spring's linear range	empirical	the static-compliance boundary, over every spring	attested: the range is the datasheet's to supply
HE2 — each spring's mass is at most one percent of the lighter body's	empirical	the pair's mass boundary, over every spring	attested: discharged when the springs are weighed
HE5 — every spring's stiffness lies within its datasheet band	empirical	the static-compliance boundary, over every spring	attested: the band is the datasheet's to supply

The census over the module reads `3 governed, 2 exempted — clean` and `4 provable, 3 empirical, 3 attested`: the two specification boundaries are listed as the subject of no requirement, and the three

attestations are the ratchet's opening count. The node below is the clause's witness in this document, in the form the capstones set.

Theorem 17.6.1. group statement"> uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***The clause's witness.*** *HP1 in full: a model boundary whose one member computes body A's static compliance from the three stiffnesses; a specification boundary whose one member computes the wall spring's own compliance; both kernel-accepted by `#kind_contract_decide`; a `boundedBy` edge between them whose witness is the linked theorem, proved over `ℝ` under the named domain — positive wall stiffness, non-negative others, a non-degenerate chain; and a spot check at `Float` on the tutorial's stiffnesses, evaluated by `#kind_requirement` and pinned with the rest of the report. Beside it, HP6 discharges a negative claim by absence, and HE1, HE2 and HE5 are the attested empirical rows the census counts.*

Lean code for Theorem 17.6.1.

1 theorem

`theorem` · defined in `Tutorial/HarmonicOscillatorRequirements.lean` — complete

```
theorem Tutorial.HarmonicOscillator.compliance_bounded
  (kA kC kB :
    PropertyKindCalculus.Quantity
      Tutorial.HarmonicOscillator.stiffnessDK ℝ)
  (h : Tutorial.HarmonicOscillator.complianceDomain kA kC kB) :
  (Tutorial.HarmonicOscillator.complianceA kA kC kB).magnitude ≤
    (Tutorial.HarmonicOscillator.complianceBound kA).magnitude
```

Proof for Theorem 17.6.1

uses 0

The bound `(k_B + k_C)/(k_A k_B + k_A k_C + k_B k_C) ≤ 1/k_A` by cross-multiplication under the domain, closed by `nlinarith` from the non-negativity of `k_B k_C`. The requirement command's report — edge, witness, spot check `true` — is pinned by `#guard_msgs`, as are the survey, the census and the gate over the module.

18. Write Once, Correctly

The chapters so far catalogue what PropertyKindCalculus *provides*. This one is about *method*: the order in which an application author reaches for those pieces when specifying a real model, and why the order is forced rather than a matter of taste. The running example is `PropertyKindCalculus.Examples.MiniWriteOnce` — a small, self-contained soil-moisture retrieval in the project's own `Examples` library — read here only to *infer the recipe*. It is a deliberately simplified subset, keeping just the salient structure each step needs (a few kinds, a typed constant table, a dedicated kind, one row of a Cholesky factor) and omitting the full retrieval. Every line of it is a *checked fact*, built in the `Examples` library under CI, so the illustrations below are machine-verified; PropertyKindCalculus supplies the five primitives named below; the example composes the first four in this order, and the fifth, the boundary, is shown on the tutorial's oscillator exhibit.

The discipline has a slogan: *write once correctly, go fast automatically*. A quantity is authored once, carrying its kind, over an abstract numeric carrier; the kind layer is checked by the elaborator and then *erases* — projected away by `.magnitude` — to a bare floating-point kernel, bit for bit. Correctness lives in the types and is paid for at compile time, while the number that runs is the same one a hand-written kernel would compute. Getting the *authoring order* right is what makes that erasure sound: each layer must be in place before the one above it can type-check.

18. This carrier-parametric discipline is inherited from TorchLean (George *et al.*, 2026)[18] Robert Joseph George, Jennifer Cruden, Will Adkisson, Xiangru Zhong, Huan Zhang, and Anima Anandkumar (2026). "TorchLean: Formalizing Neural Networks in Lean" (https://arxiv.org/abs/2602.22631). *arXiv preprint arXiv:2602.22631 (cs.MS)*. . , where a tensor is generic over its scalar carrier so that one network runs at $\mathbb{R}$ for proof and at `Float` for execution — and the executable carrier here is, in fact, a TorchLean tensor. PropertyKindCalculus lifts the discipline up the metrology tower: the same numeric carrier is varied over a kind-typed scalar quantity, a tensor-valued quantity, and — across the lawful-to-executable refinement — a numerically adequate one; the same parametric-indexing idea then carries onto an additive uncertainty descriptor, yielding an *uncertain* quantity, and onto the measured object as a further type index, yielding an object-indexed *individual* quantity. What TorchLean provides for one carrier of one tensor, PropertyKindCalculus provides for the whole tower of quantity notions.

18.1. The five steps

18.1.1. Step 1 — Start from the measurement principle: declare the kinds

Definition 18.1.1.1. group uses 3 used by 0

Before any number, name *what each quantity is* by the examination that produces it. A kind-of-property is the common defining aspect of mutually comparable properties (Dybkær §6.19); two otherwise-comparable kinds are individuated by their examination principle — the defining aspect of the procedure that measures them, so that a different principle forces a different kind. This is the layer a

description logic partly has: it can record a class, but not the operator algebra or the laws the later steps add.

In the example this is the catalogue of bare kinds — the radar `backscatter \sigma^0`, the optical `ndvi`, the surface `reflectivity`, and the Mironov dielectric fit-constant kinds — each declared as a `def` of `KindOfProperty`, named by the instrument or principle that examines it. That a different examination principle forces a different kind is itself checked there: monostatic and bistatic `reflectivity` share name, scale, and dimension yet are distinct kinds. A kind is *general* (a type); *this pixel's* backscatter is *individual* (a term, `Quantity backscatter α`). Fix the kinds first: everything above is indexed by them.

18.1.2. Step 2 — Pair each kind with its dimension

Definition 18.1.2.1. group uses 2 used by 0 X

Give each kind its dimension, and let dimension do only what it can. The dimension map is a forgetful functor — it keeps a kind's SI base-quantity exponents and discards the rest — so it answers only the coarse question, *are these even commensurable?* It respects products, which is ordinary dimensional bookkeeping proved as a law, but it *does not decide* identity. This step matters precisely because the quantities that carry the retrieval are almost all dimension one — a volume fraction, a reflectivity, an index. Dimension equates them all; the kind layer keeps them apart. Pairing a kind with its dimension here is what lets a later product be checked for dimensional consistency *and* kind consistency at once.

In the example, volumetric and gravimetric water content (`vwc`, `gwc`) are both dimension one yet distinct kinds — a units library alone would silently accept one for the other, which is exactly the failure this layer removes. (The literal `dim`-functor collapse is the Mathlib-backed dimension layer, shown in the *Units* and *Dimension* chapters; here the point is made one level up, at the kind.)

18.1.3. Step 3 — Build kind-differentiated quantities; gate the arithmetic

Definition 18.1.3.1. group uses 2 used by 0 X

Now write the model's data and forward maps as records of typed quantities, and let the interaction algebra gate the arithmetic: a product is formed only where the kinds sanction it (`KMul` for products, `KDiv` for quotients), so a wrong pairing is a compile error, not a wrong number. Same-kind addition is the gate on `+`; products and quotients change kind by the algebra. Authored over an abstract carrier `α`, the whole record re-represents at the proof carrier (`ℝ`), the rounding spec (binary32), and the executable (`Float`) by one map, preserving every kind.

The example's `MiniMironovCoeffs α` is the pattern: each Mironov-2009 fit constant is a field of its own kind — a dry refractive index `ndA0 : Quantity refractiveIndexKind α`, a relaxation time `taub0 : Quantity relaxTimeKind α`, a conductivity `sigbF0 : Quantity conductivityKind α` — so the elaborator refuses to pass a relaxation time where a conductivity is expected, even though all three erase to the same carrier. Its `MiniMironovCoeffs.map` re-represents the whole table at any carrier (the constants moved from `Int` to executable `Float` by one map), each constant keeping its kind.

18.1.4. Step 4 — Differentiate shared kinds: dedicate, or index by field

Definition 18.1.4.1. group uses 3 used by 0 X

Finally, decide what to do about quantities that share a kind. There are two cases, and they go on *different axes*.

When two quantities *should* be different kinds because their measurement target differs — same quantity, different system or component — make them distinct with a dedicated kind: holding the component (or system) apart is enough to force distinct kinds, without inventing identity strings. In the example the bound-water and free-water static permittivities (`boundStaticPerm`, `freeStaticPerm`) are one relative-permittivity kind dedicated to two components, hence provably distinct.

When the algebra *forces* a genuinely shared kind, the leftover distinction is *positional*, not metrological, and belongs on a second axis carried by *named record fields* — not arrays. One row of the Cholesky factor — the example's `CholRowQ α` — is row-homogeneous: three fields `l20 l21 l22`, all `Quantity reflectivity α`, because `A = L L^{\mathsf T}` makes every `L_{ik} L_{jk}` land at the same kind regardless of the column — a kind that separated them would make the factorization's own row sums ill-typed. The remaining "which slot" distinction is held by the distinct field names and checked off the kind axis by an erasure `rfl`: the theorem `cholRowQ_magnitudes` proves the kinded row's `.magnitudes` are *definitionally* the bare-`Float` kernel's outputs, so a same-kind slot swap still type-checks but breaks parity. A `Fin 3`-indexed array would push that positional bookkeeping onto the type and discard the named slots; the record keeps both axes separable — kind by the elaborator, position by parity-to-reference.

18.1.5. Step 5 — Declare the boundary: what enters, what leaves, and what it must satisfy

Definition 18.1.5.1. group uses 3 used by 0 X

Last, say where the model's numbers come from, where they go, and what the model claims about them. The first four steps fix what each quantity *is*; this one fixes what the *module* is. Its interface is a declared boundary of kind-typed ports — the inputs it ingests, the outputs it emits, its parameters, and the conditional outputs with their deciders — and its claim is a relation to its measurement model: that the output equals, inverts, refines, or is bounded by the measurement the module stands for, carried as a named witness theorem with its hypotheses and a license per carrier. The boundary is also what makes Step 4's erasure *deployable*: the distribution license says that only an extensive output commutes with a re-carving of its input, so sharding and batching — the deployment spectrum below — rest on the boundary's license clause. With the boundary in place the module states each requirement it makes against it: what its quantities must satisfy, and by what evidence, a theorem or an observation (the requirement clause).

`MiniWriteOnce` stops at Step 4; the step is shown by the tutorial's oscillator exhibit `Tutorial.HarmonicOscillatorRequirements`. There `complianceBoundary` takes three stiffnesses in and puts one compliance out, `complianceSpec` is the specification boundary that bounds it — one member,

the wall spring's own compliance — and the requirement `hp1` names the `boundedBy` edge between the two, with its theorem as witness and a `Bool` spot check evaluated at `Float`. `#kind_contract_decide` checks each boundary in the kernel and leaves the theorem behind; `#kind_boundary_audit` lists every site where a number enters or leaves with its evidence tier, and a site with none is a printed violation, not a review comment.

18.2. The payoff

Authored in this order, the model is *one source*. The kinds make every operation in the forward physics and the normal-equations solve a checked equation; the dimension layer certifies commensurability without conflating dimension-one quantities; the interaction algebra gates the arithmetic; dedication plus named fields carry the distinctions the kind axis cannot; and the boundary says what enters, what leaves, and what the module claims about it. Then `.magnitude` erases the whole overlay and the compiled kernel is the bare `Float` program — proved, by a definitional `rfl` for every input, to be the same numbers. *Write once correctly, go fast automatically.*

The worked example is `PropertyKindCalculus.Examples.MiniWriteOnce`, build-gated by the project's own CI, so every illustration above is a checked fact rather than a claim. The recipe scales to a full retrieval: the same five steps, composed in the same order, carry a complete SMAP–NISAR soil-moisture model — and because each step's facts are real declarations, a downstream *verifiable ATBD* can cite them by name with their proved-or-`sorry` status, keeping the document and the code in lockstep. PropertyKindCalculus supplies the five primitives this chapter names; a model composes them in this order.

18.3. Go fast, automatically — the deployment spectrum

The five steps above are the *write once, correctly* half of the slogan; this section is the *go fast, automatically* half. Once a model is authored once over an abstract carrier `[NumCarrier α]`, the kind overlay erases to a bare arithmetic kernel that is the *same* for every carrier — and the *carrier the model is instantiated at* then selects, on its own, an execution strategy. There is a spectrum of them, from a portable single core to a compiled GPU megakernel, and moving along it is *choosing a carrier, not rewriting the model*. That is the precise content of "automatically": the performance engineering is a consequence of the instantiation, paid for once in the carrier libraries rather than per model.

The spectrum has an honest dividing line. The first two strategies ask *nothing* of the model's structure — they run an arbitrary science model, including one whose control flow braids input/output and computation in ways that defeat vectorization. The last two exploit a structural property the `NumCarrier` capability *enforces*: because `NumCarrier` carries no ordering-to-`Bool`, a kernel over it cannot take a data-dependent branch on its inputs — every conditional is re-expressed with the branchless selectors `min`/`max`. "Typechecks against `NumCarrier`" therefore *means* "one fused, control-flow-free elementwise kernel", which is exactly the precondition that makes automatic parallelization and fusion sound. Good throughput on the last two strategies additionally needs the model to have favourable *arithmetic intensity*

— enough arithmetic per byte of input/output moved — which is a property of the science, not of the calculus.

18.3.1. CPU, single-threaded — the universal baseline and the oracle

Instantiate the model at the scalar executable carrier (`α := Float`). The kind overlay erases by `.magnitude` to the bare floating-point program of Step 4's payoff; every intermediate lives in a register, so the arithmetic intensity per work-item is maximal and the kernel is compute-bound on one core. This strategy is unconditional: it runs *any* science model, including those whose interleaving of I/O and computation, data-dependent iteration, or irregular memory access makes efficient parallelism genuinely hard — the cases where the later strategies do not apply. It is also the *oracle*: because it is the same erased kernel, its per-pixel result is the reference the parallel and GPU strategies are checked against, bit for bit. The cost is that it uses a single core.

18.3.2. CPU, multi-threaded — coarse-grained task parallelism

Keep the scalar carrier and split the *row-independent* (pixel) axis into contiguous slices, running the whole kernel on each slice on its own dedicated Lean `Task`. The parallelism is Lean's own task construct — no external threading library, no OpenMP shim — and it is *coarse-grained*: one task per slice runs the entire op-chain, so the compute-per-fork is maximal and per-operation fork/join overhead is nil. This is exact whenever the work-items are independent (which the per-pixel retrieval is), and it scales toward the core count *when the model has good arithmetic intensity* — enough compute per work-item that the cores are kept busy rather than starved on memory bandwidth or throttled by task overhead. A model with poor intensity, or one whose items are not independent, sees little benefit here and is better served by the single-threaded baseline. This strategy is the deployment layer's `runSharded`.

18.3.3. GPU, eager — one pre-compiled kernel per operation

Instantiate the model at TorchLean's batched device carrier (`α := CudaT`). Each `NumCarrier` operation on this carrier is a single pre-compiled CUDA kernel, launched by a foreign-function call, that reads its operand buffers from device memory and writes one output buffer back. Running the write-once source at this carrier therefore issues a *sequence* of device kernel launches, one per arithmetic operation of the model, and the GPU parallelizes *within* each launch — one thread per pixel per operation.

It is worth being exact about what is *not* happening here, because it is a common confusion: this is *not* an intermediate representation being *interpreted on the GPU*, and no IR is involved at all. The Lean-level operation sequence directly drives a sequence of hand-written device kernels through the foreign-function boundary — eager, operation-at-a-time dispatch, the device analogue of running the scalar kernel one statement at a time. There is no on-device program, no fused kernel, and nothing that walks a graph at run time.

This strategy is *automatic* in the strongest sense — the very same write-once source runs on the GPU with no GPU code written — and it wins substantially when the pixel batch is large, because each launch

amortizes over the whole batch. Its ceiling is set by *memory bandwidth*: every intermediate of the model is materialized as a full device buffer and round-tripped through DRAM, so the workload is *memory-bound* and its arithmetic intensity — arithmetic performed per byte moved — sits far below the device's roofline (for the SMAP–NISAR AVS fit, an intensity of roughly 0.17 against an ideal near 13.75, an ~80× gap). The device spends its time moving buffers, not computing. This is the current production GPU path.

18.3.4. Fused forms — a targeted middle ground

Between dispatching every operation eagerly and compiling the whole model there is a targeted lever: a *fused form*. On the eager carrier a single composed sub-expression of the model — say a two-way attenuation `exp(c · x · y)` — is several kernel launches and as many DRAM round-trips, one per operation. A fused form replaces that one recurring shape with a single kernel the carrier provides directly, computing the composition in one pass with its intermediates in registers. Crucially the fused form is a *refinement* of the composition, not an approximation of it: it is required to be bit-identical to the `NumCarrier` expression it stands in for — same operations, same association — so substituting it changes only *how fast* the shape is computed, never *what* it computes. Where a carrier offers no fused form for a shape, the model simply falls back to the plain composition; the write-once source is unchanged either way.

This lever is *domain-neutral* and *extensible*. The forms are named by their mathematical shape, not by any application: the first is the scaled product exponential `exp(c · x · y)`, of which a soil-moisture vegetation attenuation `exp(−2 · b · ndvi)` and a Beer–Lambert two-way extinction `exp(−2 · κ · ℓ)` are instances — the domain reading lives downstream, in the science model that supplies the coefficient. Further forms — an affine exponent `exp(a + c · x)` (Arrhenius, log-linear models), a Gaussian `exp(−γ · x²)` (radial-basis kernels), a numerically-stable `logSumExp`, a reciprocal square root — drop in the same way, each earned by a specific hot path rather than added speculatively.

A fused form is the *special case* of the general fusion the next strategy performs. The megakernel compiles the *whole* model into one kernel; a fused form hand-collapses *one named shape* within an otherwise eager execution. It is the right tool when a single sub-expression dominates and the rest of the model is content to run eagerly, or as the concrete kernel a compiled path targets for that shape. The same discipline governs both: a fused form is trusted because it is a refinement of its composed spec, exactly as the megakernel is trusted because it is proved faithful to the recorded model.

18.3.5. Vocabulary extensions –– tabulated data as a declared capability

A fused form refines a shape the base vocabulary can already *compose*. Some operations are not like that: a look-up into tabulated data is a data-dependent gather, and the branchless carrier vocabulary deliberately has no indexing to compose it from — the exclusion that makes every model above lower to one straight-line kernel. When a model genuinely needs a table (a tabulated transfer function, a calibration curve), the honest move is not to smuggle the gather in but to *declare* it: a vocabulary-*extension* class whose single operation is piecewise-linear interpolation into a named,

layered, uniform-abscissa table. Unlike a fused form it has *no* composed fallback — a model that uses it says so in its type, so the dependence on tabulated data is visible exactly where the write-once discipline wants it: in the instance context.

The extension keeps every leg of the discipline. Its reference semantics is one small function — clamp, floor, linear interpolation, the uniform-grid analogue of the operational interpolator — which is simultaneously the scalar oracle, the value a recording stores (recorder-faithful by construction), and the meaning the interpreter assigns a recorded fetch. On the device it is realised by *texture hardware*: the table lives in a layered texture bound once, fetched through the GPU's dedicated texture cache — a table of tens of kilobytes is simply *resident* there, so per-fetch traffic effectively vanishes. One boundary must be stated rather than wished away: the texture unit's hardware interpolation uses a low-precision fixed-point blend weight, so the *hardware-filtered* mode is a tolerance-bounded accelerator, never bit-reproducible; the *point-fetch* mode — two exact texel reads and an explicit, contraction-blocked lerp — is the bit-reproducible twin of the reference, and it is the mode every bit-exact claim quantifies over. The split is fixed at generation time, not discovered in production. Two further consequences come for free: the table's *name* rides in the recorded node, so common-subexpression elimination distinguishes fetches into different tables with no change to its key; and because texture hardware interpolates only *uniform* coordinates, an inverse table over a non-uniform abscissa must first be inverted and resampled host-side — a one-time, measured, controllable approximation that belongs to the model's accuracy budget, stated next to the table's own discretization bias.

18.3.6. GPU, megakernel-compiled — Lean → recorded IR → CUDA source

The remaining headroom is recovered by *compiling* the model rather than dispatching it operation-by-operation. Because the write-once kernel is branchless — the `NumCarrier` guarantee — the same source, instantiated at a *recording* carrier, does not compute a number: it *records* its own elementwise operation graph. That graph is common-subexpression-eliminated to its DAG of distinct sub-expressions and emitted as a single fused CUDA `__global__` kernel: one thread per pixel, the *whole* model held in registers, with device memory touched only for the model's actual inputs and outputs. This is a genuine *ahead-of-time compilation* — from the Lean source, through a recorded intermediate representation, to CUDA C source text — and, again, *not* an interpreter: the generated kernel is ordinary compiled CUDA, with no graph walked at run time.

The trade the previous strategy could not make is now available: with every intermediate kept in a register instead of round-tripped to DRAM, the workload moves from memory-bound to *compute-bound*, and its arithmetic intensity rises above the roofline knee — as close to the device's peak as the science allows. The cost is a precondition: the model must have favourable AI properties — branchless (guaranteed) and *register-fittable*, since the register pressure of the fused kernel bounds how much of the model can be held live at once (a very large model is fused per-step and looped on the device, rather than fused whole). This is why the strategy is *not* universal in the way the single-threaded baseline is: it buys the roofline in exchange for a structural demand on the model.

Two properties make this more than a code generator. First, it is *verified*: the recorded graph is proved — sorry-free, over all inputs, by the tape's forward-value faithfulness — to compute exactly the source kernel, so the generated kernel is faithful to the model *by a theorem*, not by a tested diff. Second, it is *bit-controllable*: compiled with fused-multiply-add contraction disabled and no re-association, the megakernel is bit-identical to the eager device path, because fusion changes only *where an intermediate lives* (a register versus DRAM), never the arithmetic performed — the speedup is pure memory-traffic elimination, orthogonal to the numerics. (Enabling contraction trades that exact identity for additional speed and a more-accurate-but-different result, a separate, quantified step.)

18.3.7. Julia, emitted — the sealed algorithm handed to a host that cannot call a theorem

The strategies above keep the model inside Lean: every carrier is a Lean type, and the generated CUDA is launched from a Lean program. A model's consumer is often elsewhere. An Earth-system model written in Julia, a calibration pipeline, an analysis notebook can call a function it is handed; it cannot instantiate a `[NumCarrier α]` kernel or cite a theorem. The Julia emitter (`PropertyKindCalculus.Torch.Paradigm.TapeCodegenJulia`) is how a sealed algorithm reaches such a host: the same recorded, compacted tape the megakernel is lowered from is rendered as a Julia function — a plain host loop, the bit-exact twin, and a KernelAbstractions.jl `@kernel` the one `.jl` runs on CPU, CUDA, AMDGPU or Metal, the portability twin, which is tolerance-only because a backend may fuse `a·b + c` into a multiply-add (the CPU backend does, through PoCL) and a conditioned tail — the LM step's normal equations and Cholesky solve — amplifies that ulp a thousandfold while leaving the relative error near `1e-13`. It is not a fifth carrier. It is the fourth strategy's intermediate representation given a third surface syntax, beside the CUDA kernel and the C stub, so that what crosses into the host is the tape and nothing else.

That last clause is the strategy. It rests on a theorem and on a reading of the emitter's definition, and the two are kept apart. The theorem is the seal of the computation (the second capstone), stated over the tape the emitter renders: every leaf in the backward cone of an output is either a named input or a baked constant the hypergraph's assumption ledger lists, two environments that agree on the influencers of an output give the same value at it, and every edge between observable vertices realizes one hop of one occurrence the source states, with the kinds that occurrence names — the kind-transporting weak bisimulation between the metrological provenance hypergraph and the computational tape graph. The reading is of `genJulia`: it adds no entry point to that picture — a Julia argument exists for exactly the named leaves (its input list is `collectInputs`, the function `gen` uses, name for name), a literal appears in the body for exactly the baked leaves (rendered from the double's bit pattern, so it *is* the constant the ledger lists), and a line of the body is one node's operation on its parents, over the alphabet the Float interpreter `cOp` defines. That reading is a theorem, and not a theorem about Julia: every emitter is a **printer** of one backend-neutral straight-line program that `genSL` — the only function that reads the tape — produces, one line per node, and the denotation of the rendering says the program runs as the tape evaluator, node for node, with a literal exactly at a baked leaf and an input read exactly at a named leaf (`genSL_denotes`, `genSL_lit_iff`, `genSL_input_iff`), and that the program's graph is the tape's

(`ofProg_eq_ofTape`), so the seal of the computation is a theorem about the program a host receives (`semantic_seal_program`). The Julia printer (`printJulia`) is a total function from a program line to text that cannot see the tape, so it cannot add an input, a literal or an operation the program does not have; what it leaves to trust is that each of its dozen forms means in Julia what the line means in Lean's `Float`. In the words a reviewer would use: *no naked number enters the Julia code beyond what the algorithm's contract admits* — and the same sentence holds, by the same theorem, of the C stub and the CUDA kernel, which print the same program. An input the hypergraph says cannot reach an output does not reach it in the Julia function either, because the Julia function is the tape, and the statement was proved of the tape.

What the emitter pins, so the generated file is in the regime the theorems describe (the header of every emitted file says the same):

- no `@fastmath`, no `muladd` — Julia does not contract `a·b + c` to a fused multiply-add or re-associate unless asked, which is the `nvcc --fmad=false` regime the megakernel is compiled in;
- `min`/`max` are written out as `ifelse(a <= b, a, b)` / `ifelse(a <= b, b, a)`, which is Lean's `Min Float`/`Max Float` (`minOfLe`/`maxOfLe`) to the letter, NaN cases included — the same ≤-ternary the C and CUDA renderings write, so the interpreter and all three renderings are one function on NaN inputs (`fminf`/`fmaxf`, which return the non-NaN operand, are not used);
- `sqrt` and `log` of a negative argument throw in Julia where IEEE returns a quiet NaN; the emitted helpers restore the NaN and are the identity elsewhere;
- precision is a type parameter `T`: at `Float64` the emitted function is bit-identical to `evalTape` on the IEEE-exact alphabet, because Lean's `Float` *is* binary64 — the first emitted target with no narrowing at the boundary, where the CUDA kernel is `float` with `double ↔ float32` casts in its launcher; at `Float32` it is the device kernel's precision.

What it does not do, said plainly. It does not put the host's own code under the seal: the Julia model's dynamical core, time stepping and communication are the host's, and the seal says nothing about them. It does not certify Julia's compiler or its `exp`/`log`: those are the trusted base the target adds, named as such — LLVM compiling a straight-line function, and a third libm (pure-Julia `Base.Math`) beside glibc's and CUDA's. The honest check across that base is `apps.julia_parity`, and its scope is the three recorded tapes it carries — the AVS residual and Jacobian, the whole projected LM step, and the table-fetch model of `examples.tape_codegen_lut` — not every kernel the emitter could be given: each emitted file is run under `julia` at `Float64` over deterministic pixels against the tape evaluator, bit-identical on every output whose cone has no `exp`/`log`, within an ulp bound on the outputs whose cone does. A new closure earns the same check by being added as a case. And it covers the tape's program class and no more — a branchless, fixed-shape computational tape graph over ten operations and a table fetch, with reductions only when unrolled — so a closure with a data-dependent loop, a runtime-size reduction or a stencil is outside it today; the first such closure recorded against `[NumCarrier α]` is what tells which operation to add to the alphabet, each with its interpreter case, its two renderings and its rendering-table row.

18.3.8. Recording carriers are deferred — share loop-carried state or the graph explodes

The megakernel strategy carries one authoring hazard worth stating on its own, because it is a property of the *recording carrier* rather than of any model. That carrier is *deferred*: a recorded value is a thunk that re-emits its sub-graph on every reference, not a memoized handle to an already-emitted node. Reusing an intermediate therefore records it *again* — and an iterated computation reuses its loop-carried state on every step. A fixed-point sweep whose accumulator appears `k` times in the body, composed to depth `d`, records on the order of `k^{d}` copies of the first step *before* common-subexpression elimination can collapse them. CSE dedups the *result*, but the recorder must first *build* that intermediate, so a deep, reuse-heavy fold can exhaust memory at elaboration time even though the final DAG — and every carrier's actual arithmetic — is small.

The discipline is the recording-time analogue of the megakernel's own *fuse one step, loop the device* shape: record *one* step and *materialize* the loop-carried state to a single node between steps, so the next step references it as one shared leaf instead of re-recording the entire prior step. Materialization is semantics-preserving — the shared node computes the same value — so the recording stays bit-for-bit the same kernel; only its *cost of being recorded* changes, from exponential to linear in the step count. This is invisible at the scalar, threaded, and eager-GPU carriers, where a value is just a number or a buffer and reuse is free; it surfaces only when the same write-once source is run at the deferred recording carrier, and it is a general hazard for *any* science model with an unrolled iteration recorded that way — not a quirk of one retrieval. It was found and fixed in exactly that setting: an eight-sweep loss-coupling fixed point in the SMAP–NISAR closed-form Stage-3 retrieval, its accumulator reused several times per sweep, unrolled past a hundred gigabytes at elaboration until the fold was threaded to share its accumulator per sweep — after which the same recording completed in seconds, the generated kernel unchanged.

18.3.9. Right-sizing the deployment — the memory shape is computed, the platform is asked

Choosing a strategy is only half of a deployment decision; the other half is a *number* — how many task-shards the multi-threaded strategy forks, or how many pixels a device batch carries. Both are bounded by memory before they are bounded by compute: the eager device carrier materializes a full buffer per live intermediate, and the sharded CPU path holds every concurrent slice at once. And the bound is a property of *this model on this machine*, so a hand-set constant is wrong somewhere — cores idle on the large host, or an out-of-memory kill on the confined worker. The calculus therefore treats sizing the way it treats everything else in this chapter: the model's memory shape is *computed*, from the same recording that already proves faithfulness and computes arithmetic intensity; the platform is *asked* what it will actually grant; and a small solver combines the two — with an explicit operator override that wins verbatim when someone knows better.

The model half is computed, not cited — the same standard the intensity accounting set. Memory use is affine in the pixel count for every strategy, `bytes(P) = fixed + perElement · P`, and the per-element term falls out of the recorded DAG per strategy. For the fused megakernel it is exact: inputs plus outputs, four bytes each — every intermediate is a register — with a bound table charged once per launch. For the eager carrier it is the *peak count of simultaneously-live buffers* when the DAG is evaluated in tape order, each buffer freed after its last consumer: one pass recovers every node's last use, one walk takes the running peak (`maxLiveNodes`, reported beside the intensity numbers by the same `aiReport`), and the sharded CPU path adds its host marshaling columns on top. One honesty clause belongs here: the eager number is a *lower bound*, because "freed promptly after its last consumer" is what a garbage-collected finalizer approaches rather than guarantees, and a driver that loops a recorded step holds its carry across iterations. So the static shape is the *prior* — what makes a freshly authored model safely sizable before it has ever run — and a measured probe of a real block (allocator peak on the device, child peak memory on the host) is the *authority* that calibrates it.

The platform half is asked with its provenance kept, because the budget that matters is the one enforcement reads. A confined process is killed at its cgroup limit, not at the host's advertised available memory — a container capped at 28 GiB on a host reporting 62 GiB is the observed norm on shared worker queues — so the budget query asks cgroup v2 first (the process's own cgroup, minimum over its ancestors), falls back to cgroup v1, and only then to the host's own figure, every answer labeled with its source so the log records *which limit the sizing believed*. Core counts get the same discipline: the scheduler's affinity mask intersected with the cgroup CPU quota, never the raw host core count. On the device the query is the CUDA free-memory figure, taken *after* the carrier's warm-up — the context's fixed tax is then already netted out of it, which is one fixed overhead nobody has to specify.

Two solver shapes close the loop, because the two knobs constrain differently. Task-sharding partitions *one resident tile* among concurrent shards, so the variable term is invariant in the shard count and only a per-shard fixed cost scales with it — and when the resident term *alone* exceeds the budget, no shard count fits, and the solver says exactly that (shrink the tile, not the count) instead of clamping to a plausible-looking answer. Block-concurrency — a fixed block size, choosing how many run at once — divides the remaining headroom by the per-worker cost. Either way the decision is announced on one line with every term and its source, so an out-of-memory kill or an inexplicably idle machine is diagnosable from the log alone; and the validation suite pins the peak-residency walk on hand-computed live-set structures and the capacity parsers on captured platform records, so the shapes are checked facts under the same CI as the parity theorems.

18.3.10. The through-line

The four strategies are one source, and the Julia emitter is that source handed out: not a strategy of its own but the recorded intermediate representation of the fourth given to a host that runs Julia, with the seal of the computation travelling with it because the host receives the tape and nothing else. The single-threaded baseline runs anything and is the oracle; the multi-threaded and both GPU strategies trade a structural demand on the model — independence and good arithmetic intensity, and for the megakernel, register-fittability — for progressively more of the hardware's peak. What licenses the automatic

parallelization and fusion of the latter three is not a heroic compiler analysis but the `NumCarrier` discipline itself: a model that typechecks against it is *already* branchless and fusible, so "go fast, automatically" is earned at authoring time by the same type that made "write once, correctly" true. The performance is a corollary of the specification, not a second, hand-written artifact that must be kept in step with it. And the deployment's *size* is earned the same way: the recording that licenses fusion also yields the model's memory shape, so how much of the spectrum a given machine can sustain is computed from the same single source — asked of the platform at run time, never tuned into the model.

19. Uncertainty quantification and numerical adequacy

The representation-parametric quantity of R10 carries a model's numbers at whatever carrier a task needs:

- $\mathbb{R}$ to prove,
- `Float` to run,
- `FP32` to bound rounding.

This chapter puts that parametricity to work on two questions a metrology model must answer but the calculus so far does not:

- *given the uncertainty of the input quantities, what is the uncertainty of the output?* and
- *does the floating-point representation of the model lose information that matters at the scale of those uncertainties?*

Both are developed against one shared, additive descriptor placed on a quantity — never a change to the carrier tower. The design is recorded in full in the project's `UNCERTAINTY.md`; this chapter is its blueprint face. Stages 0–3 are built and CI-checked (each item below links to the formal node that realizes it):

- the reference layer and worked examples,
- the GUM and Willink combines,
- the autograd sensitivity coefficients,
- the derivative-free SSPRC pipeline,
- the five ladder theorems:
  - T1, cumulant additivity;
  - T2, `\mathrm{gum} = \mathrm{willink}|_{\kappa_4=0}`;
  - T3, Willink as the projection of the linearized SSPRC;
  - T4, affine reference equals the mean;
  - T5, convolution adds cumulants, and
- the numerical-adequacy layer:
  - the executable analysis carrier;

    - the per-site theorems:
        - A1 absorption,
        - A2 Sterbenz,
        - A3 verdict soundness over $\mathbb{R}$;
        - the universal capstone A3′ over an arbitrary model DAG and input box;
        - A2 lifted to the genuine binary32 format `fexp32`; and
    - the executable ↔ spec bridge, certifying the runtime absorption verdict against the `round32` specification on TorchLean's computable `IEEE32Exec` model (George *et al.*, 2026)[19] Robert Joseph George, Jennifer Cruden, Will Adkisson, Xiangru Zhong, Huan Zhang, and Anima Anandkumar (2026). "TorchLean: Formalizing Neural Networks in Lean" (https://arxiv.org/abs/2602.22631). *arXiv preprint arXiv:2602.22631 (cs.MS)*. . ; and
    - the Axis-U significance wiring, coupling the autograd `c_i` with the descriptor's `u_i` into the significance scale `c_i u_i` the adequacy carrier checks against (Stage 3.4); and
    - its heterogeneous generalization, the kinded ×/÷ budget DAG, whose `ProductKind`/`QuotientKind` edges are threaded through a multi-node graph and whose threaded propagation collapses to the flat quadrature (residual 2); and
- the conformity-assessment layer (ISO/IEC Guide 98-4): tolerance and acceptance limits as bound roles, the guard band derived from a stated risk or from the costs of being wrong, a deployed band read as coverage or systematic under one classification rule (with its NaN rule), and the ratchet theorem that lets an acceptance limit tighten itself.

Since Stage 3.5 the sensitivity coefficients are also **proved** correct — the reverse pass is the adjoint of the Fréchet derivative — and Stage 3.6 carries that endpoint from the compiled tape (direct simulation) to the tape the bridge actually builds (eager provenance). The kinded ×/÷ budget DAG (residual 2) is realized, its threaded propagation proved to collapse to the flat quadrature.

19.1. Why formalize this? What the rigor buys

Uncertainty analysis is routinely done in a spreadsheet or a NumPy script, so casting it in a proof assistant earns its keep only by buying something those tools cannot. It buys five things — each **exercised by a checked example** below, not asserted here.

- **The published numbers are re-derived under CI, not transcribed.** Each worked example reproduces a headline figure of its source paper as a `#guard`, so the `UncertaintyExamples` library **building** is what certifies the figure. Degenhardt's fictive model (`UncertaintyExamples.DegenhardtFictive`, its GUM answer cross-checked against a Monte Carlo reference) and Willink's gauge block (`UncertaintyExamples.WillinkGaugeBlock`, by the cumulants

method) reproduce their papers' `u_c`, excess `\gamma_Y`, and expanded-uncertainty half-widths to the printed digits.

- **The method hierarchy is a theorem, so the three methods cross-validate for free.** GUM ⊂ Willink ⊂ SSPRC is a chain of proved homomorphisms (T2, T3, resting on T5), not a slogan. Wherever their hypotheses hold the three answers **must** agree; a discrepancy is a modelling error or a broken proof, caught mechanically rather than by eye (`UncertaintyExamples.SsprcNesting` pins the shared cumulants `(41, -1186)`).
- **The sensitivity coefficients are proved equal to the derivative, not merely computed.** The `c_i = \partial f/\partial X_i` the two linearized rungs need come from one reverse pass over the write-once model (the sensitivity bridge); since Stage 3.5 that reverse pass is **proved** to be the adjoint of the Fréchet derivative, an endpoint that holds on the very tape the bridge builds. A `c_i` entering a budget is therefore correct by theorem, not by trust in an autograd library (`UncertaintyExamples.DegenhardtSensitivity` returns `(5, 5, 2.25)` and reproduces `u_c = 1.662`).
- **"No information loss" is decided, not hoped for — and at the scale that matters.** The hazard this half of the chapter guards is silent: a small-but-significant input uncertainty rounded away in floating point, corrupting the result with no error raised. The adequacy carrier detects exactly that, and its verdict is sound and complete — the flag fires **iff** a contribution is genuinely lost — at the half-ulp scale the **same** descriptor defines. `UncertaintyExamples.AdequacySwamping` flags a contribution swamped under a `10^8` accumulator and certifies the small-accumulator variant clean, while `UncertaintyExamples.AdequacyCoupling` drives the significance coupling — the uncertainty budget and the adequacy check from one descriptor list — so a UQ-analysed model is **simultaneously** adequacy-checked at its own uncertainty scale.
- **One model serves proof, differentiation, execution, and certification with no re-implementation.** All of the above are interpretations of a single write-once kernel at different carriers, so the proved model and the model that runs cannot drift apart — they are one source. The next section makes this composition precise; the working consequence is that an edit to the model is proved, differentiated, run, and adequacy-checked from the one definition.

None of this substitutes for measurement, or for judgement about which distribution an input follows. What it removes is a specific class of **silent** errors:

- **a mis-stated propagation law** — the combine rules are **derived**, not asserted: GUM and Willink are proved projections of the finer method (T2, T3), so a wrong law surfaces as a broken proof or a numerical cross-validation mismatch rather than a plausible-looking answer;
- **a sensitivity taken on trust** — the coefficients `c_i` are proved equal to the adjoint of the Fréchet derivative on the very tape the sensitivity bridge builds (eager provenance), not read off an unverified autograd library;

- **an uncertainty contribution quietly lost to rounding** — the adequacy carrier detects absorption, and its verdict is sound and complete at the half-ulp scale the same descriptor defines, so a swamped contribution is **reported**, not dropped in silence.

The uncertainty formalism turns each into an obligation the build must discharge.

19.2. Two orthogonal axes, and why their properties compose

A science model is written **once** as a carrier-polymorphic kernel over `[NumCarrier α]` (the WO1 discipline of the previous chapter). On top of that single source sit *two orthogonal parametrization axes*:

- **the numeric carrier** `R` (Axis N) — **how** the numbers are represented: `ℝ` for proof, an autograd tape for sensitivity, `FP32`/`RInterval` for numerical adequacy, `CudaT`/ Tensor for GPU execution, `Float` for a fast CPU reference;
- **the uncertainty descriptor** (Axis U) — **what is known about an input's dispersion**: a per-quantity distribution carrying its moments, an inverse cumulative distribution function, and a support range.

The value of parametrizing both is that the guarantees **compose** — the same model yields a proof, a sensitivity, a numerical-adequacy certificate, and a GPU run as different interpretations, not as four re-implementations:

- **Proof carrier (`ℝ`).** The three uncertainty methods below are one provably nested ladder; numerical adequacy is a theorem, not a heuristic.
- **the autograd tape (sensitivity).** Exact sensitivity coefficients `c_i = \partial f/\partial X_i` for the linearized methods, and local surrogates for the sampling method.
- **the uncertainty descriptor (noise).** Measurement uncertainty becomes a first-class, composable property of a quantity, in the spirit of the VIM.
- **`FP32`/`RInterval` (no information loss).** A certificate that the model does not silently drop a small-but-significant input uncertainty — **at the scale that the same descriptor defines**.
- **`Float`/`CudaT` (fast execution).** The identical model runs, batched, on CPU and GPU.

19.3. The input uncertainty descriptor

Definition 19.3.1. **group** uses 0 used by 0 ✓

The **moment data** of an input: its mean, variance (`\kappa_2 = u^2`), fourth cumulant (`\kappa_4 = w`), and third cumulant (`\kappa_3`, asymmetry). Carrier-parametric — a bare record over any `R` — so it exists at `ℝ` for proofs and `Float` for execution alike. The cumulant constants of the named distributions (normal, uniform, triangular, arc-sine) are the values tabulated by Willink (Willink, 2005)[20] Robin

20.

Willink (2005). “A procedure for the evaluation of measurement uncertainty based on moments” (https://iopscience.iop.org/article/10.1088/0026-1394/42/5/001). *Metrologia.* **42**(5). .

Lean code for Definition **19.3.1.** 1 definition

`structure` (4 fields) · defined in `PropertyKindCalculus/Uncertainty/InputDist.lean` complete

```
structure PropertyKindCalculus.Uncertainty.MomentData (R : Type) : Type
```

Proof for Definition 19.3.1 uses 0

Realized. A four-field structure `MomentData R`; the named distributions are **values** built by smart constructors (`InputDist.normal`, `.uniform`, `.triangular`, `.arcsine`), each filling in its Willink-Table-1 cumulants — the open-world, values-not-constructors convention the whole project follows.

Definition 19.3.2. group uses 0 used by 0 ✓

The **input uncertainty descriptor**: the moment data, an optional rigorous support `[\mathrm{lo}, \mathrm{hi}]`, and an inverse-CDF sampler. Feeding a random probability to the sampler gives Monte Carlo; feeding a systematic probability `(j-\tfrac12)/N` gives the systematic sampling of the efficient method — one primitive, two methods.

Lean code for Definition **19.3.2.** 1 definition

`structure` (3 fields) · defined in `PropertyKindCalculus/Uncertainty/InputDist.lean` complete

```
structure PropertyKindCalculus.Uncertainty.InputDist (R : Type) : Type
```

Proof for Definition 19.3.2 uses 0

Realized. `InputDist R` bundles `MomentData` with a `support` and an `invCDF`. The paired `UncertainQuantity k R` places it alongside a kinded `Quantity k R` as additive metadata, exactly the pattern R11 used for vector carriers — the carrier tower is untouched.

19.4. The method ladder: GUM ⊂ Willink ⊂ SSPRC

For a model `Y = f(X_1,\dots,X_k)` with mutually independent, uncertain inputs, three methods estimate the uncertainty of `Y`, in increasing fidelity and cost. They are **one** pipeline — extract each input's

contribution, combine the independent contributions, summarize — differing only in what a contribution is:

- **GUM / law of propagation.** Linearize and combine variances only: `u_c(Y) = \sqrt{\textstyle\sum_i c_i^2\, u^2(X_i)}, \qquad c_i = \partial f/\partial X_i`. A symmetric, Gaussian answer.

21. - **Willink's cumulants method** (Willink, 2005)[21] Robin Willink (2005). "A procedure for the evaluation of measurement uncertainty based on moments" (https://iopscience.iop.org/article/10.1088/0026-1394/42/5/001). *Metrologia.* **42**(5). . The same linearization, but combine the *second and fourth* cumulants, `u_Y^2 = \sum_i c_i^2 u_i^2` and `w_Y = \sum_i c_i^4 w_i`, form the excess `\gamma_Y = w_Y/u_Y^4`, and read an expanded-uncertainty half-width off the Pearson family via the coverage-factor `k_p(\gamma_Y)`. This recovers the **tail shape** (kurtosis) GUM discards.

22. - **The SSPRC method** (Degenhardt *et al.*, 2025)[22] Johannes Degenhardt, Rainer Tutsch, Xiukun Hu, and Gaoliang Dai (2025). "A practically oriented, efficient alternative to the Monte Carlo method for measurement uncertainty estimation" (https://iopscience.iop.org/article/10.1088/1681-7575/adb3ab). *Metrologia.* **62**(2). . No linearization: systematically sample each input, propagate each **separately** through the model, reconstruct each input's deviation distribution, and combine them by *convolution*. It captures full non-linearity and each input's individual uncertainty contribution, at a fraction of the model evaluations a random Monte Carlo needs.

The organizing fact is that these are **provably nested**. All three rest on the additivity of cumulants under independent summation — Willink's `\kappa_r\!\bigl(\sum_j a_j Z_j\bigr) = \sum_j a_j^{\,r}\, \kappa_r(Z_j)`. GUM keeps only `\kappa_2`; Willink keeps `\kappa_2` and `\kappa_4`; SSPRC's convolution combines **all** cumulants numerically (Willink himself calls his method "convolution of the components in the linearized formulation"). So the ladder is a chain of homomorphisms between the methods' contribution types, and each coarser rung is a **projection** of the finer one — the claim the ladder theorems below discharge, all five now proved over $\mathbb{R}$: T1 and T2 relate GUM and Willink; T5 shows convolution adds cumulants; T3 uses it to identify Willink with the `(\kappa_2,\kappa_4)` truncation of the linearized SSPRC; and T4 pins the SSPRC reference to the mean for an affine model. The sensitivity coefficients `c_i` the two linearized rungs need come from the autograd bridge below, not by hand.

Definition 19.4.1. group uses 0 statement">used by

The GUM combined standard uncertainty `\sqrt{\sum_i c_i^2 u_i^2}`, over a list of sensitivity / moment-data pairs. The coarsest rung.

Lean code for Definition 19.4.1. 1 definition

```
def · defined in  PropertyKindCalculus/Uncertainty/Combine.lean
```

complete

```
def PropertyKindCalculus.Uncertainty.gumStdUnc
  (terms :
    List (Float × PropertyKindCalculus.Uncertainty.MomentData Float)) :
  Float
```

Proof for Definition 19.4.1 uses 0

Realized. `gumStdUnc` folds the `\kappa_2` contributions. Stage 0 supplied the `c_i` explicitly; Stage 1 now sources them from the autograd carrier (one reverse pass over the write-once model at the input means), leaving this fold unchanged — the Degenhardt example checks the two agree, both giving `u_c = 1.662`.

Definition 19.4.2. group uses 0 statement">used by

Willink's cumulant combine → `(u_Y, \gamma_Y)`, with the Pearson coverage-factor fits `k_{0.95}`, `k_{0.99}` giving the expanded-uncertainty half-width. The middle rung.

Lean code for Definition 19.4.2. 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Combine.lean` complete

```
def PropertyKindCalculus.Uncertainty.willinkCombine
  (terms :
    List (Float × PropertyKindCalculus.Uncertainty.MomentData Float)) :
  Float × Float
```

Proof for Definition 19.4.2 uses 0

Realized. `willinkCombine` combines `\kappa_2` and `\kappa_4`; `willinkK95`/`willinkK99` are the rational Pearson-percentile fits (Willink eqs. 6/7); `willinkHalfWidth99` composes them.

Definition 19.4.3. group uses 0 used by 0

The Monte Carlo **reference** propagator: sample the joint input distribution, push each draw through the model, report the sample mean and standard uncertainty of the output. Makes no linearization and no combination assumption, so it is the yardstick the cheaper rungs are validated against.

Lean code for Definition 19.4.3. 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Mcm.lean` complete

```
def PropertyKindCalculus.Uncertainty.Mcm.run (model : List Float → Float)
  (inputs : List (PropertyKindCalculus.Uncertainty.InputDist Float))
  (n : ℕ) (seed : UInt64) : Float × Float
```

Proof for Definition 19.4.3 uses 0

Realized. `Mcm.run` over the `Float` carrier, drawing each input through its `invCDF`. Efficiency is not its job — fidelity is.

Definition 19.4.4. group uses 0 statement">used by

The **sensitivity bridge**: the coefficients `c_i = \partial f/\partial X_i` that GUM and Willink need are obtained by instantiating the write-once model at an autograd carrier and running one reverse pass at the input means — not by hand. This is the `extract` step of the two linearized rungs, and the narrow, precise role of automatic differentiation here, since the SSPRC method is derivative-free.

Lean code for Definition 19.4.4. 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Sensitivity.lean` complete

```
def PropertyKindCalculus.Uncertainty.Sensitivity.gradient
  (model : PropertyKindCalculus.Uncertainty.Sensitivity.ScalarModel)
  (point : List Float) : Except String (List Float)
```

Proof for Definition 19.4.4 uses 0

Realized (Stage 1, `Sensitivity.gradient`; retargeted in Stage 3.5). The model kernel — the **same** source run at `Float` for Monte Carlo — is interpreted at TorchLean's reverse-mode `TapeBuilder` carrier; each input enters as a differentiable leaf, and one **total dense** reverse pass (`Tape.backwardDenseFrom`, seeded `1` at the scalar output) reads every `c_i`, with no model rewrite. Since Stage 3.5 this is the very entry point TorchLean's soundness theorem `backwardDenseFrom_lowerGraphToTape_adjoint_fderiv` characterizes: on a compiled graph at the `ℝ` carrier its input-prefix output **is** the adjoint of the Fréchet derivative of the forward evaluation (the eager-tape provenance — narrowed at Stage 3.6 to a closure-provenance question, see the direct-simulation theorem below — and the `Float`-vs-`ℝ` deviation remain the two documented gaps). On the Degenhardt fictive model the bridge returns `(c_1, c_2, c_3) = (5, 5, 2.25)` and reproduces the GUM `u_c = 1.662` exactly (the `DegenhardtSensitivity` example). The op class is `exp`/`log`/`sqrt` with arithmetic; trigonometric models await tape VJP nodes.

Theorem 19.4.5. group uses 0 used by 0 ✓

Direct runtime simulation, closed for compiled tapes (Stage 3.6). *The executable dense reverse pass on a compiled tape succeeds, and the input-prefix of its gradient **array** — the runtime's own shape-erased representation, block by block (`ArrCorr`) — realises exactly the adjoint of the Fréchet*

*derivative of the forward evaluation applied to the seed. Separately, and for **arbitrary** (not just compiled) value-correct tapes: the reverse pass returns `.ok` provided every backward closure is **shape-total** — a hypothesis the eager `leaf`/`add`/`mul` tape constructors are proved to provide, constructor by constructor.*

Lean code for Theorem 19.4.5.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Experiments/PRSimulation.lean` — complete

```
theorem PRSim.direct_PR_soundness_compiled {Γ ss : List Spec.Shape}
  (g : Proofs.Autograd.Graph Γ ss)
  (hg : Proofs.Autograd.GraphFDerivCorrect g)
  (x : TorchLean.TensorPack ℝ Γ)
  (seed : TorchLean.TensorPack ℝ (Γ ++ ss)) :
  ∃ out,
    (g.toAlgebra.lowerGraphToTape x ()).1.backwardDenseFrom
          seed.toShapeErasedArray =
        Except.ok out ∧
      PRSim.ArrCorr
        ((ContinuousLinearMap.adjoint
            (fderiv ℝ g.evalVec (Proofs.Autograd.flattenCtx x)))
          (Proofs.Autograd.flattenCtx seed))
        (out.extract 0 Γ.length)
```

Proof for Theorem 19.4.5

uses 0

Realized (`PRSim.direct_PR_soundness_compiled`, with `backwardDenseFrom_ok` for the totality half and `forwardSim_lowerGraphToTape` inhabiting the simulation relation; `Experiments/PRSimulation.lean`, sorry-free). The endpoint is the input-prefix projection of the Stage-3.5 theorem transported to the runtime array by two erasure lemmas (block-reads invert context flattening; `Array.extract` is the erased typed prefix) — no reverse-pass fold argument is re-derived, so the adjoint mathematics enters exactly once, through the Stage-3.5 bridge. The totality half threads the "every slot carries its node's shape" invariant through the accumulation fold, value-free; the shape-totality hypothesis is honest and necessary, since a value-correct tape with an erroring closure refutes the unconditioned claim — the closure-provenance identification for the **eager** path is the next theorem. The `AutogradDirectSim` example instantiates everything concretely and pins the axiom profiles to the classical trio.

Theorem 19.4.6.

group · uses 0 · used by 0 ✓

***Eager provenance (Stage 3.6 addendum).** The tape the eager runtime constructors actually build — input leaves, then one `add`/`mul` call per graph node, which is the construction pattern the sensitivity bridge uses — runs the **same** total dense reverse pass as the compiled tape of the corresponding*

graph: same result, same gradient array, from any erased-context seed. Therefore the Fréchet-adjoint endpoint above holds on the eagerly built tape itself, with no compilation anywhere in the trusted path.

Lean code for Theorem 19.4.6.

1 theorem

theorem · defined in `PropertyKindCalculus/Uncertainty/Experiments/EagerProvenance.lean`

complete

```
theorem PRSim.direct_PR_soundness_eager {Γ ss : List Spec.Shape}
  (g : Proofs.Autograd.Graph Γ ss)
  (hg : Proofs.Autograd.GraphFDerivCorrect g)
  (x : TorchLean.TensorPack ℝ Γ) {t : PRSim.RTape}
  (ht : PRSim.EagerBuilds g x t)
  (seed : TorchLean.TensorPack ℝ (Γ ++ ss)) :
  ∃ out,
    Runtime.Autograd.Tape.backwardDenseFrom t seed.toShapeErasedArray =
        Except.ok out ∧
      PRSim.ArrCorr
        ((ContinuousLinearMap.adjoint
            (fderiv ℝ g.evalVec (Proofs.Autograd.flattenCtx x)))
          (Proofs.Autograd.flattenCtx seed))
        (out.extract 0 Γ.length)
```

Proof for Theorem 19.4.6

uses 0

Realized (`PRSim.direct_PR_soundness_eager`, via `backwardDenseFrom_eager_eq_compiled`; `Experiments/EagerProvenance.lean`, sorry-free). `EagerBuilds` is the provenance relation — its base tape **is** the compiled tape of the empty graph, definitionally — and the identification is pure accounting, never a re-derived fold: the compiled node's dense contribution list is folded by the upstream accumulation bridge, the eager node's sparse two-element list by a new single-slot lemma (one accumulation call is a one-hot context addition), and the two context updates coincide by flatten-injectivity through the vectorization homomorphisms — the eager closure's stored product-rule payloads are exactly the `ofVec`-authored node's Hadamard vjp blocks. Push-invariance lemmas restrict the extended tape's loop to its prefix, so the structural induction consumes its hypothesis directly. Honest residuals: ops beyond `add`/`mul` (one more crank of the same machine per op), the `TapeM` state-monad sugar (a one-line wrapper per op), and the `Float`-vs-`ℝ` deviation — the workstream's own Adequacy claim. The `AutogradDirectSim` example witnesses `EagerBuilds` for the concrete product tape and pins the classical-trio axiom profile.

Theorem 19.4.7.

group uses 0 statement">used by

Cumulant additivity (T1). *Combining independent contributions adds their cumulants: the measurand's combined `(\kappa_2, \kappa_4)` for an input list is the head input's contribution plus the combined cumulants of the tail. Equivalently the contribution type is a commutative monoid under*

`combine` — ***and that it is a monoid is the independence assumption made explicit***. *This is Willink's `\kappa_r(\sum_j a_j Z_j) = \sum_j a_j^{\,r}\,\kappa_r(Z_j)` for `r \in \{2, 4\}`, the fact the whole ladder rests on.*

Lean code for Theorem 19.4.7.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Ladder.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.willinkCumulants_cons
  (t : ℝ × PropertyKindCalculus.Uncertainty.MomentData ℝ)
  (ts : List (ℝ × PropertyKindCalculus.Uncertainty.MomentData ℝ)) :
  PropertyKindCalculus.Uncertainty.willinkCumulants (t :: ts) =
    PropertyKindCalculus.Uncertainty.termCumulants t +
      PropertyKindCalculus.Uncertainty.willinkCumulants ts
```

Proof for Theorem 19.4.7

uses 0

Realized over `ℝ`: `willinkCumulants_cons`, with the `Cumulants AddCommMonoid` instance (the `combine`/`empty` monoid laws) providing the associativity/commutativity/unit that make the independent fold well-defined.

Theorem 19.4.8.

group uses 3 statement">used by

Tags

capstone

proved

GUM is the `\kappa_2`-projection of Willink (T2). *The second cumulant of the Willink combine is exactly the GUM combined variance, so `\mathrm{gum}` is the cumulant-forgetting coarsening of `\mathrm{willink}`. And when every input's fourth cumulant vanishes (`w_i = 0`, as for normal inputs) the excess `\gamma_Y` is zero, the Pearson coverage factor collapses to the standard-normal percentile (`k_{0.95}(0) = 1.96`), and the Willink expanded-uncertainty half-width equals GUM's Gaussian half-width `1.96\,u_c`. Uses the GUM combine and Willink's combine, resting on cumulant additivity.*

Lean code for Theorem 19.4.8.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Ladder.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.gum_eq_willink_of_normal
  (terms : List (ℝ × PropertyKindCalculus.Uncertainty.MomentData ℝ))
  (h : ∀ t ∈ terms, t.2.fourthCumulant = 0) :
  PropertyKindCalculus.Uncertainty.willinkHalf95 terms =
    PropertyKindCalculus.Uncertainty.gumHalf95 terms
```

Proof for Theorem 19.4.8 uses 0

Realized over $\mathbb{R}$ (theorem T2 of `UNCERTAINTY.md`, `gum_eq_willink_of_normal`; `#print axioms` shows it sorry-free — no `sorryAx`). Two parts: `willinkCumulants_kappa2` — the variance fold is literally the GUM one — and the shape collapse $w_i = 0 \Rightarrow \gamma_Y = 0 \Rightarrow k_p$ is the standard-normal percentile, so the half-widths coincide. Establishes the top edge of the ladder; the lower edge — that Willink is the fourth-cumulant truncation of the linearized SSPRC, because convolution adds cumulants — is theorem T3 below.

19.5. The SSPRC method and the lower ladder rungs

The two linearized rungs replace the model by its gradient at the input means. The **SSPRC method** [23] does no such thing: it keeps the full model but evaluates it **cheaply** by sampling each input systematically and propagating each one separately. Its four steps are systematic sampling (each input at the equidistant probabilities $P_j = (j-\tfrac12)/N_i$ through its inverse CDF), separated propagation (vary one input, hold the rest at their means, form the deviations $a_{i,j} = f(\dots,x_{i,j},\dots) - R$ from the reference $R = f(E(X))$), reconstruction of each input's deviation distribution, and **convolution** of those distributions into the measurand's. Because the sample counts **add** ($N_s = \sum_i N_i$) rather than multiply, it reaches Monte Carlo fidelity at a fraction of the model evaluations — and, keeping the full model, it recovers the **non-linear** mean the linearized rungs cannot.

[23] (Degenhardt *et al.*, 2025) Johannes Degenhardt, Rainer Tutsch, Xiukun Hu, and Gaoliang Dai (2025). "A practically oriented, efficient alternative to the Monte Carlo method for measurement uncertainty estimation" (https://iopscience.iop.org/article/10.1088/1681-7575/adb3ab). *Metrologia.* **62**(2).

Definition 19.5.1. group uses 0 used by 0 ✓

The **SSPRC pipeline**: from the model, the input descriptors, and a per-input sample count, produce the combined $(E(Y), u(Y))$ by systematic sampling, separated propagation, and convolution — a derivative-free method whose evaluation budget is additive in the inputs.

Lean code for Definition 19.5.1. 1 definition

```
def · defined in PropertyKindCalculus/Uncertainty/Ssprc.lean
```

complete

```
def PropertyKindCalculus.Uncertainty.Ssprc.run (model : List Float → Float)
  (inputs : List (PropertyKindCalculus.Uncertainty.InputDist Float))
  (ns : List ℕ) (h : ns.length = inputs.length := by decide) :
  Float × Float
```

Proof for Definition 19.5.1 uses 0

Realized (Stage 2, `Ssprc.run`) over the `Float` carrier. `run` reads `E(Y) = R + \sum_i E(A_i)` and `\mathrm{Var}(Y) = \sum_i \mathrm{Var}(A_i)` off the per-input deviation distributions (means and variances add under convolution, so the combinatorial convolution need not be materialized); `runConv` performs the actual discrete convolution and agrees, cross-checking the engine. On the Degenhardt fictive model with 100 systematic samples per input — 300 evaluations against the Monte Carlo reference's 20 000 — it returns `E(Y) = 11.5875` (recovering the non-linear `+0.3375` offset that GUM cannot see) and `u(Y) \approx 1.69`, closer to the Monte Carlo value than GUM's 1.662 (the `DegenhardtSsprc` example).

Theorem 19.5.2. group uses 0 used by 1 ✓

Convolution adds cumulants (T5). *For independent centered probability distributions the second and fourth cumulants are additive under convolution: `\kappa_r(d_1 \star d_2) = \kappa_r(d_1) + \kappa_r(d_2)` for `r \in \{2,4\}`. Equivalently the cumulant map is a homomorphism from the convolution operation onto the `Cumulants` monoid. This is the mathematical bridge under T3.*

Lean code for Theorem 19.5.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Convolution.lean` complete

```
theorem PropertyKindCalculus.Uncertainty.Dist.cumulantsOf_conv
  (d1 d2 : PropertyKindCalculus.Uncertainty.Dist) (h1 : d1.IsProb)
  (h2 : d2.IsProb) (c1 : d1.IsCentered) (c2 : d2.IsCentered) :
  (d1.conv d2).cumulantsOf = d1.cumulantsOf + d2.cumulantsOf
```

Proof for Theorem 19.5.2 uses 0

Realized over ℝ (`Convolution.lean`): convolution is **defined** as the distribution of the sum of independent contributions — every pairwise sum weighted by the product — and the additivity is **proved** from the factorization of the joint expectation (`kappa2_conv`, `kappa4_conv`; the `-3\,E[X^2]^2` correction is exactly what cancels the cross term `6\,E[X^2]E[Y^2]` in the fourth moment of the sum), not assumed. Sorry-free (`#print axioms` → `propext`, `Classical.choice`, `Quot.sound`).

Theorem 19.5.3. group uses 2 used by 0 ✓

Tags

capstone

proved

> ***Willink is the `(\kappa_2,\kappa_4)`-projection of the linearized SSPRC (T3).*** *If each input's deviation distribution is a centered probability distribution realizing its declared moments, then the cumulants of the convolved combined deviation are* ***exactly*** *the Willink combined cumulants `(\sum_i c_i^2 u_i^2,\ \sum_i c_i^4 w_i)`. So convolving the separately-propagated inputs and truncating at the fourth cumulant reproduces `\mathrm{willink}` — the lower edge of the ladder, whose upper edge is T2. Rests on convolution adding cumulants.*

Lean code for Theorem 19.5.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Convolution.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.cumulantsOf_combinedDeviation
  (xs : List PropertyKindCalculus.Uncertainty.SensitizedInput)
  (hp : ∀ t ∈ xs, t.2.1.IsProb) (hc : ∀ t ∈ xs, t.2.1.IsCentered)
  (hk2 : ∀ t ∈ xs, t.2.1.kappa2 = t.2.2.variance)
  (hk4 : ∀ t ∈ xs, t.2.1.kappa4 = t.2.2.fourthCumulant) :
  (PropertyKindCalculus.Uncertainty.combinedDeviation xs).cumulantsOf =
    PropertyKindCalculus.Uncertainty.willinkCumulants
      (PropertyKindCalculus.Uncertainty.termsOf xs)
```

Proof for Theorem 19.5.3

uses 0

Realized over $\mathbb{R}$ (`cumulantsOf_combinedDeviation`): induction over the input list, each step the scaling law `\kappa_r(c_i \cdot Z_i) = c_i^{\,r}\,\kappa_r(Z_i)` followed by the convolution homomorphism T5, landing on `willinkCumulants` term by term. Together with T2 this closes the ladder `\mathrm{GUM} \subset \mathrm{Willink} \subset \mathrm{SSPRC}` as a chain of homomorphisms (the `SsprcNesting` example applies it to concrete distributions, pinning `(\kappa_2,\kappa_4) = (41, -1186)`). Sorry-free.

Theorem 19.5.4.

group · uses 0 · used by 0 ✓

> ***The SSPRC reference equals the mean for an affine model (T4).*** *When every input's deviation distribution is centered — the case of a linear/affine model — the combined deviation is centered, so the reference value `R = f(E(X))` equals `E(Y)`. The non-linearity signature `E(Y) - R` is exactly the non-vanishing of `\sum_i c_i\,E(Z_i)` when a deviation distribution is* ***not*** *centered — the gap the Degenhardt fictive example exhibits (0.3375) and the linearized rungs structurally miss.*

Lean code for Theorem 19.5.4. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Convolution.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.combinedDeviation_isCentered
  (xs : List PropertyKindCalculus.Uncertainty.SensitizedInput)
  (hc : ∀ t ∈ xs, t.2.1.IsCentered) :
  (PropertyKindCalculus.Uncertainty.combinedDeviation xs).IsCentered
```

Proof for Theorem 19.5.4 uses 0

> Realized over $\mathbb{R}$ (`combinedDeviation_isCentered`, with `mean_combinedDeviation` giving the general gap `\sum_i c_i\,E(Z_i)`): the mean of a convolution is the sum of the means (weighted by the unit totals), so a fold of centered contributions is centered. Sorry-free.

19.6. Numerical adequacy of the floating-point representation

A floating-point representation is **numerically adequate** for a model, given the input ranges and uncertainties, exactly when evaluating it loses no information significant **at the scale of those uncertainties**. The qualifier is what makes the notion checkable, and it is why this half of the chapter shares Axis U's descriptor: the yardstick is the very `c_i\,u_i` and support the uncertainty methods already carry. The textbook failure — adding a small floating-point number to a large one and losing the small one — becomes precise: an input whose uncertainty contribution `c_i\,u_i` falls below half a unit in the last place of the accumulated sum is **numerically invisible**, a silent corruption of the uncertainty result. Its exact-cancellation dual (a difference of near-equal quantities is exact by Sterbenz, yet amplifies **relative** uncertainty) is the same two views — floating-point and uncertainty — meeting again.

The idiomatic realization is **numerical adequacy as a carrier**: because the model is WO1 over `[NumCarrier α]`, instantiating it at an analysis carrier that tracks each value's magnitude and carried uncertainty, and checks each operation for swamping and cancellation, yields the adequacy report **for free** — no model rewrite. The check is sound because the swamping threshold is exactly half a unit in the last place, a fact proved on the rounding grid over $\mathbb{R}$ and grounded in TorchLean's binary32 `FP32` unit-in-the-last-place lemmas (`Adequacy.Fp32Grounding`, which imports the genuine — `noncomputable` — Flocq port); the executable carrier computes the same threshold over `Float`.

Definition 19.6.1. group uses 0 statement proof ✓

> The **adequacy analysis carrier**: a value carrying its magnitude, its propagated uncertainty, and an accumulated report of adequacy violations. As a `NumCarrier`, any write-once model instantiates at it and is analyzed for floating-point information loss with no rewrite — the **runtime certificate**, the pragmatic first target of the numerical-adequacy layer.

Lean code for Definition 19.6.1.

1 definition

`structure` (3 fields) · defined in `PropertyKindCalculus/Uncertainty/Adequacy.lean` — complete

```
structure PropertyKindCalculus.Uncertainty.Adequacy : Type
```

Proof for Definition 19.6.1

uses 0

Realized (Stage 3, `Adequacy`) over the `Float` carrier. Addition runs a swamping check (the smaller-magnitude operand's uncertainty against half the sum's ulp), subtraction a cancellation check (relative uncertainty reaching 100%); the `NumCarrier` instance closes with these plus the arithmetic and `MathCarrier` fields, so the **same** kernel the earlier rungs evaluate runs here to emit the report. The `AdequacySwamping` example flags a large-accumulator model and certifies its small-accumulator variant clean — one WO1 kernel, checked for free.

Theorem 19.6.2.

group uses 0 statement">used by

Absorption (A1). *On the rounding grid of spacing `u` (a ulp), a representable value perturbed by strictly less than half a ulp rounds back unchanged: `|y| < u/2 \Rightarrow \mathrm{round}_u(x + y) = x`. The perturbation carries **no** information into the result — floating-point swamping made precise — and its converse (`resolve`) shows a perturbation of at least half a ulp does move the result, so half a ulp is the **exact** absorption threshold.*

Lean code for Theorem 19.6.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Adequacy/Absorption.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.absorb {u x y : ℝ}
  (hu : 0 < u)
  (hx : PropertyKindCalculus.Uncertainty.Adequacy.OnGrid u x)
  (hy : |y| < u / 2) :
  PropertyKindCalculus.Uncertainty.Adequacy.gridRound u (x + y) = x
```

Proof for Theorem 19.6.2

uses 0

Realized over `ℝ` (`Adequacy.Absorption.absorb`, with `resolve` the converse): on the uniform grid `u·ℤ`, `round_u(x+y) = u·round((x+y)/u)`, and `|y| < u/2` places `(x+y)/u` within a half-integer of `x/u`, so it rounds to `x/u`. The binary32 realization is TorchLean's `neuralRound` nearest-point optimality and `neuralUlp` (`Adequacy.Fp32Grounding`). Sorry-free.

Theorem 19.6.3. group uses 0 used by 2 ✓

> ***Sterbenz (A2).*** *The difference of two positive floating-point numbers representable at precision `p` that lie within a factor of two of each other (`y \le x \le 2y`) is **itself** representable at precision `p` — so near-equal subtraction is **exact**, introducing no rounding error. Its dual is the uncertainty hazard: that exact difference can have **relative** uncertainty ≥ 100% (`relUnc_amplifies`).*

Lean code for Theorem 19.6.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Adequacy/Sterbenz32.lean` complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.flx_sterbenz {p : ℕ} {x y :
ℝ}
  (hy : 0 < y) (hyx : y ≤ x) (hx2y : x ≤ 2 * y)
  (hxr : PropertyKindCalculus.Uncertainty.Adequacy.FLX p x)
  (hyr : PropertyKindCalculus.Uncertainty.Adequacy.FLX p y) :
  PropertyKindCalculus.Uncertainty.Adequacy.FLX p (x - y)
```

Proof for Theorem 19.6.3 uses 0

> Realized over `ℝ` (`Adequacy.Sterbenz32.flx_sterbenz`), the real-number analogue of TorchLean's `neural_generic_format_FLX_sterbenz`: align both operands to the smaller exponent, and the factor-of-two condition bounds the difference's mantissa below `2^p`. `sub_exact_on_grid` gives the grid form (the difference rounds to itself). Sorry-free. Stage 3.2 carries this to the FLT/`fexp32` format binary32 actually uses (gradual underflow) — the genuine grounding is `thm_uq_sterbenz_fp32`.

Theorem 19.6.4. group statement">uses 1 used by 0 ✓

Tags: capstone, proved

> ***Sterbenz at the binary32 format (A2, Stage 3.2).*** *The same exactness at the format binary32 **actually uses** — TorchLean's `fexp32 = \mathrm{FLTExp}\,(-149)\,24`, an `FLT` format with gradual underflow: for representable binary32 `u`, `v` within a factor of two (`u \le 2v`, `v \le 2u`), the exact difference is already on the grid, so `\mathrm{round}_{32}(u - v) = u - v`. This realizes the `FLX`-model A2 at the real gradual-underflow format, and its `FP32`-typed corollary `sub32_exact_of_sterbenz` makes a near-equal binary32 subtraction **lossless**.*

Lean code for Theorem 19.6.4.

1 theorem

theorem · defined in PropertyKindCalculus/Uncertainty/Adequacy/Fp32Grounding.lean

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.round32_sterbenz_exact
  {u v : ℝ}
  (hu :
    FloatLib.Floats.Formats.Flocq.genericFormat
      FloatLib.Numerics.binaryRadix PropertyKindCalculus.Fp32.fexp32 u)
  (hv :
    FloatLib.Floats.Formats.Flocq.genericFormat
      FloatLib.Numerics.binaryRadix PropertyKindCalculus.Fp32.fexp32 v)
  (hupos : 0 < u) (hvpos : 0 < v) (huv : u ≤ 2 * v) (hvu : v ≤ 2 * u) :
  PropertyKindCalculus.Fp32.round32 (u - v) = u - v
```

Proof for Theorem 19.6.4

uses 0

Realized over ℝ/FP32 (Stage 3.2, `Adequacy.Fp32Grounding.round32_sterbenz_exact` / `sub32_exact_of_sterbenz`), grounded in the TorchLean PR's `neural_generic_format_FLT_sterbenz`. That `FLT` Sterbenz lemma splits on the underflow boundary `β^(prec+emin)`: on the subnormal side the fixed `emin` grid subtracts exactly (`FLT_to_FIX → FIX_sub → FIX_to_FLT`), on the normal side it transports through the unbounded `FLX` family (`FLT_to_FLX → FLX_sterbenz → FLX_to_FLT_of_normal`). Composing it with `neural_round_preserves_generic` (rounding is the identity on representable reals) gives `\mathrm{round}_{32}(u - v) = u - v`, so the per-op `sub32_within_half_ulp` half-ulp bound collapses to **zero** in the Sterbenz regime. Sorry-free (`[propext, Classical.choice, Quot.sound]`), exercised by the `AdequacySterbenz32` example.

Theorem 19.6.5.

group

statement">used by 4

✓

Tags

capstone

proved

Adequacy verdict soundness (A3). *At an addition site, the carrier's flag — the operand's uncertainty is below half the accumulator's ulp — holds* ***if and only if*** *the contribution is genuinely lost (the rounded sum is unchanged by that uncertainty). So the flag is* ***sound*** *(flag ⟹ lost) and* ***complete*** *(lost ⟹ flag): the theorem that makes the carrier's report trustworthy. Rests on absorption and its converse.*

Lean code for Theorem 19.6.5.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Adequacy/Soundness.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.verdict_sound {u s unc : ℝ}
  (hu : 0 < u)
  (hs : PropertyKindCalculus.Uncertainty.Adequacy.OnGrid u s)
  (h0 : 0 ≤ unc) (h1 : unc < u) :
  PropertyKindCalculus.Uncertainty.Adequacy.AbsorptionFlag u unc ↔
    PropertyKindCalculus.Uncertainty.Adequacy.gridRound u (s + unc) = s
```

Proof for Theorem 19.6.5

uses 0

Realized over ℝ (`Adequacy.Soundness.verdict_sound`): the forward direction is A1 `absorb`, the reverse is the contrapositive of `resolve`, giving the biconditional `unc < ½ ulp ↔ fl(s + unc) = s` for a representable accumulator and an uncertainty within one ulp. The `AdequacyLadder` example applies it to concrete values with a sorry-free axiom profile. This is the per-site soundness that grounds the runtime carrier.

Theorem 19.6.6.

group uses 2 used by 0 ✓

Tags

capstone

proved

Universal adequacy soundness (A3', the capstone). *Abstract a write-once model as a binary32 evaluation DAG over the full operator class +/−/×/÷. For* ***any*** *two inputs — in particular every input in a box around a nominal one — the FP32-computed measurand's* ***variation*** *reproduces the exact ℝ-computed variation up to a proven, DAG-additive rounding budget; and when no site rounds anywhere (flag-free — no absorption at any addition, every subtraction in the Sterbenz A2 regime), the bound collapses to* ***equality****. This lifts the per-site verdict A3 across a whole evaluation, and is the theorem that binds the layer to R10's exec/spec refinement.*

Lean code for Theorem 19.6.6.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Adequacy/DagBound.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.dag_fp32_box_faithful
  (e : PropertyKindCalculus.Uncertainty.Adequacy.Expr)
```

```
  (ρ σ : ℕ → TorchLean.Floats.FP32)
  (hρ : PropertyKindCalculus.Uncertainty.Adequacy.Regular e ρ)
  (hσ : PropertyKindCalculus.Uncertainty.Adequacy.Regular e σ) :
  |(PropertyKindCalculus.Uncertainty.Adequacy.evalFP32 e σ).val -
          (PropertyKindCalculus.Uncertainty.Adequacy.evalFP32 e ρ).val -
        (PropertyKindCalculus.Uncertainty.Adequacy.evalExact e σ -
          PropertyKindCalculus.Uncertainty.Adequacy.evalExact e ρ)| ≤
    PropertyKindCalculus.Uncertainty.Adequacy.errBound e σ +
      PropertyKindCalculus.Uncertainty.Adequacy.errBound e ρ
```

Proof for Theorem 19.6.6

proof">uses 1

Realized over ℝ/FP32 (Stage 3.1, `Adequacy.DagBound`). The forward-error accumulation `dag_fp32_error_bound` composes the per-operation half-ulp bounds (`Adequacy.Fp32Grounding.{add32,sub32,mul32,div32}_within_half_ulp`, the genuine `FP32.{add,sub,mul,div}_abs_error`) with the triangle inequality **and** the first-order propagation of operand errors along the DAG; applying it at both inputs and one more triangle step gives the box-faithfulness bound `dag_fp32_box_faithful`, and `dag_fp32_box_exact_of_flagFree` is the flag-free equality. The **linear** nodes +/− pass operand error through with coefficient one; the **nonlinear** ×/÷ carry the GUM magnitude factors (`|b|`, `1/|b|`, `|a|/|b|²`), a `div` node's bound holding on the nonzero-denominator side condition `Regular` (vacuous on ÷-free DAGs). Sorry-free (`#print axioms → [propext, Classical.choice, Quot.sound]`), instantiated on concrete +/−/×/÷ DAGs in the `AdequacyDag` example (the executable-carrier bridge is built, Stage 3.3).

Theorem 19.6.7.

group uses 0 used by 0 ✓

Tags

proved

Flag-freedom is a condition on the exact evaluation. *The flag-free hypothesis of A3′ is phrased about the floating-point intermediates, so settling it means evaluating in binary32 first. Asking instead that every node's* ***exact*** *value be a binary32 number is a question about the ℝ model and its inputs — and it cuts out the same inputs.*

Lean code for Theorem 19.6.7.

1 theorem

theorem · defined in `PropertyKindCalculus/Uncertainty/Adequacy/DagBound.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.flagFree_iff_exactRepresentab
  (ρ : ℕ → TorchLean.Floats.FP32)
  (e : PropertyKindCalculus.Uncertainty.Adequacy.Expr) :
```

```
PropertyKindCalculus.Uncertainty.Adequacy.FlagFree e ρ ↔
  PropertyKindCalculus.Uncertainty.Adequacy.ExactRepresentable e ρ
```

Proof for Theorem 19.6.7

uses 0

Realized as `ExactRepresentable` and `flagFree_iff_exactRepresentable` (Stage 3.8, `Adequacy.DagBound`). Both directions of the induction need the operand condition first, which is what identifies the two readings of a node's operands; the node step is then `Fp32Grounding.round32_eq_self_iff`, that `round32` fixes exactly the representable reals. That characterization is also what makes the relocated condition **minimal** for this conclusion: nothing weaker on a node's exact value keeps that node from rounding.

The consequence worth stating is that the flag-free theorems are now about an inhabited regime. `AdequacyDag` discharges `ExactRepresentable` on a doubling chain at two input assignments and derives the box equality with no hypothesis left standing — where `mix_box_exact` and `prod_box_exact` assume flag-freedom and nothing supplies it.

What is **not** done is the weaker regime the residual list still carries: a no-absorption condition would **allow** rounding and ask only that a tracked contribution survive it, which makes equality of the variations false and so requires a different conclusion — a resolution statement lifting A1's converse `resolve` along the DAG, as `dag_fp32_error_bound` lifts the half-ulp bound.

Theorem 19.6.8.

group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

Which bridge carries which half. *Two routes connect the executable rung to the specification: the [algebraic one](), whose content is one equation per operator, and the per-operation error bounds, whose content is one bound per operator. They are not alternatives. The forward-error accumulation is metric and rests on the bounds; the exactness regime is algebraic and is a statement about the refinement's rounding — which this theorem proves by showing the two vocabularies pick out the same evaluations.*

Lean code for Theorem 19.6.8.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Uncertainty/Adequacy/RefinementBridge.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.refinementFixes_iff_exactRepre
  (ρ : ℕ → TorchLean.Floats.FP32)
  (e : PropertyKindCalculus.Uncertainty.Adequacy.Expr) :
```

```
PropertyKindCalculus.Uncertainty.Adequacy.RefinementFixes e ρ ↔
  PropertyKindCalculus.Uncertainty.Adequacy.ExactRepresentable e ρ
```

Proof for Theorem 19.6.8 proof">uses 1

Realized in `Adequacy.RefinementBridge`. The identification is definitional — the refinement rounds by the format's own rounding — so the induction does nothing but apply, at each node, that rounding fixes exactly the representable reals.

The reason to keep both routes rather than consolidate is that their side conditions are about different things. The division law is unconditional at the specification rung only because `\mathbb{R}` totalizes division by zero, which is why the executable rung instead carries the divisor's nonzero decoded mantissa — a question of **definedness**. The adequacy DAG's regularity condition is unrelated to that: the per-operation bound for division has no hypothesis at all, and regularity guards the **propagation** factors — a question of **magnitude**. Merging them would conflate the two, and the weighted mean is where that would cost something: at its single division node the two rungs' licenses are independent in both directions.

Theorem 19.6.9. group statement">used by 2 ✓

Tags

capstone

proved

Executable ↔ spec bridge (Stage 3.3). *The verdicts above are stated over the `noncomputable` ℝ/FP32 specification, but the **runtime** carrier must compute a verdict. Lean's host `Float` is binary64 (and, on the pinned toolchain, a structure over core's `Float.Model` — `examples.tape_host_float_model`), not the float32 this verdict is about, so the certified check is **stated** on TorchLean's **computable** `IEEE32Exec` model (and the runtime `Float32` carrier is then proved to compute exactly this verdict — the next node). There the ULP is genuinely computed from a bit pattern (`ulpExp?`, matching the spec ulp_{32} whenever it answers — `none` on NaNs and infinities), and the kernel's absorption test `absorbs s δ` — the float32 sum is unchanged — **certifies** the specification: the exact real sum rounds back under `\mathrm{round}_{32}`. So the computed adequacy verdict is provably the specified A3 one, on the finite fragment. The residual `Float32 ↔ IEEE32Exec` step is a FloatLib theorem over Lean's logical model of `Float32`, for finite operands; what stays trusted is that the compiled runtime primitives agree with that logical model — the irreducible hardware trust boundary.*

Lean code for Theorem 19.6.9. 1 theorem

theorem · defined in `PropertyKindCalculus/Uncertainty/Adequacy/ExecBridge.lean` complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy.exec_verdict_sound
  {s δ :
    FloatLib.Floats.ExecFloat.Binary 8 23
      FloatLib.Floats.Formats.BinaryInterchange.FloatFormat.Encoding.ieee

(FloatLib.Floats.Formats.BinaryInterchange.FloatFormat.Encoding.ieee.defaultBias
       8)
      PropertyKindCalculus.Uncertainty.Adequacy.exec_ulp_grounds._proof_1
      PropertyKindCalculus.Uncertainty.Adequacy.exec_ulp_grounds._proof_2
      PropertyKindCalculus.Uncertainty.Adequacy.exec_ulp_grounds._proof_3
      PropertyKindCalculus.Uncertainty.Adequacy.exec_ulp_grounds._proof_4}
  (hfin :
    FloatLib.Floats.ExecFloat.Binary.isFinite
        (FloatLib.Floats.ExecFloat.add s δ) =
      true)
  (hverdict : TorchLean.Floats.IEEE754.IEEE32Exec.absorbs s δ = true) :
  PropertyKindCalculus.Fp32.round32
      (s.toModel.toReal + δ.toModel.toReal) =
    s.toModel.toReal
```

Proof for Theorem 19.6.9 uses 0

Realized over ℝ/`IEEE32Exec` (Stage 3.3, `Adequacy.ExecBridge`), grounded in TorchLean's `IEEE32Exec.ulpExp?` + `bpow_eq_ulp_of_ulpExp?_eq_some` (an executable ULP answer equals ulp_{32}; the query answers on exactly the finite fragment) and `round_add_eq_left_of_absorbs` (a 3-line corollary of the `toReal_add_eq_round_of_isFinite` op-level refinement, since FloatLib's `Model.roundAt binary32` **is** round_{32}). `exec_verdict_sound` composes them: when `absorbs s δ` fires, $\mathrm{round}_{32}(\mathrm{toReal}\,s + \mathrm{toReal}\,\delta) = \mathrm{toReal}\,s$. Sorry-free (`[propext, Classical.choice, Quot.sound]`); the executable side is **run** (`#guard`) at `2^{25}` and `10^8` in the `AdequacyExecBridge` example, which also confirms the axiom profile. The binary64 `Adequacy` carrier stays the illustrative mirror (its binary32 ulp is **emulated**); the runtime carrier that is certified is `Adequacy32`, below.

Theorem 19.6.10. **group** **uses 2** used by 0 ✓

Tags

capstone

proved

The runtime adequacy carrier is certified (Stage 3.3, closed from the runtime side). *`Adequacy32` is the write-once adequacy carrier with `Float32` magnitudes and uncertainties, and its swamping test is `absorbs`-shaped: the at-risk uncertainty `δ`, added to the binary32 sum `v`, leaves it bit-for-bit unchanged (`absorbed v δ`). When that flag fires on finite operands, the exact real sum rounds back under `\mathrm{round}_{32}`: `\mathrm{round}_{32}(v + δ) = v`. So a verdict a model run **emits** — not a*

*verdict on a model of the runtime — is the specified A3 one, through the executable ↔ spec bridge. No `log2/exp2` (the `opaque` externs) appears; what stays trusted is exactly what FloatLib names: the compiled `lean_float32_*` primitives agree with core's `Float32.Model`. The certificate is exactly the flagged-+ direction of A3: a raised flag means an absorbed uncertainty. It does not certify a **clean** report, and the cancellation flag at − remains the decidable rule backed by the ℝ-level Sterbenz theorems, not by a runtime theorem.*

***Uncertified vs. certified, made visible.** The binary64 `Adequacy` carrier's verdict is a claim about a **picture**: a binary64 sum compared with an emulated binary32 spacing under the rule of thumb `u < ½\, \mathrm{ulp}_{32}`; nothing relates that sum to `\mathrm{round}_{32}`, so the claim is illustrative. `Adequacy32`'s verdict is a claim about the arithmetic in question, and the theorem above says what it means. The two part ways exactly where a rule of thumb and IEEE rounding differ — at a **tie**: an uncertainty of exactly half a ulp is not below the threshold, so the emulation reports `1 + (0 ± 2^{-24})` adequate, while genuine binary32 rounds the tie to even, absorbs it, and the certified carrier flags it (and does not on the odd-mantissa neighbour `1 + 2^{-23}`). The `AdequacyCertified` example runs both carriers on the same kernel and `#guard`s the disagreement, and instantiates `add_flag_sound` at the tie: `\mathrm{round}_{32}(1 + 2^{-24}) = 1`.*

Lean code for Theorem 19.6.10.

1 theorem

theorem · defined in `PropertyKindCalculus/Uncertainty/Adequacy32.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.Adequacy32.add_flag_sound
  {v δ : Float32} (hv : v.isFinite = true) (hδ : δ.isFinite = true)
  (h :
    PropertyKindCalculus.Uncertainty.Adequacy32.absorbed v δ = true) :
  PropertyKindCalculus.Fp32.round32
      ((FloatLib.Floats.ExecFloat.Binary.ofFloat32 v).toModel.toReal +
        (FloatLib.Floats.ExecFloat.Binary.ofFloat32 δ).toModel.toReal) =
    (FloatLib.Floats.ExecFloat.Binary.ofFloat32 v).toModel.toReal
```

Proof for Theorem 19.6.10

uses 0

Three theorems composed, each from a library the repo already depends on. Core 4.35 makes `Float32` a structure over `Float32.Model`, so the runtime bit equality is `Float32` equality (`Float32.toBits_inj`, Batteries). FloatLib's native bridge carries a finite `Float32` sum to the `ExecFloat.Binary 8 23` sum (`ExecFloat.Binary.ofFloat32_add_of_isFinite`), so `absorbed v δ` **is** TorchLean's `absorbs (ofFloat32 v) (ofFloat32 δ)` (`Adequacy32.absorbs_of_absorbed`), and finiteness transfers (`isFinite_ofFloat32`, via `Model.isFinite_toModel`). `exec_verdict_sound` then reads the absorbed model sum as `\mathrm{round}_{32}` of the exact real sum. `add_absorptions` ties the flag to

the report the carrier's + emits. Sorry-free, [propext, Classical.choice, Quot.sound] guarded; the flag is **run** (#guard) at 1 + 2^{-25} (absorbed) and 1 + 2^{-23} (not).

Definition 19.6.11. group uses 2 used by 2 ✓

The Axis-U significance yardstick (Stage 3.4). Uncertainty propagation and numerical adequacy are driven from **one** InputDist descriptor, and the GUM budget carries its metrological intent in the **types**: an input standard uncertainty is a \mathrm{Quantity}\ k_i, a sensitivity c_i = \partial f/\partial x_i a quotient \mathrm{Quantity}\ (k_o/k_i), and their product — the **contribution** u_i(y) = |c_i| \,u(x_i), the scale at which absorption becomes genuine information loss — a \mathrm{Quantity}\ k_o, formed by a term gated on (k_o/k_i)\cdot k_i = k_o (the GUM units cancellation, a typechecking obligation). analyzeQ instantiates one **carrier-raw** write-once kernel twice from that single descriptor list — at the autograd carrier for the c_i, and at the adequacy carrier seeded from the same descriptor — and returns a kind-typed budget, so a UQ-analyzed model is **simultaneously** adequacy-checked at the scale its own uncertainty defines, with the roles kept distinct by kind.

Lean code for Definition 19.6.11. 1 definition

def · defined in PropertyKindCalculus/Uncertainty/Adequacy/Significance.lean — complete

```
def PropertyKindCalculus.Uncertainty.Adequacy.analyzeQ
  {k0 kI kS : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.ProductKind kS kI k0)
  (model : PropertyKindCalculus.Uncertainty.Adequacy.ListModel)
  (inputs : List (PropertyKindCalculus.Uncertainty.InputDist Float)) :
  Except String
    (PropertyKindCalculus.Uncertainty.Adequacy.CouplingResultQ k0)
```

Proof for Definition 19.6.11 uses 2

Realized (Stage 3.4, Adequacy.Significance.analyzeQ over Uncertainty.Budget; analyzeQ32 is the certified route — binary32 descriptors, the same budget on their exact widening, analyzeQ32_budget, and the verdict at Adequacy32). The three budget roles get kinds (stdUncQ, sensitivityQ, contributionQ gated by ProductKind kₛ kᵢ kₒ, combinedQ); the model stays carrier-raw, so no kinded autograd is needed — the kinds are the caller-declared overlay on the budget. On the Degenhardt descriptors the coupling reproduces (c_i) = (5, 5, 2.25), the kinded contributions (c_i u_i) = (1.0, 1.30, 0.28) at \mathrm{ measurandY}, and their quadrature reproduces the GUM combined uncertainty u_c = 1.662359 **exactly**; #check_failure probes make the conflations (sensitivity ↔ uncertainty, heterogeneous quadrature, field swap) type errors. A 10^8-accumulator swamps a contribution (< \tfrac12\,\mathrm{ulp}_{32}(10^8) = 4); the flag's soundness is the A3 verdict, and contribution_absorbed_at_scale applies it at the contribution scale, sorry-free. (ProductKind.ofRatio is liberal, so the gate enforces the GUM shape and forbids un-sanctioned mixing

while the author asserts `k_s = k_o/k_i`.) The budget is carrier-generic (`Uncertainty.Budget` over `[NumCarrier R]`) — the **second** axis, orthogonal to the kind: the same `combinedQ` runs at `ℝ` (to prove laws), at `Float` (to run), and at the adequacy carrier, where its quadrature `\sqrt{\sum u_i^2}` adequacy-checks **itself** (the example flags a `10^{-3}` contribution swamped beside a `10^8` one).

Definition 19.6.12. group statement">statement">used by ✓

The kinded ×/÷ budget DAG (residual 2). The significance wiring covers the **homogeneous-input** case — all inputs at one shared kind, one flat quadrature. A genuinely heterogeneous multiply/divide model, each interior node with its own kind equation, needs the kind gates **threaded through the expression**. `BudgetExpr R k` is a kinded ×/÷ expression whose every `mul`/`div` node **carries its `\mathrm{ProductKind}`/`\mathrm{QuotientKind}` witness as a constructor field**, so an ill-kinded expression is unwritable; and every interpretation — the plain value `valueQ`, the node-local GUM propagation `propagateQ`, the flattened per-leaf `contribsQ` — is built exclusively by the kind-gated `\mathrm{Quantity.mul}`/`\mathrm{Quantity.div}`, never a naked carrier op re-stamped. The per-node GUM content is the product/quotient rows of GUM Table §4.4: at a product the two quadrature terms are `u_a|b|` and `|a|u_b`, at a quotient `u_a/|b|` and `(|y|u_b)/|b|`, the latter written through the transposed witness `k = k_1/k_2 \Rightarrow k\cdot k_2 = k_1` so the intermediate `|y|u_b` is itself a kinded product, not unkinded scratch.

Lean code for Definition 19.6.12. 1 definition

inductive (3 constructors, 2 parameters) · defined in `PropertyKindCalculus/Uncertainty/BudgetDag.lean` — complete

```
inductive PropertyKindCalculus.Uncertainty.BudgetExpr (R : Type) :
  PropertyKindCalculus.KindOfProperty → Type
```

Proof for Definition 19.6.12 proof">uses 1

Realized (Stage 3.4 extension, residual 2; `Uncertainty.BudgetDag`, Mathlib/Torch-free, carrier-generic over `[NumCarrier R]`). The transposed witness is `QuotientKind.toProductKind` (`QuantityClassification`); the value component rides along unperturbed (`propagateQ_value_eq_valueQ`). Deliberately no `add`/`sub` nodes — the homogeneous linear case is already the flat kinded budget fed by autograd sensitivities, so tree +/− would be a second authoring of the same quadrature. Exercised on the block-density model `\rho = m/((l\cdot w)\cdot h)` (`UncertaintyExamples.BudgetDagDensity`): five distinct kinds through one budget — untypeable under the significance wiring's single shared input kind — with `#guards` reproducing `\rho = 2500`, the per-leaf contributions `(1.0, 2.5, 5.0, 25.0)`, and `u_c = \sqrt{657.25} \approx 25.637`, and `#check_failure` probes making the operand swap, the heterogeneous quadrature, and the kind-misplaced leaf type errors.

Theorem 19.6.13. group uses 2 used by 0 ✓

Tags

capstone

proved

> ***The threaded propagation collapses to the flat quadrature.*** *Over $\mathbb{R}$, the uncertainty the budget DAG accumulates node-by-node — a `\sqrt{\ }` at every node, re-squared by the parent — telescopes **exactly**, not up to a bound, to the flat `combinedQ` quadrature over the flattened per-leaf-occurrence contributions. Relative to the flat single-quadrature baseline the DAG machine therefore adds **threading only**, never a second combination rule: it is a conservative generalization of the kinded budget, not a second authority over the same GUM content.*

Lean code for Theorem 19.6.13.

1 theorem

theorem · defined in `PropertyKindCalculus/Uncertainty/BudgetDagLaws.lean`

complete

```
theorem PropertyKindCalculus.Uncertainty.BudgetExpr.propagateQ_unc_eq_combinedQ
  {k : PropertyKindCalculus.KindOfProperty}
  (e : PropertyKindCalculus.Uncertainty.BudgetExpr ℝ k)
  (hne : e.NonnegUncs) :
  e.propagateQ.unc =
    PropertyKindCalculus.Uncertainty.combinedQ e.contribsQ
```

Proof for Theorem 19.6.13

uses 0

Realized over $\mathbb{R}$ (`Uncertainty.BudgetExpr.propagateQ_unc_eq_combinedQ`, in `BudgetDagLaws`), under a `NonnegUncs` hypothesis (nonnegativity of every leaf uncertainty — statable only at the ordered proof carrier, the core `NumCarrier` being deliberately branchless). Structural induction: `Quantity.ext` reduces each node to a magnitude identity in the `sumSq` sum-of-squares vocabulary (`sumSq_append`, the scaled families `sumSq_map_mul_left/_right/_div`), each child radicand re-expressed as `\sqrt{S}\cdot \sqrt{S}` (`Real.mul_self_sqrt`) and closed by `ring`. No divisor-nonzero hypotheses — division is $\mathbb{R}$'s totalized `x/0 = 0` and both sides degenerate identically. Sorry-free (`#print axioms → [propext, Classical.choice, Quot.sound]`, pinned in `BudgetDagDensity`). Two textbook relative-uncertainty corollaries (`mul_leaf_relative_quadrature`, `div_leaf_relative_quadrature`) give `(u_y/|y|)^2 = (u_a/|a|)^2 + (u_b/|b|)^2` for single nodes, there needing the nonzero-value hypotheses relative uncertainty divides by.

19.7. Coverage intervals (R18)

Once an output's uncertainty is a distribution, VIM 2.36–2.38 ask for a *coverage interval* — a probability attached to an interval about the mean. R18 discharges this in two tiers, graded by how much is assumed about the distribution's shape. Both are stated over $\mathbb{R}$ and depend only on the `variance` the descriptor already carries; neither needs a *true value* (the sense in which coverage is provable while *accuracy* — closeness to nature — is not). Deliberately out of scope: exact *normal* coverage (the 95% $\leftrightarrow$ `k \approx 1.96` statement), which needs the Gaussian CDF / error function Mathlib does not provide, and any bridge to the executable `Float` sampler (the R15 exec$\leftrightarrow$spec axis).

Definition 19.7.1. group uses 0 statement">used by ✓

The **standard uncertainty** `u = \sqrt{\mathrm{variance}}` (VIM 2.30) — the scale in which a coverage interval is measured. It is the square root of the `\kappa_2` the moment descriptor records.

Lean code for Definition 19.7.1. 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Coverage.lean` — complete

```
def PropertyKindCalculus.Uncertainty.Coverage.stdUnc.{u_1} {Ω : Type u_1}
  [MeasurableSpace Ω] (X : Ω → ℝ) (μ : MeasureTheory.Measure Ω) : ℝ
```

Proof for Definition 19.7.1 uses 0

Realized: `Coverage.stdUnc`, `Real.sqrt` of `ProbabilityTheory.variance`.

Theorem 19.7.2. group statement">proof">used by 1 ✓

Tags: capstone, proved

> ***Tier 1 — the distribution-free coverage bound.*** *For **any** distribution with a positive variance, the interval of `k` standard uncertainties about the mean, `[m - k u,\, m + k u]`, has coverage probability at least `1 - 1/k^2`. This is the complement of Chebyshev's inequality and assumes nothing about the distribution's shape. It is honest but conservative: `k = 2` gives `\ge 75\%`, not the Gaussian `95\%` — the famous figure is a shape assumption, out of scope. Uses the standard uncertainty.*

Lean code for Theorem 19.7.2.

1 theorem

theorem · defined in PropertyKindCalculus/Uncertainty/Coverage.lean

complete

```
theorem PropertyKindCalculus.Uncertainty.Coverage.coverageBound_stdUnc.{u_1}
  {Ω : Type u_1} [MeasurableSpace Ω] {μ : MeasureTheory.Measure Ω}
  [MeasureTheory.IsProbabilityMeasure μ] {X : Ω → ℝ}
  (hXm : Measurable X) (hX : MeasureTheory.MemLp X 2 μ) {k : ℝ}
  (hk : 0 < k) (hv : 0 < ProbabilityTheory.variance X μ) :
  1 - ENNReal.ofReal (1 / k ^ 2) ≤
    μ
      {ω |
        |X ω - ∫ (x : Ω), X x ∂μ| <
          k * PropertyKindCalculus.Uncertainty.Coverage.stdUnc X μ}
```

Proof for Theorem 19.7.2 uses 0

Realized over ℝ: `coverageBound_stdUnc`, from Mathlib's `meas_ge_le_variance_div_sq` by taking the complement of the tail event (`prob_compl_eq_one_sub`) and substituting `c = k u`. Sorry-free (`[propext, Classical.choice, Quot.sound]`). The half-width form `coverageBound` (`\ge 1 - (u/c)^2` for half-width `c`) is the underlying lemma.

Theorem 19.7.3. group uses 0 used by 0 ✓

Tier 2 — exact coverage for a bounded family. *For a uniform distribution on `[m - δ, m + δ]`, the coverage of the centred interval `[m - h, m + h]` (with `0 \le h \le δ`) is* ***exactly*** *`h/δ` — closed-form, from the distribution's definition, and tighter than the Chebyshev bound. A concrete witness (the identity on ℝ under `volume` conditioned to the support) shows the hypothesis is non-vacuous; the `Coverage` worked example checks the central half of a uniform on `[-2, 2]` has coverage exactly `1/2`.*

Lean code for Theorem 19.7.3.

1 theorem

theorem · defined in PropertyKindCalculus/Uncertainty/Coverage.lean

complete

```
theorem PropertyKindCalculus.Uncertainty.Coverage.uniform_coverage_exact.{u_1}
  {Ω : Type u_1} [MeasurableSpace Ω] {P : MeasureTheory.Measure Ω}
  {X : Ω → ℝ} {m δ : ℝ} (hδ : 0 < δ)
  (hu :
    MeasureTheory.pdf.IsUniform X (Set.Icc (m - δ) (m + δ)) P
      MeasureTheory.volume)
```

```
{h : ℝ} (_hh0 : 0 ≤ h) (hhδ : h ≤ δ) :
P (X ⁻¹' Set.Icc (m - h) (m + h)) = ENNReal.ofReal (h / δ)
```

Proof for Theorem 19.7.3 uses 0

Realized over `ℝ`: `uniform_coverage_exact`, from Mathlib's `pdf.IsUniform.measure_preimage` and `Real.volume_Icc` (the intersection with the support is the sub-interval, and the ratio of Lebesgue lengths is `2h/2δ = h/δ`). Sorry-free (`[propext, Classical.choice, Quot.sound]`).

19.8. Conformity assessment (ISO/IEC Guide 98-4, JCGM 106:2012)

A coverage interval says how far the measurand may lie from its estimate. It does not say what to **do** with that at a limit, and that gap is where guard bands come from. The decision rule "accept iff the estimate is below the limit" ignores the uncertainty entirely and runs a consumer's risk of essentially 50 % at the limit; every practitioner therefore applies a margin, and the margin is almost always a **chosen** factor rather than a derived one. A chosen factor cannot be checked, cannot be shown to be too tight or too loose, and cannot shrink as evidence accumulates, so it charges the first measurement's ignorance forever. ISO/IEC Guide 98-4 (JCGM 106:2012) is the standard that closes the gap, and `Conformity.lean` is its executable face — Mathlib-free and `Float`-carried like the `Evidence` module it builds on — with `ConformityLadder.lean` supplying, over `ℝ`, the argument that a margin derived this way may safely tighten itself. No requirement of the R1–R28 catalogue claims this layer; it sits beside R18 as what a coverage factor is **for** once a limit is in view.

JCGM 106's vocabulary, as the module carries it. The **tolerance limit** `T` (§3.3.4) is the specified bound on permissible values of the measurand — given, not computed. The **acceptance limit** `A` (§3.3.8) is the bound the decision actually compares against. The **guard band** `g = |T - A|` (§3.3.10), applied inward, is guarded acceptance: the acceptance interval sits inside the tolerance interval and the consumer's risk falls; applied outward it is guarded rejection. The **specific consumer's risk** `R_C(y)` (§9.5.2) is, for a measured value `y` with standard uncertainty `u`, the probability that the measurand is past `T` anyway; its counterpart, the specific producer's risk, is the probability that a value rejected at `A` would in fact have conformed. Nothing here is a naked number: the limits are `Quantity k` values of the measurand, a probability and a coverage factor ride distinct dimension-one kinds, so reading `1.96` as `0.95` or comparing an estimate against a limit of another kind is a type error rather than a review finding.

Definition 19.8.1. group uses 0 statement">used by

Tolerance limits as roles. A `Tolerance k R` is an optional `LowerBound k R` and an optional `UpperBound k R` — the directional roles of `Bounds.lean`, not bare endpoints — so each limit can only ever be used from its own side, and an absent bound is `none` rather than an infinity. `Tolerance.admits` asks whether a **value** conforms (the question a measurement cannot answer, which is why the rest of the module exists), and `Tolerance.guardedBy` derives the acceptance limits: it subtracts a band from an upper limit and adds it to a lower one. That direction is forced by the role, not chosen by convention, so "applied

the guard band the wrong way and turned guarded acceptance into guarded rejection" is not a mistake a caller can make while still typechecking.

Lean code for Definition **19.8.1.** 1 definition

`structure` (2 fields) · defined in `PropertyKindCalculus/Uncertainty/Conformity.lean` complete

```
structure PropertyKindCalculus.Uncertainty.Conformity.Tolerance
  (k : PropertyKindCalculus.KindOfProperty) (R : Type) : Type
```

Proof for Definition 19.8.1 uses 0

Realized: `Conformity.Tolerance` with `atMost`/`atLeast`/`ofIcc` constructors (scale-gated by `OrderKind`, since a limit presupposes an order to be on one side of), `Tolerance.admits` through each endpoint's own directional decider, and `Tolerance.guardedBy` gated by `DifferenceKind`, the metrological licence to form the difference at all.

Definition **19.8.2.** group statement">used 1 used by 0 ✓

The guard band `g = k\,u`, a quantity of the measurand's own kind. It is built through `Quantity.mul` on `expansionLaw`, so the kind of the product is derived from a stated law — a coverage factor is dimension one **in the role of expanding an uncertainty** — rather than asserted. The input is a `Dispersion` and the output is not: a guard band **is** an expanded uncertainty, but what it is used as is a displacement of a limit, and the change of role happens once, here. `accepts` is then the decision — accept iff the estimate lies within the guarded acceptance limits — with the estimate and its uncertainty at their roles, so the two cannot be exchanged into an inverted rule that guards enormously on a small uncertainty and not at all on a large one. `assess` packages the whole thing: an evidence record, a tolerance and a target risk in; the factor, the band, the acceptance limits, the decision and the risk actually run out — the shape that makes the guard band an **output**.

Lean code for Definition **19.8.2.** 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Conformity.lean` complete

```
def PropertyKindCalculus.Uncertainty.Conformity.guardBand
  {k : PropertyKindCalculus.KindOfProperty}
  (u : PropertyKindCalculus.Uncertainty.Dispersion k Float)
  (factor :
    PropertyKindCalculus.Quantity
```

```
    PropertyKindCalculus.Uncertainty.coverageFactor Float)
  (hk : k.IsRational := by rfl) : PropertyKindCalculus.Quantity k Float
```

Proof for Definition 19.8.2

uses 0

Realized: `Conformity.guardBand` as `Dispersion.expanded` unwrapped to a `Quantity`, `Conformity.accepts` over `Tolerance.guardedBy`, `consumerRisk`/`producerRisk`/`riskAtLimit` as the risks actually run, and `Conformity.assess` returning `none` — a refusal, not a factor nothing justifies — when the evidence cannot support a coverage factor at the target risk.

Definition 19.8.3.

group uses 0 used by 2 ✓

The band from a stated risk. `factorForRisk` inverts §9.5.2: to hold the specific consumer's risk at a one-sided acceptance limit to `p`, the factor is `k = \Phi^{-1}(1 - p)` — `none` unless `0 < p < 1`, since a risk of zero demands an infinite band and a risk of one is not a decision rule. The factor stops being an input and becomes a consequence of a risk the operator names. `riskForFactor` is its exact inverse, `1 - \Phi(k)`, and prices a band somebody else chose. Both are Gaussian, which is exact only when `u` is **known**; `factorForEvidence` and `riskForFactorAt` are the same pair at the evidence's own `\nu` (a Student `t` that **is** the Gaussian at `\nu = \infty`, so the correction appears and disappears by itself), and `factorForRiskRectangular`/`riskForFactorRectangular` are the closed-form pair for GUM 4.3.7's bounded posterior, which saturates at `\sqrt{3}\,u` where the risk is exactly zero.

Lean code for Definition 19.8.3.

1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Conformity.lean` — complete

```
def PropertyKindCalculus.Uncertainty.Conformity.factorForRisk
  (p :
    PropertyKindCalculus.Quantity
      PropertyKindCalculus.Uncertainty.probability Float) :
  Option
    (PropertyKindCalculus.Quantity
      PropertyKindCalculus.Uncertainty.coverageFactor Float)
```

Proof for Definition 19.8.3

uses 0

Realized: `Conformity.factorForRisk` and `riskForFactor` through `Sampling.normalQuantile` and `normalCDF`; `riskForFactorAt` through `Sampling.studentTCDF`, returning `none` when `\nu` is `NaN` or below one (fewer than two indications support no statement about a tail — the same refusal `Evidence.coverageFactorOneSided` makes, not a Gaussian fallback); and the rectangular pair around `rectangularFactorLimit = \sqrt{3}`. The direction in which a small `\nu` is dangerous is the pricing

one: at `\nu = 4` a band of `4.5\,u` buys `5.4 \times 10^{-3}`, not the `3.4 \times 10^{-6}` a Gaussian reads off it.

Definition 19.8.4. group statement">uses 1 statement">used by 0 ✓

The risk from the costs, when nobody knows what risk to ask for. Accepting a non-conforming value costs `C_a`; rejecting a conforming one costs `C_r`; at a measured value the expected costs are `R_C\,C_a` and `(1 - R_C)\,C_r`, so accepting is the cheaper action exactly while `R_C \le C_r/(C_a + C_r)`, and that break-even risk is the target to hand to `factorForRisk`. The two costs must be quantities of one kind, and the step from their ratio to a probability is an authored crossing — nothing about a ratio of two durations makes it a probability except the decision-theoretic argument. Strongly asymmetric costs then **produce** the conclusion that is usually asserted: a wrong acceptance a hundred times as costly as a wrong rejection gives `p^* \approx 1\,\%` and a one-sided `2.33\,u`, a derivation auditable in a way "we used 0.9" is not.

Lean code for Definition 19.8.4. 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Conformity.lean` complete

```
def PropertyKindCalculus.Uncertainty.Conformity.riskFromCosts
  {kc : PropertyKindCalculus.KindOfProperty}
  (costOfWrongAccept costOfWrongReject :
    PropertyKindCalculus.Quantity kc Float)
  (hc : kc.IsRational := by rfl) :
  Option
    (PropertyKindCalculus.Quantity
      PropertyKindCalculus.Uncertainty.probability Float)
```

Proof for Definition 19.8.4 uses 0

Realized: `Conformity.riskFromCosts`, dividing through `costRatioIsRisk kc hc`, `none` unless both costs are positive numbers.

Definition 19.8.5. group statement">uses 1 statement">used by 1 ✓

Reading a band that already exists. `readBand g u` divides a deployed margin by the measurand's own standard uncertainty — a quotient of two `k`-quantities, which `bandReadingLaw` declares to be a coverage factor — and says what kind of thing the band is. A `BandReading` is one of three constructors. `coverage factor risk`: a band at a few `u` is a coverage statement, dispersion that shrinks as evidence accumulates, and the risk it buys is attached. `systematic factor`: a band at tens of `u` is not — no risk target produces it under any posterior, so the margin is standing in for a **systematic the model does not carry**, which does not shrink with evidence, and tightening it as though it did is how a fleet walks into a correlated failure. `unstated`: the band cannot be read at all. The two readable cases have

opposite repairs, which is why the distinction is a constructor and not a comment; `systematicThreshold` (`6`, where the Gaussian risk reaches `10^{-9}`) is the judgement they turn on, taken as an argument so that it applies uniformly. `Evidence.readBand e g` is the same reading taken against a record's own evidence, priced at its `\nu` through `riskForFactorAt` instead of at the Gaussian limit, and declared in `Evidence`'s namespace so the `\nu` cannot be dropped at the call site. `BandReading.label` forgets the evidence to a `BandReadingLabel` — the three-valued classification a report line or a record field states.

Lean code for Definition **19.8.5.** 1 definition

inductive (3 constructors) · defined in `PropertyKindCalculus/Uncertainty/Conformity.lean` complete

```
inductive PropertyKindCalculus.Uncertainty.Conformity.BandReading : Type
```

Proof for Definition 19.8.5 proof">uses 1

Realized: `Conformity.BandReading`, `BandReadingLabel`, `BandReading.label`, `systematicThreshold`, `readBand` (pricing with `riskForFactor`) and `Evidence.readBand` (pricing with `riskForFactorAt` at `e.dof`), both one-line calls into `classifyBand`. The band is a plain `Quantity` and the `u` a `Dispersion`, which is what stops `g/u` from being transposed into `u/g` — both well-kinded coverage factors, and the transposition turns a `42\,u` systematic into `1/42` of a `u` that every downstream rule would happily shrink.

Definition **19.8.6.** group statement">used by 2 ✓

The one band-classification rule, and the NaN rule inside it. `classifyBand g u hk systematicAt price` is the single place a band is classified: the guard (`u` must be a positive number), the quotient `g/u` through `bandReadingLaw`, the comparison against the systematic threshold, and the pricing of a coverage statement through the `price` function the caller supplies — `readBand` passes the Gaussian `riskForFactor`, `Evidence.readBand` passes `riskForFactorAt` at the record's own `\nu`. Nothing else differs between the two entry points of the band reading.

The NaN rule. A coverage factor that is not a number — `\infty/\infty`, an unbounded band over an unbounded `u` — reads as `unstated`, and so does any band read against a systematic threshold that is not a number. The rule cannot be left to the comparison: `NaN ≤ t` and `NaN > t` are **both** false, so a rule written "`\le t` ⇒ coverage, else systematic" and the same rule written "`> t` ⇒ systematic, else coverage" classify the same band in opposite directions — one as a systematic that must never be tightened, the other as a coverage statement every downstream rule may shrink. Neither is right. `unstated` is chosen over the conservative `systematic` because a systematic is a positive claim — a term the model does not carry — that a factor nobody can price does not support; and over the permissive `coverage` because that is the reading whose repair, tightening, is the dangerous one. A factor nobody can price is not a reading, and a threshold nobody stated classifies nothing. Because the guard lives in `classifyBand` and both entry points are one-line calls to it, no caller can re-derive the wrong direction; and for every factor and threshold that **are** numbers the rule is unchanged — the

guard falls through to exactly the comparison it always made, so `\infty/1` is still a systematic and the threshold itself is still a coverage statement.

Lean code for Definition **19.8.6.** 1 definition

`def` · defined in `PropertyKindCalculus/Uncertainty/Conformity.lean` complete

```
def PropertyKindCalculus.Uncertainty.Conformity.classifyBand
  {k : PropertyKindCalculus.KindOfProperty}
  (g : PropertyKindCalculus.Quantity k Float)
  (u : PropertyKindCalculus.Uncertainty.Dispersion k Float)
  (hk : k.IsRational)
  (systematicAt :
    PropertyKindCalculus.Quantity
      PropertyKindCalculus.Uncertainty.coverageFactor Float)
  (price :
    PropertyKindCalculus.Quantity
        PropertyKindCalculus.Uncertainty.coverageFactor Float →
      Option
        (PropertyKindCalculus.Quantity
          PropertyKindCalculus.Uncertainty.probability Float)) :
  PropertyKindCalculus.Uncertainty.Conformity.BandReading
```

Proof for Definition 19.8.6 uses 0

Realized: `Conformity.classifyBand`, with the `isNaN` guards on the threshold and on the quotient ahead of the `>` comparison, and `none` from `price` (a `\nu` nothing qualifies) also read as `unstated`. The checked facts are the probes of `PropertyKindCalculus.Tests.Uncertainty.Evidence`, each a `#guard` that fails the build if a reading moves: `\infty/\infty` reads `unstated` through `readBand` **and** through `Evidence.readBand` at `\nu = 4` — the case where a second copy of the rule had disagreed; `41\,u` under a `NaN` threshold reads `unstated` through both; `\infty/1` reads `systematic`; `4.5\,u` and `4.39\,u` read `coverage` (at `3.4 \times 10^{-6}` under the Gaussian, `5.4 \times 10^{-3}` at `\nu = 4`) and `41\,u` reads `systematic` through both; the threshold `6\,u` itself reads `coverage` at the Gaussian limit and at `\nu = 4`; and a `u` of zero, or a `u` qualified by `\nu = 0.5`, reads `unstated`.

Theorem **19.8.7.** group statement"> used by 0 ✓

Tags

proved

Why an acceptance limit may be allowed to tighten itself. *Deriving the band from a stated risk and pooling evidence so that `u` falls gives a deployment that gets* ***less*** *conservative the longer it runs —*

> *exactly the automatic loosening that should not be allowed near anything that can be killed for using too much memory. Over $\mathbb{R}$, with the guarded upper acceptance limit $A(u) = L - k\,u$: the limit rises as u falls and never past L, so the mechanism is a ratchet with a ceiling; and — the whole content — the specific consumer's risk **at** the acceptance limit is $Q(k)$ for every $u \neq 0$, independent of u, because band and uncertainty shrink together and the standardized deviate they are read at is exactly k. A rule built for a target risk therefore runs that target at every stage of evidence, the cold start included: with one Type B prior the loop is safe and merely inefficient. What is not claimed: that the risk is unchanged by evidence is a statement about the rule, not about the world — if the pooled u is wrong, a systematic reading is the one thing the loop must never tighten, and a floor on u is the operational guard the theorem leaves to the deployment.*

Lean code for Theorem **19.8.7.** 1 theorem

theorem · defined in `PropertyKindCalculus/Uncertainty/ConformityLadder.lean` — complete

```
theorem PropertyKindCalculus.Uncertainty.ConformityLadder.riskAt_acceptanceLimit
  (Q : ℝ → ℝ) (L k : ℝ) {u : ℝ} (hu : u ≠ 0) :
  PropertyKindCalculus.Uncertainty.ConformityLadder.riskAt Q L
      (PropertyKindCalculus.Uncertainty.ConformityLadder.acceptanceLimit
        L k u)
      u =
    Q k
```

Proof for Theorem 19.8.7 uses 0

Realized over $\mathbb{R}$ in `ConformityLadder`: `acceptanceLimit_antitone` and `acceptanceLimit_le` (the ratchet and its ceiling), `riskAt_acceptanceLimit` (the invariance, by the algebra `(L - A(u))/u = k`, with the tail `Q` an arbitrary function), `riskAt_acceptanceLimit_le` (the target survives every update of `u`), `riskAt_le_of_accepted` and `riskAt_acceptanceLimit_antitone_in_factor` (needing only `Antitone Q`, which the Gaussian, `t` and rectangular tails all satisfy), and `acceptanceLimit_le_of_floor` (a floor on `u` is a floor on the limit).

19.9. Worked examples (checked facts)

Thirteen examples reproduce a headline number (or a theorem) as a `#guard`, so the `UncertaintyExamples` library building under CI is what makes the claims true rather than asserted — the project's reflection-tests discipline applied to metrology. The first two are the Stage-0 reference numbers; the next two are the Stage-1 autograd and ladder facts; the next two are the Stage-2 SSPRC pipeline and its ladder theorems; the last seven are the Stage-3 adequacy carrier, its per-site theorems, the

Stage-3.1 DAG capstone A3′, the Stage-3.2 binary32 Sterbenz, the Stage-3.3 executable ↔ spec bridge, the Stage-3.4 Axis-U significance coupling, and the Stage-3.6 direct-simulation closure.

- `PropertyKindCalculus.UncertaintyExamples.DegenhardtFictive` — the non-linear fictive model `Y = (X_1 + X_2^2)\,X_3` of Degenhardt (Degenhardt *et al.*, 2025)[24] Johannes Degenhardt, Rainer Tutsch, Xiukun Hu, and Gaoliang Dai (2025). "A practically oriented, efficient alternative to the Monte Carlo method for measurement uncertainty estimation" (https://iopscience.iop.org/article/10.1088/1681-7575/adb3ab). *Metrologia.* **62**(2). §3.1, with a normal, a uniform, and a triangular input. The Monte Carlo reference gives `E(Y) \approx 11.60`, `u(Y) \approx 1.70` (matching the paper's `11.5875`, `1.691`); the reference value `R = f(E(X)) = 11.25`, and the gap `E(Y) - R = 0.34` is the non-linearity signature of the `X_2^2` term — exactly what the linearized rungs cannot see. The GUM cross-check gives `u_c \approx 1.662`, visibly **under** the Monte Carlo value: the linearization drops the curvature, which is the whole motivation for the higher rungs.

- `PropertyKindCalculus.UncertaintyExamples.WillinkGaugeBlock` — the gauge-block calibration of Willink (Willink, 2005)[25] Robin Willink (2005). "A procedure for the evaluation of measurement uncertainty based on moments" (https://iopscience.iop.org/article/10.1088/0026-1394/42/5/001). *Metrologia.* **42**(5). Table 4 (the GUM example H.1), evaluated by the cumulants method: combining the six error variables' variances and fourth cumulants gives `u_Y = 33.4\,\mathrm{nm}`, excess `\gamma_Y = 0.124`, and a 99% expanded-uncertainty half-width `h_{0.99} = 87.6\,\mathrm{nm}` — 5% narrower than the GUM procedure's 93 nm at the same coverage, because the (mostly negative) fourth cumulants pull the tails of the output toward normal. That narrowing **is** the difference between the GUM rung and the Willink rung, reproduced as a checked number.

- `PropertyKindCalculus.UncertaintyExamples.DegenhardtSensitivity` — the Stage-1 autograd bridge on the **same** fictive kernel: interpreting it at the reverse-mode tape carrier returns the sensitivity coefficients `(c_1, c_2, c_3) = (5, 5, 2.25)` to machine precision (matching the hand-computed `\partial Y/\partial X_i` at the means), and feeding them through `gumStdUnc` reproduces `u_c = 1.662` exactly — the write-once model differentiated with no rewrite.

- `PropertyKindCalculus.UncertaintyExamples.LadderNesting` — T1/T2 applied to a concrete all-Gaussian term list over $\mathbb{R}$ (`gum = willink|κ₄=0` as a closed proof term, `#print axioms` confirming no `sorryAx`), together with the executable `Float` shadow of the collapse: for normal-only inputs the Willink 95% half-width is exactly `1.96\,u_c`.

- `PropertyKindCalculus.UncertaintyExamples.DegenhardtSsprc` — the Stage-2 SSPRC pipeline on the **same** fictive model (Degenhardt *et al.*, 2025)[26] Johannes Degenhardt, Rainer Tutsch, Xiukun Hu, and Gaoliang Dai (2025). "A practically oriented, efficient alternative to the Monte Carlo method for measurement uncertainty estimation" (https://iopscience.iop.org/article/10.1088/1681-7575/adb3ab). *Metrologia.* **62**(2). : 100 systematic samples per input (300 model evaluations against the Monte Carlo reference's 20 000, a factor `\approx 67`) recover `E(Y) = 11.5875` — including the non-linear `+0.3375` offset the linearized rungs cannot produce — and `u(Y) \approx 1.69`, closer to the Monte Carlo value than GUM's 1.662; the actual convolution (`runConv`) reproduces the moment read-off to machine precision.

- `PropertyKindCalculus.UncertaintyExamples.SsprcNesting` — the T3/T4/T5 ladder theorems applied to two concrete centered distributions over $\mathbb{R}$: convolution adds their cumulants (T5), the convolved combined deviation has exactly the Willink cumulants `(41, -1186)` (T3), and an all-centered (affine) input list yields a centered combined deviation, so `E(Y) = R` (T4) — with `#print axioms` confirming no `sorryAx`.
- `PropertyKindCalculus.UncertaintyExamples.AdequacySwamping` — the executable `Adequacy` carrier on two write-once kernels: `bias + x` with a small input uncertainty is flagged **swamped** under a large accumulator (`10^8`, where `\tfrac12\,\mathrm{ulp}_{32} = 4 > 1`) and certifies **clean** under a small one (`100`); `a − b` of near-equal uncertain operands is flagged **catastrophic cancellation** (relative uncertainty ≥ 100%) and certifies clean when the operands are well separated — the same kernel, checked for free at each configuration.
- `PropertyKindCalculus.UncertaintyExamples.AdequacyLadder` — the adequacy theorems A1 (absorption, `\mathrm{round}_8(80+1) = 80`), A2 (Sterbenz, `3 - 2` of two `FLX 24` numbers is `FLX 24`), and A3 (the verdict biconditional) applied to concrete values over $\mathbb{R}$, with `#print axioms` confirming no `sorryAx`.
- `PropertyKindCalculus.UncertaintyExamples.AdequacyDag` — the Stage-3.1 capstone A3′ instantiated on concrete model DAGs (a three-input accumulator, an add/sub variant, a deeper five-input tree): the `FP32` measurand's variation over an input box equals the $\mathbb{R}$ one up to the DAG-additive rounding budget, and **exactly** when no site rounds (flag-free) — again with `#print axioms` confirming no `sorryAx`.
- `PropertyKindCalculus.UncertaintyExamples.AdequacySterbenz32` — the Stage-3.2 A2 lift to the genuine binary32 format: `\mathrm{round}_{32}(u - v) = u - v` and `(a - b).\mathrm{val} = a.\mathrm{val} - b.\mathrm{val}` for near-equal representable binary32 values (so the `sub32_within_half_ulp` bound collapses to zero), tied back to the self-contained `FLX 24` fact — with `#print axioms` confirming no `sorryAx`.
- `PropertyKindCalculus.UncertaintyExamples.AdequacyExecBridge` — the Stage-3.3 executable↔spec bridge: the ULP query `ulpExp?` and absorption test `absorbs` **run** on concrete float32 bit patterns (`#guard`: at `2^{25}` the ULP is `4`, so a perturbation of `1` is absorbed and `4` is not; at `10^8` the ULP is `8`, so `1` is absorbed and `8` is not), while `exec_verdict` **proves** that whenever the kernel reports absorption the exact real sum rounds back under `\mathrm{round}_{32}` — the computed verdict equal to the specified one, with `#print axioms` confirming no `sorryAx`.
- `PropertyKindCalculus.UncertaintyExamples.AdequacyCoupling` — the Stage-3.4 kind-typed coupling: one `analyzeQ` on the Degenhardt descriptors yields kinded contributions `(c_i u_i) = (1.0, 1.30, 0.28)` at `\mathrm{measurandY}` whose quadrature reproduces the GUM combined uncertainty `u_c = 1.662359` **exactly**, and certifies the model adequate at that scale; `#check_failure` probes make the metrological conflations (sensitivity↔uncertainty, heterogeneous quadrature, `CouplingResultQ` field swap) type errors; and a `10^8`-accumulator swamps a contribution (one absorption), with `contribution_absorbed_at_scale` applying the A3 verdict at that scale, `#print axioms` confirming no `sorryAx`.

- `PropertyKindCalculus.UncertaintyExamples.AutogradDirectSim` — the Stage-3.6 closure instantiated concretely: the vectorization homomorphisms at a 3-vector shape; the compiled product graph `x_0 \cdot x_1` with `direct_PR_soundness_compiled` (the runtime dense reverse pass succeeds and its input-prefix realises the adjoint of the Fréchet derivative) and its inhabited forward simulation; an eager two-leaf/`mul` runtime tape whose shape-total-closure hypothesis is discharged constructor by constructor; and, via the eager-provenance addendum, `EagerBuilds` witnessing that runtime tape with `direct_PR_soundness_eager` giving the endpoint on it — no compilation involved — with `#print axioms` pinning the classical trio only.

20. ISO/IEC 80000 — The Standards Catalogue

The quantity-kinds and units specified here are grounded against the international standard series ISO/IEC 80000, *Quantities and units*. That series is **licensed and copyrighted**: no normative content from it is reproduced in this work. What is recorded is the *citable identity* of each part — body, number, part, edition, and publication date — so that every catalogued quantity-kind or unit can name its exact source. This chapter specifies that references layer; the chapters that follow specify one part's quantity-kinds and units at a time, each added as coverage of that part becomes available.

20.1. The references layer (citation metadata only)

Definition 20.1.1. group uses 0 state

A *standard reference* `StandardRef` is the bibliographic identity of one part: its publishing body (ISO or IEC), the series number (`80000`), the part number, the edition designation as printed, the publication date, and the part's English title. It carries **no** normative content — only what is needed to *cite* the source. Its `cite` renders the conventional form, e.g. `IEC 80000-6, Edition 2.0, 2022-11`.

Lean code for Definition 20.1.1. 1 definition

`structure` (6 fields) · defined in `PropertyKindCalculus/Iso80000/References.lean` complete

```
structure PropertyKindCalculus.Iso80000.StandardRef : Type
```

Proof for Definition 20.1.1 uses 0

Realized as `structure StandardRef` (Mathlib-free) with a `StandardBody` (`ISO`/`IEC`) field and the printed edition, date, and title as strings, plus `designation` (`ISO 80000-3`) and `cite` (the full reference). The values are read off each licensed part's cover page; nothing normative is transcribed.

Definition 20.1.2. group statement">us

The *catalogue* is the list of the thirteen catalogued parts in part order, each a standard reference. It is the single place this work names *which editions* of ISO/IEC 80000 the catalogued quantity-kinds and units are grounded against, so the grounding is auditable rather than implicit.

Lean code for Definition 20.1.2. 1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/References.lean` complete

```
def PropertyKindCalculus.Iso80000.catalogue :
  List PropertyKindCalculus.Iso80000.StandardRef
```

Proof for Definition 20.1.2

uses 0

Realized as `catalogue : List StandardRef`, the thirteen definitions `iso80000_1` … `iec80000_13` (with `iec80000_6` and `iec80000_13` the IEC-published parts). The edition is "Second edition" for every part except those two, both "Edition 2.0"; the dates are the cover dates of the licensed copies.

21. ISO 80000-3 — Space and Time

The first part whose quantity-kinds and units are specified in full is ISO 80000-3, *Space and time*. Every item — all of 3-1.1 … 3-26.3 — is catalogued: length, the width/height/distance/radius family, area, volume, the angles, duration, the frequencies, velocity and speed, acceleration, the wave quantities, and the rest, each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only **citation locators** are recorded; no normative content (definitions, remarks) from the licensed standard is reproduced. The defining *mathematics* of selected remarks is formalized in its own right, in the sibling modules linked below.

Part 3 turns out to be an unusually good proving ground for the calculus, because the standard itself is full of distinctions that a dimension-only or representation-only model cannot make: a dozen different length quantities, several dimensionless angles, and three families of quantities that share a dimension while remaining distinct kinds.

21.1. Quantity-kinds and units of ISO 80000-3

Definition 21.1.1. group statement"> uses 1 used by 0 ✓

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "3-3"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream rather than kept in a comment.

Lean code for Definition 21.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 21.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-3, Second edition, 2019-10 item 3-3`. The `catalogue` lists all 42 items in item order — length (3-1.1), area (3-3), volume (3-4), duration (3-9), speed (3-10.2), frequency (3-17.1), … — each paired with its coherent SI unit symbol.

Theorem 21.1.2. group statement"> uses 1 used by 0 ✓

Effort

small

Tags

proved

Area is L^2 (item 3-3). *The catalogued area kind carries length-exponent two in PhysLib's dimension group — the standard's dimensional statement reproduced as a checked computation, not an annotation. Uses [the dimension map](#).*

Lean code for Theorem 21.1.2.

1 theorem

theorem · defined in `PropertyKindCalculus/Iso80000/Part3.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.area_dim_length :
  (PropertyKindCalculus.Iso80000.Part3.area.dim
       LTMCTDimensionBase).length =
    2
```

Proof for Theorem 21.1.2

uses 0

Proved as `area_dim_length : area.dim.length = 2` in the `Iso80000` library, discharged by `Dim.area_length` from the `Dimension` layer. The companion `volume_dim_length` (item 3-4) gives length-exponent three.

Theorem 21.1.3.

group statement">uses 1 used by 0 ✓

Effort

small

Tags

proved

Speed is $L \cdot T^{-1}$ (item 3-10.2). *The catalogued speed kind's dimension equals length divided by time — a checked computation in the dimension group. Uses [the dimension map](#).*

Lean code for Theorem 21.1.3.

1 theorem

theorem · defined in `PropertyKindCalculus/Iso80000/Part3.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.speed_dim :
  PropertyKindCalculus.Iso80000.Part3.speed.dim LTMCTDimensionBase =
    PropertyKindCalculus.Dim.length / PropertyKindCalculus.Dim.time
```

Proof for Theorem 21.1.3

uses 0

Proved as `speed_dim : speed.dim = Dim.length / Dim.time`, discharged by `Dim.speed_eq`. The catalogue also records `curvature_dim_length = -1` (item 3-2, L^{-1}), `frequency_dim_time = -1` (item 3-17.1, T^{-1}), and `acceleration_dim_time = -2` (item 3-11, $L \cdot T^{-2}$).

Theorem 21.1.4.

group statement">used 1 used by 0 ✓

Effort	Tags
small	proved

The metre is a well-formed unit (item 3-1.1). *Length is a ratio-scale kind, so it bears a unit (Dybkær §13.3.5); the metre is one such chosen reference.*

Lean code for Theorem 21.1.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.metre_wellFormed :
  PropertyKindCalculus.Iso80000.Part3.metre.WellFormed
```

Proof for Theorem 21.1.4

uses 0

Proved as `metre_wellFormed : metre.WellFormed` via `KindOfProperty.rational_bears_unit`.

Theorem 21.1.5.

group statement">used 1 used by 0 ✓

Effort	Tags
small	proved

A metre and a second are not commensurable. *Length and duration are distinct kinds, so their units cannot be compared by ratio — a type-level fact, not a runtime check. Uses metrological units.*

Lean code for Theorem **21.1.5.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.metre_second_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part3.metre.Commensurable
      PropertyKindCalculus.Iso80000.Part3.second
```

Proof for Theorem 21.1.5 uses 0

> Proved as `metre_second_not_commensurable : ¬ metre.Commensurable second` by unfolding `Commensurable` to the underlying kinds and `decide` (length's kind `id` differs from duration's).

21.2. The length family: distinguishing kinds by measurement principle

ISO 80000-3 lists width (3-1.2), height (3-1.3), thickness (3-1.4), diameter (3-1.5), radius (3-1.6), path length (3-1.7), distance (3-1.8), and more as *separate items* — yet every one of them has dimension L and ratio scale. The standard distinguishes them only in *prose*: a width is a transverse extent, a distance is a shortest path, and so on. A dimension-only or representation-only model cannot capture the difference at all; it sees one type, "a real number of metres", for all of them.

This is exactly requirement *R2*, and Part 3 is where it earns its keep on the real standard. Item 3-1.1, *length*, is the broad genus. Each later item is specified as a *species* of length, distinguished **not by fiat** (not merely by a different name) but by an explicit *measurement principle* — Dybkær's examination principle (§7.5), a genuine defining aspect. The species are then provably distinct kinds, while remaining *mutually comparable* as lengths; and the specialization edges mirror the standard's own "defined in terms of" links (a diameter is a width of a circle, a radius is half a diameter, a distance is a shortest path length).

Definition **21.2.1.** group uses 2 used by 0 ✓

> A *length species* is a ratio-scale quantity-kind of dimension L carrying an examination principle — this work's own terse descriptor of the measurement principle that distinguishes it (transverse extent, shortest path, centre-to-rim, …), never the standard's normative definition. Width, height, thickness, diameter, radius, path length, distance, radial distance, the position and displacement vectors, the radius of curvature, and the wavelength are all species of the general length kind (item 3-1.1).

Lean code for Definition 21.2.1.

1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
def PropertyKindCalculus.Iso80000.Part3.lengthSpecies (id : String)
  (p : PropertyKindCalculus.ExaminationPrinciple) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase
```

Proof for Definition 21.2.1

uses 0

`lengthSpecies id p := { kind := { id, scale := .ratio, examPrinciple := some p.id }, dim := Dim.length }`. The general `length` carries no examination principle; the direct-parent edges of the family are collected in an `Edge` relation mirroring the standard's definitional links (thickness and diameter via width, radius via diameter, distance via path length, …).

Theorem 21.2.2.

group statement"> uses 1 by 0 ✓

Effort	Tags
small	proved

> ***Width and distance are distinct kinds — by measurement principle, not by fiat.*** *A width (transverse extent) and a distance (shortest path) are different kinds because they are examined by different principles, proved through `distinct_of_examPrinciple` (§7.5) rather than by appealing to their `id` strings. Same dimension `L`, same scale, different kind. Uses the examination principle.*

Lean code for Theorem 21.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.width_ne_distance :
  PropertyKindCalculus.Iso80000.Part3.width.kind LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part3.distance.kind LTMCTDimensionBase
```

Proof for Theorem 21.2.2

uses 0

`width_ne_distance : width.kind ≠ distance.kind`, from `KindOfProperty.distinct_of_examPrinciple` applied to the differing examination principles `transverse-extent` and `shortest-path` (a `decide` on the `Option String` links). The companion `width_examinedBy` checks the kind-to-principle link. Axiom-free.

Theorem 21.2.3. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***A radius specializes length — transitively.*** *Through the chain radius ⊑ diameter ⊑ width ⊑ length, a radius is a length: specialization is the reflexive-transitive closure of the family's edges, a genuine preorder (a lattice, not just direct edges). Uses specialization.*

Lean code for Theorem 21.2.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.radius_specializes_length :
  PropertyKindCalculus.Specializes
    PropertyKindCalculus.Iso80000.Part3.Edge
    (PropertyKindCalculus.Iso80000.Part3.radius.kind LTMCTDimensionBase)
    (PropertyKindCalculus.Iso80000.Part3.length.kind LTMCTDimensionBase)
```

Proof for Theorem 21.2.3

uses 0

> `radius_specializes_length`, built from `Specializes.of_edge` on `radius_diameter`, `diameter_width`, `width_length` composed by `Specializes.trans`. Axiom-free.

Theorem 21.2.4. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***Comparability is preserved.*** *Width and distance, though distinct kinds, remain mutually comparable — they share the super-kind length, so combining them is possible but only via an explicit up-cast, never silently. Uses specialization.*

Lean code for Theorem 21.2.4.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.width_distance_comparable :
  PropertyKindCalculus.MutuallyComparable
```

```
    PropertyKindCalculus.Iso80000.Part3.Edge
    (PropertyKindCalculus.Iso80000.Part3.width.kind LTMCTDimensionBase)
    (PropertyKindCalculus.Iso80000.Part3.distance.kind
      LTMCTDimensionBase)
```

Proof for Theorem 21.2.4 uses 0

> `width_distance_comparable : MutuallyComparable Edge width.kind distance.kind`, with witness `length.kind`, `width` reaching it by one edge and `distance` by two (distance ⊏ path length ⊏ length). Axiom-free.

21.3. Dimension does not classify; the kind does

Part 3 alone is dense with *dimension collisions* — distinct kinds that the dimension functor cannot tell apart. Plane angle, rotational displacement, phase angle, solid angle, rotation, and the logarithmic decrement are *all* dimension one. Frequency, rotational frequency, angular frequency, angular velocity, and the damping coefficient are *all* T^{-1}. Velocity and speed are both $\mathsf{L}\cdot\mathsf{T}^{-1}$. The dimension is the same; the kind is not — and so the units differ too, a radian is not a steradian and a hertz is not a radian-per-second, though each pair shares a dimension.

Theorem 21.3.1. group statement"> uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***The dimension-1 disambiguation, on the standard.*** *There exist distinct ISO 80000-3 kinds with the same dimension one — plane angle and solid angle witness it. No dimension-only type system can separate them; the kind layer does. This generalizes the calculus's `dim_not_injective` capstone to the published standard. Uses the dimension map.*

Lean code for Theorem 21.3.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.iso80000_3_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 21.3.1

uses 0

`iso80000_3_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨planeAngle, solidAngle, …⟩` with `planeAngle_ne_solidAngle` (a `decide`) and reflexivity. Axiom-free.

Theorem 21.3.2.

group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

A radian is not a steradian, though both are dimension one. *The SI units of plane and solid angle are not commensurable, because their kinds differ — a type-level fact the dimension cannot see. This is exactly the separation the angle-reform literature seeks to install in the dimension layer — Leonard (Leonard, 2021)*[27] B. P. Leonard (2021). "Proposal for the dimensionally consistent treatment of angle and solid angle by the International System of Units (SI)" (https://iopscience.iop.org/article/10.1088/1681-7575/abe0fc). *Metrologia.* **58**(5). *would give angle its own dimension and make solid angle its square. PKC secures it at the kind layer — so it holds whether or not the SI ever assigns angle a dimension — and, having made PhysLib's `Dimension` parametric in its basis, now realizes the reform itself in an angle-augmented basis where plane angle is a base dimension and solid angle its square. The hertz and the radian-per-second are separated the same way (both T^{-1}). Uses metrological units.*

Lean code for Theorem 21.3.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.radian_steradian_not_commensurable
  ¬PropertyKindCalculus.Iso80000.Part3.radian.Commensurable
      PropertyKindCalculus.Iso80000.Part3.steradian
```

Proof for Theorem 21.3.2

uses 0

`radian_steradian_not_commensurable : ¬ radian.Commensurable steradian` by unfolding to the underlying kinds and `decide`; `hertz_radianPerSecond_not_commensurable` is the T^{-1} companion.

Theorem 21.3.3.

group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***Dimension is not a classifier even away from dimension one.*** *Distinct ISO 80000-3 kinds also share dimensionful dimensions — frequency and angular frequency are both T^{-1}, velocity and speed both $L \cdot T^{-1}$. Uses the dimension map.*

Lean code for Theorem **21.3.3.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.iso80000_3_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 21.3.3 uses 0

> `iso80000_3_dim_collision`, witnessed by `⟨frequency, angularFrequency, …⟩` with `frequency_ne_angularFrequency` (a `decide`) and reflexivity; `velocity_ne_speed` is the $L \cdot T^{-1}$ companion. Axiom-free.

21.4. Standard prefixes: the centimetre as *centi · metre*

A unit is rarely used alone — a length is reported in millimetres, kilometres, or centimetres, all *the same kind* as the metre but differing by a decimal factor. The VIM4 records this directly: §1.19 lists the SI prefixes (a factor 10^n with a name and a symbol), §1.20 and §1.21 define a *multiple* and a *submultiple* of a unit, and §1.22 the *conversion factor* between two units of the same kind. So the centimetre is not an independent unit named `"cm"`: it is the *centi* submultiple of the metre, and its symbol and its 10^{-2} factor should *follow* from that, not be asserted by hand.

Definition **21.4.1.** group statement">us

A *prefixed unit* is a decimal (SI, §1.19) or binary (IEC 80000-13) prefix applied to a base unit. It projects to a metrological unit of the *same kind* with the prefix symbol prepended (`"c" ++ "m" = "cm"`), and exposes its §1.22 conversion factor as the exponent and its §1.20/§1.21 status as a multiple or submultiple.

Lean code for Definition **21.4.1.** 1 definition

`structure (4 fields)` · defined in `PropertyKindCalculus/UnitPrefix.lean` — complete

```
structure PropertyKindCalculus.PrefixedUnit : Type
```

Proof for Definition 21.4.1 uses 0

Realized as `structure SIPrefix` (`name`, `symbol`, `exponent : Int`) with the full §1.19 table (and a sibling `BinaryPrefix` for IEC 80000-13), and `structure PrefixedUnit` (`radix`, `exponent`, `symbol`, `base`) with `toUnit` (the projection), `conversionExponent`, `IsMultiple`, and `IsSubmultiple`. Recording the resolved `radix` lets both prefix families share one prefixed unit; the projection preserves the kind, so `toUnit_wellFormed` and `commensurable_base` follow by reflexivity.

Theorem 21.4.2. group statement">us

Effort	Tags
small	proved

The centimetre is the centi submultiple of the metre, factor 10^{-2}. *Its symbol `"cm"` and its conversion exponent `-2` are checked computations from the prefix and the base, and it stays commensurable with the metre. Uses metrological units.*

Lean code for Theorem 21.4.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.centimetre_conversionExponent :
  PropertyKindCalculus.Iso80000.Part3.centimetrePrefixed.conversionExponent =
    -2
```

Proof for Theorem 21.4.2 uses 0

`centimetre` is `(metre.withPrefix SIPrefix.centi).toUnit`. The facts `centimetre_symbol = "cm"`, `centimetre_conversionExponent = -2`, `centimetre_isSubmultiple`, and `centimetre_wellFormed` are discharged by `rfl` / `decide`; `metre_centimetre_commensurable` holds because the prefix leaves the kind unchanged.

21.5. Capturing a Remark's mathematics: the surface element of an area (item 3-3)

Each ISO 80000 item carries a *Remarks* field, and those remarks often hold the quantity's defining mathematics. Item 3-3 (area) is the example: its remark gives the *surface element* of a surface in terms of the Gaussian coordinates `u`, `v` and the determinant `g` of the metric tensor (ISO 80000-2). Rather than

transcribe the licensed text or reduce it to a dimensional annotation, this work *formalizes the relation* — as real differential-geometric objects with proved properties — so the mathematics behind the kind is itself checked.

Definition 21.5.1. group uses 0 used by 0 ✓

The *first fundamental form* (the metric tensor of ISO 80000-2) at a point is the 2×2 Gram matrix ⟪t_i, t_j⟫ of the surface's two tangent vectors. It is symmetric and positive-semidefinite, so its determinant `g` is non-negative.

Lean code for Definition 21.5.1.

1 definition

def · defined in PropertyKindCalculus/Iso80000/Part3/AreaElement.lean

complete

```
def PropertyKindCalculus.Iso80000.Part3.AreaElement.firstFundamentalForm.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 2 → E) : Matrix (Fin 2) (Fin 2) ℝ
```

Proof for Definition 21.5.1 uses 0

Realized as `firstFundamentalForm t := Matrix.gram ℝ t` over Mathlib's Gram-matrix library; symmetry is `isHermitian_gram` and `metricDet_nonneg` is the determinant of a positive-semidefinite matrix.

Definition 21.5.2. group statement">uses 1 used by 0 ✓

The *surface element* `dA = √g` is the area-element density at a point, and the *area* of a surface patch is its integral over the parameter region (`A = ∬ √g du dv`). Uses the dimension map for the `L²` reading of the resulting kind.

Lean code for Definition 21.5.2.

1 definition

def · defined in PropertyKindCalculus/Iso80000/Part3/AreaElement.lean

complete

```
def PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 2 → E) : ℝ
```

Proof for Definition 21.5.2 uses 0

`areaElement t := Real.sqrt (metricDet t)` and `surfaceArea t s := ∫ x in s, areaElement (t x)`. `surfaceArea_const` shows a uniformly-parametrized patch has area `element × (area of the parameter region)`.

Theorem 21.5.3. group uses 0 used by 0 ✓

Effort	Tags
small	proved

The element squares back to the determinant: `(dA)² = g`. *The √ in `dA = √g` is faithful — the element is exactly the square root of the determinant of the metric tensor.*

Lean code for Theorem 21.5.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3/AreaElement.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement_sq.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 2 → E) :
  PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement t ^ 2 =
    PropertyKindCalculus.Iso80000.Part3.AreaElement.metricDet t
```

Proof for Theorem 21.5.3 uses 0

`areaElement_sq : areaElement t ^ 2 = metricDet t`, from `Real.sq_sqrt` and the non-negativity of `g`.

Theorem 21.5.4. group uses 0 used by 0 ✓

Effort	Tags
medium	proved

A flat patch recovers its Euclidean area. *For a surface lying in the plane the element equals `|det A|`, the change-of-variables area density of the parametrization — so the √g reading computes the actual area. This is the correctness anchor that pins the convention the prose remark leaves implicit.*

Lean code for Theorem 21.5.4. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3/AreaElement.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement_eq_abs_det
  (t : Fin 2 → EuclideanSpace ℝ (Fin 2)) :
  PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement t =
    |(Matrix.of fun i j => (t j).ofLp i).det|
```

Proof for Theorem 21.5.4 uses 0

`areaElement_eq_abs_det : areaElement t = |(Matrix.of fun i j => t j i).det|`, via `gram_eq_conjTranspose_mul` (so `g = (det A)²`) and `Real.sqrt_sq_eq_abs`.

Theorem 21.5.5. group uses 0 used by 0 ✓

Effort	Tags
small	proved

> ***Regularity.*** *The element is strictly positive exactly when the two tangent vectors are linearly independent — a non-degenerate (regular) point. A vanishing element marks a degenerate parametrization.*

Lean code for Theorem 21.5.5. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3/AreaElement.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement_pos_iff.
{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 2 → E) :
  0 < PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement t ↔
    LinearIndependent ℝ t
```

Proof for Theorem 21.5.5 uses 0

`areaElement_pos_iff : 0 < areaElement t ↔ LinearIndependent ℝ t`, from `Real.sqrt_pos` and `det_gram_ne_zero_iff_linearIndependent`.

21.6. The volume element of a region (item 3-4)

The volume item carries the analogous remark — the *volume element* `dV = √g` for the `3 × 3` metric tensor of a three-parameter region. It is the three-dimensional analogue of the area construction, the same Gram-determinant raised one rank, and it is formalized the same way.

Definition 21.6.1. group uses 0

The *volume element* `dV = √g` is the volume-element density of a three-parameter region, and the *volume* is its integral over the parameter region (`V = ∭ √g du dv dw`). It squares back to `g`, is positive exactly at regular points, and (the correctness anchor) equals `|det A|` for a flat region — so it integrates to the actual Euclidean volume.

Lean code for Definition 21.6.1. 1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part3/VolumeElement.lean` — complete

```
def PropertyKindCalculus.Iso80000.Part3.VolumeElement.volumeElement.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 3 → E) : ℝ
```

Proof for Definition 21.6.1 uses 0

`volumeElement t := Real.sqrt (metricDet t)` over the `3 × 3` `metricTensor t := Matrix.gram ℝ t` for `t : Fin 3 → E`; `regionVolume t s := ∫ x in s, volumeElement (t x)` with `regionVolume_const` the constant-frame case. The proofs mirror `AreaElement` at one higher rank.

Theorem 21.6.2. group uses 0

Effort	Tags
medium	proved

> ***A flat region recovers its Euclidean volume.*** *For a region in Euclidean 3-space the element equals `|det A|`, the change-of-variables volume density — the three-dimensional correctness anchor, confirming the `√g` reading against the volume it must reproduce.*

Lean code for Theorem 21.6.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3/VolumeElement.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part3.VolumeElement.volumeElement_eq_abs_d
  (t : Fin 3 → EuclideanSpace ℝ (Fin 3)) :
  PropertyKindCalculus.Iso80000.Part3.VolumeElement.volumeElement t =
    |(Matrix.of fun i j => (t j).ofLp i).det|
```

Proof for Theorem 21.6.2 uses 0

`volumeElement_eq_abs_det : volumeElement t = |(Matrix.of fun i j => t j i).det|` for `t : Fin 3 → EuclideanSpace ℝ (Fin 3)`, via `gram_eq_conjTranspose_mul` and `Real.sqrt_sq_eq_abs`. The companion `volumeElement_sq` gives `(dV)² = g`.

21.7. The algebraic Remarks as kind-laws

Many Part-3 remarks state a quantity's defining relation *algebraically* — as a quotient or a reciprocal of other quantities. Curvature is `1/ρ` (item 3-2); repetency is `1/λ` (3-20); frequency is `1/T` (3-17.1); speed is `ds/dt`, path length per duration (3-10.2); a plane angle is `s/r`, arc length per radius (3-5). Each is formalized as an R12 kind-law over the catalogued kinds, using the quotient and reciprocal families.

Theorem 21.7.1. group statement">us

Effort	Tags
small	proved

Under the SI's `α = s/r`, a plane angle computes to dimension one. *Taking arc and radius as plain lengths, the remark `α = s/r` cancels the dimension — a checked computation that reproduces the current SI convention, not one that settles it. That the relation forces dimensionlessness is exactly what the metrology reform disputes: Quincey, Mohr and Phillips (Quincey, Mohr, and Phillips, 2019)*[28] Paul Quincey, Peter J. Mohr, and William D. Phillips (2019). "Angles are inherently neither length ratios nor dimensionless" (https://iopscience.iop.org/article/10.1088/1681-7575/ab27d7). *Metrologia.* **56**(4). *argue an angle is inherently neither a length ratio nor dimensionless. PKC formalizes the standard as published and leaves the dimensional stance open — plane angle remains a distinct kind from every other dimension-one quantity (the collision capstone above), which is what keeps the radian and steradian apart whatever dimension the SI assigns. Uses the dimension map.*

Lean code for Theorem 21.7.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part3/DefiningRelations.lean` complete

```
theorem PropertyKindCalculus.Iso80000.Part3.DefiningRelations.planeAngle_dim_fr
  PropertyKindCalculus.Iso80000.Part3.planeAngle.dim
      LTMCTDimensionBase =
    PropertyKindCalculus.Iso80000.Part3.pathLength.dim
        LTMCTDimensionBase /
      PropertyKindCalculus.Iso80000.Part3.radius.dim LTMCTDimensionBase
```

Proof for Theorem 21.7.1

uses 0

`planeAngle_dim_from_arc_over_radius : planeAngle.dim = pathLength.dim / radius.dim`, by reducing both sides to `Dim.length / Dim.length` and the lemma `length_div_length` (`a / a = 1`, by `ext` and `simp`).

Theorem 21.7.2.

group statement">us

Effort	Tags
small	proved

> ***Verified construction at the quantity level.*** *A frequency built as the reciprocal of a period duration carries its classification certificate by construction, over the `ℝ` carrier — and canonicity makes that certificate determine the quantity. Uses [the reciprocal kind-law].*

Lean code for Theorem 21.7.2.

1 theorem

theorem · defined in `PropertyKindCalculus/Iso80000/Part3/DefiningRelations.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.DefiningRelations.frequencyOf_isReci
  (T :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part3.periodDuration.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsReciprocal

PropertyKindCalculus.Iso80000.Part3.DefiningRelations.frequency_recip_periodDu
ration
    (PropertyKindCalculus.Iso80000.Part3.DefiningRelations.frequencyOf
      T)
    T
```

Proof for Theorem 21.7.2

uses 0

`frequencyOf T := Quantity.recip frequency_recip_periodDuration T`, and `frequencyOf_isReciprocal` is `rfl`; `frequency_certificate_canonical` is `eq_recip_of_isReciprocal`. The kind-law `frequency_recip_periodDuration` is `⟨rfl, rfl⟩` (both kinds ratio-scale). The same families give `speedOf` (speed = path length / duration) and `planeAngleOf` (plane angle = arc / radius).

21.8. Why it matters: classification and metrological consistency at the quantity level

Formalizing a remark's mathematics is not decoration. It earns its keep by turning the quantity-kind into a working *classifier* and by moving metrological checking down to the *quantity level*, where the type system enforces it.

Classifying quantities. The surface-area formula does not return a bare real number: its output is, by construction, a magnitude of the *one* catalogued kind area. Any quantity a downstream model computes this way — the area of a field, a footprint, a cross-section — is automatically *classified* as the same kind, dimension L^2, unit the square metre. The kind is the genus under which every such quantity falls, and the defining relation is what places it there. So formalizing the remark gives users a principled answer to "what *is* this quantity?" — it is a value of area, because the relation that produced it is area's defining relation.

Verifying metrological consistency. Because the kind carries its dimension and its own scalar unit, the consistency checks happen at the quantity level and cannot be skipped. Two areas combine — addition is defined on the area carrier — but an area and a length are *different types*, so adding them does not type-check: the dimensionless numeric conflation a spreadsheet would silently allow is rejected. An area is not measurable in metres, because the square metre and the metre are not commensurable, a fact of the types rather than a runtime guard. And the dimensional invariant L^2 travels with *every* value of the kind. The remark's mathematics, the kind, and these consistency guarantees are thus one chain: the relation anchors the kind, the kind classifies the quantity, and the classification *is* the metrological check.

These claims are exercised as checked facts in the `DimensionExamples` library: a surface area built from the analytic element and classified as an area quantity, the L^2 invariant, the area-plus-area combination, and the rejected area-versus-length and square-metre-versus-metre conflations.

This is the **verified instantiation** requirement (R12): a classification is a certificate that a quantity satisfies its kind's defining relation, not a bare tag. For area, the defining relation is the surface element above, and it yields two concrete certificate forms.

Theorem 21.8.1. group uses 0

Effort	Tags
small	proved

> ***A rectangle's area is verified-classified by construction.*** `rectangleArea w h` *is built as the product of two lengths (area = length · length), so its area-classification certificate holds by construction.*

Lean code for Theorem 21.8.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3/AreaClassification.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaClassification.rectangleArea_isP
  (w h :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part3.length.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part3.AreaClassification.area_is_length_times_le
ngth
    (PropertyKindCalculus.Iso80000.Part3.AreaClassification.rectangleArea
      w h)
    w h
```

Proof for Theorem 21.8.1

uses 0

`rectangleArea w h := Quantity.mul area_is_length_times_length w h`, and `rectangleArea_isProduct` is `rfl` — the certificate of the constructed product.

Theorem 21.8.2.

group

uses 0

Effort	Tags
medium	proved

Property transport. *Every certified surface area is non-negative — the surface element is `≥ 0` (a property of area's defining relation), instantiated at the quantity level through the certificate.*

Lean code for Theorem 21.8.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3/AreaClassification.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaClassification.certified_surface
  {q :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part3.area.kind LTMCTDimensionBase)
      ℝ}
```

```
{t : ℝ × ℝ → Fin 2 → EuclideanSpace ℝ (Fin 2)} {s : Set (ℝ × ℝ)}
(hs : MeasurableSet s)
(h :
  PropertyKindCalculus.Iso80000.Part3.AreaClassification.IsSurfaceArea
    q t s) :
0 ≤ q.magnitude
```

Proof for Theorem 21.8.2 uses 0

> `certified_surfaceArea_nonneg` rewrites the certificate `q.magnitude = surfaceArea t s` and applies `surfaceArea_nonneg` (the integral of a non-negative element, by `setIntegral_nonneg` and `areaElement_nonneg`).

21.9. Item index — ISO 80000-3

Every catalogued item of ISO 80000-3, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in — the length-family lattice, a dimension collision, a formalized remark, or the catalogue itself. Symbols and unit strings are **citation locators**; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
3-1.1	length	l	m	L
3-1.2	width	b	m	L
3-1.3	height	h	m	L
3-1.4	thickness	d	m	L
3-1.5	diameter	d	m	L
3-1.6	radius	r	m	L
3-1.7	path length	s	m	L
3-1.8	distance	d	m	L
3-1.9	radial distance	r_Q	m	L
3-1.10	position vector	$\mathbf{r}$	m	L
3-1.11	displacement	$\Delta\mathbf{r}$	m	L
3-1.12	radius of curvature	ρ	m	L
3-2	curvature	κ	m^{-1}	L^{-1}
3-3	area	A	m^2	L^2
3-4	volume	V	m^3	L^3
3-5	plane angle	α	rad	1
3-6	rotational displacement	φ	rad	1
3-7	phase angle	φ	rad	1
3-8	solid angle	Ω	sr	1
3-9	duration	t	s	T
3-10.1	velocity	v	m/s	$L \cdot T^{-1}$
3-10.2	speed	v	m/s	$L \cdot T^{-1}$
3-11	acceleration	a	m/s^2	$L \cdot T^{-2}$
3-12	angular velocity	ω	rad/s	T^{-1}
3-13	angular acceleration	α	rad/s^2	T^{-2}
3-14	period duration	T	s	T
3-15	time constant	τ	s	T
3-16	rotation	N	1	1

Item	Quantity	Symbol	Unit	Dimension
3-17.1	frequency	f	Hz	T^{-1}
3-17.2	rotational frequency	n	s^{-1}	T^{-1}
3-18	angular frequency	ω	$\mathrm{rad/s}$	T^{-1}
3-19	wavelength	λ	m	L
3-20	repetency	σ	m^{-1}	L^{-1}
3-21	wave vector	k	m^{-1}	L^{-1}
3-22	angular repetency	k	m^{-1}	L^{-1}
3-23.1	phase velocity	c	$\mathrm{m/s}$	$\mathsf{L}\cdot\mathsf{T}^{-1}$
3-23.2	group velocity	c_g	$\mathrm{m/s}$	$\mathsf{L}\cdot\mathsf{T}^{-1}$
3-24	damping coefficient	δ	s^{-1}	T^{-1}
3-25	logarithmic decrement	Λ	1	1
3-26.1	attenuation	α	m^{-1}	L^{-1}
3-26.2	phase coefficient	β	$\mathrm{rad/m}$	L^{-1}
3-26.3	propagation coefficient	γ	m^{-1}	L^{-1}

22. ISO 80000-4 — Mechanics

The second part specified in full is ISO 80000-4, *Mechanics*. Every item — all of 4-1 … 4-32 — is catalogued: mass and the densities, momentum, the force family, impulse and the moments, pressure and stress, the strains and elastic moduli, the viscosities, power and the energies, the flows, and action — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only **citation locators** are recorded; no normative content (definitions, remarks) from the licensed standard is reproduced. The defining *mathematics* of selected remarks is formalized in the sibling module `Part4.DefiningRelations`.

Mechanics is where the two theses of the calculus bite hardest. The standard's own quantities force the issues: a force family that is a specialization lattice; the textbook torque-versus-energy dimension collision (both $\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$, the newton metre reserved for one and the joule for the other); and a web of defining relations that build mechanical quantities out of Part-3 *space-and-time* quantities — momentum from mass and velocity, pressure from force and area.

22.1. Quantity-kinds and units of ISO 80000-4

Definition 22.1.1. group statement">uses 1 · used by 0 ✓

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "4-9.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream rather than kept in a comment.

Lean code for Definition 22.1.1. 1 definition

structure (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` — complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 22.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-4, Second edition, 2019-08 item 4-9.1`. The `catalogue` lists all 54 items in item order — mass (4-1), momentum (4-8), force (4-9.1), torque (4-12.2), pressure (4-14.1), power (4-27), action (4-32), … — each paired with its coherent SI unit symbol.

Theorem 22.1.2. group statement">uses 1 · used by 0 ✓

Effort	Tags
small	proved

> ***Force is $M \cdot L \cdot T^{-2}$ (item 4-9.1).*** *The catalogued force kind carries time-exponent minus two in PhysLib's dimension group — the standard's dimensional statement reproduced as a checked computation, not an annotation. Uses the dimension map.*

Lean code for Theorem 22.1.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part4.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part4.force_dim_time :
  (PropertyKindCalculus.Iso80000.Part4.force.dim
       LTMCTDimensionBase).time =
    -2
```

Proof for Theorem 22.1.2

uses 0

Proved as `force_dim_time : force.dim.time = -2` in the `Iso80000` library, discharged by `norm_num` over the PhysLib dimension lemmas. The companions `energy_dim_length = 2` (item 4-28), `pressure_dim_length = -1` (item 4-14.1), and `power_dim_time = -3` (item 4-27) record the rest of the mechanical algebra.

Theorem 22.1.3.

group statement"> uses 1 by 0 ✓

Effort	Tags
small	proved

> ***A kilogram and a newton are not commensurable.*** *Mass and force are distinct kinds, so their units cannot be compared by ratio — a type-level fact, not a runtime check. Uses metrological units.*

Lean code for Theorem 22.1.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part4.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part4.kilogram_newton_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part4.kilogram.Commensurable
      PropertyKindCalculus.Iso80000.Part4.newton
```

Proof for Theorem 22.1.3 uses 0

> Proved as `kilogram_newton_not_commensurable : ¬ kilogram.Commensurable newton` by unfolding `Commensurable` to the underlying kinds and `decide`. The kilogram and the newton are each `WellFormed` (mass and force are ratio-scale).

22.2. The force family: distinguishing kinds by measurement principle

ISO 80000-4 lists weight (4-9.2), static friction force (4-9.3), kinetic friction force (4-9.4), rolling resistance (4-9.5), and drag force (4-9.6) as *separate items* — yet every one has dimension $M \cdot L \cdot T^{-2}$ and ratio scale. The standard distinguishes them only in *prose*: a weight acts in a gravitational field; a static friction force resists motion *before* a body slides; a kinetic friction force resists motion *while* it slides. A dimension-only or representation-only model cannot capture the difference; it sees one type, "a real number of newtons", for all of them.

This is requirement *R2* again, now on the force family — the exact pattern Part 3 set on the length family. Item 4-9.1, *force*, is the broad genus; each later item is specified as a *species* of force, distinguished **not by fiat** but by an explicit *measurement principle* (Dybkær's examination principle, §7.5). The species are then provably distinct kinds while remaining *mutually comparable* as forces.

Definition 22.2.1. group uses 2 used by 0 ✓

> A *force species* is a ratio-scale quantity-kind of dimension $M \cdot L \cdot T^{-2}$ carrying an examination principle — this work's own terse descriptor of the measurement principle that distinguishes it (gravitational-field action, pre-slip resistance, sliding resistance, …), never the standard's normative definition. Weight, the static and kinetic friction forces, the rolling resistance, and the drag force are all species of the general force kind (item 4-9.1).

Lean code for Definition 22.2.1.

1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part4.lean` — complete

```
def PropertyKindCalculus.Iso80000.Part4.forceSpecies (id : String)
  (p : PropertyKindCalculus.ExaminationPrinciple) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase
```

Proof for Definition 22.2.1

uses 0

`forceSpecies id p := { kind := { id, scale := .ratio, examPrinciple := some p.id }, dim := MDim.force }`. The general `force` carries no examination principle; the direct-parent edges of the family are collected in an `Edge` relation, each species an edge to `force` (mirroring the standard's "force (item 4-9.1) … acting/resisting …").

Theorem 22.2.2.

group statement"> uses 1 by 0 ✓

Effort	Tags
small	proved

Static and kinetic friction forces are distinct kinds — by measurement principle, not by fiat. *They share dimension $M·L·T^{-2}$ and are both "friction forces", yet are different kinds because they are examined under different conditions (resistance before sliding vs during sliding), proved through `distinct_of_examPrinciple` (§7.5). This is the force-family analogue of width ≠ distance. Uses [the examination principle].*

Lean code for Theorem 22.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part4.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part4.staticFriction_ne_kineticFriction :
  PropertyKindCalculus.Iso80000.Part4.staticFrictionForce.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part4.kineticFrictionForce.kind
      LTMCTDimensionBase
```

Proof for Theorem 22.2.2

uses 0

`staticFriction_ne_kineticFriction : staticFrictionForce.kind ≠ kineticFrictionForce.kind`, from `KindOfProperty.distinct_of_examPrinciple` applied to the differing principles `pre-slip-resistance` and `sliding-resistance` (a `decide` on the `Option String` links). The companions `weight_ne_dragForce` and `weight_ne_force` separate the other species, and `weight_examinedBy` checks the kind-to-principle link. Axiom-free.

Theorem 22.2.3. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

Comparability is preserved. *Weight and the drag force, though distinct kinds, remain mutually comparable — they share the super-kind force, so combining them is possible but only via an explicit up-cast, never silently. The companion `weight_specializes_force` records that a weight specializes force. Uses specialization.*

Lean code for Theorem 22.2.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part4.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part4.weight_dragForce_comparable :
  PropertyKindCalculus.MutuallyComparable
    PropertyKindCalculus.Iso80000.Part4.Edge
    (PropertyKindCalculus.Iso80000.Part4.weight.kind LTMCTDimensionBase)
    (PropertyKindCalculus.Iso80000.Part4.dragForce.kind
      LTMCTDimensionBase)
```

Proof for Theorem 22.2.3 uses 0

`weight_dragForce_comparable : MutuallyComparable Edge weight.kind dragForce.kind`, with witness `force.kind`, each species reaching it by one edge. Axiom-free.

22.3. Dimension does not classify; the kind does — the torque/energy case

Part 4 holds the textbook example of a dimension collision. *Torque* (moment of force, 4-12.1/4-12.2) and *energy* (work, 4-28) are *both* $\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$ — yet they are distinct kinds, measured in distinct units: the newton metre is reserved for torque, the joule for energy, precisely because they are different kinds. And Part 4 is dense with such collisions: momentum and impulse ($\mathsf{M}\cdot\mathsf{L}\cdot\mathsf{T}^{-1}$); angular momentum, angular impulse, and action ($\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-1}$); pressure, stress, and the three elastic moduli ($\mathsf{M}\cdot\mathsf{L}^{-1}\cdot\mathsf{T}^{-2}$); and a whole

dimension-one family — the relative densities, the strains, the Poisson number, the friction factors, the drag coefficient, and the efficiency.

Theorem 22.3.1. group statement"> uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***Torque is not energy, though both are $M·L^2·T^{-2}$.*** *There exist distinct ISO 80000-4 kinds with the same dimension — torque and energy witness it. No dimension-only type system can separate them; the kind layer does. This is the textbook case that a dimension does not classify a quantity. Uses the dimension map.*

Lean code for Theorem 22.3.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part4.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part4.iso80000_4_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 22.3.1 uses 0

> `iso80000_4_dim_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim`, witnessed by `⟨torque, mechanicalEnergy, …⟩` with `torque_ne_energy` (a `decide`) and reflexivity. The companion `momentum_ne_impulse` is the $M·L·T^{-1}$ case. Axiom-free.

Theorem 22.3.2. group statement"> uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***A newton metre is not a joule, though both are $M·L^2·T^{-2}$.*** *The SI units of torque and energy are not commensurable, because their kinds differ — the unit-level form of the torque/energy collision, a type-level fact the dimension cannot see. Uses metrological units.*

Lean code for Theorem 22.3.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.newtonMetre_joule_not_commensurable
  ¬PropertyKindCalculus.Iso80000.Part4.newtonMetre.Commensurable
      PropertyKindCalculus.Iso80000.Part4.joule
```

Proof for Theorem 22.3.2

uses 0

`newtonMetre_joule_not_commensurable : ¬ newtonMetre.Commensurable joule` by unfolding to the underlying kinds and `decide` (torque's kind differs from the energy kind).

Theorem 22.3.3.

group statement">uses 1 by 0 ✓

Effort: small

Tags: proved

> ***The dimension-1 disambiguation, on the standard.*** *There exist distinct ISO 80000-4 kinds with the same dimension one — efficiency and the relative mass density witness it, alongside the strains, the Poisson number, and the friction factors. Dimension cannot separate them; the kind layer does. Uses [the dimension map](#).*

Lean code for Theorem 22.3.3.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.iso80000_4_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 22.3.3

uses 0

`iso80000_4_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨efficiency, relativeMassDensity, …⟩` with `efficiency_ne_relativeMassDensity` (a `decide`) and reflexivity. Axiom-free.

22.4. The algebraic Remarks as kind-laws: mechanics built from space and time

Many Part-4 remarks state a quantity's defining relation *algebraically* — and, in mechanics, the other quantities are often *space-and-time* quantities from Part 3. Momentum is mass × velocity (4-8); pressure is force / area (4-14.1); mass density is mass / volume (4-2); specific volume is $1/\rho$ (4-3); kinematic viscosity is η/ρ (4-25); efficiency is output power / input power (4-29); a modulus of elasticity is stress / strain (4-19.1). Each is formalized as an R12 kind-law over the catalogued kinds, several composing a Part-4 kind out of Part-3 kinds.

Theorem 22.4.1. group statement">us

Effort	Tags
small	proved

> ***Momentum is mass × velocity — a verified, cross-part construction.*** *A momentum built as the product of a mass (Part 4) and a velocity (Part 3) carries its product certificate by construction, over the $\mathbb{R}$ carrier, and canonicity makes the certificate determine the quantity. Uses the product kind-law.*

Lean code for Theorem 22.4.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4/DefiningRelations.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.DefiningRelations.momentumOf_isProdu
  (m :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part4.mass.kind LTMCTDimensionBase)
      ℝ)
  (v :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part3.velocity.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part4.DefiningRelations.momentum_prod_mass_veloc
ity
    (PropertyKindCalculus.Iso80000.Part4.DefiningRelations.momentumOf m
      v)
    m v
```

Proof for Theorem 22.4.1

uses 0

`momentumOf m v := Quantity.mul momentum_prod_mass_velocity m v`, and `momentumOf_isProduct` is `rfl`; `momentum_certificate_canonical` is `eq_mul_of_isProduct`. The kind-law `momentum_prod_mass_velocity : ProductKind mass.kind Part3.velocity.kind momentum.kind` is `⟨rfl, rfl, rfl⟩`, and `momentum_dim_from_mass_velocity` checks $\mathsf{M}\cdot\mathsf{L}\cdot\mathsf{T}^{-1}$ follows from the relation. The same families give `pressureOf` (pressure = force / area) and `efficiencyOf`.

Theorem 22.4.2. group statement">us

Effort	Tags
small	proved

> ***Efficiency is dimension one because it is a ratio of two powers.*** *From the remark `η = P_out/P_in`, the powers cancel and the dimension is computed to be one — not stipulated, the mechanics analogue of the plane angle being a ratio of two lengths. Yet efficiency remains a distinct kind from every other dimension-one quantity (the collision capstone above). Uses [the dimension map](#).*

Lean code for Theorem 22.4.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part4/DefiningRelations.lean` complete

```
theorem PropertyKindCalculus.Iso80000.Part4.DefiningRelations.efficiency_dim_fr
  PropertyKindCalculus.Iso80000.Part4.efficiency.dim
      LTMCTDimensionBase =
    PropertyKindCalculus.Iso80000.Part4.power.dim LTMCTDimensionBase /
      PropertyKindCalculus.Iso80000.Part4.power.dim LTMCTDimensionBase
```

Proof for Theorem 22.4.2 uses 0

> `efficiency_dim_from_power_ratio : efficiency.dim = power.dim / power.dim`, by reducing both sides via `div_self'` (`a / a = 1`). The kind-law `efficiency_quot_power_power` is `⟨rfl, rfl, rfl⟩` and `efficiencyOf` builds a certified efficiency from two powers.

22.5. Item index — ISO 80000-4

Every catalogued item of ISO 80000-4, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in — the force-family lattice, the torque/energy collision, a cross-part defining relation, or the catalogue itself. Symbols and unit strings are **citation locators**; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
4-1	mass	m	kg	M
4-2	mass density	ρ	kg/m^3	$M \cdot L^{-3}$
4-3	specific volume	v	m^3/kg	$M^{-1} \cdot L^3$
4-4	relative mass density	d	1	1
4-5	surface mass density	ρ_A	kg/m^2	$M \cdot L^{-2}$
4-6	linear mass density	ρ_l	kg/m	$M \cdot L^{-1}$
4-7	moment of inertia	J	$kg \cdot m^2$	$M \cdot L^2$
4-8	momentum	p	$kg \cdot m/s$	$M \cdot L \cdot T^{-1}$
4-9.1	force	F	N	$M \cdot L \cdot T^{-2}$
4-9.2	weight	F_g	N	$M \cdot L \cdot T^{-2}$
4-9.3	static friction force	F_s	N	$M \cdot L \cdot T^{-2}$
4-9.4	kinetic friction force	F_k	N	$M \cdot L \cdot T^{-2}$
4-9.5	rolling resistance	F_{rr}	N	$M \cdot L \cdot T^{-2}$
4-9.6	drag force	F_D	N	$M \cdot L \cdot T^{-2}$
4-10	impulse	I	$N \cdot s$	$M \cdot L \cdot T^{-1}$
4-11	angular momentum	L	$kg \cdot m^2/s$	$M \cdot L^2 \cdot T^{-1}$
4-12.1	moment of force	M	$N \cdot m$	$M \cdot L^2 \cdot T^{-2}$
4-12.2	torque	T	$N \cdot m$	$M \cdot L^2 \cdot T^{-2}$
4-13	angular impulse	H	$N \cdot m \cdot s$	$M \cdot L^2 \cdot T^{-1}$
4-14.1	pressure	p	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-14.2	gauge pressure	p_e	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-15	stress	σ	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-16.1	normal stress	σ_n	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-16.2	shear stress	τ	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-17.1	strain	ε	1	1
4-17.2	relative linear strain	ε	1	1
4-17.3	shear strain	γ	1	1
4-17.4	relative volume strain	ϑ	1	1

Item	Quantity	Symbol	Unit	Dimension
4-18	Poisson number	μ	1	1
4-19.1	modulus of elasticity	E	Pa	$M·L^{-1}·T^{-2}$
4-19.2	modulus of rigidity	G	Pa	$M·L^{-1}·T^{-2}$
4-19.3	modulus of compression	K	Pa	$M·L^{-1}·T^{-2}$
4-20	compressibility	ϰ	Pa^{-1}	$M^{-1}·L·T^{2}$
4-21.1	second axial moment of area	I_a	m^4	L^4
4-21.2	second polar moment of area	I_p	m^4	L^4
4-22	section modulus	Z	m^3	L^3
4-23.1	static friction factor	μ_s	1	1
4-23.2	kinetic friction factor	μ	1	1
4-23.3	rolling resistance factor	C_rr	1	1
4-23.4	drag coefficient	C_D	1	1
4-24	dynamic viscosity	η	Pa·s	$M·L^{-1}·T^{-1}$
4-25	kinematic viscosity	ν	m^2/s	$L^2·T^{-1}$
4-26	surface tension	γ	N/m	$M·T^{-2}$
4-27	power	P	W	$M·L^2·T^{-3}$
4-28.1	potential energy	V	J	$M·L^2·T^{-2}$
4-28.2	kinetic energy	T	J	$M·L^2·T^{-2}$
4-28.3	mechanical energy	E	J	$M·L^2·T^{-2}$
4-28.4	mechanical work	W	J	$M·L^2·T^{-2}$
4-29	efficiency	η	1	1
4-30.1	mass flow	j_m	$kg/(m^2·s)$	$M·L^{-2}·T^{-1}$
4-30.2	mass flow rate	q_m	kg/s	$M·T^{-1}$
4-30.3	mass change rate	q_m	kg/s	$M·T^{-1}$
4-31	volume flow rate	q_V	m^3/s	$L^3·T^{-1}$
4-32	action	S	J·s	$M·L^2·T^{-1}$

23. ISO 80000-5 — Thermodynamics

The third part specified in full is ISO 80000-5, *Thermodynamics*. Every item — all of 5-1 … 5-36, including every sub-suffixed item (5-3.1 … 5-3.3, 5-5.1/5-5.2, 5-6.1/5-6.2, 5-10.1/5-10.2, 5-16.1 … 5-16.4, 5-17.1/5-17.2, 5-20.1 … 5-20.5, 5-21.1 … 5-21.5, 5-25.1/5-25.2) — is catalogued: the temperatures, the expansion and pressure coefficients, heat and the heat-transfer quantities, the heat capacities and entropy, the energy family (the thermodynamic potentials), the specific energies, the Massieu and Planck functions, efficiency, and the humidity quantities — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only **citation locators** are recorded; no normative content (definitions, remarks) from the licensed standard is reproduced. The defining *mathematics* of selected remarks is formalized in the sibling module `Part5.DefiningRelations`.

Thermodynamics brings the one axis Parts 3 and 4 could not show, and sharpens the two the calculus already pressed. It introduces the temperature base quantity `θ`, and with it the *scale-type* distinction — thermodynamic temperature is ratio-scale, but Celsius temperature, of the very same dimension, is only interval-scale (requirement *R6*, here on the standard). It holds the textbook *entropy-versus-heat-capacity* dimension collision (both `J/K`, down to the same unit string). And item 5-20 lays out the thermodynamic potentials — internal energy, enthalpy, the Helmholtz and Gibbs energies — as a specialization lattice over energy, mirroring Part 3's length family and Part 4's force family.

23.1. Quantity-kinds and units of ISO 80000-5

Definition 23.1.1. group statement">uses 1 used by 0 ✓

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "5-20.3"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream rather than kept in a comment.

Lean code for Definition 23.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 23.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-5, Second edition, 2019-08 item 5-18`. The `catalogue` lists all 54 items in item order — thermodynamic

temperature (5-1), heat (5-6.1), entropy (5-18), the thermodynamic potentials (5-20.1 … 5-20.5), the dew-point temperature (5-36), … — each paired with its coherent SI unit symbol.

Theorem 23.1.2. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

Entropy is $M \cdot L^2 \cdot T^{-2} \cdot \Theta^{-1}$ (item 5-18). *The catalogued entropy kind carries temperature-exponent minus one in PhysLib's dimension group — the standard's dimensional statement reproduced as a checked computation over the temperature generator `Θ`, not an annotation. Uses the dimension map.*

Lean code for Theorem 23.1.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` complete

```
theorem PropertyKindCalculus.Iso80000.Part5.entropy_dim_temperature :
  (PropertyKindCalculus.Iso80000.Part5.entropy.dim
        LTMCTDimensionBase).temperature =
    -1
```

Proof for Theorem 23.1.2 uses 0

Proved as `entropy_dim_temperature : entropy.dim.temperature = -1` in the `Iso80000` library, discharged by `norm_num` over the PhysLib dimension lemmas for the temperature generator. The companions `heatCapacity_dim_temperature = -1`, `heat_dim_length = 2` (item 5-6.1), and `linearExpansionCoefficient_dim_temperature = -1` (item 5-3.1) record the rest of the thermodynamic algebra.

Theorem 23.1.3. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

A kelvin and a joule are not commensurable. *Thermodynamic temperature and energy are distinct kinds, so their units cannot be compared by ratio — a type-level fact, not a runtime check. Uses metrological units.*

Lean code for Theorem 23.1.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part5.kelvin_joule_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part5.kelvin.Commensurable
      PropertyKindCalculus.Iso80000.Part5.joule
```

Proof for Theorem 23.1.3 uses 0

> Proved as `kelvin_joule_not_commensurable : ¬ kelvin.Commensurable joule` by unfolding `Commensurable` to the underlying kinds and `decide`. The kelvin and the joule are each `WellFormed` (thermodynamic temperature and energy are ratio-scale).

23.2. Temperature: one dimension, two scales

ISO 80000-5 lists *thermodynamic temperature* (5-1) and *Celsius temperature* (5-2) as separate items of the *same* dimension `0`. But they are not the same kind of quantity. Thermodynamic temperature has an absolute zero, so it is *ratio-scale*: ratios of temperatures are meaningful, and the kelvin admits `×` and `÷`. Celsius temperature is measured from the arbitrary zero of the ice point ($t = T - T_0$), so it is *interval-scale*: only its *differences* are meaningful, and a ratio of Celsius temperatures is not a defined operation. A dimension-only model sees one type, "a real number of degrees", for both.

This is requirement *R6* — the scale type fixes which operations are even *defined* — now on the real standard, the axis Part 3 and Part 4 (every quantity ratio-scale) could not exercise. The two temperatures are distinct kinds separated *neither* by dimension *nor* by an examination principle, but by their *scale type* alone.

Theorem 23.2.1. group statement">us

Effort: small — Tags: proved

> ***Celsius temperature does not admit ×,÷.*** *It is interval-scale (an arbitrary zero at the ice point), so a ratio of Celsius temperatures is not a meaningful operation — the scale layer refuses it where the dimension would not. Thermodynamic temperature, being ratio-scale, does admit it. Uses the scale type.*

Lean code for Theorem 23.2.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part5.celsiusTemperature_not_allowsRatio
  ¬(PropertyKindCalculus.Iso80000.Part5.celsiusTemperature.kind
        LTMCTDimensionBase).scale.AllowsRatio
```

Proof for Theorem 23.2.1

uses 0

`celsiusTemperature_not_allowsRatio : ¬ ScaleType.AllowsRatio celsiusTemperature.kind.scale`, with the companion `thermodynamicTemperature_allowsRatio` for the ratio case. The degree Celsius is nonetheless `WellFormed` (`degreeCelsius_wellFormed`, via the differential scale), and `thermodynamicTemperature_ne_celsiusTemperature` (a `decide`) records that the two are distinct kinds.

Theorem 23.2.2.

group statement">us

Effort: small

Tags: proved

The scale disambiguation, on the standard. *There exist distinct ISO 80000-5 kinds with the same dimension `θ` that are separated by scale type alone — thermodynamic temperature (ratio) and Celsius temperature (interval) witness it. Neither the dimension nor an examination principle tells them apart; the scale layer does. This is requirement R6 on the standard. Uses [the scale type].*

Lean code for Theorem 23.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part5.iso80000_5_scale_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        (a.kind LTMCTDimensionBase).scale ≠
          (b.kind LTMCTDimensionBase).scale
```

Proof for Theorem 23.2.2

uses 0

> `iso80000_5_scale_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.kind.scale ≠ b.kind.scale`, witnessed by `⟨thermodynamicTemperature, celsiusTemperature, …⟩` with reflexivity on the dimension and a `decide` on the differing scales. Axiom-free.

23.3. The energy family: distinguishing kinds by defining construction

ISO 80000-5 item 5-20 lists *energy* (5-20.1) as a genus and the *thermodynamic potentials* — internal energy (5-20.2), enthalpy (5-20.3), the Helmholtz energy (5-20.4), and the Gibbs energy (5-20.5) — as items beneath it. Every one has dimension `M·L²·T⁻²` and ratio scale. The standard distinguishes them only in *prose*, by *which thermodynamic variables are natural to the potential*: internal energy is a state function of `S, V, N`; the enthalpy is `H = U + pV`; the Helmholtz energy is `A = U − TS`; the Gibbs energy is `G = H − TS`. A dimension-only model sees one type, "a real number of joules", for all of them — and cannot even separate them from a Part-4 torque.

This is requirement *R2* again, now on the thermodynamic potentials — the pattern Part 3 set on the length family and Part 4 on the force family. Item 5-20.1 is the broad genus; each later item is a *species* of energy, distinguished **not by fiat** but by an explicit defining construction (Dybkær's examination principle, §7.5). The species are then provably distinct kinds while remaining *mutually comparable* as energies.

Definition 23.3.1. group uses 2 used by 0 ✓

An *energy species* is a ratio-scale quantity-kind of dimension `M·L²·T⁻²` carrying an examination principle — this work's own terse descriptor of the defining construction that distinguishes it (natural in S-V-N, in S-p, in T-V, in T-p), never the standard's normative definition. Internal energy, the enthalpy, and the Helmholtz and Gibbs energies are all species of the general energy kind (item 5-20.1).

Lean code for Definition 23.3.1. 1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
def PropertyKindCalculus.Iso80000.Part5.energySpecies (id : String)
  (p : PropertyKindCalculus.ExaminationPrinciple) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase
```

Proof for Definition 23.3.1 uses 0

> `energySpecies id p := { kind := { id, scale := .ratio, examPrinciple := some p.id }, dim := TDim.energy }`. The general `energy` carries no examination principle; the direct-parent edges of the

family are collected in an `Edge` relation, each species an edge to `energy` (mirroring the standard's "energy (item 5-20.1) … given by …").

Theorem 23.3.2. group statement">used 1 uses 0 ✓

Effort	Tags
small	proved

The Helmholtz and Gibbs energies are distinct kinds — by defining construction, not by fiat. *They share dimension `M·L²·T⁻²` and are both "free energies", yet are different kinds because they are defined by different natural variables (`A = U − TS`, natural in T,V; `G = H − TS`, natural in T,p), proved through `distinct_of_examPrinciple` (§7.5). This is the thermodynamic-potential analogue of width ≠ distance and static ≠ kinetic friction force. Uses the examination principle.*

Lean code for Theorem 23.3.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part5.helmholtzEnergy_ne_gibbsEnergy :
  PropertyKindCalculus.Iso80000.Part5.helmholtzEnergy.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part5.gibbsEnergy.kind
      LTMCTDimensionBase
```

Proof for Theorem 23.3.2 uses 0

`helmholtzEnergy_ne_gibbsEnergy : helmholtzEnergy.kind ≠ gibbsEnergy.kind`, from `KindOfProperty.distinct_of_examPrinciple` applied to the differing principles `natural-T-V` and `natural-T-p` (a `decide` on the `Option String` links). The companions `internalEnergy_ne_enthalpy` and `internalEnergy_ne_energy` separate the other species, and `internalEnergy_examinedBy` checks the kind-to-principle link. Axiom-free.

Theorem 23.3.3. group statement">used 1 uses 0 ✓

Effort	Tags
small	proved

Comparability is preserved. *The Helmholtz and Gibbs energies, though distinct kinds, remain mutually comparable — they share the super-kind energy, so combining them is possible but only via an explicit up-cast, never silently. The companion `internalEnergy_specializes_energy` records that internal energy specializes energy. Uses specialization.*

Lean code for Theorem 23.3.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part5.helmholtzEnergy_gibbsEnergy_comparab
  PropertyKindCalculus.MutuallyComparable
    PropertyKindCalculus.Iso80000.Part5.Edge
    (PropertyKindCalculus.Iso80000.Part5.helmholtzEnergy.kind
      LTMCTDimensionBase)
    (PropertyKindCalculus.Iso80000.Part5.gibbsEnergy.kind
      LTMCTDimensionBase)
```

Proof for Theorem 23.3.3 uses 0

> `helmholtzEnergy_gibbsEnergy_comparable : MutuallyComparable Edge helmholtzEnergy.kind gibbsEnergy.kind`, with witness `energy.kind`, each species reaching it by one edge. Axiom-free.

23.4. Dimension does not classify; the kind does — the entropy/heat-capacity case

Part 5 holds another textbook dimension collision. *Entropy* (5-18) and *heat capacity* (5-15) are *both* $M·L^2·T^{-2}·Θ^{-1}$ — the joule per kelvin — yet they are distinct kinds, and the collision is unusually sharp: their units carry not only the same dimension but the *same symbol*, "J/K", and are still not commensurable, because their kinds differ. The Massieu (5-22) and Planck (5-23) functions join them at `J/K`; specific entropy (5-19), specific heat capacity (5-16.1), and the specific gas constant (5-26) collide the same way at `J/(kg·K)`; and the humidity ratios and fractions (5-29 … 5-35), with the ratio of specific heats and the efficiencies, fill out the dimension-one family.

Theorem 23.4.1. group statement"> uses 1 · used by 0 ✓

Effort	Tags
small	proved

> ***Entropy is not heat capacity, though both are $M·L^2·T^{-2}·Θ^{-1}$.*** *There exist distinct ISO 80000-5 kinds with the same dimension — entropy and heat capacity witness it. No dimension-only type system can separate them; the kind layer does. This is the textbook thermodynamic case that a dimension does not classify a quantity. Uses the dimension map.*

Lean code for Theorem 23.4.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part5.iso80000_5_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 23.4.1

uses 0

`iso80000_5_dim_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim`, witnessed by `⟨entropy, heatCapacity, …⟩` with `entropy_ne_heatCapacity` (a `decide`) and reflexivity. The companion `specificEntropy_ne_specificHeatCapacity` is the `J/(kg·K)` case. Axiom-free.

Theorem 23.4.2. group statement">uses 1 by 0 ✓

Effort: small

Tags: proved

> ***The entropy "J/K" is not the heat-capacity "J/K".*** *The units of entropy and heat capacity carry the same symbol and the same dimension $M·L^{2}·T^{-2}·Θ^{-1}$, yet are not commensurable — the sharpest form of the rule that neither a dimension nor a unit string determines a kind. Uses metrological units.*

Lean code for Theorem 23.4.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part5.entropy_heatCapacity_unit_not_commer
  ¬PropertyKindCalculus.Iso80000.Part5.joulePerKelvinEntropy.Commensurable
      PropertyKindCalculus.Iso80000.Part5.joulePerKelvinHeatCapacity
```

Proof for Theorem 23.4.2

uses 0

`entropy_heatCapacity_unit_not_commensurable : ¬ joulePerKelvinEntropy.Commensurable joulePerKelvinHeatCapacity` by unfolding to the underlying kinds and `decide` (entropy's kind differs from the heat-capacity kind, although both units are written "J/K").

Theorem 23.4.3. group statement">uses 1 by 0 ✓

Effort	Tags
small	proved

> ***The dimension-1 disambiguation, on the standard.*** *There exist distinct ISO 80000-5 kinds with the same dimension one — the ratio of specific heat capacities and the efficiency witness it, alongside the isentropic exponent and the humidity ratios and fractions. Dimension cannot separate them; the kind layer does. Uses [the dimension map](#).*

Lean code for Theorem **23.4.3.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part5.iso80000_5_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 23.4.3 uses 0

`iso80000_5_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨ratioOfSpecificHeatCapacities, efficiency, …⟩` with `ratioOfSpecificHeatCapacities_ne_efficiency` (a `decide`) and reflexivity. Axiom-free.

23.5. The algebraic Remarks as kind-laws: thermodynamics built from the earlier parts

Many Part-5 remarks state a quantity's defining relation *algebraically* — and, in thermodynamics, the other quantities are often from *earlier parts* (mass from ISO 80000-4, area from ISO 80000-3) or from temperature, the base quantity this part introduces. Heat capacity is added heat per temperature (5-15); specific heat capacity is heat capacity / mass (5-16.1); specific entropy is entropy / mass (5-19); the density of heat flow rate is heat flow rate / area (5-8); thermal conductance is `1/R` (5-13); the ratio of specific heats is `cp/cV` (5-17.1). Each is formalized as an R12 kind-law over the catalogued kinds, several composing a Part-5 kind out of an earlier part's kinds.

Theorem **23.5.1.** group statement">us

Effort	Tags
small	proved

> ***Specific heat capacity is heat capacity / mass — a verified, cross-part construction.*** *A specific heat capacity built as the quotient of a heat capacity (Part 5) by a mass (Part 4) carries its quotient certificate by construction, over the $\mathbb{R}$ carrier, and canonicity makes the certificate determine the quantity. Uses the quotient kind-law.*

Lean code for Theorem 23.5.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5/DefiningRelations.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part5.DefiningRelations.specificHeatCapaci
  (c :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part5.heatCapacity.kind
        LTMCTDimensionBase)
      ℝ)
  (m :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part4.mass.kind LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsQuotient

PropertyKindCalculus.Iso80000.Part5.DefiningRelations.specificHeatCapacity_quo
t_heatCapacity_mass

(PropertyKindCalculus.Iso80000.Part5.DefiningRelations.specificHeatCapacityOf
      c m)
    c m
```

Proof for Theorem 23.5.1

uses 0

`specificHeatCapacityOf c m := Quantity.div specificHeatCapacity_quot_heatCapacity_mass c m`, and `specificHeatCapacityOf_isQuotient` is `rfl`; `specificHeatCapacity_certificate_canonical` is `eq_div_of_isQuotient`. The kind-law `specificHeatCapacity_quot_heatCapacity_mass : QuotientKind heatCapacity.kind Part4.mass.kind specificHeatCapacity.kind` is `⟨rfl, rfl, rfl⟩`, and `specificHeatCapacity_dim_from_heatCapacity_mass` checks $\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{0}^{-1}$ follows from the relation. The same families give `densityOfHeatFlowRateOf` (= heat flow rate / area) and `thermalConductanceOf` (= the reciprocal of thermal resistance).

Theorem 23.5.2. group statement">us

Effort

small

Tags

proved

> ***The ratio of specific heats is dimension one because it is a ratio of two specific heat capacities.*** *From the remark `γ = cp/cV`, the specific heat capacities cancel and the dimension is computed to be one — not stipulated, the thermodynamic analogue of the plane angle being a ratio of two lengths and the efficiency a ratio of two powers. Yet the ratio remains a distinct kind from every other dimension-one quantity (the collision capstone above). Uses [the dimension map](#).*

Lean code for Theorem 23.5.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part5/DefiningRelations.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part5.DefiningRelations.ratio_dim_from_spe
  PropertyKindCalculus.Iso80000.Part5.ratioOfSpecificHeatCapacities.dim
      LTMCTDimensionBase =

PropertyKindCalculus.Iso80000.Part5.specificHeatCapacityConstantPressure.dim
        LTMCTDimensionBase /

PropertyKindCalculus.Iso80000.Part5.specificHeatCapacityConstantVolume.dim
        LTMCTDimensionBase
```

Proof for Theorem 23.5.2

uses 0

> `ratio_dim_from_specific_heats : ratioOfSpecificHeatCapacities.dim = specificHeatCapacityConstantPressure.dim / specificHeatCapacityConstantVolume.dim`, by reducing both sides via `div_self'` (`a / a = 1`). The kind-law `ratioOfSpecificHeatCapacities_quot_cp_cV` is `⟨rfl, rfl, rfl⟩`.

23.6. Item index — ISO 80000-5

Every catalogued item of ISO 80000-5, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in — the scale-type distinction, the energy-family lattice, the entropy/heat-capacity collision, a cross-part defining relation, or the catalogue itself. Symbols and unit strings are **citation locators**; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
5-1	thermodynamic temperature	T	K	$\mathsf{\Theta}$
5-2	Celsius temperature	t	°C	$\mathsf{\Theta}$
5-3.1	linear expansion coefficient	α_l	K^{-1}	$\mathsf{\Theta}^{-1}$
5-3.2	cubic expansion coefficient	α_V	K^{-1}	$\mathsf{\Theta}^{-1}$
5-3.3	relative pressure coefficient	α_p	K^{-1}	$\mathsf{\Theta}^{-1}$
5-4	pressure coefficient	β	Pa/K	$\mathsf{M}\cdot\mathsf{L}^{-1}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-5.1	isothermal compressibility	$\varkappa_T$	Pa^{-1}	$\mathsf{M}^{-1}\cdot\mathsf{L}\cdot\mathsf{T}^{2}$
5-5.2	isentropic compressibility	$\varkappa_S$	Pa^{-1}	$\mathsf{M}^{-1}\cdot\mathsf{L}\cdot\mathsf{T}^{2}$
5-6.1	heat	Q	J	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$
5-6.2	latent heat	Q	J	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$
5-7	heat flow rate	Φ	W	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-3}$
5-8	density of heat flow rate	q	W/m^2	$\mathsf{M}\cdot\mathsf{T}^{-3}$
5-9	thermal conductivity	λ	W/(m·K)	$\mathsf{M}\cdot\mathsf{L}\cdot\mathsf{T}^{-3}\cdot\mathsf{\Theta}^{-1}$
5-10.1	coefficient of heat transfer	K	$W/(m^2\cdot K)$	$\mathsf{M}\cdot\mathsf{T}^{-3}\cdot\mathsf{\Theta}^{-1}$
5-10.2	surface coefficient of heat transfer	h	$W/(m^2\cdot K)$	$\mathsf{M}\cdot\mathsf{T}^{-3}\cdot\mathsf{\Theta}^{-1}$
5-11	thermal insulance	M	$m^2\cdot K/W$	$\mathsf{M}^{-1}\cdot\mathsf{T}^{3}\cdot\mathsf{\Theta}$
5-12	thermal resistance	R	K/W	$\mathsf{M}^{-1}\cdot\mathsf{L}^{-2}\cdot\mathsf{T}^{3}\cdot\mathsf{\Theta}$
5-13	thermal conductance	G	W/K	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-3}\cdot\mathsf{\Theta}^{-1}$
5-14	thermal diffusivity	a	m^2/s	$\mathsf{L}^{2}\cdot\mathsf{T}^{-1}$
5-15	heat capacity	C	J/K	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-16.1	specific heat capacity	c	J/(kg·K)	$\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-16.2	specific heat capacity at constant pressure	c_p	J/(kg·K)	$\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-16.3	specific heat capacity at constant volume	c_V	J/(kg·K)	$\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-16.4	specific heat capacity at saturated vapour pressure	c_{sat}	J/(kg·K)	$\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-17.1	ratio of specific heat capacities	γ	1	1
5-17.2	isentropic exponent	$\varkappa$	1	1
5-18	entropy	S	J/K	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$
5-19	specific entropy	s	J/(kg·K)	$\mathsf{L}^{2}\cdot\mathsf{T}^{-2}\cdot\mathsf{\Theta}^{-1}$

Item	Quantity	Symbol	Unit	Dimension
5-20.1	energy	E	J	$M \cdot L^2 \cdot T^{-2}$
5-20.2	internal energy	U	J	$M \cdot L^2 \cdot T^{-2}$
5-20.3	enthalpy	H	J	$M \cdot L^2 \cdot T^{-2}$
5-20.4	Helmholtz energy	A	J	$M \cdot L^2 \cdot T^{-2}$
5-20.5	Gibbs energy	G	J	$M \cdot L^2 \cdot T^{-2}$
5-21.1	specific energy	e	J/kg	$L^2 \cdot T^{-2}$
5-21.2	specific internal energy	u	J/kg	$L^2 \cdot T^{-2}$
5-21.3	specific enthalpy	h	J/kg	$L^2 \cdot T^{-2}$
5-21.4	specific Helmholtz energy	a	J/kg	$L^2 \cdot T^{-2}$
5-21.5	specific Gibbs energy	g	J/kg	$L^2 \cdot T^{-2}$
5-22	Massieu function	J	J/K	$M \cdot L^2 \cdot T^{-2} \cdot \Theta^{-1}$
5-23	Planck function	Y	J/K	$M \cdot L^2 \cdot T^{-2} \cdot \Theta^{-1}$
5-24	Joule-Thomson coefficient	μ_JT	K/Pa	$M^{-1} \cdot L \cdot T^2 \cdot \Theta$
5-25.1	efficiency	η	1	1
5-25.2	maximum efficiency	η_max	1	1
5-26	specific gas constant	R_s	J/(kg·K)	$L^2 \cdot T^{-2} \cdot \Theta^{-1}$
5-27	mass concentration of water	w	kg/m³	$M \cdot L^{-3}$
5-28	mass concentration of water vapour	v	kg/m³	$M \cdot L^{-3}$
5-29	mass ratio of water to dry matter	u	1	1
5-30	mass ratio of water vapour to dry gas	r	1	1
5-31	mass fraction of water	w_H2O	1	1
5-32	mass fraction of dry matter	w_d	1	1
5-33	relative humidity	φ	1	1
5-34	relative mass concentration of vapour	φ	1	1
5-35	relative mass ratio of vapour	ψ	1	1
5-36	dew-point temperature	T_d	K	Θ

24. IEC 80000-6 — Electromagnetism

The fourth part specified in full is IEC 80000-6, *Electromagnetism* — the one *IEC*-published part of the series. Every item — all of 6-1 … 6-62, including every sub-suffixed item (6-2.1/6-2.2, 6-11.1 … 6-11.4, 6-14.1/6-14.2, 6-19.1/6-19.2, 6-22.1 … 6-22.4, 6-26.1/6-26.2, 6-35.1/6-35.2, 6-37.1 … 6-37.3, 6-41.1/6-41.2, 6-42.1/6-42.2, 6-51.1 … 6-51.5, 6-52.1 … 6-52.5) — is catalogued: electric current and charge, the charge and current distributions, the electric field and potential, the capacitances and permittivities, the magnetic field, flux, and potentials, the magnetic circuit, conductivity and resistivity, impedance and admittance, and the AC power quantities — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only *citation locators* are recorded; no normative content (definitions, remarks) from the licensed standard is reproduced. The defining *mathematics* of selected remarks is formalized in the sibling module `Part6.DefiningRelations`.

Electromagnetism brings the last SI base quantity the earlier parts did not exercise — *electric current* — and sharpens the two axes the calculus already pressed. The base axis enters as the ampere; PhysLib's `Dimension` takes electric *charge* `C` as its generator, so electric current appears as $C \cdot T^{-1}$ (coulomb per second). The *scale-type* distinction recurs — electric potential is gauge-dependent, fixed only up to an arbitrary additive reference, hence interval-scale, while electric potential difference, of the very same dimension `V`, is ratio-scale (requirement *R6*). And the AC power quantities give the sharpest *dimension does not classify* case yet: active, reactive, and apparent power are all $M \cdot L^2 \cdot T^{-3}$, yet the standard spends *three* different unit strings on that one dimension — the watt, the var, and the volt-ampere — and arranges them as a specialization lattice over power (requirement *R2*).

24.1. Quantity-kinds and units of IEC 80000-6

Definition 24.1.1. group statement">uses 1 used by 0 ✓

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "6-51.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream rather than kept in a comment.

Lean code for Definition 24.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` — complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 24.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `IEC 80000-6, Edition 2.0, 2022-11 item 6-51.1` (the one IEC-published part of the catalogue). The `catalogue` lists all 85 items in item order — electric current (6-1), electric charge (6-2.1), magnetic flux (6-22.1), the impedance family (6-51.1 … 6-51.5), the AC power quantities (6-56 … 6-61), active energy (6-62), … — each paired with its coherent SI unit symbol.

Theorem 24.1.2. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

A capacitance carries the charge generator squared (item 6-13). *The catalogued capacitance kind has charge-exponent two in PhysLib's dimension group — the farad's `C²·M⁻¹·L⁻²·T²` reproduced as a checked computation over the new electric-current base axis, not an annotation. Uses [the dimension map](#).*

Lean code for Theorem 24.1.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part6.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.capacitance_dim_charge :
  (PropertyKindCalculus.Iso80000.Part6.capacitance.dim
        LTMCTDimensionBase).charge =
    2
```

Proof for Theorem 24.1.2

uses 0

Proved as `capacitance_dim_charge : capacitance.dim.charge = 2` in the `Iso80000` library, discharged by `norm_num` over the PhysLib dimension lemmas for the charge generator. The companions `electricCurrent_dim_charge = 1` and `electricCurrent_dim_time = -1` (item 6-1, the ampere as `C·T⁻¹`), `resistance_dim_charge = -2` (item 6-46, the ohm), and `magneticFlux_dim_charge = -1` (item 6-22.1, the weber) record the rest of the electromagnetic algebra.

Theorem 24.1.3. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

An ampere and a volt are not commensurable. *Electric current and electric potential are distinct kinds, so their units cannot be compared by ratio — a type-level fact, not a runtime check. Uses [metrological units](#).*

Lean code for Theorem 24.1.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part6.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part6.ampere_voltPotential_not_commensural
  ¬PropertyKindCalculus.Iso80000.Part6.ampere.Commensurable
      PropertyKindCalculus.Iso80000.Part6.voltPotential
```

Proof for Theorem 24.1.3 uses 0

Proved as `ampere_voltPotential_not_commensurable : ¬ ampere.Commensurable voltPotential` by unfolding `Commensurable` to the underlying kinds and `decide`. The ampere and the ohm are each `WellFormed` (electric current and resistance are ratio-scale).

24.2. Electric potential: one dimension, two scales

IEC 80000-6 lists *electric potential* (6-11.1) and *electric potential difference* (6-11.2) as separate items of the *same* dimension `V`. But they are not the same kind of quantity. Electric potential is fixed only up to an arbitrary additive reference — the choice of where "zero potential" sits (the ground, infinity) is a gauge, so the standard notes the potential "is not unique". Only its *differences* are physically meaningful, so it is *interval-scale*: a ratio of two electric potentials is not a defined operation. Electric potential difference is a genuine difference, so it is *ratio-scale*. A dimension-only model sees one type, "a real number of volts", for both.

This is requirement *R6* — the scale type fixes which operations are even *defined* — recurring on electromagnetism exactly as it appeared on thermodynamics. The gauge-dependent electric potential is electromagnetism's Celsius temperature: same dimension as its companion, separated from it *neither* by dimension *nor* by an examination principle, but by *scale type* alone.

Theorem 24.2.1. group statement">us

Effort: small

Tags: proved

> ***Electric potential does not admit ×,÷.*** *It is interval-scale (gauge freedom — fixed only up to an arbitrary additive reference), so a ratio of electric potentials is not a meaningful operation — the scale layer refuses it where the dimension would not. Electric potential difference, being ratio-scale, does admit it. Uses the scale type.*

Lean code for Theorem 24.2.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part6.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part6.electricPotential_not_allowsRatio :
  ¬(PropertyKindCalculus.Iso80000.Part6.electricPotential.kind
        LTMCTDimensionBase).scale.AllowsRatio
```

Proof for Theorem 24.2.1

uses 0

`electricPotential_not_allowsRatio : ¬ ScaleType.AllowsRatio electricPotential.kind.scale`, with the companion `electricPotentialDifference_allowsRatio` for the ratio case. The volt of electric potential is nonetheless `WellFormed` (`voltPotential_wellFormed`, via the differential scale), and `voltPotential_voltDifference_not_commensurable` records that the two volts — same symbol, same dimension — are not commensurable, because the kinds differ in scale.

Theorem 24.2.2. group statement">us

Effort	Tags
small	proved

The scale disambiguation, on the standard. *There exist distinct IEC 80000-6 kinds with the same dimension v that are separated by scale type alone — electric potential (interval, gauge-dependent) and electric potential difference (ratio) witness it. Neither the dimension nor an examination principle tells them apart; the scale layer does. This is requirement R6 on the standard, the electromagnetic counterpart of thermodynamic versus Celsius temperature. Uses the scale type.*

Lean code for Theorem 24.2.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part6.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part6.iec80000_6_scale_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        (a.kind LTMCTDimensionBase).scale ≠
          (b.kind LTMCTDimensionBase).scale
```

Proof for Theorem 24.2.2 uses 0

> `iec80000_6_scale_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.kind.scale ≠ b.kind.scale`, witnessed by `⟨electricPotential, electricPotentialDifference, …⟩` with reflexivity on the dimension and a `decide` on the differing scales. Axiom-free.

24.3. The AC power family: distinguishing kinds by defining construction

IEC 80000-6 lists *power* (6-45) as a genus and the AC power quantities — active power (6-56), apparent power (6-57), complex power (6-59), reactive power (6-60), and non-active power (6-61) — as items of one dimension `M·L²·T⁻³`. The standard distinguishes them by *which component of the periodic process each isolates*: active power is the time-averaged real (resistive) component; reactive power the imaginary part of the complex power; apparent power the product of RMS voltage and current; and so on. Tellingly, it spends *three different unit strings* on this single dimension — the watt for active power, the var for reactive power, the volt-ampere for apparent power — precisely to keep the kinds apart in print. A dimension-only model sees one type, "a real number of watts", for all of them.

This is requirement *R2* again, now on the AC power quantities — the pattern Part 3 set on the length family, Part 4 on the force family, and Part 5 on the thermodynamic potentials. Item 6-45 is the broad genus; each AC quantity is a *species* of power, distinguished **not by fiat** but by an explicit defining construction (Dybkær's examination principle, §7.5). The species are then provably distinct kinds while remaining *mutually comparable* as powers — comparable enough to combine into the complex power `P + jQ`.

Definition 24.3.1. group uses 2 used by 0 ✓

A *power species* is a ratio-scale quantity-kind of dimension `M·L²·T⁻³` carrying an examination principle — this work's own terse descriptor of the defining construction that distinguishes it (the time-averaged real part, the RMS product, the complex sum `P + jQ`, the imaginary part, the residual `√(S² − P²)`), never the standard's normative definition. Active, apparent, complex, reactive, and non-active power are all species of the general power kind (item 6-45).

Lean code for Definition 24.3.1. 1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part6.lean` — complete

```
def PropertyKindCalculus.Iso80000.Part6.powerSpecies (id : String)
  (p : PropertyKindCalculus.ExaminationPrinciple) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase
```

Proof for Definition 24.3.1

uses 0

`powerSpecies id p := { kind := { id, scale := .ratio, examPrinciple := some p.id }, dim := EDim.power }`. The general `power` carries no examination principle; the direct-parent edges of the family are collected in an `Edge` relation, each species an edge to `power` (mirroring the standard's listing of the AC quantities beneath power).

Theorem 24.3.2.

group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

Active and reactive power are distinct kinds — by defining construction, not by fiat. *They share dimension $M \cdot L^2 \cdot T^{-3}$ and are both "powers", yet are different kinds because they isolate different components of the periodic process (the time-averaged real part vs the imaginary part of the complex power), proved through `distinct_of_examPrinciple` (§7.5). This is the AC-power analogue of width ≠ distance, static ≠ kinetic friction force, and Helmholtz ≠ Gibbs energy. Uses [the examination principle](#).*

Lean code for Theorem 24.3.2.

1 theorem

theorem · defined in `PropertyKindCalculus/Iso80000/Part6.lean`

complete

```
theorem PropertyKindCalculus.Iso80000.Part6.activePower_ne_reactivePower :
  PropertyKindCalculus.Iso80000.Part6.activePower.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part6.reactivePower.kind
      LTMCTDimensionBase
```

Proof for Theorem 24.3.2

uses 0

`activePower_ne_reactivePower : activePower.kind ≠ reactivePower.kind`, from `KindOfProperty.distinct_of_examPrinciple` applied to the differing principles `time-averaged-real` and `reactive-imaginary` (a `decide` on the `Option String` links). The companion `apparentPower_ne_complexPower` separates two further species, and `activePower_examinedBy` checks the kind-to-principle link. Axiom-free.

Theorem 24.3.3.

group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***Comparability is preserved.*** *Active and reactive power, though distinct kinds, remain mutually comparable — they share the super-kind power, so combining them (into the complex power) is possible but only via an explicit up-cast, never silently. The companion* `activePower_specializes_power` *records that active power specializes power. Uses specialization.*

Lean code for Theorem **24.3.3.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part6.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.activePower_reactivePower_comparable
  PropertyKindCalculus.MutuallyComparable
    PropertyKindCalculus.Iso80000.Part6.Edge
    (PropertyKindCalculus.Iso80000.Part6.activePower.kind
      LTMCTDimensionBase)
    (PropertyKindCalculus.Iso80000.Part6.reactivePower.kind
      LTMCTDimensionBase)
```

Proof for Theorem 24.3.3 uses 0

`activePower_reactivePower_comparable : MutuallyComparable Edge activePower.kind reactivePower.kind`, with witness `power.kind`, each species reaching it by one edge. Axiom-free.

24.4. Dimension does not classify; the kind does — the power and impedance cases

Part 6 holds the sharpest dimension collision in the series so far. *Active power* (6-56), *reactive power* (6-60), and *apparent power* (6-57) are *all* $M \cdot L^2 \cdot T^{-3}$, yet they are distinct kinds — and the collision is sharper than Part 5's joule-per-kelvin, because there even the unit string agreed, while here the standard itself spends three strings (`W`, `var`, `VA`) on the one dimension. *Resistance* (6-46) and *reactance* (6-51.4) collide the same way at the ohm — the real and imaginary parts of an impedance — and the relative permittivity (6-15), relative permeability (6-27), the susceptibilities, the coupling and leakage factors, and the quality, loss, and power factors fill out the dimension-one family.

Theorem **24.4.1.** group statement"> used by 0 ✓

Effort: small — Tags: proved

> ***Active power is not reactive power, though both are*** $M \cdot L^2 \cdot T^{-3}$***.*** *There exist distinct IEC 80000-6 kinds with the same dimension — active and reactive power witness it, alongside apparent, complex,*

> *and non-active power. No dimension-only type system can separate them; the kind layer does. This is the electromagnetic case that a dimension does not classify a quantity, sharper than Part 5's. Uses [the dimension map](#).*

Lean code for Theorem 24.4.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part6.lean — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.iec80000_6_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 24.4.1

uses 0

> `iec80000_6_dim_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim`, witnessed by `⟨activePower, reactivePower, …⟩` with `activePower_ne_reactivePower` and reflexivity. The companion `resistance_ne_reactance` is the ohm case (real vs imaginary part of an impedance). Axiom-free.

Theorem 24.4.2.

group statement">uses 1 by 0 ✓

Effort	Tags
small	proved

> ***The active-power watt is not the reactive-power var.*** *The units of active and reactive power carry the same dimension `M·L²·T⁻³`, yet are not commensurable — and the standard keeps them apart not only by kind but by string, `W` versus `var`, a sharper form of the rule that neither a dimension nor a unit string determines a kind. Uses [metrological units](#).*

Lean code for Theorem 24.4.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part6.lean — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.wattActive_varReactive_not_commensu
  ¬PropertyKindCalculus.Iso80000.Part6.wattActive.Commensurable
      PropertyKindCalculus.Iso80000.Part6.varReactive
```

Proof for Theorem 24.4.2

uses 0

`wattActive_varReactive_not_commensurable : ¬ wattActive.Commensurable varReactive` by unfolding to the underlying kinds and `decide` (active power's kind differs from the reactive-power kind; the units are written "W" and "var").

Theorem 24.4.3. group statement">uses 1 used by 0 ✓

Effort: small

Tags: proved

> ***The dimension-1 disambiguation, on the standard.*** *There exist distinct IEC 80000-6 kinds with the same dimension one — relative permittivity and relative permeability witness it, alongside the susceptibilities, the coupling and leakage factors, and the quality, loss, and power factors. Dimension cannot separate them; the kind layer does. Uses [the dimension map](#).*

Lean code for Theorem 24.4.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part6.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.iec80000_6_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 24.4.3 uses 0

`iec80000_6_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨relativePermittivity, relativePermeability, …⟩` with `relativePermittivity_ne_relativePermeability` (a `decide`) and reflexivity. Axiom-free.

24.5. The algebraic Remarks as kind-laws: electromagnetism built by the kind algebra

Many Part-6 definitions state a quantity's defining relation *algebraically* — the constitutive laws of circuit theory. Resistance is voltage per current (Ohm's law, 6-46); conductance is `1/R` (6-47); capacitance is charge per voltage (6-13); power is voltage times current (6-45); resistivity is `1/σ` (6-44); admittance is `1/Z` (6-52.1); permeance is `1/Rₘ` (6-40); the power factor is active over apparent power (6-58). Electric current itself is charge per time (6-1), crossing to ISO 80000-3 for its time factor. Each is formalized as an R12 kind-law over the catalogued kinds.

Theorem 24.5.1. group statement">use

Effort	Tags
small	proved

> ***Resistance is voltage / current — a verified construction (Ohm's law).*** *A resistance built as the quotient of a voltage by an electric current carries its quotient certificate by construction, over the* `ℝ` *carrier, and canonicity makes the certificate determine the quantity. Uses the quotient kind-law.*

Lean code for Theorem 24.5.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part6/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.DefiningRelations.resistanceOf_isQu
  (u :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part6.voltage.kind
        LTMCTDimensionBase)
      ℝ)
  (i :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part6.electricCurrent.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsQuotient

PropertyKindCalculus.Iso80000.Part6.DefiningRelations.resistance_quot_voltage_
current
    (PropertyKindCalculus.Iso80000.Part6.DefiningRelations.resistanceOf
      u i)
    u i
```

Proof for Theorem 24.5.1

uses 0

`resistanceOf u i := Quantity.div resistance_quot_voltage_current u i`, and `resistanceOf_isQuotient` is `rfl`; `resistance_certificate_canonical` is `eq_div_of_isQuotient`. The kind-law `resistance_quot_voltage_current : QuotientKind voltage.kind electricCurrent.kind resistance.kind` is `⟨rfl, rfl, rfl⟩`, and `resistance_dim_from_voltage_current` checks `M·L²·T⁻¹·C⁻²` follows from the relation. The same families give `powerOf` (= voltage × current, a `ProductKind`), `conductanceOf` (= `1/R`), and the cross-part `electricCurrent_quot_charge_duration` (= charge / time, the time crossing to ISO 80000-3).

Theorem 24.5.2. group statement">use

Effort	Tags
small	proved

> ***The power factor is dimension one because it is a ratio of two powers.*** *From the relation `λ = P/S`, the powers cancel and the dimension is computed to be one — not stipulated, the electromagnetic analogue of the plane angle being a ratio of two lengths and the thermodynamic efficiency a ratio of two powers. Yet the power factor remains a distinct kind from every other dimension-one quantity (the collision capstone above). Uses [the dimension map](#).*

Lean code for Theorem 24.5.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part6/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part6.DefiningRelations.powerFactor_dim_f
  PropertyKindCalculus.Iso80000.Part6.powerFactor.dim
      LTMCTDimensionBase =
    PropertyKindCalculus.Iso80000.Part6.activePower.dim
        LTMCTDimensionBase /
      PropertyKindCalculus.Iso80000.Part6.apparentPower.dim
        LTMCTDimensionBase
```

Proof for Theorem 24.5.2

uses 0

`powerFactor_dim_from_powers : powerFactor.dim = activePower.dim / apparentPower.dim`, by reducing both sides via `div_self` (`a / a = 1`). The kind-law `powerFactor_quot_active_apparent` is `⟨rfl, rfl, rfl⟩`. The companion `power_dim_from_voltage_current` checks `M·L²·T⁻³` follows from `P = U·I` (the charge generator cancelling between voltage and current).

24.6. Item index — IEC 80000-6

Every catalogued item of IEC 80000-6, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it (over the charge generator C). Each item number links to the formalized result it participates in — the scale-type distinction, the AC-power lattice, a dimension collision, a defining relation, or the catalogue itself. Symbols and unit strings are **citation locators**; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
6-1	electric current	I	A	I
6-2.1	electric charge	Q	C	T·I
6-2.2	elementary charge	e	C	T·I
6-3	electric charge density	ρ	C/m³	L⁻³·T·I
6-4	surface density of electric charge	σ	C/m²	L⁻²·T·I
6-5	linear density of electric charge	τ	C/m	L⁻¹·T·I
6-6	electric dipole moment	p	C·m	L·T·I
6-7	electric polarization	P	C/m²	L⁻²·T·I
6-8	electric current density	J	A/m²	L⁻²·I
6-9	linear electric current density	J_S	A/m	L⁻¹·I
6-10	electric field strength	E	V/m	M·L·T⁻³·I⁻¹
6-11.1	electric potential	V	V	M·L²·T⁻³·I⁻¹
6-11.2	electric potential difference	V_ab	V	M·L²·T⁻³·I⁻¹
6-11.3	voltage	U	V	M·L²·T⁻³·I⁻¹
6-11.4	induced voltage	U_i	V	M·L²·T⁻³·I⁻¹
6-12	electric flux density	D	C/m²	L⁻²·T·I
6-13	capacitance	C	F	M⁻¹·L⁻²·T⁴·I²
6-14.1	electric constant	ε_0	F/m	M⁻¹·L⁻³·T⁴·I²
6-14.2	permittivity	ε	F/m	M⁻¹·L⁻³·T⁴·I²
6-15	relative permittivity	ε_r	1	1
6-16	electric susceptibility	χ	1	1
6-17	electric flux	Ψ	C	T·I
6-18	displacement current density	J_D	A/m²	L⁻²·I
6-19.1	displacement current	I_D	A	I
6-19.2	total current	I_tot	A	I
6-20	total current density	J_tot	A/m²	L⁻²·I
6-21	magnetic flux density	B	T	M·T⁻²·I⁻¹
6-22.1	magnetic flux	Φ	Wb	M·L²·T⁻²·I⁻¹

Item	Quantity	Symbol	Unit	Dimension
6-22.2	protoflux	Ψ_p	Wb	M·L²·T⁻²·I⁻¹
6-22.3	linked magnetic flux	Φ_l	Wb	M·L²·T⁻²·I⁻¹
6-22.4	total magnetic flux	Ψ	Wb	M·L²·T⁻²·I⁻¹
6-23	magnetic moment	m	A·m²	L²·I
6-24	magnetization	M	A/m	L⁻¹·I
6-25	magnetic field strength	H	A/m	L⁻¹·I
6-26.1	magnetic constant	μ_0	H/m	M·L·T⁻²·I⁻²
6-26.2	permeability	μ	H/m	M·L·T⁻²·I⁻²
6-27	relative permeability	μ_r	1	1
6-28	magnetic susceptibility	κ	1	1
6-29	magnetic polarization	J_m	T	M·T⁻²·I⁻¹
6-30	magnetic dipole moment	j_m	Wb·m	M·L³·T⁻²·I⁻¹
6-31	coercivity	H_c	A/m	L⁻¹·I
6-32	magnetic vector potential	A	Wb/m	M·L·T⁻²·I⁻¹
6-33	electromagnetic energy density	w	J/m³	M·L⁻¹·T⁻²
6-34	Poynting vector	S	W/m²	M·T⁻³
6-35.1	phase speed of electromagnetic waves	c	m/s	L·T⁻¹
6-35.2	speed of light in vacuum	c_0	m/s	L·T⁻¹
6-36	source voltage	U_s	V	M·L²·T⁻³·I⁻¹
6-37.1	magnetic potential	V_m	A	I
6-37.2	magnetic tension	U_m	A	I
6-37.3	magnetomotive force	F_m	A	I
6-38	number of turns in a winding	N	1	1
6-39	reluctance	R_m	H⁻¹	M⁻¹·L⁻²·T²·I²
6-40	permeance	Λ	H	M·L²·T⁻²·I⁻²
6-41.1	inductance	L	H	M·L²·T⁻²·I⁻²
6-41.2	mutual inductance	L_mn	H	M·L²·T⁻²·I⁻²
6-42.1	coupling factor	k	1	1

Item	Quantity	Symbol	Unit	Dimension
6-42.2	leakage factor	σ	1	1
6-43	conductivity	σ	S/m	$M^{-1} \cdot L^{-3} \cdot T^{3} \cdot I^{2}$
6-44	resistivity	ρ	Ω·m	$M \cdot L^{3} \cdot T^{-3} \cdot I^{-2}$
6-45	power	P	W	$M \cdot L^{2} \cdot T^{-3}$
6-46	resistance	R	Ω	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-2}$
6-47	conductance	G	S	$M^{-1} \cdot L^{-2} \cdot T^{3} \cdot I^{2}$
6-48	phase difference	φ	rad	1
6-49	electric current phasor	I	A	I
6-50	voltage phasor	U	V	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-1}$
6-51.1	impedance	Z	Ω	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-2}$
6-51.2	impedance of vacuum	Z_0	V/A	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-2}$
6-51.3	resistance	R	Ω	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-2}$
6-51.4	reactance	X	Ω	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-2}$
6-51.5	apparent impedance	Z	Ω	$M \cdot L^{2} \cdot T^{-3} \cdot I^{-2}$
6-52.1	admittance	Y	S	$M^{-1} \cdot L^{-2} \cdot T^{3} \cdot I^{2}$
6-52.2	admittance of vacuum	Y_0	A/V	$M^{-1} \cdot L^{-2} \cdot T^{3} \cdot I^{2}$
6-52.3	conductance	G	S	$M^{-1} \cdot L^{-2} \cdot T^{3} \cdot I^{2}$
6-52.4	susceptance	B	S	$M^{-1} \cdot L^{-2} \cdot T^{3} \cdot I^{2}$
6-52.5	apparent admittance	Y	S	$M^{-1} \cdot L^{-2} \cdot T^{3} \cdot I^{2}$
6-53	quality factor	Q	1	1
6-54	loss factor	d	1	1
6-55	loss angle	δ	rad	1
6-56	active power	P	W	$M \cdot L^{2} \cdot T^{-3}$
6-57	apparent power	S	VA	$M \cdot L^{2} \cdot T^{-3}$
6-58	power factor	λ	1	1
6-59	complex power	S	VA	$M \cdot L^{2} \cdot T^{-3}$
6-60	reactive power	Q	var	$M \cdot L^{2} \cdot T^{-3}$
6-61	non-active power	Q'	VA	$M \cdot L^{2} \cdot T^{-3}$

Item	Quantity	Symbol	Unit	Dimension
6-62	active energy	W	J	$M \cdot L^2 \cdot T^{-2}$

25. ISO 80000-7 — Light and radiation

The fifth part specified in full is ISO 80000-7, *Light and radiation*. Every item — all of 7-1.1 … 7-37, including every sub-suffixed item — is catalogued: the speed of light and refractive index, radiant energy and its densities, radiant flux, intensity, and radiance, irradiance, exitance, and exposure, the luminous efficiency and efficacy, the luminous quantities, the photon quantities, colorimetry, colour temperature, the optical material properties, and the attenuation coefficients — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced. The defining *mathematics* of selected remarks is specified in the sibling module `Part7.DefiningRelations`.

Light and radiation is where *dimension does not classify the kind* reaches its widest form, and where the catalogue meets two SI base quantities PhysLib's five-generator `Dimension` does not carry — luminous intensity and amount of substance. Both are handled by the Finkelstein–Whitehead *scale-spanning* reduction (the *Scale-spanning units* chapter, requirement *R13*), not by a new generator: the candela is the dimension of *power*, the steradian and the mole are dimension one. So the luminous quantities share the dimensions of their radiant partners — luminous flux ≡ radiant flux, luminance ≡ radiance, illuminance ≡ irradiance — while staying distinct kinds, and a molar quantity drops the mole.

25.1. Quantity-kinds and units of ISO 80000-7

Definition 25.1.1. group statement">us

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "7-6.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream.

Lean code for Definition 25.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 25.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-7, Second edition, 2019-08 item 7-6.1`. The `catalogue` lists all 66 items in item order — the speed of light (7-1.1), radiant flux (7-4.1), luminous intensity (7-14), the photon quantities (7-19.1 … 7-25), the molar absorption coefficient (7-37), … — each paired with its coherent SI unit symbol.

Theorem 25.1.2. group uses 2 used by 0 ✓

Effort	Tags
small	proved

> ***The candela reduces to power (item 7-14).*** *Luminous intensity carries the dimension of power, `M·L²·T⁻³` — the scale-spanning reduction (R13) made a checked computation: PhysLib has no luminous-intensity generator, and none is needed, the spectral luminous efficiency `V(λ)` that relates the candela to the watt being dimensionless. Uses the dimension map and the scale-spanning units.*

Lean code for Theorem 25.1.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part7.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part7.luminousIntensity_dim_eq_power :
  PropertyKindCalculus.Iso80000.Part7.luminousIntensity.dim
      LTMCTDimensionBase =
    PropertyKindCalculus.Dim.power
```

Proof for Theorem 25.1.2 uses 0

`luminousIntensity_dim_eq_power : luminousIntensity.dim = Dim.power`, by reflexivity over `Dim.luminousIntensity := Dim.power`. The companions `radiantFlux_dim_{mass,length,time}` record `M·L²·T⁻³` for radiant flux, `radiantEnergy_dim_time = -2` the joule, `photonFlux_dim_time = -1` the photon count rate, and `molarAbsorptionCoefficient_dim_length = 2` the molar coefficient as an area (the mole reduced).

25.2. The radiant / luminous / photon trios

ISO 80000-7 lists most measurands three times — a *radiant* (energetic, subscript e) version, a *luminous* (photopic, subscript v) version, and a *photon* (count, subscript p) version — and pairs them explicitly ("the corresponding photometric quantity is …"). They are not the same kind of quantity: radiant flux is the total radiant power, luminous flux its `V(λ)`-weighted response in the human eye, photon flux a count rate. A dimension-only model sees one type, "a real number of watts", for radiant and luminous flux, and cannot say why a count rate is a different thing again.

This is requirement *R2* once more — the pattern Part 3 set on the length family, Part 4 on the force family, Part 5 on the thermodynamic potentials, Part 6 on the AC power quantities — now on the radiation trios. Each member is a distinct *kind* individuated *not by fiat* but by a *radiation mode* (Dybkær's examination principle, §7.5). And the trios split two ways: the luminous mode is a *dimensionless*

reweighting (luminous flux ≡ radiant flux in dimension), while the photon mode is a genuine *re-dimensioning* (photon flux is `T⁻¹`, not `M·L²·T⁻³`).

Definition 25.2.1. group uses 2 used by 0 ✓

A *radiation-mode kind* is a ratio-scale quantity-kind carrying an examination principle — this work's terse descriptor of which aspect of the radiation it isolates: the total energetics (subscript e), the `V(λ)`-weighted photopic response (subscript v), or a photon count (subscript p). The radiant, luminous, and photon members of each measurand's trio differ in this principle.

Lean code for Definition 25.2.1. 1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part7.lean` complete

```
def PropertyKindCalculus.Iso80000.Part7.modeKind (id : String)
  (m : PropertyKindCalculus.ExaminationPrinciple)
  (dim : Dimension LTMCTDimensionBase) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase
```

Proof for Definition 25.2.1 uses 0

`modeKind id m dim := { kind := { id, scale := .ratio, examPrinciple := some m.id }, dim }`, with the three modes `RadiationMode.energetic`, `.photopic`, `.photonic`. The flux trio is `radiantFlux` / `luminousFlux` / `photonFlux`, and likewise for energy, intensity, radiance, irradiance, exitance, and exposure.

Theorem 25.2.2. group statement">uses 1 by 0 ✓

Effort	Tags
small	proved

> ***Radiant and luminous flux are distinct kinds — by radiation mode, not by fiat.*** *They share dimension `M·L²·T⁻³` and are both "flux", yet are different kinds because one is the total radiant power and the other its `V(λ)`-weighted photopic response — proved through `distinct_of_examPrinciple` (§7.5). The photon member is distinct again, and even re-dimensions (`T⁻¹`). This is the AC-power-family pattern of Part 6 on the radiation trios. Uses the examination principle.*

Lean code for Theorem 25.2.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part7.lean` complete

```
theorem PropertyKindCalculus.Iso80000.Part7.radiantFlux_ne_luminousFlux :
  PropertyKindCalculus.Iso80000.Part7.radiantFlux.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part7.luminousFlux.kind
      LTMCTDimensionBase
```

Proof for Theorem 25.2.2

uses 0

`radiantFlux_ne_luminousFlux : radiantFlux.kind ≠ luminousFlux.kind`, from `KindOfProperty.distinct_of_examPrinciple` on the differing modes `radiant-energetic` and `luminous-photopic-Vlambda`. The companions `radiantFlux_ne_photonFlux` (the photon mode) and `radiance_ne_luminance` cover further trios; `radiantFlux_examinedBy` and `luminousFlux_examinedBy` check the kind-to-mode links. Axiom-free.

Theorem 25.2.3.

group uses 2 used by 0 ✓

Effort	Tags
small	proved

The luminous efficacy is dimension one because it is a ratio of two fluxes (item 7-11.1). *From `K = Φ_v/Φ_e`, with luminous and radiant flux of equal dimension (the candela reducing to power), the dimension cancels and the efficacy is computed to be one — the photometric analogue of the power factor (Part 6) and the plane angle (Part 3). The scale coefficient `K_cd = 683 lm/W` is the candela's scale-spanning constant (R13). Uses the dimension map and the quotient kind-law.*

Lean code for Theorem 25.2.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part7/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part7.DefiningRelations.luminousEfficacy_d
  PropertyKindCalculus.Iso80000.Part7.luminousEfficacy.dim
      LTMCTDimensionBase =
    PropertyKindCalculus.Iso80000.Part7.luminousFlux.dim
        LTMCTDimensionBase /
      PropertyKindCalculus.Iso80000.Part7.radiantFlux.dim
        LTMCTDimensionBase
```

Proof for Theorem 25.2.3

uses 0

`luminousEfficacy_dim_from_fluxes : luminousEfficacy.dim = luminousFlux.dim / radiantFlux.dim`, by reducing the right side via `div_self` (`a / a = 1`). The kind-law

> `luminousEfficacy_quot_fluxes` is `⟨rfl, rfl, rfl⟩`; the efficacy nonetheless remains a distinct kind from every other dimension-one quantity (the collision capstone below).

25.3. Dimension does not classify; the kind does — the widest collisions in the series

ISO 80000-7 holds the widest dimension collisions of the catalogue. The candela reducing to power and the steradian to one, *radiant flux, luminous flux, radiant intensity, and luminous intensity all share* `M·L²·T⁻³`; radiance, luminance, irradiance, illuminance, and the exitances all share `M·T⁻³`; and over two dozen kinds — refractive index, the efficiencies and efficacies, emissivity, the absorptances, reflectances, and transmittances, the optical densities, the radiance and luminance factors, the chromaticity coordinates, the colour-matching functions, the photon number — share dimension one.

Theorem 25.3.1. group statement">us

Effort	Tags
small	proved

> ***Radiant flux is not luminous flux, though both are `M·L²·T⁻³`.*** *There exist distinct ISO 80000-7 kinds with the same dimension — radiant and luminous flux witness it, alongside radiant and luminous intensity. The candela reduces to power, so no dimension-only type system can separate them; the kind layer — and the unit, `W` versus `lm` — does. This is a collision between a base-quantity unit and a derived one, the scale-spanning case (R13) on the standard. Uses the dimension map.*

Lean code for Theorem 25.3.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part7.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part7.iso80000_7_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 25.3.1 uses 0

> `iso80000_7_dim_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim`, witnessed by `⟨radiantFlux, luminousFlux, …⟩`. The companion `radiantFlux_dim_eq_radiantIntensity_dim` is the *steradian* collision (flux versus intensity, the steradian being dimension one), and `wattRadiant_lumen_not_commensurable` / `wattRadiant_wattPerSteradian_not_commensurable` record the unit-level facts. Axiom-free.

Theorem 25.3.2. group statement">use

Effort	Tags
small	proved

> ***The dimension-1 disambiguation, on the standard — the widest in the series.*** *There exist distinct ISO 80000-7 kinds with the same dimension one — refractive index and emissivity witness it, alongside the efficiencies, efficacies, absorptances, reflectances, transmittances, optical densities, the radiance and luminance factors, the reflectance factor, the chromaticity coordinates, the colour-matching functions, and the photon number. Dimension cannot separate them; the kind layer does. Uses [the dimension map](#).*

Lean code for Theorem 25.3.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part7.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part7.iso80000_7_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 25.3.2 uses 0

`iso80000_7_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨refractiveIndex, emissivity, …⟩` with `refractiveIndex_ne_emissivity` (a `decide`) and reflexivity. Axiom-free.

25.4. The algebraic Remarks as kind-laws: radiometry and photometry built by the kind algebra

Many Part-7 definitions state a quantity's defining relation *algebraically* — the constitutive laws of radiometry and photometry. Radiant flux is the time-derivative of radiant energy (7-4.1); photon flux of photon number (7-20); irradiance is the area-density of radiant flux (7-7.1); radiant intensity its solid-angle-density (7-5.1); the luminous efficacy is luminous flux over radiant flux (7-11.1). Each is specified as an R12 kind-law, several crossing to ISO 80000-3 for time, area, and solid angle.

Theorem 25.4.1. statement">used by 0

Effort	Tags
small	proved

> ***Irradiance is radiant flux / area — a verified construction (item 7-7.1).*** *An irradiance built as the quotient of a radiant flux by an area carries its quotient certificate by construction, over the* `ℝ` *carrier, and canonicity makes the certificate determine the quantity. The companion* `radiantIntensity_dim_from_flux_solidAngle` *shows radiant intensity keeps the dimension of flux because the solid angle it divides by is dimension one — the steradian dropping, as a checked computation. Uses* [*the quotient kind-law*](#)*.*

Lean code for Theorem 25.4.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part7/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part7.DefiningRelations.irradianceOf_isQuo
  (φ :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part7.radiantFlux.kind
        LTMCTDimensionBase)
      ℝ)
  (a :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part3.area.kind LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsQuotient

PropertyKindCalculus.Iso80000.Part7.DefiningRelations.irradiance_quot_flux_are
a
    (PropertyKindCalculus.Iso80000.Part7.DefiningRelations.irradianceOf
      φ a)
    φ a
```

Proof for Theorem 25.4.1

uses 0

`irradianceOf φ a := Quantity.div irradiance_quot_flux_area φ a`, and `irradianceOf_isQuotient` is `rfl`; `irradiance_certificate_canonical` is `eq_div_of_isQuotient`. The same family gives `luminousEfficacyOf` (= `Φ_v/Φ_e`), and the cross-part `radiantFlux_quot_energy_duration` (= radiant energy / time) and `photonFlux_quot_number_duration` (= photon number / time), the time crossing to ISO 80000-3.

25.5. Item index — ISO 80000-7

Every catalogued item of ISO 80000-7, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it (over the radiation reductions). Each item number links to the formalized result it participates in — a trio distinction, a dimension collision, a defining relation, or the catalogue itself. Symbols and unit strings are *citation locators*; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
7-1.1	speed of light in a medium	c	$\mathrm{m/s}$	$\mathsf{L}\cdot\mathsf{T}^{-1}$
7-1.2	refractive index	n	1	1
7-2.1	radiant energy	Q_e	J	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$
7-2.2	spectral radiant energy	$Q_{e\lambda}$	$\mathrm{J/nm}$	$\mathsf{M}\cdot\mathsf{L}\cdot\mathsf{T}^{-2}$
7-3.1	radiant energy density	w	$\mathrm{J/m^3}$	$\mathsf{M}\cdot\mathsf{L}^{-1}\cdot\mathsf{T}^{-2}$
7-3.2	spectral radiant energy density in terms of wavelength	w_λ	$\mathrm{J/(m^3\cdot nm)}$	$\mathsf{M}\cdot\mathsf{L}^{-2}\cdot\mathsf{T}^{-2}$
7-3.3	spectral radiant energy density in terms of wavenumber	$w_{\tilde{\nu}}$	$\mathrm{J/m^2}$	$\mathsf{M}\cdot\mathsf{T}^{-2}$
7-4.1	radiant flux	Φ_e	W	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-3}$
7-4.2	spectral radiant flux	$\Phi_{e\lambda}$	$\mathrm{W/nm}$	$\mathsf{M}\cdot\mathsf{L}\cdot\mathsf{T}^{-3}$
7-5.1	radiant intensity	I_e	$\mathrm{W/sr}$	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-3}$
7-5.2	spectral radiant intensity	$I_{e\lambda}$	$\mathrm{W/(sr\cdot nm)}$	$\mathsf{M}\cdot\mathsf{L}\cdot\mathsf{T}^{-3}$
7-6.1	radiance	L_e	$\mathrm{W/(sr\cdot m^2)}$	$\mathsf{M}\cdot\mathsf{T}^{-3}$
7-6.2	spectral radiance	$L_{e\lambda}$	$\mathrm{W/(sr\cdot m^2\cdot nm)}$	$\mathsf{M}\cdot\mathsf{L}^{-1}\cdot\mathsf{T}^{-3}$
7-7.1	irradiance	E_e	$\mathrm{W/m^2}$	$\mathsf{M}\cdot\mathsf{T}^{-3}$
7-7.2	spectral irradiance	$E_{e\lambda}$	$\mathrm{W/(m^2\cdot nm)}$	$\mathsf{M}\cdot\mathsf{L}^{-1}\cdot\mathsf{T}^{-3}$
7-8.1	radiant exitance	M_e	$\mathrm{W/m^2}$	$\mathsf{M}\cdot\mathsf{T}^{-3}$
7-8.2	spectral radiant exitance	$M_{e\lambda}$	$\mathrm{W/(m^2\cdot nm)}$	$\mathsf{M}\cdot\mathsf{L}^{-1}\cdot\mathsf{T}^{-3}$
7-9.1	radiant exposure	H_e	$\mathrm{J/m^2}$	$\mathsf{M}\cdot\mathsf{T}^{-2}$
7-9.2	spectral radiant exposure	$H_{e\lambda}$	$\mathrm{J/(m^2\cdot nm)}$	$\mathsf{M}\cdot\mathsf{L}^{-2}\cdot\mathsf{T}^{-2}$
7-10.1	luminous efficiency	V	1	1
7-10.2	spectral luminous efficiency	$V(\lambda)$	1	1
7-11.1	luminous efficacy of radiation	K	$\mathrm{lm/W}$	1
7-11.2	spectral luminous efficacy	$K(\lambda)$	$\mathrm{lm/W}$	1
7-11.3	maximum luminous efficacy	K_m	$\mathrm{lm/W}$	1
7-11.4	luminous efficacy of a source	η_v	$\mathrm{lm/W}$	1
7-12	luminous energy	Q_v	$\mathrm{lm\cdot s}$	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$

Item	Quantity	Symbol	Unit	Dimension
7-13	luminous flux	Φ_v	lm	$M \cdot L^2 \cdot T^{-3}$
7-14	luminous intensity	I_v	cd	$M \cdot L^2 \cdot T^{-3}$
7-15	luminance	L_v	cd/m^2	$M \cdot T^{-3}$
7-16	illuminance	E_v	lx	$M \cdot T^{-3}$
7-17	luminous exitance	M_v	lm/m^2	$M \cdot T^{-3}$
7-18	luminous exposure	H_v	$lx \cdot s$	$M \cdot T^{-2}$
7-19.1	photon number	N_p	1	1
7-19.2	photon energy	Q_p	J	$M \cdot L^2 \cdot T^{-2}$
7-20	photon flux	Φ_p	s^{-1}	T^{-1}
7-21	photon intensity	I_p	$s^{-1} \cdot sr^{-1}$	T^{-1}
7-22	photon radiance	L_p	$m^{-2} \cdot s^{-1} \cdot sr^{-1}$	$L^{-2} \cdot T^{-1}$
7-23	photon irradiance	E_p	$m^{-2} \cdot s^{-1}$	$L^{-2} \cdot T^{-1}$
7-24	photon exitance	M_p	$m^{-2} \cdot s^{-1}$	$L^{-2} \cdot T^{-1}$
7-25	photon exposure	H_p	m^{-2}	L^{-2}
7-26.1	tristimulus values for the CIE 1931 standard observer	X, Y, Z	1	1
7-26.2	tristimulus values for the CIE 1964 standard observer	X_{10}, Y_{10}, Z_{10}	1	1
7-27.1	CIE colour-matching functions for the CIE 1931 standard observer	$\bar{x}$, $\bar{y}$, $\bar{z}$	1	1
7-27.2	CIE colour-matching functions for the CIE 1964 standard observer	$\bar{x}_{10}$, $\bar{y}_{10}$, $\bar{z}_{10}$	1	1
7-28.1	chromaticity coordinates in the CIE 1931 standard system	x, y, z	1	1
7-28.2	chromaticity coordinates in the CIE 1964 standard system	x_{10}, y_{10}, z_{10}	1	1
7-29.1	colour temperature	T_c	K	0
7-29.2	correlated colour temperature	T_{cp}	K	0
7-30.1	emissivity	ε	1	1
7-30.2	emissivity at a specified wavelength	$\varepsilon(\lambda)$	1	1

Item	Quantity	Symbol	Unit	Dimension
7-31.1	absorptance	α	1	1
7-31.2	luminous absorptance	α_v	1	1
7-31.3	reflectance	ρ	1	1
7-31.4	luminous reflectance	ρ_v	1	1
7-31.5	transmittance	τ	1	1
7-31.6	luminous transmittance	τ_v	1	1
7-32.1	transmittance optical density	D	1	1
7-32.2	Napierian absorbance	A_n	1	1
7-33.1	radiance factor	β_e	1	1
7-33.2	luminance factor	β_v	1	1
7-34	reflectance factor	R	1	1
7-35.1	linear attenuation coefficient	μ	m^{-1}	L^{-1}
7-35.2	linear absorption coefficient	a_l	m^{-1}	L^{-1}
7-36.1	mass attenuation coefficient	μ_m	$kg^{-1} \cdot m^2$	$M^{-1} \cdot L^2$
7-36.2	mass absorption coefficient	α_m	$kg^{-1} \cdot m^2$	$M^{-1} \cdot L^2$
7-37	molar absorption coefficient	χ	m^2/mol	L^2

26. ISO 80000-8 — Acoustics

The sixth part specified in full is ISO 80000-8, *Acoustics*. Every item — all of 8-1 … 8-17, including the sub-suffixed pressure pair (8-2.1, 8-2.2) — is catalogued: the logarithmic frequency range, static and sound pressure, the sound-particle kinematics (displacement, velocity, acceleration), volume flow rate, the sound-energy cluster (energy density, energy, power, intensity), sound exposure, the two impedances, the three logarithmic levels, and the reverberation time — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced. The defining *mathematics* of selected remarks is specified in the sibling module `Part8.DefiningRelations`.

Acoustics brings two faces of the catalogue's central thesis, and — unusually — its boundary. The *logarithmic levels* (sound pressure level, sound power level, sound exposure level) are each `10·lg(quantity / reference)`, ratios on a logarithmic scale reported in the decibel: *dimension one*, three distinct kinds the dimension functor collapses to a single point. A genuine *dimension collision* has sound pressure and sound energy density carry the same dimension $M \cdot L^{-1} \cdot T^{-2}$ — force per area equals energy per volume — while staying distinct kinds. And the *boundary*: the two homonymous *impedances* are told apart *by* their dimension, the case where the necessary discriminator is, this time, also sufficient.

26.1. Quantity-kinds and units of ISO 80000-8

Definition 26.1.1. statement">uses 1 used by 0

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "8-2.2"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream.

Lean code for Definition 26.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` — complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 26.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-8, Second edition, 2020-02 item 8-2.2`. The `catalogue` lists all 18 items in item order — the logarithmic frequency range (8-1), static and sound pressure (8-2.1, 8-2.2), the kinematics (8-3 … 8-5), the sound-

> energy cluster (8-7 … 8-10), the impedances (8-12, 8-13), the levels (8-14 … 8-16), … — each paired with its coherent SI unit symbol.

26.2. The logarithmic levels — the acoustic face of dimension one

ISO 80000-8 lists three *levels*: the sound pressure level (8-14), the sound power level (8-15), and the sound exposure level (8-16). Each is `10·lg(quantity / reference)` — a ratio of a quantity to a stated reference, taken on a logarithmic scale, and reported in the decibel. A level is *dimension one*: the logarithm of a ratio of like quantities. So the dimension functor sends all three to the single point `1`, and a dimension-only model sees one type, "a real number of decibels", for all of them.

What keeps them apart is *which quantity each is the level of* — the pressure, the power, the exposure. This is requirement *R1* once more (the dimension does not classify the kind), in the form the part makes vivid: a shared unit symbol `dB` no more makes the sound pressure level the sound power level than the shared symbol `1` makes a Reynolds number a Froude number (ISO 80000-11).

Theorem 26.2.1. group statement">us

Effort	Tags
small	proved

> ***The decibel of sound pressure level and the decibel of sound power level are not commensurable, though both are `dB` and dimension one (items 8-14, 8-15).*** *Same symbol, same dimension, different kind — the level of pressure is not the level of power. This is the ISO 80000-11 `"1"` pattern (Reynolds versus Euler) on the acoustic levels. Uses [the metrological unit](#).*

Lean code for Theorem 26.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part8.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part8.decibelPressure_decibelPower_not_cor
  ¬PropertyKindCalculus.Iso80000.Part8.decibelPressure.Commensurable
      PropertyKindCalculus.Iso80000.Part8.decibelPower
```

Proof for Theorem 26.2.1 uses 0

> `decibelPressure_decibelPower_not_commensurable`, by `decide` after unfolding `MetrologicalUnit.Commensurable` on the two dimension-one kinds. The companion

`soundPressureLevel_ne_soundPowerLevel` is the underlying kind distinction, and `iso80000_8_dim_one_collision` packages the dimension-one capstone. Axiom-free.

Theorem 26.2.2. group statement">us

Effort	Tags
small	proved

The dimension-1 disambiguation, on the standard. *There exist distinct ISO 80000-8 kinds with the same dimension one — the sound pressure level and the sound power level witness it, alongside the sound exposure level and the logarithmic frequency range. The dimension cannot separate them; the kind layer does. Uses the dimension map.*

Lean code for Theorem 26.2.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part8.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part8.iso80000_8_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 26.2.2 uses 0

`iso80000_8_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨soundPressureLevel, soundPowerLevel, …⟩` with the kind distinction a `decide` and reflexivity. Axiom-free.

26.3. A dimension collision, and the boundary where the dimension does classify

ISO 80000-8 shows both sides of the thesis. Sound pressure (8-2.2, Pa) and sound energy density (8-7, J/m³) carry the *same* dimension M·L⁻¹·T⁻² — pressure being force per area, energy density energy per volume, and the two equal — yet are different kinds: a collision the dimension cannot resolve.

But the part also marks the *boundary*. It lists two quantities both called an *impedance*: the characteristic impedance of a medium (8-12, Pa·s/m, M·L⁻²·T⁻¹) and the acoustic impedance (8-13, Pa·s/m³, M·L⁻⁴·T⁻¹). Here the dimension *does* tell the homonyms apart, because one is pressure over particle velocity and the other pressure over volume flow rate — the extra L² shows. Dimension is a *necessary*

discriminator of kind, never a *sufficient* one; the impedances are the case where, this once, it is also sufficient, and naming them makes the limit of the dimension test explicit.

Theorem 26.3.1. statement"> uses 1 used by 0

Effort: small

Tags: proved

> ***Sound pressure is not sound energy density, though both are `M·L⁻¹·T⁻²` (items 8-2.2, 8-7).*** *There exist distinct ISO 80000-8 kinds with the same dimension — sound pressure and sound energy density witness it. Force per area equals energy per volume, so no dimension-only type system can separate them; the kind layer — and the unit, `Pa` versus `J/m³` — does. Uses [the dimension map](#).*

Lean code for Theorem 26.3.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part8.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part8.iso80000_8_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 26.3.1 uses 0

`iso80000_8_dim_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim`, witnessed by `⟨soundPressure, soundEnergyDensity, …⟩`. The dimension equality `soundPressure_dim_eq_energyDensity_dim` is a `Dimension.ext` computation (force/area = energy/volume); `pascalSound_joulePerCubicMetre_not_commensurable` records the unit-level fact. The contrasting `acousticImpedance_dim_from_pressure_volumeFlow` shows where the dimension instead discriminates. Axiom-free.

26.4. The algebraic Remarks as kind-laws: linear acoustics built by the kind algebra

Several Part-8 definitions state a quantity's defining relation *algebraically* — the constitutive laws of linear acoustics. Sound particle velocity is the time-derivative of displacement (8-4), acceleration of velocity (8-5); sound intensity is the *product* of sound pressure and particle velocity (8-10, `I = p·u`); the two impedances are quotients of sound pressure (8-12, 8-13). Each is specified as an R12 kind-law, the kinematics crossing to ISO 80000-3 for time.

Theorem 26.4.1. statement">uses 1 · used by 0

Effort	Tags
small	proved

> ***Sound intensity is sound pressure × particle velocity — a verified construction (item 8-10).*** *A sound intensity built as the product of a sound pressure and a particle velocity carries its product certificate by construction, over the `ℝ` carrier. The companion `soundIntensity_dim_from_pressure_velocity` shows the intensity is `M·T⁻³` because the length in pressure and velocity cancels — the acoustic intensity law as a checked computation. Uses the quotient kind-law.*

Lean code for Theorem 26.4.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part8/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part8.DefiningRelations.soundIntensityOf_
  (p :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part8.soundPressure.kind
        LTMCTDimensionBase)
      ℝ)
  (u :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part8.particleVelocity.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part8.DefiningRelations.soundIntensity_prod_pres
sure_velocity
    (PropertyKindCalculus.Iso80000.Part8.DefiningRelations.soundIntensityOf
      p u)
    p u
```

Proof for Theorem 26.4.1

uses 0

`soundIntensityOf p u := Quantity.mul soundIntensity_prod_pressure_velocity p u`, and `soundIntensityOf_isProduct` is `rfl`. The same family gives the quotient constructors `characteristicImpedanceOf` (= `p/u`) and the cross-part kinematic laws

> `particleVelocity_quot_displacement_duration` (= displacement / time), the time crossing to ISO 80000-3.

26.5. Item index — ISO 80000-8

Every catalogued item of ISO 80000-8, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in — a dimension collision, a dimension-one level, a defining relation, or the catalogue itself. Symbols and unit strings are *citation locators*; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
8-1	logarithmic frequency range	G	oct	1
8-2.1	static pressure	p_s	Pa	M·L⁻¹·T⁻²
8-2.2	sound pressure	p	Pa	M·L⁻¹·T⁻²
8-3	sound particle displacement	δ	m	L
8-4	sound particle velocity	u	m/s	L·T⁻¹
8-5	sound particle acceleration	a	m/s²	L·T⁻²
8-6	volume flow rate	q_V	m³/s	L³·T⁻¹
8-7	sound energy density	w	J/m³	M·L⁻¹·T⁻²
8-8	sound energy	Q	J	M·L²·T⁻²
8-9	sound power	P	W	M·L²·T⁻³
8-10	sound intensity	I	W/m²	M·T⁻³
8-11	sound exposure	E	Pa²·s	M²·L⁻²·T⁻³
8-12	characteristic impedance of a medium	Z_c	Pa·s/m	M·L⁻²·T⁻¹
8-13	acoustic impedance	Z_a	Pa·s/m³	M·L⁻⁴·T⁻¹
8-14	sound pressure level	L_p	dB	1
8-15	sound power level	L_W	dB	1
8-16	sound exposure level	L_E	dB	1
8-17	reverberation time	T	s	T

27. ISO 80000-9 — Physical chemistry and molecular physics

The seventh part specified in full is ISO 80000-9, *Physical chemistry and molecular physics*. Every item — all of 9-1 … 9-49, sixty-two including every sub-suffixed item — is catalogued: counting entities and the mole, the molar quantities, the concentrations and fractions, the chemical potential and the activity ladder, the reaction and equilibrium quantities, statistical mechanics, and the transport and electrochemical coefficients — each carrying its exact source as data. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced. The defining *mathematics* of selected remarks is specified in the sibling module `Part9.DefiningRelations`.

Physical chemistry is the home of the *mole*, and so the sharpest test in the series of the Finkelstein–Whitehead *scale-spanning* reduction (the *Scale-spanning units* chapter, requirement *R13*): the mole is a human-selected dimensionless count of entities, `N_A` of them, so `Dim.amountOfSubstance = 1`, and every quantity *per mole* drops the mole. One reduction reorganizes the whole part. Amount of substance joins the dimensionless family. Molar mass *is a mass*, molar volume *a volume*, the amount-of-substance concentration *a number density*, molality *an inverse mass*. The seven `J/mol` quantities collide on one dimension; the three `J/(mol·K)` quantities on another.

27.1. Quantity-kinds and units of ISO 80000-9

Definition 27.1.1. statement">uses 1 used by 0

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "9-12.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream.

Lean code for Definition 27.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 27.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-9, Second edition, 2019-08 item 9-12.1`. The `catalogue` lists all 62 items in item order — the mole (9-2), the molar quantities (9-4 … 9-8), the concentrations (9-9.1 … 9-12.2), the activities (9-18 … 9-27.3),

> the partition functions (9-35.1 … 9-35.4), the conductivities (9-44, 9-45), … — each paired with its coherent SI unit symbol.

27.2. The mole reduces to dimension one — R13 at its sharpest

ISO 80000-9 is built on the *mole*. In the scale-spanning reading (R13), the mole is not a base dimension but a human-selected dimensionless *count*: `N_A` entities. So amount of substance is *dimension one*, and the `/mol` in every molar quantity is invisible to the dimension functor. Molar mass (`kg/mol`) carries the dimension of *mass*; molar volume (`m³/mol`) the dimension of *volume*; the amount-of-substance concentration (`mol/m³`) the dimension of a *number density* `L⁻³` — the very same dimension as a particle concentration; molality (`mol/kg`) the dimension of an *inverse mass*.

And amount of substance itself joins the dimensionless family, distinct in kind from the number of entities it counts (`N = n·N_A`) — both dimension one, two kinds. This is requirement *R13* in the place that makes it unavoidable: a `mol` is no more a base unit than a `cd` (ISO 80000-7) or a `Sh` (IEC 80000-13).

Theorem 27.2.1. group uses 2 used by 0 ✓

Effort: small

Tags: capstone, proved

> ***Molar mass is a mass — the mole reduces (item 9-4).*** *With the mole dimension one, `kg/mol` carries the dimension of mass, `M`. The companion facts record the rest of the reduction: the amount-of-substance concentration is a number density (equal to the particle concentration), molality is an inverse mass, and amount of substance is itself dimension one. This is the scale-spanning reduction (R13) at the place the mole lives. Uses the dimension map and the scale-spanning units.*

Lean code for Theorem 27.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part9.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part9.molarMass_dim_eq_mass :
  PropertyKindCalculus.Iso80000.Part9.molarMass.dim LTMCTDimensionBase =
    PropertyKindCalculus.Dim.mass
```

Proof for Theorem 27.2.1 uses 0

`molarMass_dim_eq_mass : molarMass.dim = Dim.mass`, by reflexivity (the kind is declared at `Dim.mass`). The companions `amountOfSubstance_dim_eq_one`,

> `amountConcentration_dim_eq_particleConcentration`, and `molality_dim_eq_inverseMass` record the rest; the defining-relation versions (`molarMass_dim_from_mass_amount`, `amountConcentration_dim_from_amount_volume`) derive the reduction *from* `M = m/n` and `c = n/V` with `dim n = 1`. Axiom-free.

Theorem 27.2.2. group statement">use

Effort	Tags
small	proved

> ***The dimension-1 disambiguation, on the standard — with the mole inside it.*** *There exist distinct ISO 80000-9 kinds with the same dimension one — amount of substance (the mole, reduced by R13) and the number of entities witness it, alongside the fractions, the activities, and the partition functions. Dimension cannot separate them; the kind layer does. Uses the dimension map.*

Lean code for Theorem 27.2.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part9.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part9.iso80000_9_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 27.2.2 uses 0

> `iso80000_9_dim_one_collision`, witnessed by `⟨amountOfSubstance, numberOfEntities, …⟩` with the kind distinction a `decide` and reflexivity. Axiom-free.

27.3. The seven-fold `J/mol` collision

The mole reducing to dimension one, the seven energies-per-mole — molar internal energy, molar enthalpy, molar Helmholtz and Gibbs energies (9-6.1 … 9-6.4), the chemical potential (9-17), the standard chemical potential (9-21), and the affinity of a reaction (9-30) — all carry the *same* dimension `M·L²·T⁻²` and the *same* unit string `J/mol`, yet are seven distinct kinds. A dimension-only model sees one type, "a real number of joules per mole", for all seven; the kind layer keeps the chemical potential from being read as an affinity. Likewise the molar heat capacity (9-7), molar entropy (9-8), and molar gas constant (9-37.1) all share `J/(mol·K)`.

Theorem 27.3.1. statement">used by 0

Effort	Tags
small	proved

> ***Molar internal energy is not molar Gibbs energy, though both are `J/mol` and `M·L²·T⁻²` (items 9-6.1, 9-6.4).*** *There exist distinct ISO 80000-9 kinds with the same dimension — seven `J/mol` quantities witness it. The mole reduces to dimension one, so no dimension-only type system can separate them; the kind layer — and the unit — does. Uses the dimension map.*

Lean code for Theorem 27.3.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part9.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part9.iso80000_9_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 27.3.1

uses 0

`iso80000_9_dim_collision`, witnessed by `(molarInternalEnergy, molarGibbsEnergy, …)`. The unit fact `jPerMol_internalEnergy_gibbs_not_commensurable` is a `decide`; `molarHeatCapacity_dim_eq_molarEntropy_dim` records the parallel `J/(mol·K)` collision, and `molPerKg_molality_ionicStrength_not_commensurable` the `mol/kg` one. Axiom-free.

27.4. The algebraic Remarks as kind-laws: physical chemistry built from the other parts

The molar quantities of ISO 80000-9 are defined as quotients *by the amount of substance* — molar mass is `m/n` (9-4), molar volume `V/n` (9-5), the amount concentration `n/V` (9-12.1), molality `n/m` (9-15), the molar energies `U/n` (9-6.1) — composing Part-9 kinds out of ISO 80000-3 (volume), -4 (mass), and -5 (energy) kinds and the mole. Each is specified as an R12 kind-law.

Theorem 27.4.1. statement">uses 1 by 0

Effort	Tags
small	proved

> ***Molar mass is mass / amount of substance — a verified construction (item 9-4).*** *A molar mass built as the quotient of a mass by an amount of substance carries its quotient certificate by*

> *construction, over the `ℝ` carrier, and canonicity makes the certificate determine the quantity. The companion `molarMass_dim_from_mass_amount` shows the result is a mass because the mole is dimension one. Uses [the quotient kind-law](#).*

Lean code for Theorem 27.4.1.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part9/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part9.DefiningRelations.molarMassOf_isQuot
  (m :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part4.mass.kind LTMCTDimensionBase)
      ℝ)
  (n :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part9.amountOfSubstance.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsQuotient

PropertyKindCalculus.Iso80000.Part9.DefiningRelations.molarMass_quot_mass_amou
nt
    (PropertyKindCalculus.Iso80000.Part9.DefiningRelations.molarMassOf m
      n)
    m n
```

Proof for Theorem 27.4.1

uses 0

`molarMassOf m n := Quantity.div molarMass_quot_mass_amount m n`, and `molarMassOf_isQuotient` is `rfl`; `molarMass_certificate_canonical` is `eq_div_of_isQuotient`. The same family gives `amountConcentrationOf` (= `n/V`) and the cross-part kind-laws `molarVolume_quot_volume_amount`, `molarInternalEnergy_quot_energy_amount`, and `molality_quot_amount_mass`, crossing to ISO 80000-3/-4/-5.

27.5. Item index — ISO 80000-9

Every catalogued item of ISO 80000-9, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it (the mole reduced). Each item number links to the formalized result it participates in. Symbols and unit strings are *citation locators*; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
9-1	number of entities	N	1	1
9-2	amount of substance	n	mol	1
9-3	relative atomic mass	A_r	1	1
9-4	molar mass	M	kg/mol	M
9-5	molar volume	V_m	m^3/mol	L^3
9-6.1	molar internal energy	U_m	J/mol	$M·L^2·T^{-2}$
9-6.2	molar enthalpy	H_m	J/mol	$M·L^2·T^{-2}$
9-6.3	molar Helmholtz energy	F_m	J/mol	$M·L^2·T^{-2}$
9-6.4	molar Gibbs energy	G_m	J/mol	$M·L^2·T^{-2}$
9-7	molar heat capacity	C_m	J/(mol·K)	$M·L^2·T^{-2}·Θ^{-1}$
9-8	molar entropy	S_m	J/(mol·K)	$M·L^2·T^{-2}·Θ^{-1}$
9-9.1	particle concentration	n	m^{-3}	L^{-3}
9-9.2	molecular concentration	C	m^{-3}	L^{-3}
9-10	mass concentration	ρ	kg/m^3	$M·L^{-3}$
9-11	mass fraction	w	1	1
9-12.1	amount-of-substance concentration	c	mol/m^3	L^{-3}
9-12.2	standard amount-of-substance concentration	c°	mol/m^3	L^{-3}
9-13	mole fraction	x	1	1
9-14	volume fraction	φ	1	1
9-15	molality	b	mol/kg	M^{-1}
9-16	latent heat of phase transition	C_{pt}	J	$M·L^2·T^{-2}$
9-17	chemical potential	μ	J/mol	$M·L^2·T^{-2}$
9-18	absolute activity	λ	1	1
9-19	partial pressure	p	Pa	$M·L^{-1}·T^{-2}$
9-20	fugacity	$\tilde{p}$	Pa	$M·L^{-1}·T^{-2}$
9-21	standard chemical potential	$μ^\circ$	J/mol	$M·L^2·T^{-2}$
9-22	activity factor	f	1	1
9-23	standard absolute activity in a mixture	$λ^\circ$	1	1

Item	Quantity	Symbol	Unit	Dimension
9-24	activity of solute	a	1	1
9-25	activity coefficient	γ	1	1
9-26	standard absolute activity in a solution	λ°	1	1
9-27.1	activity of solvent	a_A	1	1
9-27.2	osmotic coefficient of solvent	φ	1	1
9-27.3	standard absolute activity of solvent	λ_A°	1	1
9-28	osmotic pressure	Π	Pa	M·L⁻¹·T⁻²
9-29	stoichiometric number	ν	1	1
9-30	affinity of a chemical reaction	A	J/mol	M·L²·T⁻²
9-31	extent of reaction	ξ	mol	1
9-32	standard equilibrium constant	K°	1	1
9-33	equilibrium constant (pressure basis)	K_p	Pa^(Σν)	1
9-34	equilibrium constant (concentration basis)	K_c	(mol/m³)^(Σν)	1
9-35.1	microcanonical partition function	Ω	1	1
9-35.2	canonical partition function	Z	1	1
9-35.3	grand-canonical partition function	Ξ	1	1
9-35.4	molecular partition function	q	1	1
9-36.1	statistical weight of subsystem	g	1	1
9-36.2	degeneracy	g	1	1
9-37.1	molar gas constant	R	J/(mol·K)	M·L²·T⁻²·Θ⁻¹
9-37.2	specific gas constant	R_s	J/(kg·K)	L²·T⁻²·Θ⁻¹
9-38	mean free path	l	m	L
9-39	diffusion coefficient	D	m²/s	L²·T⁻¹
9-40.1	thermal diffusion ratio	k_T	1	1
9-40.2	thermal diffusion factor	α_T	1	1
9-41	thermal diffusion coefficient	D_T	m²/s	L²·T⁻¹
9-42	ionic strength	I	mol/kg	M⁻¹
9-43	degree of dissociation	α	1	1

Item	Quantity	Symbol	Unit	Dimension
9-44	electrolytic conductivity	κ	S/m	$M^{-1} \cdot L^{-3} \cdot T^{3} \cdot I^{2}$
9-45	molar conductivity	Λ_m	$S \cdot m^{2}/mol$	$M^{-1} \cdot L^{-1} \cdot T^{3} \cdot I^{2}$
9-46	transport number of the ion	t	1	1
9-47	angle of optical rotation	α	rad	1
9-48	molar optical rotatory power	α_n	$rad \cdot m^{2}/mol$	L^{2}
9-49	specific optical rotatory power	α_m	$rad \cdot m^{2}/kg$	$M^{-1} \cdot L^{2}$

28. ISO 80000-10 — Atomic and nuclear physics

The eighth part specified in full is ISO 80000-10, *Atomic and nuclear physics*. Every item — all of 10-1.1 … 10-89, one hundred and twenty-five including every sub-suffixed item — is catalogued: the nucleon numbers and masses, the atomic constants and magnetic moments, the quantum numbers, the precession frequencies, the decay and activity quantities, the cross sections, the fluences, the attenuation and stopping powers, the transport-in-matter and reactor quantities, and the dosimetry family — each carrying its exact source as data. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced. The defining *mathematics* of selected remarks is specified in the sibling module `Part10.DefiningRelations`.

Atomic and nuclear physics holds the series' clearest external confirmation of the *dimension does not classify the kind* thesis — the standard itself coins *two unit names for one dimension*. The absorbed dose, specific energy imparted, and kerma (all the gray) and the dose equivalent (the sievert) all carry dimension $L^2 \cdot T^{-2}$ (energy per mass), and the standard separates them by unit name *because the kinds differ*: the gray for the physical doses, the sievert for the biologically-weighted one. This is the entropy-versus-heat-capacity `J/K` collision of ISO 80000-5, made sharper. A second case is the becquerel, the special name SI reserves for s^{-1} *as the unit of activity*, a kind the dimension cannot tell from the decay constant.

28.1. Quantity-kinds and units of ISO 80000-10

Definition 28.1.1. statement">uses 1 used by 0

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "10-81.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream.

Lean code for Definition 28.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` — complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 28.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-10, Second edition, 2019-08 item 10-81.1`. The `catalogue` lists all 125 items in item order — the nucleon numbers (10-1.1 … 10-1.3), the activity (10-27), the cross sections (10-38.1 … 10-42.2), the dosimetry family (10-80.1 … 10-89), … — each paired with its coherent SI unit symbol.

28.2. The gray/sievert collision — the standard's own `J/kg` disambiguation

Four quantities of ISO 80000-10 carry the dimension `L²·T⁻²` (energy per mass): the absorbed dose (10-81.1), the specific energy imparted (10-81.2), the kerma (10-86.1) — all in *grays* — and the dose equivalent (10-83.1) — in *sieverts*. The gray and the sievert are the *same* coherent unit, `J/kg`; the standard gives them different special names *precisely because the kinds differ*: the physical energy deposited per mass, versus that energy weighted for biological effect.

This is requirement *R1* (the dimension does not classify the kind) confirmed by the standard's own hand. A dimension-only model sees one type, "a real number of joules per kilogram", for absorbed dose and dose equivalent — and would let a gray be silently read as a sievert, the exact error the two unit names exist to prevent. It is the entropy-versus-heat-capacity `J/K` collision of ISO 80000-5, made sharper: here the collision is between two *named* units, not two uses of one.

Theorem 28.2.1. group statement">uses 1 by 0 ✓

Effort: small

Tags: capstone, proved

> ***Absorbed dose is not dose equivalent, though both are `J/kg` and `L²·T⁻²` (items 10-81.1, 10-83.1).*** *There exist distinct ISO 80000-10 kinds with the same dimension — absorbed dose and dose equivalent witness it, alongside the specific energy imparted and the kerma. The standard assigns two unit names (gray, sievert) to one dimension; no dimension-only type system can separate them, the kind layer — and the unit — does. Uses the dimension map.*

Lean code for Theorem 28.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part10.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part10.iso80000_10_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 28.2.1 uses 0

`iso80000_10_dim_collision`, witnessed by `⟨absorbedDose, doseEquivalent, …⟩`. The unit fact `gray_sievert_not_commensurable` is a `decide`. Axiom-free.

Theorem 28.2.2. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***The becquerel of activity and the reciprocal second of the decay constant are not commensurable, though both are T^{-1} (items 10-27, 10-24).*** *The becquerel is the special name SI reserves for s^{-1} as the unit of activity — a kind distinction the dimension cannot see, shared also with the particle emission rate and the Larmor and cyclotron frequencies. Uses [the metrological unit](#).*

Lean code for Theorem 28.2.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part10.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part10.becquerel_perSecond_not_commensural
  ¬PropertyKindCalculus.Iso80000.Part10.becquerel.Commensurable
      PropertyKindCalculus.Iso80000.Part10.perSecondDecay
```

Proof for Theorem 28.2.2 uses 0

`becquerel_perSecond_not_commensurable`, by `decide` after unfolding `MetrologicalUnit.Commensurable`. The underlying kind distinction is `activity_ne_decayConstant`, and `activity_dim_eq_decayConstant_dim` records the shared dimension. Axiom-free.

28.3. The dimension-one family — the widest in the physical parts

ISO 80000-10 carries the largest dimension-one family of the physical parts: the atomic, neutron, and nucleon numbers; the eight quantum numbers (10-13.1 … 10-13.8); the two g-factors; the relative mass excess and defect; the packing and binding fractions; the internal conversion and quality factors; the total ionization; and the reactor factors (resonance escape, fast fission, thermal utilization, non-leakage, multiplication). All share the dimension one; what tells them apart is the kind.

Theorem 28.3.1. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***The dimension-1 disambiguation, on the standard — the widest in the physical parts.*** *There exist distinct ISO 80000-10 kinds with the same dimension one — the atomic and neutron numbers*

witness it, alongside the quantum numbers, the g-factors, and the reactor factors. Dimension cannot separate them; the kind layer does. Uses the dimension map.

Lean code for Theorem **28.3.1.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part10.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part10.iso80000_10_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 28.3.1 uses 0

`iso80000_10_dim_one_collision`, witnessed by `⟨atomicNumber, neutronNumber, …⟩` with the kind distinction a `decide` and reflexivity. Axiom-free.

28.4. The algebraic Remarks as kind-laws: dosimetry built by the kind algebra

Several Part-10 definitions state a quantity's defining relation *algebraically*. Dose equivalent is the *product* of absorbed dose with the dimensionless quality factor (10-83.1, `H = D·Q`); mean life is the *reciprocal* of the decay constant (10-25, `τ = 1/λ`); specific activity is activity per mass (10-28); the absorbed-dose rate is absorbed dose per time (10-84); the mass attenuation coefficient is linear attenuation per mass density (10-50). Each is specified as an R12 kind-law, several crossing to ISO 80000-3 and -4.

Theorem **28.4.1.** statement"> used by 0

Effort	Tags
small	proved

Dose equivalent is absorbed dose × quality factor — a verified construction (item 10-83.1). *A dose equivalent built as the product of an absorbed dose and the dimensionless quality factor carries its product certificate by construction, over the `ℝ` carrier. The companion `doseEquivalent_dim_from_dose_quality` shows it keeps the dimension of absorbed dose `L²·T⁻²` because the quality factor is dimension one — the gray/sievert collision as a checked computation. Uses the quotient kind-law.*

Lean code for Theorem **28.4.1.** 1 theorem

theorem · defined in `PropertyKindCalculus/Iso80000/Part10/DefiningRelations.lean` complete

```
theorem PropertyKindCalculus.Iso80000.Part10.DefiningRelations.doseEquivalentOf_
  (d :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part10.absorbedDose.kind
        LTMCTDimensionBase)
      ℝ)
  (q :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part10.qualityFactor.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part10.DefiningRelations.doseEquivalent_prod_dos
e_quality
    (PropertyKindCalculus.Iso80000.Part10.DefiningRelations.doseEquivalentOf
      d q)
    d q
```

Proof for Theorem 28.4.1 uses 0

> `doseEquivalentOf d q := Quantity.mul doseEquivalent_prod_dose_quality d q`, and `doseEquivalentOf_isProduct` is `rfl`. The same families give `specificActivityOf` (= `A/m`) and the reciprocal/quotient kind-laws `meanLife_recip_decayConstant`, `absorbedDoseRate_quot_dose_duration`, and `massAttenuation_quot_linear_density`, crossing to ISO 80000-3 and -4.

28.5. Item index — ISO 80000-10

Every catalogued item of ISO 80000-10, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in. Symbols and unit strings are *citation locators*; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
10-1.1	atomic number	Z	1	1
10-1.2	neutron number	N	1	1
10-1.3	nucleon number	A	1	1
10-2	rest mass	m	kg	M
10-3	rest energy	E_0	J	$M \cdot L^2 \cdot T^{-2}$
10-4.1	atomic mass	m_a	kg	M
10-4.2	nuclidic mass	m	kg	M
10-4.3	unified atomic mass constant	m_u	kg	M
10-5.1	elementary charge	e	C	$T \cdot I$
10-5.2	charge number	c	1	1
10-6	Bohr radius	a_0	m	L
10-7	Rydberg constant	R_∞	m^{-1}	L^{-1}
10-8	Hartree energy	E_H	eV	$M \cdot L^2 \cdot T^{-2}$
10-9.1	magnetic dipole moment	μ	$m^2 \cdot A$	$L^2 \cdot I$
10-9.2	Bohr magneton	μ_B	$m^2 \cdot A$	$L^2 \cdot I$
10-9.3	nuclear magneton	μ_N	$m^2 \cdot A$	$L^2 \cdot I$
10-10	spin	s	$J \cdot s$	$M \cdot L^2 \cdot T^{-1}$
10-11	total angular momentum	J	$J \cdot s$	$M \cdot L^2 \cdot T^{-1}$
10-12.1	gyromagnetic ratio	γ	$A \cdot m^2 \cdot J^{-1} \cdot s^{-1}$	$M^{-1} \cdot T \cdot I$
10-12.2	gyromagnetic ratio of the electron	γ_e	$A \cdot m^2 \cdot J^{-1} \cdot s^{-1}$	$M^{-1} \cdot T \cdot I$
10-13.1	quantum number	N	1	1
10-13.2	principal quantum number	n	1	1
10-13.3	orbital angular momentum quantum number	l	1	1
10-13.4	magnetic quantum number	m	1	1
10-13.5	spin quantum number	s	1	1
10-13.6	total angular momentum quantum number	j	1	1
10-13.7	nuclear spin quantum number	I	1	1
10-13.8	hyperfine structure quantum number	F	1	1

Item	Quantity	Symbol	Unit	Dimension
10-14.1	Landé factor	g	1	1
10-14.2	g factor of nucleus	g	1	1
10-15.1	Larmor angular frequency	ω_L	$\mathrm{s^{-1}}$	$\mathsf{T^{-1}}$
10-15.2	Larmor frequency	ν_L	$\mathrm{s^{-1}}$	$\mathsf{T^{-1}}$
10-15.3	nuclear precession angular frequency	ω_N	$\mathrm{s^{-1}}$	$\mathsf{T^{-1}}$
10-16	cyclotron angular frequency	ω_c	$\mathrm{s^{-1}}$	$\mathsf{T^{-1}}$
10-17	gyroradius	r_g	m	L
10-18	nuclear quadrupole moment	Q	$\mathrm{m^2}$	$\mathsf{L^2}$
10-19.1	nuclear radius	R	m	L
10-19.2	electron radius	r_e	m	L
10-20	Compton wavelength	λ_C	m	L
10-21.1	mass excess	Δ	kg	M
10-21.2	mass defect	B	kg	M
10-22.1	relative mass excess	Δ_r	1	1
10-22.2	relative mass defect	B_r	1	1
10-23.1	packing fraction	f	1	1
10-23.2	binding fraction	b	1	1
10-24	decay constant	λ	$\mathrm{s^{-1}}$	$\mathsf{T^{-1}}$
10-25	mean duration of life	τ	s	T
10-26	level width	Γ	eV	$\mathsf{M \cdot L^2 \cdot T^{-2}}$
10-27	activity	A	Bq	$\mathsf{T^{-1}}$
10-28	specific activity	a	$\mathrm{Bq/kg}$	$\mathsf{M^{-1} \cdot T^{-1}}$
10-29	activity density	c_A	$\mathrm{Bq/m^3}$	$\mathsf{L^{-3} \cdot T^{-1}}$
10-30	surface-activity density	a_S	$\mathrm{Bq/m^2}$	$\mathsf{L^{-2} \cdot T^{-1}}$
10-31	half life	$T_{1/2}$	s	T
10-32	alpha disintegration energy	Q_α	eV	$\mathsf{M \cdot L^2 \cdot T^{-2}}$
10-33	maximum beta-particle energy	E_β	eV	$\mathsf{M \cdot L^2 \cdot T^{-2}}$
10-34	beta disintegration energy	Q_β	eV	$\mathsf{M \cdot L^2 \cdot T^{-2}}$

Item	Quantity	Symbol	Unit	Dimension
10-35	internal conversion factor	α	1	1
10-36	particle emission rate	$\dot{N}$	s^{-1}	T^{-1}
10-37.1	reaction energy	Q	eV	$M \cdot L^2 \cdot T^{-2}$
10-37.2	resonance energy	E_r	eV	$M \cdot L^2 \cdot T^{-2}$
10-38.1	cross section	σ	m^2	L^2
10-38.2	total cross section	σ_{tot}	m^2	L^2
10-39	direction distribution of cross section	σ_Ω	$m^2 \cdot sr^{-1}$	L^2
10-40	energy distribution of cross section	σ_E	m^2/J	$M^{-1} \cdot T^2$
10-41	direction and energy distribution of cross section	$\sigma_{\Omega E}$	$m^2/(J \cdot sr)$	$M^{-1} \cdot T^2$
10-42.1	macroscopic cross section	Σ	m^{-1}	L^{-1}
10-42.2	macroscopic total cross section	Σ_{tot}	m^{-1}	L^{-1}
10-43	particle fluence	Φ	m^{-2}	L^{-2}
10-44	particle fluence rate	$\dot{\Phi}$	$m^{-2} \cdot s^{-1}$	$L^{-2} \cdot T^{-1}$
10-45	radiant energy	R	eV	$M \cdot L^2 \cdot T^{-2}$
10-46	energy fluence	Ψ	eV/m^2	$M \cdot T^{-2}$
10-47	energy fluence rate	$\dot{\Psi}$	W/m^2	$M \cdot T^{-3}$
10-48	particle current density	J	$m^{-2} \cdot s^{-1}$	$L^{-2} \cdot T^{-1}$
10-49	linear attenuation coefficient	μ	m^{-1}	L^{-1}
10-50	mass attenuation coefficient	μ_m	$kg^{-1} \cdot m^2$	$M^{-1} \cdot L^2$
10-51	molar attenuation coefficient	μ_c	$m^2 \cdot mol^{-1}$	L^2
10-52	atomic attenuation coefficient	μ_a	m^2	L^2
10-53	half-value thickness	$d_{\frac{1}{2}}$	m	L
10-54	total linear stopping power	S	eV/m	$M \cdot L \cdot T^{-2}$
10-55	total mass stopping power	S_m	$eV \cdot m^2/kg$	$L^4 \cdot T^{-2}$
10-56	mean linear range	R	m	L
10-57	mean mass range	R_ρ	$kg \cdot m^{-2}$	$M \cdot L^{-2}$
10-58	linear ionization	N_{il}	m^{-1}	L^{-1}
10-59	total ionization	N_i	1	1

Item	Quantity	Symbol	Unit	Dimension
10-60	average energy loss per elementary charge produced	W_i	eV	$M \cdot L^2 \cdot T^{-2}$
10-61	mobility	μ	$m^2/(V \cdot s)$	$M^{-1} \cdot T^2 \cdot I$
10-62.1	particle number density	n	m^{-3}	L^{-3}
10-62.2	ion number density	n^+	m^{-3}	L^{-3}
10-63	recombination coefficient	α	$m^3 \cdot s^{-1}$	$L^3 \cdot T^{-1}$
10-64	diffusion coefficient	D	$m^2 \cdot s^{-1}$	$L^2 \cdot T^{-1}$
10-65	diffusion coefficient for fluence rate	D_φ	m	L
10-66	particle source density	S	$m^{-3} \cdot s^{-1}$	$L^{-3} \cdot T^{-1}$
10-67	slowing-down density	q	$m^{-3} \cdot s^{-1}$	$L^{-3} \cdot T^{-1}$
10-68	resonance escape probability	p	1	1
10-69	lethargy	u	1	1
10-70	average logarithmic energy decrement	ζ	1	1
10-71	mean free path	l	m	L
10-72.1	slowing-down area	L^2_s	m^2	L^2
10-72.2	diffusion area	L^2	m^2	L^2
10-72.3	migration area	M^2	m^2	L^2
10-73.1	slowing-down length	L_s	m	L
10-73.2	diffusion length	L	m	L
10-73.3	migration length	M	m	L
10-74.1	neutron yield per fission	ν	1	1
10-74.2	neutron yield per absorption	η	1	1
10-75	fast fission factor	ε	1	1
10-76	thermal utilization factor	f	1	1
10-77	non-leakage probability	Λ	1	1
10-78.1	multiplication factor	k	1	1
10-78.2	infinite multiplication factor	k_∞	1	1
10-79	reactor time constant	T	s	T

Item	Quantity	Symbol	Unit	Dimension
10-80.1	energy imparted	ε	eV	$M \cdot L^2 \cdot T^{-2}$
10-80.2	mean energy imparted	$\bar{\varepsilon}$	eV	$M \cdot L^2 \cdot T^{-2}$
10-81.1	absorbed dose	D	Gy	$L^2 \cdot T^{-2}$
10-81.2	specific energy imparted	z	Gy	$L^2 \cdot T^{-2}$
10-82	quality factor	Q	1	1
10-83.1	dose equivalent	H	Sv	$L^2 \cdot T^{-2}$
10-83.2	dose equivalent rate	$\dot{H}$	Sv/s	$L^2 \cdot T^{-3}$
10-84	absorbed-dose rate	$\dot{D}$	Gy/s	$L^2 \cdot T^{-3}$
10-85	linear energy transfer	L_Δ	eV/m	$M \cdot L \cdot T^{-2}$
10-86.1	kerma	K	Gy	$L^2 \cdot T^{-2}$
10-86.2	kerma rate	$\dot{K}$	Gy/s	$L^2 \cdot T^{-3}$
10-87	mass energy-transfer coefficient	μ_{tr}/ρ	$kg^{-1} \cdot m^2$	$M^{-1} \cdot L^2$
10-88	exposure	X	C/kg	$M^{-1} \cdot T \cdot I$
10-89	exposure rate	$\dot{X}$	C/(kg·s)	$M^{-1} \cdot I$

29. ISO 80000-11 — Characteristic numbers

The sixth part specified is ISO 80000-11, *Characteristic numbers* — and it is where the *dimension does not classify the kind* thesis (*R1*) becomes *total*. Every item is catalogued: all 115 of items 11-4.1 … 11-9.2, across the six clauses — momentum transfer (the Reynolds, Euler, Froude, Mach, Knudsen, … numbers), transfer of heat (Fourier, Péclet, Rayleigh, Nusselt, Biot, …), transfer of matter in a binary mixture (the mass-transfer analogues), constants of matter (Prandtl, Schmidt, Lewis, …), magnetohydrodynamics (the magnetic and electric numbers), and miscellaneous — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced.

A characteristic number is a *dimensionless* ratio. So **every one of the 115 is dimension one**: the dimension functor sends the Reynolds number (inertial / viscous forces), the Mach number (flow speed / sound speed), and the Prandtl number (momentum / thermal diffusivity) all to the *single point* `1`. A dimension-only model sees one type — "a real number" — for the entire part and cannot tell any two of its 115 members apart. What keeps them apart is their *measurement principle*: the physical ratio each one expresses, carried as an examination principle (requirement *R2*). This is the pattern Part 3 set on the length family, Part 4 on the force family, Part 7 on the radiation trios — here at its widest, over a part with *no* dimensional variation at all.

29.1. Quantity-kinds of ISO 80000-11 — one dimension, 115 kinds

Definition 29.1.1. group statement"> uses 1 used by 0 ✓

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "11-4.1"), the principal quantity symbol (e.g. `Re`), and the coherent SI unit symbol (`1` for every characteristic number). The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream.

Lean code for Definition 29.1.1. 1 definition

structure (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` — complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 29.1.1 uses 0

> Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-11, Second edition, 2019-10 item 11-4.1`. The `catalogue` lists all 115 items in item order across clauses 4 … 9; `catalogue_length : catalogue.length = 115` checks the count.

Definition 29.1.2. group uses 2 used by 0 ✓

A *characteristic number* is a ratio-scale quantity-kind of *dimension one*, individuated by its examination principle — the *measurement principle*, formed as `context: defining-ratio`. The context is one of ISO 80000-11's six clauses (momentum transfer, heat transfer, mass transfer, constants of matter, magnetohydrodynamics, miscellaneous); it is part of the principle because the same ratio name recurs across clauses as a *different* kind.

Lean code for Definition 29.1.2.

1 definition

`def` · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
def PropertyKindCalculus.Iso80000.Part11.charNum
  (ctx : PropertyKindCalculus.Iso80000.Part11.Context)
  (id principle : String) :
  PropertyKindCalculus.DimensionedKind LTMCTDimensionBase
```

Proof for Definition 29.1.2

uses 0

> `charNum ctx id principle := { kind := { id, scale := .ratio, examPrinciple := some (ctx.id ++ ": " ++ principle) }, dim := Dim.one }`, with the clause `Context` an enumeration (`momentum`, `heat`, `mass`, `constants`, `mhd`, `misc`). Every kind in the part is built by this one constructor.

Theorem 29.1.3. group statement"> used by 0 ✓

Effort	Tags
small	proved

> ***Every characteristic number is dimension one — the widest collapse in the series.*** *All 115 catalogued kinds share the dimensionless dimension; the dimension functor collapses the entire part to one point. What distinguishes them is the measurement principle, never the dimension. Uses the dimension map.*

Lean code for Theorem 29.1.3.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part11.catalogue_all_dim_one
  (c : PropertyKindCalculus.Iso80000.CataloguedKind) :
  c ∈ PropertyKindCalculus.Iso80000.Part11.catalogue →
    c.qk.dim LTMCTDimensionBase = 1
```

Proof for Theorem 29.1.3 uses 0

`catalogue_all_dim_one : ∀ c ∈ catalogue, c.qk.dim = 1`, by `fin_cases` over the catalogue then reflexivity (each entry is a `charNum` application, whose `dim` is `Dim.one` by construction — `charNum_dim_one`). Axiom-free.

29.2. The sub-suffixed homonyms — same name, distinct kind

ISO 80000-11 reuses one name across its transport-phenomena clauses, distinguishing the variants only by a symbol suffix and a different defining ratio. The Froude number appears as 11-4.3 (`Fr`, momentum) and 11-5.4 (`Fr*`, heat); the Fourier, Péclet, Grashof, Nusselt, Stanton, Graetz and Biot numbers each as a heat (`Fo`, …) and a mass (`Fo*`, …) member; the Bejan number *four* times; and the Stokes number *five* times (`Stk`, `Stk₁` … `Stk₄`). These are the sub-suffixed quantity kinds — and they are the sharpest form of *R2* in the whole series: a member shares its name *and* its dimension (one) with its siblings, so neither the name nor the dimension can tell them apart. Only the measurement principle can.

Theorem 29.2.1. group statement">used by 0 ✓

Effort	Tags
small	proved

The two Froude numbers are distinct kinds — by measurement principle, not by name. *Both are named "Froude number" and both are dimension one (11-4.3, `Fr`, momentum transfer; 11-5.4, `Fr*`, heat transfer), yet they isolate different physical ratios — inertial/gravitational versus gravitational/thermodiffusion forces — so the calculus proves them distinct from their differing principles. The Fourier (11-5.1, 11-6.1) and Nusselt (11-5.5, 11-6.4) numbers recur likewise. Uses [the examination principle](#).*

Lean code for Theorem 29.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part11.froudeMomentum_ne_froudeHeat :
  PropertyKindCalculus.Iso80000.Part11.froudeMomentum.kind
      LTMCTDimensionBase ≠
```

```
    PropertyKindCalculus.Iso80000.Part11.froudeHeat.kind
      LTMCTDimensionBase
```

Proof for Theorem 29.2.1

uses 0

`froudeMomentum_ne_froudeHeat : froudeMomentum.kind ≠ froudeHeat.kind`, from `KindOfProperty.distinct_of_examPrinciple` on the differing principles `momentum-transfer: …` and `heat-transfer: …`; `froudeMomentum_id_eq_froudeHeat` checks the names *are* equal. The companions `fourierHeat_ne_fourierMass` and `nusseltHeat_ne_nusseltMass` cover the other recurring pairs. Axiom-free.

Theorem 29.2.2.

group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

The five Stokes numbers are pairwise distinct kinds. *All five (11-4.32 … 11-4.36) carry the name "Stokes number" and dimension one, distinguished only by symbol suffix (`Stk`, `Stk₁` … `Stk₄`) and the measurement principle each isolates — friction/inertia for particles in flow, for vibrating particles, rotameter calibration, viscous/gravity for settling particles, drag/internal-friction. Uses [the examination principle](#).*

Lean code for Theorem 29.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part11.stokes_pairwise_distinct :
  PropertyKindCalculus.Iso80000.Part11.stokesPlasma.kind
        LTMCTDimensionBase ≠
      PropertyKindCalculus.Iso80000.Part11.stokesVibrating.kind
        LTMCTDimensionBase ∧
    PropertyKindCalculus.Iso80000.Part11.stokesVibrating.kind
          LTMCTDimensionBase ≠
        PropertyKindCalculus.Iso80000.Part11.stokesRotameter.kind
          LTMCTDimensionBase ∧
      PropertyKindCalculus.Iso80000.Part11.stokesRotameter.kind
            LTMCTDimensionBase ≠
          PropertyKindCalculus.Iso80000.Part11.stokesGravity.kind
            LTMCTDimensionBase ∧
        PropertyKindCalculus.Iso80000.Part11.stokesGravity.kind
              LTMCTDimensionBase ≠
            PropertyKindCalculus.Iso80000.Part11.stokesDrag.kind
```

```
      LTMCTDimensionBase ∧
    PropertyKindCalculus.Iso80000.Part11.stokesPlasma.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part11.stokesDrag.kind
      LTMCTDimensionBase
```

Proof for Theorem 29.2.2

uses 0

`stokes_pairwise_distinct`, a conjunction of five `distinct_of_examPrinciple` applications on the differing principles. Axiom-free.

Theorem 29.2.3.

group statement">uses 1 by 0 ✓

Effort	Tags
small	proved

The four Bejan numbers are distinct kinds — even two sharing a transport context. *The momentum (11-4.15), the two heat (11-5.9, 11-5.10) and the mass (11-6.16) Bejan numbers are four kinds. The two heat-transfer ones share name and context, and differ only in the measurement principle (mechanical-work/diffusion-losses versus heat-transfer efficiency by entropy generation) — the finest distinction the calculus draws in this part. Uses the examination principle.*

Lean code for Theorem 29.2.3.

1 theorem

theorem · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part11.bejan_four_distinct :
  PropertyKindCalculus.Iso80000.Part11.bejanMomentum.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part11.bejanHeat.kind
      LTMCTDimensionBase ∧
  PropertyKindCalculus.Iso80000.Part11.bejanHeat.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part11.bejanEntropy.kind
      LTMCTDimensionBase ∧
    PropertyKindCalculus.Iso80000.Part11.bejanEntropy.kind
      LTMCTDimensionBase ≠
    PropertyKindCalculus.Iso80000.Part11.bejanMass.kind
      LTMCTDimensionBase ∧
    PropertyKindCalculus.Iso80000.Part11.bejanMomentum.kind
      LTMCTDimensionBase ≠
```

```
PropertyKindCalculus.Iso80000.Part11.bejanMass.kind
  LTMCTDimensionBase
```

Proof for Theorem 29.2.3 uses 0

`bejan_four_distinct`, a conjunction of four `distinct_of_examPrinciple` applications. Axiom-free.

Theorem 29.2.4. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

> ***One unit symbol, many kinds.*** *The Reynolds and Euler units are not commensurable, though both carry the coherent unit symbol `1` and dimension one. Same symbol, same dimension, different kind — so a value of one cannot be read as a value of the other. The kind layer keeps them apart where the unit symbol and the dimension cannot. Uses [the metrological unit](#).*

Lean code for Theorem 29.2.4. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part11.reynolds_euler_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part11.reynoldsUnit.Commensurable
      PropertyKindCalculus.Iso80000.Part11.eulerUnit
```

Proof for Theorem 29.2.4 uses 0

`reynolds_euler_not_commensurable : ¬ reynoldsUnit.Commensurable eulerUnit`, by unfolding `Commensurable` (equality of the referenced kinds) and `decide`; `reynoldsUnit_wellFormed` checks the unit is well-formed (a ratio-scale kind bears a unit). Axiom-free.

29.3. Dimension does not classify; the measurement principle does

Theorem 29.3.1. uses 2 used by 0 ✓ L∃∀N

Effort	Priority	Tags
small	high	capstone, proved

> ***The dimension-one collision, on the standard — total, the widest in the series.*** *There exist distinct ISO 80000-11 kinds with the same dimension one — the Reynolds and Euler numbers witness it, alongside all 113 others. No dimension-only type system can separate any two characteristic numbers; the kind layer, through the measurement principle (R2), holds the entire part apart. This is dimension does not classify the kind (R1) at its absolute limit. Uses the dimension map and the examination principle.*

Lean code for Theorem **29.3.1.** 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part11.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part11.iso80000_11_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 29.3.1 uses 0

> `iso80000_11_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1`, witnessed by `⟨reynolds, euler, reynolds_ne_euler, rfl, rfl⟩`. Axiom-free.

29.4. A characteristic number is a kind, not a constant

The collapse to dimension one (the collision above) invites a tempting confusion: if every characteristic number is dimension one, isn't it just *a number* — the same sort of thing as π or e? It is not, and saying why is the clearest statement of the two-index architecture: a quantity is a *kind* carried over a *numeric representation*.

A mathematical constant such as π is a fixed *magnitude in the carrier*: it lives *below* the kind layer, in the numbers the magnitudes are drawn from, and it is the same value in every context. A characteristic number is the opposite in every respect: it is a *kind of quantity*, dimension one, living *above* the carrier; its value is *variable* — whatever the defining ratio produces in a given situation (ρvL/μ for the Reynolds number, flow speed over sound speed for the Mach number). The two never coincide:

	π (a constant)	Reynolds, Mach, … (characteristic numbers)
What it is	a fixed magnitude in the carrier	a kind of quantity (dimension one)
Layer	below the kind layer, in the representation	above the carrier, in the kind layer
Value	the same number in every context	variable — whatever `ρvL/μ`, `v/c`, … produce
How formed	shipped by the carrier	by the quotient calculus, the dimensions cancelling to 1
Identity	one value	each a distinct kind — Reynolds ≠ Mach, though both dimension one

This is why the function calculus supplies π as a carrier constant and `e = exp 1` as a *derived* one (the only mathematical constant a numeric carrier need supply directly is π), while the Reynolds and Mach numbers are *kinds* in this part, never carrier constants. The kind layer keeps the two categories apart exactly where a dimension-only model — seeing "dimension one" for all of them — cannot.

Definition 29.4.1. group statement">us

A *carrier constant* is a fixed magnitude supplied by the numeric carrier — π, and the derived `e = exp 1` — living *below* the kind layer, in the representation type. It is the same value in every context, and it is *not* a quantity-kind: it carries no kind index, no measurement principle, and is not formed from other quantities. This is the categorical opposite of a characteristic number.

Lean code for Definition 29.4.1. 1 definition

`def` · defined in `PropertyKindCalculus/QuantityFunction.lean` — complete

```
def PropertyKindCalculus.MathCarrier.e {R : Type}
  [PropertyKindCalculus.MathCarrier R] [One R] : R
```

Proof for Definition 29.4.1 uses 0

In the function calculus (`PropertyKindCalculus.QuantityFunction`), the carrier capability `MathCarrier` carries `pi` as its *only* primitive constant — the one transcendental constant a numeric carrier ships directly (matching TorchLean's `MathFunctions` (George *et al.*, 2026)[29] Robert Joseph George, Jennifer Cruden, Will Adkisson, Xiangru Zhong, Huan Zhang, and Anima Anandkumar (2026). "TorchLean: Formalizing Neural Networks in Lean" (https://arxiv.org/abs/2602.22631). *arXiv preprint arXiv:2602.22631 (cs.MS).* .). Every other constant is *derived*: `MathCarrier.e := MathCarrier.exp`

29.

> `1`, and likewise `ln 2`, `√2`, …. Adding them as carrier fields would be redundant, since the operations already determine them.

Theorem 29.4.2. group statement">us

Effort	Tags
small	proved

> ***The Reynolds and Mach numbers are distinct kinds, both dimension one — so neither is a carrier constant.*** *Two characteristic numbers forget to the same dimension one yet are different kinds, formed by the quotient calculus from different physical ratios. Unlike π (a fixed magnitude in the carrier), each is a variable quantity whose value depends on the flow. Uses the dimension map.*

Lean code for Theorem 29.4.2. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Function.lean` — complete

```
theorem PropertyKindCalculus.reynolds_ne_mach_but_both_dimensionless :
  PropertyKindCalculus.reynoldsK.kind LTMCTDimensionBase ≠
      PropertyKindCalculus.machK.kind LTMCTDimensionBase ∧
    PropertyKindCalculus.reynoldsK.toDimension = 1 ∧
      PropertyKindCalculus.machK.toDimension = 1
```

Proof for Theorem 29.4.2 uses 0

> `reynolds_ne_mach_but_both_dimensionless : reynoldsK.kind ≠ machK.kind ∧ reynoldsK.toDimension = 1 ∧ machK.toDimension = 1`, the kind inequality by `decide` on the differing identities, the two dimensions by reflexivity. The kinds `reynoldsK`/`machK` are declared in the function-calculus layer (`PropertyKindCalculus.Function`) for this contrast; the catalogued `reynolds`/`euler` of this part carry the same point across all 115 members. Axiom-free.

29.5. Item index — ISO 80000-11

Every catalogued item of ISO 80000-11, indexed by its printed item number, with the characteristic number's name, the principal quantity symbol, and the *measurement principle* — this work's terse descriptor of the defining ratio, which is what individuates the kind. The coherent SI unit and the PhysLib dimension are `1` for *every* item, so they are omitted from the table; that uniformity is exactly the point. Each item number links to the formalized result it participates in — a homonym distinction, the dimension-one collision, or the dimension-one theorem itself. Names, symbols, and principle descriptors are *citation locators*; nothing normative is reproduced.

Item	Characteristic number	Symbol	Measurement principle (dimension one)
11-4.1	Reynolds number	Re	inertial forces / viscous forces
11-4.2	Euler number	Eu	pressure drop / kinetic energy per volume
11-4.3	Froude number	Fr	inertial forces / gravitational forces
11-4.4	Grashof number	Gr	thermal buoyancy forces / viscous forces
11-4.5	Weber number	We	inertial forces / surface-tension forces
11-4.6	Mach number	Ma	speed of flow / speed of sound
11-4.7	Knudsen number	Kn	mean free path / characteristic length
11-4.8	Strouhal number	Sr	characteristic frequency / characteristic speed
11-4.9	drag coefficient	c_D	drag force / inertial force
11-4.10	Bagnold number	Bg	drag and gravitational force / inertial force
11-4.11	Bagnold number	Ba_2	drag force / viscous force, solid particles
11-4.12	lift coefficient	c_l	lift force / inertial force
11-4.13	thrust coefficient	c_t	thrust force / inertial force
11-4.14	Dean number	Dn	centrifugal force / inertial force, curved pipe
11-4.15	Bejan number	Be	mechanical work / frictional energy loss
11-4.16	Lagrange number	Lg	mechanical work / frictional energy loss, Lagrange form
11-4.17	Bingham number	Bm	yield stress / viscous stress
11-4.18	Hedström number	He	yield stress / viscous stress, at flow limit
11-4.19	Bodenstein number	Bd	convective / diffusive matter transfer
11-4.20	Rossby number	Ro	inertial forces / Coriolis forces
11-4.21	Ekman number	Ek	viscous forces / Coriolis forces
11-4.22	elasticity number	El	relaxation time / diffusion time
11-4.23	Darcy friction factor	f_D	pressure loss / pipe-wall friction
11-4.24	Fanning number	f_F	wall shear stress / dynamic pressure
11-4.25	Goertler number	Go	centrifugal effects / viscous effects, boundary layer
11-4.26	Hagen number	Hg	pressure-gradient force / viscous force
11-4.27	Laval number	La	flow speed / critical sound speed, nozzle throat

Item	Characteristic number	Symbol	Measurement principle (dimension one)
11-4.28	Poiseuille number	Poi	pressure force / viscous force, pipe flow
11-4.29	power number	Ne	agitator power / inertial power
11-4.30	Richardson number	Ri	potential energy / kinetic energy
11-4.31	Reech number	Re_e	object speed / wave speed, submerged
11-4.32	Stokes number	Stk	friction force / inertial force, particles in flow
11-4.33	Stokes number	Stk_1	friction force / inertial force, vibrating particles
11-4.34	Stokes number	Stk_2	drag / inertia, rotameter calibration
11-4.35	Stokes number	Stk_3	viscous force / gravity force, settling particles
11-4.36	Stokes number	Stk_4	drag force / internal friction force, particles
11-4.37	Laplace number	La	capillary force / viscous force, free surface
11-4.38	Blake number	Bl	inertial force / viscous force, porous medium
11-4.39	Sommerfeld number	So	viscous force / load force, lubrication
11-4.40	Taylor number	Ta	centrifugal force / viscous force, rotating flow
11-4.41	Galilei number	Ga	gravitational force / viscous force, fluid film
11-4.42	Womersley number	Wo	inertial forces / viscous forces, oscillating flow
11-5.1	Fourier number	Fo	heat conduction rate / thermal storage rate
11-5.2	Péclet number	Pe	convective / conductive heat transfer rate
11-5.3	Rayleigh number	Ra	thermal buoyancy forces / viscous forces, free convection
11-5.4	Froude number	Fr*	gravitational forces / thermodiffusion forces
11-5.5	Nusselt number	Nu	convective / conductive heat transfer at a surface
11-5.6	Biot number	Bi	internal / surface thermal resistance
11-5.7	Stanton number	St	wall heat transfer / fluid heat-capacity flow
11-5.8	j-factor	j	heat transfer / mass transfer, Colburn analogy
11-5.9	Bejan number	Be_1	mechanical work / frictional and thermal-diffusion losses
11-5.10	Bejan number	Be_S	heat-transfer efficiency / entropy generation
11-5.11	Stefan number	Ste	sensible heat / latent heat, phase change

Item	Characteristic number	Symbol	Measurement principle (dimension one)
11-5.12	Brinkman number	Br	viscous heat production / wall heat conduction
11-5.13	Clausius number	Cl	kinetic energy transfer / thermal conduction
11-5.14	Carnot number	Ca	maximum thermodynamic (Carnot) efficiency
11-5.15	Eckert number	Ec	kinetic energy / enthalpy difference
11-5.16	Graetz number	Gz	convective / conductive heat, laminar pipe
11-5.17	heat transfer number	K_Q	heat flow / kinetic energy of flow
11-5.18	Pomerantsev number	Po	generated heat / conducted heat in a body
11-5.19	Boltzmann number	Bz	convective heat / radiant heat
11-5.20	Stark number	Sk	radiant heat / conductive heat
11-6.1	Fourier number	Fo*	diffusive mass transfer / storage rate
11-6.2	Péclet number	Pe*	advective / diffusive mass transfer rate
11-6.3	Grashof number	Gr*	buoyancy forces / viscous forces, natural convection
11-6.4	Nusselt number	Nu*	mass flux / molecular-diffusion flux
11-6.5	Stanton number	St*	perpendicular / parallel surface mass transfer
11-6.6	Graetz number	Gz*	advective / radial diffusive mass transfer
11-6.7	mass transfer factor	j*	interface mass transfer / parallel flux, Chilton-Colburn
11-6.8	Atwood number	At	density difference / density sum, two fluids
11-6.9	Biot number	Bi*	interface / interior mass transfer rate
11-6.10	Morton number	Mo	gravitational forces / viscous forces, bubbles
11-6.11	Bond number	Bo	gravitational and inertial force / capillary force
11-6.12	Archimedes number	Ar	buoyancy forces / viscous forces, density difference
11-6.13	expansion number	Ex	buoyancy force / inertial force, rising bubbles
11-6.14	Marangoni number	Mg	surface-tension convection / thermal diffusion
11-6.15	Lockhart-Martinelli parameter	Lp	two-phase mass-flow-rate ratio by density
11-6.16	Bejan number	Be*	mechanical work / frictional and diffusion losses
11-6.17	cavitation number	Ca	static-vapour head / dynamic head

Item	Characteristic number	Symbol	Measurement principle (dimension one)
11-6.18	absorption number	Ab	mass flow rate / surface area, gas absorption
11-6.19	capillary number	Ca	gravitational forces / capillary forces
11-6.20	dynamic capillary number	Ca*	viscous force / capillary force, interface
11-7.1	Prandtl number	Pr	kinematic viscosity / thermal diffusivity
11-7.2	Schmidt number	Sc	kinematic viscosity / diffusion coefficient
11-7.3	Lewis number	Le	thermal diffusivity / diffusion coefficient
11-7.4	Ohnesorge number	Oh	viscous force / root of inertia times capillary force
11-7.5	Cauchy number	Cy	inertia forces / compression forces
11-7.6	Hooke number	Ho_2	inertia forces / linear-stress forces
11-7.7	Weissenberg number	Wi	shear rate times relaxation time
11-7.8	Deborah number	De	relaxation time / observation time
11-7.9	Lorentz number	Lo	electrical conductivity / thermal conductivity
11-7.10	compressibility number	Z	real-gas / ideal-gas compressibility factor
11-8.1	Reynolds magnetic number	Rm	inertial force / magneto-dynamic viscous force
11-8.2	Batchelor number	Bt	inertia / magneto-dynamic diffusion
11-8.3	Nusselt electric number	Ne_e	convective / diffusive ion current
11-8.4	Alfvén number	Al	flow speed / Alfvén wave speed
11-8.5	Hartmann number	Ha	magnetically induced stress / viscous force
11-8.6	Cowling number	Co	magnetic / kinematic energy density
11-8.7	Stuart electrical number	Se	electric / kinematic energy density
11-8.8	magnetic pressure number	N_mp	gas pressure / magnetic pressure
11-8.9	Chandrasekhar number	Q	Lorentz force / viscous force
11-8.10	Prandtl magnetic number	Pr_m	kinematic viscosity / magnetic viscosity
11-8.11	Roberts number	Ro_m	thermal diffusivity / magnetic viscosity
11-8.12	Stuart number	N_St	magnetic force / inertial force
11-8.13	magnetic number	N_mg	magnetic body force / viscous force
11-8.14	electric field parameter	E_f	Coulomb force / Lorentz force
11-8.15	Hall number	Hl	gyrofrequency / collision frequency

Item	Characteristic number	Symbol	Measurement principle (dimension one)
11-8.16	Lundquist number	Lu	Alfvén speed / magneto-dynamic diffusion speed
11-8.17	Joule magnetic number	Jo_m	Joule heating / magnetic field energy
11-8.18	Grashof magnetic number	Gr_m	thermo-magnetic buoyancy / viscous force
11-8.19	Naze number	Na	Alfvén wave speed / sound speed
11-8.20	Reynolds electric number	Re_e	fluid speed / charged-particle drift speed
11-8.21	Ampère number	Am	electric surface current / magnetic field
11-9.1	Arrhenius number	α	chemical activation energy / thermal energy
11-9.2	Landau-Ginzburg number	κ	penetration depth / coherence length

30. ISO 80000-12 — Condensed matter physics

The ninth part specified in full is ISO 80000-12, *Condensed matter physics*. Every item — all of 12-1.1 … 12-38.2, sixty including every sub-suffixed item — is catalogued: the lattice geometry, the reciprocal lattice and wavenumbers, the Debye and Grüneisen quantities, the transport and thermoelectric coefficients, the work functions and band energies, the carrier densities, and the magnetism and superconductivity quantities — each carrying its exact source as data. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced. The defining *mathematics* of selected remarks is specified in the sibling module `Part12.DefiningRelations`.

Condensed matter physics is where *dimension does not classify the kind* stops being the exception and becomes the rule: nearly every quantity collides in dimension with several others. Five families dominate, and each is one big collision — *thirteen lengths* on `L`, *seven energies* on `M·L²·T⁻²`, *five temperatures* on `Θ`, *five carrier densities* on `L⁻³`, and *five reciprocal lengths* on `L⁻¹`. The five named temperatures are especially striking: the Debye, Fermi, Curie, Néel, and superconduction-transition temperatures share one dimension *and* one unit (the kelvin), and only the kind tells them apart.

30.1. Quantity-kinds and units of ISO 80000-12

Definition 30.1.1. statement">uses 1 used by 0

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "12-27.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream.

Lean code for Definition 30.1.1. 1 definition

`structure` (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` — complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 30.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `ISO 80000-12, Second edition, 2019-08 item 12-27.1`. The `catalogue` lists all 60 items in item order — the lattice geometry (12-1.1 … 12-8), the band energies (12-24.1 … 12-27.2), the carrier densities (12-29.1 … 12-29.5), the superconductivity quantities (12-35.1 … 12-38.2), … — each paired with its coherent SI unit symbol.

30.2. Collisions everywhere — the kind classifies where the dimension cannot

ISO 80000-12 makes the catalogue's thesis the norm. Five families each collapse to one dimension:

- *Seven energies* — work function, ionization energy, electron affinity, Fermi energy, gap energy, exchange integral, superconductor energy gap — all $M \cdot L^2 \cdot T^{-2}$.
- *Five temperatures* — Debye, Fermi, Curie, Néel, superconduction-transition — all Θ, and all in the kelvin.
- *Thirteen lengths* — lattice and position vectors, spacings, mean free paths, the penetration and coherence lengths — all L.
- *Five carrier densities* and *five reciprocal lengths* — all L^{-3} and L^{-1}.

A dimension-only model sees one type per family; the kind layer keeps the Fermi energy from the gap energy, the Curie temperature from the Néel temperature, the lattice spacing from the Burgers vector. The five *named* temperatures are the sharpest case: one dimension, one unit symbol, five kinds.

Theorem 30.2.1. group statement">uses 1 used by 0 ✓

Effort: small

Tags: capstone, proved

> ***Fermi energy is not gap energy, though both are J and $M \cdot L^2 \cdot T^{-2}$ (items 12-27.1, 12-27.2).*** *There exist distinct ISO 80000-12 kinds with the same dimension — the seven energies witness it. No dimension-only type system can separate them; the kind layer — and the unit — does. Uses the dimension map.*

Lean code for Theorem 30.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part12.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part12.iso80000_12_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 30.2.1 uses 0

`iso80000_12_dim_collision`, witnessed by `⟨fermiEnergy, gapEnergy, …⟩`. The unit fact `jouleFermi_jouleGap_not_commensurable` is a `decide`; the companions `latticePlaneSpacing_dim_eq_burgersVector_dim` (the length family) and `curieTemperature_dim_eq_neelTemperature_dim` (the temperature family) record the other collisions. Axiom-free.

Theorem 30.2.2. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

The kelvin of the Curie temperature and the kelvin of the Néel temperature are not commensurable, though both are `0` (items 12-35.1, 12-35.2). *Five named temperatures — the Debye, Fermi, Curie, Néel, and superconduction-transition — share one dimension and one unit symbol, and only the kind tells them apart. Uses the metrological unit.*

Lean code for Theorem 30.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part12.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part12.kelvinCurie_kelvinNeel_not_commensu
  ¬PropertyKindCalculus.Iso80000.Part12.kelvinCurie.Commensurable
      PropertyKindCalculus.Iso80000.Part12.kelvinNeel
```

Proof for Theorem 30.2.2

uses 0

`kelvinCurie_kelvinNeel_not_commensurable`, by `decide` after unfolding `MetrologicalUnit.Commensurable`. Axiom-free.

Theorem 30.2.3. group statement">uses 1 used by 0 ✓

Effort	Tags
small	proved

The dimension-1 disambiguation, on the standard. *There exist distinct ISO 80000-12 kinds with the same dimension one — the Bragg angle and the structure factor witness it, alongside the order parameters, the atomic scattering factor, the Debye-Waller factor, the Grüneisen parameters, and the mobility ratio. Dimension cannot separate them; the kind layer does. Uses the dimension map.*

Lean code for Theorem 30.2.3. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part12.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part12.iso80000_12_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 30.2.3 uses 0

> `iso80000_12_dim_one_collision`, witnessed by `⟨braggAngle, structureFactor, …⟩` with the kind distinction a `decide` and reflexivity. Axiom-free.

30.3. The algebraic Remarks as kind-laws: thermoelectricity built by the kind algebra

Two Part-12 definitions state the thermoelectric coefficients *algebraically*: the Seebeck coefficient is the thermoelectric voltage per temperature (12-21, `S = dE/dT`), and the Peltier coefficient is the Seebeck coefficient times temperature (12-22, `Π = S·T`, the Kelvin relation). Both are specified as R12 kind-laws, crossing to ISO 80000-5 for the thermodynamic temperature.

Theorem 30.3.1. **statement">uses 1** used by 0

Effort	Tags
small	proved

> ***Peltier coefficient is Seebeck coefficient × temperature — a verified construction (item 12-22).*** *A Peltier coefficient built as the product of a Seebeck coefficient and a thermodynamic temperature carries its product certificate by construction, over the `ℝ` carrier. The companion `seebeck_dim_from_voltage_temperature` shows the Seebeck coefficient is `V/K` because it is a voltage per temperature. Uses the quotient kind-law.*

Lean code for Theorem 30.3.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part12/DefiningRelations.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part12.DefiningRelations.peltierOf_isProdu
  (s :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part12.seebeckCoefficient.kind
        LTMCTDimensionBase)
      ℝ)
  (t :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part5.thermodynamicTemperature.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part12.DefiningRelations.peltier_prod_seebeck_te
mperature
    (PropertyKindCalculus.Iso80000.Part12.DefiningRelations.peltierOf s
      t)
    s t
```

Proof for Theorem 30.3.1 uses 0

> `peltierOf s t := Quantity.mul peltier_prod_seebeck_temperature s t`, and `peltierOf_isProduct` is `rfl`. The companion `seebeckOf` (= thermoelectric voltage / temperature) carries the quotient certificate, with `seebeck_certificate_canonical` its canonicity; both cross to ISO 80000-5.

30.4. Item index — ISO 80000-12

Every catalogued item of ISO 80000-12, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in. Symbols and unit strings are *citation locators*; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
12-1.1	lattice vector	R	m	L
12-1.2	fundamental lattice vectors	a_1	m	L
12-2.1	angular reciprocal lattice vector	G	m^{-1}	L^{-1}
12-2.2	fundamental reciprocal lattice vectors	b_1	m^{-1}	L^{-1}
12-3	lattice plane spacing	d	m	L
12-4	Bragg angle	ϑ	1	1
12-5.1	short-range order parameter	r	1	1
12-5.2	long-range order parameter	R	1	1
12-5.3	atomic scattering factor	f	1	1
12-5.4	structure factor	F	1	1
12-6	Burgers vector	b	m	L
12-7.1	particle position vector	r	m	L
12-7.2	equilibrium position vector	R_0	m	L
12-7.3	displacement vector	u	m	L
12-8	Debye-Waller factor	D	1	1
12-9.1	angular wavenumber	k	m^{-1}	L^{-1}
12-9.2	Fermi angular wavenumber	k_F	m^{-1}	L^{-1}
12-9.3	Debye angular wavenumber	q_D	m^{-1}	L^{-1}
12-10	Debye angular frequency	$ω_D$	s^{-1}	T^{-1}
12-11	Debye temperature	$Θ_D$	K	Θ
12-12	density of vibrational states	g	$m^{-3}·s$	$L^{-3}·T$
12-13	thermodynamic Grüneisen parameter	$γ_G$	1	1
12-14	Grüneisen parameter	γ	1	1
12-15.1	mean free path of phonons	l_p	m	L
12-15.2	mean free path of electrons	l_e	m	L
12-16	energy density of states	n_E	$J^{-1}·m^{-3}$	$M^{-1}·L^{-5}·T^2$
12-17	residual resistivity	$ρ_0$	Ω·m	$M·L^3·T^{-3}·I^{-2}$
12-18	Lorenz coefficient	L	V^2/K^2	$M^2·L^4·T^{-6}·I^{-2}·Θ^{-2}$

Item	Quantity	Symbol	Unit	Dimension
12-19	Hall coefficient	R_H	m^3/C	$L^3 \cdot T^{-1} \cdot I^{-1}$
12-20	thermoelectric voltage	E_{ab}	V	$M \cdot L^2 \cdot T^{-3} \cdot I^{-1}$
12-21	Seebeck coefficient	S_{ab}	V/K	$M \cdot L^2 \cdot T^{-3} \cdot I^{-1} \cdot \Theta^{-1}$
12-22	Peltier coefficient	Π_{ab}	V	$M \cdot L^2 \cdot T^{-3} \cdot I^{-1}$
12-23	Thomson coefficient	μ	V/K	$M \cdot L^2 \cdot T^{-3} \cdot I^{-1} \cdot \Theta^{-1}$
12-24.1	work function	φ	J	$M \cdot L^2 \cdot T^{-2}$
12-24.2	ionization energy	E_i	J	$M \cdot L^2 \cdot T^{-2}$
12-25	electron affinity	χ	J	$M \cdot L^2 \cdot T^{-2}$
12-26	Richardson constant	A	$A \cdot m^{-2} \cdot K^{-2}$	$L^{-2} \cdot I \cdot \Theta^{-2}$
12-27.1	Fermi energy	E_F	J	$M \cdot L^2 \cdot T^{-2}$
12-27.2	gap energy	E_g	J	$M \cdot L^2 \cdot T^{-2}$
12-28	Fermi temperature	T_F	K	Θ
12-29.1	electron density	n	m^{-3}	L^{-3}
12-29.2	hole density	p	m^{-3}	L^{-3}
12-29.3	intrinsic carrier density	n_i	m^{-3}	L^{-3}
12-29.4	donor density	n_d	m^{-3}	L^{-3}
12-29.5	acceptor density	n_a	m^{-3}	L^{-3}
12-30	effective mass	m^*	kg	M
12-31	mobility ratio	b	1	1
12-32.1	relaxation time	τ	s	T
12-32.2	carrier lifetime	τ	s	T
12-33	diffusion length	L	m	L
12-34	exchange integral	J	J	$M \cdot L^2 \cdot T^{-2}$
12-35.1	Curie temperature	T_C	K	Θ
12-35.2	Néel temperature	T_N	K	Θ
12-35.3	superconduction transition temperature	T_c	K	Θ
12-36.1	thermodynamic critical magnetic flux density	B_c	T	$M \cdot T^{-2} \cdot I^{-1}$
12-36.2	lower critical magnetic flux density	B_{c1}	T	$M \cdot T^{-2} \cdot I^{-1}$

Item	Quantity	Symbol	Unit	Dimension
12-36.3	upper critical magnetic flux density	B_{c2}	T	$\mathsf{M}\cdot\mathsf{T}^{-2}\cdot\mathsf{I}^{-1}$
12-37	superconductor energy gap	Δ	J	$\mathsf{M}\cdot\mathsf{L}^{2}\cdot\mathsf{T}^{-2}$
12-38.1	London penetration depth	λ_L	m	L
12-38.2	coherence length	ξ	m	L

31. IEC 80000-13 — Information science and technology

The tenth and last part specified is IEC 80000-13, *Information science and technology* — the second IEC-published part, after electromagnetism. Every item — all of 13-1 … 13-42, forty-two in all (the part uses flat item numbers, no sub-suffixes) — is catalogued: the teletraffic quantities, storage and transfer, the powers and signal energy, and the coding- and information-theory quantities — each carrying its exact source as data. Only *citation locators* are recorded; no normative content from the licensed standard is reproduced. The defining *mathematics* of selected remarks is specified in the sibling module `Part13.DefiningRelations`.

Information science is where the *scale-spanning* reduction (the *Scale-spanning units* chapter, requirement *R13*) meets *human-selected counts of information*. Its characteristic quantities are all *dimension one* — and yet they carry special, mutually *incommensurable* units, and belong to distinct kinds. Information content and entropy are measured in the *shannon* (or hartley, or natural unit); traffic intensity in the *erlang*; storage capacity in the *bit*. All are dimension one; the dimension functor collapses them to a single point, and only the kind keeps the shannon of information from the erlang of traffic from the bit of storage. The shannon, erlang, and bit are scale-spanning units in exactly the sense of the candela and the mole: dimensionless references chosen by people.

31.1. Quantity-kinds and units of IEC 80000-13

Definition 31.1.1. statement">uses 1 used by 0

A *catalogued kind* pairs a dimensioned quantity-kind with its exact source in the series — the part, the printed item designation (e.g. "13-24"), the principal quantity symbol, and the unit symbol (for the information quantities, the special named unit — shannon, erlang, bit — rather than the bare `1`). The citation travels with the kind as data.

Lean code for Definition 31.1.1. 1 definition

structure (5 fields) · defined in `PropertyKindCalculus/Iso80000/Catalogue.lean` complete

```
structure PropertyKindCalculus.Iso80000.CataloguedKind : Type
```

Proof for Definition 31.1.1 uses 0

Realized as `structure CataloguedKind` over a `DimensionedKind`, with `cite` rendering e.g. `IEC 80000-13, Edition 2.0, 2025-02 item 13-24`. The `catalogue` lists all 42 items in item order — the

> traffic quantities (13-1 … 13-8), storage (13-9, 13-10), the information content and entropies (13-24 … 13-37), the rates (13-38 … 13-42), … — each paired with its unit symbol.

31.2. Dimension one, three kind families, incommensurable special units (R13)

IEC 80000-13's characteristic quantities all live at dimension one — and this is exactly the place the catalogue's thesis turns into a **unit** question. Three families share dimension one but carry *different special units*, and belong to *different kinds*:

- *Information content and entropy* (13-24 …) in the *shannon* `Sh`, *hartley* `Hart`, or *natural unit* `nat` — alternative units of one kind, related by the logarithm base.
- *Traffic intensity* (13-1 …) in the *erlang* `E`.
- *Storage capacity* (13-9, 13-10) in the *bit* (or octet, or byte).

A dimension-only model sees one type, "a real number, dimension one", for all of them, and would let a bit of storage be silently read as an erlang of traffic. The shannon, erlang, and bit are *scale-spanning* units (R13), human-selected references for a dimensionless count — exactly like the candela (ISO 80000-7) and the mole (ISO 80000-9). The kind layer is what keeps them apart.

Theorem 31.2.1. group statement"> used 1 by 0 ✓

Tags

Effort: small

capstone

proved

> ***The shannon of information content and the erlang of traffic intensity are not commensurable, though both are dimension one (items 13-24, 13-1).*** *Two human-selected dimensionless references for two different kinds — a bit of information is not an erlang of traffic. The companion* `shannon_bit_not_commensurable` *separates information from storage. This is the scale-spanning collision (R13) on information. Uses the metrological unit.*

Lean code for Theorem 31.2.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part13.lean` complete

```
theorem PropertyKindCalculus.Iso80000.Part13.shannon_erlang_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part13.shannon.Commensurable
      PropertyKindCalculus.Iso80000.Part13.erlang
```

Proof for Theorem 31.2.1

uses 0

`shannon_erlang_not_commensurable` and `shannon_bit_not_commensurable`, each by `decide` after unfolding `MetrologicalUnit.Commensurable` on the dimension-one kinds. Axiom-free.

Theorem 31.2.2.

group statement">used by 0 ✓

Effort	Tags
small	proved

The dimension-1 collision, on the standard. *There exist distinct IEC 80000-13 kinds with the same dimension one — information content and traffic intensity witness it, alongside storage capacity, the entropies, and the probabilities. With distinct special units (shannon, erlang, bit), the dimension cannot separate them; the kind layer does. Uses [the dimension map](#).*

Lean code for Theorem 31.2.2.

1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part13.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part13.iec80000_13_dim_one_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase ∧
        a.dim LTMCTDimensionBase = 1
```

Proof for Theorem 31.2.2

uses 0

`iec80000_13_dim_one_collision`, witnessed by `⟨informationContent, trafficIntensity, …⟩` with the kind distinction a `decide` and reflexivity. Axiom-free.

31.3. The information rates re-dimension to T^{-1}

Per-character entropies are dimension one, but the *per-second* rates re-dimension to T^{-1}: the average information rate, the average transinformation rate, the channel (time) capacity, the bit and transfer rates, the call intensities. They collide there — distinct kinds, one dimension — just as the dimension-one quantities collide at the point one.

Theorem 31.3.1. group statement">uses 1 · used by 0 ✓

Effort	Tags
small	proved

> ***The bit rate is not the call intensity, though both are T^{-1} (items 13-13, 13-7).*** *There exist distinct IEC 80000-13 kinds with the same dimension T^{-1} — the bit rate and the call intensity witness it, alongside the transfer and modulation rates and the information rates. Dimension cannot separate them; the kind layer does. Uses the dimension map.*

Lean code for Theorem 31.3.1. 1 theorem

`theorem` · defined in `PropertyKindCalculus/Iso80000/Part13.lean` — complete

```
theorem PropertyKindCalculus.Iso80000.Part13.iec80000_13_dim_collision :
  ∃ a b,
    a.kind LTMCTDimensionBase ≠ b.kind LTMCTDimensionBase ∧
      a.dim LTMCTDimensionBase = b.dim LTMCTDimensionBase
```

Proof for Theorem 31.3.1 uses 0

> `iec80000_13_dim_collision`, witnessed by `⟨binaryDigitRate, callIntensity, …⟩`. The shared dimension is `binaryDigitRate_dim_eq_callIntensity_dim`; the kind distinction is a `decide`. Axiom-free.

31.4. The algebraic Remarks as kind-laws: digital transmission built by the kind algebra

Several Part-13 definitions state a quantity's defining relation *algebraically*. The signal energy per binary digit is the *product* of carrier power and the bit period (13-19, `E_bit = P_c·T_bit`); the periods are *reciprocals* of the rates (13-12, 13-14). Each is specified as an R12 kind-law.

Theorem 31.4.1. statement">uses 1 · used by 0

Effort	Tags
small	proved

> ***Signal energy per bit is carrier power × bit period — a verified construction (item 13-19).*** *A signal energy built as the product of a carrier power and a bit period carries its product certificate by construction, over the $\mathbb{R}$ carrier. The companion `signalEnergy_dim_from_power_period` shows it is $M·L^2·T^{-2}$ because it is a power times a time. Uses the quotient kind-law.*

Lean code for Theorem 31.4.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part13/DefiningRelations.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part13.DefiningRelations.signalEnergyOf_is
  (p :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part13.carrierPower.kind
        LTMCTDimensionBase)
      ℝ)
  (t :
    PropertyKindCalculus.Quantity
      (PropertyKindCalculus.Iso80000.Part13.periodOfBinaryDigits.kind
        LTMCTDimensionBase)
      ℝ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part13.DefiningRelations.signalEnergy_prod_power
_period
    (PropertyKindCalculus.Iso80000.Part13.DefiningRelations.signalEnergyOf
      p t)
    p t
```

Proof for Theorem 31.4.1 uses 0

> `signalEnergyOf p t := Quantity.mul signalEnergy_prod_power_period p t`, and `signalEnergyOf_isProduct` is `rfl`. The reciprocal kind-laws `periodOfDataElements_recip_transferRate` and `periodOfBinaryDigits_recip_binaryDigitRate` give the periods, with `periodOfDataElements_dim_from_transferRate` the dimension. Axiom-free.

31.5. Item index — IEC 80000-13

Every catalogued item of IEC 80000-13, indexed by its printed item number, with the principal quantity symbol, the unit (the special named unit where the standard gives one), and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in. Symbols and unit strings are *citation locators*; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
13-1	traffic intensity	A	E	1
13-2	traffic offered intensity	A_o	E	1
13-3	traffic carried intensity	Y	E	1
13-4	mean queue length	L	1	1
13-5	loss probability	B	1	1
13-6	waiting probability	W	1	1
13-7	call intensity	λ	s^{-1}	T^{-1}
13-8	completed call intensity	μ	s^{-1}	T^{-1}
13-9	storage capacity	M	bit	1
13-10	equivalent binary storage capacity	M_e	bit	1
13-11	transfer rate	r	s^{-1}	T^{-1}
13-12	period of data elements	T	s	T
13-13	binary digit rate	r_{bit}	bit/s	T^{-1}
13-14	period of binary digits	T_{bit}	s	T
13-15	equivalent binary digit rate	r_e	bit/s	T^{-1}
13-16	modulation rate	r_m	Bd	T^{-1}
13-17	quantizing distortion	T_Q	W	$M \cdot L^2 \cdot T^{-3}$
13-18	carrier power	P_c	W	$M \cdot L^2 \cdot T^{-3}$
13-19	signal energy per binary digit	E_{bit}	J	$M \cdot L^2 \cdot T^{-2}$
13-20	error probability	P	1	1
13-21	Hamming distance	d_n	1	1
13-22	clock frequency	f_{cl}	Hz	T^{-1}
13-23	decision content	D_a	1	1
13-24	information content	I(x)	Sh	1
13-25	entropy	H	Sh	1
13-26	maximum entropy	H_0	Sh	1
13-27	relative entropy	H_r	1	1
13-28	redundancy	R	Sh	1

Item	Quantity	Symbol	Unit	Dimension
13-29	relative redundancy	r	1	1
13-30	joint information content	I(x,y)	Sh	1
13-31	conditional information content	I(x|y)	Sh	1
13-32	conditional entropy	H(X|Y)	Sh	1
13-33	equivocation	H_x(X|Y)	Sh	1
13-34	irrelevance	H_y(Y|X)	Sh	1
13-35	transinformation content	T(x,y)	Sh	1
13-36	mean transinformation content	T	Sh	1
13-37	character mean entropy	H′	Sh	1
13-38	average information rate	H*	Sh/s	T^{-1}
13-39	character mean transinformation content	T′	Sh	1
13-40	average transinformation rate	T*	Sh/s	T^{-1}
13-41	channel capacity per character	C′	Sh	1
13-42	channel time capacity	C*	Sh/s	T^{-1}

32. External Cross-References

This work formalizes Dybkær's **Ontology on Property** and aligns its concepts with the **International Vocabulary of Metrology** (VIM). The correspondence is recorded as typed, declaration-indexed metadata — two attributes, `@[dybkaer …]` and `@[vim4 …]`, attached to the declarations of the core spine — so the two index tables below are **generated from the source**, not maintained by hand. Each row maps an external locus to the external term and to this work's declaration, linked to its definition in this blueprint. Only locators are recorded; no normative text from either copyrighted source is reproduced.

32.1. Dybkær — An Ontology on Property

Indexed by Dybkær section, with the term as printed and this work's corresponding declaration.

René Dybkær, An Ontology on Property for Physical, Chemical, and Biological Systems, 2009

Locus	Term	This work	Note
§ 3.3	system	`System`	
§ 6	quantities of the same kind	`MutuallyComparable`	
§ 6.19	kind-of-property	`KindOfProperty`	
§ 7	examination method	`ExaminationMethod`	Ch. 7
§ 7	examination procedure	`ExaminationProcedure`	Ch. 7
§ 7.5	examination principle	`ExaminationPrinciple`	
§ 9.15	property value	`PropertyValue`	
§ 10.14	property value scale	`ValueScale`	
§ 12	operator-based scale types	`ScaleType`	the four-way division, Fig. 12.21
§ 13.3.3	quantity (number times reference)	`Quantity`	
§ 13.3.5	reciprocal of a quantity	`ReciprocalKind`	
§ 13.3.5	quotient of two quantities	`QuotientKind`	
§ 13.3.5	product of two quantities	`ProductKind`	
§ 13.5.1	unconditionally extensive unitary kind-of-quantity	`Extensive`	the value for the total equals the arithmetic sum of the values for the parts
§ 13.5.4	intensive kind-of-quantity	`Intensive`	invariant with the extent of a system OF CONSTANT COMPOSITION — the clause carried as the law's hypothesis
§ 18.12	metrological unit	`MetrologicalUnit`	
§ 20	component	`Component`	
§ 20	dedicated kind-of-property	`DedicatedKind`	the System — Component ; kind-of-property triple (IUPAC/IFCC; the System slot names the sort)
§ 20	sort of system	`SortOfSystem`	

32.2. VIM 4 2CD — International Vocabulary of Metrology

Indexed by VIM 4 2CD clause. The 2CD is a restricted committee draft; only clause numbers and terms are recorded.

The correspondence is mostly with this work's core spine — VIM's *quantity*, *ratio/interval/ordinal quantity*, *measurement scale*, *value of a quantity*, and the *measurement principle/method/procedure* chain map directly to PKC declarations. Three VIM concepts reach into the further layers: *quantity dimension* (1.9) and *quantity with the unit one* (1.10) are the `Dimension` layer's `dim` map and the dimension-one thesis; *quantity calculus* (1.29) is the `Interaction` algebra; the *international system of quantities* (1.8) is the ISO/IEC 80000 catalogue and the *coherent derived unit* (1.15) the unit each catalogued kind carries.

JCGM, International Vocabulary of Metrology — Basic and general concepts and associated terms (VIM), 4th edition, Committee Draft 2, 2023-07-31

Locus	Term	This work	Note
1.1	quantity	`Quantity`	
1.2	quantities of the same kind	`MutuallyComparable`	
1.3	ratio quantity	`IsRational`	
1.4	interval quantity	`IsDifferential`	
1.8	international system of quantities (ISQ)	`catalogue`	the ISO/IEC 80000 catalogue of quantity-kinds
1.9	quantity dimension	`toDimension`	the forgetful dim map into PhysLib
1.10	quantity with the unit one	`dim_not_injective`	the dimension-1 thesis: distinct kinds collapse to one dimension
1.12	measurement unit	`MetrologicalUnit`	
1.15	coherent derived unit	`CataloguedKind`	the coherentUnit of each kind
1.20–1.21	multiple / submultiple of a unit	`SIPrefix`	the decimal prefix factor
1.23	measurement scale	`ValueScale`	
1.24	value of a quantity	`PropertyValue`	
1.29	quantity calculus	`KMul`	kind interaction (KMul / KDiv)
1.30	quantity equation	`ProductKind`	realized as R12 kind-laws
1.33	ordinal quantity	`IsOrdinal`	
2.4	measurement principle	`ExaminationPrinciple`	
2.5	measurement method	`ExaminationMethod`	
2.7	measurement procedure	`ExaminationProcedure`	
2.12		`Relation`	

Locus	Term	This work	Note
	measurement model		the mathematical relation among the quantities involved, attached to a declared boundary as a checked theorem edge
2.13	measurement function	`RelationKind`	"f may symbolize an algorithm" — the four ways an implementing function may relate to its model: equals, inverts, refines, bounded by

Many VIM clauses have no PKC counterpart, for two reasons. Some are **metrological- procedure** concerns outside this ontology's scope — *true value* (1.26), *conventional value* (1.27), *measurement result* (2.10), *measured value* (2.11), *measurement model/function* (2.12–2.13), *influence quantity* (2.16), *correction* (2.17), and the *metrology*/*measurand*/*primary method* terms (2.2, 2.3, 2.6, 2.8, 2.9). Others are **not yet formalized**: the *base*/*derived quantity* and *base*/*derived*/*off-system unit* distinctions (1.6, 1.7, 1.13, 1.14, 1.18), *system of quantities*/*of units* as first-class objects (1.5, 1.16, 1.17, 1.19), *reference quantity* (1.11), and the *unit*/*numerical-value equation* forms (1.31, 1.32). The §1.22 *conversion factor between units* is, by contrast, now formalized — carried as a power-of-ten exponent (`conversionExponent`), with the conversion round-trip proved exactly over `\mathbb{Z}` and numerically over `\mathbb{R}` (requirement R17).

33. Using the library: annotations and generated indexes

The preceding chapters develop the calculus. This one is about **working in it**: the annotations an author writes, what each one changes, and the indexes the library generates from them.

Four things cannot be inferred from a definition and so must be stated by its author. Each is a family of annotations, and each has its own section below.

- **Where the calculus is left** — each site at which a value already carrying one kind is re-typed as another, the plumbing that must drop to the carrier, the point at which a kinded value becomes a naked one. The boundary family: `@[kindCrossing]`, `@[kindIngest]`, `@[kindConst]`, `@[carrierVocab]` and `@[kindEmission]` each name the tier that sanctions a boundary site; `@[kindCarrier]` is what brings a downstream carrier under the audit at all, `@[kindAttest]` registers a wrapper through which authored mints go on the record, and `@[kindCounterexample]` exempts a deliberate falsification probe from the provenance sweeps.
- **What a vocabulary answers for** — whether every dimension-one kind is individuated, every boundary carries a measurement model, every inversion says where it fails. The census marks: `@[kindPrincipleFree]`, `@[kindRelationFree]` and `@[kindInversionTotal]` record an honest negative with its reason, and `@[kindDiagnostic]` enrolls a conditioning output.
- **How a definition should read** — the notation, the constants, the intermediate bindings a rendered equation should keep. The rendering family: `@[pkc_math]`, steered by `@[pkc_math_symbol]`, `@[pkc_math_config]` and `@[pkc_math_transparent]`.
- **What an external document says** — which requirement a theorem discharges, which clause of Dybkær or VIM a definition formalizes. The metadata family: `@[requirement]`, `@[dybkaer]` and `@[vim4]`.

The distinction that matters most is between the first two families and the other two, and it is a distinction about what happens when an annotation is **absent**. A boundary site that carries no tier is reported as a violation by `#kind_boundary_audit`, and that report is pinned, so omitting `@[kindCrossing]` fails the build, and a dimension-one kind with neither a principle nor a `@[kindPrincipleFree]` mark fails `#kind_examination_clean` the same way: the annotation **discharges an obligation** about the code, in the way a proof discharges one about a proposition. Omitting a rendering or metadata annotation fails nothing — `@[pkc_math_symbol]` changes only how something reads. Read the **Enforcement** column below with that asymmetry in mind: it says `checked` for `@[pkc_math]` and `@[pkc_math_config]` too, but what those checks reject is a **misuse** of the annotation — an argument naming a binding the definition does not have, an attribute attached to something that is not a structure — never its absence.

Everything in the **Generated indexes** section at the end is computed from the environment when this chapter is built, scoped to this package's own worked examples. No row is maintained by hand, so a renamed declaration is a build error rather than a stale table.

33.1. The annotations at a glance

Each row links to the section that explains it.

Annotation	Attaches to	Effect	Enforcement
@[kindCrossing]	a definition	Two kinds can meet in two ways. Under a *law* — an authored edge such as `ProductKind k₁ k₂ k` or a `KindMul` entry — the calculus derives a third kind from two, and the use site needs no annotation, because the edge already licenses it. A *crossing* is the case no law covers: a value already carrying one kind is re-typed as another on the author's signature rather than on a derivation. An edge is a rule the calculus applies; a crossing is a place the calculus is left, which is why it must be declared and why the audit counts it. Both kinds are stated in the signature, on the argument side as much as the result; the mint or erasure inside is the crossing's mechanism, reviewed once.	checked — rejected at elaboration if no argument carries a registered carrier; #kind_boundary_audit also reports an untagged boundary site as a violation
@[kindIngest]	a definition	Declares a checked ingest mint: raw external/host data enters the calculus for the first time, admitted through some check. The dual of @[kindCrossing] — no argument need carry a kind, because none has been established yet.	checked — #kind_boundary_audit reports an untagged boundary site as a violation
@[kindConst]	a definition	Declares a constant mint: an adjudicated value enters the calculus as data — a cited table, a configuration bound,	checked — #kind_boundary_audit reports an untagged

Annotation	Attaches to	Effect	Enforcement
		a seed, a threshold, or a structural constant.	boundary site as a violation
@[carrierVocab]	a definition	Registers kind-preserving representation plumbing — an operation the kind algebra does not name (a branchless min, a map over a coefficient table) that must drop to the carrier.	checked — #kind_boundary_audit reports an untagged boundary site as a violation
@[kindEmission]	a definition	Marks a genuine emission boundary: a kinded value becomes a naked one for a consumer outside the calculus (a deploy driver, a tape recorder).	checked — #kind_boundary_audit reports an untagged boundary site as a violation
@[kindCarrier]	a structure	Registers a downstream single-field quantity-like carrier, so the boundary audit recognizes its mints and erasures — and so its records and operations are indexed.	checked — rejected at elaboration if the declaration is not a structure
@[kindCounterexample]	a Provenance.Contract or Provenance.Relation definition	Marks a deliberate provenance counterexample — a falsification probe kept because its checker refuses it. The by-type sweeps (#kind_contracts, #kind_contracts_decide, #kind_relations) list it as exempted instead of gating on it, and the contracts and provenance-coverage tables neither count it as a witness nor as a coverage subject.	checked — rejected at elaboration if the declaration is not a Provenance.Contract or Provenance.Relation
@[kindAttest]	a definition taking a `KindOfProperty` and a `String` argument	Registers an authored-mint attestor (`Quantity.attest` is built in): the boundary audit	checked — rejected at elaboration unless the declaration takes

Annotation	Attaches to	Effect	Enforcement
		reports each of its call sites with the reason string harvested from the argument, and skips the attestor's own body, so the wrapper's one raw mint is reviewed once, at registration, instead of hiding every site behind the wrapper's name.	both a `KindOfProperty` argument (the attested kind) and a `String` argument (the reason)
@[pkc_math]	a Quantity-valued definition	Renders the definition as typeset LaTeX into its own docstring, so doc-gen4 and the InfoView display the equation. Clauses `keeping` and `substituting` control let-bindings and derivations; a literal string argument overrides the rendering entirely.	checked — rejected if `keeping` names an unbound let, if `substituting` names a recursive definition, or if a literal override is combined with either clause
@[pkc_math_symbol "…"]	any declaration	Fixes the LaTeX a token renders as, overriding every naming heuristic. The optional `operator` modifier prints a one-argument application as juxtaposition rather than a call.	advisory
@[pkc_math_config]	a structure	Declares a configuration type: its field projections render as the qualified constant `Struct.field` rather than as a function applied to the configuration value.	checked — rejected at elaboration if the declaration is not a structure

Annotation	Attaches to	Effect	Enforcement
@[pkc_math_transparent]	a definition	Marks a notational wrapper the renderer should see through, so the wrapped term is rendered in place of a call to the wrapper.	advisory
@[requirement "…" role]	any declaration	Records that the declaration specifies, proves, implements or exemplifies a numbered requirement.	advisory
@[dybkaer "…" "…"]	any declaration	Records the Dybkær *Ontology on Property* section and term this declaration formalizes.	advisory
@[vim4 "…" "…"]	any declaration	Records the VIM 4 2CD entry and term this declaration corresponds to.	advisory
@[kindPrincipleFree "…"]	a `DimensionedKind` definition	Declares that a dimension-one kind is legitimately without an examination principle, with the reason: `#kind_examination_coverage` lists it as exempted and `#kind_examination_clean` does not fire on it. Inert on a kind that carries a principle.	checked — rejected at elaboration unless the declaration is a `DimensionedKind` and the reason is non-empty
@[kindRelationFree "…"]	a `Provenance.Contract` definition	Declares that a boundary legitimately has no theorem edge — its behavior is not a measurement model — with the reason: `#kind_relation_coverage` lists it as exempted and `#kind_relation_clean` does not fire on it. Inert on a boundary an edge names.	checked — rejected at elaboration unless the declaration is a `Provenance.Contract` and the reason is non-empty
@[kindInversionTotal "…"]	a `Provenance.Contract` definition	Declares that an inverted boundary legitimately has no conditional port — the	checked — rejected at elaboration unless the declaration is a

Annotation	Attaches to	Effect	Enforcement
		inversion is total on its input type — with the reason: `#kind_inversion_coverage` lists it as exempted and `#kind_inversion_clean` does not fire on it. Inert on a boundary that declares a conditional port.	`Provenance.Contract` and the reason is non-empty
@[kindDiagnostic "…"]	a `KindOfProperty` definition	Declares a kind to be a quality or conditioning output a consumer must read, with what it diagnoses: `#kind_diagnostic_coverage` then asks the declared boundaries in scope for a produced port carrying it. The mark is the census's enrollment — an unmarked kind is not walked, and removing a mark is the exemption.	checked — rejected at elaboration unless the declaration is a `KindOfProperty` and the description is non-empty

33.2. The commands at a glance

What the annotations **declare**, the commands **ask**. There are three forms of asking, and a command's **suffix** names which form it is — so the table below reads in pairs and triples, each base command followed by its suffixed siblings.

- A bare command **reads**: the answer goes to the InfoView, for a person.
- A reading frozen with `#guard_msgs` becomes a **record**. The whole answer is now reviewable in the diff, and a changed answer is a failed build. This is the form most of the audits take.
- A `_clean` command is the **gate that a record cannot be**, and the distinction is the reason it exists as a separate command rather than as a stricter pin. A record can be **re-blessed**: re-pinning a message whose summary reads `⚠ UNTAGGED — invariant 6/7 violation` leaves the build green with the invariant dead, and nothing about the pin itself distinguishes that from a legitimate update. The mechanism that makes an inventory reviewable is exactly the mechanism that lets the invariant be signed away. So a `_clean` command carries no message and pins nothing: it throws. There is nothing there to re-bless.

`_decide` is the fourth suffix and a different axis: the same check run in the **kernel**, leaving a theorem behind rather than a message. `#kind_contract_decide` does not print that a boundary agrees — it adds `c.kindContractOk` and lets `#print axioms` answer for it.

Reading the table, then: the **Pinned** column says what freezing a command's answer buys, and reads – for the commands whose job is to be run by hand.

Command	Library	What it answers	Pinned
#kind_boundary_audit ns …	PropertyKindCalculus	Walks every compute definition in the namespaces and reports each that mints or erases a registered carrier, with the kinds it mints and the tier that sanctions it; an untagged boundary site is reported as a violation.	a new anonymous interior mint fails the build, the way a new axiom fails a pinned axiom profile
#kind_boundary_clean ns …	PropertyKindCalculus	The invariant rather than the inventory: throws when any boundary-active declaration in scope carries no tier attribute. It carries no message and pins nothing, so re-blessing the audit's pin cannot silence it.	the command is the gate; there is no message to re-bless
#kind_mint_ratchet ns …	PropertyKindCalculus	Throws on a raw carrier mint inside a @[kindCrossing] or @[carrierVocab] body, where every mint must instead be a licensed derivation or a Quantity.attest whose reason is harvested. @[kindConst], @[kindIngest] and @[kindEmission] keep raw mints legal at the declaration granularity.	the command is the gate; at the ratcheted tiers a new anonymous mint fails the build instead of joining a list nobody re-reads
#kind_crossings [ns …]	PropertyKindCalculus	Enumerates the tagged boundary registry — every sanctioned site with the first line of its docstring, grouped by tier.	—
#kind_edges k	PropertyKindCalculus		

Command	Library	What it answers	Pinned
		Prints every authored kind-algebra edge mentioning the kind `k` — named witness theorems, call-site witnesses lifted out of definitions, and operator-table registrations.	the complete set of edges a kind supports is fixed, so a new one is a visible change
#kind_dimensional_coverage ns …	`Dimension`	Resolves every authored edge's kinds to their declared DimensionedKind and evaluates the family's dimensional rule, reporting each edge as coherent, parametric, undimensioned, conflicting, or incoherent.	an edge over a kind carrying no dimension, or one whose dimensions do not balance, fails the build
#kind_dimensional_clean ns …	`Dimension`	The coverage invariant rather than its record: throws while any edge in scope is undimensioned, conflicting or incoherent. Parametric generic vocabulary is not a violation. Its one false alarm is an under-imported closure, which the error says.	the command is the gate; there is no message to re-bless
#kind_examination_coverage ns …	`Dimension`	Walks every dimension-one `DimensionedKind` in the namespaces and reports each as individuated by its examination principle, exempted by an `@[kindPrincipleFree]` mark, or UNINDIVIDUATED — the census behind the model	a dimension-one kind declared without a principle or a mark fails the pin

Command	Library	What it answers	Pinned
		template's M6, which the dimension functor cannot supply because it sends every such kind to `1`.	
#kind_examination_clean ns …	`Dimension`	The census as a gate: throws while any dimension-one kind in scope carries neither an examination principle nor a declared exemption. Records an audit receipt exactly when it does not fire, which is what lets a document's M6 read green only in a build where the census ran.	the command is the gate; there is no message to re-bless
#kind_relation_coverage ns …	`PropertyKindCalculus`	Walks every `Provenance.Contract` in the namespaces and reports each as witnessed by the theorem edges naming it (relations walked unscoped, counterexamples excluded), exempted by a `@[kindRelationFree]` mark, or UNWITNESSED — the census behind M12.	a boundary declared anywhere in scope that no edge names fails the pin
#kind_relation_clean ns …	`PropertyKindCalculus`	The census as a gate: throws while any declared boundary in scope is unwitnessed and unexempted. Records an audit receipt exactly when it does not fire.	the command is the gate; there is no message to re-bless
#kind_mereology_coverage ns …	`PropertyKindCalculus`	Walks every produced port (`output` or `conditional`) of every `Provenance.Contract` in	a produced port with no `aggregations`

Command	Library	What it answers	Pinned
		the namespaces and reports each as classed by its declared aggregation class or UNDECLARED — the census behind M15. No exception mark: the class vocabulary names every honest negative.	entry fails the pin
#kind_mereology_clean ns …	`PropertyKindCalculus`	The census as a gate: throws while any produced port in scope carries no aggregation class. Records an audit receipt exactly when it does not fire.	the command is the gate; there is no message to re-bless
#kind_inversion_coverage ns …	`PropertyKindCalculus`	Walks every `Provenance.Contract` in the namespaces that an `inverts` edge names as its left side and reports each as guarded by a `conditional` port (with its decider where named), exempted by a `@[kindInversionTotal]` mark, or UNGUARDED — the census behind M21's domain half.	an inverted boundary with no conditional port and no exemption fails the pin
#kind_inversion_clean ns …	`PropertyKindCalculus`	The census as a gate: throws while any inverted boundary in scope is unguarded and unexempted. Records an audit receipt exactly when it does not fire.	the command is the gate; there is no message to re-bless
#kind_wellposedness_coverage ns …	`PropertyKindCalculus`	Walks every `inverts` edge (`Provenance.Relation`) in the namespaces and reports each as well-posed	an inverts edge declaring neither a

Command	Library	What it answers	Pinned
		by its `wellPosed` witness on its declared `domain`, surfaced by its `ambiguity` witness, or UNDECIDED — the census behind M20 and M21's ambiguity half. No exception mark: the two fields are total over the honest negatives.	well-posedness nor an ambiguity fails the pin
#kind_wellposedness_clean ns …	`PropertyKindCalculus`	The census as a gate: throws while any `inverts` edge in scope answers for its inversion with neither a well-posedness witness nor a surfaced ambiguity. Records an audit receipt exactly when it does not fire.	the command is the gate; there is no message to re-bless
#kind_diagnostic_coverage ns …	`PropertyKindCalculus`	Walks every `@[kindDiagnostic]`-marked kind in the namespaces and reports each as exported by the produced ports carrying it or SIDECHANNELED — the census behind M22. Enrollment is the mark, so a scope with no marks records that visibly.	a marked kind no declared boundary exports fails the pin
#kind_diagnostic_clean ns …	`PropertyKindCalculus`	The census as a gate: throws while any diagnostic kind in scope is carried by no produced port. Records an audit receipt exactly when it does not fire.	the command is the gate; there is no message to re-bless
#kind_scc [ns …]	`Graph`	The kind-level component report: the authored kinds in scope, their licensed-	the reviewed enumeration to pin beside

Command	Library	What it answers	Pinned
		derivation edge count, every inter-derivability cluster of two or more kinds with the edges that wire it, and whether the kind digraph is acyclic.	#kind_edges — a new cluster is a visible change
#kind_scc_clean [ns …] [(k …) …]	`Graph`	Inter-derivability as a gate: every cluster of two or more mutually derivable kinds must be declared as a parenthesized group at the same size; an undeclared cluster throws with its wiring printed.	a new witness registration that merges two kind roles fails the build at the vocabulary level, before any value walks the new cycle
#kind_scc_d2 "dir" [ns …]	`Graph`	Writes the D2 diagram sources of the component findings into a directory — the condensation and one file per inter-derivability cluster. The script-facing twin of #kind_scc, run from a `lake env lean` driver.	—
#kind_footprint c ns…	`Graph`	The SCC footprint of a declared boundary against the kind graph: the component partition of its port and interior kinds, the same-kind port groups, the review-strength sites, and the verdict naming where the module's guarantee comes from.	the boundary's exposure to inter-derivability is fixed, so a widened footprint is a visible change
#kind_ports d	`PropertyKindCalculus`	Prints every port a declaration's kind-typed signature states — inputs,	a port that changes kind, or an

Command	Library	What it answers	Pinned
		configuration reads, outputs, each with its stated kind — followed by the signature's unkinded positions.	unkinded position that appears, fails the pin
#kind_occurrences d	`PropertyKindCalculus`	Prints every authored license discharged inline in a declaration's value — each consuming application's edge with the operand quantities that met there, in body order, with multiplicity.	the licenses a body actually discharges are fixed, so a new one is a visible change
#kind_graph d	`PropertyKindCalculus`	Prints the constructed step graph — ports, introduction events, wired occurrences with their assumed/partial markers, exits — and the evaluated well-formedness verdict.	the wiring is fixed, so a rewired body is a visible change
#kind_graph_decide d	`PropertyKindCalculus`	Reflects the constructed graph into a term and adds the theorem `d.kindGraphWf`, proved by `decide` — kernel reduction of the structural checker on the harvested object. Errors out before troubling the kernel when the graph is not well-formed.	the command is the gate; the pin freezes the kernel receipt
#kind_assembly [d_1, d_2, …]	`PropertyKindCalculus`	Assembles the listed declarations into one multi-step graph and prints it — each level with its inclusion mode, the union graph, the citation relation, and the evaluated verdict.	the assembled wiring is fixed across the whole set, not one declaration at a time

Command	Library	What it answers	Pinned
#kind_assembly_decide [d_1, d_2, …]	PropertyKindCalculus	Reflects the assembled union graph into a term and adds the theorem d_1.kindAssemblyWf, proved by decide. Errors out before troubling the kernel when the assembly is not well-formed.	the command is the gate; the pin freezes the kernel receipt
#kind_contract c	PropertyKindCalculus	Assembles the members a contract declares and compares that assembly's boundary with the boundary the contract declares — parameters and decided conditional ports, then what each side has and the other does not, then the verdict. Where #kind_assembly says the wiring holds together, this says the members are the ones the declared interface belongs to.	a boundary that stops being the one its members compute fails the pin
#kind_contract_decide c	PropertyKindCalculus	Adds the theorem c.kindContractOk : c.Agrees (graph), proved by decide — kernel reduction of the boundary comparison against the author's own contract definition, which the proposition names rather than copies. Errors out when the boundaries disagree.	the command is the gate; the pin freezes the kernel receipt
#kind_contracts ns …	PropertyKindCalculus	Surveys every Provenance.Contract declared under the	a boundary declared anywhere in

Command	Library	What it answers	Pinned
		namespaces — membership is by type, so declaring a boundary enrolls it — re-checking each as `#kind_contract` does, with violations as ✗ rows and `@[kindCounterexample]` declarations as exempted ⊘ rows.	scope and inconsistent with what its members compute fails the pin, including one nobody remembered to check by name
#kind_contracts_decide ns …	`PropertyKindCalculus`	The same sweep as a hard gate with kernel receipts: any violation is an error, and each passing contract gains the kernel theorem `c.kindContractOk` unless it already stands.	the command is the gate; the pin freezes the kernel receipts
#kind_relation r	`PropertyKindCalculus`	Checks the theorem edge a `Provenance.Relation` declares between two contracts — a sorry-free witness, the conclusion in the claimed shape, the tolerance, hypothesis and license clauses each answered for by name — and prints it.	every failure throws, so the command is the report and the gate at once — there is no reading of it that states a violation
#kind_relations ns …	`PropertyKindCalculus`	The by-type edge survey: every `Provenance.Relation` under the namespaces re-checked as `#kind_relation` checks one, failures as ✗ rows and counterexamples as ⊘ rows, the header counting	a newly violated edge fails the pin, and so does a vanished one or an exemption nobody declared

Command	Library	What it answers	Pinned
		edges, violations and exemptions.	
#kind_discharges c d	`PropertyKindCalculus`	Compares two contracts with no graph and no harvest: what the deploying contract did with each parameter the deployed contract handed it, and whether anything was left unanswered.	a parameter that stops being answered is a visible change
#kind_discharges_decide c d	`PropertyKindCalculus`	Adds the theorem `c.kindDischarges.d : c.Discharges d`, proved by `decide` — kernel reduction of the tier relation on the two authors' own contract definitions. Errors out when the tiers do not stack, printing what is unanswered.	the command is the gate; the pin freezes the kernel receipt
#kind_unkinded c	`PropertyKindCalculus`	Prints the unkinded ledger of the scope a contract declares: the count, then one line per naked position and per unkinded flow, deduplicated by member.	the pin fails when a silence appears AND when one is fixed, which is what keeps the ledger honest about which direction it moved
#kind_unkinded_clean c	`PropertyKindCalculus`	The ledger as a gate: errors unless the scope's ledger is empty. Separate from the pin above for the reason every violation-capable pin needs a	the command is the gate; there is no message to re-bless

Command	Library	What it answers	Pinned
		separate gate — pinning a violation is how it stops being noticed.	
#kind_contract_propagation c	`PropertyKindCalculus`	The propagation relation of a contract over the assembly its members compute: which source ports reach which produced ports.	—
#kind_output_ledger c	`PropertyKindCalculus`	The per-output assumption ledger and trust decomposition of every produced port of a contract, over the assembly its members compute.	—
#kind_budget b c	`Uncertainty`	Checks an uncertainty budget's attachment to a boundary and renders it: the port must be a produced port at the budget's kind, the assembly acyclic, every term naming a source that influences the port — and the influencing sources the budget omits render as `unbudgeted source(s)`, so what is not propagated is in the report rather than absent.	every violation throws, so the command is the report and the gate at once
#pkc_index "…" [ns …]	`Index`	Prints one generated index as plain text — the same table a document renders, available without building a document.	a derived table that stops matching is a build failure rather than an absence

Command	Library	What it answers	Pinned
			nobody notices
#pkc_summary_overflow [ns …]	`Index`	Reports the docstrings an index table quotes whose first paragraph is too long for the column that quotes it, widest first.	—
#pkc_index_page "…" ["…", …] [ns …]	`Index`	Writes the named indexes into the elaborating module's own module docstring, so doc-gen4 renders them as that module's page.	—

33.3. Where the calculus is left — the boundary family

A kind crossing must be **authored**. The calculus will not infer that two kinds may meet, and the audit will not accept that they did without being told. That is the whole design: the set of authored crossings is a closed, enumerable statement of where the type discipline is deliberately set aside, in the way `#print axioms` is a closed statement of what a proof rests on.

The audit detects a boundary structurally, not textually. A grep cannot see through `⟨…⟩` constructor sugar, structure-eta projections, or the `match_N` and `_proof_N` auxiliaries Lean lifts out of a definition — and it cannot tell a parity **theorem about** `.magnitude` (legitimate, a statement in `Prop`) from a compute body that exits the calculus (a defect). `#kind_boundary_audit` walks the environment instead, and reports every declaration that constructs or projects a registered carrier.

33.3.1. `@[kindCrossing]` — an authored crossing

Two kinds can meet in two ways, and only one of them is a crossing. Under a **law** — an authored edge, `ProductKind k₁ k₂ k` or a `KindMul` entry — the calculus derives a third kind from two, and the use site needs no annotation at all, because the edge already licenses it. Those are the edge audits' subject, not this family's.

A **crossing** is the case no law covers: a value already carrying one kind is re-typed as another on the author's signature rather than on a derivation. An edge is a rule the calculus applies; a crossing is a place the calculus is left. That asymmetry is why one is inferred at every use and the other must be declared once and counted.

Both kinds are stated in the signature — on the argument side as much as the result: `add` rejects a `@[kindCrossing]` declaration none of whose arguments carries a registered carrier, because a crossing goes FROM an already-kinded value, and a declaration with no kinded argument has nothing to cross from. The mint or erasure inside is the crossing's mechanism, reviewed once.

```
/-- A tagged crossing minting the probe output kind from an already-kinded gain value. -/
@[kindCrossing]
def probeCrossing (g : Quantity gainKind Float) : Quantity outputKind Float :=
⟨g.magnitude⟩
```

A declaration with no kinded argument at all — `probeCrossing (x : Float) : Quantity outputKind Float := ⟨x⟩` — is rejected at the `@[kindCrossing]` line itself, with a message pointing at the two tiers below it is actually one of: `@[kindIngest]` if something is checked on the way in, `@[kindConst]` if it is a fixed literal.

In the InfoView. Nothing else changes on the declaration itself. The effect is on the audit: `#kind_boundary_audit` prints one line per boundary-active declaration, and the tier tag is what distinguishes a sanctioned site from a violation.

```
boundary audit:
[kindCrossing] …probeCrossing — mints: outputKind
[carrierVocab] …probeVocab — mints: gainKind
[kindEmission] …probeEmission — erases (emission-only)
[kindIngest] …probeIngest — mints: outputKind
⚠ UNTAGGED …violationSite — mints: outputKind
```

The last line is the point. An untagged mint is reported as a violation, and because the report is pinned with `#guard_msgs` in an indexed probe, a **new** interior boundary fails the build — the same discipline by which a pinned axiom profile catches a proof silently relocated behind a `sorry`.

And the two gates beside it. `#kind_boundary_clean ns…` states the same invariant without a message: it throws when any boundary-active declaration in scope carries no tier. The audit says what the boundary **is**; this says every one of its sites has been adjudicated, and re-blessing the first cannot silence the second because there is nothing here to re-bless. `#kind_mint_ratchet ns…` tightens the granularity. A tier tag sanctions a whole **declaration**, so a `@[kindCrossing]` body can hold any number of interior `⟨…⟩` under one sanction; the ratchet requires that at the crossing and carrier-vocabulary tiers every mint be a licensed derivation or a `Quantity.attest` whose reason is harvested — the raw column stays empty, so a new anonymous mint fails the build instead of joining a list nobody re-reads. `@[kindConst]` and `@[kindIngest]` keep raw mints legal at the declaration granularity, because there the declaration's own evidence is the sanction.

In the generated indexes. Every tagged site appears in the kind-crossing table with the kinds it mints, taken from the audit's own walk so the two cannot disagree. `#kind_crossings [ns…]` is the same registry from the InfoView — every sanctioned site with the first line of its docstring, grouped by tier. It reads the

registry where the audit walks the **environment**, which is why the audit can report a site the registry does not contain: that site is the violation.

33.3.2. `@[kindIngest]` — a checked ingest mint

The dual of `@[kindCrossing]`: raw, external data — a host column, a JSON blob, an env-var's text — enters the calculus for the first time, admitted through some check (a `KindAdmissible`/ `BatchAdmissible` instance, an inline range test, a fallible parse). No argument carries a kind, because none has been established yet — that is the entire point of an ingest boundary, and the reason `@[kindCrossing]`'s own check must not reject it: the check performed is this declaration's evidence, stated once, rather than presupposed.

```
/-- A checked ingest mint: a raw carrier value enters the calculus as the probe output
kind. -/
@[kindIngest]
def probeIngest (x : Float) : Quantity outputKind Float := ⟨x⟩
```

33.3.3. `@[kindConst]` — a constant mint

The third mint tier, and the one a nullary crossing candidate turns out to be. Where `@[kindCrossing]` goes **from** an already-kinded value and `@[kindIngest]` admits external data through a check, `@[kindConst]` mints from neither: the value is adjudicated **data** — a cited coefficient table, a configuration bound or box, a seed, a threshold, or a structural constant of the model such as a zero accumulator or the vacuum index `1`.

```
/-- The probe's adjudicated threshold, cited to the table it is read from. -/
@[kindConst]
def probeThreshold : Quantity outputKind Float := ⟨0.02⟩
```

The mint's value is data rather than dataflow, so its provenance is the declaration's docstring rather than its signature — there is no argument to trace it to. That is why the tier exists as its own tag instead of collapsing into `@[kindCrossing]`: a declaration with no kinded argument is **definitionally** this tier, having nothing to cross from, and the crossing attribute's rejection message names it for exactly that reason.

33.3.4. `@[carrierVocab]` — a carrier-vocabulary exception

Representation plumbing for an operation the kind algebra does not name: a branchless minimum, a square root on the complex carrier, a `map` over a coefficient table. Kind-preserving by construction, which is what distinguishes it from a crossing.

```
/-- A carrier-vocabulary exception on the probe kinds. -/
@[carrierVocab]
```

```
def probeVocab (a b : Quantity gainKind Float) : Quantity gainKind Float :=
  ⟨min a.magnitude b.magnitude⟩
```

33.3.5. `@[kindEmission]` — a genuine emission boundary

Where a kinded value legitimately becomes a naked one for a consumer outside the calculus: a deploy driver, a tape recorder, a serializer.

```
/-- An emission boundary out of the probe world. -/
@[kindEmission]
def probeEmission (x : Quantity outputKind Float) : Float := x.magnitude
```

33.3.6. `@[kindCarrier]` — teaching the audit a new carrier

The audit recognizes a boundary as an application of a **carrier structure**'s constructor or first-field projection. `Quantity` and `CertifiedQuantity` are built in; a downstream layer registers its own — soil-moisture-model registers its `DedicatedQuantity`, which carries a `DedicatedKind` rather than a `KindOfProperty`.

```
/-- A toy single-field carrier, standing in for the model's `DedicatedQuantity`. -/
@[kindCarrier]
structure Tagged (k : KindOfProperty) (R : Type) where
  magnitude : R
```

Registering a carrier is the single opt-in that also earns a layer the two **structural** indexes: the kinded-record and kinded-operation tables recognize a structure or definition by whether a registered carrier appears in its fields or its result type. A layer therefore cannot register for the audit and be missed by the indexes, or the reverse.

Carrier	Origin	What it is
`Quantity`	built in	**A quantity (ℝ10).** A magnitude of a fixed kind `k`, carried at a representation type `R`.
`CertifiedQuantity`	built in	**A quantity bundled with its discharged admissibility evidence** — `cert` is the gate's receipt, kept in the type rather than asserted at the call site.
`NominalValue`	built in	**§12.4 nominal property value**, type-indexed. A designation of the fixed nominal kind `k`, carried at representation `R`. The nominal dual of `Quantity`: same indexing …

Why the crossing table is not here. The core spine has no kind crossings, and that is a fact about what the spine **is** rather than an omission: a crossing is where a model meets a measurement, so

crossings live downstream. The populated table — twenty-three sites, with the kinds each one crosses — is in soil-moisture-model's technical reference. This chapter's boundary examples come from the validation probe that pins the audit's behaviour.

33.3.7. `@[kindAttest]` — a registered attestor

An *attested* mint is an authored `⟨…⟩` written through a registered attestor that names its reason at the call site; `Quantity.attest` is the built-in one. The audit cannot see through a named wrapper — a helper's body is walked once, under the helper's own name, so a declaration that mints through an unregistered wrapper shows nothing at its own site. Registration turns that hiding into accountability: the audit recognizes a registered attestor's applications, reports each site with the reason string harvested from the argument, and skips the attestor's own body, whose one raw mint is the sanctioned mechanism, reviewed at registration — the same reason a carrier's own constructor is not a site. The attribute derives the attestor's footprint from its signature, and refuses a declaration that does not carry both arguments it needs:

```
`@[kindAttest]` expects a declaration taking a `KindOfProperty` argument (the attested
kind) and
a `String` argument (the reason) — '…' has neither or only one. Without both, attested
sites
could not be harvested, and the wrapper would HIDE its mint from the walk instead of
putting it
on the record.
```

The discipline the registry serves: raw mints are the **suspect** column — a declaration-level tier covers however many its body holds, multiplicity unseen — and attested mints the **reviewed** one, each surviving site carrying its own one-line justification into the pinned report. An attestation is a claim with no machine-checkable evidence; where evidence exists, the licensed route — a `KindAdmissible` check at ingest, a `ProductKind` or `QuotientKind` witness edge — is what to write instead, and the reason should say why no check applies.

33.4. What the kind algebra rests on — the edge audits

The boundary audit asks where the calculus is **left**. These ask what it **rests on** — the laws under which two kinds may meet without a crossing.

A kind-level law — `ProductKind k₁ k₂ k`, a `KindMul` table entry, a `PowerKind` — is authored, not derived. The calculus signs any ratio-scale triple its author writes, exactly as a proof assistant accepts any axiom its author declares, and for the same reason: a claim about which physical quantities compose to which is not something a type checker can settle. What follows from that is not that the claims go unchecked, but that they must be **enumerable**.

`#kind_edges k` is that enumeration, per kind. It finds all three authoring styles in one scan over constant types — named witness theorems, the `_proof_N` auxiliaries Lean lifts out of call-site witnesses, and operator-table instances — so a reader sees every edge a kind participates in regardless of how it was written. This is what makes the **Algebra** column of the kind tables below worth reading: a kind whose Algebra cell is empty supports no arithmetic at all.

`#kind_dimensional_coverage ns …` then cross-checks those edges against the dimension layer, and it is the one place an authored edge can be **refuted** rather than merely listed. Each edge's kinds are resolved to their declared dimensioned kinds and the family's rule is evaluated in PhysLib's `Dimension` group — products add exponents, quotients subtract, powers scale by the rational exponent, a transcendental demands dimension one on both sides. An edge reports as:

```
[coherent]      relPermKind · relPermKind → relPermSqKind
[parametric]    pureNumber · k → k
⚠ UNDIMENSIONED angFreqKind · relaxTimeKind → dimlessKind — no DimensionedKind for:
relaxTimeKind
⚠ INCOHERENT    angFreqKind · relaxTimeKind → angFreqKind
⚠ CONFLICTING   … — disagreeing DimensionedKinds for: …
```

`[parametric]` is generic vocabulary — an edge quantified over a kind variable has no fixed dimensional content, and its instantiations are audited as their own rows. The two ⚠ verdicts that matter are different failures: **undimensioned** is a coverage gap, an edge nobody mirrored into the dimension layer, while **incoherent** is a genuine error, $T^{-1} \cdot T = T^{-1}$ asserted where the group says otherwise.

The division of labour with the curated interaction algebras is worth stating, because the two look similar and are not. This command is **total** and mechanical: it can only affirm that an edge's dimensions balance, and every dimension-one triple balances trivially — which is most of a soil-moisture model. An `InteractionAlgebra` is **curated**: it says which products are physically sanctioned, which is the thing dimensional analysis discards and no walk can recover. Coverage is checked; curation stays authored.

`#kind_dimensional_clean ns…` is the coverage report's gate, and stands to it exactly as `#kind_boundary_clean` stands to the boundary audit: no message, nothing pinned, it throws while any edge in scope is undimensioned, conflicting or incoherent. `[parametric]` does not fire it. It has one false alarm, and the error message says so: both the edges and the `DimensionedKind` declarations are harvested from the environment, so a module that reaches an edge but not the module declaring its kinds' dimensions reports that edge undimensioned while the codebase is coherent. That is the same import-closure sensitivity the generated indexes have, pointing the other way — an under-imported **report** silently misses rows, an under-imported **gate** invents violations. The asymmetry is the safe one, because a false alarm is loud and a silent omission is not.

33.5. What one declaration wires — the graph and incidence commands

The audits so far sweep **namespaces**. These read a single declaration, or the kind vocabulary as a whole, and they are what an author runs while writing rather than what a probe pins — though most of them pin, because a wiring that changes silently is the thing worth catching.

What a signature states, and what a body does. `#kind_ports d` prints the ports the declaration's kind-typed signature states — inputs, configuration reads, outputs, each with its kind — and then the signature's **unkinded** positions, which is the half a reader forgets to ask for. `#kind_occurrences d` prints the other side: every authored license the body discharges inline, each consuming application's edge with the operands that met there, in body order and with multiplicity. `#kind_graph d` puts the two together as the constructed step graph and evaluates its well-formedness, and `#kind_graph_decide d` proves that verdict in the kernel, leaving `d.kindGraphWf` behind. `#kind_assembly` and `#kind_assembly_decide` are the same pair over a **list** of declarations, assembled into one multi-step graph — which is what a boundary spanning several definitions needs.

What the kind vocabulary looks like as a graph. `#kind_scc [ns…]` reports the kinds in scope, their licensed-derivation edges — the edges `#kind_edges` reports, read from the same harvest, so a witness a theorem merely assumes wires nothing here either — and, the finding it exists for, every **inter-derivability cluster**: a set of two or more kinds each manufacturable from the others by licensed steps. Inside such a cluster the kind algebra alone cannot refuse a substitution, so a cluster is a real weakening of the discipline and one worth knowing about. `#kind_scc_clean` is its gate, and it is declarative rather than absolute: every cluster must be declared as a parenthesized group at its exact size, so a new witness registration that merges two kind **roles** fails the build at the vocabulary level, before any value walks the new cycle. `#kind_scc_d2` writes the same findings as D2 diagram sources for rendering, and `#kind_footprint c ns…` asks the question a module owner asks: which of those clusters this boundary's own kinds land in, and therefore where the module's guarantee actually comes from.

33.6. What a boundary claims — the provenance sweeps

The boundary audit asks where the calculus is left; the edge audits ask what it rests on. These ask whether a declared boundary is **the one its members compute**.

A `Provenance.Contract` is that declaration: the ports a boundary consumes, the exits it produces, and the member steps that realize it. It can disagree with itself — an input the members read and the contract never declared, a clause naming a supplier that answers no port — and the disagreement is not visible in any type, because a contract is data about a computation rather than a constraint on it.

Enrollment is by type, which is the design. Every constant whose type is headed by `Provenance.Contract` under the swept namespaces is a subject, so declaring a boundary enrolls it. There is no per-declaration command to remember and no registration to forget: a boundary cannot be

declared and left out of the sweep, which is the property a per-name command (`#kind_contract`, one contract) cannot offer however diligently it is used.

`#kind_contracts ns…` re-checks every subject exactly as `#kind_contract` checks one — the decider, aggregation and supplier clauses, then the declared boundary against the one the members compute — and renders one line per contract in declaration-name order, as a single `info` message suitable for `#guard_msgs` pinning. Namespaces elided, this is the validation probe's own pinned output:

```
kind contracts — 3 contract(s), 1 violated, 1 exempted
  ✗ …Bad.forgottenInput: 'sweep probe forward, x forgotten' — undeclared input fwd/x : aK
  …Good.fwdBoundary: 'sweep probe forward' — 2 ports, 0 exits, 1 member step(s)
  …Good.invBoundary: 'sweep probe retrieval' — 2 ports, 0 exits, 1 member step(s)
  ⊘ …Exempt.keptCounterexample: counterexample, exempted
```

Three things in that header rather than one. Because it counts the **checked subjects** as well as the violations, a pin fails on a boundary that vanished as readily as on one that broke; because it counts the exemptions beside them, exemption creep is a failed pin too (`#kind_relations` and `#kind_requirements` head their surveys the same way). And a scope with no contracts pins `0 contract(s)` rather than passing invisibly — the distinction between **checked and clean** and **nothing was checked**, which an empty report cannot make.

`#kind_contracts_decide ns…` is the same sweep as a hard gate. A violation is an error rather than a `✗` row, so no reading of the output can state one, and each passing contract gains the kernel theorem `c.kindContract0k`. Sweeping again re-proves nothing: a standing receipt reports as `already stands`. The two tiers answer different questions — whether the boundaries in scope agree, and whether the kernel has been made to say so.

The same three forms, per name. `#kind_contract c` and `#kind_contract_decide c` are the sweep's singular siblings, and what they add over `#kind_assembly` is the comparison: the assembly says the wiring holds together, the contract says the members are the ones the declared interface belongs to.

And the edges between boundaries. A `Provenance.Relation` states what one boundary claims of another — an inversion, a bound. `#kind_relation r` checks one: a sorry-free witness, the conclusion in the claimed shape, and the tolerance, hypothesis and license clauses each answered for by name. Every failure throws, so that command is report and gate at once. `#kind_relations ns…` is its by-type sweep, with the same `✗`/`⊘` rows and the same counted header. `#kind_discharges c d` asks the tier question instead — what a deploying contract did with each parameter the deployed one handed it — and `#kind_discharges_decide` proves the answer.

What is not kinded, and what is not propagated. Two commands report **absences**, which is the thing a checker cannot notice on its own. `#kind_unkinded c` prints the naked positions and unkinded flows in a contract's scope; pinned, it fails when a silence appears **and** when one is fixed, which is what keeps the ledger honest about which direction it moved — and `#kind_unkinded_clean` is the gate for a scope that has cleared them. `#kind_budget b c` checks an uncertainty budget's attachment to a boundary and

renders the influencing sources the budget **omits** as `unbudgeted source(s)`, so what the model is not propagating appears in the report rather than being absent from it. `#kind_contract_propagation` and `#kind_output_ledger` are the reading aids beside them: which source ports reach which produced ports, and the per-output assumption ledger.

33.6.1. `@[kindCounterexample]` — a deliberate misdeclaration, kept

A sweep that enrolls by type has one cost: a **falsification probe** — a contract written wrong on purpose, kept because its checker refuses it — enrolls too, and would fail the gate it exists to demonstrate. Deleting it would leave the checker's refusal untested; leaving it untagged would make the gate unpinnable.

```
/-- A deliberately misdeclared boundary, kept because the sweep must refuse it. -/
@[kindCounterexample]
def keptCounterexample : Provenance.Contract := …
```

A tagged subject renders as a ⊘ row and is **not** checked, so the probe stands beside the gate it exercises. The exemption is not a way out of the discipline, because it is counted: the header's exemption count is part of the pin, so a contract quietly exempted to make a build pass changes the pinned output. The mark reaches the generated tables on the same terms — a tagged contract is not a coverage subject, and a tagged relation is not a witness, since a deliberate misdeclaration must not discharge a real boundary's edge obligation.

The attribute checks what it is attached to, and says why:

```
`@[kindCounterexample]` expects a 'Provenance.Contract' or a 'Provenance.Relation' — '…
stray' is
neither. The mark exempts a declaration from the by-type provenance sweeps, and only those
two
types are swept.
```

33.7. What a vocabulary answers for — the coverage censuses

The audits so far ask about one declaration, one namespace's boundary, or one boundary's agreement with itself. The censuses ask the question the model template's rubrics put to a whole vocabulary: over a **population** the environment can enumerate — every dimension-one kind, every declared boundary, every produced port, every inversion edge — does each member declare what the rubric obliges it to? Six censuses share one shape, set by `#kind_examination_coverage`: a record command whose sorted `info` message is pinned, a `_clean` gate that throws and pins nothing, a declared exception where the population has honest negatives, and an audit receipt at each success point — which is what lets a

document's conformance table read a rubric as green only in a build where its census ran over the document's scope.

- `#kind_examination_coverage ns …` (M6) walks every dimension-one `DimensionedKind` in scope and reports it `[individuated]` by its examination principle, `⊘ exempted` by a `@[kindPrincipleFree]` mark, or `⚠ UNINDIVIDUATED`. Inside the dimension-one fiber the principle is the only defining aspect that separates two kinds, so a kind declared without one is named, not individuated. A `KindOfProperty` that no `DimensionedKind` wraps is reported `⚠ UNDIMENSIONED` rather than decided: "dimension one" is a fact about a dimension the vocabulary never declared, and a gate that passed over such kinds would pass over nothing.
- `#kind_relation_coverage ns …` (M12) walks every `Provenance.Contract` in scope and reports it `[witnessed]` by the theorem edges naming it, `⊘ exempted` by a `@[kindRelationFree]` mark, or `⚠ UNWITNESSED`. Relations are walked unscoped, since a theorem edge may live beside a deployment rather than beside the boundary it is about, and a `@[kindCounterexample]` relation is not a witness. Whether a named edge is **valid** is `#kind_relations`' question; this census asks only whether one exists.
- `#kind_mereology_coverage ns …` (M15) walks every produced port — `output` or `conditional` — of every contract in scope and reports it `[classed]` by its aggregation class or `⚠ UNDECLARED`. There is no exception mark: the `AggregationClass` vocabulary is total over the honest negatives — a port that must never be summed declares `.intensive`, one its parts do not determine declares `.wholeProper` — so declaring nothing is exactly the finding.
- `#kind_inversion_coverage ns …` (M21, the domain half) walks every contract that an `inverts` edge names as its left side — the boundary that recovers what another consumed — and reports it `[guarded]` by a `conditional` port, with its decider where one is named, `⊘ exempted` by a `@[kindInversionTotal]` mark, or `⚠ UNGUARDED`.
- `#kind_wellposedness_coverage ns …` (M20, and M21's ambiguity half) walks every `inverts` edge in scope and reports it `[well-posed]` by a `wellPosed` witness on its declared `domain`, `[surfaced]` by an `ambiguity` witness, or `⚠ UNDECIDED`. No exception mark: an inversion either has exactly one answer on a declared domain or it does not, and either answer is a declaration.
- `#kind_diagnostic_coverage ns …` (M22) walks every `@[kindDiagnostic]`-marked kind in scope and reports it `[exported]` by a produced port carrying it or `⚠ SIDECHANNELED`. Enrollment is the mark, so a scope with no marks records that visibly rather than passing in silence.

Each has its `_clean` sibling — `#kind_examination_clean`, `#kind_relation_clean`, `#kind_mereology_clean`, `#kind_inversion_clean`, `#kind_wellposedness_clean`, `#kind_diagnostic_clean` — which throws while any member of its population is a `⚠` row, and records the audit receipt exactly when it does not fire. A `@[kindCounterexample]` contract is a subject of none of them and is listed as exempted, so a census says what it skipped; a mark on a member that satisfies the predicate anyway is inert, so a stale exemption cannot hide a declaration made later; and a contract declared at node or kind types the census cannot read is reported `⚠ UNREADABLE` rather than walked past. Every census is import-closure

sensitive in the direction the other environment walks are: an exemption declared in a module the probe does not reach is a loud false alarm, never a silent pass.

The pinned form of the first, from the validation suite — the namespace prefix and two further `[individuated]` rows elided:

```
examination coverage:
[individuated] …probeIndividuated (probe individuated) — principle: probe-principle
⊘ exempted …probeGeometry (probe geometry) — the wave calculus's own geometry, the product
of no phenomenon
⚠ UNDIMENSIONED …probeBareKind (probe bare kind) — no DimensionedKind wraps it
⚠ UNINDIVIDUATED …probeBare (probe bare)
5 dimension-one kind(s): 3 individuated, 1 exempted, 1 UNINDIVIDUATED; 1 kind(s)
UNDIMENSIONED — examination-coverage violation
```

33.7.1. `@[kindPrincipleFree "…"]` — a dimension-one kind that is correctly principle-free

The declared exception to M6. A geometry vocabulary — an illumination cosine, a normalized wavenumber — is the product of no measurement phenomenon; a nominal designation has no principle to carry; a bookkeeping fraction is examined by nothing. Each is dimension one and each is correctly without an examination principle, and the mark is that decision as data the census can read, with its reason:

```
/-- Dimension one, no principle, exempted with a reason. -/
@[kindPrincipleFree "the wave calculus's own geometry, the product of no phenomenon"]
def probeGeometry : DimensionedKind :=
  { kind := { id := "probe geometry", scale := .ratio }, dim := 1 }
```

It attaches to the `DimensionedKind`, not to the `KindOfProperty`, because the population it exempts a kind from is "dimension one", a fact about the `DimensionedKind`. On a kind that does carry a principle the mark is inert — the census reports the principle and ignores the mark — so a stale exemption cannot hide an individuation supplied later. The attribute checks its target, and requires a non-empty reason:

```
`@[kindPrincipleFree]` expects a 'DimensionedKind' — '…notAKind' is not one. The mark
exempts a
dimension-one kind from the examination-coverage sweep, and dimension is a fact about the
DimensionedKind, so that is where the mark goes.
```

33.7.2. `@[kindRelationFree "…"]` — a boundary with no measurement model

The declared exception to M12, on a `Provenance.Contract`: a boundary whose behavior is not a measurement model — pure data movement, or the reference the theorem edges are **about** — said in the author's words where the census reads them.

```
/-- No edge, on purpose. Its port is classed. -/
@[kindRelationFree "a data-movement boundary: its behavior is the identity on kinds, and \
  there is no measurement model to carry"]
def dataMove : Provenance.Contract NodeId KindRef where …
```

Inert on a boundary some edge names. Rejected at elaboration unless the declaration is a `Provenance.Contract` and the reason is non-empty: the mark exempts a declared boundary from a census over declared boundaries, so that is where it goes.

33.7.3. `@[kindInversionTotal "…"]` — an inversion with no outside

The declared exception to M21's domain half, on the `Provenance.Contract` an `inverts` edge names as its left side: the inversion is total on its input type, so there is no failure outside the domain to detect and no `conditional` port to guard it.

```
/-- Inverted, no conditional port, exempted as total. -/
@[kindInversionTotal "total on its input type: every integer is the forward's image of one, \
  so there is no outside to detect"]
def totalInverse : Provenance.Contract NodeId KindRef where …
```

Inert on a boundary that declares a conditional port, and checked exactly as `@[kindRelationFree]` is.

33.7.4. `@[kindDiagnostic "…"]` — a conditioning output, enrolled

Not an exception but an **enrollment**, on a `KindOfProperty`: the kind is a quality or conditioning output a consumer must read, and the string says what it diagnoses.

```
/-- A quality kind the retrieval exports: marked diagnostic, and carried by a produced
port of `Good.invBoundary` below. -/
@[kindDiagnostic "whether the retrieval's answer was decided inside its domain"]
def qcK : KindOfProperty := { id := "contract coverage probe quality", scale := .ordinal }
```

`#kind_diagnostic_coverage` then asks the declared boundaries in scope for a produced port carrying it; a marked kind no boundary exports is a side channel. An unmarked kind is not walked, so removing the mark is the exemption, and a scope with no marks records that visibly. Rejected at elaboration unless the declaration is a `KindOfProperty` and the description is non-empty.

33.8. How a definition reads — the rendering family

`@[pkc_math]` renders a `Quantity` definition to LaTeX and writes it into that declaration's **own docstring**. Nothing else in the toolchain has to know: doc-gen4 typesets the docstring with MathJax and the InfoView renders the same markdown on hover, so one annotation reaches both surfaces.

Every rendering is **faithful** — it denotes exactly what the definition computes. Substitution is delta-expansion and keeping a `let` is a naming choice; neither asserts algebra the definition does not perform. The full policy, and the escape hatch for the cases it cannot serve, are in `RENDERING.md`.

33.8.1. `@[pkc_math]` — render this definition

```
/-- The vegetation attenuation `τ = exp(−2·b·ndvi)`, kind `attenuationK`. -/
@[pkc_math]
def attenuationQ (cfg : AvsConfig α) (b : Quantity paramB α)
    (ndvi : Quantity vegetationIndex α) : Quantity attenuationK α := …
```

The effect, appended to that declaration's docstring and typeset on its doc-gen4 page:

```
\tau = e^{-2\,b\,\mathrm{NDVI}}
```

Two clauses control the shape. `keeping` binds a definition's `let`s as named auxiliary equations below the main one instead of inlining them — right when a model shares one subexpression across several outputs, where inlining both explodes the LaTeX and hides the structure the author wrote. `substituting` adds a **derivation**: one further equation per clause, each inlining the named helpers, so a reader sees the equation as written and then the same equation expanded.

33.8.2. `@[pkc_math_symbol]` — fix the notation

Overrides every naming heuristic for one token.

```
attribute [pkc_math_symbol "n_{d}"]  MironovNK.nd
attribute [pkc_math_symbol "m_{vt}"] MironovNK.mvt
```

This is not always cosmetic. The rendering heuristic maps a Greek-letter **spelling** to its letter, so a refractive-index field named `nu` would typeset as `\nu` — an angular frequency. Notation stated once, on the field, wins over every heuristic, and the index below is therefore a table of the places where a heuristic was deliberately overruled.

Declaration	Renders as	Layout
`avsForward`	`\sigma^0`	function head
`byPolarization`	`P`	function head
`columns`	`J`	function head
`combined`	`D`	function head
`decay`	`\tau`	function head
`grad`	`\nabla`	prefix operator
`letExample`	`q`	function head
`model`	`F`	function head
`noted`	`\gamma`	function head
`residual`	`R`	function head
`scaled`	`g`	function head
`scaledByTwo`	`u`	function head
`shared`	`S`	function head
`twiceBound`	`T`	function head

33.8.3. `@[pkc_math_config]` — a configuration type

A projection of an ordinary structure renders as a function applied to its receiver: `\mathrm{two}\left(\mathrm{cfg}\right)`, which is the field name stripped of the type it belongs to plus a spurious application. Tagging the structure makes its projections render as the qualified constant `\mathrm{AvsConfig.two}` instead, with the configuration value elided.

```
@[pkc_math_config]
structure AvsConfig (α : Type) where
  two : Quantity pureNumber α
```

The type must opt in: a blanket "drop the receiver" rule would wreck `Prod.fst` and `Prod.snd`. Note what this does **not** do — it does not render `cfg.two` as `2`, which would assert something about a `def` parameter rather than report what the definition computes.

Type	What it is
`AvsConfig`	The AVS forward's configuration: the two-way optical-path factor `2`, as a kinded quantity — the sole numeral of the model.
`DemoConfig`	A configuration type: one dimensionless scale factor, the demo's sole configured constant.

33.8.4. `@[pkc_math_transparent]` — a notational wrapper

Marks a wrapper the renderer should see through, so the wrapped term is rendered in place of a call to it. The carrier's numeral injection is the motivating case: without this, every literal in a rendered equation appears wrapped in a helper nobody wants to read.

Wrapper	What it is
`ofN`	The carrier's numeral injection: a transparent wrapper.

33.8.5. The rendered definitions

Every `@[pkc_math]` application in the worked examples, with the clauses each requested.

Definition	Rendering mode	What it is
`attenuationQ`	plain	The vegetation attenuation `τ = exp(−2·b·ndvi)`, kind `attenuationK`.
`lavsForwardQ`	substituting (1 step(s))	The LAVS forward `σ⁰ = a·ndvi + (τ·c)·r + d`, kind `backscatter`.
`lavsJacResidualQ`	substituting (1 step(s))	The four analytic Jacobian columns `∂residual/∂(a,b,c,d)`, each at its forced kind.
`lavsResidualQ`	substituting (2 step(s))	The LAVS residual `s0 − σ⁰`, kind `backscatter` (subtraction forces the shared kind).
`avsForward`	plain	The AVS-style backscatter forward model, rendered on its doc page as `σ⁰ = a·ndvi + exp(−2·b·ndvi)·c·r + d`.
`byPolarization`	plain	Case analysis on the polarization, written vertical-first.
`columns`	plain	Three columns returned as a triple: exercises the `Prod.mk` flattening and both head layouts.
`combined`	plain	Consumes a destructured pair.
`model`	substituting (1 step(s))	One derivation step.
`noted`	plain	Base backscatter.
`residual`	substituting (2 step(s))	The residual of `model`: two cumulative steps, so the reader sees the call, then the call expanded, then the constant it bottoms out in.
`scaled`	plain	Reads a configuration constant.
`scaledByTwo`	plain	Doubling through the numeral injection.
`shared`	keeping	A model with a shared intermediate: `d` is used by both `s` and the sum.
`spread`	plain	A record-valued model: renders as the system `hi = a·b`, `lo = a − b`.
`twiceBound`	keeping	A definition that binds `u` twice, for different values.

33.9. External correspondence — the metadata family

`@[requirement]`, `@[dybkaer]` and `@[vim4]` record what an external document says about a declaration. All three are advisory, and all three already have their own generated index: the requirement-traceability

matrix in the **Requirements** section maps every numbered requirement to the declarations that specify, prove, implement or exemplify it, and the two correspondence tables in the **External Cross-References** chapter map Dybkær sections and VIM clauses to this work's declarations. They are listed in the reference table above for completeness and are not duplicated here.

33.10. The generated indexes

What these tables index. Every table in this section is scoped to `PropertyKindCalculus.Examples` — the calculus's own worked examples, which are what this package has to index. They are **not** the soil-moisture model: that model is a separate downstream repository, and its technical reference renders the same eight tables over its own namespaces, populated with its real kinds and crossings.

The rows nonetheless read as soil-moisture material, and it is worth saying why rather than letting it look like a leak. The calculus's worked examples deliberately run on the domain that motivated it: a vegetation-attenuation forward model, Mironov coefficients, soil and soil-water as systems. That is the point of a worked example — it has to be **of** something — and the alternative, examples over invented quantities, would demonstrate the machinery while hiding whether it survives contact with a real model. The generic examples are here too, in the same tables: length and width as distinct kinds, a blood group as a nominal scale, Mohs hardness as an ordinal one, the principle/method/procedure examination chain.

So the reading to avoid is that these tables document the model. They document **the examples this package ships**, and they exist here for the same reason the boundary probe does: a derived table nobody renders is a table nobody notices has broken.

33.10.1. Kinds and their algebra

The column that earns this table is **Algebra**. A kind's authored edges are the complete statement of what may be done with it — multiplied by what, divided into what, raised to what power — because the calculus infers none of them. A kind whose Algebra cell is empty supports no arithmetic at all, and being able to see that is the point.

Kind	Identity	Scale	Algebra	Theorems
`attenExponent`	AVS attenuation exponent	ratio	*produced (1)* • `paramB · vegetationIndex → attenExponent` *consumed (1)* • `transcendental : attenExponent → attenuationK`	`attenuation_roles_distinct`
`attenuationK`	AVS vegetation attenuation	ratio	*produced (1)* • `transcendental : attenExponent → attenuationK` *consumed (3)* • `jacB · attenuationK → jacB` • `attenuationK · paramC → paramC` • `attenuationK · reflectivity → reflectivity`	`attenuation_roles_distinct`
`backscatter`	AVS backscatter coefficient σ^0	ratio	*produced (2)* • `paramA · vegetationIndex → backscatter` • `paramC · reflectivity → backscatter`	
`jacB`	AVS backscatter times NDVI	ratio		

Kind	Identity	Scale	Algebra	Theorems
			produced (2) • `jacB · attenuationK → jacB` • `jacBPartial · reflectivity → jacB`	
`jacBPartial`	AVS NDVI times gain c	ratio	*produced (1)* • `vegetationIndex · paramC → jacBPartial` *consumed (1)* • `jacBPartial · reflectivity → jacB`	
`paramA`	AVS gain a (σ^0 per NDVI)	ratio	*consumed (1)* • `paramA · vegetationIndex → backscatter`	
`paramB`	AVS rate b (inverse NDVI)	ratio	*produced (1)* • `pureNumber · paramB → paramB` *consumed (1)* • `paramB · vegetationIndex → attenExponent`	
`paramC`	AVS gain c (σ^0 per reflectivity)	ratio	*produced (1)* • `attenuationK · paramC → paramC`	

Kind	Identity	Scale	Algebra	Theorems
			consumed (2) • `paramC · reflectivity → backscatter` • `vegetationIndex · paramC → jacBPartial`	
`pureNumber`	AVS pure number	ratio	*consumed (2)* • `pureNumber · paramB → paramB` • `pureNumber · vegetationIndex → vegetationIndex`	`attenuation_roles_distinct`
`reflectivity`	AVS surface reflectivity	ratio	*produced (1)* • `attenuationK · reflectivity → reflectivity` *consumed (2)* • `paramC · reflectivity → backscatter` • `jacBPartial · reflectivity → jacB`	
`vegetationIndex`	AVS vegetation index	ratio	*produced (1)* • `pureNumber · vegetationIndex → vegetationIndex`	

Kind	Identity	Scale	Algebra	Theorems
			consumed (3) • `paramB · vegetationIndex → attenExponent` • `paramA · vegetationIndex → backscatter` • `vegetationIndex · paramC → jacBPartial`	
`length`	length	ratio	*produced (1)* • `speed · time → length`	`length_is_speed_times_time`
`speed`	speed	ratio	*consumed (1)* • `speed · time → length`	`length_is_speed_times_time`
`time`	time	ratio	*consumed (1)* • `speed · time → length`	`length_is_speed_times_time`
`massFraction`	mass fraction	ratio		
`volumeFraction`	volume fraction	ratio	*produced (1)* • `volumeFraction ± volumeFraction → volumeFraction`	
`dl`	1	ratio	*produced (2)* • `dl · dl → dl`	`pk`, `tk`

Kind	Identity	Scale	Algebra	Theorems
			• `transcendental : dl → dl`	
`height`	height	ratio		
`length`	length	ratio		
`width`	width	ratio		
`mass`	mass	ratio		`mass_extensive`
`lengthK`	length	ratio		
`massK`	mass	ratio		`instAssemblesGridSMassK`
`length`	length	ratio		`hLen`
`mass`	mass	ratio		`hMass`
`lengthKind`	length	ratio	*produced (1)* • `lengthKind ± lengthKind → lengthKind`	`hLen`
`displacement`	displacement	ratio	*produced (2)* • `displacement ± displacement → displacement` • `factor · displacement → displacement` *consumed (1)* • `displacement · displacement → displacementSq`	`doubled_east`, `hDisp`, `hScale`, `hSquare`, `vector_add_comm`

Kind	Identity	Scale	Algebra	Theorems
`displacementSq`	displacement squared	ratio	*produced (1)* • `displacement · displacement → displacementSq`	`hSquare`
`factor`	scale factor	ratio	*consumed (1)* • `factor · displacement → displacement`	`hScale`
`backscatter`	backscatter coefficient	ratio		
`conductivityKind`	ionic conductivity	ratio		
`gwc`	gravimetric water content	ratio		
`ndvi`	vegetation index	ratio		
`reflBistatic`	reflectivity	ratio		
`reflMonostatic`	reflectivity	ratio		
`reflectivity`	reflectivity	ratio		`cholRowQ_magnitudes`
`refractiveIndexKind`	refractive index (n)	ratio		
`relPermKind`	relative permittivity	ratio		
`relaxTimeKind`	dielectric relaxation time	ratio		
`vwc`	volumetric water content	ratio		

Kind	Identity	Scale	Algebra	Theorems
`bloodGroup`	abo-blood-group	nominal		
`length`	length	ratio		
`mass`	mass	ratio		
`mohsHardness`	mohs-hardness	ordinal		
`bloodGroup`	abo-blood-group	nominal		
`length`	length	ratio		
`mass`	mass	ratio		
`height`	height	ratio		
`length`	length	ratio		
`width`	width	ratio		

33.10.2. The ontology

The Dybkær spine is inhabited by ordinary definitions, so nothing marks them as a family: the criterion is simply "a constant whose type is that structure". The dedicated-kind table renders each one as the `System – Component ; kind-of-property` triple it is — the `System` slot naming the **sort** — cross-linked to the sort and component tables rather than repeating their identity strings. The systems table lists the particulars; nothing dedicates to a particular, so it carries no dedicated-kinds column.

Sort	Identity	Dedicated kinds	Theorems
`soil`	soil	`airContent`, `gwc`, `vwc`	
`gridS`	four-cell grid		`instAssemblesGridSMassK`
`soilWater`	soil water	`boundStaticPerm`, `freeStaticPerm`	

System	Identity	Theorems

Component	Identity	Dedicated kinds	Theorems
`air`	air	`airContent`	
`water`	water	`gwc`, `vwc`	
`boundWater`	bound water	`boundStaticPerm`	
`freeWater`	free water	`freeStaticPerm`	

Dedicated kind	Sort of system	Component	Kind of property	Theorems
`airContent`	`soil`	`air`	`volumeFraction`	
`gwc`	`soil`	`water`	`massFraction`	
`vwc`	`soil`	`water`	`volumeFraction`	
`boundStaticPerm`	`soilWater`	`boundWater`	`relPermKind`	
`freeStaticPerm`	`soilWater`	`freeWater`	`relPermKind`	

Two dedicated kinds that share a kind-of-property are held apart only by their sort and component, and the **Theorems** column lists what proves it. A dedicated kind with no distinctness theorem is one that nothing yet relies on separating.

The examination chain needs no annotation either: a method is based on the principle it cites and a procedure on the method it cites, so `BasedOn` is structural, and the table below is the generating edges of the `Refines` closure.

Examination	Layer	Based on	Principle
`widthPrinciple`	principle	—	caliper-perpendicular
`widthMethod`	method	`widthPrinciple`	iso-width-method
`widthProcedure`	procedure	`widthMethod`	iso-width-procedure-v1

33.10.3. Kinded records and operations

Neither table needs an annotation to decide membership. A **kinded record** is a structure at least one of whose fields is a registered carrier applied to a kind; a **kinded operation** is a definition whose result type is one. `@[kindCarrier]` is the only opt-in, and it is already required for the audit.

For a record the useful column is the notation, because the fields are where `@[pkc_math_symbol]` attaches and a rendered equation's readability is decided.

Record	Field	Carrier	Kind	Notation
`AvsConfig`	`two`	`Quantity`	`pureNumber`	
`Bounds`	`hi`	`Quantity`	`dl`	
	`lo`	`Quantity`	`dl`	
`DemoConfig`	`scale`	`Quantity`	`dl`	
`CholRowQ`	`l20`	`Quantity`	`reflectivity`	
	`l21`	`Quantity`	`reflectivity`	
	`l22`	`Quantity`	`reflectivity`	
`MiniMironovCoeffs`	`ndA0`	`Quantity`	`refractiveIndexKind`	
	`taub0`	`Quantity`	`relaxTimeKind`	
	`sigbF0`	`Quantity`	`conductivityKind`	

For an operation it is the kind signature together with the authored crossings its body routes through — which makes the row a contract rather than a signature, since an operation that changes kind must reach a sanctioned crossing to do it.

Field projections are excluded. `MironovNK.nd` satisfies the criterion literally, its result type being `Quantity nIndexKind α`, but it is the record's accessor rather than an operation on kinds and is already a row of the table above.

Operation	Argument kinds	Result kind	Crossings
`attenuationQ`	`paramB`, `vegetationIndex`	`attenuationK`	
`lavsForwardQ`	`paramA`, `paramB`, `paramC`, `backscatter`, `vegetationIndex`, `reflectivity`	`backscatter`	
`lavsResidualQ`	`paramA`, `paramB`, `paramC`, `backscatter`, `vegetationIndex`, `reflectivity`, `backscatter`	`backscatter`	
`d`		`length`	
`dFake`		`length`	
`v`		`speed`	
`τ`		`time`	
`avsForward`	`dl`, `dl`, `dl`, `dl`, `dl`, `dl`	`dl`	
`byPolarization`	`dl`, `dl`	`dl`	
`combined`	`dl`, `dl`	`dl`	
`decay`	`dl`	`dl`	
`gain`	`dl`	`dl`	
`grad`	`dl`	`dl`	
`letExample`	`dl`	`dl`	
`model`	`dl`, `dl`	`dl`	
`noted`	`dl`, `dl`	`dl`	
`ofN`		`dl`	
`residual`	`dl`, `dl`, `dl`	`dl`	
`scaled`	`dl`	`dl`	
`scaledByTwo`	`dl`	`dl`	
`shared`	`dl`, `dl`	`dl`	
`twiceBound`	`dl`, `dl`	`dl`	
`lenA`		`length`	
`lenB`		`length`	
`lenF`		`length`	
`massA`		`mass`	

Operation	Argument kinds	Result kind	Crossings
`a`		`lengthKind`	
`aSpec`		`lengthKind`	
`b`		`lengthKind`	
`bSpec`		`lengthKind`	
`doubled`		`displacement`	
`east`		`displacement`	
`up`		`displacement`	

33.11. Reading an index without building a document

Every table above is also available from the InfoView, under the name the directive uses:

```
#pkc_index "crossings" PropertyKindCalculus.Examples
```

which is how a pinned probe fixes one. `tests/PropertyKindCalculus/Tests/Core/Index.lean` authors a small closed world — three kinds with one edge between them, a record, an operation, one site of each boundary tier — and pins the rendered index of each with `#guard_msgs`. That is what turns **the table came back empty** into a build failure, which matters more here than for a hand-written table: a derived index that stops matching does not look broken, it looks like an absence.

The second command asks after the **docstrings** the tables quote:

```
#pkc_summary_overflow PropertyKindCalculus.Examples
```

The `What it does` column is a declaration's first paragraph — the unit its author composed, read in preference to its first line, which ends wherever a hard wrap fell. A paragraph much longer than the column is cut, and the cut is made on the parsed inline runs rather than on characters, so a code span is dropped whole rather than severed and an emphasized phrase keeps its delimiters. No docstring length can produce a cell that fails to parse.

The third writes rather than prints. `#pkc_index_page "…" […] ns…` renders the named indexes into the **elaborating module's own module docstring**, so doc-gen4 publishes them as that module's page — the same route `@[pkc_math]` takes to reach two surfaces through one docstring, applied to a table instead of an equation.

What a length can still produce is a cell that says nothing: an ellipsis three words in, where a summary should be. `#pkc_summary_overflow` reports the quoted docstrings that will do that, widest first, so the question is asked where the docstrings are rather than raised as a warning against the directive that

happened to render them. It is not an error, and the fix where one is wanted is a paragraph break — a first paragraph saying what the declaration **is**, with the argument below it.

34. Applying the calculus to a domain: the model template

The preceding chapters say what the calculus is and what it proves. This one is guidance for the domain expert who builds on them: what a **domain model** owes to its reader, and how a document can report, on every build, which of those obligations it has discharged.

The obligation is not obvious, and getting it wrong is cheap. A model that declares its kinds, proves its algebra and ships green nodes can still leave a reader unable to answer the questions that decide whether the model may be used — what it is a model **of**, which of its quantities the dimension layer cannot separate, at which carrier each theorem holds, and which of its numbers were proved rather than measured. None of those is a matter of taste: each has an answer, and a document either contains it or it does not. The template below is that list of questions.

34.1. What a template is, and what it is not

A rubric is an obligation to **answer**, not a section to write. Two documents that discharge the same rubric may look nothing alike: one may answer M2 in a paragraph of its introduction and another in the closing page of a chapter, and both conform. What the template fixes is the question and the **kind of evidence** that settles it.

Of those two, it is the evidence that is worth stating carefully: a checklist that names its questions but not what settles them can be checked off by asserting the answers. Four kinds of evidence appear, and they are not interchangeable:

- **Exposition.** A tagged section of the document states the answer. There is nothing to prove — the obligation is that a reader can find it — so the evidence is a cross-reference, and a cross-reference to a section that does not exist is a render error.
- **Generated.** A view read off the compiled environment: an index, a specification sheet, a graph. The obligation is specifically that it is **generated**. A table transcribed by hand sits on the page looking identical and satisfies the letter of the rubric, so the evidence required is not the table but the declaration that produces or gates it — which only the generated one has.
- **Checked.** A kernel-checked declaration: a theorem edge, a proof, an audit that fails the build when the property it checks does not hold.
- **Measured.** A reported measurement. This is weaker evidence than a proof and the template keeps it weaker on purpose: a measured result is discharged by naming the parameter it was swept over, the denominator it is relative to, and the statistic it reports.

Reporting all four as a single "done" would let a cross-reference pass for a proof. Reported separately, a document's own matrix says which of its answers are proved and which are merely written down.

Each rubric therefore carries two texts, written for different moments. The **rubric** is a property of a finished document — what a reader can check by reading it. The **author's part** is an instruction: the artifact to produce, in the imperative. Neither is derivable from the other by someone who has not built such a document before, which is why the template carries both rather than leaving the second to be inferred from the first.

The template has 26 rubrics: 5 discharged by a section that states the answer, 7 by a view generated from the compiled environment, 13 by a kernel-checked declaration, and 1 by a labeled measurement.

34.2. The template

The rows below are generated from `PropertyKindCalculus.Rubrics.modelTemplate`, so a rubric added to the catalogue appears here without this chapter being edited, and one removed cannot linger in it. The **author's part** column is the one to read while building a document; the sections that follow argue why each group asks for what it asks. The argument is not repeated in the table and the instruction is not repeated in the prose, so neither can drift from the other.

	Requires	The rubric	The author's part
The premise			
`M1`	exposition	The measurand and the deliverable are named and kept apart from what a reference measurement reports	Name the measurand and the deliverable in the model's own quantities, and say what a reference measurement reports instead — with the conversion that relates them where they differ, which is the step a validation comparing the two must perform
`M2`	exposition	Why the kind layer is load-bearing in this domain: which quantities are of dimension one, and which confusions survive a unit check	State which of this model's quantities land on dimension one and which of its confusions survive a unit check, and cite the model's own dimensional-coverage table for the census the paragraph summarizes
`M3`	exposition	The typing discipline is cited, not re-argued	Name what the document assumes of the calculus and point at where it is established; do not restate the case for kinds
`M4`	exposition	The measurement chain in the order the physics runs it, and the layer map saying where each claim lives	Give the measurement chain in the order the physics runs it, and a map from each claim the document makes to the library layer that holds it
The kind vocabulary			
`M5`	generated	Every quantity the document names is a declared kind-typed quantity, inventoried rather than surveyed	Declare every quantity the document names as a kind-typed declaration, and render the inventory with the generated index rather than a table written beside it
`M6`	generated	Kinds of dimension one are individuated by their examination principle, with the procedure ⊑ method ⊑ principle refinement recorded	Give every kind of dimension one an examination principle and record the procedure ⊑ method ⊑ principle chain; where such a kind is deliberately principle-free, declare the exception so an audit can read it
`M7`	checked	Dedications to systems and components are declared, and each separation the	Declare the dedications to systems and components, and prove each separation

	Requires	The rubric	The author's part
		model relies on is proved rather than assumed	an argument in the document rests on rather than asserting it
M8	generated	The arithmetic surface is closed: the complete list of products, quotients and powers the model is permitted to form	Author every product, quotient and power the model may form, and publish the closed surface as a generated table — the empty cells included, since those are the combinations the model has decided are meaningless
M9	checked	Every departure from the algebra is authored, and an audit that gates the build makes the inventory complete rather than a survey	Author every departure from the algebra — a genuine meeting of two kinds, kind-preserving plumbing, a point where a kinded value leaves the calculus — and run the audit that fails the build on an unauthored one
M10	generated	Dimensional coverage is audited edge by edge, and inter-derivability the algebra cannot refuse is reported rather than hidden	Pin the dimensional-coverage reading and the inter-derivability report, so a newly undimensioned edge or a newly merged cluster is a change a reader sees in the diff
The model as modules			
M11	checked	Each boundary is a declared metrological module of kind-typed ports	Declare each boundary as a contract of kind-typed ports, and sweep the namespace so that a boundary declared anywhere is enrolled without being listed by name
M12	checked	A module's behavior is a measurement model carried as a theorem edge, not as prose beside it	Carry each module's behavior as a relation naming a theorem, not as prose beside the module
M13	checked	The implementation's relation to the measurement model is named — equals, inverts, refines, or bounded by a kinded tolerance	Name the relation the implementation bears to the measurement model — equals, inverts, refines, or bounded by a kinded tolerance — and let the sweep check that the witness is sorry-free and about those two boundaries
M14	checked		

	Requires	The rubric	The author's part
		Licenses are stated per carrier rung, because laws transfer across the carrier ladder and side conditions do not	List on the edge itself one license per carrier rung the claim is extended to beyond the rung its witness is stated at, each answered for by a named repair or a named per-rung restatement
`M15`	checked	Mereology is declared per output port, and any recarving or sharding is licensed by that declaration	Declare per output port whether it aggregates over parts, and license any recarving or sharding of a batch by that declaration rather than by the fact that it worked
`M16`	generated	One specification sheet per boundary, generated from the checked declaration rather than written beside it	Generate one specification sheet per boundary from the declared contract, so the commit that changes a contract either regenerates the sheet or fails
Carriers, adequacy and uncertainty			
`M17`	checked	The specification carrier and the executable carrier are distinguished, and the refinement bridge between them is named	Register the specification carrier and the executable carrier as distinct, and name the bridge that carries a claim from one to the other
`M18`	checked	The deployed representation is shown numerically adequate at the scale of the input uncertainties	Show that the deployed representation loses no information at the scale of the input uncertainties — a claim about this model's numbers, not a preference for wider floats
`M19`	checked	Uncertainty names the rung of the method ladder it was computed at, and the coverage its recorded variance certifies	Name, for each reported uncertainty, the rung of the method ladder it was computed at and the coverage its recorded variance certifies
Well-posedness and honest failure			

	Requires	The rubric	The author's part
M20	checked	Where the model is inverted, existence and uniqueness are proved on a declared domain	Where the model is inverted, prove existence and uniqueness on a domain the document declares — both the proof and the domain it is about
M21	checked	Failure outside that domain is detected and reported, and ambiguity is surfaced rather than silently resolved	Detect and report failure outside that domain, and surface ambiguity rather than resolving it silently to whichever root the algorithm reached first
M22	checked	The conditioning and quality outputs a consumer must read are part of the declared contract, not a side channel	Put the conditioning and quality outputs a consumer must read into the declared contract as ports, not into a side channel a downstream stage may or may not read
Assurance			
M23	generated	Every load-bearing definition and theorem links to the declaration that realizes it and carries that declaration's machine-checked status	Cite each load-bearing definition and theorem by the declaration that realizes it, so the rendered claim carries that declaration's machine-checked status
M24	generated	A dependency graph and an assurance ledger are generated from those same nodes, so neither can omit a cited claim	Generate the dependency graph and the assurance ledger from the same nodes the chapters cite, so neither view can omit a claim the other carries
M25	exposition	The limits of a green status are stated plainly: which rung each theorem holds at, and which results are measured rather than proved	State plainly what this document's green status does not mean: which rung each theorem holds at, and which of its numbers are measured rather than proved
M26	measured	Each reported measurement names the parameter it was swept over, the denominator it is relative to, and the statistic it reports	For each reported measurement, name the parameter it was swept over, the denominator it is relative to, and the statistic it reports

34.3. The premise — what the model is a model of

The first four rubrics are the ones most often assumed rather than stated, because to the model's authors they are obvious.

- **M1 — Measurand and deliverable.** The measurand and what the model delivers, kept apart from what a reference measurement reports. These are routinely different quantities that share a name, and the gap between them is where validation quietly goes wrong: comparing a retrieved value against a field measurement of a different kind is an error no amount of downstream rigor recovers from, because the two are related only through a conversion the comparison never performed.
- **M2 — Why kinds are load-bearing here.** Why the kind layer is load-bearing **in this domain** rather than in general. This blueprint argues the general case; a model document's job is the specific one — which of its own quantities land on dimension one, and which of its own confusions survive a unit check. A model whose quantities are mostly dimensioned has a weaker claim on the machinery than one where the dimension layer discriminates almost nothing, and saying which it is tells a reader how much of the model's safety is coming from where.
- **M3 — Cite the discipline, do not re-argue it.** The converse obligation, and a prohibition as much as a requirement. A domain document that restates the case for kinds ages against this blueprint, contradicts it in the details, and buries its own subject. The right form is a section that names what is assumed and points at where it is established.
- **M4 — The spine: measurement chain and layer map.** The measurement chain in the order the physics runs it, and the map from the document's claims to the layers of the library that hold them. This is what lets a reader who cares about one step find the one chapter and the one layer that answer for it.

34.4. The kind vocabulary –– and the audits that close it

A model's kind vocabulary is only as good as its **closure**. Two things can escape it: a quantity the document names but the code carries as a bare number, and an operation on kinded values the declared algebra never licensed — a product formed outside the arithmetic surface, or a crossing between two kinds performed without being authored. The difference between a model that has declared some kinds and a model whose kind discipline is load-bearing is whether anything checks for either, and rubrics M5 through M10 are all variations on that question. The closure asked for is over the document's own vocabulary and not over its domain: it makes no claim that the model covers the physics, only that every quantity the model **does** name is inside the discipline, and every step out of the discipline is authored.

- **M5 — The kind inventory.** Every quantity the document names is a declared kind-typed quantity, reported as an **inventory** rather than a survey — a table generated by walking the compiled environment, which cannot omit a declaration the way a grep can.

- **M6 — Individuating dimension one.** Kinds of dimension one are individuated by their examination principle, with the procedure ⊑ method ⊑ principle refinement recorded: inside the dimension-one fiber the principle is the **only** thing that separates two kinds, so a model that declares such kinds without recording what examines them has named them, not individuated them.
- **M7 — Dedications, and the separations proved.** Dedications to systems and components are declared, and — the part that makes it a `checked` rubric rather than a `generated` one — each separation the model **relies on** is proved. A dedication nothing depends on separating is inventory; a dedication a chapter's argument rests on is a theorem.
- **M8 — The arithmetic surface.** The complete list of products, quotients and powers the model is permitted to form. Read the other way, it is the list of combinations the model has decided are meaningless, which is the more useful reading and the one a generated table gives for free.
- **M9 — Authored departures, gated by an audit.** The rubric the group exists for. Every departure from the algebra — a genuine meeting of two kinds, kind-preserving plumbing the algebra does not name, a point where a kinded value leaves the calculus — must be **authored**, and an audit that fails the build on an unauthored one is what turns the resulting list from a survey into an inventory. Without that audit the list means "the crossings someone thought to write down"; with it, the list is closed.
- **M10 — Dimensional coverage and inter-derivability.** Two audits that report rather than gate. Dimensional coverage checks each authored edge against the dimension functor — the one layer that can mechanically refute an authored axiom, since an edge over genuinely dimensioned kinds whose exponents fail to balance is **wrong**, not merely undeclared — and its value is the inventory it produces: where the dimensioned content actually lives, and how much of the model is dimension one and therefore the kind layer's problem alone. And where kinds are inter-derivable — where each can be manufactured from the others by licensed steps, so the algebra cannot refuse a substitution between them — the template asks that this be drawn and pinned rather than left implicit. A model with such clusters is not defective; a model that has them and does not say so is.

34.5. The model as modules

Rubrics M11 through M16 are the **Metrological modularity** chapter's paradigm read as an obligation on a document.

- **M11 — Boundaries as declared modules.** Each boundary is a declared metrological module of kind-typed ports.
- **M12 — Behavior as a theorem edge.** A module's behavior is a measurement model carried as a theorem edge rather than as prose beside it.
- **M13 — The implementation relation, named.** The implementation's relation to that model is **named**: equals, inverts, refines, or bounded by a kinded tolerance.

- **M14 — Licenses per carrier rung.** The rubric this project has been bitten by, and it generalizes past metrology: laws transfer across the carrier ladder and side conditions do not. A theorem proved over the reals is a statement about the reals; the licenses under which it survives to the executable carrier are separate, per rung, and must be stated there. A document that proves at one rung and deploys at another without saying so has published a claim about something other than the thing it ships.
- **M15 — Mereology per output port.** Mereology is declared per output port — which outputs aggregate over parts and which do not — and any recarving or sharding of a batch is licensed by that declaration rather than by the fact that it happened to work.
- **M16 — A specification sheet per boundary.** One specification sheet per boundary, generated from the checked declaration: the page one hands a prospective consumer, which the same commit that changes a contract either regenerates or fails on.

34.6. Carriers, adequacy and uncertainty

Three rubrics separate the specification from the thing that runs.

- **M17 — Two carriers, and the bridge.** The specification carrier and the executable carrier are distinguished, and the bridge between them is named.
- **M18 — Numerical adequacy.** The deployed representation is shown **numerically adequate**: it loses no information at the scale of the input uncertainties. That is a claim about the model's own numbers rather than a general preference for wider floats.
- **M19 — The uncertainty rung and its coverage.** Uncertainty names the rung of the method ladder it was computed at and the coverage its recorded variance certifies; an uncertainty reported without either is a number without an interpretation.

34.7. Well-posedness and honest failure

- **M20 — Existence and uniqueness on a declared domain.** Where a model is inverted, existence and uniqueness are proved **on a declared domain** — a proof, and a domain the proof is about.
- **M21 — Failure outside the domain.** What happens outside it. This is the rubric that separates a retrieval that is right where it applies from one that is dangerous where it does not: failure outside the domain must be detected and reported, and ambiguity must be surfaced rather than silently resolved to whichever root the algorithm happened to reach.
- **M22 — Conditioning in the contract.** The conditioning and quality outputs a consumer needs in order to act on either are part of the declared contract, not a side channel a downstream stage may or may not read.

34.8. Assurance — and what a green status does not mean

The last group is about the document's own evidence.

- **M23 — Claims linked to declarations.** Every load-bearing definition and theorem links to the declaration realizing it and carries that declaration's machine-checked status, so a reader can follow any equation to the source the deployed code is built from.
- **M24 — Graph and ledger from the same nodes.** A dependency graph and an assurance ledger generated from those same nodes — generated from the same nodes specifically, so that neither view can omit a claim the chapters cite.
- **M25 — The limits of a green status.** The rubric that keeps the other twenty-five honest. A green status means a particular thing, and the two ways it is over-read are both worth stating in the document rather than leaving to the reader: a theorem proved at the real-number carrier is not a statement about the floating-point kernel that runs, and a measured quantity is not a proved one. A document that states both limits plainly is stronger than one that does not, not weaker.
- **M26 — What a measurement must name.** The measurement half of that, made concrete. A speedup is a curve and not a number; a best-of-`n` is biased by `n`; a threshold below the dispersion of the measurement it is compared against decides nothing. The rubric is discharged by naming, for each reported measurement, the parameter it was swept over, the denominator it is relative to, and the statistic it reports.

34.9. What the machine can hold, and what it cannot

Everything above is about what a document must contain. This section is about something else: whether anything can check that it contains **all** of it.

Of the 26 rubrics, 5 have no census and can have none — their population is what the document says. Of the other 21, 16 are walked today (10 by an audit that fails the build, the rest by a generated view or a pinned reading), and 5 have a population a sweep could enumerate and no sweep walking it: for those, the template is reporting exemplars, and is only as complete as the author's memory.

The two are easy to confuse, because a conformance matrix reports them in one word. A status is computed from whether the document offers a **site** — one declaration, one section — in each channel the rubric's evidence kind requires. A site is never a claim about the rest, so two annotated theorems and two hundred read identically. Where a rubric's population can be walked by the machine, that gap can be closed; where it cannot, the rubric is reporting an exemplar and has to be read as one.

A census needs three things, and each of them either exists or does not.

- **A population that is a type.** `#kind_contracts` sweeps every declared contract under a namespace — membership is by type, so declaring a boundary enrolls it. That is the whole

difference between a census and a list: enrolment has to be a side effect of doing the work, never a second act of remembering. Where the population is what the document **says** — the measurand, the case for kinds in this domain, the limits of a green status — no such type exists, and no census can.

- **A decidable per-member predicate.** M6 has one: a kind of dimension one either carries an examination principle or it does not. M7 does not — "each separation the model **relies on**" is not a machine notion, and the nearest mechanical proxy, every pair of dedicated kinds that meet in one signature, is a different claim. Choosing that proxy is a design decision to be made and written down, not a formality to be skipped on the way to an audit.
- **Declarable exceptions.** A census with honest negatives and no way to record them is a census someone will switch off. Kinds of dimension one that are principle-free on purpose — a geometry vocabulary, a nominal designation — are the ordinary case, and an audit that cannot be told so is an audit that gets deleted. `@[kindCounterexample]` and the declared clusters `#kind_scc_clean` takes are the library's two idioms for it: the legitimate gap becomes data rather than a sentence in a docstring no audit can read.

Where all three hold, the population can be held at one of three strengths, and the library names them. A **reading** answers a question. A **record** is a reading pinned with `#guard_msgs`: the whole answer is reviewable in the diff and a changed answer fails the build. A **gate** is the `_clean` form, which throws and carries no message to re-bless — kept separate from the record for the reason every violation-capable pin needs a gate beside it, that pinning a violation is how it stops being noticed.

The table below says where each rubric of this template stands. The rows to watch are the ones reading **census possible**: their population could be enumerated and nothing enumerates it, so the rubric is discharged by whichever instances the author remembered, and a green cell on such a row is a statement about those instances and not about the model.

	Closure	What holds the population, or what a sweep would walk
The premise		
M1	no census	—
M2	no census	—
M3	no census	—
M4	no census	—
The kind vocabulary		
M5	pinned	the generated kind, record and operation index, walked from the compiled environment
M6	gated	`#kind_examination_clean ns …`, with `#kind_examination_coverage` as the record and `@[kindPrincipleFree]` the declared exception; a bare `KindOfProperty` no `DimensionedKind` wraps is reported, not skipped
M7	census possible	the pairs of dedicated kinds a signature makes meet — the mechanical proxy for `relied on`, which has to be chosen before it can be swept
M8	pinned	`#kind_edges k`, and the Algebra column of the generated kinds table
M9	gated	`#kind_boundary_clean ns …` and `#kind_mint_ratchet ns …`
M10	pinned	`#kind_dimensional_coverage ns …` and `#kind_scc ns …`, pinned with `#guard_msgs`; the `_clean` forms gate where a model can afford it
The model as modules		
M11	gated	`#kind_contracts ns …`, or `#kind_contracts_decide ns …` for kernel receipts
M12	gated	`#kind_relation_clean ns …`, with `#kind_relation_coverage` as the record and `@[kindRelationFree]` the declared exception
M13	gated	`#kind_relations ns …`
M14	gated	`#kind_relations ns …`, which fails on a license clause not answered for by name
M15	gated	

	Closure	What holds the population, or what a sweep would walk
		`#kind_mereology_clean ns …`, with `#kind_mereology_coverage` as the record; no exception mark, since the aggregation vocabulary names every honest negative
`M16`	pinned	the module card, generated from the declared contract
Carriers, adequacy and uncertainty		
`M17`	census possible	registered carriers with no declared bridge — the carriers are enumerable (the `Carrier`/`NumCarrier` instance vocabulary, specification from executable by `LawfulCarrier`), but a bridge that is a theorem rather than a `CarrierRefinement` instance has no declaration form, so a sweep today would report every real bridge as missing; the form comes first
`M18`	census possible	deployed output ports carrying no adequacy claim
`M19`	census possible	reported uncertainties that name neither a rung nor a coverage
Well-posedness and honest failure		
`M20`	gated	`#kind_wellposedness_clean ns …` — every `inverts` edge declares its well-posedness (`wellPosed`, an `∃!`-concluding sorry-free theorem, with the `domain` its statement mentions) or surfaces its ambiguity — with `#kind_wellposedness_coverage` as the record; no exception mark, the two fields are total over the honest negatives; the fields themselves are validated by `#kind_relation`
`M21`	gated	`#kind_inversion_clean ns …` for the domain half — every inverted boundary declares a `conditional` port — with `#kind_inversion_coverage` as the record and `@[kindInversionTotal]` the declared exception; `#kind_wellposedness_clean ns …` for the ambiguity half — an inverts edge that does not prove well-posedness surfaces its `ambiguity` as a declaration
`M22`	gated	`#kind_diagnostic_clean ns …` — every `@[kindDiagnostic]`-marked kind is carried by a produced port of some declared boundary — with `#kind_diagnostic_coverage` as the record; enrollment is the mark itself,

	Closure	What holds the population, or what a sweep would walk
		so removing a mark is the exemption, and a scope with no marks records that visibly rather than passing in silence
Assurance		
`M23`	pinned	the document's own harvest of cited declarations, which cannot omit a node it renders
`M24`	pinned	the graph and ledger directives, generated from the cited nodes
`M25`	no census	—
`M26`	census possible	reported measurements, once each is declared as data rather than written as a number in a sentence

34.10. How a document declares conformance

A document answers to the template by declaring **where** it addresses each rubric, through two channels with different guarantees. A **declaration site** is the `@[rubric "M12"]` attribute on a real declaration, applied from a separate module the way the requirement annotations are; because it attaches to a declaration, a site cannot name one that does not exist, and a rename is a compile error. A **section site** names the tag of the section that states the rubric; the conformance matrix renders it as a cross-reference, so a dangling tag is a render error.

Every rubric requires a section site, whatever its evidence kind. The template is a claim about a document, so a rubric discharged only by a declaration is one the document never mentions, and reporting that as addressed would be the false green the whole arrangement exists to prevent. What the evidence kind adds is whether a declaration is required **as well**: for everything but exposition, it is.

A rubric with sites in some but not every required channel therefore reads `partial` rather than passing, and it does so in both directions. A generated view cited by section with no generator named is a hand-maintained table claiming to be a live one; a theorem annotated with no section citing it is a green declaration the document is silent about. Both are the case a single "done" would hide.

The status of each rubric is therefore computed from the sites and the rubric's own evidence kind. There is no switch to forget to flip, and the worked instance of the whole arrangement is soil-moisture-model's technical reference, whose conformance matrix is generated this way against this template.

One limit of that computation is worth stating where an author will meet it. The status asks whether a site **exists**, never what the site covers. A rubric whose closure is a census can therefore be discharged by a declaration that is not that census — two theorems standing in for a sweep — and no cell of the matrix distinguishes the two. Until a rubric names the audit that closes it and the status requires that audit

among its sites, a green cell on a **census possible** row means the author remembered; the closure table above is where a reader finds out which cells those are.

35. Deploying a domain model: the deployment template

A model document says what a boundary **is**. A deployment document says what deploying one **adds**, and the two are different enough that a single document doing both does neither well. The model's reader wants to know what is proved and at which carrier; the deployment's reader wants to know what will run, on what, against which contract, and what would have to fail for the result to be rejected.

The separation is not editorial tidiness. Almost everything the deployment adds is **unproved by construction**: an image is built, not derived; a shard size is measured, not theorem-backed; a queue is chosen. Mixing that material into the model document lets the model's green nodes lend it an assurance it does not have. Keeping it separate forces the deployment to carry its own evidence, which is what this template enumerates.

The template has 17 rubrics: 9 discharged by a section that states the answer, 2 by a view generated from the compiled environment, 3 by a kernel-checked declaration, and 3 by a labeled measurement.

35.1. The template

The rows below are generated from `PropertyKindCalculus.Rubrics.deploymentTemplate`. The **author's part** column is the one to read while building a document; the sections that follow argue why each group asks for what it asks. Conformance is declared exactly as for the model template: declaration sites through the `@[rubric …]` attribute, section sites through the document's own tags, and a status computed from the two against each rubric's evidence kind.

	Requires	The rubric	The author's part
What is deployed			
`D1`	exposition	What is deployed, and which model boundary each deployed unit realizes	List the deployed units, and name for each the model boundary it realizes
`D2`	exposition	The document states what deploying the model adds, and cites the model document for what the module is	State what deploying the model adds, and cite the model document for what the module is
The contracts			
`D3`	checked	The data contract is declared, compiled in, and validated against a producer's own claims before any payload is decoded	Declare the payload interface, compile it into the executable, and validate it against the producer's own claims before any payload is decoded
`D4`	checked	The platform contract is kind-typed and held to the registration payload, with the discharge relation decided by the kernel	Kind-type the argument plane the scheduler passes, hold it to the registration payload, and let the kernel decide the discharge relation between the two
`D5`	generated	One deployment card per deployed unit, each panel read off a declared object rather than written beside it	Generate one deployment card per deployed unit, each panel read off a declared object rather than written beside it
Sizing and knobs			
`D6`	measured	The run is sized from measured records, not from constants typed by hand	Size the run from measured records, and name the records each size came from
`D7`	measured	Operating points come from a sweep whose own turnover or plateau names them, reported with the dispersion any threshold has to beat	Sweep the parameter, let the machine's own turnover or plateau name the operating point, and report the dispersion any threshold has to beat
`D8`	exposition		

	Requires	The rubric	The author's part
		Every environment knob a job may set is enumerated with what it decides	Enumerate every environment knob a job may set, with what each one decides
The platform			
`D9`	exposition	The image facts are stated: what ships per unit, the toolchain and architecture spectrum it was built for, and the ABI floor it runs against	State what ships per unit, the toolchain and architecture spectrum it was built for, and the ABI floor it runs against
`D10`	exposition	Registration and version derivation are mechanized, so a deployed version is derived from the artifact rather than asserted beside it	Derive the deployed version from the artifact and mechanize registration, so neither is asserted beside what actually shipped
`D11`	exposition	Backend interchange is stated: what the same source does on each target, and what changes when the target changes	State what the same source does on each target, and what changes when the target does
Acceptance			
`D12`	checked	Acceptance is a gate against the specification the model document proves — run as proof-by-execution, not asserted	Run acceptance as a gate against the specification the model document proves — proof by execution, not an assertion that the two agree
`D13`	measured	Performance claims are a paired A/B in the deployed executable, never a microbenchmark quoted as a prediction	Make every performance claim a paired A/B in the deployed executable, never a microbenchmark quoted as a prediction
`D14`	exposition	Each green signal names which of the changed lines it actually compiled or ran, so a passing pipeline is not quoted as evidence for a path it never touched	Say, for each green signal quoted as evidence, which of the changed lines it actually compiled or ran
Reproducibility and limits			

	Requires	The rubric	The author's part
D15	exposition	Failure, checkpoint and resume semantics are stated: what a partially completed job leaves behind, and what resuming it does	State what a partially completed job leaves behind, and what resuming it does with that
D16	generated	A reproducibility manifest carries the pins and digests of what actually ran, generated from the artifacts rather than transcribed	Generate the reproducibility manifest from the artifacts — the pins and digests of what actually ran, not of what was intended
D17	exposition	What is trusted is stated plainly: the boundaries the verification does not cross, and what a consumer is therefore taking on faith	Name the boundaries this deployment's verification does not cross, and what a consumer is therefore taking on faith

35.2. What is deployed

- **D1 — The deployed units.** The list — which units are deployed, and which model boundary each one realizes. A deployment that cannot name the boundary it implements has no specification to be judged against, and its acceptance gate is therefore a comparison against itself.
- **D2 — What deploying adds.** The counterpart of the model template's M3, pointed the other way: state what deploying adds, and cite the model document for what the module is. A deployment document that re-explains the physics duplicates a text that will move without it.

35.3. The contracts

Three rubrics fix what a deployed unit is held to, and all three ask for the contract to be **decidable** rather than documented.

- **D3 — The data contract.** The payload interface a unit reads and writes, declared, compiled into the executable, and validated against a producer's own claims **before any payload is decoded**. Checking after decoding is checking too late — by then a mis-channeled array has already been read as something it is not, and the failure surfaces as a wrong number rather than a rejection.
- **D4 — The platform contract.** The argument plane the job scheduler passes, kind-typed and held to the registration payload, with the discharge relation between the two decided by the kernel rather than kept in step by hand. This is the seam where a deployment most often rots — the registration record and the executable's expectations drift apart across a rename — and it is the one seam where a machine-checked relation costs almost nothing.

- **D5 — A deployment card per unit.** One deployment card per deployed unit, each panel read off a declared object. The card is the specification sheet of the **deployment** rather than of the model: the argument plane, the payload contract faces, the sizing, the knobs, the container facts. Its value comes entirely from being read off declarations, so that changing a contract either regenerates the card or fails.

35.4. Sizing and knobs

D6 and D7 are the rubrics that a deployment document is most tempted to skip, because the numbers involved are easy to type and hard to justify.

- **D6 — Sizing from measured records.** A run is sized from measured records rather than from constants typed by hand. A hand-typed size does not report the machine; it reports how far from saturation the author's memory happened to be.
- **D7 — Operating points from a sweep.** Operating points come from a sweep whose own turnover or plateau names them, reported with the dispersion any threshold has to beat. A threshold below the resolution of the measurement that produced it decides nothing, and — the part worth watching for — it fails **silently**, printing an optimum chosen by scatter. Reporting the dispersion beside the decision is what makes the failure visible, and refusing a comparison the data cannot support is a better answer than making one.
- **D8 — The environment knobs.** Every environment knob a job may set is enumerated with what it decides. A knob that is settable and undocumented is a way for a run to differ from the run that was accepted.

35.5. The platform

- **D9 — The image facts.** What ships per unit, the toolchain and architecture spectrum it was built for, and the ABI floor it runs against. The reason to state the spectrum rather than "it builds" is that the development box and the deployment image routinely differ in both compiler version and target set, and either difference can hide a failure that cannot appear locally.
- **D10 — Mechanized registration and version derivation.** Registration and version derivation are mechanized, so a deployed version is **derived from the artifact** rather than asserted beside it.
- **D11 — Backend interchange.** What backend interchange means for this deployment: what the same source does on each target, and what changes when the target does.

35.6. Acceptance

- **D12 — Acceptance as a gate.** Acceptance is a gate against the specification the model document proves — run, as proof-by-execution, rather than asserted. This is the load-bearing link between the two documents: without it, everything the model proved stops at the model's own boundary.
- **D13 — A paired A/B in the executable.** Performance claims are a paired A/B in the deployed executable. A microbenchmark ranks; it does not predict. An isolated win of a factor of two and a half can be zero in the binary that ships, and the only measurement that settles the question is the one taken in the executable whose behavior is being claimed.
- **D14 — What each green signal covered.** The rubric that costs the least and is skipped the most: each green signal must name which of the changed lines it actually compiled or ran. A pipeline that passes without compiling the changed path is evidence about the paths it did compile and about nothing else, and quoting it as though it covered the change is how two defects survive three green signals.

35.7. Reproducibility and limits

- **D15 — Failure, checkpoint and resume semantics.** What a partially completed job leaves behind, and what resuming it does with that.
- **D16 — The reproducibility manifest**. A manifest carrying the pins and digests of what actually ran, generated from the artifacts rather than transcribed from intent.
- **D17 — What is trusted.** The rubric that closes the template the way M25 closes the model's. Every deployment has boundaries its verification does not cross — a foreign function interface, a vendor runtime, a scheduler — and naming them is what lets a consumer distinguish what was checked from what was assumed. A deployment document that claims no limits is claiming the strongest thing in it, which is never true.

35.8. What the machine can hold here

Of the 17 rubrics, 7 have no census and can have none — their population is what the document says. Of the other 10, 5 are walked today (2 by an audit that fails the build, the rest by a generated view or a pinned reading), and 5 have a population a sweep could enumerate and no sweep walking it: for those, the template is reporting exemplars, and is only as complete as the author's memory.

The framing is the model template's — a census needs a population that is a type, a decidable predicate on its members, and a way to declare the honest exceptions — but this template's profile differs in kind, for the reason the chapter opened with. Almost everything a deployment adds is unproved by construction: an image is built, a queue is chosen, an ABI floor is what it is. More of this template is therefore prose, and a document that states those facts plainly has done what can be done about them.

What is worth seeing in the table below is that the rows reading **census possible** are nearly all one move from being held, and it is the same move each time. The sizing records, the sweep behind each operating point, the settable knobs, the paired run behind a performance claim — every one of them is data the deployment already has and currently writes into a sentence. Declared instead of narrated, each becomes a population something can walk, which is the step M9 took for crossings in the model template and the reason that rubric gates where its neighbours only report.

	Closure	What holds the population, or what a sweep would walk
What is deployed		
`D1`	no census	—
`D2`	no census	—
The contracts		
`D3`	census possible	declared payload faces with no validation ahead of the first decode
`D4`	gated	`#kind_discharges_decide c d`
`D5`	pinned	the deployment card, generated per declared unit
Sizing and knobs		
`D6`	census possible	sizing constants, once the measured records behind each are declared
`D7`	census possible	reported operating points, once the sweep behind each is declared
`D8`	census possible	settable environment knobs, once declared as data rather than listed in prose
The platform		
`D9`	no census	—
`D10`	pinned	the deployed version and registration payload, derived from the artifact
`D11`	no census	—
Acceptance		
`D12`	gated	the acceptance run itself, executed against the model document's specification
`D13`	census possible	performance claims, once each names the paired run in the deployed executable behind it
`D14`	no census	—
Reproducibility and limits		
`D15`	no census	—
`D16`	pinned	the manifest, generated from the artifacts that ran

	Closure	What holds the population, or what a sweep would walk
D17	no census	—

36. Terminology — the dictionary and the index

A reader who meets three phrases for one object concludes there are three objects. The calculus names many objects that are close neighbors — several graphs, several tiers, several boundaries — and a document that drifts between phrasings for any of them costs the reader the very distinction the calculus was built to keep. So the vocabulary is managed the way the requirements and the templates are: as data, rendered here, and gated.

Three rules govern it. **One defining site**: each term is defined by exactly one `{deftech}` in the blueprint, under the section the dictionary names, and every other mention is a `{tech}` link to that site or plain prose in the same words. **Canonical first, short after**: a chapter, a module docstring, or a plan document uses the canonical phrase at a concept's first mention and may use the sanctioned short form afterward. **Retired means refused**: a phrasing the dictionary lists as retired fails the build's documentation gate wherever the gate reads — every blueprint chapter and every root plan document — and is counted in the library's own docstrings by a ratchet that may only fall.

The dictionary is `PropertyKindCalculus.Terminology.dictionary`. The table below is rendered from it, sorted by canonical phrase; the **Defined in** column links to the section whose prose explains the term, and the declarations column is checked, at blueprint build, against the environment — a renamed declaration fails the render rather than surviving as a stale name. A new concept gets its entry before its first use, so that the entry, the defining site and the gate move together.

The dictionary carries 39 terms: 8 with a sanctioned short form, 5 with retired phrasings the gate refuses, and 51 declaration citations that the blueprint build checks exist.

36.1. The dictionary

Term	Meaning	Declarations
assembly	The harvest's output for one declared boundary: its provenance hypergraph together with the signature positions found to carry no kind and the level each node belongs to.	`PropertyKindCalculus.KindIncidence.Assembly`
assumption ledger	An output's influencers split by what each source is — the ports left open at the boundary, the gated ingests, the attested mints with their reasons.	`PropertyKindCalculus.Provenance.assumptionLedger`, `PropertyKindCalculus.Provenance.trustSplit`
attestation	A kind claim with no machine-checkable evidence, stated where used with its reason: a hypothesis of the model, not an axiom of the calculus.	`PropertyKindCalculus.Quantity.attest`
blueprint dependency graph — short form *dependency graph*	The graph of this blueprint's own nodes and their `uses` edges — a rendering of the document, and not one of the graphs the calculus defines.	-—
boundary audit	The environment walk `#kind_boundary_audit` that lists every site where a value acquires or sheds a	-—

Term	Meaning	Declarations
	kind, each with the tier that sanctions it, and fails the build on a site no tier sanctions.	
carrier ladder	The number types one carrier-parametric definition is instantiated at, ordered from proof to execution — the reals for proof, binary32 as the rounding specification, `Float` for execution, the tape for the kernel — each a rung, reached from the abstract carrier by one map. Laws survive the climb up to one rounding per operation; side conditions phrased in a carrier's own arithmetic do not survive it at all.	`PropertyKindCalculus.Carrier`, `PropertyKindCalculus.CarrierRefinem`
checked ingest	The entry of raw, external data into the calculus through a check — the `@[kindIngest]` tier — whose evidence is the check itself.	—
compacted tape	A tape after its identical sub-expressions are merged: the object	`PropertyKindCalculus.Paradigm.TapeCSE.cseCompact`

Term	Meaning	Declarations
	the code generator lowers to one kernel.	
computational tape graph — short form *tape graph*	The unkinded, computational counterpart of the metrological provenance hypergraph: the directed acyclic graph a tape determines, one vertex per tape node, an edge from each parent to its node, in-edges ordered as the parents are; leaves are the named inputs and the baked constants, roots the recorded outputs. A tape node is its own operation, so a vertex's ordered in-edges are the one hyperedge into it and the hypergraph reading adds nothing — unlike the provenance hypergraph, whose occurrences are objects apart from its nodes. The object the second capstone relates to the provenance hypergraph.	`PropertyKindCalculus.Paradigm.TapeGraph`, `PropertyKindCalculus.Paradigm.TapeCodegen.ofTape`, `Runtime.Autograd.Node`
constant mint	The entry of an adjudicated constant	-—

Term	Meaning	Declarations
	— a cited coefficient, a threshold, a structural constant of the model — through the `@[kindConst]` tier, its provenance the declaration's docstring.	
contracted tape graph	The computational tape graph with each realized occurrence's sub-graph collapsed to one ordered hyperedge — its operands the sub-graph's frontier in operand order, its results the sub-graph's roots, its label the family; defined under the clauses acceptance decides of the sub-graphs — closed, interior-private, progressing, used, pairwise disjoint, covering every operation. On it the kind-transporting weak bisimulation becomes strong, the silent steps gone.	`PropertyKindCalculus.Paradigm.Match.contracted`, `PropertyKindCalculus.Paradigm.Match.isBisimulation_contracted`
crossing	A re-typing of a value that already carries one kind as another, with no law of the algebra to derive it: trust granted by hand, tagged	—

Term	Meaning	Declarations
	`@[kindCrossing]` and enumerated by the boundary audit.	
declared boundary — short form *contract*	The interface a metrology module declares: its members, its kind-typed ports with the role each plays, its exits, and its deciders; decided against what the members compute, in both directions.	`PropertyKindCalculus.Provenance.Contract`, `PropertyKindCalculus.Provenance.Contract.agrees`
denotation bridge	The predicate relating a recording-carrier value to the tensor it computes, preserved by every carrier operation, so that a module's parity with its own eager value is a chain of per-operation lemmas.	`PropertyKindCalculus.Paradigm.TapeParity.Evaluates`
dimensional homogeneity	The third capstone: along every occurrence of a well-formed provenance hypergraph the dimension of the result is the one the family's rule computes from the operands — dimension is a homomorphism along every path.	`PropertyKindCalculus.DimensionalHomogeneity.dimensional_homogene` `PropertyKindCalculus.Provenance.homogeneity`, `PropertyKindCalculus.InteractionAlgebra.dim_homomorphism`
emission		—

Term	Meaning	Declarations
	Where a kinded value leaves the calculus as a bare number for a consumer outside it — a deploy driver, a serializer, the parity apparatus — tagged `@[kindEmission]`.	
empirical requirement	A requirement whose statement names a referent the model does not define, so no theorem about the model alone can discharge it — the falsifiable kind, discharged by a gate against the referent or deferred by an attestation.	`PropertyKindCalculus.Provenance.RequirementKind`
evidence tier — short form *tier*	How a value came to carry its kind: derived through a law of the algebra, gated by a check at ingest, or attested by its author with a reason.	`PropertyKindCalculus.Provenance.IntroTier`
harvest	The elaboration-time walk that reads a declared boundary's member definitions and builds their provenance hypergraph: ports from the signatures, occurrences from the kinded operations, sources from the	`PropertyKindCalculus.KindIncidence.Assembly`

Term	Meaning	Declarations
	boundary audit's tiers and the attestations' reasons.	
influencers	The sources among an output's pedigree: the term list of an uncertainty budget over the graph's edges, and the source set whose attached conditions a downstream flag may claim for that output.	`PropertyKindCalculus.Provenance.influencers`
kind-of-property — short form *kind*	The common defining aspect of mutually comparable properties, carried as a type index of every quantity; a kind-of-quantity when its scale has magnitude.	`PropertyKindCalculus.KindOfProperty`
kind-transporting weak bisimulation — short form *bisimulation*	The relation tying the two graphs of one module together: the metrological provenance hypergraph, harvested from the source, where every kind originates, and the computational tape graph, recorded by evaluation, which carries none. It matches sources to leaves and each occurrence to the tape sub-graph	`PropertyKindCalculus.Paradigm.Match`, `PropertyKindCalculus.Paradigm.Match.accepts`, `PropertyKindCalculus.Paradigm.Match.weak`, `PropertyKindCalculus.Paradigm.Match.isWeakBisimulation_weak`

Term	Meaning	Declarations
	realizing its family; weak because it is silent on the interior of a carrier-level expansion, a bisimulation because it simulates in both directions — every occurrence realized on the tape, every tape path projected to a hypergraph path — and along it the hypergraph's kinds are transported onto the tape graph. A weak bisimulation in the textbook sense, over labelled transition systems: the provenance layer's hops labelled by family and operand position, the tape's edges labelled through the match, wires and interior steps silent. As data it is a match — each node with the tape vertices realizing it, each occurrence with the interior of its sub-graph — whose acceptance is decidable.	
license clause	The part of a measurement-model relation that extends	`PropertyKindCalculus.Provenance.RelationLicense`, `PropertyKindCalculus.Provenance.TransferStatus`

Term	Meaning	Declarations
	its witness beyond the carrier rung it is proved at: one entry per further rung, each a named repair theorem — exact, where rounding is the identity on the values in play, or nonneg, where nonnegative on-grid weights rule cancellation out — or an independent witness restated at that rung; a rung with neither is refused.	
measurement model	The VIM relation among the quantities involved in a measurement, attached to a declared boundary as a checked theorem edge rather than as prose.	`PropertyKindCalculus.Provenance.Relation`
metrological provenance hypergraph — short form *provenance hypergraph*	The object every audit report presents one relation of, and the one where every kind originates: kinded values as nodes, occurrences of the witness families as ordered hyperedges, sources carrying their evidence tier, and exits where a value leaves the calculus; prelude-only data,	`PropertyKindCalculus.Provenance`, `PropertyKindCalculus.Provenance.wellFormed`

Term	Meaning	Declarations
	authored by hand in a probe or produced by the harvest from the source.	
metrology module	A software component — the declarations one boundary names as its members, which need not be a file or a namespace — whose interface is a declared boundary, whose behavior is a measurement model attached as a theorem edge, whose implementation is a carrier-parametric measurement function, and whose licenses and mereology are declared.	-—
pedigree	Everything a node set is derived from, the set included: the backward closure through the occurrences.	`PropertyKindCalculus.Provenance.ancestorsOf`, `PropertyKindCalculus.Provenance.eq_or_operand_of_mem_ancestorsOf` `PropertyKindCalculus.Provenance.operand_mem_ancestorsOf`
propagation relation	Which source ports of a declared boundary reach which of its produced ports through the occurrences — the reading of an interface that says	`PropertyKindCalculus.Provenance.Contract.propagation`

Term	Meaning	Declarations
	how information moves through it, not only what it takes and returns.	
provable requirement	A requirement whose statement closes over the model's own declarations and kinded quantities, so a theorem about the model discharges it: a theorem edge from the governed boundary to a specification boundary, or a theorem, with spot checks where the model runs; a negative one, by the absence of a license.	`PropertyKindCalculus.Provenance.RequirementKind`
recording carrier	The carrier whose values are thunks appending their sub-expression to a tape; instantiating a module at it builds the module's tape.	`PropertyKindCalculus.Paradigm.TapeBuilder`
referent	A kinded quantity the model does not define — a datasheet value, a prior, a reference measurement — marked `@[kindReferent]` with where it comes from: what an empirical requirement is	`PropertyKindCalculus.Provenance.Requirement.referents`

Term	Meaning	Declarations
	decided against, and what a provable requirement's statement may not name.	
requirement	What a metrology module's quantities must satisfy, stated as data beside the declared boundary it governs: the statement in the author's words, the governed boundary and port, the objects it quantifies over, and the evidence by which it is met — provable or empirical, or attested with a reason.	`PropertyKindCalculus.Provenance.Requirement`
requirement scope	The objects a requirement quantifies over: every object the boundary is evaluated for, every object of a named sort (reached through `Sorted`), named objects, or the objects a named decider selects.	`PropertyKindCalculus.Provenance.RequirementScope`
seal of a module	The first capstone: once a module's gates are declared, every leaf of an output's pedigree is a declared source — no	`PropertyKindCalculus.Provenance.pedigree_seal`, `PropertyKindCalculus.Provenance.sealed`

Term	Meaning	Declarations
	undeclared raw datum reaches the outputs.	
seal of the computation	The second capstone: the seal of a module carried onto the computational tape graph at every carrier — an output depends only on its influencers and the listed constants, and every kind change in the computation is one an occurrence of the hypergraph states.	`PropertyKindCalculus.Paradigm.TapeSeal.semantic_seal`
specification boundary	A declared boundary whose one member computes the bound a requirement is checked against, told from a model boundary by its role: the right side of a provable requirement's theorem edge, and itself the subject of no requirement.	`PropertyKindCalculus.Provenance.BoundaryRole`, `PropertyKindCalculus.Provenance.Contract.role`
spot check	A named `Bool` declaration a provable requirement cites and the requirement command evaluates, requiring `true`: the runtime evidence that the proof reaches the carrier the model runs	`PropertyKindCalculus.Provenance.Requirement.spotChecks`

Term	Meaning	Declarations
	on and the operational inputs — a check of the proof's reach, not a discharge.	
tape	Automatic differentiation's word for the record of the operations a program performs, in the order it performs them: a grow-only array of nodes, each one operation with its parent ids in operand order and its forward value, every operation appending one node. A carrier-generic module writes one when instantiated at the recording carrier; the calculus reads it forward and never runs the backward pass the name comes from. A straight-line program, not a graph, with kinds erased.	`Runtime.Autograd.Tape`, `PropertyKindCalculus.Paradigm.TapeBuilder`
value-flow digraph — short form *flow digraph*	The binary shadow of a provenance hypergraph: one vertex per node, one edge per operand-to-result pair of an occurrence; the graph reachability is stated over, so that the executable closures	`PropertyKindCalculus.Provenance.flowDigraph`, `PropertyKindCalculus.Provenance.mem_ancestorsOf_iff`

Term	Meaning	Declarations
	can be proved to compute it.	

36.2. Index

Every defining site also marks an index entry, and chapters may mark further entries at the places a reader would look a term up. The index below is generated from those marks.

A | B | C | D | E | H | I | K | L | M | P | R | S | T | V

A

assembly

assumption ledger

attestation

B

blueprint dependency graph

boundary audit

C

carrier ladder

checked ingest

compacted tape

computational tape graph

constant mint

contracted tape graph

crossing

D

declared boundary

denotation bridge

dimensional homogeneity

E

emission

empirical requirement

evidence tier

H

harvest

I

influencers

K

kind-of-property

kind-transporting weak bisimulation

L

license clause

M

measurement model

metrological provenance hypergraph

metrology module

P

pedigree

propagation relation

provable requirement

R

recording carrier

referent

requirement

requirement scope

S

seal of a module

seal of the computation

specification boundary

spot check

T

tape

V

value-flow digraph

37. License

PropertyKindCalculus and this blueprint are released under the Apache License, Version 2.0 (SPDX `Apache-2.0`; license text (https://www.apache.org/licenses/LICENSE-2.0)).

Copyright (c) 2026 California Institute of Technology (Caltech). U.S. Government sponsorship acknowledged.